

z/OS
3.1

*DFSMSrmm Implementation and
Customization Guide*



Note

Before using this information and the product it supports, read the information in [“Notices” on page 547](#).

This edition applies to IBM® z/OS® 3.1 (5655-ZOS) and to all subsequent releases and modifications until otherwise indicated in new editions.

Last updated: 2025-06-18

© **Copyright International Business Machines Corporation 1992, 2025.**

US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

Figures.....	xv
Tables.....	xxv
About this document.....	xxix
How to use this document.....	xxix
How to find samples in this document.....	xxix
Required product knowledge.....	xxx
z/OS information.....	xxx
Notational conventions.....	xxx
How to read syntax diagrams.....	xxx
How to abbreviate commands and operands.....	xxxii
How to use continuation characters.....	xxxii
Delimiters.....	xxxiii
Character sets.....	xxxiii
How to provide feedback to IBM.....	xxxv
Summary of changes.....	xxxvii
Summary of changes for z/OS 3.1.....	xxxvii
Chapter 1. Introducing DFSMSrmm.....	1
What is a RMMplex?.....	1
What libraries and locations can DFSMSrmm manage?.....	1
What is in a removable media library?.....	1
What is in a system-managed tape library?.....	2
What is in a non-system-managed tape library?.....	3
What is in a storage location?.....	3
How does DFSMSrmm manage these libraries and locations?.....	3
Setting up your installation options.....	4
Selecting the retention method.....	4
Defining retention and movement policies for the VRSEL retention method.....	5
Running DFSMSrmm utilities.....	7
What resources does DFSMSrmm manage?.....	9
Shelf locations.....	9
Volumes.....	10
Data sets.....	14
Year 2000 support.....	14
Software products.....	15
Owner information.....	15
How does DFSMSrmm help you create reports?.....	15
Using DFSMSrmm report generator.....	15
Using DFSMSrmm ISPF dialog and RMM TSO subcommands.....	15
Using the EDGAUD and EDGRPTD report utilities.....	16
Using the DFSMSrmm EDGRRPTE exec.....	16
Using the DFSORT ICETOOL utility.....	16
Using the DFSMSrmm application programming interface.....	16
How does DFSMSrmm authorization and security work?.....	16
What tape usage does DFSMSrmm support?.....	17

How does DFSMSrmm validate tape mounts?.....	17
Why does DFSMSrmm reject tape volumes?.....	18
Rejects caused by installation controls.....	19
Rejects caused by validation failure.....	19
Rejects caused by DFSMSrmm rules.....	20
Who can use DFSMSrmm and how?.....	20
General user.....	20
Tape librarian.....	20
Storage administrator.....	21
Application programmer.....	21
System programmer.....	21
Operator.....	21
Using DFSMSrmm.....	22
Using DFSMSrmm WUI.....	22

Chapter 2. Implementing DFSMSrmm..... 23

Step 1: Preparing to implement DFSMSrmm.....	23
Step 2: Running the installation verification procedure (optional).....	24
Step 3: Updating JES3 (optional).....	24
Step 4: Updating installation exits.....	24
Step 5: Updating SYS1.PARMLIB members.....	25
Updating IEFSSNxx.....	25
Updating IKJTSOxx to authorize DFSMSrmm commands.....	26
Updating SMFPRMxx (optional).....	27
Updating GRSRNLxx (optional).....	27
Updating AUTORxx (optional).....	30
Enabling DFSMSrmm.....	42
Step 6: Using the Problem Determination Aid Facility (optional).....	43
Step 7: Setting up DFSMSrmm disposition processing (optional).....	43
Step 8: Updating the procedure library.....	43
Step 9: Assigning DFSMSrmm a RACF user ID.....	46
Step 10: Defining parmlib member EDGRMMxx.....	47
Step 11: Tailoring parmlib member EDGRMMxx.....	47
Step 12: Creating the DFSMSrmm control data set.....	48
Roadmap for creating the control data set.....	48
Defining the DFSMSrmm control data set.....	49
Calculating DASD space for the DFSMSrmm control data set.....	49
Placing the DFSMSrmm control data set.....	50
Allocating space for the control data set.....	51
Protecting the control data set.....	51
Initializing the control data set.....	52
Planning to back up the control data set.....	52
Step 13: Creating the journal.....	53
Roadmap for creating the journal.....	53
Calculating DASD space for the journal.....	53
Placing the journal.....	54
Allocating space for the journal.....	54
Protecting the journal.....	55
Backing up the journal.....	55
Step 14: Authorizing users.....	55
Step 15: Making the DFSMSrmm ISPF dialog available to users.....	55
Adding DFSMSrmm to an ISPF selection panel.....	56
Modifying an ISPF selection panel.....	57
Enabling ISPF data set list (DSLIST) support.....	58
Step 16: Restarting z/OS with DFSMSrmm implemented.....	59
Step 17: Tailoring DFSMSrmm set up.....	59
Step 18: Updating the workload management service definition for DFSMSrmm.....	60

Step 19: Starting DFSMSrmm.....	60
Stopping DFSMSrmm.....	61
Quiescing DFSMSrmm.....	62
Restarting DFSMSrmm.....	62
Checking DFSMSrmm status.....	62
Step 20: Defining resources.....	63
Defining shelf locations.....	63
Defining owner information to DFSMSrmm.....	63
Defining volumes.....	64
Defining vital record specifications.....	66
Step 21: Updating the operational procedures.....	66
Step 22: Initializing the DFSMSrmm subsystem and tape recording.....	66
Enabling the DFSMSrmm subsystem interface.....	66
Changing the DFSMSrmm running mode.....	67
Activating the tape volume interface.....	67
Restarting the DFSMSrmm subsystem.....	67
Step 23: Setting up DFSMSrmm utilities.....	68
Step 24: Installing PTFs and the SMP/E maintenance to DFSMSrmm.....	69
Chapter 3. Setting up DFSMSrmm client and server systems.....	71
Implementing DFSMSrmm client and server systems.....	72
Using the DFSMSrmm client and server systems.....	73
Managing catalogs in an RMMplex.....	74
Chapter 4. Organizing the removable media library.....	75
Organizing the library by pools.....	75
Pooling overview.....	75
Pooling considerations.....	77
Calculating pool size.....	78
Defining pools in parmlib member EDGRMMxx.....	78
Changing pool definitions.....	79
Designing rack pools.....	80
Designing scratch pools.....	80
Requesting and using scratch pools.....	81
Using SMS tape storage groups for DFSMSrmm scratch pooling.....	82
Making an ACS storage group assignment.....	82
A pooling example.....	84
Managing pools with job name and data set name.....	85
Using scratch pooling for System managed and Non-System Managed Tape.....	85
Assigning policies.....	87
Using SMS management class to retain non-system-managed volumes with VRSEL retention method.....	89
Managing volumes with special dates.....	91
Using volumes with special expiration dates.....	91
Using management class to retain system-managed volumes managed by VRSEL retention method.....	91
Using the SMS pre-ACS interface.....	93
Exploiting SMS management class attributes.....	93
Managing volumes with duplicate volume serial numbers.....	95
Using volumes with duplicate volume serial numbers.....	95
Changing duplicate volume serial numbers.....	96
Adding a duplicate volume into a system-managed tape library.....	97
Managing undefined volume serial numbers.....	97
Segregating WORM tapes in separate scratch pools.....	97
Chapter 5. Running DFSMSrmm with system-managed tape libraries.....	99
Using DFSMSrmm with system-managed tape libraries.....	99

Associating volumes and system-managed libraries.....	100
Cartridge entry processing.....	100
Manual cartridge entry processing.....	100
Managing scratch pools.....	101
Ejecting volumes from system-managed libraries.....	101
Returning volumes to the system-managed library.....	102
Volume-not-in-library processing.....	103
Confirming volume movement for system managed libraries.....	105
Defining system-managed volume information.....	106
Initializing scratch volumes in system-managed libraries.....	107
Using storage group names.....	108
Using DFSMSrmm with the IBM totalstorage virtual tape server.....	108
Defining logical volumes in a virtual tape server library.....	108
Logical volume cartridge entry processing.....	109
Managing stacked volumes.....	110
Deleting stacked volume information.....	112
DFSMSrmm support for stacked volumes when stacked volume support is enabled.....	112
DFSMSrmm support for stacked volumes when stacked volume support is not enabled.....	118
Enabling stacked volume support.....	120
Performing a virtual export of logical volumes.....	121
Recovering a logical volume from an exported stacked volume.....	122
Setting up DFSMSrmm for the system-managed tape library.....	122
Using the system-managed tape library with new volumes.....	122
Using the system-managed tape library with volumes already defined in dfsmsrmm.....	123
Using the system-managed tape library with existing volumes.....	124
Using DFSMSrmm with an existing automated tape library.....	124
Returning volumes to scratch status.....	124
Partitioning system-managed tape libraries.....	125
Sharing a system-managed library and a BTLS-managed library.....	127
Moving from a non-system-managed to a system-managed IBM automated tape library.....	127
Chapter 6. Running DFSMSrmm with BTLS.....	129
Setting up scratch pools for BTLS-managed volumes.....	129
Running DFSMSrmm inventory management with BTLS.....	130
Running EDGINERS for BTLS-managed volumes.....	130
Restrictions.....	130
Defining volume information for BTLS-managed volumes.....	131
Returning BTLS-managed volumes to scratch.....	131
Chapter 7. Managing storage locations.....	133
Types of storage locations.....	133
Defining storage locations.....	134
Implementing installation defined storage locations.....	134
Implementing storage locations as home locations.....	135
Managing shelf space for home locations.....	136
Reusing bins in storage locations.....	136
Moving volumes to storage locations.....	136
Moving volumes by location.....	136
Moving volumes by media shape.....	137
Moving volumes manually.....	138
Assigning bins in storage locations.....	139
Changing storage locations.....	139
Deleting storage locations.....	139
Switching volumes to installation defined storage locations.....	140
Converting from built-in storage locations.....	140
Going back to built-in storage locations.....	140

Chapter 8. Using the parmlib member EDGRMMxx.....	143
Defining storage locations: LOCDEF.....	143
LOCDEF command syntax.....	144
LOCDEF command operands.....	144
Defining media information: MEDINF.....	148
MEDINF command syntax.....	149
MEDINF command operands.....	149
Using MEDINF REPLACE commands to implement volume replacement policies.....	151
Example of using the MEDINF STK command.....	153
Defining mount and fetch messages: MNTMSG.....	153
MNTMSG command syntax.....	154
MNTMSG command operands.....	155
ONLYIF.....	156
ONLYIF command syntax.....	156
Controlling the use of tape volumes: OPENRULE.....	157
OPENRULE command syntax.....	157
OPENRULE command operands.....	158
Defining system options: OPTION.....	161
OPTION command syntax.....	162
OPTION command operands.....	168
Using the OPTION command to switch from SMF record types to IBM-assigned SMF record types.....	196
Partitioning tape volumes: PRTITION.....	196
PRTITION command syntax.....	197
PRTITION command operands.....	197
Implementing PRTITION and OPENRULE parmlib commands.....	200
Reviewing installation exits.....	200
Identifying basic rules.....	200
Examples of OPENRULE and PRTITION commands.....	201
Defining tapes not available on systems: REJECT.....	202
REJECT command syntax.....	203
REJECT command operands.....	203
Converting REJECT commands to PRTITION and OPENRULE commands.....	204
Defining security classes: SECCLS.....	206
SECCLS command syntax.....	207
SECCLS command operands.....	208
Defining pools: VLPOOL.....	209
VLPOOL command syntax.....	209
VLPOOL command operands.....	210
 Chapter 9. Authorizing DFSMSrmm users and ensuring security.....	 215
Protecting DFSMSrmm resources with RACF profiles.....	215
Creating profiles.....	219
Setting the level of access for the DFSMSrmm resources.....	219
Authorizing resources.....	225
General user functions.....	228
Storage administrator functions.....	228
System programmer functions.....	229
Librarian functions.....	229
Inventory management functions.....	230
Operator functions.....	230
Using the tape relabeling resources.....	231
Creating audit trails.....	232
Control data set information.....	232
SMF audit information.....	232
RACF audit information.....	232

Using security classification processing.....	232
Preventing the use of IEHINITT.....	233
Controlling RACF tape profile processing.....	233
Recommendations for tape security.....	236
Recommendations for using RACF tape profile processing.....	237
Rejecting volumes on specific systems in a system complex.....	238
Maintaining the user access list.....	238
Using RACF with DFSMSrmm.....	239
DFSMSrmm RACF tape security support.....	239
DFSMSrmm automatic tape security support processing.....	239
Data set profile processing implications.....	239
RACF installation exit conversion.....	240
Using RACF options for authorizing RMM TSO subcommands.....	241
Using the SAF interface.....	241
SAF calls for authorization checking.....	241
SAF and RACF calls for creating, updating and deleting security profiles.....	244
Chapter 10. Using DFSMSrmm programming interfaces.....	249
Releasing tapes: EDGTVEXT.....	249
Invocation.....	250
Input.....	250
Output.....	250
Processing.....	250
Environment.....	250
Managing DFSMSHsm tapes: EDGDFHSM.....	250
Invocation.....	251
Input.....	251
Output.....	251
Processing.....	251
Environment.....	251
Managing system-managed tape library volumes: EDGLCSUX.....	251
Input.....	252
Output.....	253
Processing.....	255
Environment.....	263
Processing fetch and mount messages: EDGMSGEX.....	263
Input.....	263
Output.....	263
Processing.....	263
Environment.....	263
Processing JES3 messages: EDG3X71.....	264
Input.....	264
Output.....	264
Processing.....	264
Environment.....	264
Setting up parallel processing.....	264
Setting up parallel processing using SMP/E.....	265
Setting up parallel processing outside of SMP/E.....	266
Chapter 11. Using DFSMSrmm installation exits.....	267
Using the DFSMSrmm EDG_EXIT100 installation exit.....	267
Planning to manage scratch pools with EDG_EXIT100	268
Managing scratch pools.....	269
Specifying the retention method to be used for new tape data sets.....	271
Copying data set attributes in a tape copy application.....	273
Excluding specific data sets from VRSEL processing as they are created or rewritten.....	274
Ignoring duplicate or undefined volume serial numbers.....	275

Using vital record specification management values to retain tape volumes.....	279
Using the EDG_EXIT100 installation exit from pre-ACS processing.....	281
Creating sticky labels.....	281
Modifying DFSMSrmm label output.....	284
Controlling tape volume data set recording.....	285
Changing location information with EDG_EXIT100.....	287
EDG_EXIT100 exit routine processing.....	287
Setting up the EDG_EXIT100 routine environment.....	293
Installing the EDGUX100 default exit routine.....	293
Deleting the EDG_EXIT100 exit routines.....	294
Writing an exit routine for the EDG_EXIT100 exit.....	295
EDG_EXIT100 installation exit routine return codes.....	300
Using the EDG_EXIT200 installation exit.....	300
EDG_EXIT200 exit routine processing.....	300
Setting up the EDG_EXIT200 routine environment.....	301
Installing the EDGUX200 default exit routine.....	301
Deleting the EDG_EXIT200 exit routines.....	302
Writing an exit routine for the EDG_EXIT200 exit.....	303
EDG_EXIT200 installation exit routine return codes.....	305
Using the EDG_EXIT300 installation exit.....	305
EDG_EXIT300 exit routine processing.....	305
Setting up the EDG_EXIT300 routine environment.....	306
Installing the EDGUX300 default exit routine.....	306
Deleting the EDG_EXIT300 exit routines.....	307
Writing an exit routine for the EDG_EXIT300 exit.....	308
EDG_EXIT300 installation exit return codes.....	309

Chapter 12. Running DFSMSrmm with DFSMSHsm.....311

Defining DFSMSHsm to RACF.....	311
Authorizing DFSMSHsm to DFSMSrmm resources.....	311
Authorizing ABARS to DFSMSrmm resources.....	312
Setting DFSMSrmm options when using DFSMSHsm.....	312
Setting DFSMSHsm options when using DFSMSrmm.....	313
Setting DFSMSHsm system options.....	313
Setting DFSMSHsm dump definitions.....	313
DFSMSrmm support for DFSMSHsm naming conventions.....	314
DFSMSrmm support for retention and pooling.....	314
Retaining DFSMSHsm tapes using expiration dates.....	314
Using the EXPDT retention method to manage DFSMSHsm tapes.....	314
Using the VRSEL retention method to manage DFSMSHsm tapes.....	315
Retaining all DFSMSHsm tapes.....	315
Retaining open data sets.....	316
Retaining single file format migration tapes.....	316
Retaining multifile format migration tapes.....	317
Retaining single file format backup tapes.....	317
Retaining multifile format backup tapes.....	317
Retaining and moving TAPECOPY tapes or DUPLEX tapes.....	318
Retaining and moving dump tapes.....	319
Retaining and moving tapes written by ABARS.....	321
Retaining and moving ABARS accompany tapes.....	321
Retaining DFSMSHsm control data set backup tapes.....	322
Retaining cycles of dump tapes.....	322
Retaining DFSMSHsm tapes with extra days retention.....	324
Disaster recovery using DFSMSHsm alternate tapes with DFSMSrmm.....	324
Securing tapes when running DFSMSHsm and DFSMSrmm.....	325
Recommendations for using DFSMSrmm and DFSMSHsm.....	325

Chapter 13. Performing inventory management.....	327
Scheduling DFSMSrmm utilities.....	327
Running inventory management.....	328
Inventory management considerations for EXPROC and VRSEL processing.....	329
Managing exceptions in retention.....	329
Inventory management considerations.....	332
DFSMSrmm inventory management considerations when client/server support is enabled.....	332
Allocating data sets for inventory management.....	333
Creating an extract data set.....	335
JCL for EDGHSKP.....	337
EXEC parameters for EDGHSKP.....	337
SYSIN file for the EDGHSKP utility.....	342
EDGSPLCS file for the EDGHSKP utility.....	345
Running vital record processing.....	345
JCL for vital record processing.....	347
Using the vital records retention report.....	348
Using the inventory management ACTIVITY file.....	352
How vital record processing works.....	352
Running storage location management processing.....	357
JCL for storage location management processing.....	357
How storage location management processing works.....	357
Running expiration processing.....	359
JCL for expiration processing.....	359
How expiration processing works.....	360
Running DFSMSrmm catalog synchronization.....	365
DFSMSrmm catalog processing.....	366
JCL for catalog synchronization.....	367
Synchronizing the DFSMSrmm control data set with user catalogs in a fully shared catalog environment.....	367
Synchronizing the DFSMSrmm control data set with user catalogs when catalogs are not fully shared.....	368
Confirming global volume movement.....	369
Confirming global release actions.....	369
Backing up the control data set.....	370
Ensuring a consistent copy of the DFSMSrmm CDS.....	371
JCL for using backup to create a CDS copy.....	372
JCL for backing up the control data set and journal.....	372
Backing up the journal.....	373
JCL for backing up the journal.....	373
Steps for automating control data set backup and journal clearing.....	374
Return codes for EDGHSKP.....	374
 Chapter 14. Maintaining the control data set.....	 377
DFSMSrmm considerations when client/server support is enabled.....	378
Using EDGBKUP.....	379
JCL for EDGBKUP.....	379
EXEC parameters for EDGBKUP.....	380
DD statements for EDGBKUP.....	382
SYSIN file for EDGBKUP.....	382
Return codes for EDGBKUP.....	384
Additional EDGBKUP return code information.....	384
Customizing the DSSOPT DD statement.....	384
Backing up the control data set.....	386
Backing up the DFSMSrmm control data set and journal.....	387
Updating the active control data set.....	387
JCL for EDGUPDT.....	388

DD statements for EDGUPDT	388
SYSIN file for EDGUPDT.....	389
Restoring the control data set.....	389
Controlling the control data set recovery point.....	390
Restoring the control data set with forward recovery.....	390
Restoring the control data set without forward recovery.....	391
Forward recovering the control data set.....	391
Restoring the control data set at a recovery site.....	392
Using non-dfsmsrmm utilities to restore the control data set.....	392
Reorganizing the control data set.....	393
Monitoring the space used by the control data set.....	394
Changing the size of the control data set and journal.....	395
Recovering from control data set update failures.....	395
Recovery processing.....	395
Handling I/O requests following a failure.....	396
Moving the control data set and journal to a different device.....	397
Steps for moving the control data set and journal using the DFSMSrmm EDGHSKP utility with the PARM='BACKUP' parameter.....	397
Steps for moving the control data set and journal using DFSMSrmm utility EDGHSKP utility with the PARM='BACKUP(DSS)' parameter.....	398
Moving the journal using DFSMSrmm utilities.....	399
Steps for moving the control data set using non-dfsmsrmm utilities.....	400
Using EDGUTIL for tasks such as creating and verifying the control data set.....	401
JCL for EDGUTIL.....	401
EXEC parameters for EDGUTIL.....	402
SYSIN file for VERIFY and MEND processing.....	406
How EDGUTIL performs VERIFY and MEND processing for volumes.....	408
Creating or updating the control data set control record.....	409
Verifying the contents of the control data set.....	411
Verifying the control data set and tape configuration database.....	413
Synchronizing the contents of the control data set.....	413
Mending the control data set.....	414
Setting up DFSMSrmm stacked volume support.....	415
Setting up DFSMSrmm common time support.....	416
Enabling extended bin support.....	417
EDGSPLCS file for the EDGUTIL utility.....	418
Return codes for EDGUTIL.....	418
Using EDGSPLCS to issue commands to OAM for system-managed volumes.....	418
EXEC parameters for EDGSPLCS.....	419
INDD input file.....	419
OUTDD output file.....	421
Return codes for EDGSPLCS.....	421
Sharing the DFSMSrmm control data set.....	421
Running DFSMSrmm inventory management when sharing the control data set.....	421
Running EDGINERS when sharing the control data set.....	422
Defining volume information when sharing the control data set.....	422
Confirming volume movement when sharing the control data set.....	422
Returning volumes to scratch when sharing the control data set.....	422
Chapter 15. Initializing, erasing, and scanning tape volumes.....	423
Replacing IEHINITT with EDGINERS.....	424
Using EDGINERS.....	424
Initializing and erasing volumes automatically.....	425
Initializing,Erasing, and scanning volumes manually.....	425
Initializing and erasing volumes automatically using multiple tape drives.....	426
JCL for EDGINERS.....	426
SCAN output.....	436

Using EDGINERS with system-managed tape libraries.....	438
Controlling access to EDGINERS.....	441
How DFSMSrmm selects an ISO/ANSI label Version.....	441
Producing label symmetry.....	441
How EDGINERS processing works.....	442
Return codes for EDGINERS.....	442
EDGINERS examples.....	443
Example 1: Write IBM standard labels on three tapes.....	443
Example 2: Write an ISO/ANSI label on a tape.....	443
Example 3: Place two groups of serial numbers on six tape volumes.....	444
Example 4: Place serial numbers on eight tape volumes.....	444
Example 5: Relabel a volume.....	445
Example 6: Automatically initialize or erase 3480 volumes.....	445
Example 7: Automatically initialize and erase volumes in a system-managed library.....	446
Example 8: Automatically initialize 50 scratch enhanced capacity cartridges.....	446
Example 9: Manually erase a volume.....	446
Example 10: Automatically initialize volumes using multiple tape drives.....	447
Example 11: Manually labeling duplicate volumes using EDGINERS.....	447
Example 12: Selecting EHPCT volumes for processing automatically.....	447
Example 13: Manually scanning a system managed DFSMSrmm volume.....	448
Chapter 16. Customizing DFSMSrmm.....	449
Changing the initial entry point to the DFSMSrmm dialog.....	449
Adding local dialog extensions.....	450
Customizing the local dialog with 'U' line command.....	451
Customize point-and-shoot fields in the DFSMSrmm dialog.....	451
Changing the ADD product volume defaults.....	451
Customizing DFSMSrmm messages for report titles and user notification.....	451
Customizing DFSMSrmm report titles.....	452
Customizing notification messages and notes.....	452
Managing VM tape volumes.....	458
Replenishing scratch volumes in a system-managed library.....	458
Automating backup.....	459
Using the LABEL procedure.....	459
Processing NL label tapes: EDG019VM.....	460
Input.....	460
Output.....	460
Processing.....	460
Environment.....	460
Chapter 17. Using the Problem Determination Aid Facility	461
Roadmap for using the Problem Determination Aid.....	461
Planning to use the PDA facility.....	462
Determining how long to keep trace information.....	462
Short-term trace history.....	462
Long-term trace history.....	462
Determining Problem Determination Aid (PDA) log data set size.....	462
Enabling the Problem Determination Aid (PDA) facility.....	462
Allocating the Problem Determination Aid (PDA) log data sets.....	463
Increasing the size of the Problem Determination Aid (PDA) log data sets.....	463
Maintaining a history of Problem Determination Aid (PDA) log data.....	464
Printing the Problem Determination Aid (PDA) log data sets.....	465
Chapter 18. Setting up DFSMSrmm disposition processing.....	467
Implementing DFSMSrmm disposition control file processing.....	467
Modifying the contents of the disposition control file.....	468
Selecting the method used for label processing.....	470

Modifying tape labels.....	471
Chapter 19. Running DFSMSrmm with JES3.....	473
Preventing JES3 from validating volumes.....	473
Updating JES3 fetch and mount messages.....	473
Steps for using the EDG3UX71 USERMOD.....	473
Using the EDG3IIP1 USERMOD.....	474
Using the EDG3LVVR USERMOD.....	474
Using the EDG3UX62 USERMOD to create and mount no label tapes.....	474
Chapter 20. Running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS.....	477
Using a Tivoli special resource when running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS.....	477
Setting up DFSMSrmm to use the IBM Tivoli Workload Scheduler for z/OS.....	478
Descriptions of DFSMSrmm jobs to run with the IBM Tivoli Workload Scheduler for z/OS.....	478
IBM Tivoli Workload Scheduler for z/OS applications for DFSMSrmm.....	480
Customizing the IBM Tivoli Workload Scheduler for z/OS batch loader statements.....	481
Setting up IBM Tivoli Workload Scheduler for z/OS workstations.....	481
Event triggered tracking.....	482
Chapter 21. Using the parmlib member EDGDEFxx.....	483
Introduction to the defaults table.....	483
Converting from using UXTABLE to using the defaults table.....	485
Modifying the defaults table.....	486
Defining defaults table entries: DEFAULT.....	486
DEFAULT command syntax.....	486
DEFAULT command operands.....	487
Sharing the defaults table on multiple systems.....	493
Chapter 22. The DFSMSrmm WUI.....	495
Setting up the DFSMSrmm WUI.....	495
Installing the DFSMSrmm WUI.....	495
Using the DFSMSrmm WUI.....	497
Volumes tab.....	497
Data Sets tab.....	500
Defaults Table tab.....	503
Monitoring tab.....	505
Settings tab.....	506
Network Logs tab.....	507
Troubleshooting.....	508
Appendix A. DFSMSrmm mapping macros.....	511
OAM interface: EDGLCSUP.....	511
Installation exit mapping macro: EDGPL100.....	517
Installation exit mapping macro: EDGPL200.....	524
Installation exit mapping macro: EDGPL300.....	526
Sticky label data: EDGSLAB.....	529
Appendix B. Using DFSMSrmm samples.....	533
Appendix C. Evaluating removable media management needs.....	539
Appendix D. Problem Determination Aid log data set size work sheet for long-term trace history.....	541

Appendix E. Problem Determination Aid log data set size work sheet for short-term trace history.....	543
Appendix F. Accessibility.....	545
Notices.....	547
Terms and conditions for product documentation.....	548
IBM Online Privacy Statement.....	549
Policy for unsupported hardware.....	549
Minimum supported hardware.....	549
Programming interface information.....	550
Trademarks.....	550
Index.....	551

Figures

1. Defining a vital record specification chain.....	6
2. Example of a list of volumes owned by a single user.....	15
3. Defining DFSMSrmm to z/OS through IEFSSNxx with subsystem inactive.....	26
4. Updating IKJTSoxx to authorize RMM commands.....	26
5. Updating IKJTSoxx to call DFSMSrmm through TSO.....	27
6. Converting the SYSTEMS enqueue to a local SYSTEM enqueue.....	28
7. Changing an existing definition from SPECIFIC to GENERIC.....	28
8. Converting the RESERVE to a SYSTEMS enqueue.....	28
9. Changing an existing SPECIFIC definition to GENERIC.....	29
10. Creating a procedure in SYS1.PROCLIB using the recommended JCL.....	43
11. Creating a procedure in SYS1.PROCLIB using additional parameters.....	44
12. Allocating DASD space for the control data set.....	51
13. Initializing the control data set.....	52
14. Allocating space for the journal.....	55
15. Adding DFSMSrmm to ISPF.....	57
16. Enabling ISPF data set list (DSLST) support.....	58
17. Starting DFSMSrmm.....	60
18. Starting DFSMSrmm with additional parameters.....	61
19. Stopping DFSMSrmm.....	61
20. Disabling the DFSMSrmm subsystem interface.....	61
21. Quiescing the DFSMSrmm subsystem interface.....	62
22. Defining minimum volume information.....	64
23. Defining volumes in a system-managed library.....	64

24. Defining volumes in a manual tape library.....	64
25. Changing SYS1.PARMLIB IEFSSNxx	66
26. Restarting the DFSMSrmm subsystem.....	67
27. Message EDG0103D.....	68
28. Default VLPOOL command.....	79
29. Defining pools with the DFSMSrmm EDGRMMxx parmlib VLPOOL command.....	83
30. Sample management class routine.....	83
31. Storage group routine sample.....	84
32. Defining pools with VLPOOL commands.....	84
33. Defining vital record specifications for non-system-managed volumes.....	90
34. Sample management class routine for managing non-system-managed volumes.....	90
35. Assigning management class to data sets on tape.....	92
36. Special date 99000 vital record specification.....	92
37. Managing management classes using a data set name mask.....	93
38. Specifying a library name for a volume.....	100
39. Requesting the eject of a volume.....	102
40. Contents of the shipped table: TAPEUNITS.....	103
41. Changing volume type.....	109
42. Searching the required location for logical volumes.....	114
43. Confirming volume moves for exported volumes.....	115
44. Building a list of stacked volumes to be imported from a single stacked volume.....	115
45. Building a list of logical volumes to be imported from a single stacked volume.....	116
46. Building a list of logical volumes to be imported from multiple stacked volumes.....	116
47. Creating a volume export list.....	119
48. Confirming volume moves for exported volumes.....	119

49. Creating a volume import list.....	120
50. Converting A logical volume to a physical volume.....	122
51. Reusing volume information for a recovered volume.....	122
52. Sample JCL to return volumes to scratch.....	131
53. Using media name to control volume movement.....	138
54. Keeping volumes on-site.....	138
55. Sending a volume to another system.....	139
56. Returning a volume from another system.....	139
57. Identifying an installation defined storage location.....	140
58. Parmlib member EDGRMMxx LOCDEF command example.....	144
59. Parmlib member EDGRMMxx LOCDEF command for defining storage locations.....	144
60. Parmlib member EDGRMMxx LOCDEF command for defining shelf and system-managed libraries..	144
61. Using a medianame not defined in the LOCDEF command.....	145
62. Parmlib member EDGRMMxx MEDINF command example for OEM media products.....	148
63. Parmlib member EDGRMMxx MEDINF command syntax.....	149
64. Parmlib member EDGRMMxx MNTMSG command examples for 4-digit devices.....	154
65. Parmlib member EDGRMMxx MNTMSG command syntax.....	154
66. ONLYIF statement.....	156
67. Parmlib member EDGRMMxx ONLYIF command syntax.....	156
68. Parmlib member EDGRMMxx OPENRULE command.....	158
69. Parmlib member EDGRMMxx OPTION command examples.....	162
70. Parmlib member EDGRMMxx OPTION command syntax.....	162
71. Parmlib member EDGRMMxx PRTITION command.....	197
72. Volumes not used and partitioned.....	201
73. Volumes not used for output on this system.....	201

74. Volumes partitioned and not used on system.....	201
75. Volumes partitioned, not used on system, but allowed to be read.....	202
76. Three systems using a range of system-managed volumes.....	202
77. Parmlib member EDGRMMxx REJECT command examples.....	203
78. Parmlib member EDGRMMxx REJECT command syntax.....	203
79. Parmlib member EDGRMMxx SECCLS command examples.....	207
80. Parmlib member EDGRMMxx SECCLS command syntax.....	207
81. Parmlib member EDGRMMxx VLPOOL command examples.....	209
82. Parmlib member EDGRMMxx VLPOOL command syntax.....	210
83. Creating a RACF profile.....	219
84. Creating a RACF profile with audit options.....	219
85. Limiting the use of IEHINITT.....	233
86. Using the REJECT parmlib option.....	238
87. Creating an ACEE for a user defined to RACF.....	242
88. Creating an ACEE for a user not defined to RACF.....	242
89. Checking authorization.....	247
90. Sample USERMOD for setting up running exits in parallel.....	265
91. Sample JCL for setting up running exits in parallel.....	266
92. Example JCL for Setting Up Running Exits in Parallel.....	266
93. Defining pools for a specific application.....	270
94. Sample retention method selection table.....	272
95. Sample VRSELEXCLUDE selection table.....	275
96. Managing special date 99000 with vital record management value.....	280
97. Specifying data set masks for vital record management values.....	280
98. Sample EDGUX100 exit module sticky label support.....	282

99. Sample EDGUX100 exit module sticky label support.....	283
100. Sample EDGUX100 exit module sticky label support location.....	283
101. Sample EDGUX100 exit module sticky label support own labels.....	284
102. Sticky label area.....	284
103. Addressing the sticky label area.....	285
104. Mapping a custom sticky label.....	285
105. Sample table for controlling data set recording.....	286
106. Sample EDGUX100 pool selection table.....	290
107. Sample EDGUX100 exit module system name table.....	292
108. Pool selection table for system 3.....	292
109. Building an SMP/E USERMOD to apply the updated EDGUX100 exit module.....	294
110. Building an SMP/E USERMOD to apply the updated EDGUX200 exit module.....	302
111. Building an SMP/E USERMOD to apply the updated EDGUX300 exit module.....	307
112. Retaining DFSMSHsm tapes that require no movement.....	315
113. Retaining DFSMSHsm tapes with the DFSMSHsm and ABARS procedure name.....	316
114. Keeping all single file format migration tapes.....	316
115. Keeping multifile format migration tapes.....	317
116. Keeping single file format backup tapes.....	317
117. Keeping multifile format backup tapes.....	317
118. Keeping tapes created by the DFSMSHsm TAPECOPY command and DUPLEX tape feature.....	318
119. Keeping tapes used for dump.....	319
120. Managing cycles of dumps.....	319
121. Retaining and moving volumes by cycles.....	320
122. Keeping ABARS tapes.....	321
123. Keeping ABARS accompany tapes.....	321

124. Keeping DFSMSHsm control data set and journal backup tapes.....	322
125. Moving dumps to storage locations.....	322
126. Keeping ABARS backup tapes using GDG names.....	323
127. JCL for adding a DD statement to the EDGHSKP job step.....	335
128. JCL for specifying the extract data set.....	336
129. JCL for creating an extract data set.....	336
130. Example of JCL for EDGHSKP.....	337
131. EDGHSKP EXEC parameters.....	338
132. RPTEXT command.....	344
133. Sample JCL for allocating the EDGSPLCS file prior to EDGHSKP processing.....	345
134. Example of JCL for vital record specification processing.....	347
135. Example of JCL for trial run vital record specification processing.....	348
136. Example of JCL for storage location management processing.....	357
137. Example of JCL for storage location management processing using the LOCATION, INSEQUENCE, and REASSIGN parameters.....	357
138. Automatically ejecting volumes from system-managed libraries.....	358
139. Example of JCL for expiration processing.....	359
140. Example of JCL for trial run expiration processing.....	359
141. Example of JCL for expiration processing.....	359
142. Expiration processing report.....	364
143. Example of JCL for catalog synchronization processing.....	367
144. Example of JCL for trial run catalog synchronization processing.....	367
145. Example of JCL for using backup to create a CDS copy.....	372
146. Example of JCL for backing up the control data set and the journal to tape.....	373
147. Example of JCL for backing up the control data set and journal to DASD.....	373
148. Example of JCL for backing up and clearing the journal.....	374

149. JCL example for EDGBKUP.....	380
150. EDGBKUP EXEC parameters.....	380
151. EDGBKUP SYSIN file.....	383
152. DFSMSdss commands that are issued by DFSMSrmm.....	385
153. JCL example for EDGUPDT.....	388
154. EDGUPDT SYSIN file.....	389
155. Restoring the control data set with forward recovery.....	390
156. JCL example for backing up the control data set and journal.....	400
157. Creating the control data set.....	402
158. Updating the control data set.....	402
159. Verifying the contents of the control data set.....	402
160. Mending the control data set.....	402
161. EDGUTIL EXEC parameters.....	403
162. EDGUTIL SYSIN file.....	406
163. EDGUTIL SYSIN commands.....	409
164. Example of JCL for VERIFY(STORE).....	411
165. Sample EDGUTIL SYSPRINT output.....	412
166. Sample JCL for verifying the control data set and the TCDB.....	413
167. Changing volume type and volume location.....	415
168. Sample JCL for allocating the EDGSPLCS file during EDGUTIL processing.....	418
169. EDGSPLCS EXEC parameters.....	419
170. JCL for EDGINERS automatic processing.....	426
171. JCL for EDGINERS manual processing.....	426
172. JCL for initializing volumes with ISO/ANSI Version 4 VOL1 and HDR1 labels.....	426
173. EDGINERS EXEC parameters.....	427

174. EDGINERS SYSIN commands (erasing and initializing).....	433
175. EDGINERS SYSIN command (for scanning).....	436
176. Example of scan output.....	436
177. Defining scratch volumes to be initialized.....	440
178. Changing the initialization action for a volume.....	440
179. Using IEBGENER.....	441
180. Writing IBM standard labels on three tapes.....	443
181. Writing an ISO/ANSI label on a tape.....	444
182. Numbering tape volumes.....	444
183. Placing serial numbers on eight tape volumes.....	445
184. Relabeling a volume.....	445
185. Automatically initialize or erase 3480 volumes.....	445
186. Automatically initialize and erase volumes in a system-managed library.....	446
187. Initialize 50 scratch enhanced capacity cartridges.....	446
188. Erase a volume.....	446
189. Initialize volumes using multiple tape drives.....	447
190. Labeling duplicate volumes using EDGINERS.....	447
191. Selecting volumes for automatic processing.....	447
192. Scanning a system managed volume.....	448
193. RMMISPF exec syntax diagram.....	449
194. Adding DFSMSrmm Librarian Option to ISPF.....	450
195. Before modifying the Trailer 1 for EDGRPTD.....	452
196. After modifying the Trailer 1 for EDGRPTD.....	452
197. Notify owner messages.....	453
198. Default notification text.....	453

199. Modified messages.....	454
200. Modified notification text.....	454
201. Notifying product owner.....	454
202. Customizing message numbers 2450-2463 for release notification.....	456
203. Example of customizing message numbers 2450-2463 for release notification.....	456
204. EDGXPROC procedure.....	459
205. Example BACKUPPROC procedure.....	459
206. Sample label procedure.....	460
207. Disposition control file record format.....	468
208. Sample JCL to request disposition processing.....	470
209. Default label format for a tape cartridge.....	471
210. Default label format for a round tape.....	471
211. Overriding vital record specification management processing with the RMM CHANGEDATASET TSO subcommand.....	478
212. Parmlib member EDGDEFxx DEFAULT command syntax.....	486
213. JCL for allocating and cataloging PDA log data sets.....	543

Tables

1. Character sets.....	xxxiii
2. Special characters used in syntax.....	xxxiii
3. DFSMSrmm movement priority.....	7
4. How the DFSMSrmm running mode affects tape mount validation.....	18
5. DFSMSrmm resource symbolic names.....	29
6. Data sets requiring access by the DFSMSrmm RACF user ID.....	46
7. Creating DFSMSrmm parmlib definitions.....	47
8. DFSMSrmm control data set DASD space requirements.....	49
9. DFSMSrmm journal DASD space requirements.....	53
10. Default libraries to concatenate.....	58
11. DFSMSrmm volumes in a pool determined by pool prefix.....	78
12. DFSMSrmm entry processing decisions.....	109
13. Differences between built-in and installation defined storage locations.....	134
14. Storing media of the same shape.....	137
15. Storing media of different shapes.....	137
16. How OPMODE honors the settings of various options.....	181
17. How OPMODE value affects system-managed tape library support.....	181
18. Converting REJECT commands to OPENRULE and PRTITION commands.....	205
19. Resources you protect with RACF profiles.....	215
20. Authorized functions.....	219
21. Suggested resource access to limit scope of tasks.....	225
22. Suggested resource access without limited tasks.....	226
23. General user functions.....	228

24. Storage administrator functions.....	228
25. System programmer functions.....	229
26. Librarian functions.....	229
27. Inventory management functions.....	230
28. Operator functions.....	231
29. RACF processing performed by DFSMSrmm.....	235
30. Installation exits used by DFSMSrmm.....	249
31. OAM installation exits.....	252
32. EDGLCSUX return and reason codes returned in register 15 and register 0.....	253
33. EDGLCSUX return and reason codes based on DFSMSrmm reason code setting.....	254
34. Processing for the change use attribute, cartridge entry and cartridge eject parameter list.....	256
35. Processing for the change use attribute parameter list.....	260
36. Processing for the cartridge entry parameter list.....	261
37. Processing for the cartridge eject parameter list.....	261
38. Processing for the volume-not-in-library parameter list.....	261
39. DFSMSrmm OAM return codes from EDGLCSUX register 15.....	262
40. EDG_EXIT100 installation routine exit return codes.....	300
41. EDG_EXIT200 installation exit routine return codes.....	305
42. EDG_EXIT300 installation exit return codes.....	309
43. Authorization required to use scratch tapes with DFSMShsm.....	311
44. Authorization required to use DFSMShsm with a DFSMShsm scratch pool.....	312
45. Authorization required to use DFSMSrmm with ABARS.....	312
46. Scheduling DFSMSrmm utilities.....	327
47. DFSMSrmm EDGHSKP data sets.....	333
48. DFSMSrmm extract data set DASD space requirements.....	335

49. DFSMSrmm movement priority default values.....	356
50. EDGHSKP return codes.....	374
51. DFSMSrmm EDGBKUP data sets.....	382
52. EDGBKUP return codes.....	384
53. DFSMSrmm EDGUPDT data sets.....	388
54. Information checked by EDGUTIL for inconsistencies.....	413
55. EDGUTIL return codes.....	418
56. INDD input file for the EDGSPLCS utility.....	420
57. EDGSPLCS return codes.....	421
58. Label data comparison with DFSMSrmm data.....	437
59. EDGINERS return codes.....	442
60. Customizing report titles.....	452
61. Setting the message route code.....	469
62. Coding sticky label text.....	469
63. IBM Tivoli Workload Scheduler for z/OS applications.....	480
64. Structure LCSUP.....	512
65. Constants for LCSUP.....	514
66. Cross Reference for LCSUP.....	515
67. Structure PL100.....	518
68. Structure PL100_LABDS.....	520
69. Constants for PL100.....	521
70. Cross Reference for PL100.....	521
71. Structure PL200.....	525
72. Constants for PL200.....	525
73. Cross Reference for PL200.....	526

74. Structure PL300.....	527
75. Constants for PL300.....	527
76. Cross Reference for PL300.....	528
77. Structure SLAB.....	529
78. Constants for SLAB.....	530
79. Cross Reference for SLAB.....	531
80. SAMPLIB and SMPSTS Members.....	533
81. Evaluating Removable Media Management Needs.....	539

About this document

This document is intended for storage administrators and system programmers who are responsible for implementing and customizing DFSMSrmm.

Before using this document, you should read *z/OS Upgrade Workflow* for detailed migration information for DFSMSrmm as well as other DFSMS functional components. Additionally, you should have installed DFSMSrmm with SMP/E using the directions in *z/OS Program Directory* in the *z/OS Internet library* (www.ibm.com/servers/resourcelink/svc00100.nsf/pages/zosInternetLibrary).

How to use this document

This document explains how to implement DFSMSrmm for users with no existing media management system.

Refer to these topics to implement DFSMSrmm:

- [Chapter 1, “Introducing DFSMSrmm,” on page 1](#)
- [Chapter 2, “Implementing DFSMSrmm,” on page 23](#)
- [Chapter 3, “Setting up DFSMSrmm client and server systems,” on page 71](#)
- [Chapter 4, “Organizing the removable media library,” on page 75](#)
- [Chapter 5, “Running DFSMSrmm with system-managed tape libraries,” on page 99](#)
- [Chapter 6, “Running DFSMSrmm with BTLS,” on page 129](#)
- [Chapter 7, “Managing storage locations,” on page 133](#)
- [Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143](#)
- [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215](#)
- [Chapter 10, “Using DFSMSrmm programming interfaces,” on page 249](#)
- [Chapter 11, “Using DFSMSrmm installation exits,” on page 267](#)
- [Chapter 12, “Running DFSMSrmm with DFSMSshm,” on page 311](#)
- [Chapter 19, “Running DFSMSrmm with JES3,” on page 473](#)
- [Chapter 13, “Performing inventory management,” on page 327](#)
- [Chapter 14, “Maintaining the control data set,” on page 377](#)
- [Chapter 15, “Initializing, erasing, and scanning tape volumes,” on page 423](#)

To set up and run DFSMSrmm utilities, refer to:

- [Chapter 13, “Performing inventory management,” on page 327](#)
- [Chapter 14, “Maintaining the control data set,” on page 377](#)
- [Chapter 15, “Initializing, erasing, and scanning tape volumes,” on page 423](#)

To customize DFSMSrmm, refer to:

- [Chapter 10, “Using DFSMSrmm programming interfaces,” on page 249](#)
- [Chapter 11, “Using DFSMSrmm installation exits,” on page 267](#)
- [Chapter 16, “Customizing DFSMSrmm,” on page 449](#)
- [Chapter 17, “Using the Problem Determination Aid Facility,” on page 461](#)
- [Chapter 18, “Setting up DFSMSrmm disposition processing,” on page 467](#)
- [Chapter 20, “Running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS,” on page 477](#)

How to find samples in this document

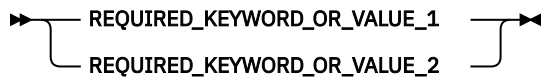
Throughout this document, when a task has an available sample, we point you to the sample using a figure like the one that follows:

CBRUXENT Programming Interface to EDGLCSUX

Required product knowledge

- RACF, a component of the Security Server for z/OS for defining resources and creating access lists
- ISPF, for using and customizing the DFSMSrmm ISPF dialog
- DFSMSHsm, for customizing the interaction between DFSMSHsm and DFSMSrmm
- OAM, for customizing the interface between DFSMSrmm and OAM
- DFSMSdss, for backing up the DFSMSrmm control data set and journal

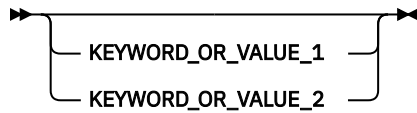
Notational conventions



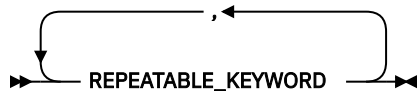
- Optional keywords and values appear below the main path line. You can choose not to code optional keywords and values.



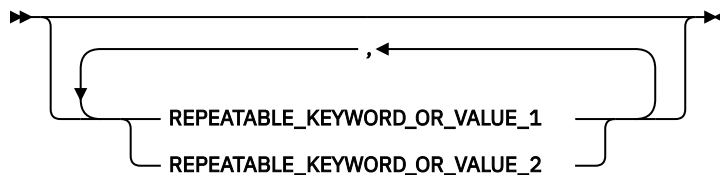
If several mutually exclusive optional keywords or values exist, they are stacked vertically in alphanumeric order below the main path line.



- An arrow returning to the left above a keyword or value on the main path line means that the keyword or value can be repeated. The comma means that each keyword or value must be separated from the next by a comma.



- An arrow returning to the left above a group of keywords or values means more than one can be selected, or a single one can be repeated.



- A word in all uppercase is a keyword or value you must spell exactly as shown. In this example, you must code **KEYWORD**.

►► **KEYWORD** ►►

If a keyword or value can be abbreviated, the abbreviation is discussed in the text associated with the syntax diagram.

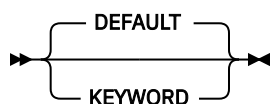
- If a diagram shows a character that is not alphanumeric (such as parentheses, periods, commas, and equal signs), you must code the character as part of the syntax. In this example, you must code **KEYWORD=(001,0.001)**.

►► **KEYWORD=(001,0.001)** ►►

- If a diagram shows a blank space, you must code the blank space as part of the syntax. In this example, you must code **KEYWORD=(001 FIXED)**.

►► **KEYWORD=(001 FIXED)** ►►

- Default keywords and values appear above the main path line. If you omit the keyword or value entirely, the default is used.



- A word in all lowercase italics is a *variable*. Where you see a variable in the syntax, you must replace it with one of its allowable names or values, as defined in the text.

►► *variable* —¹►►

Notes:

¹ An example of a syntax note.

- References to syntax notes appear as numbers enclosed in parentheses above the line. Do not code the parentheses or the number.

►► KEYWORD ►►

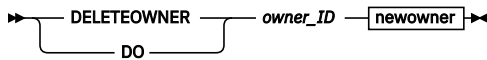
- Some diagrams contain *syntax fragments*, which serve to break up diagrams that are too long, too complex, or too repetitious. Syntax fragment names are in mixed case and are shown in the diagram and in the heading of the fragment. The fragment is placed below the main diagram.

►► Reference to syntax fragment ►►

Syntax fragment

►► 1ST_KEYWORD,2ND_KEYWORD,3RD_KEYWORD ►►

The following is an example of a syntax diagram.



newowner



Notes:

¹ Must be specified if the owner owns one or more volumes.

The possible valid versions of the RMM DELETEOWNER command are:

```

RMM DELETEOWNER owner
RMM DO owner
RMM DELETEOWNER owner NEWOWNER(new_owner)
RMM DO owner NEWOWNER(new_owner)

```

How to abbreviate commands and operands

The TSO abbreviation convention applies for all DFSMSrmm commands and operands. The TSO abbreviation convention requires you to specify as much of the command name or operand as is necessary to distinguish it from the other command names or operands.

Some DFSMSrmm keyword operands allow unique abbreviations. All unique abbreviations are shown in the command syntax diagrams.

How to use continuation characters

The symbol - is used as the continuation character in this document. You can use either - or +.

-

Do not ignore leading blanks on the continuation statement

+

Ignore leading blanks on the continuation statement

Delimiters

When you type a command, you must separate the command name from the first operand by one or more blanks. You must separate operands by one or more blanks or a comma. Do not use a semicolon as a delimiter because any character you enter after a semicolon is ignored.

Character sets

To code job control statements, use characters from the character sets in Character sets.

Special characters used in syntax lists the special characters that have syntactical functions in job control statements.

Table 1. Character sets. This table lists the special characters that have syntactical functions in job control statements.		
Character Set	Contents	
Alphanumeric	Alphabetic Numeric	Capital A through Z 0 through 9
National (See note)	“At” sign Dollar sign Pound sign	@ (Characters that can be \$ represented by hexadecimal # values X'7C', X'5B', and X'7B')
Special	Comma Period Slash Apostrophe Left parenthesis Right parenthesis Asterisk Ampersand Plus sign Hyphen Equal sign Blank	, . / ' () * & + - =
EBCDIC text	EBCDIC printable character set	Characters that can be represented by hexadecimal X'40' through X'FE'
Note: The system recognizes the following hexadecimal representations of the U.S. National characters; @ as X'7C'; \$ as X'5B'; and # as X'7B'. In countries other than the U.S., the U.S. National characters represented on terminal keyboards might generate a different hexadecimal representation and cause an error. For example, in some countries the \$ character might generate a X'4A'.		

Table 2. Special characters used in syntax	
Character	Syntactical Function
,	To separate parameters and subparameters
=	To separate a keyword from its value, for example, BURST=YES
(b)	To enclose subparameter list or the member name of a PDS or PDSE
&	To identify a symbolic parameter, for example, &LIB
&&	To identify a temporary data set name, for example, &&TEMPDS, and, to identify an in-stream or sysout data set name, for example, &&PAYOUT
.	To separate parts of a qualified data set name, for example, A.B.C., or parts of certain parameters or subparameters, for example, nodename.userid
*	To refer to an earlier statement, for example, OUTPUT=*name, or, in certain statements, to indicate special functions: //label CNTL * //ddname DD * RESTART=* on the JOB statement
'	To enclose specified parameter values that contain special characters
(blank)	To delimit fields

How to provide feedback to IBM

We welcome any feedback that you have, including comments on the clarity, accuracy, or completeness of the information. For more information, see [How to send feedback to IBM](#).

Summary of changes

This information includes terminology, maintenance, and editorial changes. Technical changes or additions to the text and illustrations for the current edition are indicated by a vertical line to the left of the change.

Note: IBM z/OS policy for the integration of service information into the z/OS product documentation library is documented on the z/OS Internet Library under [IBM z/OS Product Documentation Update Policy](http://www.ibm.com/docs/en/zos/latest?topic=zos-product-documentation-update-policy) (www.ibm.com/docs/en/zos/latest?topic=zos-product-documentation-update-policy).

Summary of changes for z/OS 3.1

The following content is new, changed, or no longer included in z/OS 3.1.

New

The following content is new.

March 2025 release

- Added note. For more information, see [“Partitioning tape volumes: PRTITION”](#) on page 196.

February 2025 release

- Added DFSMSHsm process related to RMM CDS. For more information, see [“Planning to back up the control data set”](#) on page 52
- Added Note. For more information, see [“VLPOOL command operands”](#) on page 210
- Added Note. For more information, see [“Defining the DFSMSrmm control data set”](#) on page 49.

September 2023 release

- The HSKP keyword is added. For more information, see [“Defining system options: OPTION”](#) on page 161 and [“OPTION command operands”](#) on page 168.
- VLPOOL RACF values are updated for TPRACF(NONE), TPRACF(PREDEFINED), and TPRACF(AUTOMATIC). For more information, see [“OPTION command operands”](#) on page 168.
- A new procedure in the installation is added for the z/OSMF "Import Manager" plug in, see [“Installing the DFSMSrmm WUI”](#) on page 495 and a description was updated for the management class expiration attributes processed by DFSMSrmm, see [“OPTION command operands”](#) on page 168.

Changed

The following content is changed.

June 2025 release

- Updated data set mask naming rules for DSNAME in [“DEFAULT command operands”](#) on page 487

March 2025 release

- The Note description in [“Partitioning system-managed tape libraries”](#) on page 125 is updated.
- The Manual mode description is updated in [“OPTION command operands”](#) on page 168.

February 2025 release

- Updated PDA log data sets > SPACE CYCLE value in [“Allocating the Problem Determination Aid \(PDA\) log data sets”](#) on page 463.

June 2024 refresh

- “Installing the DFSMSrmm WUI” on page 495 and “Settings tab” on page 506 is updated to add support for APAR [OA65420: RMM zosmf proclib concatenation \(www.ibm.com/support/pages/apar/OA65420\)](http://www.ibm.com/support/pages/apar/OA65420) (which applies to z/OS V2R5 and newer).
- “OPENRULE command operands” on page 158 is updated to add OPENRULE changes. (which also applies to z/OS V2R5).

March 2024 release

- Copy export from the TS7740 Virtualization Engine is updated. For more information, see “Copy export from the TS7740 Virtualization Engine (3957-V06 or 3957-V07)” on page 116 (APAR [OA65964: DFSMSRMM - COPY EXPORT RECLAIM CBR3750I - R0000 DOES NOT MOVE THE STACKED VOLUME TO REQUIRED LIBRARY LOCATION. \(www.ibm.com/support/pages/apar/OA65964\)](http://www.ibm.com/support/pages/apar/OA65964), which also applies to z/OS V2R4 and V2R5).
- Modifications were made to the publication to correct identified inaccuracies. For more information, see the following
 - “Step 9: Assigning DFSMSrmm a RACF user ID” on page 46
 - “Moving the journal using DFSMSrmm utilities” on page 399
 - “Stopping DFSMSrmm” on page 61
 - “Defining retention policies” on page 5
 - “Running DFSMSrmm utilities” on page 7
 - “OPTION command operands” on page 168
 - “Selecting the retention method” on page 4
 - “DD statements for EDGBKUP” on page 382
 - “Defining DFSMSrmm to z/OS” on page 25
 - “How does DFSMSrmm authorization and security work?” on page 16
 - “Allocating data sets for inventory management” on page 333
 - “Managing volumes with special dates” on page 91
 - “Data sets” on page 14
 - “Managing volumes with duplicate volume serial numbers” on page 95
 - “Shelf locations” on page 9
 - “Owner information” on page 15
 - “SYSIN file for VERIFY and MEND processing” on page 406
 - “Volumes” on page 10
 - “Step 10: Defining parmlib member EDGRMMxx” on page 47
 - “Step 8: Updating the procedure library” on page 43
 - “Using the DFSORT ICETOOL utility” on page 16
 - “Updating SMFPRMxx (optional)” on page 27
 - “Updating GRSRNLxx (optional)” on page 27

September 2023 release

- The OPENRULE command syntax figure is updated. For more information, see “OPENRULE command syntax” on page 157.
- New OPENRULE command operands are added. For more information, see “OPENRULE command operands” on page 158.
- The Table 19 on page 215 table is updated.
- Updated EXEC parameters are added for EDGHSKP. For more information, see “Expiration processing trial run” on page 364, “EXEC parameters for EDGHSKP” on page 337, and “JCL for expiration processing” on page 359.

- The description of OVERRIDE is updated. For more information, see [“Converting from using UXTABLE to using the defaults table”](#) on page 485.
- The new keyword NODATE is added to the parmlib member EDGDEFxx DEFAULT command syntax. For more information, see [“DEFAULT command syntax”](#) on page 486.
- The description of NOKEYDATE is updated. For more information, see [“DEFAULT command operands”](#) on page 487.
- The [“Defaults Table tab”](#) on page 503 of Using the DFSMSrmm WUI is updated.

Deleted

The following content was deleted.

September 2023 release

- *What health checks does DFSMSrmm provide?* is removed from [“Who can use DFSMSrmm and how?”](#) on page 20

Chapter 1. Introducing DFSMSrmm

DFSMSrmm is a z/OS feature. In your enterprise, you probably store and manage removable media in several types of media libraries. For example, in addition to your traditional tape library, a room with tapes, shelves, and drives, you might have several automated, virtual, and manual tape libraries. You probably also have both on-site libraries and off-site storage locations, also known as vaults or stores.

With DFSMSrmm, you can manage your removable media as one enterprise-wide library across systems and sysplexes. DFSMSrmm manages your installation's tape volumes and the data sets on those volumes. DFSMSrmm also manages the shelves where volumes reside in all locations except in automated tape libraries.

DFSMSrmm manages all tape media, such as cartridge system tapes and 3420 reels, as well as other removable media you define to it. For example, DFSMSrmm can record the shelf location for optical disks and track their vital record status; it does not manage the objects on optical disks.

This topic discusses basic tape management concepts and introduces terminology used throughout the DFSMSrmm publications and in the DFSMSrmm Interactive System Productivity Facility (ISPF) dialog. This topic also discusses tape management tasks you can perform with DFSMSrmm.

What is a RMMplex?

An RMMplex is one or more z/OS systems each running a DFSMSrmm subsystem sharing a control data set. An RMMplex can optionally include one or more DFSMSrmm subsystems as servers, one or more client subsystems, in addition to standard DFSMSrmm subsystems. The server subsystems and standard subsystems have direct access to and share the DFSMSrmm control data set. The client systems have no direct access to the DFSMSrmm control data set, but share the control data set through the server. All systems that share a control data set in this way are part of the same RMMplex.

What libraries and locations can DFSMSrmm manage?

You decide where to store your removable media based on how often you access the media and for what purpose you retain the media. For example, you might keep volumes that are frequently accessed in an automated tape library. You probably use at least one storage location to retain volumes for disaster recovery and audit purposes. You might also have locations to which you send volumes for further processing. These locations might be other data centers within your company or at your customers and vendors locations.

DFSMSrmm can manage:

- A removable media library, which incorporates all other libraries, such as:
 - System-managed tape libraries; for example, the automated IBM TotalStorage™ Enterprise Automated Tape Library (3494), IBM TotalStorage Virtual Tape Servers (VTS), and manual tape libraries
 - Non-system-managed tape libraries or traditional tape libraries
- Storage locations that are on-site and off-site
- Storage locations defined as home locations

What is in a removable media library?

A *removable media library* contains all the tape and optical volumes that are available for immediate use, including the shelves where they reside. A removable media library usually includes other libraries: *system-managed libraries* such as *automated* or *manual tape libraries*; and *non-system-managed libraries*, containing the volumes, shelves, and drives not in an automated or a manual tape library.

In the removable media library, you store your volumes in shelves, where each volume occupies a single *shelf location*. This shelf location is referred to as a *rack number* in the RMM TSO subcommands and in the DFSMSrmm ISPF dialog. A rack number matches the volume's external label. In DFSMSrmm volume serial numbers are used to identify volumes and to identify the volume label. DFSMSrmm allows you to define a volume using a different serial number than is recorded in the volume label. In this way you can define volumes with duplicate volume serial numbers. DFSMSrmm uses the external volume serial number to assign a rack number when adding a volume, unless you specify otherwise. The format of the volume serial and rack you define must be one to six alphanumeric, national, or special characters.

What is in a system-managed tape library?

A *system-managed tape library* consists of tape volumes and tape devices that are defined in the tape configuration database. The tape configuration database is an integrated catalog facility user catalog marked as a volume catalog (VOLCAT) containing tape volumes and tape library records. A system-managed tape library can be either automated or manual.

You can have several automated tape libraries or manual tape libraries. You use an installation-defined library name to define each automated tape library or manual tape library to the system. DFSMSrmm treats each system-managed tape library as a separate location or destination. See [*z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries*](#) for additional information.

Automated tape libraries

An *automated tape library* is a hardware device that automates the retrieval, storage, and control of tape cartridges (physical or virtual). Refer to [*z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries*](#) for information on the automated, virtual, and manual tape libraries supported through system-managed tape and DFSMSrmm.

DFSMSrmm can automatically replenish the scratch volumes in an automated tape library when the supply of volumes becomes low. See [“Replenishing scratch volumes in a system-managed library” on page 458](#) for information about how DFSMSrmm reclaims volumes that are eligible for returning to scratch.

DFSMSrmm provides a cartridge entry installation exit that you can use to help partition volumes in one or more system-managed tape libraries across multiple systems. Use the installation exit for support for sharing between z/OS and other systems and also for partitioning between z/OS systems and SMS complexes. When using the cartridge entry installation exit, consider these factors.

- DFSMSrmm supports partitioning when you use the DFSMSrmm parmlib options to identify volumes that you want to partition. Volumes can be identified based on volume naming conventions and as individual volumes defined to DFSMSrmm on z/OS.
- You can use the DFSMSrmm partitioning support, defined using the PRTITION commands in parmlib, to partition between multiple z/OS systems, regardless of whether each z/OS partition has a separate DFSMSrmm control data set.
- When you want to share the DFSMSrmm control data set across multiple z/OS partitions, you must use the PRTITION commands in parmlib and ensure that the CBRUXENT exit shipped with DFSMSrmm is used to control the partitioning.

Manual tape libraries

A *manual tape library* is a set of tape drives and the set of system-managed volumes the operator can mount on those drives. The manual tape library provides more flexibility, enabling you to use various tape volumes in a given manual tape library. This support allows volumes to be associated with manual tape libraries so that only those volumes defined for a specific manual tape library can be mounted on drives in that library.

Unlike the automated tape library, the manual tape library does not use the library manager. With the manual tape library, a human operator responds to mount messages that are generated by the host and displayed on a console. This manual tape library implementation completely replaces the IBM 3495-M10 implementation. IBM no longer supports the 3495-M10.

See “Setting up DFSMSrmm for the system-managed tape library” on page 122 for implementation details for these scenarios.

What is in a non-system-managed tape library?

A *non-system-managed tape library* is all the volumes, shelves, and drives not in an automated tape library or manual tape library. You might know this as the traditional tape library in a data center or as an automated environment that is not system-managed. DFSMSrmm provides complete tape management functions for the volumes and shelves in this traditional tape library.

All tape media and drives supported by z/OS are supported in this environment. Use DFSMSrmm to fully manage all types of tapes in a non-system-managed tape library, including 3420 reels, 3480, 3490, and 3590 cartridge system tapes.

You can also use DFSMSrmm to manage volumes in any automated tape library that has special software including an IBM Tape Library Data server that is managed using Basic Tape Library Support (BTLS).

What is in a storage location?

Storage locations are not part of the removable media library because the volumes in storage locations are not generally available for immediate use. A storage location is comprised of shelf locations that you define to DFSMSrmm. A shelf location in a storage location is identified by a *bin number*. Storage locations are typically used to store removable media that are kept for disaster recovery or vital records. DFSMSrmm manages two types of storage locations: installation-defined storage locations and DFSMSrmm built-in storage locations.

You can define an unlimited number of installation-defined storage locations, using any eight-character name for each storage location. Within the installation-defined storage location, you can define the type or shape of the media in the location. You can also define the bin numbers that DFSMSrmm assigns to the shelf locations in the storage location. You can request DFSMSrmm shelf-management when you want DFSMSrmm to assign a specific shelf location to a volume in the location.

You can also use the DFSMSrmm built-in storage locations, which are LOCAL, DISTANT, and REMOTE. Although the names of these locations imply their purpose, they do not mandate their actual location. All volumes can be in the same or separate physical location. For example, an installation could have the LOCAL storage location on-site, as a vault in the computer room, the DISTANT storage location could be a vault in an adjacent building, and the REMOTE storage location could be a secure facility across town or in another state. DFSMSrmm provides shelf-management for storage locations so that storage locations can be managed at the shelf location level.

Although DFSMSrmm automatically shelf-manages built-in storage locations, you must first define the bins you want to use to DFSMSrmm. For bin numbers in built-in storage locations, the numbers are fixed in range, starting at bin number 000001. For installation-defined storage locations, you can use any alphanumeric characters.

How does DFSMSrmm manage these libraries and locations?

DFSMSrmm records the complete inventory of the removable media library and storage locations in the DFSMSrmm control data set, which is a VSAM key-sequenced data set. DFSMSrmm records all changes made to the inventory, such as adding or deleting volumes, and also keeps track of all movement between libraries and storage locations.

DFSMSrmm manages the movement of volumes among all library types and storage locations. This lets you control where a volume, and hence a data set, resides and how long it is retained.

DFSMSrmm helps you manage the movement of your volumes and retention of your data over their full life, from initial use to the time they are retired from service. Among the functions DFSMSrmm performs for you are:

- Automatically initializing and erasing volumes.
- Recording information about volumes and data sets as they are used.

- Expiring volumes based on controls you define.
- Identifying volumes with high error levels that require replacement.

To use all of the DFSMSrmm functions, you specify installation setup options and define retention and movement policies. DFSMSrmm provides you with utilities to implement the policies you define.

Setting up your installation options

The DFSMSrmm parmlib member EDGRMMxx includes many options for setting up DFSMSrmm, such as:

- Defining system options, such as the date format for reports and messages, default retention periods, and notification to volume owners when their volumes are ready to be released.
- Controlling the use of tape volumes.
- Partitioning tape volumes.
- Specifying if a pool has tape profile processing as provided by RACF, a component of the Security Server for z/OS.
- Tailoring mount messages with either the volume's shelf location or the pool identifier.
- Defining security classes for data sets and volumes.
- Defining the DFSMSrmm running mode to determine when DFSMSrmm records volume usage and performs tape validation as described in [“How does DFSMSrmm validate tape mounts?” on page 17](#).
- Defining how DFSMSrmm bypass label processing is performed.
- Defining storage locations to DFSMSrmm.
- Defining media information and media replacement policies.
- Defining a system-wide default retention method for tapes.

Your system programmer or storage administrator defines your DFSMSrmm installation options during implementation. For information on the options that can be set in EDGRMMxx, see [Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143](#).

Selecting the retention method

When data is created you can select an appropriate retention method (seen denoted as RM) for a volume set. The basic choice is either VRSEL retention method or EXPDT retention method. See the “Retention Methods” topic in [z/OS DFSMSrmm Managing and Using Removable Media](#) for considerations when deciding which retention method to use.

A system-wide default retention method can be specified in parmlib. This is applied by default to volumes added to DFSMSrmm (if the volume is added in a status of MASTER or USER) and for each new volume set created during Open Close End-of-volume (OCE). In addition, for each new volume set created during OCE, you can use the Management Class, the installation exit EDG_EXIT100, or the defaults table to select the retention method. The management class has the highest priority, followed by the EDG_EXIT100, the defaults table, and the parmlib RETENTIONMETHOD(EXPDT|VRSEL) option, which has the least priority.

Volumes and data sets retained by the EXPDT retention method are not subject to VRSEL. Only inventory management expiration processing (seen denoted as EXPROC) determines whether the volumes are retained or expired. Volume sets retained by the EXPDT retention method can be retained by VOLUME, SET, or FIRSTFILE. Return to scratch is attempted in a single run of inventory management expiration processing, as if the *scratchimmediate* VRS release option had been used for these volumes. Any movement of EXPDT managed volumes must be done with a manual move.

Volumes and data sets retained by the VRSEL retention method are processed by VRSEL processing, which tries to match data sets and volumes to vital record specifications, and if a match is found, to determine whether the data sets or volumes are to be retained by vital record specifications. When the volume is no longer retained by any vital record specification, it is a subject for expiration processing. With vital record specifications and vital record specification chains, you can also specify movement policies to automate the movement of your volumes through locations.

Defining retention and movement policies for the VRSEL retention method

If you use the VRSEL retention method, you must define retention and movement policies to DFSMSrmm. These policies are known as *vital record specifications*. You use them to specify how long and where you want to keep data sets or volumes. You also use them to define how volumes are to be moved among the libraries DFSMSrmm supports and to define the storage locations for vital records and disaster recovery purposes.

Use vital record specifications to control retention requirements for production data in accordance with the service level agreement and known requirements. Use COUNT(0) to specify that a vital record specification is not to retain a data set.

Use the DFSMSrmm parmlib member EDGRMMxx to set expiration and retention defaults for your installation. When users are allowed to specify expiration and retention period to override vital record specifications, use the parmlib OPTION MAXRETPD operand to set a maximum retention period for your installation. See [Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143](#) for information.

You can define policies, or vital record specifications, for data sets and volumes. You can also define name vital record specifications to provide retention information for data sets and volumes that must be moved through multiple locations before they expire.

Defining home location and target destinations

DFSMSrmm records the starting location for a volume when the volume is initially defined to DFSMSrmm or when volume information is changed. This starting location is known to DFSMSrmm as a *home* location. Home is the location where volumes start from and are returned to when the identified retention and movement actions have been completed. You can use system-managed libraries, storage locations, and SHELF as home locations. A non-system-managed library identified as SHELF can only be used as a home location and not as a target location in a vital record specification. SHELF can be used as a target location when a volume is moved by issuing a command.

You can give any system-managed library or storage location as a target destination for a volume move.

You can also use the DFSMSrmm-reserved location name CURRENT to avoid moving a volume from its current location.

Defining retention policies

Use data set names and data set name masks to define retention policies for data sets. Use job names and job name masks to define retention policies to further qualify the criteria for applying retention and movement policies. For data sets, you can request the different types of retention:

Retention by cycles

You can use retention by cycles for generation data groups (GDGs), pseudo-GDG data set names, or data sets with the same name. For non-GDG data sets, DFSMSrmm considers each occurrence of a data set to be a cycle.

Retention by cycles by days

You can use retention by cycles by days when you want DFSMSrmm to manage all copies of a data set created on a single calendar day as a single cycle.

Retention by number of elapsed days

You can retain a data set specifying a number of days since it was created.

Retention for a number of extra days

You can retain a data set for a number of days beyond the date when the data set is no longer retained by the previous vital record specification in a vital record specification chain.

Retention by days since last referenced

You can retain each copy of a data set produced for a set number of days since the data set was read or written.

Retention while data set is cataloged

You can retain any data set as long as it remains cataloged.

Retention to a specific date

You can set a deletion date for a vital record specification. When that date is reached, the vital record specification is deleted. All data sets and volumes that would match the vital record specification become eligible for release processing, or might match a less specific vital record specification that might specify different retention and movement information.

Retention by expiration date

You can retain the data set on a volume as long as the volume expiration date has not yet been reached.

Retention of open data sets

You can specify a separate policy to apply to all data sets that are currently open.

Retention of data sets closed by abend processing

You can specify a separate policy to apply to all data sets that were open at the time of an application or system abend.

Retention of deleted data sets

You can specify a separate policy to apply to all data sets that were created with a normal disposition of DELETE.

You can also use a *vital record specification management value* (seen denoted as management value or VRS value) instead of a data set name to define retention policies. A vital record specification management value assigns management and retention policies to tape data sets and is defined by your installation. You can define data set vital record specifications for vital record specification management values to provide support for special JCL-specified expiration dates. You can use the installation exit EDG_EXIT100, as well as the defaults table, to assign vital record specification management values.

You can use management class names when you are managing data sets. See [“Managing volumes with special dates” on page 91](#) for information on assigning management class names and vital record specification management values using EDG_EXIT100.

Defining vital record specification chains

You can combine retention types and movements within a vital record specification to manage data sets and volumes by defining vital record specifications chains. A vital record specification chain can be one or more vital record specifications linked together using NEXTVRS and ANDVRS operands. The first vital record specification in the chain is a data set or volume vital record specification. You can link vital record specifications to the first vital record specification to add additional retention types and movement policies for a data set or volume. When combining retention types, you can include the WHILECATALOG operand and the UNTILEXPIRED operand with one other retention type in a single vital record specification. To combine more retention types together so that all must be true concurrently, you link multiple vital record specifications using the ANDVRS operand. To combine different retention types to be implemented consecutively or to move a data set or volume through multiple locations consecutively, you link multiple vital record specifications using the NEXTVRS operand. An exception is that when the EXTRADAYS operand is used as a retention type. EXTRADAYS must be the only retention type at that point in the vital record specification chain. Do not link to or from the vital record specification that specifies the EXTRADAYS operand using the ANDVRS operand.

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information about chaining vital record specifications.

Figure 1 on [page 6](#) is an example of creating a vital record specification chain. The data set name vital record specification moves a data set from the installation media library to the REMOTE storage location. The name vital record specification, MOVE2, moves the volume from the REMOTE storage location to the DISTANT storage location.

```
RMM ADDVRS DSNAME(STEGO.SAURUS) COUNT(1855) DAYS LOCATION(REMOTE) -  
    STORENUMBER(30) NEXTVRS(MOVE2)  
RMM ADDVRS NAME(MOVE2) LOCATION(DISTANT) STORENUMBER(1825)
```

Figure 1. Defining a vital record specification chain

You can define retention and movement policies for specific volumes and volumes that match a generic volume serial number. If you use a specific volume serial number, DFSMSrmm retains the volume that matches that volume serial number. If you use a generic volume serial number, DFSMSrmm retains a number of volumes matching the generic volume serial number based on the number you specify.

When multiple vital record specifications retain a volume, and each vital record specification contains a different destination, DFSMSrmm decides where to move the volume based on priority number. Lower priority numbers have higher priorities. Table 3 on page 7 shows the movement priority DFSMSrmm uses if you do not assign movement priority numbers. For example, if you define both REMOTE and DISTANT as the destination for the volume, DFSMSrmm selects the REMOTE storage location as the destination. The selection is based on the REMOTE priority number of 100 which has a higher priority selection value than the DISTANT priority number of 200.

Table 3. DFSMSrmm movement priority

Priority Number	Location Name or Location Type
100	REMOTE DFSMSrmm built-in storage location name
200	DISTANT DFSMSrmm built-in storage location name
300	LOCAL DFSMSrmm built-in storage location
2000	Installation defined storage locations
4800	AUTO automated tape libraries
4900	MANUAL manual tape libraries
5000	SHELF location name

Running DFSMSrmm utilities

DFSMSrmm provides utilities to manage your inventory, create reports, maintain the DFSMSrmm control data set (CDS), and erase and initialize volumes. DFSMSrmm uses IKJTSEV, if necessary, to establish a TSO environment for each of its batch utilities. See *z/OS TSO/E Programming Services* for additional information about IKJTSEV.

All data sets used by DFSMSrmm can be eligible for allocation in the extended addressing space of an EAV. This includes the DFSMSrmm CDS, journal, and any dynamically allocated temporary files.

Temporary sort files created by DFSMSrmm for use by DFSORT are always large format EAS-eligible data sets when inventory management or EDGUTIL are used. EAS-eligible data sets can be placed in an extended addressing space (EAS) on an extended address volume (EAV). See *z/OS DFSMS Using the New Functions* for more information on EAS and EAV. DFSMSrmm specifies EATTR=OPT on the dynamic allocation that creates the temporary data set.

All DFSMSrmm utilities and programs support NON_VSAM_XTIOT=YES options in the DEVSUPxx parmlib member.

For the correct and full functioning of DFSMSrmm, DFSORT must be installed. If you have another sort product instead of DFSORT, you need equivalent sort product capability to support large format sequential input and output data sets.

If you do not have DFSORT installed and either EDGUTIL or inventory management fails with ABEND 213-14, pre-allocate the temporary data sets as non-large format data sets:

- For EDGUTIL, the DD name is VCINOUT and must be included in the EDGUTIL JCL.
- For EDGHSKP (inventory management), the DD names are SRTINOUT, DATUPD, VOLCON, VOLSET, and CONSET and must be included in the started procedure JCL, usually called DFRMM.
- Example JCL is:

```
//SRTINOUT DD DISP=(,DELETE),
//          SPACE=(472,(pp,ss)),AVGREC=U,
```

```
//      BLKSIZE=0, LRECL=472,  
//      RECFM=FB, UNIT=(SYSALLDA, v) , DSORG=PS
```

- The LRECL in this example is approximate and will be overridden by DFSMSrmm at run time.
- pp - primary space. This is usually half the number of the data set records (TCDB volume records for EDGUTIL). See messages EDG2223E and EDG6840E for an explanation.
- ss - same as pp.
- v - volume count. Usually 1 is enough.

Use the EDGHSKP utility to run inventory management activities.

- For volumes managed by the VRSEL retention method, vital record processing determines which data sets to retain and what volume moves are required based on the retention and movement policies you define to DFSMSrmm.
- Expiration processing (EXPROC) identifies data sets ready to expire and volumes ready to be released and returned to scratch for both VRSEL and EXPDT retention methods.
- Catalog Synchronization processing (CATSYNCH) identifies and attempts to fix any inconsistencies between the CDS and user catalogs.
- For CATSYNCH, VRSEL and EXPROC, DFSMSrmm supports trial run (HSKP parameter: VERIFY) on top of normal processing (referred to as production run). This is helpful when you are defining your initial set of policies and when you need to make changes as you gain more experience with DFSMSrmm.
 - For VRSEL, VERIFY vital record processing does not change data set or volume information in the DFSMSrmm control data set. Use VERIFY processing to analyze the effect that your movement and retention policies will have. Based on your analysis, you can then determine if you need to change vital record specifications before performing production run vital record processing.
 - For EXPROC, VERIFY expiration processing does not expire data sets or volumes in the DFSMSrmm control data set. Use VERIFY to analyze the effect that your retention and expiration policies will have prior to production expiration processing. This allows you to make adjustments to your expiration specifications before performing any job that may be prematurely expire data.
- Storage location management processing assigns shelf locations to volumes being moved to storage locations.
- Back up the CDS and the journal, reset the journal data set when the CDS and journal are backed up, and create an extract data set.

Use the EDGAUD and EDGRPTD utilities and the EDGRRPTE exec to get information about your removable media library and storage locations. You can also get security trail information about volumes and data sets defined to DFSMSrmm and audit trail information about volumes, shelf assignments, and user activity.

Use the EDGUTIL utility to create, update, mend, and verify the control data set.

Use the EDGBKUP utility to back up and recover the control data set and journal.

Use DFSMSrmm backup utilities instead of other backup utilities, such as access method services EXPORT, because DFSMSrmm provides the necessary serialization and forward recovery functions using the journal data sets. DFSMSrmm backup utilities check whether the control data set is in use and tell the DFSMSrmm subsystem that backup or recovery is in process.

Use the EDGINERS utility to erase and initialize tape volumes either automatically or manually and to scan tape labels manually. You can use EDGINERS instead of the DFSMSdfp utility IEHINITT. See [“Replacing IEHINITT with EDGINERS”](#) on page 424 for details on the differences between the utilities and for information on controlling the use of IEHINITT.

You can schedule these utilities to run at the time and frequency that are best for your installation. Use a scheduling system such as IBM Tivoli® Workload Scheduler for z/OS for this purpose, or see Chapter 13, [Performing inventory management](#) section [Scheduling DFSMSrmm utilities](#) for more information.

What resources does DFSMSrmm manage?

This topic describes how DFSMSrmm helps you manage shelf locations, volumes, data sets, information about volume owners, and software products in all libraries and storage locations.

Shelf locations

DFSMSrmm automatically:

- Assigns a rack number to a volume that matches the volume serial number you specify except when:
 - You already specified a rack number or pool ID when you first defined a volume to DFSMSrmm.
 - There is no rack number that matches the volume.
 - The volume is a logical volume.
- Assigns bin numbers for the shelf locations in shelf-managed storage locations
- Provides a specific rack number or pool information in the fetch/mount message to help the operator respond to mount requests

You can specify that DFSMSrmm make bins available for reuse when a volume move has started or make bins available only when a volume move has been confirmed. You can request that DFSMSrmm move volumes by specific location and request that DFSMSrmm move the volumes to bins in sequential order, beginning with the lowest volume serial number and the lowest bin number. You can also request that DFSMSrmm reassign the volumes to bins during storage location management processing.

DFSMSrmm also helps you delete rack or bin numbers for obsolete, empty shelf locations. You can create lists containing information about the shelf locations in your removable media library and in storage locations.

You can have more control over where volumes reside and how they are managed by defining your removable media into pools. A pool is a group of shelf locations defined by a common prefix or a group of volumes that require DFSMSrmm pool management functions. For example, based on your pool definitions, DFSMSrmm can ensure that RACF® profiles exist for all volumes in a pool. You could also request that all volumes in a pool that are protected with an expiration date are processed automatically. See [“Organizing the library by pools” on page 75](#) for information about defining your pools in parmlib member EDGRMMxx. If you do not define any pools, DFSMSrmm considers all volumes part of one default pool.

In DFSMSrmm, there are two categories of pools:

Rack pool

A *rack pool* is shelf space that can be assigned to hold any volumes. Although you can add scratch volumes to these pools, you cannot normally use these volumes to satisfy non-specific mount requests.

Each rack pool can contain private volumes and scratch volumes:

- Private volumes
 - Are not generally available for use because the data on them is not yet expired
 - Can have a master status or user status

A master status indicates that the volume contains data that can only be overwritten based on criteria set by the EDGRMMxx MASTEROVERWRITE operand.

A user status indicates that the volume contains data that can be overwritten even when a data set name does not match.

- Scratch volumes
 - Are available for use because they are either unused volumes or they contain expired data
 - Are used to satisfy scratch requests.

With scratch volumes in rack pools, you can provide a user or group-based pool of volumes that is selected using the RMM GETVOLUME subcommand or with the EDG_EXIT100 installation exit. Scratch volumes in a rack pool cannot be used where DFSMSrmm system-based pooling is in use.

Scratch pool

A *scratch pool* is shelf space assigned to hold volumes for use with the DFSMSrmm system-based scratch pooling or exit-based pooling. The volumes assigned to this shelf space can be used to satisfy scratch requests as long as the volumes are in scratch status. Once the volume has been written to, it becomes a volume with master status, until the volume is returned to scratch status.

Pools within system-managed libraries are based on the external volume serial number, which is also used as the rack number. Most volumes in a system-managed library either belong to a general scratch pool or they can be separated into multiple scratch pools, one for each media type. For example, volumes can be divided into one pool for cartridge system tape and one pool for enhanced capacity cartridge system tapes. If volumes from other data centers are entered into a system-managed library for processing, a rack pool can be defined to allow you to take advantage of DFSMSrmm pool management functions. These pools are not directly used for scratch volume selection. Rather, media type and category are used to determine which volumes to choose from. The tape library chooses from its own pooling of those volumes and ultimately passes reference of the selected volume to DFSMSrmm.

Pools in non-system-managed libraries are logical sections of shelf space. The shelf space is divided based on rack number, logical groups of volumes based on volume prefix, or logical groups of volumes with the same storage group name. You can control shelf location assignment by specifying a pool ID when you define a volume to DFSMSrmm. DFSMSrmm automatically assigns the volume to the next available rack number in that pool. If you do not specify a shelf location or pool ID for a volume, DFSMSrmm attempts to assign the rack number that matches the volume serial number.

Volumes

DFSMSrmm provides support for these volumes:

- DFSMSrmm allows you to use all types of physical volumes, tracking their use and the data they contain, and providing vaulting and retention services according to policies assigned to the data sets on the volumes. DFSMSrmm also allows you to use duplicate volumes. These are volumes that are defined to DFSMSrmm with a unique external volume serial number and a VOL1 label that might duplicate another volume but that does not match its own external volume serial number.
- Logical volumes reside in a VTS or on exported stacked volumes. Applications can access the data on these volumes only when they reside in a VTS. The data in a VTS is accessed in its tape volume cache or after the data has been copied to a physical volume through the use of special utilities. DFSMSrmm tracks the use of logical volumes and the data that resides on the volumes. Retention services are provided according to policies assigned to the data sets on the volumes.

DFSMSrmm supports exporting of logical volumes on stacked volumes and provides vaulting services for these stacked volumes based on the policies assigned to the data sets on the contained logical volumes. Logical volumes can be removed from a VTS using export processing described in the [“Using DFSMSrmm with the IBM totalstorage virtual tape server” on page 108](#).

DFSMSrmm supports export of logical volume copies and optionally provides vaulting services for the copy exported stacked volumes based on volume VRSeS, and not the data on the contained logical volume copies. Logical volume copies can be removed from a VTS using export processing described in the [“Using DFSMSrmm with the IBM totalstorage virtual tape server” on page 108](#).

- Stacked volumes have a one-to-one association with physical tape media and are used in a VTS to store logical volumes. Stacked volumes are not used by z/OS applications but by the VTS and its associated utilities. Stacked volumes can be removed from a VTS to allow transportation of logical volumes or their copies to a vault or to another VTS. DFSMSrmm can be used to track stacked volumes and automatically records the logical volumes contained on stacked volumes during export processing when stacked volume support is enabled. DFSMSrmm provides support for importing logical volumes on stacked volumes either into the VTS from which it was exported or into a different VTS.

DFSMSrmm automatically validates volumes to ensure that only valid scratch volumes are mounted for non-specific mount requests and that the right volume is mounted for a specific mount request. This eliminates unintentionally overwriting a valid master volume or a volume retained for disaster recovery or vital record management.

DFSMSrmm offers two retention methods, EXPDT and VRSEL, for retention of your volumes and data sets on your volumes. For each volume set, you can specify whether the volumes and data sets in that volume set should be managed by the expiration date or by VRS policies.

The EXPDT retention method is based on the expiration date and avoids VRSEL processing. As a result, the retention information can be known when a tape data set is created. You can also manually override an expiration date to release a volume before the original expiration date is reached. The movement of the volumes must be managed manually.

The VRSEL retention method can retain a volume beyond its expiration date by using a vital record specification to define a retention policy. The VRSEL retention method can also be used for setting a movement policy. Volumes and data sets managed by this retention method are subject to frequent VRSEL processing. In every run of VRSEL processing, the matching of your data sets and volumes to the vital record policies is done again, so the retention and movement management can change from one inventory management run to another.

See *z/OS DFSMSrmm Managing and Using Removable Media* for more information about using the EXPDT and VRSEL retention methods.

When a data set on a volume is opened and closed, DFSMSrmm automatically performs these functions:

- Sets the retention method for the volume if we are writing the first data set on a new volume chain. The RETENTIONMETHOD for volumes can be taken from the following sources, listed in priority order, with the highest priority first:
 - “Retention Method” in the Management Class
 - PL100_RETENTIONMETHOD, returned by the EDG_EXIT100 exit routine
 - RETENTIONMETHOD in the defaults table in EDGDEFxx
 - The default EDGRMMxx parmlib OPTION RM
- For the EXPDT retention method, the RETAINBY value is also set. The RETAINBY value for volumes can be taken from the following sources, listed in priority order, with the highest priority first:
 - “Volume Set Management Level” in the Management Class
 - RETAINBY in the defaults table in EDGDEFxx
 - The default EDGRMMxx parmlib OPTION RM(EXPDT(RETAINBY(value)))
- Changes the volume status from scratch to master for non-specific mount requests at open time.
- Sets the expiration date and time for the volume and ensures that the maximum expiration date is not exceeded at open time.
- For the EXPDT retention method, LASTREF and WHILECATALOG are set for the data set. These can be taken from the following sources, listed in priority order, with the highest priority first:
 - The Management Class
 - The defaults table in EDGDEFxx
 - The default options specified in the EDGRMMxx parmlib options
- Records information about data sets on the volume (the data set name is recorded at open time; all other information is recorded at close time).
- Counts the number of times a volume is used since the volume was last in scratch status at open time.
- Counts the number of temporary and permanent errors encountered at close time only.
- Sets a security classification based on the data sets that reside on the volume at open time.
- Prevents reading of data on a volume in scratch status when DFSMSrmm is running in protect mode at open time.

- When option TAPEAUTHDSN=YES in parmlib member DEVSUPxx in SYS1.PARMLIB is in use, the RACF 'erase on scratch' setting sets the ERASE release action for the volume. Secure erasure of this sort may have performance implications when dealing with large volumes.

DFSMSrmm automatically controls the movement of volumes within the removable media library and the built-in or installation-defined storage locations, based on criteria you specify in vital record specifications.

When volumes no longer reside in the removable media library or storage locations, you can define loan locations where these volumes can be found. DFSMSrmm does not, however, manage the movement of volumes residing in loan locations.

DFSMSrmm automatically releases volumes that have reached their expiration date and time. Any non-scratch volume defined to DFSMSrmm has an expiration date and time indicating when the volume is to be considered for release. The volume expiration date can be:

- An expiration date or retention period, given in the JCL by the user when writing a data set to the volume.
- The date specified by the user when a scratch volume is requested using the RMM GETVOLUME subcommand or when information about the volume is manually added or changed.
- The 'Expiry after Date/Days', given by a management class applied to the data set (if enabled by the MCATTR option in parmlib member EDGRMMxx), when writing a data set to a volume.
- If the data set is on a volume managed by the EXPDT retention method, and the LASTREF attribute is set, then the expiration date and time will be dynamically updated based on the last write or read date of the data set.
- A default retention period for data sets set by your installation with the RETPD option in parmlib member EDGRMMxx in SYS1.PARMLIB.
- The expiration date that is set using the DFSMSrmm EDG_EXIT100 installation exit. If the exit sets a zero date, the installation default retention period is used.

A cataloged data set may prevent the volume from expiring on its expiration date if it has WHILECATALOG(ON). In that case, (KEPTBYCATLG) is appended to the expiration date in the LISTDATASET and LISTVOLUME display. If (ORUNCATLG) is appended to the expiration date, the date may decrease after a dataset with WHILECATALOG(UNTILEXPIRED) is uncatalogued (more information on this in [Using the parmlib member EDGDEFxx on page 492](#)).

Your installation can also set a maximum retention period in parmlib member EDGRMMxx. An expiration date or retention period set for a volume cannot exceed this value. If the MAXRETPD is used to overwrite a retention period, it does so generally silently, and does not cancel a submitted job. For more information, see OPTION MAXRETPD in [Using the parmlib member EDGRMMxx on page 177](#).

When a volume is eligible for release, DFSMSrmm can perform *release actions* for the volume based on information you provide. Examples of release actions include:

- Returning a volume to scratch status
- Initializing a volume
- Erasing a volume
- Notifying a volume's owner of the volume's release
- Returning a volume to its owner and deleting the volume record in the DFSMSrmm control data set

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information about setting release actions and how processing is performed.

DFSMSrmm tracks the number of I/O errors recorded for a volume. During expiration processing, DFSMSrmm automatically checks the volumes against volume replacement policies. DFSMSrmm also tracks SARS MIM alerts and uses these to identify volumes that should be replaced.

You can get additional information to help you analyze volumes with temporary errors from these tools and services:

- DFSMSrmm records volume use and temporary I/O errors. You can obtain this information through the extract data provided by EDGHSKP.
- The Environmental Record Editing and Printing Program (EREP) uses LOGREC records to identify tapes with unacceptably high error conditions.
- Service Director™ identifies volumes with high error levels. See your IBM service representative for information about using this tool.
- System management facilities (SMF) records, produced by DFSMSdfp, track information on volume use, such as how much data is written to a volume. You can use this information to analyze volumes with errors.

You can then use this information to identify volumes you want replaced or managed by DFSMSrmm. Use the RMM CHANGEVOLUME RELEASEACTION(REPLACE) subcommand or the dialog change volume function when you want DFSMSrmm to manage the replacement.

DFSMSrmm provides support for using the system TAPEAUTHDSN and TAPEAUTHF1 options (specified in parmlib member DEVSUPxx in SYS1.PARMLIB) and the RACF standard tape volume security protection with any combination of RACF TAPEVOL and TAPEDSN options. See *z/OS MVS Initialization and Tuning Reference* for information about the TAPEAUTHxxx keywords to enable tape authorization checks. For more information on TAPEVOL and TAPEDSN options, see [Authorizing DFSMSrmm users and ensuring security](#) on page 234, specifically Table 29.

DFSMSrmm allows you to use volumes with duplicate volume serial numbers as well as volumes undefined to DFSMSrmm. You can request DFSMSrmm to ignore a volume so it can be used, even if another volume with the same volume serial number is already defined in the DFSMSrmm control data set. If DFSMSrmm ignores a volume, it does not track volume use. If the volume is not defined to DFSMSrmm, DFSMSrmm cannot provide any management functions for the volume. See [Ignoring duplicate or undefined volume serial numbers](#) on page 275 for more information about using EDG_EXIT100 or OPENRULE within EDGRMM to ignore duplicate volume serial numbers.

DFSMSrmm tape label support

DFSMSrmm supports these tape label types:

- IBM standard labels (SL)
- ISO/ANSI labels (AL)
- Both IBM standard and user header or trailer labels (SUL)
- Both ISO/ANSI and user header or trailer labels (AUL)
- No labels (NL)

You can explicitly use AL, SL, and NL label types when you define volumes to DFSMSrmm. Although a tape is defined to DFSMSrmm as SL or AL, a user can still process it as SUL or AUL. A user could ask for a non-specific volume by specifying LABEL=(,SUL) and create an SL volume or SUL volume without affecting DFSMSrmm. DFSMSrmm records the value specified in the JCL.

DFSMSrmm provides support for ISO/ANSI Version 3 and ISO/ANSI Version 4 labels. You can provide the current ISO/ANSI label version of a volume that you define to DFSMSrmm. Then you can use the DFSMSrmm EDGINERS utility to initialize the tape with either ISO/ANSI Version 3 or ISO/ANSI Version 4 labels.

DFSMSrmm supports no label (NL) for private volumes and for scratch tapes in a system-managed automated tape library and private volumes in a non-system-managed tape library. If you use the IFG019VM volume mount installation exit, DFSMSrmm supports no label (NL) for scratch tapes in a non-system-managed tape library. All other scratch tapes must be standard label tapes. DFSMSrmm allows the creation of no label tapes from standard label scratch volumes by changing the volume to master status and setting the initialize release action so that the tape is relabeled as a standard label tape when returned to scratch. DFSMSrmm also overrides the logical volume serial number generated by OPEN for NL output to scratch tapes, so that the correct volume serial number is used for cataloging data sets. Nonstandard labels (NSL) can be used but only if the volume is not defined to DFSMSrmm. [“How does](#)

DFSMSrmm validate tape mounts?” on page 17 describes how DFSMSrmm validates volume usage and does not allow the processing of certain label types.

DFSMSrmm also supports the use of bypass label processing (BLP) for:

- Volumes that are either not defined to DFSMSrmm or volumes that DFSMSrmm should ignore.
- Non-scratch (user and master status) volumes that are used for input processing, and only those user status volumes that are used for output processing. You can select this system option in the EDGRMMxx parmlib member using the OPTION BLP(RMM) command.
- Non-scratch (user and master status) volumes that are used for input processing, or user, master and scratch tapes that are used for output processing. You can select this option in the EDGRMMxx parmlib member using the OPTION BLP(NORMM) command.

When BLP is used with scratch tapes, DFSMSrmm changes the volume to master status. DFSMSrmm also sets the initialize volume release action to ensure that the volume has a valid standard label before it returns to scratch status. When a volume is written with BLP, DFSMSrmm can no longer perform the 44-character data set name check for the files on the volume.

For BLP output to a non-specific volume, DFSMSrmm does not check the file sequence number. For example, LABEL=(2,BLP) can be used. For non-BLP requests, DFSMSrmm restricts the use to LABEL=(1,label_type).

The data set information for files processed with BLP is only updated for output to first file on a volume. All other types of BLP requests change only the volume information such as the date last read and the date last written. Also for BLP output requests, the data set information is only updated in the DFSMSrmm control data set when the data set is closed.

Data sets

DFSMSrmm automatically records information about a data set when the data set is opened. DFSMSrmm can automatically record information when you request data set recording and one of these conditions is true:

- The data set is the first file on the volume.
- You already processed all files preceding it on the volume.
- You have previously defined the data sets on the volume to DFSMSrmm.

Data class, management class, storage class, and storage group are included in the information that DFSMSrmm records for system-managed data sets.

When the data set is created on a tape volume managed by the EXPDT retention method, the data set VRSELEXCLUDE attribute is automatically set. If the data set is created on a tape volume managed by the VRSEL retention method, the VRSELEXCLUDE attribute can be set by the Management Class, the defaults table or the EDG_EXIT100 installation exit, and can be set or reset later by command. Volumes with the VRSELEXCLUDE attribute are never processed by an EDGHSKP inventory management run with the VRSEL parameter specified. VRSELEXCLUDE volumes are only affected by EXPROC inventory management processing.

You can manually add information about a data set if the volume on which the data set resides is already defined to DFSMSrmm and the volume has not been already processed as a result of open processing and close processing. There are some limitations on the information that you can change if it was originally recorded by DFSMSrmm during open, close, and end-of-volume processing.

DFSMSrmm supports generic data set names as filter criteria for searching the control data set which makes it easier to create lists of resources.

Year 2000 support

DFSMSrmm provides support for dates beyond the 20th century by ensuring that DFSMSrmm stores and displays all dates using a 4-digit year. DFSMSrmm also allows you to specify dates using the 4-digit year. DFSMSrmm provides the same support for dates as DFSMSdftp.

Software products

You can also define software products to DFSMSrmm and associate volumes with the products.

Owner information

DFSMSrmm can help you keep track of volume owners. It provides functions to electronically notify owners when their volumes are being considered for release. DFSMSrmm automatically records owner IDs as volumes are used. If you want DFSMSrmm to use owner information to automatically notify owners, you must manually define the owner's electronic address to DFSMSrmm. Additionally, the notify action must have been requested for the volume.

DFSMSrmm ensures that there is an owner ID for each volume. If a job is started and does not have a known RACF user ID, DFSMSrmm uses the job name as the owner ID.

DFSMSrmm also provides for notification to product owners when new volumes are added for a product.

DFSMSrmm keeps track of volume ownership. To delete an owner ID who still owns volumes, DFSMSrmm optionally allows you to transfer ownership of those volumes before deleting the owner record. If no other ID exists to be able to take ownership, DFSMSrmm will attempt to create an ID and set ownership to this new ID.

How does DFSMSrmm help you create reports?

You can obtain information and create reports using these methods:

- DFSMSrmm Report Generator
- DFSMSrmm ISPF dialog or RMM TSO commands
- EDGAUD and EDGRPTD report utilities
- DFSMSrmm EDGRRPTE exec shipped in SAMPLIB
- The DFSORT ICETOOL utility
- The DFSMSrmm application programming interface

Using DFSMSrmm report generator

The DFSMSrmm Report Generator is an ISPF application that you use to create reports to show the status of the resources that DFSMSrmm manages for you. These resources include volumes, data sets, racks, owners, and the retention and movement policies that are established for your installation. Refer to [z/OS DFSMSrmm Reporting](#) for detailed information.

Using DFSMSrmm ISPF dialog and RMM TSO subcommands

You can search on-line using either the DFSMSrmm ISPF dialog or RMM TSO subcommands to create lists of resources and display information recorded in the DFSMSrmm control data set. Here are some examples:

- Operators can create lists of scratch volumes to be pulled for use.
- Tape librarians and system programmers can create lists of software products and the volumes on which they reside.
- General users can create lists of volumes they own, such as the example in [Figure 2 on page 15](#):

Volume	Owner	Rack	Assigned date	Expiration date	Location	Dsets	St	Act	Dest.
VOL600	AMYW01	RAC500	06/11/2000	11/11/2000	SHELF	0	UR	SI	
VOL601	AMYW01	RAC501	06/11/2000	11/11/2000	SHELF	0	UR	SI	
VOL603	AMYW01	RAC502	06/11/2000	11/11/2000	SHELF	0	UR	SI	
EDG3011I 3 ENTRIES LISTED									

Figure 2. Example of a list of volumes owned by a single user

With DFSMSrmm, you can use the RMM TSO SEARCH subcommands with the CLIST operand to create a data set of executable subcommands. For example, you can create subcommands to confirm volume movement for volumes identified during a SEARCHVOLUME request.

Using the EDGAUD and EDGRPTD report utilities

You can create several types of reports using two DFSMSrmm report utilities. Use EDGRPTD to create movement and inventory reports and EDGAUD to create security and audit reports. EDGRPTD uses the DFSMSrmm extract data set as input. EDGAUD uses SMF records as input.

You can use the reports for these purposes:

- Identifying volumes that should be moved between the removable media library and storage locations.
- Determining your volume inventory in the removable media library and storage locations.
- Identifying volumes that are in transit or that should be marked as moved.
- Identifying all accesses to volumes and changes to information recorded in the DFSMSrmm control data set.
- Separating volumes that are waiting to return to scratch from those that are private or have other release actions pending.
- Producing new scratch volume reports or scratch volume inventory reports.

Using the DFSMSrmm EDGRRPTE exec

Use the DFSMSrmm-supplied EDGJRPT JCL to run the EDGRRPTE exec to produce reports using the extract data set as input. See [z/OS DFSMSrmm Reporting](#) for more information.

Using the DFSORT ICETOOL utility

You can use DFSORT or a similar program to generate a formatted report using the information in the extended extract data set created by the EDGHSKP utility. For example, you could produce an extract data set listing all volumes to be used on VM with information about volume owners. Then use the DFSORT ICETOOL utility to sort the information by volume and produce a report, complete with title and header information.

Using the DFSMSrmm application programming interface

You can use the DFSMSrmm application programming interface to obtain information about resources defined to DFSMSrmm. See [z/OS DFSMSrmm Application Programming Interface](#) for information about how to use the DFSMSrmm application programming interface.

How does DFSMSrmm authorization and security work?

You can choose the authorization levels of users for all DFSMSrmm functions. DFSMSrmm uses z/OS System Authorization Facility (SAF) for its authorization checking. You define DFSMSrmm resources to RACF (or other similar software security package) for use during authorization checking. DFSMSrmm can create volume profiles, change them, and delete them on registration, expiration, or release of volumes. DFSMSrmm provides an access list you can use to set the access list in RACF. You can use the DFSMSrmm access list for authorization checking on non-RACF systems. Use the RMM LISTVOLUME subcommand or the DFSMSrmm ISPF dialog to display the DFSMSrmm access list. You can also view the access list in the volume records in the report extract data set.

DFSMSrmm provides automatic security classification through installation-specified criteria based on data set names. DFSMSrmm security includes these elements:

- An audit trail of access and change of status through SMF. This audit trail produces information about RACF user IDs, groups, and job names.
- Required operator confirmation prior to using certain volumes.

- Erasure of data when a volume is released prior to the volume returning to scratch status.

DFSMSrmm provides these ways of optionally keeping an audit trail for volumes defined to it:

- Control data set information
- SMF audit records
- RACF audit information

See Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215 for additional information about DFSMSrmm authorization and security.

What tape usage does DFSMSrmm support?

DFSMSrmm supports these tape usage:

- Using BLP to read any private volume.
- Using BLP to write to any volume.
- Using nonstandard label volumes that are not defined to DFSMSrmm.
- Using both AL and SL scratch tapes.
- Using no label (NL) processing for any private volume.
- Performing no label (NL) output to any system-managed scratch volume or standard label scratch volume.
- Using no label (NL) for private volumes and for scratch tapes in a system-managed automated tape library and private volumes in a non-system-managed tape library.
- Using no label (NL) also for scratch tapes in a non-system-managed tape library if you use the IFG019VM volume mount installation exit.
- Overwriting a volume that is in master status. For volumes in master status, DFSMSrmm allows the overwriting of data based on criteria set by the EDGRMMxx MASTEROVERWRITE operand.
- Overwriting a volume that is in user status no matter what the data set name is.
- Using volumes not defined to DFSMSrmm.
- Using duplicate volumes when running in OPMODE(Protect).
- Automatically labeling scratch tapes in an automated tape library.
- Reusing 36-track recorded scratch tapes on 18-track drives and 256-track recorded scratch tapes on 128-track drives.
- Using multifile and multivolume data sets.
- Recording and validating only the first file on the volume.
- Creating checkpoint data sets on scratch volumes in an automated system-managed tape library.
- Creating non-checkpoint data sets on scratch volumes in an automated system-managed library, where the scratch volumes were previously a checkpoint secure volume.
- Exploiting high-speed cartridge tape positioning when an application does not exploit it.
- Automatic recovery from multivolume tape label anomalies.

How does DFSMSrmm validate tape mounts?

DFSMSrmm performs validation for all data sets on a volume that have been recorded by DFSMSrmm when a data set is opened. DFSMSrmm validates tape volumes as follows:

- For specific tape requests, DFSMSrmm checks that the correct private volume is mounted.
- For non-specific tape requests, DFSMSrmm checks that a scratch volume is mounted and that the volume is from an acceptable rack, scratch pool, or storage group.

- For volumes where DFSMSrmm has recorded the first file creation at open time, DFSMSrmm checks that the last 17 characters of the data set name from the tape volume HDR1 label match the first file data set name known to DFSMSrmm.
- For specific requests to overwrite data on a master volume, DFSMSrmm allows the overwriting of data based on criteria set by the EDGRMMxx OPTION MASTEROVERWRITE operand. DFSMSrmm checks that the data set name used matches the data set name that DFSMSrmm has recorded. For generation data group data (GDG) sets, DFSMSrmm removes the GDG suffix before checking the data set name.
- At open time for the file being referenced, DFSMSrmm checks that the data set name used matches the one DFSMSrmm has recorded. If only the first file on a volume is being recorded, DFSMSrmm only validates the first file on the volume.

If DFSMSShsm is reading a tape volume, only the last 17 characters of the data set name need to match the data set information in the tape header label.

- For scratch tapes mounted in a fully functional automated tape library, that have incorrect or missing VOL1 labels, DFSMSrmm checks the external and internal volume serial numbers. DFSMSrmm ensures that both the external volume serial number and internal volume serial number, if one exists, are either defined as scratch to DFSMSrmm or are not defined to DFSMSrmm. If validation is successful, DFSMSrmm uses the external volume serial number (obtained by the vision system) to request the tape VOL1 label is created by OPEN processing.
- DFSMSrmm ensures that the mounted volume has the correct WWID. DFSMSrmm obtains the WWID for a WORM tape from the tape drive when the volume is mounted and used or when you define the WORM tape to DFSMSrmm using the RMM TSO subcommands.
- When DFSMSrmm has a Write Mount Count for a WORM media, DFSMSrmm ensures that the mounted volume write mount count matches the value recorded in the DFSMSrmm control data set.

DFSMSrmm performs tape mount validation based on the DFSMSrmm running mode set in the parmlib member EDGRMMxx with the OPTION command and OPMODE operand as shown in [Table 4 on page 18](#).

Table 4. How the DFSMSrmm running mode affects tape mount validation

DFSMSrmm Running Mode	Tape Validation
Manual	DFSMSrmm does not validate volume usage.
Record-only	DFSMSrmm does not validate volume usage.
Warning	DFSMSrmm validates tape volume usage and issues warnings if errors are encountered. DFSMSrmm does not reject volume usage.
Protect	DFSMSrmm validates tape volume usage and rejects volume usage under certain conditions.

Use warning mode as you perform testing during conversion from another tape management system. When you are running DFSMSrmm in warning mode, DFSMSrmm validates your tape volumes but does not prevent their use. See [“Defining system options: OPTION” on page 161](#) for information about setting options in the DFSMSrmm parmlib member.

Why does DFSMSrmm reject tape volumes?

In addition to validation, there are a number of reasons why a tape can be rejected when you are running DFSMSrmm in protect mode. Volumes can be rejected based on installation-controlled parmlib options and volume definitions or based on DFSMSrmm and system rules.

You can use PRTITION and OPENRULE commands to control library partitioning and the use of volumes by applications. See [“Implementing PRTITION and OPENRULE parmlib commands” on page 200](#) for additional information. If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. See [“Converting REJECT commands to PRTITION and OPENRULE commands” on page 204](#) for information about converting from the use of REJECT commands.

Rejects caused by installation controls

With DFSMSrmm you can define conditions such as where volumes should reside, ranges of shelf locations that should not be used, and volumes that can only be used on certain systems. If you are running DFSMSrmm in protect mode, the volume is rejected under these conditions:

- The volume mounted for a scratch request has not been defined to DFSMSrmm.
- The volume mounted for a scratch request is not in a scratch pool associated with this system or does not match installation-defined requirements.
- The volume mounted for a scratch request is from a scratch pool associated with another system.
- The volume is not to be used on z/OS systems.
- A user asks for the use of a volume to be ignored, but is not authorized to do so for that volume.
- A volume in an automated tape library is not defined to DFSMSrmm and cannot be defined to DFSMSrmm for some reason.
- You have defined OPENRULE commands in parmlib member EDGRMMxx, and:
 - An OPENRULE command with the unqualified REJECT action is defined for a set of volumes, preventing a volume from being used on a particular system.
 - An OPENRULE command with the REJECT action qualified by SYSID or CATLG is defined for a set of volumes, preventing a volume from being used when the qualified condition is not met.
 - An OPENRULE command with the IGNORE action is defined for a set of volumes, asking for the use of a volume to be ignored, but the user is not authorized to do so for that volume.
- You have defined REJECT commands in parmlib member EDGRMMxx, and:
 - You have defined a REJECT command for a range of shelf locations or volumes, preventing a volume from being used on a particular system.
 - The volume is not defined to DFSMSrmm and a REJECT command requests that all non-defined volumes are to be rejected.

Rejects caused by validation failure

DFSMSrmm performs validity checking on volumes when you read or write to them if DFSMSrmm is recording information about the data sets on the volumes. If you are running DFSMSrmm in protect mode the volume is rejected under these conditions:

- The wrong volume is mounted for a specific volume request.
- An attempt is made to use a specific scratch volume. In DFSMSrmm, when you want a specific volume, you must request a specific, non-scratch volume, and when you want a scratch volume, you must request a non-specific mount.
- A private volume (master or user) is mounted in response to a scratch request.
- The data set information for the first file on the volume that DFSMSrmm has recorded during open, close, or end-of-volume processing does not match the information on the volume.
- An attempt is made to overwrite a data set on a master volume and the specified data set name does not exactly match the data set name that DFSMSrmm has recorded. You can control the overwriting of data sets on master volumes using the EDGRMMxx OPTION MASTEROVERWRITE operand. If both the data set being written, and the data set DFSMSrmm has recorded are generation data group data sets, DFSMSrmm ignores the generation data group suffix when comparing the data set names.

If the volume is part of a multivolume sequence containing multiple data sets, DFSMSrmm uses only the first volume, first file data set name for validation; for all other volumes the sequence of volumes is validated to prevent overwrite.

- An attempt is made to read a data set that DFSMSrmm has not recorded, and the volume information was previously recorded by DFSMSrmm.
- An attempt is made to automatically label a non-scratch volume in an automated tape library.
- An attempt is made to write on a volume that should be replaced (release action REPLACE).

Rejects caused by DFSMSrmm rules

DFSMSrmm checks that volumes are labeled correctly and that the volume status and intended usage is acceptable to DFSMSrmm. If you are running DFSMSrmm in protect mode the volume is rejected under these conditions:

- The DSNEXPIRE(BLOCK) option is used and the data set has an expiration date in the past and is not retained. Or, CHANGEDATASET FORCEEXPIRE was executed against the data set.
- An attempt is made to read a scratch volume.
- An attempt is made to read a volume obtained using the RMM GETVOLUME subcommand and the volume has not yet been written to. You use the RMM GETVOLUME subcommand to request a scratch volume and assign it to an owner defined to DFSMSrmm.
- Bypass label processing (BLP) is being used to write to a scratch or master volume unless you requested BLP processing through EDGRMMxx in the parmlib.
- An attempt is made to read or write to a volume using nonstandard labels, and the volume is defined to DFSMSrmm.
- An attempt is made to overwrite standard labels on a master volume, and the user is not authorized to do so.
- An attempt is made to write standard labels on a master volume that has no labels, and the user is not authorized to do so.
- An attempt is made to overwrite standard labels on a scratch volume by a user that is not authorized to do so.
- A scratch volume is requested for a nonstandard label request. In DFSMSrmm, scratch volumes must have standard labels.
- A volume is in an automated tape library and is defined to DFSMSrmm, but it is defined as not having a standard label or has different internal and external volume labels.
- An attempt is made to read or write to a volume that is waiting to be initialized.
- An attempt is made to read or write to a volume that is pending release.
- An attempt is made to write to a data set on the scratch volume other than the first one.
- An attempt is made to write to a data set that was specified with a sequence number that is not the next in the sequence from the last file DFSMSrmm has recorded. This applies only if DFSMSrmm is recording information about all the data sets on the subject volume.

Who can use DFSMSrmm and how?

This topic describes DFSMSrmm users and the tasks they can perform.

General user

General users need only limited access to DFSMSrmm functions. They might want to manage volumes they own and request information about resources that are defined to DFSMSrmm.

General users can use DFSMSrmm to perform these tasks:

- Manually request a scratch volume
- Change information about an owned volume or data set
- Update information about your owner ID
- Manually release an owned volume
- Create lists of resources and display information about most resources that are defined to DFSMSrmm

Tape librarian

Tape librarians can use DFSMSrmm to perform these tasks:

- Define new volumes
- Add shelf locations to the removable media library and to storage locations
- Add, change, and delete information about resources defined to DFSMSrmm
- Manually release any volume
- Confirm volume movements and actions
- Create lists of resources and display information about any resource defined to DFSMSrmm

Storage administrator

Storage administrators can use DFSMSrmm to perform these tasks:

- Define retention and storage policies for data sets and volumes
- Change information about any volume they own, their owner ID, and any vital record specification
- Manually request a scratch volume
- Manually release a volume they own
- Delete a vital record specification
- Create lists of resources and display information about resources defined to DFSMSrmm

Application programmer

Application programmers can use DFSMSrmm to perform these tasks:

- Add, change, and delete information about any volume owner
- Manually request a volume and manually release a volume they own
- Create a list of software products and display information about any resource defined to DFSMSrmm
- Code calls to the DFSMSrmm application programming interface in assembler language or in any high-level language that can use the object-oriented interfaces and classes supplied with DFSMSrmm, including Java.

System programmer

System programmers can use the DFSMSrmm support menu to perform these tasks:

- Display parmlib options and control data set control information
- Add, change, and delete information about any volume owner
- Manually request a volume and manually release a volume they own
- Create a list of software products and display information about any resource defined to DFSMSrmm

Operator

Operators can use DFSMSrmm to perform these tasks:

- Fetch and mount tapes from specific pools or shelf locations, as specified in mount messages
- Manually erase and initialize tapes
- Manually request a scratch volume
- Manually release a volume they own
- Create lists of scratch tapes available for use

See [*z/OS DFSMSrmm Managing and Using Removable Media*](#) for a complete description of operator procedures.

Using DFSMSrmm

You define RACF profiles to establish the authorization scheme for using DFSMSrmm functions. The basic authorization scheme recognizes there are different types of users and each user type will request common DFSMSrmm functions. See [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215](#) for information on how authorization is set up for each user type.

In DFSMSrmm, you can use either the DFSMSrmm ISPF dialog or the set of TSO subcommands to request DFSMSrmm functions. The RACF profiles control whether or not DFSMSrmm responds to requests for functions. If you request a function you are not authorized to use, your request will fail. For descriptions of the TSO subcommands available within DFSMSrmm, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

DFSMSrmm offers menus in the DFSMSrmm ISPF dialog that are tailored specifically to a user group's needs and level of access authorization. For example, only tape librarians are authorized to add software products to DFSMSrmm, so only the DFSMSrmm Librarian Menu includes an option to add software products. DFSMSrmm provides a specific menu for general users, storage administrators, tape librarians, and system programmers. It does not provide a menu for operators.

Using DFSMSrmm WUI

DFSMSrmm WEB GUI(WUI) provides an alternative to ISPF green screens. It is deployed as a z/OSMF plug-in and interacts with RMM by using the z/OSMF REST API interface and RMM TSO commands. A subset of those features is offered that can be controlled or displayed in the DFSMSrmm ISPF panels. This modern user interface allows users that are not experienced with RMM to quickly get started and get an easier overview of RMM status and volume information. Most of the fields in the DFSMSrmm WUI have hints with helpful instructions. Also, input fields have data validation that prevents users from entering incorrect values.

The set of functions that are provided includes:

- Monitoring of RMM server status.
- Displaying filtered list RMM volumes and their details.
- Displaying filtered list of RMM data sets and their details.
- Displaying RMM Defaults Table data.
- Displaying settings for the plug-in.
- Exporting table data into CSV.

For more information, see [“Using the DFSMSrmm WUI” on page 497](#).

Chapter 2. Implementing DFSMSrmm

This topic describes tasks you need to perform to implement DFSMSrmm. This topic also includes some optional tasks that might not need to be performed for your installation. Review all the steps in this topic as well as information in the [z/OS Upgrade Workflow](#) before you begin implementing DFSMSrmm. If you have multiple z/OS images or a RMMplex, you must repeat several of these steps for each z/OS image. We provide recommendations for steps that should be repeated for each image. [z/OS Upgrade Workflow](#) describes steps that are required before you install DFSMSrmm.

To access IBM Redbooks on topics such as converting to DFSMSrmm from another tape management product, visit [IBM Redbooks \(www.ibm.com/redbooks\)](http://www.ibm.com/redbooks).

Here are the steps that you can follow to implement DFSMSrmm.

- [“Step 1: Preparing to implement DFSMSrmm” on page 23](#)
- [“Step 2: Running the installation verification procedure \(optional\)” on page 24](#)
- [“Step 3: Updating JES3 \(optional\)” on page 24](#)
- [“Step 4: Updating installation exits” on page 24](#)
- [“Step 5: Updating SYS1.PARMLIB members” on page 25](#)
- [“Step 6: Using the Problem Determination Aid Facility \(optional\)” on page 43](#)
- [“Step 7: Setting up DFSMSrmm disposition processing \(optional\)” on page 43](#)
- [“Step 8: Updating the procedure library” on page 43](#)
- [“Step 9: Assigning DFSMSrmm a RACF user ID” on page 46](#)
- [“Step 10: Defining parmlib member EDGRMMxx” on page 47](#)
- [“Step 11: Tailoring parmlib member EDGRMMxx” on page 47](#)
- [“Step 12: Creating the DFSMSrmm control data set” on page 48](#)
- [“Step 13: Creating the journal” on page 53](#)
- [“Step 14: Authorizing users” on page 55](#)
- [“Step 15: Making the DFSMSrmm ISPF dialog available to users” on page 55](#)
- [“Step 16: Restarting z/OS with DFSMSrmm implemented” on page 59](#)
- [“Step 17: Tailoring DFSMSrmm set up” on page 59](#)
- [“Step 18: Updating the workload management service definition for DFSMSrmm” on page 60](#)
- [“Step 19: Starting DFSMSrmm” on page 60](#)
- [“Step 20: Defining resources” on page 63](#)
- [“Step 21: Updating the operational procedures” on page 66](#)
- [“Step 22: Initializing the DFSMSrmm subsystem and tape recording” on page 66](#)
- [“Step 23: Setting up DFSMSrmm utilities” on page 68](#)
- [“Step 24: Installing PTFs and the SMP/E maintenance to DFSMSrmm” on page 69](#)

Step 1: Preparing to implement DFSMSrmm

You should have installed DFSMSrmm, along with the other DFSMS components, using SMP/E. Refer to the *z/OS Program Directory* you received with the product tape for complete installation instructions.

If you have multiple z/OS images or systems on which you want DFSMSrmm to manage tape processing, there are some basic decisions you must make about how to implement DFSMSrmm. If the systems have access to shared DASD, you can set up one DFSMSrmm control data set and share it among your systems. Systems in a sysplex are best managed using a single control data set. When you have multiple sysplexes and have shared DASD between them, you can choose to have a single control data set to be shared or

one control data set per sysplex. If you do not have shared DASD on which to place a control data set for all systems to share, consider using DFSMSrmm client server support for z/OS to enable a single control data set to be used. See [Chapter 3, “Setting up DFSMSrmm client and server systems,”](#) on page 71 for additional information.

Recommendation: After you have installed DFSMSrmm using SMP/E, IPL your system without performing any DFSMSrmm implementation tasks and have DFSMSrmm take no part in removable media management. This is especially helpful if you are running another tape management product in production.

See [Appendix C, “Evaluating removable media management needs,”](#) on page 539 for a checklist to help you plan for implementing DFSMSrmm.

Step 2: Running the installation verification procedure (optional)

If this is the first time you are implementing DFSMSrmm, use the supplied IVP to verify that DFSMSrmm has installed correctly. You can run the IVP at any time, for example, after installing maintenance on your system.

See the *z/OS Program Directory* for the IVP procedures.

Step 3: Updating JES3 (optional)

DFSMSrmm samples provided in SAMPLIB

- EDG3IIP1 Sample to Update IATIIP1
- EDG3LVVR Sample to Update IATLVVR
- EDG3UX29 Sample to Update IATUX29
- EDG3UX62 Sample to Update IATUX62
- EDG3UX71 Sample to Update IATUX71

If you use DFSMSrmm and have JES3-managed tape devices that are not in an automated tape library, continue with this step. Otherwise, skip this step because allocation and DFSMSdftp handle all mount and volume verification.

DFSMSrmm provides SMP/E USERMODs that you can apply to the standard supplied JES3 user exits and other JES3 modules. The USERMODs can be used to prevent JES3 from validating certain volume mounts, to update JES3 fetch and mount messages, and to enable the use of no label tape volumes with JES3. If you have installed DFSMSrmm on a JES3 system, you might want to apply the JES3 USERMODs as described in [Chapter 19, “Running DFSMSrmm with JES3,”](#) on page 473. You must consider how applying a USERMOD might affect any locally-developed user exits.

Step 4: Updating installation exits

DFSMSrmm samples provided in SAMPLIB

- EDGCVRSX Sample to Use the Installation Exit EDG_EXIT100
- EDGUX100 Sample to Use the Installation Exit EDG_EXIT100
- EDGUX200 Sample to Use the Installation Exit EDG_EXIT200
- EDGUX300 Sample to Use the Installation Exit EDG_EXIT300

DFSMSrmm samples provided in AEDGSR1 or SMPSTS

- CBRUXCUA Programming Interface to EDGLCSUX
- CBRUXEJC Programming Interface to EDGLCSUX
- CBRUXENT Programming Interface to EDGLCSUX

- CBRUXVNL Programming Interface to EDGLCSUX
- IGXMSGEX Programming Interface to EDGMSGEX

DFSMSrmm provides three sample installation exit routines, EDGUX100, EDGUX200, and EDGUX300. You must install EDGUX100, EDGUX200, and EDGUX300 if you want to use them. [Chapter 11, “Using DFSMSrmm installation exits,” on page 267](#) describes the functions that the DFSMSrmm installation exits provide and how to use the exits. The installation exits that DFSMSrmm provides for IGXMSGEX and the OAM exits are implemented when DFSMS is installed. The installation exits that are installed for IGXMSGEX or the OAM exits replace any exits you had previously installed.

Recommendation: Convert your code to run when called by the DFSMSrmm exits and modify the supplied exits to call your code. You might need to modify your DFSMSrmm installation exits to provide the functions available in the exits you are currently using as well as provide functions required by DFSMSrmm.

Related reading:

- [“Processing fetch and mount messages: EDGMSGEX” on page 263](#) for details about the DFSMSrmm programming interface that you use with IGXMSGEX.
- [“Managing system-managed tape library volumes: EDGLCSUX” on page 251](#) for details about the DFSMSrmm programming interface that you use with the OAM installation exits.

Step 5: Updating SYS1.PARMLIB members

Update the SYS1.PARMLIB members IEFSSNxx and IKJTSOxx. Optionally, you can update SMFPRMxx, GRSRNLxx, and AUTORxx. You should also ensure that IFAPRD00 is updated correctly during installation of DFSMS.

Perform this step once for each z/OS image.

Updating IEFSSNxx

There are two changes to make in the IEFSSNxx member in SYS1.PARMLIB:

- Define a DFSMSrmm subsystem name to z/OS
- Add the name of the subsystem interface initialization program, EDGSSSI, to fully enable DFSMSrmm

Recommendation: Implement the changes in two steps. Define the DFSMSrmm system at this time as described in [“Defining DFSMSrmm to z/OS” on page 25](#). Then you can add the name of the subsystem interface as described in [“Step 22: Initializing the DFSMSrmm subsystem and tape recording” on page 66](#). If you make only this first change at this time, you can choose whether to start the DFSMSrmm procedure and choose a time to best suit your particular situation. If you implement both changes at once, DFSMSrmm rejects all tape mounts until the subsystem procedure is active or you use the EDGRESET utility to disable the interface.

Defining DFSMSrmm to z/OS

Add an entry to IEFSSNxx to define the DFSMSrmm subsystem to z/OS. Specify the DFSMSrmm subsystem name shown in [Figure 3 on page 26](#). [Figure 3 on page 26](#) shows where to place the subsystem name. Place the DFSMSrmm entry after the JES2 or JES3 entry and before the NetView entry to ensure that NetView automation receives the write-to-operator messages once DFSMSrmm has updated them. Additionally, if you have any software identified as a subsystem that is dependent on open/close/end-of-volume processing, place the names after the DFSMSrmm entry. DFSMSrmm has no other dependencies on the sequence of subsystem names. Although, we do recommend that RMM should be configured and running prior to other system automation tools.

Note: The DFSMSrmm subsystem can be defined to allow the SSI initialization routine for DFSMSrmm to be invoked in parallel with other SSI initialization routines. See [z/OS MVS Initialization and Tuning Guide](#) for more information.

```

SUBSYS SUBNAME(JES2)           /* JES2 PRIMARY SUBSYSTEM START */
PRIMARY(YES) START(YES)
SUBSYS SUBNAME(DFRM)           /* Name of the DFSMSrmm subsystem */
SUBSYS SUBNAME(AOPA)           /* Netview */

```

Figure 3. Defining DFSMSrmm to z/OS through IEFSSNxx with subsystem inactive

where:

DFRM

Specifies the subsystem name. You can use any unique subsystem name, one to four characters long.

Although the subsystem name can be one-to-four characters long, the subsystem name must be four characters and must match the first four characters of the procedure name under these conditions:

- You have not added EDGSSSI, as described in [“Step 22: Initializing the DFSMSrmm subsystem and tape recording”](#) on page 66, and you want DFSMSrmm to perform the EDGSSSI processing when it starts by replying RETRY to message EDG0103D.
- You use the EDGRESET utility or the START command

```
S DFRMM,OPT=RESET
```

and plan to restart DFSMSrmm without an IPL.

Run DFSMSrmm as a subsystem that is started under the JES2 or JES3 subsystem, rather than under the master subsystem. When choosing the DFSMSrmm subsystem name, the subsystem name must not exactly match the DFSMSrmm procedure name, unless you specify SUB=JES2 or SUBJ=JES3 on each START command. For example, if you use DFRM as the subsystem name, use DFRMM as the procedure name.

You must run DFSMSrmm under the JES2 or JES3 subsystem to use these functions:

- Use the DFSMSrmm NOTIFY function which automatically notifies volume and product owners when the volumes they own become eligible for release or when product volumes are added.
- Use the SYSOUT facilities like //SORTOUT DD SYSOUT=*.

Dynamically adding the DFSMSrmm subsystem

You can dynamically add the DFSMSrmm subsystem without an IPL. Use the z/OS system command SETSSI to add the DFRM subsystem. Issue the SETSSI command from a console that has master authority or a console that has sufficient RACF authority.

To activate the DFSMSrmm subsystem, issue the command:

```
SETSSI ADD,SUBNAME=DFRM
```

Enabling DFSMSrmm and tape recording

You enable DFSMSrmm later, in [“Step 22: Initializing the DFSMSrmm subsystem and tape recording”](#) on page 66. Refer to this topic now if you are interested in the changes you will make later to IEFSSNxx to enable the DFSMSrmm tape recording interface.

Updating IKJTSOxx to authorize DFSMSrmm commands

Update IKJTSOxx to authorize RMM commands and utilities. Refer to [z/OS TSO/E Customization](#) if you use the TSO CSECT facility rather than IKJTSOxx for these updates.

To authorize the TSO command RMM, make the specifications as shown in [Figure 4 on page 26](#):

```
AUTHCMD NAMES(RMM)
```

Figure 4. Updating IKJTSOxx to authorize RMM commands

To authorize the DFSMSrmm utilities that you can use within TSO/E or call through the TSO/E Service Facility, make the specifications as shown in [Figure 5 on page 27](#):

```
AUTHTSF NAMES(EDGHSKP
              EDGUTIL
              EDGRPTD
              EDGBKUP
              EDGUPDT
              EDGAUD)
AUTHPGM NAMES(EDGHSKP
              EDGUTIL
              EDGRPTD
              EDGBKUP
              EDGUPDT
              EDGAUD)
```

Figure 5. Updating IKJTSOxx to call DFSMSrmm through TSO

To dynamically obtain the updated version of the IKJTSOxx member, use the TSO PARMLIB UPDATE(**) command in your TSO session.

Updating SMFPRMxx (optional)

If you want DFSMSrmm security-type SMF records or audit-type SMF records, update the SMFPRMxx member to ensure that the DFSMSrmm SMF records are generated. Define the SMF records you require in the DFSMSrmm parmlib member EDGRMMxx using the OPTION command and the SMFAUD operand for auditing records and the SMFSEC operand for security records. For more information on these commands, see [Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143](#).

IBM recommends that you use the IBM assigned SMF record type of 42, the audit records subtype of 22, and the security records subtype of 23. To use the IBM record type, you specify OPTION SMFAUD(YES) and SMFSEC(YES) within the RMM parmlib member as outlined in [Chapter 8: Using the parmlib member EDGRMMxx](#). Alternatively, to use SMF record types from the user-written range, select two SMF record types in the user-written range 128 through 255 that your installation is not currently using. For example:

```
OPTION SMFAUD(128) SMFSEC(129)
```

Add your record types to the SMFPRMxx parmlib member as described in [z/OS MVS System Management Facilities \(SMF\)](#).

Updating GRSRNLxx (optional)

Before you begin

Skip this step if you are not using global resource serialization (GRS) to serialize access to resources. If you have multiple systems connected with global resource serialization, decide now how you want GRS to handle the reserve on the control data set. For reference, SYSZRMM MASTER.RESERVE is used during initialization of RMM, and SYSZRMM MASTER.RESERVE.cdsid is used post initialization.

Recommendations

Ensure that the volume where the DFSMSrmm control data set resides does not contain any other data on it. The handling of the SYSZRMM MASTER.RESERVE resource depends on how the SYSIGGV2 and SYSZVDS resources are being treated. Ideally, these resources are all being sharing inside the GRSPLEX and can be converted from using real hardware reserves by putting them all in the GRS CONVERSION RNL. SYSZRMM MASTER.RESERVE is by default managed as a hardware reserve and in some circumstances can get into a deadlock with Catalog if all of these resources are not in the same GRS RNL list. If you are converting the SYSIGGV2 resource for the ICF Catalog that contains the RMM CDS, then you should also consider converting SYSZRMM MASTER.RESERVE resource and the SYSZVDS resource for the specific volume that the RMM CDS resides on.

- If real reserves could impact other systems or cause DASD contention problems, convert the RESERVE to a SYSTEMS enqueue.

Example: Add the reserve names to the global resource serialization exclusion list as shown in [Figure 6 on page 28](#). This leaves the real hardware reserve in effect, and causes the associated

SYSTEMS enqueue to be converted to a local SYSTEM one. You must specify both the major name (QNAME) and the minor name (RNAME) as shown in [Figure 6 on page 28](#) when converting the SYSTEMS enqueue to a local SYSTEM enqueue. If you only specify the major name, all enqueues used by DFSMSrmm are converted and you will be unable to run DFSMSrmm on multiple systems.

Note: In [Figure 6 on page 28](#), replace *cdsid* with your selected CDSID, as specified in the EDGRMMxx parmlib member OPTION statement.

```
RNLDEF RNL(EXCL) TYPE(SPECIFIC)
QNAME(SYSZRMM)
RNAME(MASTER.RESERVE)
RNLDEF RNL(EXCL) TYPE(SPECIFIC)
QNAME(SYSZRMM)
RNAME(MASTER.RESERVE.cdsid)
```

Figure 6. Converting the SYSTEMS enqueue to a local SYSTEM enqueue

In the case of RNAME for CDS specification denoted as RNAME(MASTER.RESERVE.*cdsid*), the length of RNAME must be exactly 23 characters. If the length of your CDSID is less than 8 characters, we recommend you use a generic entry as described below in [figure 7](#) and [figure 9](#). Otherwise, you must specify RNAME as a hexadecimal constant: RNAME(X'D4C1E2E3C5D94BD9C5E2C5D9E5C54Bxx..xx4040') where X'D4C1E2E3C5D94BD9C5E2C5D9E5C54B' represents MASTER.RESERVE, and X'xx..xx4040', your CDSID appended with blanks to 8 characters in length.

If your CDSID contains nondisplayable characters, you must specify RNAME as hexadecimal constant and use two hexadecimal digits to specify each character of the name.

An alternative to creating multiple exclusion RNL entries is to change the existing definition from SPECIFIC to GENERIC. See [Figure 7 on page 28](#) for an example.

```
RNLDEF RNL(EXCL) TYPE(GENERIC)
QNAME(SYSZRMM)
RNAME(MASTER.RESERVE)
```

Figure 7. Changing an existing definition from SPECIFIC to GENERIC

- If the volume contains other critical data and real reserves could impact other systems or cause DASD contention problems, convert the RESERVE to a SYSTEMS enqueue. If you do not change your GRSRNLxx parmlib member, DFSMSrmm still functions correctly; but any performance gain from using the real hardware reserve feature is negated by the overhead created from sending the global SYSTEMS enqueue around the GRS ring.

Example: Add the reserve name to the global resource serialization reserve conversion list as shown in [Figure 8 on page 28](#) to leave the global SYSTEMS enqueue in effect and remove the real hardware reserve. You must specify both the major name (QNAME) and the minor name (RNAME) as shown in [Figure 8 on page 28](#) when converting the RESERVE to a SYSTEMS enqueue.

Note:

1. Do not convert the reserve if the control data set volume is shared with any other system outside the sysplex.
2. In [Figure 8 on page 28](#), replace *cdsid* with your selected CDSID, as specified in the EDGRMMxx parmlib member OPTION statement.

```
RNLDEF RNL(CON) TYPE(SPECIFIC)
QNAME(SYSZRMM)
RNAME(MASTER.RESERVE)
RNLDEF RNL(CON) TYPE(SPECIFIC)
QNAME(SYSZRMM)
RNAME(MASTER.RESERVE.cdsid)
```

Figure 8. Converting the RESERVE to a SYSTEMS enqueue

The length of RNAME must be exactly 23 characters. If the length of your CDSID is less than 8 characters, we recommend you use a generic entry as described below. Otherwise, you must specify

RNAME as a hexadecimal constant: RNAME(X'D4C1E2E3C5D94BD9C5E2C5D9E5C54Bxx..xx4040') where X'D4C1E2E3C5D94BD9C5E2C5D9E5C54B' represents MASTER.RESERVE, and X'xx..xx4040' your CDSID appended with blanks to 8 characters in length.

If your CDSID contains nondisplayable characters, you must specify RNAME as hexadecimal constant and use two hexadecimal digits to specify each character of the name.

An alternative to creating multiple conversion RNL entries is to change the existing definition from SPECIFIC to GENERIC. See [Figure 9 on page 29](#) for an example.

```
RNLDEF RNL(CON) TYPE(GENERIC)
QNAME(SYSZRMM)
RNAME(MASTER.RESERVE)
```

Figure 9. Changing an existing SPECIFIC definition to GENERIC

[Table 5 on page 29](#) is the list of DFSMSrmm resource symbolic names. Do not use GRS to change or alter any of these ENQs, except for the case shown in [Figure 9 on page 29](#). The resource symbolic names are provided for information only because DFSMSrmm processing manages serialization for you.

Table 5. DFSMSrmm resource symbolic names

Major Name	Minor Name	Resource	Scope
SYSZRMM	HSKP.dsn.volser	Inventory management data set serialization	SYSTEMS
SYSZRMM	MASTER.RESERVE	DFSMSrmm control data set serialization at startup and when the CDSID is not yet known	SYSTEMS
SYSZRMM	MASTER.RESERVE.cdsid	DFSMSrmm control data set serialization	SYSTEMS
SYSZRMM	MHKP.ACTIVE	Serialize inventory management functions on the same DFSMSrmm subsystem	SYSTEM
SYSZRMM	MHKP.dsn.volser	Inventory management data set serialization	SYSTEMS
SYSZRMM	RMM.ACTIVE	Ensure only one system run per z/OS image	SYSTEM
SYSZRMM	BUFFER.CONTROL	Buffer management	STEP
SYSZRMM	EDGINERS.volser	Serialize volume labeling	SYSTEMS
SYSZRMM	SHUTDOWN	Serialize DFSMSrmm shutdown and refresh processing	SYSTEM
SYSZRMM	INACTIVE	Serialize DFSMSrmm activation enabling only a single WTOR to be issued to the operator	SYSTEM
SYSZRMM	EXIT_IS_ACTIVE	Exit recovery serialization	SYSTEM
SYSZRMM	WTOR_ENQ	Exit recovery serialization	SYSTEM
SYSZRMM	WTORIPC	TCP/IP error recovery serialization on CLIENT systems	SYSTEM

Table 5. DFSMSrmm resource symbolic names (continued)

Major Name	Minor Name	Resource	Scope
SYSZRMM	EXIT_id_UNAVAIL	Exit recovery serialization where id can be 100, 200, or 300 representing the last three characters of the DFSMSrmm installation exits EDG_EXIT100, EDG_EXIT200, or EDG_EXIT300.	SYSTEM

See [z/OS MVS Planning: Global Resource Serialization](#) for additional information about global resource serialization.

Updating AUTORxx (optional)

The z/OS auto-reply (AUTOR) facility provides replies to write-to-operator with reply (WTOR) messages in cases where there is no automation or when the operator is unaware of the outstanding request or is taking too long trying to determine what the response should be. When AUTOR is active, z/OS will use the auto-reply policies provided for the subset of the DFSMSrmm WTORS that are included in the default AUTOR00 parmlib member provided by z/OS. AUTOR00 also contains comments that provide the message text for each WTOR and the rule used to select the WTOR.

You can define your own AUTORxx member to customize the auto-reply policies for DFSMSrmm. You can override or supplement the policies in AUTOR00 to add more WTORS to be handled automatically and to change the replies for these WTORS already included in AUTOR00. DFSMSrmm provides these AUTORxx parmlib members, which you can use as-is or use as the base for your customization:

AUTORRM

Includes, for all DFSMSrmm WTORS, an automated response suitable for production mode running.

AUTORRP

Includes, for all DFSMSrmm WTORS, an automated response suitable for use when running DFSMSrmm in a mode other than OPMODE(PROTECT).

Note: When converting to DFSMSrmm from another tape management program, you can use OPMODE(RECORD) or OPMODE(WARNING) to allow the other program to manage your tapes, while DFSMSrmm only records the actions it would have taken if OPMODE(PROTECT) were in effect. This is known as *parallel mode*. When running in parallel mode, you can scan your job or system logs for DFSMSrmm messages to compare the other product and DFSMSrmm reports for differences.

“[Information for customizing AUTORxx](#)” on page 30 provides additional information about the contents of the AUTOR00, AUTORM, and AUTORRP members that you might find useful in when creating your own AUTORxx member.

The automated responses in both AUTORM and AUTORRP allow a short time for either the operator or your system automation to make a reply. If no reply is made, the automated response is used.

Use the AUTOR=(xx,00) keyword in IEASYSyy to specify your selected parmlib member. If you are new to DFSMSrmm, you might use AUTOR=(RP,00) until you are ready for production. You can then use AUTOR=(RM,00) or your customized AUTORxx.

For additional information about the AUTORxx and IEASYSyy parmlib members, see [z/OS MVS Initialization and Tuning Guide](#).

Information for customizing AUTORxx

This topic lists, for each DFSMSrmm WTOR message, information that you should consider when deciding what changes (if any) you want to implement in your AUTORxx member:

- Message number and text

- **Severity** (None, Low, Medium, High) indicates the level of impairment to the operation of DFSMSrmm and its ability to manage tape processing if the operator does not respond to the WTOR or gives an inappropriate response (for example CANCEL, IGNORE).
- **Tape Processing stopped?** shows whether tape use is prevented while this WTOR is outstanding.
- **Parallel mode** shows the delay in seconds (s) or minutes (m) and the provided reply for this WTOR when PARMLIB(AUTORRP) is in effect
- **Production mode** shows the delay in seconds (s) or minutes (m) and the provided reply for this WTOR when PARMLIB(AUTORRM) is in effect
- Additional comments about each message.

For more information about the messages in this topic, refer to *z/OS MVS System Messages, Vol 5 (EDG-GLZ)* and to the DFSMSrmm operator procedures topic in *z/OS DFSMSrmm Managing and Using Removable Media*.

EDG0103D DFSMSrmm SUBSYSTEM INTERFACE IS INACTIVE - ENTER "IGNORE", "CANCEL" OR "RETRY"

Severity

None

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: RETRY

Production mode

Delay: 1m Reply: RETRY

This message is normal for starting RMM after IPL if you have not included INITRTN(EDGSSSI) in the IEFSSNxx parmlib entry for the rmm subsystem

EDG0107A ENTER SUFFIX OF INITIALIZATION MEMBER OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: P0

Production mode

Delay: 1m Reply: 00

If INITRTN(EDGSSSI) was specified in IEFSSNxx or you already successfully started DFRMM since IPL you cannot use tape until DFRMM starts successfully

EDG0110D ENTER TODAY'S DATE WITH FORMAT *date_string* OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

N/A

Production mode

N/A

There is no pre-selected response for WTOR EDG0110D because we recommend you do not use system automation to reply. You should use other facilities, such as the DFSMSrmm OPTION IPLDATE(NO), to ensure the correct systems date.

EDG0115D THE DFSMSrmm SUBSYSTEM IS NOT RUNNING UNDER A JOB ENTRY SYSTEM - SOME DFSMSrmm FUNCTIONS ARE NOT AVAILABLE. REPLY "IGNORE" OR "CANCEL"
Severity

Medium

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: IGNORE

Production mode

Delay: 1m Reply: CANCEL

Next time ensure DFRMM is started under JES in order to get full functional capability. REPLY "IGNORE" if you do not need the JES facilities.

EDG0117D DFSMSrmm DYNAMIC INSTALLATION EXITS INITIALIZATION FAILED - REPLY "IGNORE", "CANCEL" OR "RETRY"
Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: IGNORE

Production mode

Delay: 1m Reply: CANCEL

In production environment do not run rmm if any of the required installation exits cannot be initialized correctly

EDG0123D function FOUND TO BE ACTIVE ON SYSTEM *system_name* - REPLY "Y" TO RESET STATUS OR "N"
Severity

Medium

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: N

Production mode

Delay: 5m Reply: N

Determine if RMM housekeeping is running on the system specified in the message. If so, reply "N". If RMM housekeeping failed on the system specified in the message, or the system has been reset then reply "Y"

EDG0127D RECOVERY OF CONTROL DATA SET ON *date* AT *time* POSSIBLY INCOMPLETE - REPLY "CONTINUE" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: CONTINUE

Production mode

Delay: 5m Reply: CANCEL

CDS recovery was incomplete. Run EDGUTIL VERIFY to clean up or repeat the recovery process.

EDG0215D ERRORS DETECTED IN INITIALIZATION PARAMETERS - ENTER "Y" TO CONTINUE OR "N" TO CANCEL

Severity

Medium

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: Y

Production mode

Delay: 2m Reply: N

Reply "Y" in parallel, Reply "N" in production. Correct parmlib members for next start up.

EDG0228D REUSEBIN(STARTMOVE) REQUIRES EXTENDED BIN ENABLED - USING (CONFIRMMOVE) - REPLY .CONTINUE. OR .CANCEL.

Severity

Medium

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: Y

Production mode

Delay: 2m Reply: CANCEL

Reply CONTINUE in parallel and CANCEL in production

EDG0358D SERVER *server_name* COMMUNICATION ERROR - REPLY "CANCEL", OR "RETRY"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 10m Reply: CANCEL

Production mode

Delay: 10m Reply: CANCEL

This WTOR will be handled automatically by DFSMSrmm with automated retry. IF TCP/IP is not working reply CANCEL

EDG0361D SERVER STARTUP ERROR - REPLY "RETRY" OR "IGNORE"**Severity**

Medium

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 10m Reply: IGNORE

Production mode

Delay: 10m Reply: RETRY

This WTOR will be handled automatically by DFSMSrmm with automated retry. IF TCP/IP is not working reply IGNORE

EDG1107D REQUESTS WAIT TO BE PROCESSED - REPLY "STOP", "QUIESCE", "RESTART", OR "M=xx"**Severity**

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: STOP

Production mode

Delay: 2m Reply: RESTART

You stopped DFRMM while QUIESCED. Reply STOP in parallel and RESTART or M=xx in production

EDG1200D I/O ERROR ON CONTROL DATA SET WHEN PROCESSING MESSAGE *msg_number*, REPLY EITHER "RETRY" OR "CANCEL"**Severity**

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 30s Reply: CANCEL

Production mode

Delay: 30s Reply: CANCEL

Low on scratch processing is not needed in parallel, and in production cannot wait during a WTO

EDG1203D INVENTORY MANAGEMENT PREVENTED PROCESSING OF MESSAGE *msg_number*, REPLY EITHER "RETRY" OR "CANCEL"**Severity**

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 30s Reply: CANCEL

Production mode

Delay: 30s Reply: CANCEL

Low on scratch processing is not needed in parallel, and in production cannot wait during a WTO

EDG2103D PERMANENT JOURNAL ERROR - REPLY "R" TO RETRY, "I" TO IGNORE, "D" TO DISABLE OR "L" TO LOCK

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 30s Reply: D

Production mode

Delay: 30s Reply: L

In parallel - Reply "D" (disable) the journal. In production we cannot allow any tape usage without the journal, so reply L (lock). Notify the system programmer to run EDGHSKP BACKUP to re-enable the journal.

EDG2106D JOURNAL AND CONTROL DATASET DO NOT MATCH - REPLY "C" TO CANCEL, "D" TO DISABLE OR "L" TO LOCK

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 30s Reply: D

Production mode

Delay: 30s Reply: L

Reply LOCK and notify the system programmer to run EDGHSKP BACKUP to re-enable the journal.

EDG3213D ANOTHER GETVOLUME CURRENTLY IN PROGRESS - ENTER "RETRY", "CANCEL", OR "IGNORE"

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 30s Reply: RETRY

Production mode

Delay: 30s Reply: RETRY

Reply "RETRY" until GETVOLUME satisfied.

**EDG4000D JOURNAL FILE IS LOCKED DURING *action* FOR *volser* BY *jobname*, *stepname*, *ddname*;
ENTER "RETRY" OR "CANCEL"**

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 30s Reply: CANCEL

Production mode

Delay: 2m Reply: RETRY

Reply "RETRY" so production tape process is good and run EDGHSKP BACKUP to re-enable the journal ASAP

**EDG4008A SECURE *security_number* VOLUME *volser* IN USE BY *jobname*, *stepname*, *ddname* REPLY
WHEN READY**

Severity

Medium

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: U

Production mode

N/A

Reply to allow rmm processing to continue and review your SECCLS definitions. In production this WTOR is not automated. You should follow your local procedures.

**EDG4010D BACKUP IN PROGRESS DURING *action* FOR *volser* BY *jobname*, *stepname*, *ddname*;
ENTER "RETRY" OR "CANCEL"**

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

Reply "RETRY" so production tape process is good. Switch to using a non-intrusive backup technology with EDGHSKP BACKUP(DSS)

**EDG4012D DFSMSrmm INACTIVE FOR *action* *volser* BY *jobname*, *procname*, *stepname*, *ddname*;
ENTER "RETRY", "CANCEL" OR "CONTINUE"**

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: CONTINUE

Production mode

Delay: 1m Reply: RETRY

You should start DFRMM and the system automatically continues processing, or Reply "RETRY" so production tape processing is good.

EDG6605D JOURNAL FILE IS LOCKED DURING DFSMSrmm SUBSYSTEM PROCESSING - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 10m Reply: RETRY

Reply "RETRY" so production tape process is good and run EDGHSKP BACKUP to re-enable the journal ASAP

EDG6607D INVENTORY MANAGEMENT FUNCTIONS ARE ACTIVE - ENTER "RETRY" OR "CANCEL"

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 10m Reply: RETRY

Reply "RETRY" so production tape process is good.

EDG6609D FAILURE DURING DFSMSrmm SUBSYSTEM PROCESSING - ENTER "RETRY" OR "CANCEL"

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 10m Reply: RETRY

Reply "RETRY" so production tape process is good.

EDG6626A SPECIFY VOLUME "INIT" OR "ERASE" COMMAND OR "END"

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 10m Reply: END

Production mode

Delay: 10m Reply: END

Reply "END" if no reply from anyone after a few minutes.

EDG6627A M drive_number V(volser) R(rack_number) TO BE action vol1_volser, label_type

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 10m Reply: S

Production mode

Delay: 10m Reply: S

Reply "S" if volume not mounted after a few minutes.

EDG6628A drive_number, REPLY WITH RACK NUMBER OR VOLUME SERIAL NUMBER FOR NL VOLUME

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

N/A

Production mode

N/A

There is no pre-selected response for WTOR EDG6628A. There is no impact to tape processing, and the operator must verify the mounted tape in order to reply.

EDG6629D DFSMSrmm SUBSYSTEM NOT ACTIVE - ENTER "RETRY" OR "CANCEL"

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: CANCEL

EDGINERS is not needed if RMM is not active.

EDG6663D *drive_number*, REPLY "R" TO RETRY OR "F" TO FAIL THE REQUEST, OR "A" TO ACCEPT THE MOUNTED VOLUME

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 10m Reply: F

Production mode

Delay: 10m Reply: F

Fail the EDGINERS request if operator does not handle.

EDG6671D *drive_number*, ERROR PROCESSING VOLUME *volser*, REP "S" TO SKIP, "F" TO FAIL OR "R" TO RETRY

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

No

Parallel mode

Delay: 10m Reply: F

Production mode

Delay: 10m Reply: F

Fail the EDGINERS request if operator does not handle.

EDG8008D DFSMSrmm I/O ERROR DURING *task function* REQUEST FOR *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGTVEXT - DFRMM is probably QUIESCED. Automation should start DFRMM again, meanwhile we RETRY

EDG8010D BACKUP IN PROGRESS DURING *task function* REQUEST FOR *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGTVEXT - Reply RETRY so production tape is good. Ensure EDGHSKP uses BACKUP(DSS) with concurrent copy.

EDG8011D DFSMSrmm SUBSYSTEM IS NOT ACTIVE DURING *task function* REQUEST FOR *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGTVEXT - Reply RETRY so production tape is good - ensure DFRMM is always active

EDG8013D DFSMSrmm JOURNAL FILE IS LOCKED DURING *task function* REQUEST FOR *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGTVEXT - Reply RETRY so production tape is good. Run EDGHSKP BACKUP(DSS) to empty and enable the journal.

EDG8102D DFSMSrmm SUBSYSTEM NOT ACTIVE DURING *function* PROCESSING FOR *volser* - ENTER "RETRY", "IGNORE", OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: IGNORE

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY so production tape is good. This occurs when SMS tape is in use and volume not in library.

EDG8108D DFSMSrmm I/O ERROR DURING *function* PROCESSING FOR VOLUME *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY so production tape is good.

EDG8110D BACKUP IN PROGRESS DURING *function* PROCESSING FOR VOLUME *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY so production tape is good and ensure EDGHSKP uses BACKUP(DSS) with concurrent copy.

EDG8113D DFSMSrmm JOURNAL FILE IS LOCKED DURING *function* PROCESSING FOR VOLUME *volser* - ENTER "RETRY" OR "CANCEL"

Severity

High

Tape Processing Stopped?

Yes

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY so production tape is good. Run EDGHSKP BACKUP(DSS) to empty and enable the journal.

EDG8121D ENTER *volume req_volser* INTO LIBRARY *lib_name* AND REPLY "RETRY", OTHERWISE REPLY "CANCEL" OR "CONTINUE"

Severity

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CONTINUE

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY and let operator enter the tape.

EDG8122D ENTER *volume req_volser* INTO LIBRARY *lib_name* AND REPLY "RETRY", OTHERWISE REPLY "CANCEL"**Severity**

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY and let operator enter the tape.

EDG8123D LOGICAL VOLUME *req_volser* EXPORTED IN STACKED VOLUME *stack_volser* LOCATION *loc_name* SHELF *shelf_number* HOME LOCATION *home* - IMPORT VOLUME TO LIBRARY *lib_name* AND REPLY "RETRY", OTHERWISE REPLY "CANCEL"**Severity**

Low

Tape Processing Stopped?

No

Included in AUTOR00 ?

Yes

Parallel mode

Delay: 1m Reply: CANCEL

Production mode

Delay: 1m Reply: RETRY

EDGLCSUX - Reply RETRY and let operator enter the tape.

Enabling DFSMSrmm

The IFAPRDxx parmlib member is used to define the enablement policy for products or product features. To allow DFSMSrmm to execute on a z/OS system, the IFAPRDxx member that is in effect on the system must include FEATURENAME(DFSMSRMM) as an enabled feature.

To update the IFAPRDxx member dynamically, use the z/OS SET system command:

```
SET  PROD=xx
```

For more information on IFAPRDxx, see [z/OS MVS Initialization and Tuning Reference](#).

Step 6: Using the Problem Determination Aid Facility (optional)

Perform this step once for each z/OS image. You only need to perform this step if you want an external DASD record of trace data. IBM recommends that you keep a history of DFSMSrmm PDA trace data in case a problem is reported to IBM.

The problem determination aid (PDA) facility gathers DFSMSrmm processing information to enable analysis to pinpoint module flow and resource usage that is related to DFSMSrmm problems. The PDA facility is required for IBM Support Center because it traces module and resource flow. The PDA facility consists of in-storage trace, optional DASD log data sets, EDGRMMxx parmlib member options, and operator commands to control tracing. See Chapter 17, “Using the Problem Determination Aid Facility,” on page 461 for information about setting up the PDA facility.

Step 7: Setting up DFSMSrmm disposition processing (optional)

DFSMSrmm disposition processing is optional and provides support for these tasks:

- Providing operators with information to assist them in performing tasks like moving a tape to a specific location
- Generating sticky labels
- Updating the location where a volume resides

See Chapter 18, “Setting up DFSMSrmm disposition processing,” on page 467 for information about setting up DFSMSrmm disposition processing.

Step 8: Updating the procedure library

DFSMSrmm sample provided in SAMPLIB

- EDGDFRMM Sample to Create a Procedure in SYS1.PROCLIB

Perform this step once for each z/OS image.

Create a procedure in SYS1.PROCLIB to start the DFSMSrmm subsystem address space.

Tip: Figure 10 on page 43 shows sample JCL that you can use to create the procedure.

```
//DFRMM PROC M=00,OPT=MAIN  
//IEFPROC EXEC PGM=EDG&OPT.,PARM='&M',TIME=1440,REGION=40M  
//EDGPDOX DD DISP=SHR,DSN=RMM.&SYSNAME..RMMPDOX  
//EDGPDOY DD DISP=SHR,DSN=RMM.&SYSNAME..RMMPDOY  
//dispdd OUTPUT DEST=SYSTEMX,FORMS=LABEL,CLASS=L
```

Figure 10. Creating a procedure in SYS1.PROCLIB using the recommended JCL

The sample JCL in Figure 11 on page 44 shows the use of additional parameters for specifying the MASTER DD statement, the JOURNAL DD statement, the PARMLIB DD statement, and the IEFRDER DD statement.

Note: It is not necessary to create entries in SCHEDxx for DFSMSrmm. If entries are present in SCHEDxx, be sure integrity is not compromised by specifying NODSI. With NODSI specified, an enqueue will not be raised for MASTER and JOURNAL DD statements and possible corruption could occur.

```
//DFRMM PROC M=00,OPT=MAIN
//IEFPROC EXEC PGM=EDG&OPT.,PARM='&M',TIME=1440,REGION=40M
//PARMLIB DD DDNAME=IEFRDER
//IEFRDER DD DISP=SHR,DSN=SYS1.PARMLIB
//MASTER DD DISP=SHR,DSN=RMM.CONTROL.DSET
//JOURNAL DD DISP=SHR,DSN=RMM.JOURNAL.DSET
//EDGPDOX DD DISP=SHR,DSN=?UID..?HOSTID..RMMPDOX
//EDGPDOY DD DISP=SHR,DSN=?UID..?HOSTID..RMMPDOY
//dispdd OUTPUT DEST=SYSTEMX,FORMS=LABEL,CLASS=L
```

Figure 11. Creating a procedure in SYS1.PROCLIB using additional parameters

where:

DFRMM

Specifies the procedure name. You can use any procedure name, one-to-eight characters long, subject to the restriction documented under IEFSSNxx parmlib member.

IEFPROC

The first four characters of the task name cannot match the subsystem name defined in step 5, otherwise it is not possible to restart DFSMSrmm after using the EDGRESET utility.

dispdd

dispdd is an OUTPUT JCL statement that you code when you want to create label output data. The name of the output statement must match the name you specified in the DFSMSrmm EDGRMMxx parmlib OPTION DISPDDNAME operand as described in [“Defining system options: OPTION” on page 161](#). You can use any JCL keywords supported on the OUTPUT statement as described in [z/OS MVS JCL Reference](#).

EDGPDOX DD and EDGPDOY DD

EDGPDOX DD and EDGPDOY DD are required statements for external trace output recording. The specified data sets must exist on the same volume. If you do not specify these DD statements, no logging of the PDA trace output is performed. When DFSMSrmm swaps the PDA log data sets EDGPDOX and EDGPDOY, DFSMSrmm uses an intermediate data set name for the log data sets. The started task must have ALTER access to this intermediate data set. The data set name is ?UID.RMMPDO.TEMP.RMMPDO.H?SYSID. The ?UID is the DFSMSrmm ID, and ?SYSID is the first seven characters of the DFSMSrmm SYSID that is defined in the DFSMSrmm EDGRMMxx parmlib OPTION SYSID operand, as described in [Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143](#).

IEFRDER DD

IEFRDER is an optional statement. Use the IEFRDER DD statement to enable operators to specify PARMLIB DD keywords on the START command for the DFRMM procedure. If you do not code the IEFRDER DD and code the PARMLIB DD, you will not be able to specify a different PARMLIB data set name when starting the DFRMM procedure. If you do not code SYS1.PARMLIB, it is not necessary to code the IEFRDER DD statement.

JOURNAL DD

JOURNAL is an optional statement. If you code the JOURNAL DD statement to name the DFSMSrmm journal data set, you cannot easily change the journal name by switching to a different PARMLIB member. You must supply the journal data set name in the DFSMSrmm EDGRMMxx parmlib member if you do not code the JOURNAL DD statement and if you require journaling. You must catalog the journal.

M

Use M on the PROC statement to specify a parmlib member suffix. When you specify M=00, DFSMSrmm uses member EDGRMM00.

MASTER DD

MASTER is an optional statement that identifies the DFSMSrmm control data set name. If you code the MASTER DD statement to name the DFSMSrmm control data set, you cannot easily change the control data set name by switching to a different parmlib member. You must supply the control data set name in the DFSMSrmm EDGRMMxx parmlib member if you do not code the MASTER DD statement.

OPT

Use OPT on the PROC statement to specify whether to enable or disable subsystem interface:

OPT=RESET to disable the subsystem interface

OPT=MAIN to enable the DFSMSrmm subsystem

PARMLIB DD

PARMLIB is an optional statement. Use the PARMLIB DD statement to identify the data set that contains DFSMSrmm startup parameters when you do not use the system PARMLIB concatenation. If you do not code the PARMLIB DD statement, do not code the IEFORDER DD statement because it is not required. See “Step 19: Starting DFSMSrmm” on page 60 for parameters you can specify with the START command if you specify the PARMLIB DD statement.

If you specify the parmlib data set in the DFSMSrmm procedure, the data set remains allocated while DFSMSrmm is active. If you do not specify the parmlib data set name, DFSMSrmm dynamically allocates PARMLIB by using the concatenated parmlib function of the z/OS system.

Recommendation: Do not specify the PARMLIB or IEFORDER DD statement in the DFRMM procedure. Let DFSMSrmm dynamically allocate the PARMLIB to 'SYS1.PARMLIB' or use the concatenated parmlib support.

REGION

As you determine the REGION size for the DFSMSrmm started procedure, the amount of virtual storage that DFSMSrmm uses depends on the resources you have defined. DFSMSrmm virtual storage usage can be affected by any REGION size controls or restrictions that your systems might have in place such as in IEFUSI. The sample DFRMM procedure specifies REGION=40M, which normally provides all the private region below 16MB and 40MB above 16MB. To enable DFSMSrmm to use all available virtual storage, specify REGION=0M. If you want to set a specific region size, consider these tips along with the current region size of your DFRMM started procedure, to determine if you need to make any changes to the REGION size:

- The VSAM local shared resources (LSR) buffer pool that is built by the DFSMSrmm subsystem for the control data set is obtained above 16 MB.

DFSMSrmm builds an LSR buffer pool for the DFRMM started procedure, and also for the EDGUTIL utility batch address space, which has a predetermined size. The LSR buffer pool is $800 \times \text{data CISZ} + 200 \times \text{index CISZ}$. Assuming 10 240 for the control data set data CISZ and 2 048 for the control data set index CISZ, the value is $800 \times 10\,240 + 200 \times 2\,048 = 8.4$ MB. If you use larger CI sizes, more buffer space is required. For example, if you use a 26K data CISZ, a 21.2 MB buffer size is required.

Recommendation: If the buffer space is larger than 8.6 MB, add the difference to the 40 MB region size that is used by DFSMSrmm and use this value as the REGION size for the DFRMM started procedure.

- DFSMSrmm uses DFSORT during inventory management. Increase the DFSMSrmm REGION size to allow DFSORT to use storage rather than SORTWKxx DASD files. The use of storage rather than DASD files can potentially decrease the time that is needed to run DFSMSrmm inventory management. If you change the REGION size, use 1 additional MB for every 2 000 data sets on private volumes.

If you code the MASTER DD statement and the JOURNAL DD statement in the started procedure, you can switch to different data set names by specifying the names in the EDGRMMxx parmlib member. To make the switch:

1. Quiesce DFSMSrmm by specifying:

```
F DFRMM,QUIESCE
```

DFSMSrmm frees the current file allocations and pauses the DFSMSrmm processing.

2. Specify the new parmlib member name that contains the data set names.
3. Start DFSMSrmm by specifying:

```
F DFRMM,M=xxx
```

If you decide to automate EDGBKUP, EDGLABEL or EDGXPROC, you must update the procedure library to include them as well. See [Automating backup](#) on page 455 for more information on the supplied EDGBETT procedure with BACKUPPROC parameter. See [“Using the LABEL procedure”](#) on page 459 for information on the supplied EDGLABEL procedure. See [“Replenishing scratch volumes in a system-managed library”](#) on page 458 for information on the supplied EDGXPROC procedure with SCRATCHPROC parameter.

Step 9: Assigning DFSMSrmm a RACF user ID

Perform this step once for each z/OS image.

When running on a system with RACF installed, assign DFSMSrmm a RACF user ID by adding a definition to the RACF started procedures table, ICHRIN03, or in the RACF STARTED class. The RACF user ID can be the name of the DFSMSrmm procedure you created in [“Step 8: Updating the procedure library”](#) on page 43 or any installation-selected RACF user ID you specify. As data sets are created for use by the DFSMSrmm procedure, add the RACF user ID to the access list for the data sets. [Table 6 on page 46](#) lists the data sets to which DFSMSrmm requires access.

Table 6. Data sets requiring access by the DFSMSrmm RACF user ID

DDNAMES	Access Required
ACTIVITY	Update
EDGPDOX	Alter
EDGPDOY	Alter
EDGSPLCS	Update
JOURNAL	Update
MASTER	Control
MESSAGE	Update
Parmlib member	Read
REPORT	Update
REPTXT	Update
XREPTXT	Update

If you plan to use the DFSMSrmm automation parameters such as SCRATCHPROC, BACKUPPROC, or LABEL, you must define the associated procedures in ICHRIN03 or the STARTED class. For more information on updating ICHRIN03 or the RACF STARTED class, see [z/OS Security Server RACF System Programmer's Guide](#).

When assigning a RACF user ID to DFSMSrmm, it is recommended to define either the PRIVILEGED(YES) or the TRUSTED(YES) attributes for the STARTED profile. This allows DFSMSrmm to perform its functions without modifying the ACEE control block, and eliminates an ACEE modification message that can occur. Use of TRUSTED instead of PRIVILEGED makes authorization requests for the started task subject to auditing.

The DFSMSrmm procedures EDGXPROC and EDGBKUP require READ access to STGADMIN.EDG.HOUSEKEEP and ALTER access to the data sets specified in the BACKUP and JRNLBKUP DD statements. The LABEL procedure requires UPDATE access to STGADMIN.EDG.OPERATOR. If you are using the EDGRESET utility, you should make sure it has ALTER access to the STGADMIN.EDG.RESET.SSI access list. For additional information on authorization needed for the DFSMSrmm user ID, see [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,”](#) on page 215.

To run DFSMSrmm with DFSMSshm, ABARS, Tivoli Storage Manager, OAM, or any other external data managed tool, you must define their procedure names to RACF with the STARTED class. See [z/OS DFSMSdfp Storage Administration](#) for information about defining the procedure names.

You must define any user ID that requires DFSMSrmm services and makes use of OPERATIONS or privileged attributes to RACF.

If you are using an equivalent security product, review the RACF-related information to determine the changes that might be required to run DFSMSrmm with the equivalent security product.

Step 10: Defining parmlib member EDGRMMxx

Perform this step once for each z/OS image.

Related reading: See “Implementing DFSMSrmm client and server systems” on page 72 for details about setting up DFSMSrmm systems.

Create a parmlib member EDGRMMxx definition for each DFSMSrmm standard system, server system, and client system. The member name is in the form EDGRMMxx, where xx is a two character alphanumeric suffix of your choice. The default is EDGRMM00. A sample parmlib member is available as EDGIVPPM in SAMPLIB.

You can use system symbols to enable easier sharing of the EDGRMMxx parmlib member. See [z/OS MVS Initialization and Tuning Reference](#) for how to use system symbols in parmlib members.

You might need some information in the EDGRMMxx parmlib to be specific to a subset of your systems; for example, the PRTITION, OPENRULE, or VLPOOL entries might need to be different. To enable this information to be handled on a system-by-system basis, the ONLYIF parameter can be used to keep all this in one EDGRMM member, or you can specify a second parmlib member to be used. Use the MEMBER operand on the OPTION command to identify the second parmlib member. Using system symbols for the MEMBER value enables a different second parmlib member to be used on each system. See Chapter 8, [Using the parmlib member EDGRMMxx](#) on page 143 for more details on these parameters.

Recommendation: Specify the PARMLIB member EDGRMMxx as a member of SYS1.PARMLIB or the PARMLIB concatenation. If you follow this recommendation, you can use the DFSMSrmm startup parameters to avoid stopping and restarting DFSMSrmm. For example, you can use the modify command (F DFRMM,M=xx) to implement updates to the data set and avoid stopping and restarting DFSMSrmm. You can also restart DFSMSrmm using another data set with parameters to implement changes.

The RACF ID associated with the DFSMSrmm started procedure requires READ access to the parmlib member EDGRMMxx data set.

Table 7 on page 47 shows several ways to define the parmlib member.

Table 7. Creating DFSMSrmm parmlib definitions

Way to Define	Example
Member of SYS1.PARMLIB	SYS1.PARMLIB(EDGRMMxx)
Member of PARMLIB concatenation	SYSC.PARMLIB(EDGRMMxx)
Member of separate data set	MY.PARMLIB(EDGRMMxx)
Separate sequential data set	USER.PARMLIB

Step 11: Tailoring parmlib member EDGRMMxx

Perform this step once for each z/OS image.

Tailor the options for parmlib member EDGRMMxx. To see the options and parmlib member examples, see Chapter 8, [“Using the parmlib member EDGRMMxx,”](#) on page 143.

Using EDGRMMxx, you can perform these tasks:

- Define system options, such as the date format for reports and messages, and the mode in which DFSMSrmm runs
- Control the use of tape volumes
- Partition tape volumes
- Define pools, such as the range of shelves to use for a pool and whether a pool has RACF tape profile processing
- Tailor mount messages, such as the position of the volume serial number and identifier
- Define security classes for data sets and volumes
- Control the use of bypass label processing for tape volumes
- Define storage locations
- Define controls for running DFSMSrmm vital record processing to apply retention and movement policies
- Define media information for non-IBM media
- Define volume replacement policies.
- Define actions for volumes being purged by callers of EDGTVEXT or EDGDFHSM.
- Define default retention method.
- If the EXPDT retention method is selected as the default, define the default RETAINBY value for volume sets and the default LASTREF extra days for data sets.
- Define the usage of Management Class expiration attributes for data sets.

Step 12: Creating the DFSMSrmm control data set

DFSMSrmm sample provided in SAMPLIB

- DFSMSrmm EDGJMFAL sample JCL for allocating the control data set
- DFSMSrmm EDGJUTIL sample JCL for creating the control data set

Perform this step once for each RMMplex, or for each z/OS image.

Create the DFSMSrmm control data set, a VSAM key-sequenced data set that contains the complete inventory of the removable media library. DFSMSrmm records all changes made to the inventory, such as adding or deleting volumes, in the control data set. The DFSMSrmm control data set can be allocated in the extended addressing space of an extended address volume. The DFSMSrmm CDS is eligible for, and supports, CA Reclaim.

You can create the control data set for each RMMplex or for each z/OS image. If you have a control data set for each z/OS image, each control data set only contains information for that system. DFSMSrmm cannot track tapes that are accidentally moved to another system that has a different control data set.

Recommendation: Create one control data set for each RMMplex. Sharing the control data set across systems allows DFSMSrmm to keep all tape usage information in one place.

Roadmap for creating the control data set

This table shows the subtasks and associated procedures for creating the control data set.

Subtask	Associated procedure
Define the control data set.	“Defining the DFSMSrmm control data set” on page 49
Calculate DASD space for the control data set.	“Calculating DASD space for the DFSMSrmm control data set” on page 49

Subtask	Associated procedure
Place the control data set.	“Placing the DFSMSrmm control data set” on page 50
Allocate space for the control data set.	“Allocating space for the control data set” on page 51
Protect the control data set.	“Protecting the control data set” on page 51
Initialize the control data set for DFSMSrmm subsystem use by running the EDGUTIL utility.	“Initializing the control data set” on page 52
Back up the control data set.	“Planning to back up the control data set” on page 52

Defining the DFSMSrmm control data set

You can define the DFSMSrmm control data set as either an extended format (EF) or a basic format VSAM data set. Using a basic format data set for the DFSMSrmm control data set, limits the control data set size to a maximum of 4 GB. Using an EF data set enables you to use VSAM functions such as multivolume allocation, compression, or striping. EF also enables you to define a control data set that uses VSAM extended addressability (EA) to enable the control data set to grow above 4 GB. To define an EF control data set, you must include the DATACLASS keyword on the AMS DEFINE command and reference the correct data class. Refer to [z/OS DFSMS Using Data Sets](#) for more information on EF data sets, and refer to [z/OS DFSMSdfp Storage Administration](#) for information on defining data classes with DSNTYPE=EXT and EXTENDED ADDRESSABILITY=Y.

Note: Make sure the ACS routines do not select a Management Class that specifies Partial Release.

DFSMSrmm requires CONTROL access to the control data set. The control data set cannot be a SYS1.xx data set if the control data set is to be shared. Additionally, batch LSR cannot be used with the DFSMSrmm control data set.

Note: Refer to this technote for more information on how to define and convert/move from basic format VSAM Control Data Set (CDS) to Extended Format (EF). Refer to [DFSMSrmm: Procedure to define and convert/move from basic format VSAM control data set\(CDS\) to extended format \(EF\)](#) (www.ibm.com/support/pages/node/7165095).

Calculating DASD space for the DFSMSrmm control data set

Table 8 on page 49 helps you to calculate DASD space requirements for the DFSMSrmm control data set. To determine the number of resources in the library, such as the number of software products you have, see your answers to [Appendix C, “Evaluating removable media management needs,”](#) on page 539.

Table 8. DFSMSrmm control data set DASD space requirements

Control Data Set Content	DASD Space
Control record	1 MB (MB equals approximately 1 000 000 bytes)
Data sets	512 KB for every 1000 data sets
Shelf locations in the library that do not contain volumes	140 KB for every 1000 shelf locations
Shelf locations in storage locations	140 KB for every 1000 shelf locations
Owners	38 KB per 1000 volumes
Software products, average five volumes per product	420 KB for every 1000 software products
Volumes	1 MB for every 1000 volumes

Table 8. DFSMSrmm control data set DASD space requirements (continued)

Control Data Set Content	DASD Space
Vital record specifications	212 KB for every 1000 vital record specifications

After calculating the previous figures, increase the total by approximately 50% for free space in the VSAM key-sequenced data set. For example, if the calculated size is 3000 KB, increase it by 50% to 4500 KB to allow for free space. Also consider your expected future growth in numbers of tape volumes and data sets, such as acquiring a new virtual tape solution, and build this expected growth into your size calculations. Use this calculated value in the access method services KILOBYTES parameter on the DEFINE CLUSTER command shown in Figure 12 on page 51. See “Monitoring the space used by the control data set” on page 394 for information about monitoring the DFSMSrmm control data set space usage.

The size of the records in the DFSMSrmm control data set is variable, and record sizes increase as new function is added. Thus, the space required for your DFSMSrmm control data set increases over time. For example, each time a volume record is updated, such as during inventory management or when a new data set is written on a volume, the associated control data set volume records change in size or increase in size as they migrate to the new level dynamically. The value you use for free space enables DFSMSrmm and VSAM to handle the record size changes and allows you the ability to easily add new resources to DFSMSrmm at any time. Because of the way that DFSMSrmm handles variable length records, it is recommended that you do not use FREESPACE(0 0) when defining the DFSMSrmm control data set.

Placing the DFSMSrmm control data set

For each RMMplex, create one control data set on the main, or most active, system in your complex.

If the volume where you place the control data set is system-managed, select a storage class name that has the GUARANTEED SPACE attribute, and substitute it in the example for the STORAGECLASS parameter in Figure 12 on page 51. If it is not system-managed, remove the STORAGECLASS parameter.

If you are running DFSMSrmm on more than one system, decide which systems in an RMMplex will share the control data set using DASD sharing and which will share the control data set using DFSMSrmm client/server support for z/OS. Each standard system and server system needs to share the control data set using DASD sharing. The control data set must be cataloged in a user catalog shared by all the standard systems and server systems in the RMMplex.

Put the control data set on a different volume than the journal, which is also shared. Separating the two data sets optimizes data integrity, since the journal is a copy of changes made to the control data set.

If you plan to use DFSMSdss to back up the control data set, place the control data set on a concurrent copy capable volume.

To avoid a potential deadlock on the volume where the DFSMSrmm control data set is placed, you should consider the other data that you place on the volume. A deadlock can occur when a program other than DFSMSrmm reserves the volume where the control data set resides and requests DFSMSrmm for service. If the DFSMSrmm control data set requires additional extents because of the request, then a deadlock can occur.

DFSMSrmm uses RESERVE/RELEASE to control access to the control data set and ensure integrity. If your installation is using global resource serialization, see “Step 4: Updating installation exits” on page 24 and “Updating GRSRNLxx (optional)” on page 27.

Place the DFSMSrmm control data set and journal on the highest performing DASD in your installation. DFSMSrmm can benefit from features like caching and DASD fastwrite and support for concurrent copy. Consider using the storage class attribute AVAILABILITY=CONTINUOUS for the control data set. This does not remove the need for journaling in DFSMSrmm, however, as the journal is required when reconstructing the control data set from backups.

Allocating space for the control data set

When you allocate the DFSMSrmm control data set index and data components, the index and data components must be on the same volume because DFSMSrmm does not support them being allocated on separate volumes.

Use JCL similar to that shown in [Figure 12 on page 51](#) to allocate space for the control data set on the master system:

```
//IDCAMS EXEC PGM=IDCAMS,REGION=1M
//SYSPRINT DD SYSOUT=*
//DASD DD DISP=SHR,UNIT=SYSDA,VOL=SER=8E5U04
//SYSIN DD *
DEFINE CLUSTER(NAME(RMM.CONTROL.DSET) -
              FILE(DASD) -
              FREESPACE(15 0) -
              KEYS(56 0) -
              REUSE -
              RECSZ(512 9216) -
              SHR(3 3) -
              KILOBYTES(4500 1500) -
              STORAGECLASS(gspace) -
              VOLUMES(8E5U04)) -
      DATA(NAME(RMM.CONTROL.DSET.DATA) -
            BUFFERSPACE(829440) -
            CISZ(26624)) -
      INDEX(NAME(RMM.CONTROL.DSET.INDEX) -
            CISZ(2048))
/*
```

Figure 12. Allocating DASD space for the control data set

Recommendation: Use a CISZ of 26624 bytes and BUFFERSPACE of 829440 bytes for the data component of the control data set to help improve inventory management run times through reduced I/O to the control data set. Any suitable CISZ between 10240 and 26624 that meets your needs can be used.

where:

KILOBYTES

Is the space value you calculated in [“Calculating DASD space for the DFSMSrmm control data set” on page 49](#). Choose a secondary space value that allows the control data set to grow in the future.

NAME()

Specifies the name of the control data set. Use the same name you assigned in the parmlib member EDGRMMxx under OPTION DSNAME(name) or in the MASTER DD statement in the DFSMSrmm procedure.

FREESPACE

Specifies how much free space VSAM reserves in the data set for future additions. Free space is specified for control intervals (CI) to allow for the variable length records used by DFSMSrmm. Specifying a zero value for control area (CA) freespace ensures that CA splits create free space where it is required. For more information on changing the FREESPACE values, see [z/OS DFSMS Using Data Sets](#).

BUFFERSPACE

When the buffer space value is large enough to accommodate a data CA plus one data CI and one index CI, you benefit from improved performance during IDCAMS REPRO.

Protecting the control data set

Protect the control data set by ensuring that a RACF DATASET profile protects it. To prevent inadvertent updates to the control data set, specify a RACF universal access of NONE. Give READ access to users running the inventory management backup function, and give UPDATE access to users running any other backup procedure. Give CONTROL access only to users authorized to perform functions independent of the subsystem, such as restoring and reorganizing the control data set and using program EDGUTIL.

Give CONTROL access to the RACF user ID assigned to the DFSMSrmm procedure in [“Step 9: Assigning DFSMSrmm a RACF user ID”](#) on page 46.

Initializing the control data set

To initialize the control data set, run the DFSMSrmm EDGUTIL utility. [Figure 13 on page 52](#) is sample JCL you can use to create the control data set.

```
//EDGUTIL EXEC PGM=EDGUTIL,PARM='CREATE'  
//SYSPRINT DD SYSOUT=*  
//MASTER DD DSN=RMM.CONTROL.DSET,DISP=SHR  
//SYSIN DD *  
CONTROL CDSID(cds_id) EXTENDED BIN(YES) STACKED VOLUME(YES)  
/*
```

Figure 13. Initializing the control data set

See [“Creating or updating the control data set control record”](#) on page 409 for additional information on SYSIN values you can use.

Planning to back up the control data set

Plan to use the DFSMSrmm utilities EDGBKUP and EDGHSKP to back up the control data set. You can automatically back up the control data set as part of inventory management if you use EDGHSKP which also clears the journal. You can also use the parmlib OPTION command JOURNALFULL operand threshold value to start a backup procedure.

Back up the control data set to DASD. You can also back up the control data set to tape using DFSMSdss. Once the backup is complete, you can move or copy the backup file to any storage medium. DFSMSHsm should not be allowed to process the RMM CDS volume for either migration or for backup purposes as this risk potential lockouts with DFSMSrmm. Therefore care should be taken which Management Class is selected for the control data set.

DFSMSrmm Support for Copy Services: When you use XRC, PPRC, Metro or Global Mirror, or other DASD subsystem facilities for maintaining a copy of the DFSMSrmm CDS, you must plan some minimal recovery actions to ensure that the CDS copy used in recovery situations is a consistent copy of the CDS. A consistent copy ensures that there is no multi-record update in progress and that any transaction to update the CDS has been completed. You should also mirror the DFSMSrmm journal and use this for forward recovery of the mirrored CDS copy to make it consistent.

Alternatively, you can use a DFSMSrmm utility to initiate the creation of a CDS copy at the data set level. EDGHSKP and EDGBKUP provide a COPY option that requests DFSMSrmm use DFSMSdss COPY. By default, DFSMSrmm attempts fast replication, but you can tailor the COPY command used to meet your needs. A CDS copy created in this way is already a consistent copy of the CDS and needs no further processing to enable use by DFSMSrmm, unless you need to forward recover to a specific point in time.

When you use DFSMSrmm forward recovery support in the EDGBKUP utility, you can specify a specific point in time to which the CDS should be recovered, or you can forward recover to the latest consistent set of CDS records. Forward recovery uses journal records from journal backups and the active journal, or can optionally use DFSMSrmm SMF type 42 records.

When you plan to switch to a mirrored copy of the CDS, you must plan to run the EDGBKUP utility with forward recovery using a mirrored copy of the DFSMSrmm JOURNAL data set. This ensures that the CDS copy is forward recovered to the latest consistent set of journal records. This might include records that have been successfully journaled, but not yet updated in the CDS. These journaled records can be used to recover the latest level of the CDS.

Recommendation: Keep the latest backup available on DASD to provide the fastest recovery time for the control data set.

See [“Backing up the control data set”](#) on page 370 for more information.

Step 13: Creating the journal

DFSMSrmm sample provided in SAMPLIB

- EDGJNLAL Sample JCL for Allocating the Journal

Perform Step 13 once for each RMMplex or z/OS image, depending on how you created the control data set. Create one control data set and one journal for each RMMplex.

The journal contains a record of all changes made to the control data set since the last backup. Create the journal and use it to forward recover changes made since the last backup. Each time the control data set is backed up successfully using the EDGHSKP utility, the journal data set is cleared.

DFSMSrmm requires UPDATE access to the journal. The journal cannot be an extended sequential data set.

When not shared with a z/OS release below z/OS V1R12, the DFSMSrmm journal can be placed in an extended addressing space (EAS) on an extended address volume (EAV). See [z/OS DFSMS Using the New Functions](#) for more information on EAS and EAV.

You should plan to back up the journal and maintain multiple generations of it. See [Chapter 13, “Performing inventory management,”](#) on page 327 for additional information.

You can create a large format journal data set by deleting and reallocating the journal data set by specifying DSNTYPE=LARGE in the JCL. A large format data set does not actually have to be more than 65 535 tracks.

If you want to implement a large format journal data set in an existing DFSMSrmm installation, see [Chapter 14, “Maintaining the control data set,”](#) on page 377 for the procedures on reallocating or moving the journal. Also, look at the EDGJNLAL and EDGPBKP samples. They include the DSNTYPE=LARGE keyword.

Roadmap for creating the journal

This table shows the subtasks and associated procedures for creating the journal.

Subtask	Associated procedure
Calculate DASD space for the journal.	“Calculating DASD space for the journal” on page 53
Place the journal.	“Placing the journal” on page 54
Allocate space for the journal.	“Allocating space for the journal” on page 54
Protect the journal.	“Protecting the journal” on page 55
Back up the journal.	“Backing up the journal” on page 55

Calculating DASD space for the journal

[Table 9 on page 53](#) helps you to calculate DASD space requirements for the journal. To determine the number of resources in your library, such as the number of scratch mounts that you have, see your answers to [Appendix C, “Evaluating removable media management needs,”](#) on page 539. Base these calculations on your requirements from one backup to the next. To ensure that the journal has enough space when there is an unexpected increase in tape activity, increase the calculated amount by 50% or more. The journal size can exceed 65 535 tracks.

Table 9. DFSMSrmm journal DASD space requirements

Journal Content	DASD Space
Changes by users	1.5 KB for each change made

Table 9. DFSMSrmm journal DASD space requirements (continued)

Journal Content	DASD Space
Data sets	1.5 KB for each data set retained by a vital record specification
Data sets no longer retained by a vital record specification	1.5 KB for each data set no longer retained by a vital record specification
Expiring volumes	1.5 KB for each expiring volume
Non-scratch mounts	6.7 KB for each mount
Scratch mounts	8.3 KB for each mount
Volumes	1.5 KB for each volume retained by a vital record specification
Volume checked in/out	2.6 KB for each volume in or out of the library
Volumes returned to scratch	3.0 KB for each volume returned to scratch
Volumes to and from storage locations	3.5 KB for each volume moved to or from a storage location
Volumes no longer retained by a vital record specification	1.5 KB for each volume that has not reached its expiration date and is no longer retained by a vital record specification
Volumes that are exported or imported	1.5 KB for each logical volume exported or imported
Vital record specifications	1.3 KB for each vital record specification created

Convert the final figure into a space allocation. The journal has a record format of variable-length blocked records. You do not need to specify the record format information when allocating the journal, because DFSMSrmm sets the correct values when it opens the journal data set.

To calculate the space required, divide the total KB of space by 4, as allocation will be by average record size using a 4 K value. This calculation gives you the number to use in the space allocation (SPACE=) shown in [Figure 14 on page 55](#).

On test and recovery systems where you plan to use the JRNLTRAN(YES) function, increase your estimated journal size by 33% to allow for the extra journal records that are created.

Placing the journal

Place the journal on a different volume than the DFSMSrmm control data set. If the selected volume is system-managed, use the STORCLAS parameter as shown in [Figure 14 on page 55](#), and select a storage class with the GUARANTEED SPACE attribute. By using the GUARANTEED SPACE attribute, you ensure that the journal is allocated to a specific volume. This volume is different than the one on which the control data set resides. Remove the STORCLAS parameter if the volume is not system-managed.

Place the DFSMSrmm journal on the highest performing DASD in your installation. DFSMSrmm can benefit from features like caching and DASD fastwrite and support for concurrent copy. Consider using the storage class attribute AVAILABILITY=CONTINUOUS for the journal.

Systems sharing a control data set must also share the same journal.

Allocating space for the journal

[Figure 14 on page 55](#) shows JCL for allocating space for the journal.

```
//JOURNAL EXEC PGM=IEFBR14
//SYSPRINT DD SYSOUT=*
//JOURNAL DD DISP=(NEW,CATLG),DSN=RMM.JOURNAL.DSET,
//          UNIT=SYSALLDA,VOL=SER=volser,STORCLAS=store_class,
//          DSNTYPE=LARGE,
//          AVGREC=U,SPACE=(4096,(pp))
```

Figure 14. Allocating space for the journal

where:

pp

Specifies the calculated primary space.

DSN=

Specifies the name of the journal. Use the same name you assigned in the parmlib member EDGRMMxx for OPTION JRNLNAME(name) or in the JOURNAL DD statement in the DFSMSrmm procedure.

SPACE=(4096,(pp))

Specifies the space allocation for the journal. Use the value you calculated in [“Calculating DASD space for the journal”](#) on page 53.

You must catalog the journal.

You do not have to specify any DCB information for the data set because the required values are set when DFSMSrmm opens the data set. Any conflicting values that you supply are overridden.

You can create the journal in multiple extents, but increases in size are not permitted once DFSMSrmm uses the data set. If you specify a secondary space allocation for the data set, DFSMSrmm ignores it.

Protecting the journal

Protect the journal by ensuring that a RACF DATASET profile protects it. To prevent inadvertent updates to the journal, specify a RACF universal access of NONE. Give READ access only to users that are authorized to perform functions independent of the subsystem, such as restoring and reorganizing the control data set. Give UPDATE access to the RACF user ID assigned to the DFSMSrmm procedure in [“Step 9: Assigning DFSMSrmm a RACF user ID”](#) on page 46.

Backing up the journal

Plan to use the DFSMSrmm utilities EDGBKUP and EDGHSKP to back up the journal. You can automatically back up the journal as part of inventory management if you use EDGHSKP which also clears the journal. You can also use the parmlib OPTION JOURNALFULL to automatically start a backup procedure. DFSMSrmm uses IDCAMS to back up the journal.

Back up the journal to DASD. Once the backup is complete, you can move or copy the backup file to any storage medium. Be sure to maintain multiple generations of the journal.

Step 14: Authorizing users

Perform this step once for each RMMplex.

Refer to Chapter 9, [“Authorizing DFSMSrmm users and ensuring security,”](#) on page 215 to authorize your users to DFSMSrmm resources.

Step 15: Making the DFSMSrmm ISPF dialog available to users

Modify the ISPF environment so you can run the DFSMSrmm ISPF dialog. Use one or more of these methods to make the dialog available:

- Add DFSMSrmm to an ISPF dialog as described in [“Adding DFSMSrmm to an ISPF selection panel”](#) on page 56.

- Use the method supplied by DFSMS. DFSMSrmm is selection option 'R' from the ISMF primary option menu.
- Use the RMMISPF EXEC to enter the dialog.
- Enable ISPF Data Set List access to the ISPF dialog. See [“Enabling ISPF data set list \(DSLIST\) support” on page 58](#) for additional information.
- Use the EDGRPD34 exec or its aliases RMMI and TI from the ISMF data set list or ISPF data set list. See [“Enabling ISPF data set list \(DSLIST\) support” on page 58](#) for additional information.

To implement those methods, use one of these techniques:

- Concatenate the DFSMSrmm target ISPF libraries with your existing ISPF libraries.
- Use the ISPF LIBDEF facility to make the DFSMSrmm target ISPF libraries available to your users, and use the EDGRMLIB EXEC to enter the dialog.

If you are using the LIBDEF facility and you are not using the same target library names as listed in the *z/OS Program Directory* in the *z/OS Internet library* (www.ibm.com/servers/resourcelink/svc00100.nsf/pages/zosInternetLibrary) and *ServerPac: Installing Your Order*, you must modify the EDGRMLIB EXEC or produce your own similar exec or CLIST.

Adding DFSMSrmm to an ISPF selection panel

You can add a selection to an ISPF selection panel so that users can choose DFSMSrmm. To add the selection:

1. Add a selection for DFSMSrmm to the body of the chosen ISPF selection panel. For example, add:

```
R   DFSMSrmm      Invoke DFSMSrmm
```

2. Add one of these statements to the ZSEL processing list in the)PROC section of the chosen ISPF panel:

- If you are not using LIBDEF:

```
R, 'CMD(%RMMISPF) NEWAPPL(EDG) '
```

- If you are using LIBDEF:

```
R, 'CMD(%EDGRMLIB) '
```

[Figure 15 on page 57](#) shows the ISPF Utility Selection Menu with step 1 and 2 changes made. LIBDEF was not used:


```

%----- UTILITY SELECTION MENU -----
%OPTION  ==>_ZCMD
%
% 1 +LIBRARY   - Compress or print data set.  Print index listing.
+             Print, rename, delete, or browse members
% 2 +DATASET   - Allocate, rename, delete, catalog, uncatalog, or
+             display information of an entire data set
% 3 +MOVE/COPY - Move, copy, or promote members or data sets
% 4 +DSLIST    - Print or display (to process) list of data set names
+             Print or display VTOC information
% 5 +RESET     - Reset statistics for members of ISPF library
% 6 +HARDCOPY  - Initiate hardcopy output
% 8 +OUTLIST   - Display, delete, or print held job output
% 9 +COMMANDS  - Create/change an application command table
% 10 +CONVERT  - Convert old format menus/messages to new format
% 11 +FORMAT   - Format definition for formatted data Edit/Browse
% 12 +SUPERCE - Compare data sets (Standard dialog)
% 13 +SUPERCE - Compare data sets (Extended dialog)
% 14 +SEARCH-FOR - Search data sets for strings of data
% R +DFSMSrmm - Invoke DFSMSrmm
)INIT
  .HELP = ISR30000
)PROC
  &ZSEL = TRANS( TRUNC (&ZCMD, '.')
                1, 'PGM(ISRUDA) PARM(ISRUDA1)'
                2, 'PGM(ISRUDA) PARM(ISRUDA2)'
                3, 'PGM(ISRUMC)'
                4, 'PGM(ISRUDL) PARM(ISRUDLP)'
                5, 'PGM(ISRURS)'
                6, 'PGM(ISRUHC)'
                8, 'PGM(ISRUOLP)'
                9, 'PANEL(ISPUCMA)'
                10, 'PGM(ISRQCM) PARM(ISRQCMP)'
                11, 'PGM(ISRFMT)'
                12, 'PGM(ISRSSM)'
                13, 'PGM(ISRSEPRM) NOCHECK'
                14, 'PGM(ISRSFM)'
                R, 'CMD(%RMMISPF) NEWAPPL(EDG)'
                , , ,
                *, '?' )
  &ZTRAIL = .TRAIL
)END

```

Figure 15. Adding DFSMSrmm to ISPF

Modifying an ISPF selection panel

You can modify the selection so that only the USER option of the DFSMSrmm dialog is available to the majority of end users.

- If you are not using LIBDEF, add to the)PROC section:

```
R, 'CMD(%RMMISPF USER) NEWAPPL(EDG)'
```

- If you are using LIBDEF, modify the supplied EDGRMLIB EXEC to include the USER parameter on the RMMISPF EXEC call.

For example, enter this REXX statement to replace the one supplied in EDGRMLIB.

```
address "ISPEXEC" "SELECT CMD(%RMMISPF USER) NEWAPPL(EDG) PASSLIB"
```

Make the DFSMSrmm panel library, tables, skeletons, messages, and REXX execs available to users. If you are not using LIBDEF, [Table 10 on page 58](#) shows the names of the default libraries that you concatenate to the DD statements in the TSO logon procedure or a user-supplied start-up CLIST. If you are using LIBDEF, the EDGRMLIB EXEC allocates these default libraries to the user. If you changed the names of the target libraries on your system, modify the EDGRMLIB exec to contain the new library names.

[Table 10 on page 58](#) lists the DFSMSrmm libraries by the default target names.

Table 10. Default libraries to concatenate

DFSMSrmm Data Set Name	DD Statement	Content
SYS1.SEDGEXE1	SYSPROC (or SYSEXEC)	REXX execs
SYS1.SEDGMENU	ISPMLIB	English messages
SYS1.SEDGPENU	ISPLIB	English panels
SYS1.DGTSLIB	ISPSLIB	Skeletons
SYS1.DGTTLIB	ISPTLIB	Tables

The target library SYS1.SEDGEXE1 has a fixed-block record format. The SYS1.DGTSLIB library and the SYS1.DGTTLIB library are shared with other DFSMS components.

To customize the DFSMSrmm Report Generator library names, you can modify the EDGRMAIN member of SYS1.SEDGEXE1. Refer to [z/OS DFSMSrmm Reporting](#) for details.

Use the ISPF ISPPREP facility to build preprocessed versions of the panels. For more information on using ISPPREP, see [z/OS ISPF User's Guide Vol I](#).

Enabling ISPF data set list (DSLIST) support

To enable direct entry into the DFSMSrmm ISPF dialog from the ISPF Data Set List Utility, use the ISPF Configuration Utility to update the ISPF Configuration Table. To enable this function, select the Enable RM/Tape Commands option. For details of how to use the ISPF Configuration Utility, refer to [z/OS ISPF Planning and Customizing](#). Figure 16 on page 58 shows that the data set list support is enabled, as well as showing the default values for the RM/Tape Command EDGRPD34 and Command APPLID EDG. You do not need to change these values.

```
ISPPMOD3          Modify PDF Configuration Settings
Command ===>

                                More:  -

When to use COPY or COPYMOD
2  1. Use COPY if the target block size is equal to or greater than the
    source block size, COPYMOD otherwise
    2. Use COPY if the target block size is equal to the source block size,
    COPYMOD otherwise
    3. Always use COPYMOD

DSLIST Removable Media Settings
Enter "/" to select option
/  Enable RM/Tape Commands

RM/Tape Command  . . %EDGRPD34
Command APPLID   . . . EDG

Other PDF Settings
Default PDF Unit . . . . . SYSALLDA
Volume for Migrated Data Sets . . . . . MIGRAT
Delete Command for Migrated Data Sets  HDELETE
Allowed Allocation Units . . . . . ANY
Maximum IEBCOPY Return Code . . . . . 0
```

Figure 16. Enabling ISPF data set list (DSLIST) support

The line commands supported by DFSMSrmm are I, S, M, and D.

- I - Displays a search results list showing all data sets in the multivolume set for the selected data set.
- S - Displays the individual data set details. DFSMSrmm determines the first file on the selected volume that matches the selected data set. If other data sets of the same name exist on the volume, the wrong details could be displayed. In that case, use the M line command and then the DFSMSrmm I line command from that results list.
- M - Displays a search results list showing all data sets defined to DFSMSrmm that match the selected data set name.

- D - Releases the volume. If the volume is part of a multivolume set, there is the option to release all volumes in the set.

There is additional capability provided in the EDGRMAIN exec to customize how the ISPF Data Set List line commands are handled by DFSMSrmm. You can apply any of the DFSMSrmm results to any of the line commands, as well as these options:

- Display the volume details for the selected entry.
- Display the search results for all the volumes in the same set as the selected entry.
- Display a search results list showing all data sets on the same volume as the selected data set.

Once you see the DFSMSrmm results, you are in the DFSMSrmm ISPF dialog and can use any of the available functions including fast path commands.

You can use the DFSMSrmm support either by enabling the support and using the DSLIST line commands, or you can use the EDGRPD34 exec as a command directly in the DSLIST results or in the ISMF data set and mountable tape volumes lists results. For example:

```
EDGRPD34 I
```

This command executes the DSLIST I line command. You can use one of the exec aliases (RMMI or TI), or you can rename it to any value you wish. However you choose to run the command outside of the ISPF DSLIST built-in capability, the exec expects that optionally the first parameter might be a line command, such as I, S, M or D. If you do not specify an optional line command:

- When EDGRPD34 is used, the default line command is "I".
- When any other exec or alias name is used, the exec or alias name is used as the line command.

You can also customize the EDGRPD34 exec to handle your selected ISPF environment. If you plan to use ISPF LIBDEF, you must edit EDGRPD34 to use EDGRMLIB instead of EDGRMAIN to enter the DFSMSrmm dialog. Update this line in the exec:

```
UseIsplibdef = false           /* <<   true or false           */
```

When you use ISPF LIBDEF, whether for normal entry to the DFSMSrmm dialog or from the ISPF DSLIST utility, you must copy, or make available, the EDGRMLIB and the EDGRPD34 execs in an exec library normally available to ISPF users.

Step 16: Restarting z/OS with DFSMSrmm implemented

You are ready to start the system with DFSMSrmm implemented. You cannot restart z/OS without an IPL, however an IPL can usually be avoided. For example, you can avoid an IPL when performing one of these tasks:

- Changing subsystem name information in IEFSSNxx
- Setting SMF information in SMFPRMxx
- Changing GRS RNL definitions in GRSRNLxx
- Using the PARMLIB UPDATE command to implement changes to IKJTSOxx

If you performed an IPL during Step 1, described in “Step 1: Preparing to implement DFSMSrmm” on page 23, you have to re-IPL only if you cannot dynamically implement changes to the z/OS system parmlib members or modified installation exits. You must re-IPL with CLPA to include new levels of DFSMSrmm code that have LPALIB as the target library.

Step 17: Tailoring DFSMSrmm set up

At this time, you must perform some additional set up for some of the DFSMSrmm functions. See [z/OS Upgrade Workflow](#) for information about steps that you need to perform prior to installing DFSMSrmm.

See these sections for more information:

- Using volumes with special expiration dates, see [“Managing volumes with special dates”](#) on page 91.
- Using volumes with duplicate volume serial numbers, see [“Managing volumes with duplicate volume serial numbers”](#) on page 95.
- Using DFSMSrmm to control tape label types that can be used on volumes, see [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,”](#) on page 215.
- Enabling the use of no label output for scratch volumes with JES3, see [Chapter 19, “Running DFSMSrmm with JES3,”](#) on page 473.
- Synchronizing the DFSMSrmm control data set and other user catalogs, see [“Running DFSMSrmm catalog synchronization”](#) on page 365.
- Enabling DFSMSrmm stacked volume support, see [“Setting up DFSMSrmm stacked volume support”](#) on page 415.
- Managing volumes using ACS routines, see [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82.
- Enabling UTC or common time support (also known as GMT), see [“Setting up DFSMSrmm common time support”](#) on page 416.

Step 18: Updating the workload management service definition for DFSMSrmm

At this time, you should make sure that the Workload Management classification of the DFSMSrmm started procedure is appropriate. IBM recommends that you classify the DFSMSrmm started procedure into service class SYSSTC, or into a single period service class with an appropriate velocity goal. The importance level should be above or the same as the work that DFSMSrmm serves.

In addition, the Workload Management classification of any other started procedure, such as EDGXPROC and EDGBKUP, should be such that it minimizes the risk of delaying the DFSMSrmm processing. IBM recommends that you classify any DFSMSrmm started procedures into service class SYSSTC, or into a single period service class with an appropriate velocity goal and an importance level that is the same as the DFSMSrmm started procedure.

Step 19: Starting DFSMSrmm

Set the OPMODE operand in parmlib member EDGRMMxx to M to start DFSMSrmm in manual mode. Tape mounts are processed as they were before you began implementing DFSMSrmm. Refer to [“Defining system options: OPTION”](#) on page 161 for information about setting OPMODE. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information about operator procedures.

Then, issue the z/OS START command to start the DFSMSrmm subsystem, as shown in [Figure 17](#) on page 60:

```
S DFRMM,M=xx,SUB=jesp
```

Figure 17. Starting DFSMSrmm

where:

DFRMM

Specifies the procedure name.

M=xx

Specifies your chosen parmlib member name suffix.

SUB=jesp

This is an optional keyword to identify the name of the job entry subsystem. DFSMSrmm must not run under the MSTR subsystem. If the DFSMSrmm procedure name you use matches the subsystem name in IEFSSNxx as described in [“Updating IEFSSNxx”](#) on page 25, you must specify the SUB keyword when starting the DFSMSrmm procedure. The value *jesp* is the procedure name of your job entry subsystem and can be specified as SUB=JES3 or SUB=JES2.

If your DFSMSrmm procedure has a PARMLIB DD statement with DDNAME=IEFRDER, there are several other parameters that you can specify with the START command. You can specify keyword parameters that are supported on a DD statement. For example, you can specify DSN= to override the control data set name on the IEFRDER statement, as shown in [Figure 18 on page 61](#):

```
S DFRMM.name,M=xx,DSN=parmlib_name
```

Figure 18. Starting DFSMSrmm with additional parameters

where:

name

Specifies a name other than DFRMM by which you can call the DFRMM procedure. You can then use this name on STOP and MODIFY commands.

M=xx

Specifies a specific parmlib member with which DFSMSrmm should be started instead of the default parmlib member.

DSN=parmlib_name

Specifies an alternative data set name to be used as a parameter for this restart of the DFSMSrmm subsystem.

DFSMSrmm is now running in manual mode.

Stopping DFSMSrmm

You can stop the DFSMSrmm subsystem with the z/OS STOP command, as shown in [Figure 19 on page 61](#):

```
P DFRMM
```

Figure 19. Stopping DFSMSrmm

If the operator defers the reply to the WTORs, DFSMSrmm shutdown might wait until the operator enters a reply. This is also true if active or new tasks are in hold. If shutdown is delayed, DFSMSrmm issues message EDG0154I to notify the operator that action is required. If the STOP command is rejected, message EDG1108E is issued.

When DFSMSrmm is stopped, DFSMSrmm completes these tasks:

- DFSMSrmm completes any requests being processed at the time DFSMSrmm is stopped.
- DFSMSrmm completes any requests accepted by DFSMSrmm but that are currently waiting to be processed, except for requests to start DFSMSrmm inventory management. DFSMSrmm fails requests to start DFSMSrmm inventory management.
- If DFSMSrmm is already quiesced, DFSMSrmm does not process the waiting requests. DFSMSrmm issues message EDG1105I indicating that the requests are still waiting to be processed. If any of the requests are for catalog activity notifications, DFSMSrmm provides additional message text in message EDG1106I. DFSMSrmm issues a WTOR EDG1107D with options to STOP, QUIESCE, RESTART DFSMSrmm with the same parmlib member or RESTART DFSMSrmm with a new parmlib member.
- DFSMSrmm fails new requests with the RMM not active reason.

To stop DFSMSrmm and still allow tapes to be used, issue the commands shown in [Figure 20 on page 61](#) to disable the DFSMSrmm subsystem interface until the next time you IPL or start the DFSMSrmm subsystem.

```
S DFRMM,OPT=RESET  
P DFRMM
```

Figure 20. Disabling the DFSMSrmm subsystem interface

Before DFSMSrmm disables the subsystem interface, it ensures that the user who made the request is authorized. To disable the DFSMSrmm subsystem interface by using the RESET option, you must have a security profile in place. In RACF, a user desiring to use the RESET option needs ALTER access on the STGADMIN.EDG.RESET profile. If your installation does not have RACF or an equivalent security product installed, DFSMSrmm allows the reset request. In order to control the request, you can write a RACROUTE exit to test for the security profile and return an acceptable return code. See Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215 for additional information.

Tell the operator to reply to any DFSMSrmm WTORs issued between issuing the commands shown in Figure 20 on page 61. The operator must reply to any DFSMSrmm WTORs before entering the STOP command. If the RESET option is used to allow tape usage to continue, the operator should first enter the command:

```
S DFRMM,OPT=RESET
```

The operator should reply to the outstanding WTORs and then enter the STOP command. If the operator defers the reply to the WTORs, DFSMSrmm shutdown might hang until the operator enters a reply. If shutdown is delayed, DFSMSrmm issues message EDG0154I to notify the operator that action is required.

Quiescing DFSMSrmm

When you quiesce DFSMSrmm:

- DFSMSrmm completes requests that are being processed when the quiesce is requested. Note that DFSMSrmm might fail the request if manual recovery is to be performed or if there are I/O errors on the control data set.
- Any requests accepted by DFSMSrmm are left on work queues until DFSMSrmm recovery and refresh is completed.
- DFSMSrmm fails any new requests with an I/O error during manual recovery or RMM not ACTIVE when DFSMSrmm is quiesced by command.
- In a multihost environment, conditions, which result in an automatic quiesce of DFSMSrmm (such as control data set errors from which DFSMSrmm cannot automatically recover), cause the quiesce on all hosts sharing the control data set. Only after all hosts have successfully quiesced can the control data set be recovered. Manually issuing a DFSMSrmm quiesce affects only the host on which you issue the command. If you want all hosts quiesced, you must issue the command on each host that is sharing the control data set.

To quiesce DFSMSrmm, the operator should issue the command to quiesce the subsystem, allow DFSMSrmm to deallocate the control data set and the journal, and allow recovery processing to be performed.

```
MODIFY DFRMM,QUIESCE
```

Figure 21. Quiescing the DFSMSrmm subsystem interface

Restarting DFSMSrmm

To restart DFSMSrmm after it has been quiesced or to change the DFSMSrmm options, the operator can issue the command specified with a member name.

```
MODIFY DFRMM,M=xx
```

Checking DFSMSrmm status

Your operator can obtain DFSMSrmm status information by using any of the following methods:

- F DFRMM,QUERY ACTIVE operator command from an operator console at any time to display the status of DFSMSrmm, its local tasks, and request queues.

The operator can also use the abbreviations of QUERY and ACTIVE: F DFRMM,Q ACT or F DFRMM,Q A.

- TSO subcommand RMM LISTCONTROL STATUS from TSO or with any of the DFSMSrmm APIs when DFSMSrmm is active and able to process subsystem requests. v
- CONTROL STATUS ISPF dialog (fast path command, Selection 4.8.12, or Selection 5.7.12 from a TSO ISPF session to interactively display status and manage active tasks when DFSMSrmm is active and able to process subsystem requests

Status information that is available includes:

- The number of requests waiting to be processed.
- The number of local requests and server requests.
- If DFSMSrmm is active or quiesced.
- If the journal is enabled, disabled, or locked.
- The status of the listener task, either active or inactive.
- The list of active tasks including function, originated system, job name, status, token, and started time.
- The number of current active tasks, the number of current active tasks in hold and whether new tasks are in hold.
- The DEBUG status and PDA trace level .

Step 20: Defining resources

This topic helps you define these resources to DFSMSrmm: shelves, volumes, owners, and vital record specifications. For detailed information about how to perform these tasks, see *z/OS DFSMSrmm Managing and Using Removable Media*. If you are migrating to DFSMSrmm, refer to the IBM Redbook *DFSMSrmm Primer*, SG24-5983.

Defining shelf locations

Shelf locations in the removable media library are called rack numbers. Shelf locations in storage locations are called bin numbers. To define shelf locations to DFSMSrmm, use RMM ADDRACK or ADDBIN subcommands or the DFSMSrmm ISPF dialog.

You can optionally define a rack number for every volume that you plan to define to DFSMSrmm. You do not have to define all possible shelf locations now because you can add them at any time. If you want to logically divide the library into pools, see [“Organizing the library by pools” on page 75](#).

If volumes are designated for use in an automated tape library, you must define rack numbers for the volumes when the external and internal volume serial numbers are not the same.

You can optionally define a bin number for every shelf location in a storage location. For shelf locations in DFSMSrmm built-in storage locations, LOCAL, DISTANT, and REMOTE, you provide an initial count and DFSMSrmm assigns the bin numbers.

You can use the LOCDEF command in the EDGRMMxx member of parmlib to define additional locations to DFSMSrmm and to further define existing locations. See [“Defining storage locations: LOCDEF” on page 143](#) and [Chapter 7, “Managing storage locations,” on page 133](#) for additional information. For shelf locations in installation defined storage locations, provide a count and an initial bin number.

Defining owner information to DFSMSrmm

Use the RMM ADDOWNER subcommand or the Add Owner Details panel in the DFSMSrmm ISPF dialog to define owner information to DFSMSrmm. You must define owner information to DFSMSrmm before defining owner information for non-scratch volumes, software products, and vital record specifications.

If you plan to define volumes using the RMM ADDVOLUME subcommand and want to assign volume ownership, you must predefine those owners to DFSMSrmm before you add the volumes.

You must also define owners before you use the RMM ADDVRS subcommand or the DFSMSrmm Add Data Set VRS, DFSMSrmm Add Name VRS, or DFSMSrmm Add Volume VRS panel in the DFSMSrmm ISPF dialog.

DFSMSrmm automatically adds owner information to the control data set when volumes are read or data is written to volumes. DFSMSrmm uses the RACF user ID for a job, when available, as the volume owner. When there is no RACF user ID for a job, DFSMSrmm uses the job name as the volume owner ID.

To use DFSMSrmm automatic owner notification, you should include the owner's electronic user ID and node, or owner's e-mail address, when you define an owner to DFSMSrmm. For example, if you want DFSMSrmm to automatically notify owners when their volumes become eligible for release, DFSMSrmm can send the release notification to the owner's electronic address, an z/OS or VM user ID and node, or an Internet e-mail address, that you have defined.

Defining volumes

Use the RMM ADDVOLUME subcommand or the DFSMSrmm ISPF dialog to add physical volumes, logical volumes, and stacked volumes to DFSMSrmm. The volume serial number and volume status are the basic information needed for DFSMSrmm to recognize a volume. You can add more information using the subcommand or dialog or use the DFSMSrmm running modes to record information about volumes as they are used.

You can add information about volumes that reside in an automated tape library before they are entered into the automated tape library using the RMM ADDVOLUME subcommand or the DFSMSrmm ISPF dialog. For more information about adding volumes to a system-managed tape library and using DFSMSrmm with system-managed tape libraries, see [Chapter 5, “Running DFSMSrmm with system-managed tape libraries,” on page 99](#).

When you add a volume to DFSMSrmm, DFSMSrmm automatically assigns a retention method to the volume. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for details about how retention methods are assigned.

You set running modes with the OPMODE operand in the parmlib member EDGRMMxx. See [“Defining system options: OPTION” on page 161](#) for information about the DFSMSrmm running modes. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for details about adding volume information to DFSMSrmm.

Adding volumes for a new removable media library

If you have a new removable media library or system with all scratch tapes, define all volume information to DFSMSrmm at once.

1. Use the RMM ADDVOLUME subcommand or the DFSMSrmm ISPF dialog to define volume information to DFSMSrmm. [Figure 22 on page 64](#) shows the minimum information you must add for each volume: volume serial number and volume status.

```
RMM ADDVOLUME volser STATUS(SCRATCH)
```

Figure 22. Defining minimum volume information

To add volumes that are in a system-managed library, use the command as shown in [Figure 23 on page 64](#). You specify the STATUS(VOLCAT) to obtain volume information from the TCDB.

```
RMM ADDVOLUME volser STATUS(VOLCAT)
```

Figure 23. Defining volumes in a system-managed library

To add volumes that are in a manual tape library, use the command as shown in [Figure 24 on page 64](#). You must specify the MEDIATYPE operand and the RECORDINGFORMAT operand.

```
RMM ADDVOLUME volser LOCATION(mtlname) MEDIATYPE(HPCT) -  
RECORDINGFORMAT(128TRACK) STATUS(SCRATCH)
```

Figure 24. Defining volumes in a manual tape library

When you are adding a volume to DFSMSrmm, you can specify whether the volume is a logical volume, a physical volume, or a stacked volume. The default volume type is physical volume. If a volume resides in a virtual tape server (VTS), the default is either a logical volume or a stacked volume. If you want DFSMSrmm to automatically manage the movement of stacked volumes, you must enable stacked volume support as described in [“Setting up DFSMSrmm stacked volume support” on page 415](#).

If you do not specify the RETENTIONMETHOD operand on ADDVOLUME, then DFSMSrmm uses the parmlib RETENTIONMETHOD default.

When you are adding a volume that is managed by the EXPDT retention method and you do not specify the RETAINBY operand, then DFSMSrmm uses the parmlib default for the EXPDT retention method RETAINBY value.

You can issue the subcommands in batch.

2. Set the DFSMSrmm running mode in the parmlib member EDGRMMxx to record only mode, warning mode, or protect mode. DFSMSrmm records information about the volumes as they are used. When DFSMSrmm is running in warning mode, DFSMSrmm validates magnetic tapes volumes as you use them. If DFSMSrmm discovers errors, it issues error messages instead of rejecting tapes. When DFSMSrmm is running in protect mode, DFSMSrmm validates magnetic tapes volumes as you use them and rejects magnetic tape volume mounts when errors are encountered. Protect mode is the only mode that provides full verification and validation of volumes.

Adding volumes from an existing removable media library

If you have an existing removable media library with no record of tape usage, define basic volume information to DFSMSrmm. Set the DFSMSrmm running mode to record-only mode. DFSMSrmm monitors all tape volume mounts and automatically records and updates information about your defined tape volumes when they are used. DFSMSrmm cannot automatically record optical disk information.

1. Define all required tape volumes to DFSMSrmm as private volumes using the RMM ADDVOLUME subcommand. This example shows the minimum information you should add:

```
RMM ADDVOLUME volser STATUS(USER) EXPDT(yyyy/ddd)
```

DFSMSrmm uses the parmlib RETPD default if you do not use EXPDT or RETPD to define an expiration date for the volumes. Specifying the EXPDT or RETPD operand allows you to define a time period long enough to gather information about the volumes under DFSMSrmm. Use the information to determine if the volume should be released. Use EXPDT(99365) if you are unsure how long volumes should be retained. Later, after running DFSMSrmm, defining DFSMSrmm vital record specifications, you should use the information to determine if the volume should be released.

If you do not specify the RETENTIONMETHOD operand, then DFSMSrmm uses the parmlib RETENTIONMETHOD default. When you are adding a volume that is managed by the EXPDT retention method and you do not specify the RETAINBY operand, then DFSMSrmm uses the parmlib default for the RETAINBY value.

Specifying STATUS(USER) ensures that DFSMSrmm does not change the expiration date when the first file on the volume is rewritten.

2. Set the DFSMSrmm running mode to record-only mode. DFSMSrmm records information about the volumes as they are used but does not validate or reject volumes.
3. When you believe that DFSMSrmm has recorded enough information, set the DFSMSrmm running mode to warning mode. When DFSMSrmm is running in warning mode, DFSMSrmm validates tape volumes. If DFSMSrmm discovers errors, it issues error messages but does not reject tape usage.
4. After DFSMSrmm has been running for a while, check volume usage to determine if some of the volumes you added can be released.

Adding known volumes

See the IBM Redbook *DFSMSrmm Primer*, SG24-5983, for information about converting from other products to DFSMSrmm.

1. When you have existing information about your media library, you can build a set of RMM TSO subcommands, one for each volume, and define the information you know. You can also write a REXX exec, CLIST or procedure that converts volume information from your existing tape management system into DFSMSrmm subcommand requests.
2. After adding the information you have available, set the DFSMSrmm running mode to protect mode. Protect mode is the only mode that provides full verification and validation of volumes.

Defining vital record specifications

Vital record specifications are retention and movement policies for volumes with retention method VRSEL and the data sets on these volumes that are not excluded from VRSEL processing. Define vital record specifications to use DFSMSrmm retention and movement management. The only requirement is that you define them before running your first expiration processing, which you should run once a day.

To define vital record specifications, use the RMM ADDVRS subcommand or the DFSMSrmm ISPF dialog. For more information about ADDVRS and the dialog, see *z/OS DFSMSrmm Managing and Using Removable Media*. See “Managing volumes with special dates” on page 91 for information about defining vital record specifications that use special expiration dates for volumes using the EDG_EXIT100 installation exit.

To implement the retention and movement policies you define, you must run DFSMSrmm inventory management vital record processing as described in “Running vital record processing” on page 345. You can control the way vital record processing runs using the DFSMSrmm parmlib member EDGRMMxx OPTION command as described in Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143.

Step 21: Updating the operational procedures

Ensure that operators understand how to use DFSMSrmm by documenting changes in your operational procedures. *z/OS DFSMSrmm Managing and Using Removable Media* has an operator procedures topic that you should use to educate operators and update your procedures.

Step 22: Initializing the DFSMSrmm subsystem and tape recording

Perform this step once for each z/OS image to initialize the DFSMSrmm subsystem interface to enable tape usage.

Enabling the DFSMSrmm subsystem interface

Enable the DFSMSrmm subsystem interface to ensure that the interface starts every time you IPL and to ensure that users cannot use tapes before the DFSMSrmm subsystem starts. To enable DFSMSrmm, change the IEFSSNxx member of SYS1.PARMLIB. Add EDGSSSI, as the DFSMSrmm subsystem initialization program, as shown in [Figure 25 on page 66](#).

Note: The DFSMSrmm subsystem can be defined to allow the SSI initialization routine for DFSMSrmm to be invoked in parallel with other SSI initialization routines. See *z/OS MVS Initialization and Tuning Guide* for more information.

```
SUBSYS SUBNAME(JES2)          /* JES2 PRIMARY SUBSYSTEM START */
  PRIMARY(YES) START(YES)
SUBSYS SUBNAME(DFRM)          /* Name of the DFSMSrmm subsystem */
  INITRTN(EDGSSSI)            /* RMM initialization routine */
SUBSYS SUBNAME(AOPA)          /* Netview */
```

Figure 25. Changing SYS1.PARMLIB IEFSSNxx

Figure 25 on page 66 shows the correct relative position of the DFSMSrmm subsystem, updated to include the initialization program.

where:

DFRM

Specifies the subsystem name, which was chosen in step 5. Any unique subsystem name, one to four characters long can be used.

Although the subsystem name can be one to four characters long, the subsystem name must be four characters and must match the first four characters of the procedure name under this condition:

- You use the EDGRESET utility or the following START command and plan to restart DFSMSrmm without an IPL.

S DFRMM,OPT=RESET

Changing the DFSMSrmm running mode

The DFSMSrmm running modes are set with the parmlib OPTION command OPMODE operand. Each DFSMSrmm running mode provides different levels of information recording and volume validation. If you want to record information about volumes, set the running mode to record-only by setting the OPMODE in the parmlib member EDGRMMxx to R and starting or restarting DFSMSrmm.

Set the running mode to warning mode if you want DFSMSrmm to validate tape volumes. DFSMSrmm issues error messages but does not reject tape usage. If you want to ensure that DFSMSrmm provides full validation and recording functions, set the running mode to protect mode.

After you have verified that DFSMSrmm is providing the required functions, change the DFSMSrmm running mode to protect mode. Protect mode is the only mode that provides full verification and validation of volumes.

If you are converting to DFSMSrmm from another media management system, this step would be part of the final conversion phase. In the final phase, you might be reconverting some data set and volume information. Changing to protect mode ensures that DFSMSrmm controls the management of the volumes defined to it, and controls the use of scratch tapes. When changing to protect mode, either stop running the current media management system or ensure that the two products do not overlap in their function.

Activating the tape volume interface

To enable the DFSMSShsm tape volume interface to be used by other similar products, follow the directions in “Releasing tapes: EDGTVEXT” on page 249. Also, see the TVEXTPURGE parmlib option in Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143 for how to select the correct option for the tape volume interface.

Restarting the DFSMSrmm subsystem

If you changed the DFSMSrmm running mode as described in “Changing the DFSMSrmm running mode” on page 67 and need to activate the DFSMSrmm subsystem interface to implement the change, use the operator MODIFY command shown in [Figure 26 on page 67](#):

```
F DFRMM,M=xx
```

Figure 26. Restarting the DFSMSrmm subsystem

where:

M=xx

Specifies the suffix of parmlib member EDGRMMxx.

The subsystem temporarily stops and reinitializes itself with the new OPMODE. Use this command whenever you update DFSMSrmm parameters and want to implement the change.

During this restart, DFSMSrmm issues the message shown in Figure 27 on page 68, unless you have IPLed the system since adding EDGSSSI to IEFSSNxx in SYS1.PARMLIB, in which case the interface is already initialized.

```
EDG0103D DFSMSrmm SUBSYSTEM INTERFACE IS INACTIVE -  
ENTER "IGNORE", "CANCEL" OR "RETRY"
```

Figure 27. Message EDG0103D

Reply **RETRY** to this message. In response, DFSMSrmm initializes its subsystem interface. If you reply IGNORE, the DFSMSrmm tape recording function is not activated.

Step 23: Setting up DFSMSrmm utilities

There are several DFSMSrmm utilities that you should now set up to run. Run the utilities on the system with the highest level of code to ensure you are taking advantage of DFSMSrmm enhancements and changes to the control data set.

Related reading:

- See [“Scheduling DFSMSrmm utilities”](#) on page 327 for a sample schedule of all DFSMSrmm utilities. See Chapter 20, [“Running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS,”](#) on page 477 for information about setting up the IBM Tivoli Workload Scheduler for z/OS to manage the scheduling of DFSMSrmm functions.
- See Chapter 13, [“Performing inventory management,”](#) on page 327 for information about EDGHSKP to run inventory management activities.
 - Run vital record processing to determine which volumes to retain and what volume moves are required based on vital record specifications. Vital record processing can be run to validate vital record specification information without updating volume and data set information in the DFSMSrmm control data set.
 - Run expiration processing to identify volumes that are ready to be released and returned to scratch.
 - Run storage location management processing to assign shelf locations to volumes that are being moved to storage locations.
 - Run backup of the control data set and the journal to ensure the integrity of the control data set and journal.
- See [z/OS DFSMSrmm Reporting](#) for information about utilities that help you get information about your removable media library and storage locations, security-related information about volumes and data sets defined to DFSMSrmm, and audit trail information about volumes, shelf assignments, and user activity. Create movement and inventory reports by producing an extract data set from the control data set and creating a report from the control data set with EDGRPTD.
- See Chapter 14, [“Maintaining the control data set,”](#) on page 377 for information about using EDGBKUP and EDGUTIL to back up and recover the control data set, back up the journal, and check the integrity of the information contained in the control data set. Use the EDGBKUP utility to back up and recover the control data set and back up the journal, and the EDGUTIL utility to create, update, and verify the control data set.

Use DFSMSrmm backup utilities rather than other backup utilities, such as DFSMS access method services EXPORT, because DFSMSrmm provides the necessary serialization and forward recovery functions. Use EDGBKUP to backup and recover the DFSMSrmm control data set when DFSMSrmm is inactive, stopped, or quiesced.

- See Chapter 15, [“Initializing, erasing, and scanning tape volumes,”](#) on page 423 for information about EDGINERS, the DFSMSrmm utility you use to erase and initialize tape volumes either automatically or manually and to scan tape labels manually. You can use EDGINERS to replace the DFSMSdfp utility IEHINITT.

Step 24: Installing PTFs and the SMP/E maintenance to DFSMSrmm

Installing PTFs to DFSMSrmm results in updates to programs in the system libraries such as LPALIB and LINKLIB. These changes are normally implemented by an installation-defined process that would normally include an IPL of the system. If any DFSMSrmm code resident in LPALIB is changed, you must IPL with the CLPA option.

If you implement DFSMSrmm updates without an IPL, you should be aware that:

- You cannot do this for programs that are in LPALIB.
- A consistent set of DFSMSrmm programs is required to avoid problems.

Recommendation: Stop the DFSMSrmm started procedure. Copy the updated programs into the system libraries. This is usually LINKLIB. Next, refresh or restart LLA, and start the DFSMSrmm procedure.

Note: If any dialog-related parts are updated, your end users should exit ISPF, and re-enter the DFSMSrmm dialog.

See Chapter 11, “Using DFSMSrmm installation exits,” on page 267 for details about maintenance to installation exits such as EDG_EXIT100, EDG_EXIT200, and EDG_EXIT300.

Chapter 3. Setting up DFSMSrmm client and server systems

When a server subsystem starts, identify some basic information for TCP/IP to the DFSMSrmm subsystem using the EDGRMMxx parmlib OPTION SERVER operand. See [“Defining system options: OPTION” on page 161](#) for information on the OPTION command. After the startup of the standard subsystem, DFSMSrmm communicates with TCP/IP and prepares to handle DFSMSrmm requests from client systems. The server verifies that TCP/IP and the specified PORT is available, determines its own default IP address and informs the operator that initialization is successful or issues error messages. As well as normal DFSMSrmm subsystem operation, such as processing of local requests, the server waits for and accepts connection requests, and processes requests from DFSMSrmm client systems. If the server task is unable to start successfully, you can still use DFSMSrmm as a standard subsystem until the server task problem is resolved.

When a client subsystem starts, identify some basic information for TCP/IP to the DFSMSrmm subsystem with the EDGRMMxx parmlib OPTION CLIENT operand. See [“Defining system options: OPTION” on page 161](#) for information on the OPTION command. During startup, the subsystem communicates with TCP/IP and prepares to send DFSMSrmm requests from the client to a server system. The client verifies that TCP/IP is available and that the server can be reached with the specified PORT, determines its own IP address, verifies the control data set ID matches that of the server, and informs the operator that initialization is successful or issues error messages. DFSMSrmm ignores any parmlib options not required for a client system. The client can connect to only one server system at a time. If the defined server is not available, the client issues a WTOR EDG0358D and waits for either the operator to reply with CANCEL, RETRY, or for the server to be available for connection.

After the DFSMSrmm client and server are started, DFSMSrmm fails requests with an I/O error when:

- There is a client or server communication error that cannot be successfully completed.
- The server is restarted using the command:

```
F DFRMM,M=xx
```

DFSMSrmm recovers from the error when a new server is available and processing continues.

DFSMSrmm issues a WTOR when a TCP/IP error occurs. RETRY processing relies on the operator replying to the WTOR. If the error is resolved and the operator has not replied to the WTOR, DFSMSrmm processing automatically continues and cancels the outstanding WTOR.

Once the client is started, no further verification of the server availability is performed unless a DFSMSrmm request is to be processed. When a request is processed and server communication fails or a time out occurs, and retry still cannot process the request, DFSMSrmm issues message EDG0358D to describe the error and prompts the operator to reply CANCEL or RETRY, and DFSMSrmm automatically continues if the error is resolved.

The DFSMSrmm client system processes most requests by communicating with the server but also by processing those local requests which can be completely processed on the client system. When multiple tasks are being processed, DFSMSrmm maintains a queue in FIFO order. The operator can issue this command to display the tasks and a summary of the queues.

```
F DFRMM,Q A
```

The DFSMSrmm server processes local requests as if it runs as a standard system. In addition, client requests are accepted and processed synchronously while the requester on the client waits. There is no queue of client requests maintained on the server. The request queues maintained on the server are for local requests only. When you list the active tasks, using 'QUERY ACTIVE', the active local requests are listed together with the accepted client requests.

You must update your firewall to ensure that communication between DFSMSrmm clients and servers is allowed only for the defined IP addresses and ports. The DFSMSrmm subsystem does no authentication, encryption, or verification of connect requests received on the server other than to verify that it is a valid DFSMSrmm request and that control data set IDs match. You should also consider using RACF to protect the use of the IP addresses defined for DFSMSrmm and limit use of the IP address to the DFSMSrmm started task.

Tracing of the IP communication is enabled by the DFSMSrmm support. You will use TCP/IP facilities, such as TCP/IP component trace to gather information about the DFSMSrmm socket processing. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information on operator procedures.

Implementing DFSMSrmm client and server systems

This topic describes how to implement DFSMSrmm client and server systems.

Related reading: See:

- “DFSMSrmm inventory management considerations when client/server support is enabled” on [page 332](#) for details about DFSMSrmm inventory management considerations when you have set up DFSMSrmm client/server systems.
- [z/OS DFSMSrmm Managing and Using Removable Media](#) for a complete description of operator procedures.

Before you begin:

- All client and server systems must be at a currently supported release level.
- DFSMSrmm on z/OS V1R12 and later releases is an IPv6 enabled application that supports both IPv4 and IPv6 sockets. You can continue to use IPv4 on all systems or you can run a mixed environment with one or more V1R12 systems using IPv6 and other systems using IPv4. Once all systems are V1R12, you have the option of moving all systems to IPv6. In a mixed environment, dual-mode IP stacks are required. Refer to [z/OS Communications Server: IPv6 Network and Appl Design Guide](#) for more information. DFSMSrmm client/server processing is dependent on Internet Protocol V4.
- You can share the control data set with other systems that run any supported level of DFSMSrmm with any supported level of DFSMSrmm that has appropriate toleration maintenance installed.

You can convert existing DFSMSrmm systems to be either client or server systems, or add new DFSMSrmm systems to the RMMplex. In addition you can merge existing DFSMSrmm systems into the RMMplex by merging the control data sets as described in the IBM Redbook *DFSMSrmm Primer*, SG24-5983.

- To implement a client system, follow these steps.
 1. Ensure TCP/IP definition files are updated to identify the server host name, IP address, and port number.
 2. Ensure that the RACF user ID associated with the DFRMM started task is configured correctly. Refer to "Requirement for an OMVS segment" in [z/OS Communications Server: IP Configuration Guide](#).
 3. If you have a firewall installed, update your firewall to ensure that communication between DFSMSrmm clients and servers is allowed only for the defined IP addresses and ports. The DFSMSrmm subsystem does no authentication, encryption, or verification of connect requests received on the server other than to verify that it is a valid DFSMSrmm request and that the control data set IDs match. You should also consider using RACF to protect the use of the IP addresses defined for DFSMSrmm and to limit the use of the IP address to the DFSMSrmm started task.
 4. Update the parmlib options with the `CLIENT` operand and select appropriate values for the sub operands. The parmlib operand, `CDSID`, has to be the same as on the server. If `CATSYSID` is not already set, add the operand now to define the list of system IDs that share catalogs with the client system or specify that catalogs are shared. Define the list of system IDs that share catalogs with the client system (`CATSYSID(list)`) or specify that catalogs are shared (`CATSYSID(*)`).
 5. Refresh DFSMSrmm with the new EDGRMMxx parmlib member.
 6. Run EDGHSKP CATSYNCH to synchronize catalogs if needed.

7. Run EDGHSKP with EXPROC regularly to return volumes to scratch status.
- To implement a server system, follow these steps.
 1. Update the parmlib options with the SERVER operand and select appropriate values for the sub operands.
 2. If CATSYSID is not already set, add the operand to define the list of system IDs that share catalogs with the server system or specify that catalogs are shared.
 3. Ensure that TCP/IP definition files are updated to identify the server host name, IP address, and port number.
 4. Ensure your firewall is updated with the client and server IP addresses and port numbers.
 5. Refresh DFSMSrmm with the new EDGRMMxx parmlib member.
 6. Run EDGHSKP CATSYNCH to synchronize catalogs if needed.
 - You must perform these steps to implement a standard system that is part of an RMMplex that contains client and server systems.
 1. If CATSYSID is not already set, add the operand to define the list of system IDs that share catalogs with the server system.
 2. Refresh DFSMSrmm with the new EDGRMMxx parmlib member.
 3. Run EDGHSKP CATSYNCH to synchronize catalogs if needed.

Using the DFSMSrmm client and server systems

This topic contains recommendations for using the client or server system.

In order to manage a single tape inventory across multiple sysplexes where there is no shared DASD available, you can create one or more DFSMSrmm client systems. Any tape usage on a client system uses the server system to dynamically validate and record tape usage. If the server system is not available for any reason, for example, the IP connection is unavailable or fails, you cannot use tapes on the client system until the server is reconnected or restarted.

All DFSMSrmm users on a client system can use the DFSMSrmm ISPF dialog, RMM TSO subcommands, and batch utilities, including use of DFSMSrmm API and High Level Language API.

Recommendation: Users who need regular access to DFSMSrmm data should log on to the server system. Storage administrators and tape librarians should use a server system or an DFSMSrmm system with direct access to the control data set except when there is a specific reason for using a client system.

- Performing a task for a system-managed tape library that is known to the client and not the server.
 - Ejecting a volume from a system-managed tape library
 - Adding volumes to a system-managed tape library using STATUS(VOLCAT)
 - Changing volume attributes that are also maintained in the TCDB
 - Running expiration processing
 - Confirming moves for exported stacked volumes
 - Running EDGUTIL VERIFY with the TCDB and optionally the Library Manager
- Using DFSMSrmm catalog processing for cataloged data sets that are cataloged only on the client system.
 - Confirming the erasure or initialization of a volume
 - Returning volumes to scratch status and DFSMSrmm is to perform the return to scratch cleanup actions
 - Deleting volumes that contain cataloged data sets

Managing catalogs in an RMMplex

You can implement DFSMSrmm client/server support with or without sharing catalogs across all of the systems in the RMMplex. You must, however, identify to DFSMSrmm whether catalogs are shared or not using the EDGRMMxx parmlib OPTION CATSYSID command described in [“OPTION command operands”](#) on page 168 in the parmlib member for each system.

DFSMSrmm uncatalogs data sets on a volume, when you specify the EDGRMMxx parmlib OPTION UNCATALOG command under these conditions:

- The commands must be processed on the system the data set was created on if all the catalogs are not shared.
- A volume is returned to scratch status, DFSMSrmm uncatalogs all the data sets on the volume.
- The RMM DELETEVOLUME FORCE subcommand is issued for a volume, DFSMSrmm uncatalogs all the data sets on the volume.
- The RMM CHANGEVOLUME DSNAME subcommand is issued for a volume, DFSMSrmm uncatalogs all the data sets on the volume. If the data set name specified on the RMM CHANGEVOLUME subcommand matches the data set name on the volume, then DFSMSrmm only uncatalogs subsequent data sets.
- The RMM DELETEDATASET subcommand is issued for a data set, DFSMSrmm uncatalogs the data set. Also, DFSMSrmm uncatalogs all data sets recorded on the same volume with higher data set sequence numbers.
- A tape data set is overwritten, DFSMSrmm uncatalogs the data set. Also, all data sets recorded on the same volume with higher data set sequence numbers are uncataloged.
- When the volume on which data sets resides is returned to scratch status, DFSMSrmm uncatalogs data sets.
- Confirming the erasure or initialization of a volume
- Returning volumes to scratch status, and DFSMSrmm performs the return to scratch cleanup actions

To use the DFSMSrmm catalog processing, you must synchronize the catalogs with the DFSMSrmm control data set. The catalog status for all data sets is maintained in the server system control data set. With unshared catalogs, the UNCATALOG parmlib option, on the server system, cannot be honored for data sets created on the client systems because the server cannot communicate with the client to initiate uncatalog processing. However, when processing is requested from the client, there is special recognition and handling of the request so that any catalog or RACF profile updates can be initiated on the client system. To synchronize the control data set and the catalogs, specify the EDGRMMxx parmlib OPTION CATSYSID(list_of_sysids). See [“Running DFSMSrmm catalog synchronization”](#) on page 365 for additional information. Then run EDGHSKP with CATSYNCH, VERIFY, and EXPROC on the client system.

For example, you have 3 systems; SystemA is to run as a client and has no shared DASD in common with SystemB. SystemC will run in a sysplex and share their own catalogs and the DFSMSrmm control data set.

- For client SystemA, specify the EDGRMMxx parmlib OPTION CATSYSID(SystemA) SYSID(SystemA) CLIENT(....) command
- For SystemB, specify the EDGRMMxx parmlib OPTION CATSYSID(SystemB, SystemC) SYSID(SystemB) SERVER(....) command
- For SystemC, specify the EDGRMMxx parmlib OPTION CATSYSID(SystemB, SystemC) SYSID(SystemC) command

Run EDGHSKP CATSYNCH once on the client system SystemA, once on one of the systems SystemB or SystemC, and then run EDGUTIL with PARM=UPDATE and SYSIN containing the statement; CONTROL CATSYNCH(YES).

Chapter 4. Organizing the removable media library

You can organize your removable media library by performing these tasks:

- Create pools of volumes and shelves in your removable media library
- Use Storage Management Subsystem (SMS) management class and storage group to manage volumes in your removable media library
- Use special dates to manage volumes
- Use duplicate volume serial numbers

Organizing the library by pools

Related reading: See [“Defining pools: VLPOOL”](#) on page 209 for details about the DFSMSrmm EDGRMMxx VLPOOL command.

You can use DFSMSrmm EDGRMMxx VLPOOL command operands to define pool of volumes and to set installation-defined policies for the volumes in the pool. You can define pools based on criteria like:

- Systems or subsets of systems
- z/OS or VM usage
- Use of rack pools to hold foreign tapes
- Media names your installation defines, like 3480, TAPE, CASSETTE
- SMS storage group names
- Release actions for volumes in a pool

Pooling overview

A *pool* is a group of rack numbers or volumes that share a common prefix. In DFSMSrmm, there are two categories of pools: rack and scratch.

A *rack pool* is shelf space that can be assigned to hold any volumes. Although you can add scratch volumes to these pools, you cannot normally use these volumes to satisfy non-specific mount requests. A rack pool cannot be used with the DFSMSrmm system-based scratch pooling. Rack pools can perform these functions:

- Hold volumes that are temporarily brought into the library but will be returned to the owner after a period of time
- Hold customer, foreign tapes, and software product volumes
- Contain scratch volumes for use with DFSMSrmm exit-selected scratch pooling

A *scratch pool* is shelf space assigned to hold volumes for use with the DFSMSrmm system-based scratch pooling. The volumes assigned to this shelf space can be used to satisfy scratch requests as long as the volumes are in scratch status. Once the volume has been written to, it becomes a volume with MASTER status, until the volume is returned to scratch status. The volume remains in the same DFSMSrmm system based scratch pool, in that it occupies the same shelf space regardless of status.

The scratch volumes in a system-managed tape library can be from one or more pools. DFSMSrmm does not provide pool selection or validation for volumes in an automated tape library because ACS routines use storage class and storage group to make allocation decisions, and the library manager picks a scratch volume. DFSMSrmm provides pool validation for volumes that reside in a manual tape library. You can pool by storage group, exit-selected pool prefix, or DFSMSrmm system-based pooling when using manual tape libraries.

DFSMSrmm allows you to use these basic types of pooling:

- Pools of shelf space that are based on rack number prefixes. Each range identifies characteristics like management criteria and media name. Use shelf space pools to store volumes that do not match your installation-selected volume ranges and to store duplicate volumes. Define shelf space pools by using the DFSMSrmm EDGRMMxx VLPOOL command that is described in [“Defining pools: VLPOOL” on page 209](#).
- Pools of volumes that are based on the volume serial number prefix. These volumes do not have a rack number or the rack number matches the volume serial number. Each range identifies characteristics like management criteria and media name. Define volume prefix pools by using the DFSMSrmm EDGRMMxx VLPOOL command that is described in [“Defining pools: VLPOOL” on page 209](#).
- Scratch pools. These can be one or more pools of volumes. Scratch pools can be based on name, SMS storage group, prefix, or system. You define scratch pools by using the SYSID, PREFIX, and NAME operands of the DFSMSrmm EDGRMMxx VLPOOL command that is described in [“Defining pools: VLPOOL” on page 209](#).
- Storage groups. When you pool by storage group and use SMS ACS processing to assign a storage group to a tape data set, or the volume is in a system-managed manual tape library, DFSMSrmm ensures that a volume from the correct storage group is mounted. The storage group can be the same across multiple VLPOOL entries. You can use storage group for scratch pooling for system-managed manual tape libraries and non-system managed tape. Volumes that have been assigned storage group names cannot be used to satisfy scratch mount requests that do not request volumes from a specific storage group unless the mount request is in a manual system-managed tape library.

Pools can be used in these ways:

- Adding shelf space - DFSMSrmm matches the rack number prefix to the most specific VLPOOL prefix. The rack media name is taken from the matching VLPOOL entry.
- Adding volumes - You can optionally specify RACK or POOL operands to override default processing. Default processing matches the volume serial number prefix to the most specific VLPOOL prefix. If a rack number is found that matches the volume serial number and the specified media name, the volume is stored in the matching shelf pool. If no rack number is found, the volume is in a volume pool. When adding a volume you can specify a storage group name so that the volume can be in a specific scratch pool. If no storage group is specified by the command, DFSMSrmm checks to see if the matching VLPOOL NAME is a storage group, and uses that value as the storage group name. In this case, the scratch pool matches the volume pool.
- Managing access to a volume - For system-managed tapes in a manual tape library, DFSMSrmm validates the mounted volume for the requested pool. You can use the installation exit to ignore the storage group pool and use DFSMSrmm system-based scratch pooling described in [“Using storage group for manual tape library pooling” on page 292](#).
- Defining actions that should be taken when volumes are ready for release - You can define release actions for volumes on the pool level that might not already be set for individual volumes. For example, you can set the NOTIFY options so that DFSMSrmm sets notification on the release action if it is not already on at the volume level.
- Selecting scratch pools for new tape data sets - For system-managed tape, SMS ACS processing assigns a storage group. DFSMSrmm uses the storage group name to pool the volumes into a scratch pool. For non-system managed tape, when SMSACS(YES) is specified in the PARMLIB, DFSMSrmm calls the SMS ACS processing to allow a storage group to be assigned.

It is also possible to instead use the defaults table or the EDG_EXIT100 installation exit to assign a scratch pool.

Also, EDG_EXIT100 can cause the storage group assigned for manual tape libraries to be ignored (this is the default behavior for the sample exits supplied by DFSMSrmm).

When you pool by pool prefix selected by the defaults table, EDG_EXIT100, or SYSID, and the VLPOOL prefix has an associated NAME, DFSMSrmm uses the pool name for mount messages and drive displays but always validates mounted volumes by using the pool prefix and SYSID. Multiple VLPOOL entries can have the same SYSID values and the same NAME values.

Pooling considerations

If you do not currently use application or user-oriented scratch pools based on job names and data set names, use DFSMSrmm system-based pooling, where the pool is chosen based on the system name specified in the VLPOOL statement or use a general, installation-wide scratch pool. If you currently use application or user-oriented scratch pools that are based on job names and data set names, use SMS ACS routines to assign storage groups. You can use storage groups to pool volumes based on job names and data set names as described in [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82. You can also implement DFSMSrmm pooling using DFSMSrmm VLPOOL options.

If you are already using some form of scratch pooling based on information such as job name and data set name, review your current use of these pools in order to plan or change your implementation under DFSMSrmm.

Pool types

You use scratch pools with DFSMSrmm system-based scratch pooling. Define scratch volumes in rack pools and use the RMM GETVOLUME subcommand to claim them or assign them to a user. You can also use the defaults table or the EDG_EXIT100 exit to use rack pools or scratch pools for non-specific tape output requests.

When you define pools for use with specific groups of users or applications, you also need to consider how volumes are defined to DFSMSrmm. You can use the DFSMSrmm ISPF dialog or RMM ADDVOLUME subcommand to define volumes. To add a volume to a specific pool, provide the pool prefix when you issue the RMM ADDVOLUME subcommand or use the DFSMSrmm ISPF dialog. You only need to specify a pool prefix if the volume serial number does not start with the same characters as the pool prefix. When defining scratch volumes you will probably use volume serial numbers that match the rack numbers you include in the pool definition. When adding private volumes which come from another library you will have to specify a pool prefix. You can also use the DFSMSrmm ISPF dialog or RMM TSO CHANGEVOLUME subcommand to move an existing volume to a pool you define, so that the volume becomes part of the pool.

Tape drive availability

If you are using specific pools, plan how to set up your scratch tape drives so that a correct scratch tape can be mounted promptly. The benefit of using scratch tapes is that almost any unused tape can be mounted and DFSMSrmm will record new information for the volume. However, as you become more selective about which scratch tapes are used for each non-specific volume request, the benefits of using scratch tapes decrease as your requests become more and more specific.

To make mounting tape volumes easier, consider setting aside certain tape drives for use with identified scratch pools. When using tape cartridge loaders with DFSMSrmm pooling, you need to load each drive with volumes from a single scratch pool, run them in system mode, and direct only the correct non-specific requests to them. Cartridge loaders must not be run in automatic mode because DFSMSrmm processing depends on the mount message, which is not issued when the cartridge loader is in automatic mode and a volume is already loaded. Consider defining different esoteric unit names for each of the tape drives you choose to load with scratch volumes from a single pool.

You can use the EDG_EXIT100 exit to request DFSMSrmm to disable the tape drive cartridge loader to prevent specific scratch pool requests from emptying the loaders through volume rejection. Alternatively, you can direct requests to the correct drives or run the loaders in a mode that prevents them being automatically indexed when a mount is received. Also DFSMSrmm disables the loader if a volume is rejected for a scratch request to prevent the loader from being emptied.

Operator messages and tape drive displays

To use ACS routines or exit selected scratch pooling, you must define messages IEC501A, IEF233A, IAT5110, and IAT5210 using the DFSMSrmm MNTMSG commands in parmlib as described in [“Defining mount and fetch messages: MNTMSG”](#) on page 153. These messages provide the pool identifiers to the operator, the job name, and data set name information to the EDG_EXIT100 exit.

DFSMSrmm can update mount messages and tape drive displays with the pool prefix, storage group name, or pool name. DFSMSrmm can also provide information to some tape automation software installed on your system through the tape mount messages. DFSMSrmm updates messages IEF233A and IEC501A, and IAT5210 for JES3 to include the pool identifier that you select. You can define Basic Tape Library Support (BTLS) category names so that DFSMSrmm can be used to manage volumes in a BTLS-managed tape library. If you use BTLS scratch category names as DFSMSrmm pool names, DFSMSrmm can be used to direct which volumes BTLS will use for each scratch mount.

For JES3, to find out which pool was used to satisfy a request, install the USERMOD EDG3UX71; users can see mount messages with the pool identifier in the output from their batch jobs. Users allowed to view the SYSLOG can see the updated messages in the SYSLOG.

Calculating pool size

When planning the size of your pools, remember that the number of characters in the pool prefix determines the maximum pool size. A pool prefix consists of one to five alphanumeric, national, or special characters followed by an *, for example, ABADA*. [Table 11 on page 78](#) shows the maximum number of volumes each range of pool prefixes can contain:

Table 11. DFSMSrmm volumes in a pool determined by pool prefix

Pool Prefix	Maximum Number of Volumes
A*	100 000
AA*	10 000
AAA*	1000
AAAA*	100
AAAAA*	10

The maximum numbers in [Table 11 on page 78](#) are for numeric suffixes. If you use nonnumeric suffixes, such as pool prefix A with suffixes A00000 through AZZZZZ, you can increase the maximum number of volumes for that pool. For example, using the suffix AAAAA* with alphanumeric characters, \$, #, @, and the special characters, you can have a maximum of 51 volumes. However, by using nonnumeric suffixes, you limit the use of the COUNT operand on RMM TSO rack and volume related subcommands.

Plan ahead when choosing your pool prefixes, ensuring that the pool prefixes can satisfy all future volume quantities. It is not necessary to define all shelf locations to DFSMSrmm at once. Using pool definitions, you can reserve space for library expansion without taking up space in the control data set. For instance, you could assign a pool prefix ABC*, allowing space for 1000 volumes, but only defining shelves ABC000 - ABC099 initially.

When you use the VLPOOL NAME or SMS storage group name for scratch pooling, you can include multiple VLPOOL PREFIX ranges into the same scratch pool by specifying the same NAME value on multiple VLPOOL definitions or by assigning storage group names for each volume.

Defining pools in parmlib member EDGRMMxx

You must define any pool that you plan to use with DFSMSrmm in the parmlib member EDGRMMxx using the VLPOOL command. This applies to pools used by DFSMSrmm pooling and by the EDG_EXIT100 exit. See [“Defining pools: VLPOOL” on page 209](#) for information on using VLPOOL command in the parmlib member EDGRMMxx. Also consider shelf space and whether volumes are to be segregated based on media names for any scratch pooling method you use.

Define at least one pool using the VLPOOL command in the DFSMSrmm parmlib member. Use the pools to assign a media name for shelf space and volumes based on a prefix. Define a default pool to ensure that volumes and racks that do not match to a more specific VLPOOL are added with the correct media name and pool name.

Figure 28 on page 79 shows a VLPOOL command example of the default pool that DFSMSrmm assigns if you do not specify any VLPOOL commands. This default pool includes all shelf locations, all volumes in those shelf locations, and all volumes with no defined shelf location.

```
VLPOOL RACF(N) TYPE(S) -  
  EXPDTCHECK(0) MEDIANAME(parmlib_default_medianame) -  
  DESCRIPTION('DEFAULT POOL') PREFIX(*)
```

Figure 28. Default VLPOOL command

In the example:

RACF(N)

Specifies whether to provide RACF tape profile processing for this tape pool. The value N indicates that RACF tape profiles are not created, changed, or deleted for tapes in this pool, regardless of the setting of the system-wide option TPRACF.

TYPE(S)

Specifies the type of pool. The value S means that it is a scratch pool.

EXPDTCHECK(O)

Specifies the processing tape data sets on volumes in this pool that are expiration date protected. The value O means that DFSMSrmm takes no action but allows the operator or automated software to reply as necessary to any write-to-operator messages (IEC507D).

MEDIANAME(parmlib_default_medianame)

Specifies a media name for all volumes in this pool. The MEDIANAME value for the default pool is the same value that you specify with the EDGRMMxx parmlib OPTION MEDIANAME value.

To separate storage locations by media name, any LOCDEF command media names that you define must be the same as the VLPOOL media names or a subset of the media names. See [“Defining storage locations: LOCDEF” on page 143](#) for information on defining storage locations.

DESCRIPTION('DEFAULT POOL')

Describes the pool.

PREFIX(*)

Specifies a generic shelf location prefix that is used in operator messages, TSO subcommands, and the DFSMSrmm ISPF dialog. A single asterisk defines the default pool that contains all rack numbers or volumes not specifically included in any pool. You also have the option to define a pool name that can be used to update operator messages, displays, and can be used as a storage group.

You can also specify a SYSID operand on the VLPOOL command. SYSID associates a scratch pool with a particular system. DFSMSrmm matches the value with the SYSID operand of the OPTION command, also in EDGRMMxx. If the VLPOOL SYSID matches the OPTION SYSID, DFSMSrmm performs these functions:

- A volume from this pool can be used to satisfy a scratch mount request on this system.
- A volume from any other pool where its SYSID also matches the OPTION SYSID can be used to satisfy a scratch request on this system.

A volume from a pool that has no explicit VLPOOL SYSID can be used to satisfy a scratch request on a system if there are no other VLPOOL definitions that have a SYSID matching the OPTION SYSID. The SYSID values are ignored when a storage group is selected by ACS processing and when a pool is set for a non-specific volume request by EDG_EXIT100 processing. The system level pools can still be used as long as SMS ACS processing does not assign a storage group and the EDG_EXIT100 exit does not set a specific pool to be use.

Changing pool definitions

Use the VLPOOL command in the EDGRMMxx parmlib member to change pool definitions. See [“Defining pools: VLPOOL” on page 209](#) for information about using the VLPOOL command.

If you change a pool definition that include racks defined to an existing pool, you might need to redefine the shelf locations or volumes to DFSMSrmm. To redefine the shelf locations or volumes when the media name for the new pool does not match the existing pool, use the RMM subcommands or DFSMSrmm ISPF

dialog to change information. For example, to redefine the shelf locations and volumes, perform these tasks:

1. Add or alter the VLPOOL statements in parmlib and refresh the DFSMSrmm parameters.
2. Add rack numbers for the new VLPOOL prefixes and those with changed MEDIANAMES.
3. Issue the RMM CHANGEVOLUME volser MEDIANAME for each volume affected.
4. Delete the old rack numbers specifying the old MEDIANAME.

See *z/OS DFSMSrmm Managing and Using Removable Media* for more information about using the RMM subcommands or the DFSMSrmm ISPF dialog to change the pool definitions.

Designing rack pools

If you have several types of media, define pools based on type of media. For example, you can separate your reels and cartridges from each other by placing them in separate pools.

The volumes in these pools have no relationship to any specific device type. Although you can use device type names rather than media type names to define these pools, DFSMSrmm does not associate device type names with devices.

You might want to provide a pool of scratch volumes for a particular group of users or application. With EDG_EXIT100, you can use an exit-selected scratch pool to satisfy non-specific mount requests.

Users can also obtain volumes from a pool using the DFSMSrmm ISPF dialog or the RMM GETVOLUME subcommand. When you use the GETVOLUME subcommand, all tape mounts are specific mounts, which means that you cannot use a scratch pool and cartridge loaders.

If you have any volumes that do not meet your chosen volume naming conventions, or have a number of volumes that regularly enter and leave the removable media library, define a pool of empty shelf locations in which you can store these volumes.

When volumes arrive at the installation, define the volume to DFSMSrmm using the chosen pool ID. DFSMSrmm assigns the next available empty shelf location to the volume. Place an external label specifying the rack number on the volume to identify it.

If your removable media library is split across rooms or sites, define pools that allow easy separation and identification of volumes based on shelf location number.

Designing scratch pools

You can define scratch pools based on the system where the volumes are used. For example, define a pool prefix A* to cover all volumes in a pool reserved for system MVSA. Define a pool prefix VM* for a pool reserved for volumes used on a VM system.

When you define scratch pools by system, you can only use scratch volumes from those pools on the systems specified for them. If you try to mount a scratch volume on a different system than the one with which it is associated, DFSMSrmm rejects the volume. If a system has no pool defined for it, the system accepts all volumes except those you have defined to a specific system. Make your scratch volumes available for use on all systems if you do not want these limitations.

Recommendation: Use one scratch pool for the entire library. You can use a VLPOOL command similar to the default, shown in [Figure 28 on page 79](#). When defining scratch pools, do not use no label tapes in the pool.

You can define multiple pools for each system by associating the pools with a system using the DFSMSrmm EDGRMMxx parmlib VLPOOL command with the SYSID operand as described in [“Defining pools: VLPOOL” on page 209](#). DFSMSrmm accepts a volume from any of the pools defined to that system. DFSMSrmm updates mount messages and drive displays to indicate the pool from which the scratch volume should be pulled. The pools are searched in alphabetical order and the first suitable pool encountered is added to mount and fetch messages. You can assign the same name to multiple pools to make it easier to satisfy mount requests.

When you define multiple scratch pools, DFSMSrmm does not use the media name when selecting the pool to use. If you are not using system based scratch pools, the operator can mount any volume. Operators use local conventions and operator procedures to select a scratch volume. For example, the operator should select a reel tape for a mount on a 3420 drive. For a 3490 drive, the operator should select a cartridge tape. In most cases, scratch tapes are already pulled and are available for immediate mounting.

Within the pool for use on a VM system, you can define each volume for use only on VM, or for use on z/OS and VM. DFSMSrmm uses this information to reject non-z/OS volumes on z/OS. You can use VM volumes on VM systems.

You can design pools at the user, group, or application level by using DFSMSrmm calls to SMS ACS to assign storage group names, or you can use the defaults table or the EDG_EXIT100 installation exit to control scratch pool selection. See [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82, [“Using the DFSMSrmm EDG_EXIT100 installation exit”](#) on page 267, and Chapter 21, [“Using the parmlib member EDGDEFxx,”](#) on page 483 for more information about using pool names and implementing exit-selected scratch pools.

Requesting and using scratch pools

DFSMSrmm system-based scratch pooling is always available for you to use, however, you cannot use it for automated system-managed tape. To use exit-selected scratch pooling, use the defaults table as described in Chapter 21, [“Using the parmlib member EDGDEFxx,”](#) on page 483. Or, instead install the sample EDGUX100 exit module on your system and customize it to set a specific pool for a non-specific volume request. See Chapter 11, [“Using DFSMSrmm installation exits,”](#) on page 267 for information about how to use the EDG_EXIT100 installation exit.

DFSMSrmm updates the mount message and drive display with the appropriate pool prefix, pool name, or storage group name. DFSMSrmm validates the mounted scratch volume against the selected scratch pool prefix, pool name, or storage group during OPEN processing.

You can set up DFSMSrmm to satisfy each request for a new scratch tape in one of these ways:

- Set up a default pool using the EDGRMMxx parmlib OPTION VLPOOL command. DFSMSrmm uses the VLPOOL definitions to select a default DFSMSrmm pool by using DFSMSrmm system-based pooling when you have not set up other pools. DFSMSrmm sets a pool prefix and a pool name if a specific pool is selected. Set up DFSMSrmm system-based scratch pooling using the DFSMSrmm VLPOOL command with the SYSID operand. This method allows you to control which scratch pools can be used based on the system on which the volumes are used. This enforces pooling based on VLPOOL prefix pools. DFSMSrmm uses the DFSMSrmm system-based scratch pooling when you do not select any other method.
- Use ACS routines for scratch pooling based on tape storage group names. Using ACS processing to set a storage group name overrides all other pool selection methods. DFSMSrmm provides support for non-system-managed tape and for system-managed manual tape libraries. This support enables pooling at the individual volume level. You assign a storage group name to each volume by using DFSMSrmm TSO subcommands or by using pooling information that you define with the DFSMSrmm EDGRMMxx parmlib VLPOOL command. See [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82 for additional information. DFSMSrmm calls ACS routines passing environment information, including the pool identified by DFSMSrmm system-based pooling. The ACS routine can optionally set a storage group name, which overrides the DFSMSrmm system-based pool.
- Customize the defaults table defined in EDGDEFxx parmlib member. Use the job name and the data set name to determine which pool to use. The defaults table returns a pool prefix value that overrides the DFSMSrmm system-based pool, and is described in Chapter 21, [“Using the parmlib member EDGDEFxx,”](#) on page 483.
- Customize the sample EDGUX100 exit module that is described in Chapter 11, [“Using DFSMSrmm installation exits,”](#) on page 267 to select a pool prefix. You can use information in the system to determine which pool to use. DFSMSrmm calls the EDG_EXIT100 installation exit if ACS processing does not return a storage group name. The EDG_EXIT100 installation exit can return a pool prefix value

that overrides the DFSMSrmm system-based pool. The pool name is set based on the selected pool prefix.

- Use pre-ACS processing to obtain the DFSMSrmm system-based pool or the EDG_EXIT100 installation exit pool prefix as an input value to the ACS routines in the MSPPOOL read-only variable. During pre-ACS processing, DFSMSrmm does not make the RMMPOOL environment call to the ACS routine. During pre-ACS processing for new allocations:
 - DFSMSrmm uses the VLPOOL definitions to select a default DFSMSrmm pool using DFSMSrmm system-based pooling. DFSMSrmm sets a pool prefix if a specific pool is selected.
 - DFSMSrmm calls the EDG_EXIT100 installation exit to obtain a pool prefix value. If a pool prefix value is returned, the pool prefix value returned by the EDG_EXIT100 installation exit overrides the DFSMSrmm selected pool.
 - DFSMSrmm returns the selected value in the MSPPOOL read-only variable if the MSPPOOL variable is not already set by the pre-ACS exit.

Using SMS tape storage groups for DFSMSrmm scratch pooling

This topic describes how to use storage groups for DFSMSrmm scratch pooling.

Before you begin: To use SMS tape storage groups, first define tape storage groups using ISMF. To define tape storage groups, you must have at least one system-managed tape library defined to ISMF. The library can be an automated tape library or a manual tape library. If you do not have a system-managed tape library or do not want to associate the new SMS tape storage groups with an existing library, define a new dummy library to ISMF. Use the ISMF define library application to define a system-managed tape library. Use a dummy library ID to identify the library and use appropriate values to complete the other data fields required for a tape library definition. When you define a dummy library, OAM issues the CBR3006I and CBR3002E messages at startup time. Ignore these messages when they are issued for your dummy library. Once you have defined the dummy library, you do not need to start OAM or define the OAMn subsystem to support this dummy system-managed library.

Making an ACS storage group assignment

To assign a storage group, you must have the SMS subsystem active and have a valid SMS configuration.

You use the ACS routines to process the special calls that DFSMSrmm makes to the SMS subsystem for ACS processing. DFSMSrmm requests that the data class, management class, and storage group routines are run. The environment variable is set to *RMMPOOL* so that you can differentiate allocation requests for system-managed data sets from requests by DFSMSrmm for a storage group name. When DFSMSrmm calls the ACS routines with the &ACSENVIR variable set to either RMMPOOL or RMMVRS, the storage class (&STORCLAS variable) is set to the word USERMM.

The SMS read-only variables that are set for DFSMSrmm requests are:

- &ACCT_JOB
- &ACCT_STEP
- &ACSENVIR
- &DD
- &DSN
- &DSORG
- &DSTYPE
- &EXPDT
- &FILENUM
- &GROUP
- &HLQ
- &JOB

- &LABEL
- &LIBNAME
- &LLQ
- &NQUAL
- &PGM
- &STORGRP
- &SYSNAME
- &SYSPLEX
- &UNIT
- &USER
- &XMODE

The &DD, &PGM, &EXPDT, and &FILENUM variables are optional for JES3 mount messages. &LIBNAME is optional for JES3 fetch messages.

1. Define your pools by using the DFSMSrmm EDGRMMxx parmlib VLPOOL command as shown in [Figure 29 on page 83](#). See [“Defining pools: VLPOOL” on page 209](#) for information about the VLPOOL command.

```

VLPOOL PREFIX(123*)  NAME(Poola) MEDIANAME(CARTS) -
  DESCRIPTION('PART OF POOL A')
VLPOOL PREFIX(99975*) NAME(Poola) MEDIANAME(REELS) -
  DESCRIPTION('PART OF POOL A')
VLPOOL PREFIX(C0*)   NAME(TEMP) MEDIANAME(CARTS) -
  DESCRIPTION('TEMPORARY POOL')
VLPOOL PREFIX(*)     NAME(DEFAULT) MEDIANAME(CARTS) -
  DESCRIPTION('DEFAULT POOL')
...

```

Figure 29. Defining pools with the DFSMSrmm EDGRMMxx parmlib VLPOOL command

2. Code your selection criteria and rules into the management class and storage group ACS routines as shown in [Figure 30 on page 83](#).

```

PROC 1 MGMTCLAS
...
/*****
/*  RMM POOLING FILTER LISTS
*/
*****/
...
FILTLIST POOLA INCLUDE(USERA.**,USERY.KEEP.*) -
  EXCLUDE(USERA.POOLB.***)
...
IF &ACSENVIR = 'RMMPOOL' THEN
/*****
/*  RMM POOLING DECISIONS
*/
*****/
DO
  SELECT (&DSN)
    WHEN (&POOLA) SET &MGMTCLAS = 'POOLA'
    OTHERWISE     SET &MGMTCLAS = 'DEFAULT'
  END
  /* ALLOCATE TEMP POOL FOR TEMPORARY DATA SETS */
  IF &DSTYPE = TEMP THEN SET &MGMTCLAS = TEMP
...
END
...
END

```

Figure 30. Sample management class routine

If your ACS processing does not set the storage group name, DFSMSrmm processing continues without using storage group names.

The processing that the STORGRP ACS routine performs is limited to the variables it can reference. You might want to make your decisions in the MGMTCLAS ACS routine and use the STORGRP routine to set the STORGRP variable to the selected value as shown in [Figure 31 on page 84](#).

Any management class value that you set during ACS processing for the RMMPOOL ACS environment call is ignored by DFSMSrmm processing.

```
PROC 1 STORGRP
...
IF &ACSENVIR = 'RMMPOOL' THEN
/*****
/*  RMM POOLING DECISIONS  */
*****/
DO
  SELECT (&MGMTCLAS)
  WHEN ('POOLA') SET &STORGRP = 'POOLA'
  WHEN ('DEFAULT') SET &STORGRP = 'DEFAULT'
  WHEN ('TEMP') SET &STORGRP = 'TEMP'
END
...
END
END
```

Figure 31. Storage group routine sample

A pooling example

This example describes how pooling might be implemented for an installation that has:

- An automated tape library and no non-system-managed drives.
- A manual tape library.
- Ranges of scratch tapes using a numbering system with consecutive numbers for volumes spread across a few well identified ranges.

Volume serial numbers, 210000 - 219999, 710000-719999, and 800000 - 805999.

- Foreign tapes from IBM and other vendors with standard labels and random volume serial numbers that use alphanumeric, national, or special character prefixes, such as DZ1100 or DP3100.
- Tapes sent from another site for input processing. The other site uses a range of tapes numbered 401000 - 401999. The tapes are scratch at the other site, but are private volumes here.

The installation defines pools using the VLPOOL command as shown in [Figure 32 on page 84](#):

```
VLPOOL PREFIX(21*) TYPE(S) MEDIANAME(CART) RACF(Y) EXPDTCHECK(N) -
  DESCRIPTION('scratch pool of cartridge system tape')
VLPOOL PREFIX(71*) TYPE(S) MEDIANAME(CART) RACF(Y) EXPDTCHECK(N) -
  DESCRIPTION('scratch pool manual tape library') -
  NAME(SGMTL01)
VLPOOL PREFIX(80*) TYPE(S) MEDIANAME(CART) RACF(Y) EXPDTCHECK(N) -
  DESCRIPTION('scratch pool enhanced capacity cartridge') -
  NAME(TWOTONE)
VLPOOL PREFIX(401*) TYPE(R) MEDIANAME(CART) RACF(Y) EXPDTCHECK(Y) -
  NAME(SITE2) -
  DESCRIPTION('tapes sent from site 2')
VLPOOL PREFIX(*) TYPE(R) MEDIANAME(CART) RACF(Y) EXPDTCHECK(Y) -
  NAME(SITE2) -
  DESCRIPTION('miscellaneous check-in and check-out tapes')
```

Figure 32. Defining pools with VLPOOL commands

The installation can now define volumes in the 21*, 71* and 80* pools which DFSMSrmm manages as scratch pools. The storage group SGMTL01 used as a VLPOOL NAME must be defined as a valid storage group name to SMS. The installation can also define some empty racks for the 401* volumes. Each time a foreign volume comes in, it is defined to DFSMSrmm, with RLSE(RETURN) RETPD(nn), labeled with a sticky label, and entered into the system-managed tape library.

RACF and EXPDTCHECK provide:

- RACF(Y) is specified for all pools to ensure that tape volume security is managed by DFSMSrmm as described in Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143.
- EXPDTCHECK(N) is used for scratch pools, so DFSMSrmm automatically replies to any IEC507D expiration date protection messages allowing scratch volumes to be reused. DFSMSrmm does not reply to the IEC507D message issued for tape volumes when you use the EDG_EXIT100 installation exit to ignore a volume.
- EXPDTCHECK(Y) is used for all non-scratch volumes to ensure that DFSMSrmm automatically replies to IEC507D to prevent overwrite of expiration date protected data.

For the remainder of the volumes, when a volume comes in, the installation defines a rack number that matches the VOL1 label, and then defines the volume using RLSE(RETURN) RETPD(nnn). This approach should be used since the external and internal volume serial number must be the same.

In this example, all scratch volumes are system-managed. DFSMSrmm validates scratch pools for requests in the manual system-managed library, but does not validate scratch pools for automated tape libraries. DFSMSrmm always ensures that a scratch volume is mounted in response to a non-specific mount request. Defining a rack pool is an alternative to using a scratch pool as described here. The rack pool should have a finite size based on known requirements, and accommodate any volume serial number as long as it does not conflict with the installation's naming conventions.

Anytime the installation wants to have more scratch volumes, existing pools can be made larger or new pools reserving a new range of numbers can be defined.

This approach for system-managed libraries differs from the way pools might be used with a non-system-managed library. With a non-system-managed library, you need to have physical shelving. You also have the option of using different internal to external labels.

For installations with mixed environments, a combination of these approaches provides a workable solution.

Managing pools with job name and data set name

You can manage scratch tape pools based on job names and data set names.

Recommendation: Use SMS ACS Storage Group assignment to manage scratch pooling with jobname and data set name for system-managed volumes. For information, see “Making an ACS storage group assignment” on page 82. For non-system-managed volumes, use the EDG_EXIT100 installation exit. See Chapter 11, “Using DFSMSrmm installation exits,” on page 267 for information about using the EDG_EXIT100 installation exit.

Use the defaults table or the EDG_EXIT100 installation exit to select the correct scratch pool for non-specific requests for non-system-managed volumes and to decide whether the cartridge loader should be used. You can also use criteria other than job names and data set names, but you must modify the supplied EDGUX100 exit module to do this.

Using scratch pooling for System managed and Non-System Managed Tape

Scratch pooling in a IBM VTS or IBM ATL is available by assigning a different media type. For example, in a IBM VTS making use of media1 and media2, in a IBM 3494 ATL making use of media3 and media4, and in IBM TS3500 ATL, making use of media5 to media13.

In a non-IBM VTS or ATL, there are two methods available; this being non-System Managed Tape or System Managed Tape. System Managed Tape would be defined as Manual Tape Library (MTL) in SMS configuration.

Non-System Managed Tape: There are three ways you can accomplish this:

Note: Neither method requires the use of exit EDGUX100.

1. Use NAME subparameter in EDGRMMxx VLPOOL parameter and MNTMSG parameter. IEF233A is modified to include POOL=poolname. For more information about VLPOOL, refer to [“Defining pools: VLPOOL” on page 209.](#)

2. Using SMSACS(YES).

Using SMSACS(YES) allows you more flexibility. That is, it allows you to use SMS ACS routines to make decisions on assigning appropriate storage group and implementing these changes via SMS ACS routine is easier than using an exit.

Use SMSACS(YES) parameter, MNTMSG parameter in EDGRMMxx, update SMS management class ACS routine and storage group ACS routine to check for &ACSENVIR RMMVRS and RMMPOOL. RMMVRS is used to assign a management class that can be used to assign a DSN VRS type to manage your tape. For example D99000 management class can be assigned to tape data set using EXPDT=99000 and DSN VRS of D99000 would be used to manage your tape data set.

RMMPOOL is used to assign a storage group name which will modify IEF233A and add POOL=storgrp.

&ACSENVIR RMMVRS and RMMPOOL in SMS ACS routine checking are only available for Non-System Managed Tape. After initial allocation, &ACSENVIR=ALLOC, DFSMSrmm calls the ACS routines for the second time with the &ACSENVIR variable set to either RMMPOOL or RMMVRS, the storage class (&STORCLAS variable) is set to the word USERMM, then proceeds to management class and storage group ACS routines. Checking for this storage class in ACS routine is not required.

For the examples of ACS routines, refer to [“Making an ACS storage group assignment” on page 82.](#)

More ACS routines examples you can find in the *z/OS DFSMSdfp Storage Administration* manual, chapter *Defining ACS routines*, topic *ACS routine environments*, subtopic *Tape management system support*.

3. Use the Defaults Table to select the scratch pool. For more information, see [Chapter 21, “Using the parmlib member EDGDEFxx,” on page 483.](#)

System Managed Tape: System Managed manual tape library (MTL), IBM VTS and IBM ATL scratch pooling is done as follows.

In a non-IBM VTS or ATL would be defined as Manual Tape Library in SMS configuration as a library.

- a. For MTL, if you plan to use SMS ACS routines to assign storage group using &ACSENVIR=ALLOC and you are using EDGUX100, MTL code must be commented out or removed. If you plan not to use SMS ACS routines for pooling, then EDGUX100 MTL code can be used. Refer to [“Using storage group for manual tape library pooling” on page 292.](#)

Additionally, define MNTMSG parameter in the EDGRMMxx parmlib member for placement of storage group name in IEF233A. SMSACS parameter in EDGRMMxx is NOT taken into account here and has no effect, as it is used for Non-System Managed tape; since it has been designed to control whether DFSMSrmm calls SMS ACS processing to enable use of storage group and management class values with DFSMSrmm for non-system managed tape. NAME subparameter in VLPOOL is ignored and is not used, storage group name will be used for IEF233A.

During initial allocation; SMS ACS routine in &ACSENVIR=ALLOC, assignment of management class and storage group name is done. Second call to ACS routines are not done for system managed tape. The storage group name is used in IEF233A if MNTMSG is used in EDGRMMxx and add POOL=storgrp. Management class that can be used to assign a DSN VRS type to manage your tape. For example D99000 management class can be assigned to tape data set using EXPDT=99000 and DSN VRS of D99000 would be used to manage your tape data set.

The defaults table can also be used for scratch pooling when using an MTL library. For more information, see [Chapter 21, “Using the parmlib member EDGDEFxx,” on page 483.](#)

- b. IBM VTS and IBM ATL

DFSMSrmm does not provide pool selection or validation for volumes in an automated tape library because ACS routines use storage class and storage group to make allocation decisions, and the library manager picks a scratch volume. Scratch pooling is not allowed within the same media type, even though POOL=storgrp name can be done if MNTMSG is defined. However, you can use

different media types. For example in a IBM VTS making use of media1 and media2, in a IBM 3494 ATL making use of media3 and media4, and in IBM TS3500 ATL, making use of media5 to media13. You would do this by assigning appropriate data class and storage group in your SMS ACS routines in &ACSENVIR=ALLOC. Management class ACS routine in &ACSENVIR=ALLOC can also be used to assign a management class, where you would then define DSN VRS type to manage your tape.

Non-System Managed Tape and System Managed Tape: If you have both Non-System Managed Tape and System Managed Tape and would like to do scratch pooling for both or one or the other, use methods described above.

Assigning policies

DFSMSrmm provides these methods for the assignment of retention and management policies:

- Specify the retention method (EXPDT or VRSEL) to be used for the tape volume set.
- Use ACS routines to assign a management class to a data set to allow DFSMSrmm to:
 - Retrieve and use the management class attributes relevant for tape management. DFSMSrmm calls ACS routines directly to provide support for non-system-managed tape if you have enabled it with the EDGRMMxx parmlib option SMSACS. This DFSMSrmm processing enables you to use an SMS management class to manage system-managed and non-system-managed tape data sets. Use EDGRMMxx parmlib option MCATTR to enable and tailor the usage of the management class attributes. The following Management Class attributes can be retrieved:

Management Class attributes	Corresponding attributes used by RMM
Expiration attribute: Expire after Date/Days	Data set EXPDT
Expiration attribute: Expire after Days Non-Usage	Data set LASTREF
Tape Volume Attribute: Retention Method	Volume Retention Method
Tape Volume Attribute: Volume Set Management Level'	Volume RETAINBY
Tape Data Set Attribute: Exclude from VRSEL	Data set VRSELEXCLUDE
Tape Data Set Attribute: Retain While Cataloged	Data set WHILECATALOG

- Match a DFSMSrmm data set vital record specification to the management class name. This processing applies only when the VRSEL retention method is used. DFSMSrmm calls ACS routines directly to provide support for non-system-managed tape, if you have enabled it with the EDGRMMxx parmlib option SMSACS. This DFSMSrmm processing enables you to use an SMS management class to manage system-managed and non-system-managed tape data sets.
- Define DFSMSrmm vital record specifications with data set name masks to identify the retention and movement policies. This method applies only when the VRSEL retention method is used.
- Define a vital record specification management value that DFSMSrmm can use to apply retention and movement policies to data sets. Use the EDG_EXIT100 installation exit described in Chapter 11, “Using DFSMSrmm installation exits,” on page 267 to select a vital record specification management value. You

can use information in the system to determine which vital record specification management value to use. This method applies only when the VRSEL retention method is used.

- During pre-ACS processing, use the EDG_EXIT100 installation exit to obtain information that is used by the ACS routines. The EDG_EXIT100 installation exit can provide a vital record specification management value in the MSPOLICY read-only variable, which is used by the ACS routines. During pre-ACS processing, DFSMSrmm does not make the RMMVRS environment call to the ACS routine. This method applies to both the EXPDT and VRSEL retention methods.
- You can set the EXPDT in one of these ways:
 - Specify RETPD or EXPDT at allocation time or in the JCL
 - Use data class
 - Use the EDG_EXIT100 installation exit
 - Use the defaults table defined in the EDGDEFxx PARMLIB member
 - Use the DFSMSrmm default retention - EDGRMMxx parmlib OPTION RETPD
 - Use the DFSMSrmm ADD and CHANGE command for EXPDT and RETPD
 - From the TCDB tape volume record (VOLCAT)
 - Use the SMS management class expiration attribute 'Expire after Date/Days' (must be enabled by the EDGRMMxx parmlib option MCATTR)
 - Use the LASTREF data set attribute (only for volumes managed by the EXPDT retention method). LASTREF can be set by:
 - the SMS management class expiration attribute 'Expire after Days Non-usage'
 - the default EDGRMMxx parmlib OPTION RM(EXPDT(LASTREF(value)))
 - a DFSMSrmm ADDDATASET or CHANGEDATASET LASTREF command.

DFSMSrmm records how the EXPDT and retention method are set.

Using SMS ACS processing you can have an automatism to set the expiration date and LASTREF extra days of a data set from the management class expiration attributes 'Expire after Date/Days' and 'Expire after Days Non-usage'. You can tailor the usage of the management class attributes by DFSMSrmm with the EDGRMMxx parmlib option MCATTR.

Using SMS ACS processing you can consolidate policy assignment decisions in a single place whether you use system-managed tape or not. You can use SMS ACS routines to assign management class for your data sets instead of using vital record specification management values assigned by the EDG_EXIT100 exit. You can assign a management policy by name to either a non-system-managed or a system-managed tape data set. If the use of the management class attributes is enabled by the EDGRMMxx parmlib OPTION MCATTR, you can use the management class Expiration Attributes ('Expire after Date/Days' and 'Expire after Days Non-usage') to set the data set EXPDT and LASTREF attributes, where appropriate.

For non-system-managed tape, DFSMSrmm calls the ACS routines to obtain a management class. The management class is used in place of the vital record specification management value assigned by the EDG_EXIT100 installation exit. When a management class name is assigned using ACS routines, the EDG_EXIT100 installation exit is not called for a vital record specification management value. The decision to call the EDG_EXIT100 installation exit is made each time a new data set is created on a tape based on whether a construct is assigned by ACS processing. You have the flexibility to identify one request to be handled by ACS and the next request to be handled by the EDG_EXIT100 installation exit. For compatibility, DFSMSrmm passes the vital record specification management value that is determined by the EDG_EXIT100 installation exit by using the pre-ACS interface in the MSPOLICY variable. You might use the vital record specification management value in the MSPOLICY variable as the base for MC assignment for system-managed tape. Even when you use SMS ACS support to assign management class names you can have separate policies for retention and movement by using a primary data set name vital record specification and a secondary management class vital record specification.

Use the ACS routine to assign the management class as the secondary vital record specification and the DFSMSrmm data set name vital record specification to assign the primary vital record specification.

You can still use the EDG_EXIT100 installation exit to check for either EXPDT= or ACCODE= specifying special values and override them to ensure correct retention processing by DFSMSrmm.

Using SMS management class to retain non-system-managed volumes with VRSEL retention method

You can define retention and movement policies using a vital record specification data set name mask and a management class, or a vital record specification management value, can also be assigned to manage a data set.

If you want to use management class expiration attributes, you must enable this by specifying the OPTION MCATTR in EDGRMM parmlib, as described in [“Exploiting SMS management class attributes” on page 93](#).

For each new tape data set, DFSMSrmm performs these tasks:

1. During pre-ACS processing for new tape data sets:
 - a. DFSMSrmm calls the EDG_EXIT100 installation exit for a vital record specification management value. The vital record specification management value provides management policy and retention policy information for data sets.
 - b. DFSMSrmm returns the selected value in the MSPOLICY read-only variable.
2. During OPEN processing:
 - a. DFSMSrmm calls ACS routines and passes environment information. Your ACS routine can set a management class name.
 - b. If ACS processing does not return a management class name, DFSMSrmm calls the EDG_EXIT100 installation exit to obtain a vital record specification management value.
 - c. At OPEN time, DFSMSrmm records the specified management class or vital record specification management value at the data set level.

Making an ACS management class assignment

To assign a management class, you must have the SMS subsystem active and have a valid SMS configuration.

You use the ACS routines to process the special calls that DFSMSrmm makes to the SMS subsystem for ACS processing. DFSMSrmm requests that the management class routine is run. The environment variable is set to RMMVRS so that you can differentiate allocation requests for system-managed data sets from requests by DFSMSrmm for a management class name. When DFSMSrmm calls the ACS routines with the &ACSENVIR variable set to either RMMPOOL or RMMVRS, the storage class (&STORCLAS variable) is set to the word USERMM.

The SMS read-only variables that are set for DFSMSrmm requests are:

- &ACCT_JOB
- &ACCT_STEP
- &ACSENVIR
- &DD
- &DSN
- &DSORG
- &DSTYPE
- &EXPDT
- &FILENUM
- &GROUP
- &HLQ

- &JOB
- &LABEL
- &LIBNAME
- &LLQ
- &NQUAL
- &PGM
- &STORGRP
- &SYSNAME
- &SYSPLEX
- &UNIT
- &USER
- &XMODE

1. For volumes managed by the VRSEL retention method, define vital record specifications for the management class names you plan to use as shown in [Figure 33 on page 90](#).

```

...
RMM ADDVRS DSNAME('CATALOG') WHILECATALOG
RMM ADDVRS DSNAME('ONSITE') DAYS COUNT(31)
RMM ADDVRS DSNAME('OFFSITE') CYCLES COUNT(5) LOCATION(STORE1) -
    NEXTVRS(DAYS90)
RMM ADDVRS NAME(DAYS90) DAYS COUNT(90)
...

```

Figure 33. Defining vital record specifications for non-system-managed volumes

2. You code your selection criteria and rules into the management class ACS routine as shown in [Figure 34 on page 90](#).

```

PROC 1 MGMTCLAS
...
/*****
/*      RMM POLICY FILTER LISTS          */
*****/
FILTLIST POOLA INCLUDE(USERA.**,USERY.KEEP.*) -
    EXCLUDE(USERA.POOLB.***)
...
IF &ACSENVIR = 'RMMVRS' THEN
/*****
/*      RMM VRS ASSIGNMENT DECISIONS      */
*****/
DO
    SELECT (&DSN)
        WHEN (&OFFSITE) SET &MGMTCLAS = 'OFFSITE'
        WHEN (&ONSITE) SET &MGMTCLAS = 'ONSITE'
    END
    /* CATCH EXPDT=99000                      */
    IF &EXPDT = '1999000' THEN SET &MGMTCLAS = 'CATALOG'
...
END
...
END

```

Figure 34. Sample management class routine for managing non-system-managed volumes

If ACS processing does not set the management class name, DFSMSrmm processing continues without using management class names.

When your ACS routine sets a management class, no call is made to the EDG_EXIT100 installation exit to obtain a vital record specification management value. However, EDG_EXIT100 is still called to update the DFSMSrmm copy of the JFCB expiration date if necessary.

Managing volumes with special dates

If your current tape management system allows special expiration dates (such as EXPDT=99000) in JCL to identify management and retention requirements for system-managed volumes, you can manage those volumes by using management class to handle the special dates.

You can also use vital record specification management values to manage both system-managed volumes and non-system-managed tape volumes with JCL expiration dates that are specified with special dates. A vital record specification management value is a one-to-eight character name defined by your installation and used to assign management and retention values to tape data sets.

You define the management policies for management classes and for vital record specification management values using vital record specifications.

Use the EDG_EXIT100 installation exit as described in [Chapter 11, “Using DFSMSrmm installation exits,”](#) on [page 267](#) or the defaults table as described in [Chapter 21, Using the parmlib member EDGDEFxx](#) to assign vital record specification management values to new tape data sets.

During inventory management VRSEL processing, DFSMSrmm uses the vital record specification management value as either the primary or secondary VRS, as determined by the VRS matching performed by VRSEL processing.

Using volumes with special expiration dates

If you have volumes with JCL expiration dates specified with special dates, you can use DFSMSrmm to manage those volumes. You use management classes for system-managed volumes. You use vital record specification management values for both system-managed and non-system-managed tape volumes.

For system-managed tape volumes, perform these tasks:

- Define management classes to cover all the special dates used by your installation.
- Update your ACS routine to assign those management classes to data sets on tape.
- Define vital record specifications identifying the management policies for management classes.
- Run DFSMSrmm inventory management vital record processing to identify the volumes that should be retained based on the management classes that you define.

For both system-managed and non-system-managed tape volumes, perform these tasks:

- Define vital record specification management values to cover all the special dates used by your installation.
- Tailor the sample EDGUX100 exit module to assign the vital record specification management values you define to new tape data sets.
- Define vital record specifications identifying the management policies for vital record specification management values.
- Run DFSMSrmm inventory management vital record processing to identify the volumes that should be retained based on the vital record specification management values that you define.

See [“Managing volumes with special dates”](#) on [page 91](#) for more information.

Using management class to retain system-managed volumes managed by VRSEL retention method

This topic presents the steps you need to take to use management class to retain system-managed volumes that are managed by the VRSEL retention method.

Step 1: Define management class names

You need to define management class names that cover all the special dates used by your installation. For example, you can define a management class, M99000, for the special date 99000 and then define a vital record specification using the data set name M99000.

The management class name can be one to eight alphanumeric characters and must begin with an alphabetic character to be acceptable to the z/OS data set naming standards that apply to DFSMSrmm vital record specifications.

Step 2: Update ACS routine

Update your ACS routine to use an appropriate filter list and logic to assign management classes to data sets on tape. When a volume is opened, DFSMSrmm records the management class name you assign to new tape data sets using your ACS routine.

Figure 35 on page 92 shows a management class ACS routine that maps the expiration date 99000 to a management class name M99000, and the expiration dates 99001 through 99009 to the management class name M99009.

```
PROC 1 &MGMTCLAS
/*****
/* DEFINE FILTER FOR KEEP WHILE CATALOGED EXPDT=99000 */
*****/
FILTLIST SPECIAL_DATE9 INCLUDE('1999000')
/*****
/* DEFINE FILTER FOR UP TO 9 CYCLES */
*****/
FILTLIST SPECIAL_DATE9 INCLUDE(1999000*)
/*****
/* DEFINE FILTER FOR TAPE UNITS */
*****/
FILTLIST TAPEUNITS INCLUDE(TAPE*,T3420*,T3480*,T3490*,
                          '3420','3480','3490',
                          T3590*,'3590')
/*****
/* SELECT VALID MANAGEMENT CLASS */
*****/
SELECT
  WHEN (&EXPDT = &SPECIAL_DATE9 && &UNIT = &TAPEUNITS)
    SET &MGMTCLAS = 'M99000'
  WHEN (&EXPDT = &SPECIAL_DATE9 && &UNIT = &TAPEUNITS)
    SET &MGMTCLAS = 'M99009'
  OTHERWISE
    SET &MGMTCLAS = 'NORMAL'
END
END
```

Figure 35. Assigning management class to data sets on tape

Step 3: Define retention policies for management class names

You define policies for management classes by defining vital record specifications. Use the RMM subcommand ADDVRS with the DSNAME operand, or the DFSMSrmm Add Data Set VRS panel in the DFSMSrmm ISPF dialog to define vital record specifications. Use the data set name masks that match the management class names you have defined. See *z/OS DFSMSrmm Managing and Using Removable Media* for more information on defining vital record specifications.

Issue the command shown in Figure 36 on page 92 to define a vital record specification that manages the special date 99000.

```
RMM ADDVRS DSNAME('M99000') WHILECATALOG
```

Figure 36. Special date 99000 vital record specification

You can define a vital record specification data set name mask that matches multiple management class names. For example you could define a data set name mask of M99* and use this mask to cover several management classes. Because the RMM ADDVRS subcommand with DSNAME operand can also be used to define a vital record specification for an individual data set, the management class name mask you specify for ADDVRS DSNAME is first used to match a data set name before any other mask.

Figure 37 on page 93 uses a data set name mask to define a vital record specification to manage any management class starting with M99. For example, with this data set name mask you could manage special dates in the range 99001 through 99365.

```
RMM ADDVRS DSNAME('M99*')
```

Figure 37. Managing management classes using a data set name mask

Step 4: Run DFSMSrmm inventory management vital record processing

Run DFSMSrmm inventory management vital record processing to identify the volumes that should be retained based on the management classes you have defined. DFSMSrmm uses the management class assigned to the data set, rather than the data set name to select the appropriate vital record specifications. See [“How vital record processing works” on page 352](#) for information about vital record processing.

Using the SMS pre-ACS interface

Using the SMS pre-ACS calls to DFSMSrmm, you can use any existing DFSMSrmm and EDG_EXIT100 exit scratch pool and EDG_EXIT100 exit policy assignment decisions as input to your SMS ACS logic to enable you to direct new data set allocations to the correct media. You can use SMS ACS to decide if data sets are to be system-managed or not system-managed. For system-managed data sets, you can decide if they are to be DASD or tape. For system-managed tape data sets, you can decide to which library or library type you would like the allocation directed. This can help you with VTS implementation.

To use DFSMSrmm and EDG_EXIT100 decisions within your ACS processing, use the pre-ACS processing. Implement or customize the sample EDGUX100 exit module (or the EDGCVRSX sample of EDGUX100) to feed pool and VRS management value information into ACS. Base your ACS processing decisions on the MSPPOOL and MSPOLICY values that come from DFSMSrmm scratch pool processing and EDG_EXIT100 exit processing. You can plan to move the EDG_EXIT100 exit decisions into your SMS ACS routines to enable the EDG_EXIT100 exit processing to be ignored or removed some time in the future.

DFSMSrmm always attempts to pass values for the MSPPOOL and MSPOLICY ACS read-only variables. If you do not use an EDG_EXIT100 exit, the MSPPOOL variable is set to the DFSMSrmm system-based scratch pool decision. If you do not use the EDG_EXIT100 exit, there is never a value set for MSPOLICY. DFSMSrmm sets a pre-ACS variable only if the variable has not yet been set using the pre-ACS exit. Since DFSMSrmm gets control after the installation exit, any vendor decisions or any customer decisions take precedence.

Exploiting SMS management class attributes

You can exploit the SMS management class attributes for both system-managed and non-system-managed volumes. For non-system-managed volumes, you must set the EDGRMMxx parmlib option SMSACS(YES).

The EDGRMMxx parmlib option operand MCATTR enables the use of SMS management class attributes that are relevant for tape management by DFSMSrmm. These attributes are the 'Expire after Date/Days', 'Expire after Days Non-usage', 'Retention Method', 'Volume Set Management Level', 'Retain While Cataloged' management class attributes, but not the 'Retention Limit' attribute.

Depending on your MCATTR setup, the management class attributes are:

- Not used at all, if you specify MCATTR(NONE).
- All used as appropriate, if you specify MCATTR(ALL).
- All used as appropriate, except that the 'Expire after Date/Days' and 'VRSEL Exclude' are ignored if the dataset is on a volume managed by the VRSEL retention method, if you specify MCATTR(VRSELXDI). This option is recommended if you want that the processing of volumes managed by the VRSEL retention method will not change starting from release V2R1.

The management class attributes are retrieved by DFSMSrmm during OPEN for output for a new data set. They are used to set the Expiration Date, Last Reference Extra Days, WHILECATALOG and VRSELEXCLUDE values for the data set as well as the Retention Method and RETAINBY for volumes, depending on the MCATTR setting and the volume retention method. This is a one-time action, thus avoiding any overhead of repeating the policy decisions as part of inventory management. At OPEN for input or at OPEN for output

with disposition MOD for an existing data set, the management class attributes are not considered for processing.

Regardless of whether the management class attributes are used or not for a tape data set, the management class name is recorded in the data set record and, if the volume is managed by the VRSEL retention method, it is used for VRS matching during VRSEL processing. You can change the management class name with a TSO RMM CV MANAGEMENTCLASS command, but this will not cause the exploitation of the management class attributes.

The following policy will assign the management class EX2015 to all data sets with the high level qualifier DS1:

```
PROC &MGMTCLAS
/*****
/* RMM POLICY FILTER LISTS          */
/*****
  FILTLIST TAPEUNITS INCLUDE(TAPE*,T3420*,A3480*,A3490*,
                             *TAPE,'3420','3480','3490E')

/*****
/* SELECT VALID MANAGEMENT CLASS
/*****

SELECT
  WHEN (&HLQ EQ 'DS1')
    SET &MGMTCLAS = 'EX2015'
  OTHERWISE
    SET &MGMTCLAS = ''
END

WRITE 'ACS ROUTINE - MGMTCLAS ASSIGNED: ' &MGMTCLAS

END
```

You need to define a management class with the expiration attributes that you want to assign to your tape data sets. For example, you can define a management class that will set the expiration date of the data set to 2015/12/31, and the LASTREF extra days of the data set to 20. Expiration attributes are retrieved only once when the data set is created. The values are bound to the DFSMSrmm data set record.

```
MANAGEMENT CLASS DISPLAY                               Page 1 of 6

Command ==>

CDS Name      . . . . . : SYS1.SMS.SCDS
Management Class Name : EX2015

Description   : Keep until end of 2015 or as long as referenced

Expiration Attributes

  Expire after Days Non-usage . : 20
  Expire after Date/Days . . . : 2015/12/31
  Retention Limit . . . . . : NOLIMIT

Use DOWN Command to View next Panel;
Use HELP Command for Help; Use END Command to Exit.
```

If 'Expire after Date/Days' and 'Expire after days Non-usage' are both NOLIMIT then the expiration date of the DFSMSrmm data set record is set to PERMANENT.

If 'Expire after Date/Days' is NOLIMIT and 'Expire after days Non-usage' is set then the expiration date of the DFSMSrmm data set record is set from the value from the default PARMLIB option RETPD.

If 'Expire after days Non-usage' is NOLIMIT then the LASTREF extra days of the DFSMSrmm data set record is set from the default PARMLIB option RETENTIONMETHOD(EXPDT(LASTREF(extra_days))).

Additionally you can set 'Retention Method', 'VRSELEXCLUDE', 'RETAINBY', and 'WHILECATALOG' attributes. For example:

```
MANAGEMENT CLASS ALTER                                     Page 1 of 6
Command ==>

SCDS Name . . . . . : SYS1.RMM.SCDs
Management Class Name . : RMEXPDT

To ALTER Management Class, Specify:

Tape Volume Attributes
Retention Method . . . . EXPDT          (VRSEL, EXPDT or blank)
Volume Set Management LevelFIRSTFILE    (VOLUME, FIRSTFILE, SET or blank)

Tape Data Set Attributes
Exclude from VRSEL . . . . Y            (Y, N or blank)
Retain While Cataloged . . ON          (ON, OFF, UNTILEXPIRED or blank)

Use ENTER to Perform Verification; Use UP Command to View previous Panel;
Use HELP Command for Help; Use END Command to Save and Exit; CANCEL to Exit.
```

Managing volumes with duplicate volume serial numbers

In DFSMSrmm, a duplicate volume has these attributes:

- Is defined to DFSMSrmm with a unique external volume serial number.
- Has a VOL1 label.
- The VOL1 label does not match its own external volume serial number.

You can manage volumes with duplicate volume serial numbers by performing these tasks:

- Define the volumes with duplicate volume serial numbers so that you can use them without changing the volume serial number as described in [“Using volumes with duplicate volume serial numbers” on page 95](#).
- Change the volume serial numbers as described in [“Changing duplicate volume serial numbers” on page 96](#).
- Use the volumes with duplicate volume serial numbers and request that they not be managed by DFSMSrmm. See [“Ignoring duplicate or undefined volume serial numbers” on page 275](#) for information about using EDG_EXIT100 or OPENRULE within EDGRMM to ignore duplicate volume serial numbers.

Note: Although you can define, list, and manage duplicate volumes at any time, DFSMSrmm only fully supports the use and validation of duplicate volumes when you run with OPMODE(Protect).

Using volumes with duplicate volume serial numbers

Define a VLPOOL rack pool to reserve shelf space for duplicate volumes. You can use the next available empty rack number as the unique volume serial number for a new duplicate volume. For example, define a VLPOOL with the PREFIX(D*) to manage duplicate volumes. Volume ABC001 is a duplicate volume because it has standard labels that contain the volser ABC001 and it matches the volume ABC001 that is defined to DFSMSrmm. You can define ABC001 to DFSMSrmm as a duplicate volume by using a unique external volume serial number.

Use the RMM VOLUME subcommands with the VOL1 operand to define a unique volume serial number for each volume that has a duplicate VOL1 volume serial number. Change the external label, and optionally the barcode, to match the new unique volume serial number. In this example, the next available rack in the D* pool is D00010. Use the D00010 value to add the duplicate volume.

For example:

```
RMM ADDVOLUME D00010 STATUS(MASTER) VOL1(ABC001)
```

When you use the DFSMSrmm ISPF dialog to define volumes, DFSMSrmm automatically detects when you have added a volume that duplicates a volume that was already defined to DFSMSrmm. The dialog prompts you with options so that you can add the new volume as a duplicate, or change the existing volume to a duplicate and add the new volume using its existing name.

Example: Obtain information about duplicate volumes defined to DFSMSrmm by issuing the RMM SEARCHVOLUME TSO subcommand. You can also use the DFSMSrmm ISPF dialog by selecting the VOL1 field of the Search Volume panel.

```
RMM SEARCHVOLUME VOLUME(*) VOL1(*) LIMIT(*) OWNER(*)
```

To label a volume as a duplicate volume, use the EDGINERS utility manual processing with the VOL1 operand on the command in the SYSIN file. Then you can label the volume. If DFSMSrmm already knows the correct VOL1 label volume serial number, you do not need the VOL1 operand on the EDGINERS INIT command. Specify the VOL1 operand on the INIT command to override the value recorded in the DFSMSrmm control data set. DFSMSrmm writes this value to the tape label.

Example: Label a volume as a duplicate volume.

```
//MANUAL EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(tape,,DEFER)
//SYSIN DD *
INIT VOLUME(D000001) LABEL(SL) VOL1(ABC123)
/*
```

If the volume is not defined to DFSMSrmm, specify the VOL1 operand to label the volume with a value different from the volume serial number used to define the volume to DFSMSrmm. Add the volume as a MASTER volume rather than a SCRATCH volume. This example shows how to label a duplicate volume using the EDGINERS utility.

Example: Define a volume to DFSMSrmm with the external volume serial number D00001 and label it with the VOL1 label volume serial number ABC123.

```
INIT VOLUME(ABC123,D00001) VOL1(ABC123) LABEL(SL) POOL(F*) -
MEDIANAME(3590)
```

When you use EDGINERS automatic processing, the new volume label is written so that the VOL1 label volume serial number matches the volume serial number and the VOL1 label volume serial number value recorded by DFSMSrmm is cleared.

Changing duplicate volume serial numbers

If you want DFSMSrmm to manage volumes with duplicate volume serial numbers and you cannot remove one of the duplicate volumes serial numbers, you must change volume serial numbers.

Add the duplicate volume serial number, as follows:

- For volumes with no label (NL), file the duplicate volume under a different unique volume serial number. Assign the volume a volume serial number equal to the shelf location, change its external label, and inform the owners of the change.
- For IBM standard label (SL) volumes and ISO/ANSI label (AL) volumes, move all data sets on the volume to a different, unique volume. Change the external labels and RACF profiles for the new volume. Inform the owners of the change. The owners must catalog the data sets on the new volume if the data sets were cataloged on the original volume.

You might need to perform this step if you have received volumes that contain software products.

Example: Set the initialize action for a volume that is defined to DFSMSrmm.

```
RMM CHANGEVOLUME D00001 INIT(Y)
```


Adding a duplicate volume into a system-managed tape library

To add a duplicate volume into a system-managed tape library, do these steps:

1. Define a duplicate volume in order to get an unique external volume serial number.
2. Change the barcode on the cartridge to match the external volume serial number.
3. Enter the volume into the system-managed tape library.

Managing undefined volume serial numbers

DFSMSrmm does not manage volumes that are not defined to DFSMSrmm. These types of volumes are also known as foreign tape volumes. You do not need to have volumes that are foreign to DFSMSrmm because you can define any volume to DFSMSrmm including duplicates. DFSMSrmm always rejects foreign volumes when they are mounted in response to a request for a scratch tape. To control the use of foreign tapes for non-scratch requests, you can set up DFSMSrmm so that DFSMSrmm rejects foreign volumes when they are opened.

To have DFSMSrmm reject foreign volumes, specify (in the DFSMSrmm EDGRMMxx parmlib member) an OPENRULE command with TYPE(NORMM) and an action of REJECT. You can use OPENRULE for any set of volumes that are identified with the VOLUME and VOLUMERANGE operands. See [“Controlling the use of tape volumes: OPENRULE” on page 157](#) for more information about the OPENRULE command.

To prevent DFSMSrmm from rejecting undefined or foreign volumes when you use the OPENRULE command with action REJECT, also use an action of IGNORE to request that DFSMSrmm ignore the undefined volumes. All volumes that are ignored with the IGNORE action must be authorized to be ignored. When DFSMSrmm ignores a volume, DFSMSrmm does not perform these tasks:

- Record information about the volume in the DFSMSrmm control data set.
- Perform management functions for the volume.

When DFSMSrmm is configured to ignore a volume, DFSMSrmm does, however, perform these tasks:

- Checks your authorization to ignore the volume using profiles in the FACILITY class.
- DFSMSrmm validates that the correct volume has been mounted in all cases, including bypass label processing(BLP). The only exception is for the case when both the mounted and the requested volumes are with no label (NL). If the wrong volume is inserted, EDG4064D is issued to ask the operator whether RMM should ignore the volume, or reject it.
- If you are authorized to ignore the volume by the FACILITY class profiles, this overrides any decision that is taken by RACF during OPEN processing.

Note: OPENRULE is the preferred command for rejecting and ignoring volumes. The previously used REJECT command and EDGUX100 exit module continue to work if they have already been specified.

Segregating WORM tapes in separate scratch pools

When you use scratch pooling for non-system-managed tape or use a manual tape library, ensure that WORM tapes are in a separate scratch pool. If you mix WORM tape and non-WORM tape in a scratch pool, you cannot control the type of tape that will be mounted for a non-specific volume request.

Chapter 5. Running DFSMSrmm with system-managed tape libraries

DFSMSrmm provides this support for system-managed tape libraries:

- Cartridge entry processing
- Cartridge eject processing
- Expiration management
- Volume-not-in-library support
- Movement between libraries and storage locations
- Partitioning the libraries with other systems
- Tracking and managing logical, physical, and stacked volumes
- VTS import/export and TS7700 copy export processing

If you have volumes in an automated tape library, you need to reserve shelf space outside the automated tape library if you plan to eject the volumes and retain them in a manual tape library or non-system-managed tape library. Volumes in a manual tape library can be logically ejected, left in the same shelf location and used as non-system-managed volumes.

DFSMSrmm identifies all volumes including volumes that reside in system-managed tape libraries by using the volume serial number and the rack number.

- The volume serial number is the internal volume serial number recorded in VOL1 label on SL and AL type labels.
- The rack number is the external volume serial number

For volumes that reside in system-managed tape libraries, a rack number is optional. If you specify a rack number for a volume residing in a system-managed tape library, the volume serial number and the rack number must be the same. You cannot change the rack number for volumes that are associated with system-managed libraries. You cannot define a rack number for logical volumes.

You can manage volume movement between system-managed libraries and storage locations, between system-managed libraries, and between non-system managed libraries and system-managed libraries by defining vital record specifications.

This topic describes how to use DFSMSrmm with system-managed tape volumes. Refer to the EDGRMMxx OPTION command SMSTAPE operand described in [“Defining system options: OPTION”](#) on page 161 for details about controlling the functions that are provided by DFSMSrmm.

Using DFSMSrmm with system-managed tape libraries

DFSMSrmm automatically adds volumes to the DFSMSrmm control data set during entry processing and when you use volumes and when no REJECT ANYUSE command for the volumes is specified. If a volume defined to DFSMSrmm is entered into a system-managed tape library without using the RMM CHANGEVOLUME command to set the volume location to the library name, DFSMSrmm updates the DFSMSrmm control data set volume record with the library name, library type, and volume entry status. DFSMSrmm does not update the home location name for the volume.

If the volume is not defined to DFSMSrmm and is entered into an automated tape library, DFSMSrmm automatically defines the volume in the DFSMSrmm control data set with information it obtains from the OAM entry user exit parameter list as follows: LOCATION, STATUS, STORGRP, MEDIATYPE, RECORDINGFORMAT, SPECIALATTRIBUTES, COMPACTION, READDATE, WRITEDATE, EXPDT, LIBRARY NAME, TYPE, ENTRY STATUS, and HOME LOCATION. This means that all volumes entered into an automated tape library must be managed by DFSMSrmm, unless the library is to be partitioned and

certain volumes left undefined. See [“Partitioning system-managed tape libraries” on page 125](#) for details on partitioning the system-managed tape library.

Recommendation: Define all volumes to DFSMSrmm before entering them into a library. Define private volumes prior to entry into the automated tape library and set the ISMF default cartridge entry status to scratch.

If you are inserting cartridges without predefining them to DFSMSrmm, and the default entry use attribute is private, then DFSMSrmm assigns a default owner. The default owner is set to the RACF User ID associated with the DFRMM started procedure name.

Associating volumes and system-managed libraries

Specify a system-managed tape library name for a volume by using the DFSMSrmm ISPF dialog or the RMM ADDVOLUME or RMM CHANGEVOLUME subcommand with the LOCATION operand. [Figure 38 on page 100](#) shows how to use the ADDVOLUME subcommand to specify the name of an automated tape library, ATL1, where volume A00001 resides:

```
RMM ADDVOLUME A00001 LOCATION(ATL1) STATUS(VOLCAT)
```

Figure 38. Specifying a library name for a volume

You can provide the library name even if the volume does not yet reside in a system-managed tape library. When you provide a library name, DFSMSrmm checks if the library name is already defined in the TCDB. If the library name is defined in the TCDB, processing continues. If the library name is not defined, then processing fails.

DFSMSrmm records the library name and the library type; manual tape library or automated tape library. DFSMSrmm also records whether the volume is currently in the library.

If the volume is not defined in the TCDB, and the library name specified is an automated tape library, then DFSMSrmm sets the volume destination to the library name and waits for the volume to be entered. If the volume is not defined in the TCDB, and the library name specified is a manual tape library, then DFSMSrmm uses manual cartridge entry processing so the TCDB is updated for the volume.

Cartridge entry processing

DFSMSrmm is called at cartridge entry time by the CBRUXENT installation exit which DFSMSrmm supplies. During cartridge entry processing in an automated tape library, the installation exit passes the external volume serial number. DFSMSrmm checks for the existence of a volume or a rack with the same number. When there is a conflict, the cartridge entry is denied. DFSMSrmm sets a return code telling OAM not to process the volume on this system under these conditions.

- If a volume is defined to DFSMSrmm as "not for use on z/OS".
- If a volume is not defined to DFSMSrmm and it matches the REJECT ANYUSE prefixes, and is entered into a tape library. If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. See [“Converting REJECT commands to PRTITION and OPENRULE commands” on page 204](#) for information about converting from the use of REJECT commands.

If a volume is entered into a library that does not match the library name already defined for it, DFSMSrmm updates the library name in the control data set. The home location name is not updated. If a volume is 'intransit' and the destination is different than the library the volume is entering, the cartridge entry is prevented.

Manual cartridge entry processing

DFSMSrmm uses manual cartridge entry processing to add information about a volume residing in a manual tape library into the TCDB. The OAM macro CBRXLCS is used to get the volume information added to the TCDB when a volume is to reside in a manual tape library. The information provided by DFSMSrmm

using CBRUXENT, is the same as for automated libraries, and includes: storage group name, recording format, compaction, special attributes, owner name, and expiration date.

DFSMSrmm uses the volume's media type as defined to DFSMSrmm when requesting manual cartridge entry; you must ensure that DFSMSrmm has the correct media type to avoid allocation and mount problems when the volumes are later used. You use the RMM ADDVOLUME or CHANGEVOLUME subcommands with the MEDIATYPE operand to set the media type.

When volumes are entered into a manual tape library, DFSMSrmm ensures that both private and scratch volumes meet these DFSMS requirements: that only supported label types are used and that internal volume serial number matches external volume serial number.

Managing scratch pools

The DFSMSrmm scratch pool function in VLPOOL definitions is not honored for scratch mount requests on automated system-managed library resident drives, even if the storage group assigned matches a VLPOOL name. In an automated system-managed library, DFSMS and the library manager control the pooling of scratch volumes with one pool for each media type. There is currently no method to direct mount processing to select a subset of the volumes in a scratch category.

For scratch mount requests on manual system-managed library resident drives, you can decide to implement scratch pools based on storage group names or on any other pooling method provided by DFSMSrmm. When you are pooling based on the storage group name assigned by your ACS routine, any mounted volume must be from that storage group; either in a VLPOOL associated with a tape storage group or the volume has the requested storage group name. When you are pooling using any other method provided by DFSMSrmm, the storage group names are ignored for validation. DFSMSrmm records the storage group name assigned by the SMS ACS routine and ignores any existing storage group name for that volume.

When system managed volumes are moved from being system-managed to non-system-managed, be sure to clear the storage group name unless you plan to continue using storage group pooling. For non-system-managed scratch mount requests that do not request a volume by storage group, DFSMSrmm rejects any volume that already has a storage group name.

Ejecting volumes from system-managed libraries

You can eject physical volumes from a system-managed library by using various methods. One way is to list a number of volumes and then enter eject commands from the list. You normally use export processing to eject logical volumes. For information about export processing, see [“Using DFSMSrmm with the IBM totalstorage virtual tape server”](#) on page 108.

You can also eject volumes from a system-managed library by changing the library name with the RMM CHANGEVOLUME subcommand. When you issue the command, DFSMSrmm requests that the volume is ejected from the original library. Information about the volume is changed in the control data set, and the TCDB record is optionally deleted. If the target library is an automated tape library, the operator must physically transfer the volume. If the source and target libraries are both manual tape libraries, then physical movement is only needed if the volume is to be moved to another shelf location.

You can also mark a volume as ejected from a system-managed library or change the location name to SHELF when the library is not known or offline by using the RMM CHANGEVOLUME subcommand with the FORCE operand. To use the FORCE operand, you must have access to the STGADMIN.EDG.MASTER and STGADMIN.EDG.FORCE RACF profiles. When you use the FORCE operand to update a volume in a system-managed library, DFSMSrmm attempts to update the TCDB. Your request to update DFSMSrmm is completed even when the TCDB is not updated.

DFSMSrmm does not automatically eject volumes from system-managed libraries after completing vital record processing and storage location management processing. See [“How storage location management processing works”](#) on page 357 for information about adding a job step to eject volumes at a time you determine.

The DFSMSrmm control data set is updated even if ejects are not driven by DFSMSrmm. The eject is detected using the OAM eject installation exit, and DFSMSrmm can control the disposition of the

TCDB volume information. During eject processing, DFSMSrmm checks the setting of the DFSMSrmm EDGRMMxx OPTION SMSTAPE(PURGE) operand to determine whether to request deletion of volume information from the TCDB. This optionally overrides the ISMF default cartridge eject option you might have specified. If the volume record is not deleted, DFSMSrmm sets the volume destination information into the TCDB volume shelf location field.

Figure 39 on page 102 shows how you can eject a volume from an automatic tape library or from a manual tape library to a convenience output station using the RMM CHANGEVOLUME subcommand or the RMM DELETEVOLUME subcommand.

```
RMM CHANGEVOLUME A12345 EJECT(CONVENIENCE)
RMM CHANGEVOLUME A12345 LOCATION(new_lib)
RMM CHANGEVOLUME A12345 LOANLOC(set_loc)
RMM DELETEVOLUME A12345 REMOVE
RMM DELETEVOLUME A12345 FORCE
```

Figure 39. Requesting the eject of a volume

You can specify the EJECT operand on the RMM CHANGEVOLUME and DELETEVOLUME subcommands. Specify the EJECT(BULK) operand to direct the ejected volume to the high capacity input/output station. Specify the EJECT(CONVENIENCE) operand to direct the ejected volume to the convenience output station. If you specify EJECT(BULK) and the high capacity output station is not installed, then the eject is routed to a convenience output station. If there is no convenience output station, the eject is then routed to a single cell facility. In a manual tape library, DFSMSrmm ignores the BULK values and the CONVENIENCE values and the cartridge is ejected.

The RMM CHANGEVOLUME subcommand with the HOME operand does not cause an eject or a move as it is only used to update the home location name.

Returning volumes to the system-managed library

Normally system-managed volumes remain resident in the system-managed library, unless they are removed for movement to an offsite location for physical inspection or for error recovery. When volumes are to be returned to an automated system-managed library, enter the volumes into the appropriate entry station. DFSMSrmm is notified that the move has taken place. When volumes are returned to a manual system-managed library, you must confirm the movement to DFSMSrmm so that DFSMSrmm can initiate entry processing.

There might be times when a volume is not in a system-managed library but is needed for processing to continue. z/OS includes a function called Volume-Not-In-Library (VNL) processing that allows a volume to be identified for re-entry to a system-managed library at various stages of system processing:

- Job step setup
- Device allocation
- Library mount

The CBRUXVNL OAM installation exit provides support so that a volume that is needed for processing to continue can be entered back into the library. Logical volumes that are exported must be imported to enable them to be used for processing.

For volumes that are defined in the DFSMSrmm control data set, information is provided to determine if the volume can be entered into the library. DFSMSrmm issues messages that provide location, movement, and status information during VNL processing.

If the volume entering the library is marked as "intransit" to other than a system-managed library, ensure that the move for the volume is confirmed as completed. Otherwise the volume entry will be rejected.

Refer to [“Managing system-managed tape library volumes: EDGLCSUX” on page 251](#) for information on using CBRUXVNL and other OAM installation exits.

When you regularly store system managed volumes outside of a system managed library and also have the home location set to SHELF (or some other LOCDEF-defined storage location with a type of HOME), you can ensure that the volumes will be called for by CBRUXVNL by using either the:

- SMSTAPE(PURGE(NO)) option, or
- SMSTAPE(PURGE(ASIS)) option and ensure that the library eject default is set to KEEP. This enables you to be selective about the libraries to which the purge/keep option applies.

Either option results in the TCDB volume entry being retained while the volume is ejected. The DFSMSrmm supplied sample exit checks if the TCDB entry exists and, for DFSMSrmm managed volumes, will prompt the operator to re-enter the volume into a library.

Volume-not-in-library processing

When a volume is requested for a job and is not in a system-managed tape library, the DFSMSrmm supplied CBRUXVNL exit is called so the volume can be inserted into a library to prevent job failures from occurring. The CBRUXVNL exit can be modified to suit your requirements.

When called, the supplied, sample CBRUXVNL exit determines whether the request is for a tape volume. If so, CBRUXVNL calls EDGLCSUX to retrieve the DFSMSrmm information for the volume. An informed decision can be made when catalog information is available that confirms whether a volume is a tape or not. To obtain the best results for your installation, you might need to customize CBRUXVNL to change the criteria for calling EDGLCSUX, which is partly selectable by conditional assembly to enable easy customization by you. By default, the initial call to EDGLCSUX is made in these cases:

- A TCDB entry for the volume exists.
- Job information is unavailable.
- Catalog information is available, and the device type is a tape device. When catalog information is available, but the device type is not tape, the exit skips the link to EDGLCSUX.

DFSMSrmm provides the possibility to further enhance the EDGLCSUX calls by providing an option to compare the JCL esoteric unit name against a list of your known tape unit names. You can enable this additional option by customizing the default processing by:

- Turning on the tape unit name checking code by changing the setting &TAPEDEC SETC to 'YES', and
- Customizing the hardcoded list of tape unit names. The tape unit names are listed in the shipped table called TAPEUNITS. Ensure that you include in this table all the tape unit names your users code in their JCL. You can include esoteric, generic, and specific unit names. The default table is also shown.

```
TAPEUNITS DC      A(TAPFIRST,8,TAPLAST)
*****
* Tape units list - may be customized
*****
TAPFIRST DC      CL8'TAPE'
* insert your location tape unit names here
* or edit any entry
      DC      CL8'3400-6'
      DC      CL8'3420-8'
      DC      CL8'CART'
      DC      CL8'3480'
      DC      CL8'3480X'
      DC      CL8'3490'
      DC      CL8'3490E'
      DC      CL8'3590'
      DC      CL8'3590-1'
      DC      CL8'3590-E'
      DC      CL8'3590-B'
      DC      CL8'3590-H'
      DC      CL8'3590L'
      DC      CL8'3592'
      DC      CL8'3592-J'
      DC      CL8'SYS3480R'
      DC      CL8'SYS348XR'
      DC      CL8'VTS'
      DC      CL8'MTL'
      DC      CL8' '      any blank entry is ignored
TAPLAST  DC      CL8'ATL'
```

Figure 40. Contents of the shipped table: TAPEUNITS

When you have &TAPEDEC set to 'YES', CBRUXVNL checks the supplied unit name and calls EDGLCSUX only in the case where it matches some tape unit name within the TAPEUNITS table. Otherwise, the EDGLCSUX call is skipped.

In addition, the decision to issue message EDG8197I for non-RMM managed volumes is made selectable by an option flag in the EDGLCSUP parameter list. The CBRUXVNL exit shipped by DFSMSrmm sets this option flag ON when a TCDB entry of the volume exists, when the device type supplied by catalog is a tape device, or when &TAPEDEC is set to 'YES'. Otherwise, this option flag is OFF. To change this decision, you can customize the CBRUXVNL source code. This ensures that you will see message EDG8197I when you believe the volume is a tape volume, and the tape volume is not managed by DFSMSrmm.

CBRUXVNL calls DFSMSrmm once to retrieve volume information, and then for a limited set of circumstances, it calls DFSMSrmm a second time to request the operator to enter the volume into a system-managed tape library. The second call is made under these conditions:

- CBRUXVNL is called from other than job setup.
- CBRUXVNL is called from job setup and one of the following is true:
 - A TCDB entry for the volume exists, or
 - &ALWAYSPROMPT set to 'YES', or
 - The volume resides in a system-managed library, or
 - The volume needs to be directed to a system-managed tape library, or
 - The volume is an exported logical volume, or
 - The volume resides in location SHELF or in a location with type STORE,HOME, and is not moving to a storage location nor a SHELF location, and its home location is system-managed

The CBRUXVNL exit does not allow a volume to be used under these conditions:

- If the volume resides in a loan location and &FAILONLOAN set to 'YES' and the volume is system-managed, the exit fails the request.
- The volume is pending release or is in scratch status.
- The initialize volume action is pending.
- The volume is not to be used on MVS.

CBRUXVNL checks for the requested volume as follows:

- If the volume resides in a loan location and the volume is system-managed, the exit fails the request.
- The exit attempts to get a volume entered into a system-managed library if the volume can be used on the system and when:
 - The volume is in location SHELF and is not moving to a storage location, and its home location is system-managed.
 - DFSMSrmm volume information indicates the volume resides in a system-managed library.
 - The volume destination is a system-managed location.

DFSMSrmm uses the information to build a WTOR message that includes the volume location and that prompts the operator to enter the volume into the system-managed library. For volumes that reside in a VTS, DFSMSrmm returns additional information for a volume. DFSMSrmm indicates if the volume is a physical volume or a logical volume and provides the stacked volume on which the logical volume is exported.

The CBRUXVNL exit does not allow a volume to be used under these conditions:

- The volume is pending release or is in scratch status.
- The initialize volume action is pending.
- The volume is not to be used on z/OS.

In all other cases, the CBRUXVNL exit allows the volume to be used, but a non-system managed tape drive is allocated by the system.

The sample CBRUXVNL exit includes simple customization options that you can easily change with little or no assembler language knowledge:

&ALWAYSPROMPT

Controls when CBRUXVNL is to call DFSMSrmm a second time to request the operator to enter the volume into a system-managed tape library. The default processing in the sample exit does not need to be changed unless you do not retain TCDB volume entries when volumes are ejected.

NO

CBRUXVNL calls DFSMSrmm a second time to request the operator to enter the volume into a system-managed tape library only for a limited set of circumstances (as previously described). NO is the default.

YES

A DFSMSrmm-managed volume is always to be re-entered into a system managed library, regardless of the system managed volume information defined to DFSMSrmm and regardless of the presence of a TCDB.

&API

Controls whether the DFSMSrmm API can be used within the system exit to obtain DFSMSrmm information about volumes and data sets.

NO

CBRUXVNL does not permit use of the DFSMSrmm API within the system exit routine. NO is the default.

YES

CBRUXVNL is to permit the use of the DFSMSrmm API within the system exit routine. The customer must provide the necessary changes within the exit routine to invoke the desired DFSMSrmm commands and process their output. See the sample CBRUXVNL for more information.

&FAILONLOAN

Controls the handling of requests to use volumes that are on loan.

YES

Specifies that attempts to use volumes that are on loan should be failed in PROTECT mode, but allowed with a warning in WARN mode. YES is the default.

NO

Disables the on loan check and allows them to be processed. If you need to process volumes not defined to DFSMSrmm that duplicate those defined to DFSMSrmm, and the volumes defined to DFSMSrmm are on loan, it is best to select the NO value. The EXPDT=98000 required to have DFSMSrmm ignore the undefined volume is processed after the CBRUXVNL processing has occurred.

&TAPEDEC

Controls when CBRUXVNL is to call DFSMSrmm, based on the tape unit name.

NO

Disables the tape unit name check. CBRUXVNL does not use the supplied unit name when deciding whether to call EDGLCSUX. NO is the default.

YES

Turns on the tape unit name check. CBRUXVNL calls EDGLCSUX only when the supplied unit name matches a tape unit name in the TAPEUNITS table.

Confirming volume movement for system managed libraries

DFSMSrmm automatically confirms volume moves during cartridge entry processing. When you manually confirm the moves for volumes to an automated tape library, DFSMSrmm checks for the volume and library name in the TCDB and only confirms movements if the volume is library resident. DFSMSrmm If there is a mismatch between the information in the TCDB and the DFSMSrmm control data set, use the RMM CHANGEVOLUME subcommand with the LOCATION operand to update the destination information in the control data set to match the library name in the TCDB.

If a volume is returning to a manual tape library, there is no automatic confirmation. You must explicitly confirm that the move has taken place when the volume is entered into the library. When you confirm that a volume has moved to a manual tape library, DFSMSrmm requests that the volume location in the TCDB is updated with the new location. DFSMSrmm also changes the volume location information in the TCDB when you issue the DFSMSrmm `CHANGEVOLUME volser LOCATION (mtl_name)` to change the volume location to a manual tape library.

If you specify the RMM `CHANGEVOLUME` subcommand with the `EJECT` operand for a volume for which no move has been identified, DFSMSrmm does not set a destination for the volume but the volume is set to be 'intransit'. Once the volume is ejected and cannot be entered again, you should specify the location where the volume is placed and then confirm the move.

When you specify the RMM `CHANGEVOLUME` subcommand with the `LOCATION` operand, DFSMSrmm sets a destination for the volume. You only need to confirm the move after the eject.

Defining system-managed volume information

System-managed tape volumes are defined in the tape configuration database (TCDB). The TCDB is a catalog that is marked as a volume catalog (VOLCAT) containing tape volume and tape library records. For more information about the TCDB, see [z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries](#).

System-managed tapes should be defined to the DFSMSrmm control data set for the volumes to be managed by DFSMSrmm. To define volumes to both DFSMSrmm and the TCDB, use the RMM `ADDVOLUME` subcommand for volumes not yet known to DFSMSrmm, and the RMM `CHANGEVOLUME` subcommand for volumes already defined to DFSMSrmm but not the TCDB. For information on using DFSMSrmm commands, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

When you use DFSMSrmm to request that a volume should move to a system-managed tape library, DFSMSrmm ensures volume information is recorded in the TCDB.

Keeping system-managed volume information consistent

DFSMSrmm uses information in the TCDB to verify requests or to obtain information about volumes that are being added to DFSMSrmm. When you define volumes to DFSMSrmm by using the RMM `ADDVOLUME` subcommand with the `STATUS(VOLCAT)` operand, DFSMSrmm uses TCDB information to update the DFSMSrmm control data set. When there is a conflict between information in the TCDB and information in the control data set, DFSMSrmm uses the value in the TCDB.

To control the way DFSMSrmm updates the information in the TCDB, specify the DFSMSrmm `EDGRMMxx` parmlib `OPTION` command `SMSTAPE` operand and the `OPTION` command `OPMODE` operand. See “Defining system options: `OPTION`” on page 161 for information about the DFSMSrmm `EDGRMMxx` parmlib `OPTION` command. When you run DFSMSrmm in `PROTECT` mode, DFSMSrmm updates the TCDB. If you are running DFSMSrmm in other modes, you can decide if and when you want DFSMSrmm to update the TCDB. DFSMSrmm updates this TCDB information:

Volume owner information

DFSMSrmm uses the first 8 characters of the 64 bytes of owner information in the TCDB to store the DFSMSrmm volume owner information. DFSMSrmm does not overlay any information you might have entered into the first 8 bytes. Keep the first bytes of the field blank and DFSMSrmm maintains the volume information for you.

Expiration date

The expiration date is updated in the tape volume record:

- When RMM subcommands are used to change the status of a volume
- During OAM installation exit processing, as described in [Table 34 on page 256](#).

Volume use attribute (status)

DFSMSrmm updates this information when an RMM TSO subcommand is issued and a volume's status is changed. For example, if you issue the RMM `CHANGEVOLUME` subcommand with the `STATUS(USER)` operand or the RMM `GETVOLUME` subcommand, the TCDB volume use attribute is

changed to PRIVATE. Volume status is also changed when volumes are returned to scratch status during inventory management.

Storage group name

DFSMSrmm updates this information when an RMM TSO subcommand is issued to change TCDB information.

You can use the DFSMSrmm EDGUTIL utility to perform these tasks:

- Check the consistency of the control data set with the TCDB and the library manager database and use EDGUTIL MEND(SMSTAPE) to synchronize the TCDB and library manager database from the DFSMSrmm control data set.
- Mend information in the control data set based on information in the TCDB and the library manager database.

For more information about EDGUTIL, see [“Using EDGUTIL for tasks such as creating and verifying the control data set” on page 401.](#)

Use the RMM CHANGEVOLUME subcommand with the LOCATION(*mtl_name*) operand to rebuild manual tape library information in the TCDB.

If you use DFSMSrmm, ISMF, or operator commands to eject a volume, DFSMSrmm notes that the volume is 'intransit'. During eject processing, DFSMSrmm checks the setting of the DFSMSrmm EDGRMMxx OPTION SMSTAPE(PURGE) operand to determine whether to request deletion of volume information from the TCDB. This optionally overrides the ISMF default cartridge eject option you might have specified. If the volume record is not deleted, DFSMSrmm sets the volume destination information into the TCDB volume shelf location field.

Initializing scratch volumes in system-managed libraries

You can use these methods to label volumes in an automated tape library:

- Use EDGINERS or IEHINITT to label them before they are entered.
- For scratch volumes, let the volume be labeled the first time that the volume is used for output after it enters the automated tape library. DFSMSdftp performs labeling during OPEN processing.
- Use EDGINERS or IEHINITT for volumes in all system-managed tape libraries.

To label volumes by using the EDGINERS initialize function, set the DFSMSrmm initialize action by using the DFSMSrmm ISPF dialog or the RMM TSO subcommand.

If you set the initialize action for a scratch volume before it enters an automated tape library and then enter the volume into the automated tape library, DFSMSrmm defers initialization to DFSMSdftp labeling support. If you later eject the scratch volume before it is used DFSMSrmm reinstates the initialize action.

If you set the initialize action for a scratch volume while it is resident in an automated tape library, the initialize action is set outstanding and you must use the DFSMSrmm EDGINERS utility to create the labels. See [“Using EDGINERS with system-managed tape libraries” on page 438](#) for additional information.

When you use the DFSMSdftp labeling support in an automated tape library, DFSMSdftp obtains the volume serial number from the library vision system, but still issues the WTOR IEC704A to prompt the operator to provide optional owner information. The IEC704A message text includes an indicator that the volume is system managed. Normally, DFSMSrmm provides information to bypass the issuing of the message. If DFSMSrmm is running in record mode or a volume is rejected in warning mode, the WTOR is issued and must be replied to by the operator.

The DFSMSdftp labeling support for manual tape libraries cannot obtain the volume serial number from the vision system. DFSMSdftp still uses the WTOR IEC704A to prompt the operator for it.

You can decide to exploit DFSMSdftp automatic labeling to avoid unnecessary mounting of volumes or use EDGINERS before volumes enter the tape library or after volumes are resident in the tape library.

Using storage group names

You can specify a storage group name for each volume defined in the control data set. Setting the name is optional.

DFSMSrmm validates the storage group name by using services that are provided by the Storage Management Subsystem. For system-managed volumes, DFSMSrmm passes the storage group name to OAM unless the volume is in scratch status. For non-system managed volumes, you can use the assigned storage group names for scratch pooling. For more information about using SG for scratch pooling, see [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82.

When a volume is moved from one system-managed tape library to another, DFSMSrmm does not change the storage group name. OAM validates the storage group name during entry to the new library and fails the entry if the storage group is not defined for use in that new library.

DFSMSrmm uses the OAM installation exits to record any changes that ISMF and Automatic Class Selection Routine processing make to the storage group name. This function enables DFSMSrmm to provide the storage group name at cartridge re-entry time, if the volume information had been removed from the TCDB.

For information on how DFSMSrmm records data class, management class, storage class, and storage group information, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

Using DFSMSrmm with the IBM totalstorage virtual tape server

DFSMSrmm supports Virtual Tape Server (VTS) libraries as automated system-managed libraries with extensions to that support to handle the special requirements for different volume types and functional capability. DFSMSrmm supports virtual tape server import/export and copy export in different ways, depending on whether or not DFSMSrmm stacked volume support is enabled.

When you enable stacked volume support, DFSMSrmm associates exported logical volumes with stacked volumes and manages movement of stacked volumes. When you do not enable stacked volume support, DFSMSrmm tracks export and copy export only if you have previously defined stacked volumes, but does not manage movement of stacked volumes.

DFSMSrmm supports the IBM TS7700 grid and IBM Peer-to-Peer VTS implementations. With these, you can use the names of distributed VTS libraries with DFSMSrmm. For other libraries, you must only use the names of consolidated libraries with DFSMSrmm.

Related reading:

- See [z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries](#) for information about using OAM for virtual tape server support.
- See [z/OS DFSMSrmm Reporting](#) for information about the DFSMSrmm EDGRPTD utility that you can use to obtain information about volumes that reside in virtual tape server libraries.
- See [IBM TotalStorage Enterprise Automated Tape Library \(3494\) Introduction and Planning Guide](#), [IBM TotalStorage Enterprise Automated \(3494\) Tape Library Operator Guide](#), [IBM System Storage TS3500 Tape Library Introduction and Planning Guide](#), and [IBM System Storage TS3500 Tape Library Operator Guide](#) for information about the format for the import and export lists.

Defining logical volumes in a virtual tape server library

A logical volume is a volume that resides in a virtual tape library either on DASD (in the tape volume cache as a virtual volume) or on a stacked volume (as a logical volume) and is referenced from the host as if it were a physical volume.

The way you define and enter logical volumes is common regardless of how you set up DFSMSrmm. Other functions are dependent on how you request stacked volumes to be managed. For details of how DFSMSrmm supports export and import processing for logical volumes refer to [“DFSMSrmm support for stacked volumes when stacked volume support is enabled”](#) on page 112 and [“DFSMSrmm support for stacked volumes when stacked volume support is not enabled”](#) on page 118.

When volumes are automatically added to DFSMSrmm, DFSMSrmm sets the volume type based on where the volume resides. If the volume resides in a VTS, DFSMSrmm sets the volume type to logical. If the volume does not reside in a VTS, DFSMSrmm sets the volume type to physical. You can chose to define the volumes to DFSMSrmm rather than having the volumes automatically defined when they are entered into the library. Specify the correct volume type for each volume that you define. If you do not specify the correct volume type for volumes residing in system-managed tape libraries, these volumes will not be processed successfully. TYPE(LOGICAL) can also be set for non-system managed virtual tape libraries. However, non-IBM virtual tape libraries being managed as manual tape libraries are considered system managed and should not set the logical indicator.

To ensure that all logical volumes are correctly identified to DFSMSrmm as logical volumes, use the DFSMSrmm TSO subcommand example shown in [Figure 41 on page 109](#) to change the volume type of previously defined volumes. DFSMSrmm allows you to identify any volume as a logical volume as long as it is not resident in a system-managed automated tape library.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(*) LOCATION(vts) -
CLIST('RMM CHANGEVOLUME ',' TYPE(LOGICAL)')
EXEC EXEC.RMM.CLIST
```

Figure 41. Changing volume type

Logical volume cartridge entry processing

DFSMSrmm processes new logical volumes and imported volumes differently at entry time.

- DFSMSrmm automatically defines both new logical volumes and imported logical volumes if they are not defined to DFSMSrmm already. DFSMSrmm does not define rack numbers for these volumes.
- DFSMSrmm checks the rack number for a logical volume that is already defined to DFSMSrmm. If the rack number does not match the logical volume's volume serial number, DFSMSrmm fails the entry request and issues message EDG8189I. If the rack number matches the volume serial number, DFSMSrmm continues processing the request. DFSMSrmm frees the rack number for the logical volume so the logical volume is no longer associated with the rack number.
- DFSMSrmm checks that an imported logical volume that is already defined to DFSMSrmm, is also a logical volume and does not reside in a virtual tape server library. When a logical volume is being imported, DFSMSrmm checks that the volume, if it is already defined to DFSMSrmm, is correctly defined as an exported logical volume. DFSMSrmm issues message EDG8183I when the entry request fails.
- DFSMSrmm accepts new scratch logical volumes that match existing scratch logical volumes. DFSMSrmm updates the volume information with information for the new volume. If the volume serial numbers match but the volume is not defined as a scratch volume, DFSMSrmm fails the entry request and issues message EDG8182I.

Note: If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. See [“Converting REJECT commands to PRTITION and OPENRULE commands” on page 204](#) for information about converting from the use of REJECT commands.

[Table 12 on page 109](#) describes the entry processing decisions DFSMSrmm makes.

Table 12. DFSMSrmm entry processing decisions

Case	Physical Volume	Stacked Volume	Logical Volume	Imported Volume
Volume is not defined; no REJECT.	Added	Not applicable	Added scratch or private	Added private
Volume is not defined; REJECT.	Ignored	Not applicable	Ignored	Ignored
Volume is defined for use with z/OS.	Updated	Not applicable	Updated	Updated
Volume is defined for use with VM.	Ignored	Not applicable	Ignored	Ignored
Rack number associated with the volume is the same as the volume serial number.	Updated	Not applicable	Updated; DFSMSrmm clears the rack number.	Updated; DFSMSrmm clears the rack number.

Table 12. DFSMSrmm entry processing decisions (continued)

Case	Physical Volume	Stacked Volume	Logical Volume	Imported Volume
Rack number associated with the volume is not the same as the volume serial number.	Entry fails; DFSMSrmm issues EDG8189I.	Not applicable	Entry fails; DFSMSrmm issues EDG8189I.	Import fails; DFSMSrmm issues EDG8189I.
No rack number is present.	Updated	Not applicable	Updated	Updated
Volume is a scratch and not logical volume.	Updated	Not applicable	Entry fails; DFSMSrmm issues EDG8182I.	Import fails; DFSMSrmm issues EDG8182I.
Volume is private and not a logical volume.	Updated	Not applicable	Entry fails; DFSMSrmm issues EDG8182I.	Import fails; DFSMSrmm issues EDG8182I.
Volume is a logical volume and not an exported volume.	Entry fails; DFSMSrmm issues EDG8184I.	Not applicable	Update	Import fails; DFSMSrmm issues EDG8183I.
Volume is a logical exported volume.	Entry fails; DFSMSrmm issues EDG8184I.	Not applicable	Entry fails; DFSMSrmm issues EDG8184I.	Updated
Volume destination is not the library.	Entry fails; DFSMSrmm issues EDG8192I.	Not applicable	Entry fails; DFSMSrmm issues EDG8192I.	Import fails; DFSMSrmm issues EDG8192I.
Volume owner is not valid.	Entry fails; DFSMSrmm issues EDG8195I.	Not applicable	Entry fails; DFSMSrmm issues EDG8195I.	Import fails; DFSMSrmm issues EDG8195I.
Defined rack number=vol is not available.	Entry fails; DFSMSrmm issues EDG8198I.	Not applicable	Entry fails; DFSMSrmm issues EDG8198I.	Import fails; DFSMSrmm issues EDG8198I.
Maximum retention period is exceeded.	Entry fails; DFSMSrmm issues EDG8196I.	Not applicable	Not applicable	Import fails; DFSMSrmm issues EDG8196I.

Managing stacked volumes

A *stacked volume* is a volume in a virtual tape server library that is used to store one or more logical volumes.

DFSMSrmm can manage volume movement based on the logical volume or the stacked volume. You control the way DFSMSrmm manages stacked volumes by enabling stacked volume support as described in [“Enabling stacked volume support” on page 120](#). If you do not enable stacked volume support, you can always use the DFSMSrmm TSO subcommands to add, change, or delete information about stacked volumes. You can also perform export, copy export, and import processing of logical volumes, but DFSMSrmm provides no management for the movement of the stacked volumes.

If you do not enable stacked volume support, DFSMSrmm bases volume movement on the logical volume. DFSMSrmm records the stacked volume as the in container value but does not use the stacked volumes to determine volume movement. See [“DFSMSrmm support for stacked volumes when stacked volume support is not enabled” on page 118](#) for details about how DFSMSrmm supports stacked volumes when stacked volume support is not enabled.

If you enable stacked volume support, DFSMSrmm tracks stacked volumes and bases volume movement on the stacked volume. See [“DFSMSrmm support for stacked volumes when stacked volume support is enabled” on page 112](#) for details about how DFSMSrmm supports stacked volumes when stacked volume support is enabled.

If you define stacked volumes to DFSMSrmm, export, copy export, and import always updates the stacked volume information, regardless of the enablement of stacked volume support

Stacked volumes normally start and end their movement in a VTS automated tape library. The host system is not notified when stacked volumes enter or leave an automated tape library, so the movement

of stacked volumes cannot be managed in the same way that physical volumes in an automated tape library are managed.

When you use a DFSMSrmm command for a stacked volume, DFSMSrmm checks both the TCDB and the library manager database to see if the volume is known. If the volume is not known to the library manager, you cannot define the volume as being in the VTS library, but the destination is set to the VTS library name. Once the volume is entered into the VTS, you can use the CHANGEVOLUME subcommand with CONFIRMMOVE to inform DFSMSrmm that the volume is now in the VTS.

When a stacked volume containing exported logical volumes is ejected from the library, as the logical volumes expire, DFSMSrmm places the volumes in pending release status. When the logical volumes are imported back into the library, DFSMSrmm completes the return to scratch process enabling the volumes to be reused. As the exported logical volumes expire, you can do off-site stacked volume management to determine when to bring a stacked volume back on-site for possible reuse. Use REPORT17: Inventory of Stacked Volumes by Percent Active to report on the percentage of active data on a stacked volume and the percentage of active logical volumes on a stacked volume. This EDGRRPTE REXX report, REPORT17, is a helpful tool to aid with stacked volume management. See [z/OS DFSMSrmm Reporting](#) for additional information on REPORT17.

Defining stacked volumes to DFSMSrmm

Before you begin: Use the library manager console to ensure that the ranges of stacked volumes for containers are defined correctly. By doing so, you can avoid accidentally entering stacked volumes into the library as physical volumes.

Recommendation: Always define stacked volumes to DFSMSrmm. This ensures that the volumes cannot be used outside of the VTS. It also ensures that DFSMSrmm checks at cartridge entry processing time that a physical volume or logical volume does not duplicate a stacked volume.

You can define stacked volumes whether stacked volume support is enabled or not. Once stacked volumes are defined, DFSMSrmm records export, copy export, and import results only for the predefined stacked volumes. Once stacked volume support is enabled, DFSMSrmm automatically defines exported and copy exported stacked volumes to record details of this activity. There is no automatic host notification when stacked volumes enter the VTS, leave the VTS, or change category, so you must use DFSMSrmm commands if you want to have all stacked volumes defined to DFSMSrmm.

To define stacked volumes to DFSMSrmm and ensure that they can only be used as stacked volumes inside a VTS, perform these steps:

1. Issue the RMM ADDVOLUME subcommand with the TYPE(STACKED) operand to manually define stacked volumes to DFSMSrmm.
 2. Take one of these steps to prevent the use of stacked volumes outside the VTS:
 - Specify the DFSMSrmm EDGRMMxx parmlib command OPENRULE VOLUMERANGE('start':'end') TYPE(ALL) ANYUSE(REJECT) to prevent the use of ranges of stacked volumes.
 - Specify the DFSMSrmm EDGRMMxx parmlib command REJECT ANYUSE(*) to prevent the use of any volumes not defined to DFSMSrmm. See [“Defining tapes not available on systems: REJECT”](#) on page 202.
- Note:** If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. See [“Converting REJECT commands to PRTITION and OPENRULE commands”](#) on page 204 for information about converting from the use of REJECT commands.
- Specify the DFSMSrmm EDGRMMxx parmlib command REJECT ANYUSE(prefix*) to prevent the use of ranges of stacked volumes.
 - Specify the RMM ADDVOLUME subcommand with the USE(TM) operand and the TYPE(STACKED) operand to manually define stacked volumes that cannot be used on z/OS to DFSMSrmm.

Changing stacked volume information

Once you have defined a stacked volume to DFSMSrmm you do not normally need to change information about that volume. DFSMSrmm manages the information about stacked volumes and logical volumes during export, import, and copy export, but stacked volume support must be enabled if you want DFSMSrmm to manage the movement of stacked volumes as they are exported or copy exported from the VTS library.

You do need to use the CHANGEVOLUME command to confirm the movement of the stacked volume as you would for a physical volume that was moving.

Assigning a shelf location for a stacked volume

You can optionally assign a shelf location to a stacked volume. When a stacked volume is resident in a VTS library, no shelf location is needed; however, if it is stored outside the VTS assigned to location SHELF you can assign a rack number using either the RACK or POOL operands. Specify RACK with a specific rack number or use POOL to allow DFSMSrmm to select the first empty rack number in that pool of shelf locations. During normal processing, DFSMSrmm storage location management processing assigns shelf locations to a stacked volume that must move to a shelf-managed storage location.

Deleting stacked volume information

You can only delete an empty stacked volume. Ensure that all logical volumes have been imported or removed from the stacked volume. To remove a volume from a stacked volume, use the RMM CHANGEVOLUME subcommand with the CONTAINER(' ') operand to clear the container name.

DFSMSrmm support for stacked volumes when stacked volume support is enabled

With DFSMSrmm stacked volume support enabled, you can manage the movement of logical volumes by using stacked volumes and manage the movement of copy exported stacked volumes. To enable stacked volume support, see [“Enabling stacked volume support” on page 120](#). When stacked volume support is enabled, DFSMSrmm tracks each logical volume export saving the highest priority logical volume required location in the stacked volume required location.

To ensure that stacked volumes are managed correctly, DFSMSrmm inventory management processing (this applies to VRSEL/DSTORE/EXPROC only) checks that stacked volumes reside in a library.

At the completion of export processing for a single stacked volume, DFSMSrmm uses the required location of the stacked volume to attempt to start its movement. If the required location is not shelf-managed, DFSMSrmm sets the destination, but does not yet mark the stacked volume as being 'intransit'. The next time inventory management is run, DFSMSrmm checks the library to see if the stacked volume has been ejected, and if so, the stacked volume is marked as being 'intransit'.

If the required location is shelf-managed, DFSMSrmm storage location management processing starts the volume move. Storage location management processing starts movement of stacked volumes by assigning bin numbers for storage locations and setting the destination for the volume. Inventory management checks to see if closed, not-empty, or copy exported stacked volumes still reside in their location, and marks those no longer resident as 'intransit'.

Stacked volumes that are 'intransit' to storage locations are confirmed only if a global confirm move has been requested for the location and destination pair. Later, when stacked volumes move from storage locations they are set 'intransit' by storage location management processing. After apply of PTFs for APAR OA63639, stacked volumes that are 'intransit' to a destination VTS are not checked for library residence and the move can be confirmed with either global or volume-specific confirm move commands.

The result is that DFSMSrmm can automatically determine some of the moves for stacked volumes and needs you to confirm those to storage locations.

Resolving movement conflicts

If there is a movement conflict for multiple logical volumes in a stacked volume, DFSMSRmm resolves the conflict by using location priority to determine where the stacked volume should be moved. See [“Moving volumes” on page 355](#) for information about how DFSMSRmm prioritizes volume movement.

Confirming stacked volume movement

You confirm the movement of stacked volume containers to storage locations when their movement is completed. Use the RMM CHANGEVOLUME * CMOVE subcommand. The READYTOSCRATCH option is not applied to stacked volume moves and container moves are confirmed either with the ALL option or the NOTREADYTOSCRATCH option.

Using a stacked volume

The stacked volume is used to identify the media that is used for the export container volume, to track its location, and to aggregate the logical volumes contained in the volume. All movement for exported logical volumes is managed and tracked using the stacked volume. You can move a stacked volume manually, but normally you control the movement automatically using vital record specifications. The vital record specifications identify movement for data sets on logical volumes. The movement specified for the logical volumes in the stacked volume container directs the movement of the stacked volume. DFSMSRmm automatically defines stacked volumes at EXPORT time. DFSMSRmm keeps track of the "in container" information only while the logical volume is exported.

DFSMSRmm support for export processing when stacked volume support is enabled

Note: The information in this topic applies only to VTS import/export processing; it does not apply to TS7700 copy export processing.

For best results, and to ensure that stacked volumes are moved to the location determined by DFSMSRmm inventory management vital record processing, ensure that EDGHSKP has been run with the VRSEL option before starting export processing. As logical volumes are exported, DFSMSRmm tracks the "in container" stacked volume and drives the stacked volume movement based on the required location of the contained logical volumes.

DFSMSRmm export processing support is based on the following process steps, which are recommended:

1. Define vital record specifications that specify locations to which data sets in the VTS should be moved. Use the vital record specification location name as the key for managing export. You could associate location names with different types of retention to have more control over the volumes that you want to export. The intention is that all the volumes with the same required location name, in the same export request, get exported together. In step 3, you will use the required location to help build the export list, but you can use your own selection of 'location names' in the export list.
2. Run DFSMSRmm inventory management vital record processing to determine which logical volumes should be moved and to set the required location for each volume to be moved.
3. Create a list of volumes to export after you run vital record processing. The required moves are volume moves that are identified during vital record processing or by changes made when you issue the RMM CHANGEVOLUME subcommand. Use the RMM SEARCHVOLUME subcommand with the REQUIRED and CLIST operands for each storage location to obtain volume information for the list as shown in [Figure 42 on page 114](#). DFSMSRmm returns a list of volumes to export that you can use as input for creating the export list logical volume file 1. Consider using the RETDATE operand to group the volumes that are expiring in the same time period into the same export request. You can direct the CLIST created from the SEARCHVOLUME using the RMMCLIST DD name to a data set allocated in the format required by EXPORT processing. Otherwise the volume list must be reformatted; the EDGJIMPC sample JCL provides an example of how to do this.

In this example, the target storage location is 'STORE1'. This is specified for the REQUIRED operand. Substitute your VTS library name for 'vts' in the LOCATION operand. The CLIST operand causes a

CLIST record to be written for each logical volume that matches the search; the record contains 'volser STORE1'. You can specify any value instead of STORE1, the library uses this value to group volumes on to stacked volumes by destination.

```
RMM SEARCHVOLUME VOLUME(*) OWNER(*) LIMIT(*) REQUIRED(STORE1) LOCATION(vts) -  
CLIST(' ', ' STORE1') RETDATE(2001/020)
```

Figure 42. Searching the required location for logical volumes

See *z/OS DFSMSrmm Managing and Using Removable Media* for information about the RMM SEARCHVOLUME subcommand. See these examples in SYS1.SAMPLIB for the format and required files for the export list volume:

- CBRSPSXP is the sample JCL for export list volume scratch request.
- CBRSPXP is the sample JCL for export list volume private request.

When stacked volume support is enabled, you can no longer track the movement of logical volumes using the LOCATION and DESTINATION fields of the volume information. Any volume exported on a stacked volume resides in a container and has no assigned location name. DFSMSrmm continues to manage the volume movement using vital record specifications for data sets and logical volumes, but uses this information only to set the REQUIRED location. DFSMSrmm never assigns a destination to a logical volume.

4. Start the export process as described in the *z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries* using the LIBRARY EXPORT command, the EDGSPLCS utility, or the CBRSPSXP sample program.

The VTS export process runs asynchronously to DFSMSrmm processing. Once the export request has been initiated, VTS library signals trigger DFSMSrmm actions related to export processing.

During the VTS export process, logical volumes are copied to a stacked volume and the stacked volume is completed. For each logical volume copied to the stacked volume, DFSMSrmm is notified of both the logical volume and stacked volume. DFSMSrmm updates the DFSMSrmm control data set with the volume serial number of the stacked volume as the 'in container'.

During export processing, DFSMSrmm updates the 'In container' value for each logical volume exported. DFSMSrmm updates the stacked volume and tracks export completion, marking the stacked volume closed.

As export completes, DFSMSrmm uses the required location of the stacked volume to attempt to start its movement. If the required location is not shelf-managed, DFSMSrmm sets the destination, but does not yet mark the stacked volume as being 'intransit'.

5. On completion of export processing, use the export status list file to verify that all the logical volumes are processed successfully.
6. You are now ready to manage the movement of the exported stacked volumes. The inventory management functions required depend on whether your storage locations are bin managed. If all the target storage locations are not bin managed, the stacked volumes already have their destination set. You can create a report extract and continue with the next step. If any of the target storage locations are shelf managed:
 - Set the destinations for the volumes by running inventory management storage location management processing.
 - Create a report extract.

The above function can be a single run of DFSMSrmm inventory management.

7. Create movement report and pick lists by using the DFSMSrmm EDGRPTD report utility.
8. Use the movement reports to eject the stacked volumes from the VTS using the library manager and move them to the destination storage location. Transfer the stacked volumes in the export hold category to the exit station using the library manager console. Physically move the stacked volumes listed in your movement report from the VTS exit station to the destination storage location.

9. Confirm that volumes have been moved. When the volumes have been moved, use the RMM CHANGEVOLUME subcommand, as shown in [Figure 43 on page 115](#) to confirm the completion of the movement of volumes.

```
RMM CHANGEVOLUME * CONFIRMMOVE(vts,ALL)
```

Figure 43. Confirming volume moves for exported volumes

The next run of inventory management processes the global confirmation of movement and updates all the stacked volumes that you have moved to reflect the completion of movement. If you want to update the stacked volume movement status in the CDS, run inventory management now.

10. After apply of PTFs for APAR OA63639 it is now possible to manually CONFIRM MOVE a stacked volume that is returned to an SMS tape library.

DFSMSrmm support for import processing when stacked volume support is enabled

DFSMSrmm supports the importing of logical volumes that are defined to DFSMSrmm or not defined to DFSMSrmm. During import processing for a logical volume, DFSMSrmm automatically adds the volume information to the DFSMSrmm control data set or leaves the volume to be processed on another system. DFSMSrmm does not automatically add rack numbers for logical volumes because rack numbers are not supported for logical volumes.

You can initiate the import of logical volumes as follows.

1. Use the library manager console to transfer stacked volumes to be imported from the Unassign category to the Import category. See *IBM System Storage TS3500 Tape Library Introduction and Planning Guide* and *IBM System Storage TS3500 Tape Library Operator Guide* for more information about the library manager.
2. Create an import list. See these examples in SYS1.SAMPLIB for the format and required files for the import list volume.
 - CBRSPSIM is the sample JCL for import list volume scratch request.
 - CBRSPPIIM is the sample JCL for import list volume private request.

To create the volume list for file 1 of the import list logical volume, you can search in the DFSMSrmm control data set, tailor the DFSMSrmm report extract data set, or use any other method to identify the volumes to be imported.

You can use the RMM SEARCHVOLUME subcommand to build the list of logical volumes with their containing stacked volume and status as shown in these examples. When you specify the TYPE(LOGICAL) operand, DFSMSrmm returns the container volume serial number, logical volume serial number, and volume status between your specified CLIST prefix and suffix strings. Use the RMMCLIST DD to direct the CLIST output to a data set of the correct record format or use the EDGJIMPC sample to reformat it. You can then use the SEARCHVOLUME output file as input for creating the import list logical volume file 1.

When stacked volumes are returned to the library, you can build import lists by either using the logical volumes or using the stacked volumes. To build a list of stacked volumes to be imported, you can issue the DFSMSrmm TSO subcommand shown in [Figure 44 on page 115](#).

```
RMM SEARCHVOLUME VOLUME(*) OWNER(*) LIMIT(*) TYPE(STACKED) DESTINATION(vts) -  
CLIST
```

Figure 44. Building a list of stacked volumes to be imported from a single stacked volume

To build a list of logical volumes to import from a single stacked volume, you can issue the DFSMSrmm TSO subcommand shown in [Figure 45 on page 116](#).

```
RMM SEARCHVOLUME CONTAINER(S12345) VOLUME(*) OWNER(*) LIMIT(*) TYPE(LOGICAL) CLIST
```

Figure 45. Building a list of logical volumes to be imported from a single stacked volume

To build a list of logical volumes to import from multiple stacked volumes, you can issue the DFSMSrmm TSO subcommand shown in [Figure 46 on page 116](#).

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(*) CONTAINER(*) -  
REQUIRED(vts) TYPE(LOGICAL) CLIST
```

Figure 46. Building a list of logical volumes to be imported from multiple stacked volumes

3. Request import processing by using the EDGSPLCS utility, the CBRXLCS macro, the OAM CBRSP LCS sample programs, or the LIBRARY command.

During import processing, the imported volumes can be automatically defined to DFSMSrmm or the volumes can be left for processing by another system if the library is partitioned. If the volume is already known to DFSMSrmm, the volume is accepted for processing if it is defined as an exported logical volume, otherwise the volume is rejected.

4. To complete the import process, DFSMSrmm uses the cartridge entry processing installation exit to track logical volumes that are imported by the VTS. OAM calls the exit once for each logical volume imported. For logical volumes, DFSMSrmm removes the stacked volume association. The stacked volume is updated to remove information for each imported volume.
5. When all logical volumes on a stacked volume are imported, the count of contained volumes is set to 0. You now must decide what to do with the stacked volume. For example, you can return the scratch volume to the pool of empty stacked volumes for use by the VTS; use the library manager console to transfer empty stacked volumes to other categories within the VTS so they are ready for reuse.

Copy export from the TS7740 Virtualization Engine (3957-V06 or 3957-V07)

A copy of the data within a TS7740 Virtualization Engine (3957-V06 or 3957-V07) can be removed from the library, both in a stand-alone and in a multi-grid configuration. Copy export enables a copy of the data to be used for disaster recovery purposes while the original data remains accessible for production use. For the recovery of the copy exported volumes, a modified disaster recovery process is used instead of an import. The copy export support creates a secondary copy of a logical volume in a secondary physical pool. The information in this section discusses the copy export capability provided by DFSMSrmm.

For information on copy export support, refer to *z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries*, the IBM Redbook, *IBM System Storage Virtualization Engine TS7700: Tape Virtualization for System z Servers*, and the following IBM Whitepapers: *IBM Virtualization Engine TS7700 Series Bulk Volume Information Retrieval User's Guide* and *IBM Virtualization Engine TS7700 Series Copy Export User's Guide*.

When a stacked volume associated with a copy export operation is ejected from a library (placed in export-hold or is physically ejected from the library) status message E0006 is issued by the library. RMM intercepts this message and:

- If the stacked volume is not pre-defined to DFSMSrmm and stacked volume support is not enabled, no further processing is performed by DFSMSrmm.
- If the stacked volume is not pre-defined to DFSMSrmm and stacked volume support is enabled, DFSMSrmm automatically adds the stacked volume to its CDS.
- For pre-defined and newly-defined volumes DFSMSrmm sets the location and home location, the volume type, status, assigned date and time and marks it eligible for automatic movement.
- When the E0006 message includes the 'IN EJECT' text the volume is marked intransit to indicate that the volume is being ejected and the volume movement date is set.

Then, during inventory management (which occurs only if stacked volume support is enabled), RMM invokes the CBRXLCS QVR interface to determine when an export-hold volume has been ejected.

The following E0006 messages are issued by the library and appear as part of the message text for message CBR3750I.

```
E0006 STACKED VOLUME YYYYYY FROM LLLLLLLL IN EJECT (microcode prior to 3.2)
E0006 STACKED VOLUME YYYYYY FROM LLLLLLLL QUEUED FOR EJECT (microcode 3.2 to 4.1)
E0006 STACKED VOLUME YYYYYY FROM LLLLLLLL IN EJECT-QUEUE (microcode 4.1.2 and above).
```

And will be displayed at the host as part of the message text for message CBR3750I:

```
CBR3750I Message from library library-name: message.
```

The results of volume processing for EXPORT-HOLD and EJECT are as follows:

When the volume is not yet defined to DFSMSrmm, and the STACKEDVOLUME(YES) control option is in use, the DFSMSrmm processing is as if the following command was used to add the stacked volume:

```
RMM AV volser TYPE(STACKED) STATUS(MASTER) MEDIATYPE(ETC) RECORDINGFORMAT(EFMT1) OWNER(EXPORT)
DESC('Created by Export') EXPDT(99365) HOME(lib) LOCATION(lib)
```

For copy export volumes (both pre-defined and newly defined volumes) in EXPORT-HOLD, the DFSMSrmm processing is as if the following command was used to add the stacked volume:

```
RMM CV volser TYPE(STACKED) STATUS(MASTER) LOCATION(lib) HOME(lib)
RMM CV volser AUTOMOVE CLOSE ASDATE(date) ASTIME(time)
```

For copy export volumes (both pre-defined and newly defined volumes) in EJECT the DFSMSrmm processing is as if the following command was used to add the stacked volume:

```
RMM CV volser TYPE(STACKED) STATUS(MASTER) LOCATION(lib) HOME(lib)
RMM CV volser EJECT AUTOMOVE CLOSE ASDATE(date) ASTIME(time)
```

To have DFSMSrmm policy management manage the retention and movement for volumes created by copy export processing, stacked volume support must be enabled. You must also define one or more volume VRs. For example, assuming all copy exports are targeted to a range of volumes STE000-STE999 you could define a VRS;

```
RMM AS VOLUME(STE*) COUNT(99999) LOCATION(location)
```

When stacked volume support is enabled, and you want DFSMSrmm to manage the movement of copy exported stacked volumes you must run inventory management vital records and storage location management processing. (EDGHSKP with parameters including VRSEL and DSTORE). The required steps are:

1. Perform copy export either to EXPORT-HOLD or to EJECT
2. When all required copy exports are completed eject the stacked volumes in EXPORT-HOLD from the VTS using the library manager.

Note: If your TS7700 Virtualization Engine. is running microcode lower than Release 1.7 PGA4 (8.7.0.143) do not yet remove the ejected volumes from the I/O station. Otherwise, DFSMSrmm will not be able to determine the status of the volumes during inventory management correctly and will not set Intransit ON.

3. Run EDGHSKP with VRSEL and DSTORE. Optionally include a report extract to enable movement reports to be created.
4. Create movement report and pick lists by using the DFSMSrmm EDGRPTD report utility.
5. Physically move the stacked volumes listed in your movement report from the VTS exit station to the destination storage location.
6. Confirm that volumes have been moved. When the volumes have been moved, use the RMM CHANGEVOLUME subcommand, as shown in [Figure 48 on page 119](#) to confirm the completion of the movement of volumes.

A copy exported stacked volume can become eligible for reclaim based on the reclaim policies defined for its secondary physical volume pool or through the host console request command (LIBRARY REQUEST). When it becomes eligible for reclaim, the exported stacked volume no longer contains active data and can be returned from its offsite location for re-use. When the TS7700 Virtualization Engine has successfully

completed reclaim processing for a previously exported stacked volume, one of the following messages will be issued by the library based on the TS7700 micro code level and also appears as part of the message text for CBR3750I:

```
CBR3750I Message from library library-name: R0000 RECLAIM SUCCESSFUL FOR EXPORTED STACKED VOLUME volser.
```

```
CBR3750I Message from library library-name: R0000 RECLAIM SUCCESSFUL FOR EXPORTED STACKED VOLUME volser - YOU CAN RETURN IT TO THE SPECIFIED LIBRARY LLLLLLLL.
```

Note: This message is generated only for reclaims initiated by LIBRARY REQUEST. This message is not issued for reclaims of copy exported volumes that were made eligible through normal pool policy.

RMM intercepts this status message and, if a move is possible and required, marks the volume to be moved back to the library. The results of volume processing for a reclaimed volume are as if the following commands were issued:

```
RMM CV volser HOME(library-name)
```

Then, if the volume is not library resident, and is not currently moving, the following command:

```
RMM CV volser LOCATION(HOME)
```

Finally, the volume's required location is set to the library-name to ensure that if any further movement is needed, the correct destination can be set by inventory management DSTORE processing and the volume is placed under manual move control. The use of the MANUALMOVE setting ensures that the volume will not be moved again by existing VRS policies until reused for a later copy export.

Reclaim support is provided regardless of whether stacked volume support is enabled.

DFSMSrmm support for stacked volumes when stacked volume support is not enabled

A stacked volume is a volume in a virtual tape server library that is used to store one or more logical volumes. When stacked volume support is not enabled, DFSMSrmm does not track stacked volumes, but allows you to define them to DFSMSrmm. DFSMSrmm records the name of the exported stacked volume as the 'In container' information for each logical volume. DFSMSrmm keeps track of the 'In container' information only while the logical volume is exported.

DFSMSrmm support for export processing when stacked volume support is not enabled

Note: The information in this topic applies only to VTS import/export processing; it does not apply to TS7700 copy export processing.

To use DFSMSrmm for export processing, you must first make sure that volume information reflects the most current location information. Perform DFSMSrmm inventory management storage location processing to set the destinations for any volume moves that are required.

As logical volumes are exported, DFSMSrmm tracks the 'in container' stacked volume and drives movement based on the destination of logical volumes. Use DFSMSrmm export processing to remove logical volumes from a VTS.

1. Define vital record specifications that specify locations to which data sets in the VTS should be moved.
2. Run DFSMSrmm inventory management vital records and storage location management processing to determine which logical volumes should be moved. DFSMSrmm allocates a bin number to the volume if the volume's destination is shelf-managed. There will be unused bin numbers for logical volumes that are moving to shelf-managed storage locations. You will waste bin numbers because EDGRPTD ignores the assigned bin numbers for logical volumes.
3. Create a list of volumes to export. Create the list after you run storage location management. The moves are those that are identified during vital record processing or those changes that are

made when you issue the RMM CHANGEVOLUME subcommand. Use the RMM SEARCHVOLUME subcommand to obtain volume information for the list as shown in [Figure 47 on page 119](#).

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) LOCATION(vts)-  
  DESTINATION(dest) CLIST(' ','',dest') -  
  INTRANSIT(N)
```

Figure 47. Creating a volume export list

DFSMSrmm returns a list of volumes to export that you can use as input for creating the export list logical volume file 1. See these examples in SYS1.SAMPLIB for the format and required files for the export list volume.

- CBRSPSXP — sample JCL for export list volume scratch request.
- CBRSPXP — sample JCL for export list volume private request.

You can track the movement of logical volumes by using the LOCATION field and DESTINATION field of the volume information.

4. Start the export process as described in the *z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries* using the LIBRARY EXPORT command or use the CBRSPCLS sample program.

The VTS export process runs asynchronously to DFSMSrmm processing. Once the export request has been initiated, VTS library signals trigger DFSMSrmm actions related to export processing.

During the VTS export process, logical volumes are copied to a stacked volume, and the stacked volume is completed. For each logical volume copied to the stacked volume, DFSMSrmm is notified of both the logical volume and stacked volume. DFSMSrmm updates the DFSMSrmm control data set with the volume serial number of the stacked volume as the 'in container'.

5. After export VTS processing is completed, you can run DFSMSrmm inventory management report extract processing to obtain information about the volumes that were processed.
6. Create movement report and pick lists by using the DFSMSrmm EDGRPTD report utility.
7. Use the movement reports to eject the stacked volumes from the VTS using the library manager and move them to the destination storage location. Transfer the stacked volumes in the export hold category to the exit station using the library manager console.
8. Physically move the stacked volumes listed in your movement report from the VTS exit station to the destination storage location.
9. Confirm that volumes have been moved. When the stacked volume is moved, you must confirm the move for all the logical volumes that reside on the volume. Use the RMM CHANGEVOLUME subcommand as shown in [Figure 48 on page 119](#) to confirm the completion of the movement of volumes.

```
RMM CHANGEVOLUME * CONFIRMMOVE(vts,ALL)
```

Figure 48. Confirming volume moves for exported volumes

DFSMSrmm support for import processing when stacked volume support is not enabled

DFSMSrmm supports the importing of logical volumes that are defined to DFSMSrmm or not defined to DFSMSrmm. During entry processing for a logical volume, DFSMSrmm automatically adds the volume information to the DFSMSrmm control data set or leaves the volume to be processed on another system. DFSMSrmm does not automatically add rack numbers for logical volumes because rack numbers are not supported for logical volumes.

You can initiate the import of logical volumes independently of DFSMSrmm by using the library manager console and creating an import list volume.

1. Use the library manager console to transfer stacked volumes to be imported from the Unassign category to the Import category. See *IBM System Storage TS3500 Tape Library Introduction and*

Planning Guide and *IBM System Storage TS3500 Tape Library Operator Guide* for more information about the library manager.

2. Create an import list volume. See these examples in SYS1.SAMPLIB for the format and required files for the import list volume.
 - CBRSPSIM -- sample JCL for import list volume scratch request.
 - CBRSPDIM -- sample JCL for import list volume private request.

To create the volume list for file 1 of the import list logical volume, you can search in the DFSMSrmm control data set, tailor the DFSMSrmm report extract file tailoring, or use any other method to identify the volumes to be imported.

You can use the RMM SEARCHVOLUME subcommand to build the list of logical volumes with their containing stacked volume and status as shown in [Figure 49 on page 120](#). Specify the TYPE(LOGICAL) operand and DFSMSrmm returns the container volume serial number, logical volume serial number, and volume status between your specified CLIST prefix and suffix strings. The resultant output file can be used as input for creating the import list logical volume file 1 after it is reformatted using the EDGJIMPC sample.

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(*) DESTINATION(vts) -  
TYPE(LOGICAL) CLIST
```

Figure 49. Creating a volume import list

3. Request import processing by using the CBRXLCS macro, the OAM CBRSPICS sample programs, or the LIBRARY command.

During import processing, the imported volumes can be automatically defined to DFSMSrmm or the volumes can be left for processing by another system if the library is partitioned. If the volume is already known to DFSMSrmm, the volume is accepted for processing if it is defined as an exported logical volume, otherwise the volume is rejected.

4. To complete the import, DFSMSrmm uses the cartridge entry processing installation exit to track logical volumes that are imported by the VTS. OAM calls the exit once for each logical volume imported. DFSMSrmm removes the stacked volume association for logical volumes. DFSMSrmm removes the 'in container' value for the logical volumes.
5. When all logical volumes on a stacked volume are imported, you must decide what to do with the stacked volume. For example, you can return the scratch volume to the pool of empty stacked volumes for use by the VTS; use the Library Manager console to transfer empty stacked volumes to other categories within the VTS so they are ready for reuse.

Enabling stacked volume support

With DFSMSrmm stacked volume support enabled, you can manage the movement of logical volumes using stacked volumes. Without stacked volume support enabled, you can define and list stacked volumes using the DFSMSrmm ADDVOLUME TSO subcommand with the TYPE(STACKED) operand. DFSMSrmm inventory management ignores any stacked volumes that you have defined and uses just the container name in the volume records.

Prior to enabling stacked volume support, consider these conditions:

- You cannot disable stacked volume support once it is enabled.
- Do not enable support unless all systems using a control data set support stacked volumes. If the control data set is shared with a lower level and support is enabled you can create inconsistent information in the control data set. Correct inconsistencies by using the DFSMSrmm EDGUTIL MEND(VOLUME) utility before you can run inventory management.

To enable stacked volume support, perform these tasks:

1. Update all systems sharing a control data set to the DFSMSrmm level of code containing stacked volume support.

2. If you have stacked volumes defined to DFSMSrmm, but not as stacked volumes, you need to change volume information. Use the RMM SEARCHVOLUME subcommand to set the correct location information and volume type for non-exported stacked volumes. You need to do this before running EDGUTIL MEND(VOLUME). Although the EDGUTIL processing changes the volume type to stacked, it does not set the correct location information unless the stacked volume is exported and contains volumes.

You can use the RMM SEARCHVOLUME subcommand to build the command to change the volumes:

```
RMM SEARCHVOLUME VOLUME(ST*) OWNER(*) LIMIT(*) -  
CLIST('RMM CHANGEVOLUME ', ' TYPE(STACKED) LOCATION(vts_name) NORACK')
```

3. Use EDGUTIL UPDATE with the STACKEDVOLUME(YES) operand on the CONTROL statement of the SYSIN file.

Once you have enabled support, you can use EDGUTIL with VERIFY(VOLUME) to check if the container information is consistent.

4. Use the RMM LISTCONTROL CNTL subcommand to display the status of support.

If support shows MIXED, run the EDGUTIL MEND(VOLUME) utility to make the container information consistent. The inconsistency is the result of having container information in volume records in the control data set.

During EDGUTIL MEND(VOLUME) processing, DFSMSrmm creates the necessary stacked volumes if you have not previously defined them to DFSMSrmm using the DFSMSrmm TSO subcommands.

During EDGUTIL MEND(VOLUME) processing, DFSMSrmm converts each volume in a container as being in the container instead of a DFSMSrmm location. The location and bin number information in volumes that are in a container is removed, and bins returned to empty status. A bin number is assigned to the stacked volume if the location it is in is bin managed. The bin number is selected from one of the contained volumes by using location priority. The bin number choice should reflect the processing used by EDGRPTD in producing movement reports before stacked volume support is enabled.

From now on, when you run storage location processing, DFSMSrmm moves the stacked volumes based on the required location and priority of the contained logical volumes.

5. Update your procedures used to export or import logical volumes to use the required location of the logical volume instead of using the destination. See [“DFSMSrmm support for stacked volumes when stacked volume support is enabled” on page 112](#).

Performing a virtual export of logical volumes

You can use DFSMSrmm subcommands to perform a virtual export for a private logical volume to an existing exported stacked volume container. A virtual export is when you use the DFSMSrmm subcommands rather than VTS export processing to export a volume. This is possible if you have imported a logical volume from a stacked volume, processed the logical volume only for input, and now want to re-associate the logical volume with the previously exported volumes on the original stacked volume container. You use the CHANGEVOLUME subcommand with the CONTAINER operand to do the virtual export. For example:

```
RMM CHANGEVOLUME V12345 CONTAINER(S26901)
```

DFSMSrmm changes the status of the logical volume to scratch in the library manager database prior to ejecting the volume. This results in a logical eject which deletes the logical volume from the library manager database. DFSMSrmm then adds the volume logically back into the container.

For the logical eject to work and delete the logical volume from the TCDB and the library manager data base, the scratch categories in the VTS must be defined with the 'Fast Ready' attribute. If you do not use the fast ready attribute for your scratch categories, do not use virtual export. If you do so, you will receive messages CBR3650I and EDG3726I with error code '06' and the volume will not be deleted from the TCDB and the library manager data base.

Recovering a logical volume from an exported stacked volume

Note: The information in this topic applies only to VTS import/export processing; it does not apply to TS7700 copy export processing.

DITTO is used to recover a logical volume from an exported stacked volume in situations where no VTS is available. DITTO does not go through OPEN processing nor issue regular mount messages. DITTO recovers the logical volume as a physical volume. As a result, managing these recovered logical volumes requires some additional consideration and processing.

Here are some considerations:

- When you specify DFSMSrmm EDGRMMxx REJECT command values to manage volumes, DFSMSrmm relies on information available at OPEN. These REJECT command values do not apply because DITTO does not issue OPEN requests.

Note: If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRITITION and OPENRULE commands. See [“Converting REJECT commands to PRITITION and OPENRULE commands” on page 204](#) for information about converting from the use of REJECT commands.

- DFSMSrmm does not update existing volume information when DITTO copies the data.
- The DITTO output volume is a physical volume but might have the same volume serial number as the original logical volume. If you use the original logical volume serial number for the recovered volume, you can convert the logical volume in the DFSMSrmm control data set to a physical volume by issuing the DFSMSrmm CHANGEVOLUME subcommand as shown in [Figure 50 on page 122](#)

```
RMM CHANGEVOLUME volser TYPE(PHYSICAL) CONTAINER(' ')-  
LOCATION(SHELF) CONFIRMMOVE FORCE
```

Figure 50. Converting A logical volume to a physical volume

Use the recovered volume as a normal physical volume. If the output volume serial number is different from the original logical volume serial number, the volume information, the data set information and other details can be reused by issuing the DFSMSrmm CHANGEVOLUME subcommand as shown in [Figure 51 on page 122](#).

```
RMM CHANGEVOLUME volser TYPE(PHYSICAL) CONTAINER(' ')-  
LOCATION(SHELF) CONFIRMMOVE NEWVOLUME(pvolser)
```

Figure 51. Reusing volume information for a recovered volume

- You can also add data set information for the DITTO output volume from the original logical volume.

To copy details for the original logical volume to the new physical volumes, use DFSMSrmm subcommands to obtain the data set and volume details for the volumes you want to redefine.

Setting up DFSMSrmm for the system-managed tape library

To get started with the system-managed tape library, information about the volumes in the system-managed tape library must be defined to DFSMSrmm. This topic describes scenarios for getting volumes defined to DFSMSrmm.

Using the system-managed tape library with new volumes

If you intend to populate the system-managed tape library with new scratch volumes, you do not need to explicitly define them to DFSMSrmm. During entry processing, with DFSMSrmm active, DFSMSrmm automatically records information about each volume in its control data set. DFSMSrmm uses the defaults you specified in ISMF for the library entry values; you should set the default entry status to scratch. See [“Partitioning system-managed tape libraries” on page 125](#) for additional information.

Using the system-managed tape library with volumes already defined in dfsmsrmm

Volumes that are already defined to DFSMSrmm have location and other information known to DFSMSrmm. If you plan to use these volumes with the system-managed tape library, you need to update the volume location and home location to a system-managed library name. Here are two methods to implement the system-managed tape library with DFSMSrmm.

Method 1

1. For volumes that are going to a system-managed tape library, use the RMM CHANGEVOLUME subcommand with the LOCATION operand to change the location to the correct automated tape library name. This sets the automated tape library name as the destination name and home location name for each volume going to an automated tape library. For volumes that are going to a manual tape library, specify a manual tape library name and you are now ready to use the volumes.

You can use the RMM SEARCHVOLUME subcommand with the CLIST operand to build a data set containing the required CHANGEVOLUME LOCATION requests.

```
RMM SEARCHVOLUME VOLUME(A*) LIMIT(*) -  
  OWNER(*) LOCATION(SHELF) -  
  CLIST('RMM CHANGEVOLUME ',' LOCATION(ATL1)')
```

This subcommand builds a command list that you can execute. Repeat this step for each storage location from which you intend to move volumes.

2. Run EDGHSKP with the PARM='REPTXT' and use EDGRPTD to generate a list of volumes to insert into the automated tape libraries.
3. Insert the volumes. You are now ready to use the volumes.
4. To return volumes from storage locations to the system-managed tape library when the volumes no longer need to be retained, use the RMM CHANGEVOLUME subcommand to change the volume HOME location from SHELF to the library name.

example:

```
RMM SEARCHVOLUME VOLUME(A*) LIMIT(*) -  
  OWNER(*) LOCATION(LOCAL) -  
  CLIST('RMM CHANGEVOLUME ',' HOME(ATL1)')
```

Method 2

1. Pull all volumes that are to go into the system-managed tape library and physically load them.
2. Run the inventory process of the system-managed tape library and let it drive the adding of volumes to the volume catalog. This also flags the volume location as an automated tape library, but does not change the home location name.
3. Use the RMM SEARCHVOLUME subcommand with the CLIST operand to create a CLIST to create CHANGEVOLUME commands for all the volumes in the library to set the HOME location to the automated tape library. For example, specify:

```
RMM SEARCHVOLUME VOLUME(A*) LIMIT(*) -  
  OWNER(*) LOCATION(ATL1) -  
  CLIST('RMM CHANGEVOLUME ',' HOME(ATL1)')
```

If you do not change the home location name, all private volumes are eligible for moving out of the automated tape library when you next run vital record processing as part of inventory management.

4. If you have volumes in storage locations that you want returned to the system-managed tape library when they no longer need to be retained, change the volume HOME location from SHELF to the library name, using the RMM CHANGEVOLUME subcommand. For example, specify:

```
RMM SEARCHVOLUME VOLUME(A*) LIMIT(*) -  
OWNER(*) LOCATION(LOCAL) -  
CLIST('RMM CHANGEVOLUME ',' HOME(ATL1)')
```

Using the system-managed tape library with existing volumes

To use the system-managed tape library with existing volumes that are not defined to DFSMSrmm or if you are using DFSMSrmm to manage these volumes for the first time, ensure that information about your volumes is recorded in the DFSMSrmm control data set.

You need to ensure that the correct status is recorded for each private and scratch volume you want to use with the system-managed tape library. One way to do this is to predefine all the private volumes. You can use any existing information you have for the volumes either from an existing tape management system or based on data set catalog entries. You use this information to build a list of RMM ADDVOLUME subcommands to define the volumes to DFSMSrmm. For example, for a private volume:

```
RMM ADDVOLUME volser LOCATION(ATL1) STATUS(MASTER) -  
OWNER(owner) STORGRP(storagegroup)
```

Adding rack numbers is optional. You can define shelf space first by using the ADDRACK subcommand so there are empty rack numbers for each volume you add.

Depending on the ISMF library entry defaults, you might be able to avoid defining scratch volumes to DFSMSrmm as this can happen automatically during entry processing.

Using DFSMSrmm with an existing automated tape library

If you have been using an automated tape library without DFSMSrmm, volume status information and retention requirements from the TCDB must be recorded in the DFSMSrmm control data set for private and scratch volumes in the automated tape library.

Assuming that you are using a single range of volume serial numbers that start with S00000 in the automated tape library, follow these steps to obtain the information:

1. Adding shelf space is optional for physical volumes and not supported for logical volumes. Define shelf space for the volumes to DFSMSrmm using the RMM ADDRACK subcommand:

```
RMM ADDRACK S00000 COUNT(5000)
```

2. Use the RMM ADDVOLUME subcommand to define the volumes to DFSMSrmm and get the correct status information from the volume catalog. You can provide an owner on the RMM ADDVOLUME subcommand. If you do not specify the EXPDT operand, DFSMSrmm obtains the expiration date from the volume catalog. If there is no date in the volume catalog, DFSMSrmm uses the DFSMSrmm parmlib RETPD default.

```
RMM ADDVOLUME S00000 COUNT(5000) STATUS(VOLCAT) OWNER(owner) -  
EXPDT(yyyy/dd)
```

If you do not provide an owner, DFSMSrmm assigns a default owner as described in *z/OS DFSMSrmm Managing and Using Removable Media*. If you are not using a single range of volumes in the automated tape library, issue multiple ADDVOLUME subcommand requests to define all volumes.

It is likely that while implementing DFSMSrmm with the automated tape library you are converting from an existing tape management system. If so, add this step to your overall conversion plan. Instead of using the RMM ADDVOLUME subcommand to define the volumes, use information from your existing tape management system to define the volumes to DFSMSrmm during the conversion.

Returning volumes to scratch status

When a volume in a system-managed tape library is returning to scratch status, DFSMSrmm can inform OAM during expiration processing for each volume being changed to scratch status. Return to scratch processing for system-managed volumes can be either synchronous or asynchronous with EXPROC

processing. You use the SYSIN EXPROC command EDGSPLCS operand to select the type of processing you want. This function also allows DFSMSshm volumes to return to the scratch category.

If you select asynchronous processing, EDGSPLCS provides for concurrent (parallel) return to scratch processing across libraries through specification of a library name. EDGSPLCS then runs against a single list of volumes generated by RMM and through specification of a library name, the list can be executed multiple times (once for each library). EDGSPLCS also supports concurrent processing of a list of volumes within the same library. EDGSPLCS uses an ENQ to ensure that each volume can be processed by only one instance of EDGSPLCS. In this way, the parallelism is limited to the number of available logical devices and the number of copies of EDGSPLCS that are run.

See [“Using EDGSPLCS to issue commands to OAM for system-managed volumes”](#) on page 418 for additional information about the EDGSPLCS utility. See [“EDGSPLCS file for the EDGHSKP utility”](#) on page 345 for information about the EDGSPLCS file for the EDGHSKP utility.

If DFSMSrmm is to return volumes to scratch during EXPROC processing, DFSMSrmm informs OAM to update the TCDB volume status during expiration processing or when the status of a volume is changed using the RMM CHANGEVOLUME subcommand based on the DFSMSrmm EDGRMMxx parmlib OPTION SMSTAPE operand value you specify. See [“Defining system options: OPTION”](#) on page 161 for information about the DFSMSrmm EDGRMMxx parmlib OPTION SMSTAPE operand.

DFSMSrmm uses the OAM status change exit, CBRUXCUA, to allow DFSMSrmm to be notified of all volume status changes made by others, for example, by ISMF. CBRUXCUA performs this processing:

- If a volume is not defined to DFSMSrmm, but is resident in an automated tape library, it is automatically defined to DFSMSrmm. If any errors are encountered, as for cartridge entry processing, the status change is rejected.
- If a volume is defined to DFSMSrmm:
 - All changes from PRIVATE to SCRATCH status are returned to scratch by any OAM CBRUXCUA request, such as those generated by the EDGSPLCS utility or the ISMF mountable tape volume list processing. Attempts to use OAM CBRUXCUA requests to change the status of a non-scratch-candidate volume to a scratch volume fail with message EDG8194I.
 - All changes from PRIVATE to PRIVATE and SCRATCH to SCRATCH are supported to enable OAM to correct discrepancies that might exist with the library manager inventory.
 - All changes from SCRATCH to PRIVATE status are accepted. This includes open processing.
 - Storage group name changes are recorded in the DFSMSrmm control data set.
 - Any change to make TCDB information consistent with DFSMSrmm information is accepted.

The objective is to keep the TCDB and the DFSMSrmm control data set information consistent.

Recommendation: Use the STGADMIN.IGG.LIBRARY resources to protect the new catalog facilities define, alter, and delete of library entries, and volume entries to ensure changes go through the OAM installation exits. Use IDCAMS as an error recovery tool.

Partitioning system-managed tape libraries

You can partition a system-managed library including a VTS by performing these tasks:

- Specify the USE operand value on the RMM ADDVOLUME or RMM CHANGEVOLUME subcommands. You can set this value to MVS or VM or both. If you do not specify MVS for a volume, DFSMSrmm prevents the volume from being defined in the volume catalog on this system.
- Through the use of the PRTITION and OPENRULE parmlib commands, you can simplify the maintenance of the parmlib members as your libraries and volume ranges change. Operands on the OPENRULE and PRTITION commands allow global actions to be set. You can use one or more specific overrides based on volume sets that have different requirements. Typically, you could add a new range of volumes for use by a single partition and only that one system would need to be updated. The OPENRULE and PRTITION commands allow you to define whether they apply to volumes defined to DFSMSrmm or not. You can use operands on the OPENRULE command to automatically ignore volumes, rather than using EXPDT=98000 or a customized EDGUX100.

Note: PRTITION is the preferred method for partitioning. REJECT commands, although still supported, should not be used in new installations. If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. See [“Converting REJECT commands to PRTITION and OPENRULE commands”](#) on page 204 for information about converting from the use of REJECT commands. PRTITION rules are not applied when DFSMSrmm running mode is MANUAL (OPMODE=M).

When you enter a volume into a system-managed tape library, if the volume is defined to DFSMSrmm and you have specified the USE operand without MVS, or the volume matches an OPENRULE specification that prevents a volume from being defined in the system-managed tape library on the current system, EDGLCSUX sets a return code of 12 to pass to OAM. OAM leaves the volume in the system-managed tape library in the INSERT category; it does not create a volume entry in the TCDB. The volume is then available for another sharing system to process the insert request. The sharing system could be another VM or z/OS system.

If DFSMSrmm allows the volume entry to be performed, OAM creates an entry in the TCDB. If the volume matches an OPENRULE specification that limits the volume's use to input processing, at OPEN time DFSMSrmm fails any requests for output processing, while allowing requests for input processing.

Using the PRTITION commands, you can control partitioning entry/insert, export/import, eject, and CUA processing for system managed volumes:

- DFSMSrmm partitioning is based on a global setting that you can change. The default is that all system-managed and non-system managed volumes are accepted. This can be represented by this command:

```
PRTITION VOLUME(*) TYPE(ALL) SMT(ACCEPT) NOSMT(ACCEPT LOCATION(SHELF))
```

You can change the global setting so that all volumes are ignored by using this command:

```
PRTITION VOLUME(*) TYPE(ALL) SMT(IGNORE) NOSMT(IGNORE)
```

- Using a global command like this:

```
PRTITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)
```

- All system-managed volumes undefined in the DFSMSrmm CDS are left in insert category to be accepted by another system and non-system-managed volumes are not added automatically to the CDS.
- The remainder of the default command [PRTITION VOLUME(*) TYPE(RMM) SMT(ACCEPT) NOSMT(ACCEPT)] that is not overridden by your global commands in parmlib is used to handle TYPE(RMM) volumes.
- With this approach, you must predefine system-managed volumes to DFSMSrmm to enable ownership of volumes during entry/insert processing.
- No PRTITION commands - When REJECT commands are used, processing is as for earlier releases unless any OPENRULE statements are defined. In the latter case and when no REJECT commands are defined, the defaults are used for partitioning.
- Using selective PRTITION statements on top of a global command, you can be very specific about which volumes are to become owned by the current system/partition.
 - All ACCEPTed volumes that are not defined to DFSMSrmm are added automatically to the DFSMSrmm CDS and owned by this system/partition, unless it is eject processing.
 - For system-managed volumes, 'Owned by this system/partition' means that the volume is defined to DFSMSrmm, has an entry in the TCDB, and the scratch category used by the Library Manager (LM) is set based on your values in this systems DEVSUPxx.
 - The ISMF library default entry status is used for added volumes.
 - Pre-defined volumes are only considered when the PRTITION command specifies TYPE(RMM) or TYPE(ALL). The volume status for the TCDB entry is set by DFSMSrmm from the volume information.

As a result of this flexibility, you should no longer need to customize the CBRUXENT exit.

In addition to the system-managed processing, the PRTITION commands also allow all volumes to be partitioned during inventory management and during OPEN processing. During EXPROC return to scratch processing, a volume matching to a PRTITION command with TYPE(ALL/RMM) and an action of IGNORE causes the return to scratch processing for the volume to be skipped. This occurs even if a TCDB volume entry exists (for example, the TCDB is shared or a TCDB entry was created manually for private volume sharing).

You can use the DFSMSrmm unshared catalog support, by means of OPTION CATSYSID, to control which volumes are processed on which system. For each scratch candidate volume, DFSMSrmm:

1. Checks PRTITION commands. If they are not ignored or skipped, DFSMSrmm continues with the next check.
2. Checks that a TCDB entry exists if system-managed. If yes, DFSMSrmm continues with the next check. If no, DFSMSrmm skips the volume.
3. Checks the unshared catalog. If the first file data set was not created on one of the CATSYSID systems, DFSMSrmm skips the volume.

You must run EXPROC once per scratch category set that you use for system-managed library partitioning.

Sharing a system-managed library and a BTLS-managed library

When you share an automated tape library between DFSMS and BTLS, there are restrictions on the sharing of volumes between the systems. For example, although a private volume is defined in the TCDB on DFSMS, it cannot be shared unless it is also defined in the BTLS catalog.

Consider partitioning the automated tape library using some volumes under DFSMS and other volumes under BTLS. This is important when you plan to use scratch volumes. Volumes that are part of scratch pools cannot be effectively shared.

Only volumes that are long-term private volumes can be shared effectively. You can define each private volume to both systems, but if the volume changes status, it is likely that the BTLS catalog and the TCDB will not match the volume status defined in the DFSMSrmm control data set. You can modify the CBRUXENT exit supplied with DFSMSrmm to force the Library Control System to not process the volumes intended for BTLS management.

If you want to designate specific scratch volumes for use on DFSMS and others for use with BTLS:

1. Modify the DFSMSrmm supplied CBRUXENT exit by setting the return code to 12 (UXEIGNOR) for all volumes that are to be managed by BTLS.
2. Use the AMS LIBRARY SETCATEGORY command to set the appropriate BTLS private or scratch category for all volumes entering the automated tape library that are ignored by the Library Control System.

If you modify the CBRUXENT exit, issue a WTO that you trap in Netview or equivalent. Use this event to trigger the start of the AMS LIBRARY SETCATEGORY command on the system where BTLS resides.

An easier implementation would be possible using separate control data sets, one for DFSMSrmm with system-managed tapes, and one for DFSMSrmm with BTLS volumes. The disadvantage of this is extra administration and total separation of volumes.

Moving from a non-system-managed to a system-managed IBM automated tape library

When moving volumes from a non-system-managed to a system-managed IBM automated tape library, update the DFSMSrmm control data set to reflect the new locations for the volumes. Issue the RMM CHANGEVOLUME subcommand on the DFSMSrmm system to update both the home location and current location of the volumes.

```
RMM CHANGEVOLUME volser LOCATION(sms_lib_name)
```

To return volumes from a storage location to a system-managed instead of a non-system-managed IBM automated tape library, use the RMM CHANGEVOLUME subcommand to change the home location for the volumes.

```
RMM CHANGEVOLUME volser HOME(sms_lib_name)
```

See [“Defining pools: VLPOOL” on page 209](#) for information about retaining and moving volumes.

Chapter 6. Running DFSMSrmm with BTLS

You can use DFSMSrmm with Basic Tape Library Support (BTLS). BTLS is an IBM program offering that provides basic automation support for IBM tape libraries in a non-system-managed library environment. With BTLS, DFSMSrmm does not interact directly with BTLS or with the IBM tape library, so you must update the BTLS catalog to reflect changes to volumes that are managed by DFSMSrmm.

DFSMSrmm adds information to the DFSMSrmm control data set when you define volumes to DFSMSrmm. If you plan to use any of the volumes defined to DFSMSrmm with BTLS, use the access methods services LIBRARY command to define the volumes in the BTLS catalog. Refer to the *Basic Tape Library Support Version 1 Release 1 User's Guide and Reference* for more information.

If you plan to use DFSMSrmm for volumes managed by BTLS, set up procedures to return scratch volumes to scratch status in the BTLS catalog after DFSMSrmm expiration processing.

Here is a summary of the steps you follow to use BTLS with DFSMSrmm:

1. Use the NAME operand on the VLPOOL parmlib command to identify the BTLS scratch pools to be used.
2. Optionally, if you use data set name and jobname for your volume pools, use the EDG_EXIT100 installation exit to select a pool for new tape data sets.
3. Set up the procedures to return BTLS-managed volumes to scratch status after you run inventory management.
4. Set up the procedures to update BTLS when volumes are added to or removed from the installation media library.

Setting up scratch pools for BTLS-managed volumes

“Defining pools: VLPOOL” on page 209 provides information on defining VLPOOL and other EDGRMMxx options.

Both DFRMM and DFSMSrmm use VLPOOL scratch pool definitions to perform these functions:

- Update mount messages.
- Update 3480 and 3490 drive displays if the MSGDISP installation exit IGXMSGEX is called.
- Reject scratch volumes that are not from the correct pools.

If you plan to use DFSMSrmm with BTLS, you must define and use scratch pools with special care to prevent DFSMSrmm from rejecting volumes needlessly. DFSMSrmm does not know that volumes managed by BTLS reside in an automated tape library and attempts to control scratch tape assignment for mounts inside the automated tape library. DFSMSrmm, on the other hand, does not attempt to control scratch tape assignment for mounts inside a system-managed automated tape library.

If you have scratch volumes that reside in an automated tape library managed by BTLS, define pool definitions to include all the volumes that you want to use. DFSMSrmm accepts or rejects volumes that are based on the VLPOOL definitions that you provide in parmlib member, EDGRMMxx. For volumes that reside in an automated tape library and are managed by BTLS, ensure that the VLPOOL definitions do not cause conflicts when scratch volumes are used.

For example, consider these VLPOOL definitions:

```
VLPOOL PREFIX(A*) SYSID(A) TYPE(S)
VLPOOL PREFIX(B*) SYSID(B) TYPE(S)
```

On system A when a scratch volume is requested, if the automated tape library contains volumes from both pools A* and B*, it can select a scratch volume from either pool. If the Library Manager selects a volume in pool B*, DFSMSrmm rejects it. The Library Manager might *never* select a volume that can be

used because all available scratch volumes are selected until a volume that is acceptable to DFSMSrmm is found.

If you want DFSMSrmm to manage pool selection when using BTLS, use the NAME operand on the VLPOOL definition for the pool to specify a pool name. For example, you could define VLPOOL definitions as follows:

```
VLPOOL PREFIX(A*) NAME(SCRTCH5) TYPE(S) SYSID(SY1) DESC('BTLS pool 5')
VLPOOL PREFIX(B*) NAME(SCRTCH3) TYPE(R) DESC('BTLS pool 3')
```

DFSMSrmm uses the value in the NAME operand to update messages and tape drive displays for non-specific mount requests. Do not use the NAME operand if your operations depend on a pool prefix rather than a pool name.

For scratch pools of volumes that are contained in an automated tape library that are managed by BTLS, use the NAME operand when specifying the SYSID operand on VLPOOL definitions. For more information, refer to [“Returning BTLS-managed volumes to scratch” on page 131](#).

Running DFSMSrmm inventory management with BTLS

Use the EDGHSKP utility to run inventory management activities which include: vital record processing, expiration processing, storage location management processing, backing up the control data set and journal and creating an extract data set. See [Chapter 13, “Performing inventory management,” on page 327](#) for more information.

Consider how to perform inventory management if you are using DFSMSrmm with BTLS. You need to complete the update to the BTLS catalog after inventory management is completed. Use the Access Method Services (AMS) LIBRARY SETCATEGORY command to update the status of the volumes managed by BTLS.

For example, to keep track of volumes that are managed by BTLS and to update information about volumes that are returning to scratch in the BTLS catalog, perform these tasks:

1. Put all your BTLS-managed volumes in racks with a specific media name, for example: BTLS. Any other method to identify BTLS volumes, such as rack number or volume prefix could be used instead.
2. After inventory management is complete, issue this command to create a list of BTLS racks in scratch status.

```
RMM SEARCHRACK RACK(*) LIMIT(*) MEDIANAME(BTLS) SCRATCH -
CLIST NOLIST
```

3. Use the resulting data set with the scratch volume list as the LIBIN DD input to an IDCAMS job with this command:

```
LIBRARY SETCATEGORY UNIT(xxx) CATEGORY(SCRTCH)
```

Running EDGINERS for BTLS-managed volumes

For volumes that are managed only by BTLS, use EDGINERS. Use the TAPE DD to allocate a tape drive in the automated tape library. Use the POOL parameter or any other execution parameter to restrict processing to specific volumes that are managed by BTLS.

Restrictions

DFSMSrmm does not ensure that the volumes you use with BTLS in the automated tape library meet the z/OS labeling restrictions that apply for system-managed tape volumes. You must make sure that only z/OS standard label volumes with the same external and internal volume serial number are entered into the automated tape library. Failure to do so can result in incorrect processing by DFSMSrmm. For example, DFSMSrmm records the internal volume serial number of the mounted volume in the DFSMSrmm control data set, while BTLS uses the external volume serial number to request a mount.

When you migrate from BTLS management to system-managed tape, migration is easier if you use only z/OS standard label volumes. Also, under system-managed tape processing, DFSMSrmm ensures that the volume serial number and rack number match, rejecting volumes that do not meet this requirement. Considering these restrictions during BTLS implementation will make migration to system-managed tape easier.

Defining volume information for BTLS-managed volumes

DFSMSrmm does not interface with BTLS so volumes are not automatically defined to the BTLS catalog. If you plan to use any of the volumes defined to DFSMSrmm in an automated tape library managed by BTLS, you must also define the volumes in the BTLS catalog using the AMS LIBRARY SETCATEGORY command.

When you delete volumes from DFSMSrmm, if the volumes are managed by BTLS you also need to ensure the volumes are deleted from the BTLS catalog and ejected from the automated tape library, if appropriate.

Returning BTLS-managed volumes to scratch

When volumes that reside in an automated tape library are returned to scratch by DFSMSrmm, information needs to be updated in the TCDB or the BTLS catalog. DFSMSrmm provides no support to automatically update the BTLS catalog.

If you perform scratch management of volumes that reside in an automated tape library that are managed by BTLS, ensure that BTLS volume status information matches DFSMSrmm volume status information. After you run expiration processing, prepare a list of volumes that are in scratch status. Issue the LIBRARY SETCATEGORY command to update the status of each volume to scratch. Issue this command for each volume even if the status of the volume is scratch:

```
LIBRARY SETCATEGORY UNIT(addr) VOLSER(volser) CATEGORY(SCRATCH)
```

You can use DFSMSrmm to help build the input to the SETCATEGORY function. The TMP step shown in [Figure 52 on page 131](#) uses the RMM SEARCHVOLUME subcommand to produce a simple list of scratch volumes in the CLIST data set. This list is then input to IDCAMS in the LIBIN DD file. You can tailor the RMM SEARCHVOLUME subcommand to list volumes that are based on more selective criteria.

```
//jobname JOB .....
//TMP      EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN  DD *
RMM SEARCHVOLUME VOLUME(*) OWN(*) STATUS(SCRATCH) LIMIT(*) -
CLIST
/*
//AMS      EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//LIBIN DD DSN=userid.EXEC.RMM.CLIST,DISP=SHR
//SYSIN DD *
LIBRARY SETCATEGORY UNIT(addr) CATEGORY(SCRATCH)
/*
```

Figure 52. Sample JCL to return volumes to scratch

You need to modify this example if you are using system-managed tape with DFSMS and BTLS-managed volumes. Limit the search to volumes that are in the BTLS scratch pool. Otherwise all scratch volumes will become BTLS scratch and will not be available for use as scratch on the DFSMS system. Refer to *Basic Tape Library Support Version 1 Release 1 User's Guide and Reference* for more information on using the LIBRARY command.

Chapter 7. Managing storage locations

Storage locations are those places outside the removable media library where you send removable media. Storage locations can be destinations for disaster recovery related activities or any other purpose your installation chooses.

DFSMSrmm provides shelf-management of storage locations by assigning bin numbers to shelf locations within a storage location. You can specify that DFSMSrmm make bins available for reuse when volume moves have started or make bins available only when volume moves have been confirmed.

DFSMSrmm automatically provides shelf-management for the built-in locations. This means that DFSMSrmm assigns bin numbers to each volume in a built-in storage location. You can request shelf-management of installation defined storage locations when you create your location definitions using the parmlib LOCDEF command.

You can also specify media names in the parmlib LOCDEF command to provide a way to segregate shelf locations in an installation defined storage location. The media names you specify for a storage location should be the same as or a subset of the media names you specify for pools in your removable media library. See [“Defining pools in parmlib member EDGRMMxx” on page 78](#) for information about pooling.

The movement and retention of volumes in and out of storage locations is done during DFSMSrmm inventory management. During vital record processing, DFSMSrmm sets the required location for each volume using information from vital record specifications. The volume does not move to the required location during vital record processing. During storage location management processing, DFSMSrmm sets the destination for a volume using the required location information if that volume can be moved. If a move is required, and the move is to a shelf-managed storage location, DFSMSrmm assigns an empty bin in the target location to the volume based on the media name and LOCDEF information. If no empty bin numbers of the required media name are available, DFSMSrmm issues message EDG2403E, and inventory management processing continues.

When the volume is returned from the storage location, the bin number is identified for reuse as part of move confirmation processing. See [Chapter 13, “Performing inventory management,” on page 327](#) for information about inventory management vital record processing and storage location management.

When stacked volume support is enabled, DFSMSrmm sets the destination for a stacked volume at the completion of export processing. There is no destination set for logical volumes. Logical volumes use the stacked volume destination.

You can also override the inventory management processing of specific volumes by manually assigning destinations.

Types of storage locations

DFSMSrmm recognizes two types of storage locations: DFSMSrmm built-in storage locations and installation defined storage locations. There are three DFSMSrmm built-in storage locations named: LOCAL, DISTANT, and REMOTE.

Installation defined storage locations can be used for volumes for disaster or vital records, or for controlling any media moving outside your installation. Using DFSMSrmm installation defined storage locations, you can perform these tasks:

- Define more than three storage locations.
- Use any name up to 8 characters in length to name the storage location.
- Change the movement priority for the installation defined storage locations.
- Continue to use the DFSMSrmm built-in storage locations.
- Select shelf-management or no shelf-management for the installation defined storage locations.

Defining storage locations

Use the LOCDEF parmlib command to define installation defined storage locations. Storage locations can be any 1 to 8 character named location except for the DFSMSrmm reserved location names ALL, HOME, and CURRENT. You can also use the DFSMSrmm built-in storage location names as installation defined storage locations. You can also define characteristics of system-managed storage locations using the LOCDEF parmlib command. [Table 13 on page 134](#) shows the differences between built-in and installation defined storage locations.

<i>Table 13. Differences between built-in and installation defined storage locations</i>		
To	For Built-in Storage Locations	For Installation-defined Storage Locations
Segregate shelf locations by media name	You cannot segregate built-in storage locations by media name or type of media.	You can segregate shelf locations by specifying media names
Define storage locations	You do not need to define built-in storage location names. The three are: LOCAL, DISTANT, and REMOTE.	You can define an unlimited number of storage location names.
Define bin numbers	DFSMSrmm uses bin numbers 1 through 999999.	You can use any 6 character value.
Shelf-manage	DFSMSrmm automatically provides shelf-management.	You can decide.
Determine location priority	DFSMSrmm uses the default priority.	You can define the priority in the LOCDEF location definitions.

Implementing installation defined storage locations

You might implement installation defined storage locations for these reasons:

- To choose the storage location names you want
- To change the dominance priority of the locations for inventory management vital record processing for use when moves conflict
- To separate volumes by shape when sending them to storage locations
- To use storage locations without shelf management
- To use more than 3 storage locations
- To use storage locations that are not applicable for automated movement.

To implement installation defined storage locations, follow these steps:

1. Identify:
 - a. The number of storage locations you require. DFSMSrmm provides you with three built-in storage locations. If you use more than three storage locations, use the LOCDEF parmlib command to define additional storage locations. You can identify any location as an installation defined storage location except the locations ALL, HOME, and CURRENT, which are DFSMSrmm reserved location names.
 - b. The location names you will use.
 - c. The priority you want to use for each location. Priority is used to resolve movement conflicts that occur when more than one policy applies to a volume or when multiple logical volumes reside on a stacked volume. The relative priority of the locations is used to determine where a volume is sent. Include the location name SHELF and any system-managed libraries to develop your location priority.

- d. The media names you will use in your installation. If you have different media shapes in your installation, you can set movement policies based on the different shapes. For example, you might have separate shelving for tape reels, cartridge tape, and optical media due to differences in their shape.
 - e. If you require storage locations without shelf management.
 - f. If no movement will be initiated by DSTORE processing.
2. Define LOCDEF parameters in parmlib. See [“Defining storage locations: LOCDEF” on page 143](#).
 3. Restart or refresh the DFSMSrmm procedure to use the updated parmlib member.
 4. Define the bin numbers for the shelf-managed storage locations using the RMM ADDBIN subcommand. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information.

When the new location and bin numbers are defined, they are available for assignment to volumes moving to the storage location. In order for DFSMSrmm to use the new storage locations and bin numbers for storage management, you must continue with step “5” on page 135. Inventory management sequentially assigns the bin numbers to moving volumes. If you do not define vital record specifications and run inventory management, the locations and bin numbers can only be assigned manually by using the RMM CHANGEVOLUME subcommand.
 5. Create new vital record specifications or update existing vital record specifications specifying the location names you defined using LOCDEF.
 6. Run inventory management vital record processing to produce a Vital Records Retention Report. Check the report to ensure that the correct destination is selected for each data set and volume retained by a vital record specification. You might find it helpful to do a trial run of VRSEL with an ACTIVITY file and then run EDGJACTP to review just the changes in retention as a result of the new and changed VRSEs.
 7. If you plan to export logical volumes, run export processing before running DFSMSrmm storage location management. See [“Managing stacked volumes” on page 110](#) for information on running export processing.
 8. Once the retention report is correct, run inventory management storage location management and report extract processing to assign destinations and bin numbers and to prepare an extract data set which you use as input to EDGRPTD.
 9. Run EDGRPTD to produce the movement and inventory reports for use to pull and ship the volumes to the correct locations.
 10. Once volumes have been moved, use the RMM CHANGEVOLUME subcommand to confirm volume movement.

Implementing storage locations as home locations

When you use storage locations as the home location for volumes you can decide how volumes are shelf managed. Shelf management is required if you want volumes stored in a specific slot such as a rack number or a bin number. A shelf location is not required if the volume is stored in a robotic tape library.

To avoid using shelf locations, define the storage location using MANAGEMENTTYPE(NOBI NS) and do not define rack numbers that match to the volume serial numbers.

To use the rack number as the shelf location, define the storage location using MANAGEMENTTYPE(NOBI NS) and either, define rack numbers that match to the volume serial numbers or use the POOL operand when adding or moving volumes.

To use the bin number as a shelf location define the storage location using MANAGEMENTTYPE(BI NS) and do not define rack numbers that match to the volume serial numbers.

To implement storage locations as home locations, follow these steps:

1. Update the DFSMSrmm PARMLIB LOCDEF commands to include TYPE(STORAGE,HOME) for those storage locations you want to also define as home locations.
2. Refresh DFSMSrmm parameters by issuing the command

```
F DFRMM,M=xx
```

3. Use the RMM CHANGEVOLUME subcommand to set the home location of volumes to be assigned to a specific storage home location.

```
RMM CHANGEVOLUME volser HOME(storname)
```

4. Use the RMM CHANGEVOLUME subcommand to set the current location of volumes already at the storage home location.

```
RMM CHANGEVOLUME volser LOCATION(storname)
```

If the storage location is not shelf-managed, you can include the CONFIRMMOVE operand to confirm the move is completed. If the storage location is shelf-managed, you must run EDGHSKP storage location management processing to assign shelf locations to the volumes. If you want specific shelf locations assigned, you can include the BIN operand on the RMM CHANGEVOLUME subcommand.

Managing shelf space for home locations

To assign or to change the assignment of shelf locations for volumes in the SHELF location or a system-managed library, use the RMM ADDVOLUME subcommand and the RMM CHANGEVOLUME subcommand with the RACK operand or the POOL operand. DFSMSrmm does not automatically initiate assignment of rack numbers as the shelf location for these volumes.

To automatically manage shelf space in a home location, use a shelf-managed storage location as the volume's home location. When a volume moves from or to a storage location, DFSMSrmm automatically assigns a bin number as the shelf location for the volume.

You can use either the rack number or the bin number as the shelf location for a volume. DFSMSrmm issues message EDG4013I at mount time when a volume is in a storage location, so that the location and bin number is available for the operator.

Reusing bins in storage locations

If you enable extended bin support, you can specify that DFSMSrmm make bins available for reuse when volume moves have started. To make bins available for reuse when a volume move is started, specify the DFSMSrmm parmlib OPTION command REUSEBIN(STARTMOVE) operand. The default operand is REUSEBIN(CONFIRMMOVE), whether extended bin support is enabled or is not enabled. See [“Defining system options: OPTION” on page 161](#) for detailed information.

To enable extended bin support, create or update the control data set control record using EDGUTIL with the EXTENDEDBIN(YES) option. See [“Creating or updating the control data set control record” on page 409](#) for detailed information.

Moving volumes to storage locations

You can move volumes to storage locations using these methods:

- By location. See [“Moving volumes by location” on page 136](#) for additional information.
- By media shape. See [“Moving volumes by media shape” on page 137](#) for additional information.
- Manually. See [“Moving volumes manually” on page 138](#) for additional information.

Moving volumes by location

You can request that DFSMSrmm move volumes from specific locations by running the EDGHSKP utility and using the LOCATION parameter, specifying both the originating location and destination. If you use the INSEQUENCE parameter, DFSMSrmm assigns volumes to bins in sequential volume serial number order and bin number order.

Moving volumes by media shape

You can describe removable media by their shape. For example, you can identify all round media, all square media, all small size media, or all cartridge media. You can use the VLPOOL parmlib command to define pools of media that are based on shape and to set the default media name. You can use media shape to identify the type of media that is allowed in a storage location. For example, you can keep tape reels, cartridges, and optical disks in different ranges of shelf space, where each type of media requires a differently shaped slot for storage.

When you decide which media names to use, consider the different media you have or might have in the future. You should use the same media names for storage locations that you use for pools. Use the LOCDEF command MEDIANAME operand to define each media name for each location to allow or restrict storage using media name. You can also aggregate similar media to use the same range of shelf space by using a media name of *.

Consider an installation with these types of media:

- Mini tape reels
- Tape reels
- Cartridge system tape
- Enhanced capacity cartridge system tape

There are four different types of media that fall into three basic media shapes: mini reels, reels, and cartridges. The number of reels in use in the installation is declining. [Table 14 on page 137](#) shows how the VLPOOL MEDIANAME and the LOCDEF MEDIANAME values can be defined. A different VLPOOL MEDIANAME has been defined for each different type of media. The media names describe the shape of the volumes or a physical characteristic like TWOTONE for enhanced capacity CST. In [Table 14 on page 137](#) (Case 1), the same LOCDEF media names are used to describe the media that can reside in the storage location. All volumes are segregated by their media name. In (Case 2), the cartridge system tape and enhanced capacity cartridge system tape are defined with the same media name and can be stored together because they are the same shape.

<i>Table 14. Storing media of the same shape</i>			
Type of Media	VLPOOL MEDIANAME	LOCDEF MEDIANAME (Case 1)	LOCDEF MEDIANAME (Case 2)
Mini tape reels	MINI	MINI	MINI
Tape reels	REELS	REELS	REELS
Cartridge system tape	CART	CART	*
Enhanced capacity CST	TWOTONE	TWOTONE	*

In [Table 15 on page 137](#), the media names REELS and CARTRDGE are used in the VLPOOL command to describe media shape. Each basic type of media has been allocated a different media name that gives information about the volumes. When the volumes are moved to a storage location, as shown in [Table 15 on page 137](#) (Case 3), all volumes are segregated by their media name. In (Case 4), all volumes are segregated by their media name but use is made of the * media name.

<i>Table 15. Storing media of different shapes</i>			
Type of Media	VLPOOL MEDIANAME	LOCDEF MEDIANAME (Case 3)	LOCDEF MEDIANAME (Case 4)
Mini tape reels	REELS	REELS	REELS
Tape reels	REELS	REELS	REELS

Table 15. Storing media of different shapes (continued)			
Type of Media	VLPOOL MEDIANAME	LOCDEF MEDIANAME (Case 3)	LOCDEF MEDIANAME (Case 4)
Cartridge system tape	CARTRDGE	CARTRDGE	*
Enhanced capacity CST	CARTRDGE	CARTRDGE	*

By careful selection of media names you can segregate shelf space in your removable media library using the VLPOOL parmlib command and in your storage locations using LOCDEF. See [“Organizing the library by pools”](#) on page 75 for information about the use of VLPOOL.

You can also use media name to control movement of volumes to non-shelf-managed storage locations. For example, you might have a customer that can only accept cartridge system tape. You can control the type of media sent to that customer by defining the LOCDEF command with a specific media name. In [Figure 53 on page 138](#), only volumes with a media name of CART can be moved to the CUST1 location.

```
LOCDEF LOCATION(CUST1) TYPE(STORAGE) MANAGEMENTTYPE(NO BINS) -
MEDIANAME (CART)
```

Figure 53. Using media name to control volume movement

During inventory management DFSMSrmm checks the media name for the location and prevents volume movement when the media name does not match. DFSMSrmm issues message EDG2412E for each volume that cannot be moved because its media name is not supported at the location.

Moving volumes manually

You can override automatic processing and control volume movement manually by using the RMM CHANGEVOLUME subcommand with the MANUALMOVE operand. To return the volume to automatic movement control, use the RMM CHANGEVOLUME subcommand with the AUTOMOVE operand.

When you put a volume under manual move control, DFSMSrmm does not move the volume anywhere automatically, even when it expires and is pending release. Volume movement occurs only if you request it using the RMM CHANGEVOLUME subcommand with the LOCATION operand.

To allow release processing, you must remove the volume from manual move control unless the volume is in its home location. When a volume is in its home location, DFSMSrmm performs release processing even if the volume is under manual move control.

You might use manual move control to keep a volume on-site even though the volume is flagged to be sent off-site for disaster recovery. To keep the volume on-site or to request that the volume be moved back to its home location, you could use this command:

```
RMM CHANGEVOLUME volser MANUALMOVE LOCATION(HOME)
```

Figure 54. Keeping volumes on-site

When a volume is put under manual move control, any outstanding move is canceled. Moves can also be canceled by issuing the RMM CHANGEVOLUME command with the LOCATION operand. The operand LOCATION(HOME) is specified in [Figure 54 on page 138](#) to cancel any pending moves because the volume is in its home location.

You might use manual move control for volumes you create on one system and then send to other systems for processing. Define the other systems as locations using the parmlib LOCDEF command. When a volume is ready to be sent to the other system, you can confirm the volume move and put the volume under manual move control at the same time. For example, to send a volume to another system defined on a LOCDEF command as OTHER1, you could issue this command:

```
RMM CHANGEVOLUME volser LOCATION(OTHER1) CONFIRMMOVE MANUALMOVE
```

Figure 55. Sending a volume to another system

The CONFIRMMOVE operand shown in [Figure 55 on page 139](#) confirms that the volume move has completed. The MANUALMOVE operand shown in [Figure 55 on page 139](#) puts the volume under manual move control and prevents the volume from being moved automatically. When the volume is returned from the other system, remove the volume from manual move control. Then confirm that the volume is back in its home location by issuing this command.

```
RMM CHANGEVOLUME volser LOCATION(HOME) CONFIRMMOVE AUTOMOVE
```

Figure 56. Returning a volume from another system

Assigning bins in storage locations

If you use the REASSIGN parameter, DFSMSrmm reassigns volumes to bins during storage location management processing. See [“EXEC parameters for EDGHSKP” on page 337](#) for a detailed description. Use both INSEQUENCE and REASSIGN and specify the DFSMSrmm parmlib OPTION command REUSEBIN(STARTMOVE) operand to maximize reuse of bins.

Changing storage locations

To change the bin management for a storage location, follow these steps:

1. Identify the storage locations information you want to change.
2. Update the DFSMSrmm EDGRMMxx parmlib member LOCDEF command MANAGEMENTTYPE operand for a storage location.
3. Restart the DFSMSrmm subsystem (F DFRMM,M=xx.) described in [“Step 16: Restarting z/OS with DFSMSrmm implemented” on page 59](#).
4. Use the RMM ADDBIN subcommand to add bins required for the storage location if the new MANAGEMENTTYPE is BIN.
5. If you changed the management type for a location defined with TYPE(STORE,HOME), issue the RMM CHANGEVOLUME subcommand with the HOME operand for all the volumes that have the changed location defined as their home location. Even though the home location name has not changed, the change volume subcommand with operand HOME internally updates the location type of the home location in the volume record. Use this command:

```
RMM SEARCHVOLUME VOLUME(*) LIMIT(*) OWNER(*) -  
  HOME(home_storage_location_name) -  
  CLIST('RMM CV ', 'HOME(home_storage_location_name)')
```

followed by EXEC EXEC.RMM.CLIST to make this change.

6. Perform inventory management vital record processing and storage location management to move volumes to the updated storage location.

Deleting storage locations

You can delete storage locations by removing the EDGRMMxx parmlib LOCDEF definition for the location. Before removing the LOCDEF definition in parmlib, check that:

- All bin numbers have been deleted in the location you want to delete.
- There are no volumes that are marked to be moved to the location.
- DFSMSrmm does not list any volumes for the location by using the RMM SEARCHVOLUME subcommand with the LOCATION operand.
- There are no vital record specifications that use the location you want to delete.

Switching volumes to installation defined storage locations

You can switch volumes from built-in storage locations to the installation defined storage locations without moving the volumes.

Make sure you have defined the LOCDEF command in parmlib to identify the target installation defined storage location and its attributes as shown in [Figure 57 on page 140](#).

```
LOCDEF LOCATION(REMOTE) TYPE(STORAGE) MEDIANAME(CART) -  
MANAGEMENTTYPE(BINS)
```

Figure 57. Identifying an installation defined storage location

Issue the RMM CHANGEVOLUME subcommand with the BIN operand to assign bin numbers for the volumes you are switching from the built-in storage location to the installation defined storage location. For example, the volume MIKE01 resides in the built-in storage location REMOTE in bin number 000010. To switch the volume to the installation defined storage location REMOTE, issue:

```
RMM CHANGEVOLUME MIKE01 LOCATION(REMOTE) BIN(000010) CONFIRMMOVE
```

The volume is allocated to the bin number in the installation defined storage location and the original bin number in built-in location REMOTE is empty. To move all the volumes you can use the data in the report extract file to generate the RMM CHANGEVOLUME subcommands.

DFSMSrmm provides sample JCL as EDGJCVB in SAMPLIB. You can use this sample to switch the storage location name to any installation defined storage location name while keeping the assigned shelf location number the same. Substitute the LOCATION(REMOTE) with LOCATION(*storage_locname*) you choose.

Converting from built-in storage locations

If you already have volumes stored in the DFSMSrmm built-in storage locations, you need to decide if you migrate the volumes to installation defined storage locations or allow the built-in storage locations to fall out of use when volumes are moved out and not replaced.

Once you decide to move to installation defined storage locations, whether or not you decide to continue to use the same names (LOCAL, DISTANT, and REMOTE), you must redefine the vital record specification definitions so that DFSMSrmm knows you are using installation defined storage locations.

The built-in storage locations LOCAL, DISTANT and REMOTE can be defined on LOCDEF parameters. This gives you the ability to use the existing built-in names rather than having to change the storage location names in use. When you use the DFSMSrmm built-in storage locations in a LOCDEF parmlib command, the DFSMSrmm built-in storage locations are treated like any other installation defined storage location.

You cannot use the LOCDEF command to change the PRIORITY of LOCAL, DISTANT or REMOTE and have them otherwise continue to work as before. Once you have defined the built-in names using LOCDEF:

- You can no longer use previously assigned bin numbers. You must define new bin numbers using the media names that are specified in the LOCDEF command.
- You must redefine the vital record specifications which reference these locations so that volumes can be scheduled to move to the installation defined location.

If the built-in storage location names are defined using LOCDEF without the PRIORITY operand, the default installation defined location priority is used.

Going back to built-in storage locations

If you converted built-in storage locations to installation defined storage locations as described in [“Converting from built-in storage locations” on page 140](#), you can go back to using the storage locations as built-in storage locations.

1. Remove the LOCDEF commands that use the built-in names.
2. Restart or refresh the DFSMSrmm procedure to use the updated parmlib member.

3. Define the bin numbers for the built-in storage locations using the RMM ADDBIN subcommand.
4. If LOCAL, DISTANT, or REMOTE were used as installation defined storage location names, any vital record specification using the names must be deleted and then redefined.
5. Run inventory management vital record processing to produce a Vital Records Retention report.
6. Run inventory management storage location management processing to assign destinations and bin numbers. Request a report extract file for EDGRPTD.
7. Run EDGRPTD to produce the movement and inventory reports for use to pull and ship the volumes to the correct locations.
8. Once volumes have been moved, use the RMM CHANGEVOLUME subcommand to confirm volume movement.
9. Delete the empty bins used for the installation-defined storage location.

Chapter 8. Using the parmlib member EDGRMMxx

This topic describes the options that you can specify in the parmlib member EDGRMMxx. You can use system symbols to enable easier sharing of the EDGRMMxx parmlib member. See [z/OS MVS Initialization and Tuning Reference](#) for how to use system symbols in parmlib members.

Do not specify duplicate operands. If you do, DFSMSrmm uses the last value you specified.

Note: DFSMSrmm does not ignore columns 72 to 80 in the parmlib member. However, if you wish to take advantage of the symbolic parmlib parser, you should avoid placing any data, even comments, in columns 72 to 80.

Specify the options using this format:

```
command operand1(value1) operand2(value2) -
        operand3(value3)
```

```
command operand1(value1,value2, +
        value3,value4) +
```

You can include comments in the parmlib member by enclosing your comments within `/* */` as shown in this example. Comments can precede and follow the parameters as well as appear within the parameters:

```
/*Your command syntax example*/
command operand1(value1) operand2(value2) /* comment */ -
        operand3(value3) /*end of command*/
```

DFSMSrmm processes the commands in the EDGRMMxx parmlib member and then, if there is a second member named, processes the second member. Any `OPTION` operands specified, other than `MEMBER`, override the values set in the first member. The `MEMBER` operand is ignored if it is also specified in the second parmlib member. Any other parmlib commands can add to, but not replace, update or duplicate, any command from the first parmlib member. The processing by DFSMSrmm is as if all of the parmlib contents of both members had been specified in a single parmlib member.

The parmlib member EDGRMMxx uses these commands:

- `LOCDEF` that is described in [“Defining storage locations: LOCDEF”](#) on page 143.
- `MEDINF` that is described in [“Defining media information: MEDINF”](#) on page 148.
- `MNTMSG` that is described in [“Defining mount and fetch messages: MNTMSG”](#) on page 153.
- `OPENRULE` that is described in [“Controlling the use of tape volumes: OPENRULE”](#) on page 157.
- `OPTION` that is described in [“Defining system options: OPTION”](#) on page 161.
- `PRTITION` that is described in [“Partitioning tape volumes: PRTITION”](#) on page 196.
- `REJECT` that is described in [“Defining tapes not available on systems: REJECT”](#) on page 202.
- `SECCLS` that is described in [“Defining security classes: SECCLS”](#) on page 206.
- `VLPOOL` that is described in [“Defining pools: VLPOOL”](#) on page 209.

Use the `OPTION` command to set defaults for DFSMSrmm on a system. Use the `VLPOOL` command to override the system-wide defaults for particular tape pools. Commands in the parmlib member EDGRMMxx are processed for symbols substitution before the commands are parsed.

Defining storage locations: LOCDEF

Use the `LOCDEF` command to define installation defined storage locations to DFSMSrmm. You can also use `LOCDEF` to set the priority for shelf locations and system-managed tape libraries. The `LOCDEF` command in [Figure 58 on page 144](#) defines the location MIKESLOC, which accepts media with the media name SQUARE, and which is shelf-managed.

```

/* LOCDEF - Add a location definition                                */
LOCDEF LOCATION(MIKESLOC) MEDIANAME(SQUARE) TYPE(STORAGE) -
MANAGEMENTTYPE(BINS)

```

Figure 58. Parmlib member EDGRMMxx LOCDEF command example

LOCDEF command syntax

Figure 59 on page 144 shows the syntax of the LOCDEF Command for defining storage locations.

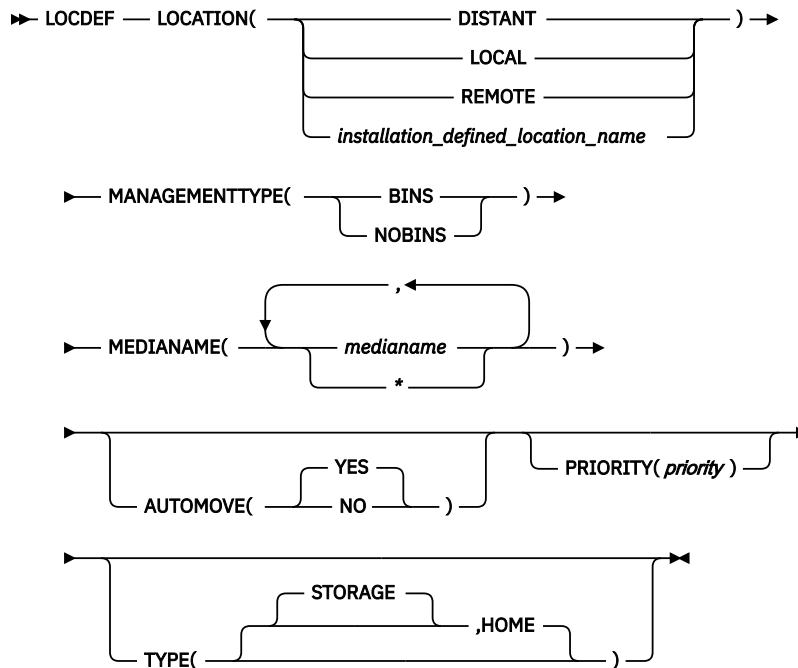


Figure 59. Parmlib member EDGRMMxx LOCDEF command for defining storage locations

Figure 60 on page 144 shows the syntax of the LOCDEF Command for defining shelf locations and system-managed libraries.

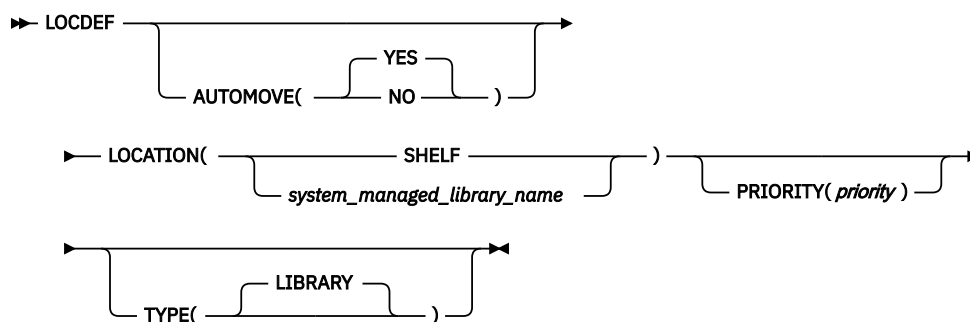


Figure 60. Parmlib member EDGRMMxx LOCDEF command for defining shelf and system-managed libraries

LOCDEF command operands

LOCATION(*installation_defined_location_name*| *system_managed_library_name*|LOCAL|DISTANT|REMOTE|SHELF)

Specifies the name of the location being defined. *installation_defined_location_name* can be any 1 to 8 character name or LOCAL, DISTANT, REMOTE, or SHELF. *system_managed_library_name* can be any 1 to 8 character name starting with a nonnumeric. The DFSMSrmm reserved location names HOME, CURRENT or ALL cannot be used as location names.

All LOCDEF location names must be unique. DFSMSrmm issues message EDG0225E if you define duplicate location names. You cannot have a storage location with the same name as a system-managed library. If you specify the same name, DFSMSrmm issues message EDG0233E. You can specify a distributed library name only if the library is an IBM Virtualization Engine. For other libraries, if you specify a distributed library name, DFSMSrmm issues message EDG0235E.

MANAGEMENTTYPE(BINS|NOBINS)

Use this operand to identify the shelf-management technique you want for the location. This operand is required when defining a location of type STORAGE.

BINS

DFSMSrmm shelf-manages the location by assigning bin numbers to volumes in the location.

NOBINS

Specifies that the location is not to be shelf-managed. Volumes sent to this location are not assigned bin numbers. However, they will still only be eligible to be sent to the location if their medianame or * appears in the LOCDEF media name list.

You can change the type of a location by specifying the LOCDEF command with the TYPE operand. During vital record processing, DFSMSrmm changes the required location and location type for each volume. See [“Changing storage locations” on page 139](#) for additional information.

Although inventory management vital record processing must be run to implement the new LOCDEF MANAGEMENTTYPE, you can use the new LOCDEF MANAGEMENTTYPE when issuing DFSMSrmm CHANGEVOLUME subcommands with the LOCATION operand.

MEDIANAME(*medianame*|*)

Specifies a list of the media names that are acceptable for the storage location. *medianame* can be any 1 to 8 character name you choose to describe a media name, type of media, a shape, or size. Examples of MEDIANAME include: CART, ROUND, SQUARE, 3490, 3590, TAPE, OPTICAL, CASSETTE, and so on. The media names you use on the LOCDEF commands should be the same as or a subset of the media names you use for your installation VLPOOL commands. See [“Defining pools: VLPOOL” on page 209](#).

When you specify MEDIANAME(*) using the parmlib LOCDEF command or the RMM ADDBIN or RMM ADDRACK subcommands, any volume with any media name can be sent to the location and DFSMSrmm does not segregate the volumes by shape. For example, if you specified a LOCDEF command with the media names:

```
LOCDEF LOCATION(MYLOC) MEDIANAME(3480,*) MANAGEMENTTYPE(BINS)
```

then you could use these media names on RMM ADDBIN subcommand requests:

```
RMM ADDBIN KG0002 LOCATION(MYLOC) MEDIANAME(3480)
```

and

```
RMM ADDBIN KG0100 LOCATION(MYLOC) MEDIANAME(*)
```

The example shown in [Figure 61 on page 145](#) is not valid because the MEDIANAME(3420) was not listed on the LOCDEF parameters for the storage location.

```
RMM ADDBIN KG0005 LOCATION(MYLOC) MEDIANAME(3420)
```

Figure 61. Using a medianame not defined in the LOCDEF command

AUTOMOVE

Use this operand to identify libraries or storage locations that hold tape volumes that are not eligible for automated movement initiated by inventory management. During inventory management DSTORE processing the volume's current location is validated against the matching LOCDEF. Consider this operand for libraries that contain virtual volumes or other volumes that either cannot be moved or for which you do not want DFSMSrmm to initiate the movement.

YES

DSTORE will initiate automated movement when the required location does not match the current location of the volume. This is the default.

NO

DSTORE will not initiate automated movement

PRIORITY(1-9999)

Defines the priority of this location relative to other locations. Lower numbers have higher priorities. PRIORITY is used to determine where to move a volume when a volume is assigned multiple destinations based on matching to two or more vital record specification definitions. PRIORITY is used to select a location in case of a move conflict. Move conflicts include:

- Multiple data sets with different required locations on one volume.
- Multiple volumes with different required locations in one volume set and the DFSMSrmm EDGRMMxx parmlib option is set for movement by volume set.
- Multiple logical volumes with different required locations on one stacked volume.

You can override this value by specifying a PRIORITY on the vital record specification. If PRIORITY is omitted, priority is based on the location type:

Priority**Location Name or Type****2000**

Installation defined STORAGE type, including LOCAL, DISTANT, and REMOTE if they are specified on LOCDEF.

4800

AUTO automated tape libraries

4900

MANUAL manual tape libraries

5000

SHELF location

For each location you do not specify in a LOCDEF, the default priority is determined from this list:

Priority**Location Name or Type****100**

REMOTE location

200

DISTANT location

300

LOCAL location

2000

installation-defined STORAGE type

4800

AUTO automated tape libraries

4900

MANUAL manual tape libraries

5000

SHELF location

TYPE(STORAGE|LIBRARY)

Use this operand to identify the type of location you are defining. The value can be either STORAGE or LIBRARY. This operand is optional. If MEDIANAME or MANAGEMENTTYPE are specified, only TYPE(STORAGE) is valid. If you do not specify the TYPE operand DFSMSrmm sets a default value based on whether you specify the MEDIANAME or MANAGEMENTTYPE operands.

STORAGE,HOME

Specifies that the location is a storage location, either shelf-managed or non-shelf-managed. You can identify a storage location as a home location by specifying TYPE(STORAGE,HOME). When you identify a storage location as a home location, you can manage volumes in a storage location like volumes that reside in a LIBRARY location. You can:

- Schedule release actions for volumes when they return to their storage home location
- Return volumes to scratch status while they reside in their storage home location

You can use STORAGE,HOME to define location names for non-system-managed robot libraries in your installation and assign a storage location name to that library instead of using the default value of SHELF.

LIBRARY

Specifies that the location is either SHELF or a system-managed library. The only reason to define these locations on LOCDEF parameters is to change the default priority, the default inventory management DSTORE processing (AUTOMOVE), or both for the location. For a LIBRARY type location the only LOCDEF operands you can use are AUTOMOVE, LOCATION, and PRIORITY.

You can change the type of a location by specifying the LOCDEF command with the TYPE operand. During vital record processing, DFSMSrmm changes the required location and location type for each volume. See [“Changing storage locations” on page 139](#) for additional information.

Although inventory management vital record processing must be run to implement the new LOCDEF TYPE value, you can use the new LOCDEF TYPE when issuing DFSMSrmm CHANGEVOLUME subcommands with the LOCATION operand.

Points on usage:

1. A LOCDEF parameter is required in the DFSMSrmm parmlib member for each installation defined storage location name and when you want to define a movement priority for the location.

If you have multiple systems sharing a control data set, use the same LOCDEF commands on all sharing systems. However, you only need to define storage locations to the systems where you will use any RMM ADDVRS, ADDBIN or CHANGEVOLUME subcommands, and where you plan to run inventory management. During storage location management processing, DFSMSrmm ensures that the storage locations used during vital record processing are defined by LOCDEF commands. If you specify a system-managed library name on a LOCDEF command, DFSMSrmm validates that it is a library defined on the current system. If your system-managed libraries are not defined on all systems, DFSMSrmm assumes a TYPE(LIBRARY) location is manual tape library if it is not defined as a system-managed library. During inventory management vital record selection processing, if no priority is obtained from a vital record specification, DFSMSrmm obtains the priority from the LOCDEF commands. If no LOCDEF command is specified and DFSMSrmm knows that the location is a system-managed library, DFSMSrmm uses the default priority for the location type. Otherwise DFSMSrmm uses a priority of 9999.

2. If a LOCDEF parameter is altered or removed, existing bin numbers that are now in a location that is not defined by LOCDEF, or which have a media name no longer listed under the LOCDEF MEDIANAME operand, are handled correctly. Bin numbers are freed up when a volume is moved from the storage location or can be removed using the RMM DELETEBIN subcommand. You cannot add more bin numbers nor assign the bin numbers to new volumes going to the location.
3. Changes to LOCDEF PRIORITY or MEDIANAME operands take effect during the first run of vital record processing after DFSMSrmm has been stopped and restarted, or the MODIFY command used to update the changed LOCDEF parameters in parmlib.
4. Changes to LOCDEF MANAGEMENTTYPE or TYPE operands take effect only after inventory management vital record processing has been run, or when using the LOCATION operand on the RMM ADDVOLUME or CHANGEVOLUME subcommands. See [“Changing storage locations” on page 139](#) for additional information.
5. See Chapter 7, [“Managing storage locations,” on page 133](#) for more information on LOCDEF and storage locations.

Defining media information: MEDINF

Use the MEDINF command to define media characteristics for OEM media products. The MEDINF values convert external values for media type and recording technology to internal values when supplied in the DFSMSrmm TSO subcommand input, as well as externalizes recorded values in reports and DFSMSrmm TSO subcommand output. To connect a volume to a specific media information entry, assign the media information name with the RMM TSO subcommands ADDVOLUME and CHANGEVOLUME using the MEDINF operand.

If a volume has no media information assigned, the default media information name is IBM, and the internal IBM table is used. To override the use of the internal table for IBM media, you must define media information in your parmlib specifying a media information name for IBM. The internal IBM table continues to be used for any values that you do not specify for IBM media. The internal values cannot be changed for IBM media. The internal values you specify identify the external values and capacity you want to override.

The internal IBM table cannot be displayed. It contains entries for each of the permissible combinations of IBM media types and recording formats and includes published media capacities. For the list of available external values, refer to the MEDIATYPE and RECORDINGFORMAT operand descriptions for the ADDVOLUME and CHANGEVOLUME subcommands in *z/OS DFSMSrmm Managing and Using Removable Media*. The internal values are defined by the CBRTDSI mapping macro; for example, for MEDIA1/CST the internal value is 1.

There is no default media information entry.

Once you have defined MEDINF commands and assigned a non-IBM media information name to a volume, DFSMSrmm no longer automatically maintains the media type and recording format. Unless you also implement the EDG_EXIT300 installation exit, the media type and recording format for non-IBM media is only maintained manually.

Use the MEDINF command with operand REPLACE to define volume replacement policies.

Figure 62 on page 148 shows part of the parmlib member EDGRMMxx, which provides sample MEDINF commands for OEM media products.

```
/*-----*/
/*
/* RMM (HDZ1A10)   SAMPLE MEDINF PARMLIB COMMANDS
/*
/* MERGE THIS MEMBER INTO YOUR EDGRMMxx PARMLIB MEMBER
/* AVOID REPLACE AND CAPACITY on SYNONYM ENTRIES unless the
/* combination of internal numerical values is unique.
/*
/*-----*/
/* Global replace policy */
MEDINF NAME(STK) REPLACE(PERM(1))
/* 3480 cartridges */
MEDINF NAME(STK) MEDIATYPE(1,STANDARD) RECORDINGFORMAT(1,18TRACK) +
CAPACITY(200)
MEDINF NAME(STK) MEDIATYPE(1,CST) /* synonym */ +
RECORDINGFORMAT(1,18TRACK)
MEDINF NAME(STK) MEDIATYPE(1,STANDARD) RECORDINGFORMAT(2,36ATRACK) +
CAPACITY(400)
MEDINF NAME(STK) MEDIATYPE(1,CST) /* synonym */ +
RECORDINGFORMAT(2,36ATRACK)
MEDINF NAME(STK) MEDIATYPE(1,STANDARD) RECORDINGFORMAT(3,36BTRACK) +
CAPACITY(400)
:
:
```

Figure 62. Parmlib member EDGRMMxx MEDINF command example for OEM media products

The full sample is delivered in SYS1.SAMPLIB(EDGMEDST). Another sample, SYS1.SAMPLIB(EDGMEDOP), provides sample MEDINF commands for media available on open systems and ready to be used with IBM's Integrated Removable Media Manager for the Enterprise on IBM Z® (IRMM).

MEDINF command syntax

Figure 63 on page 149 shows the syntax of the MEDINF command:

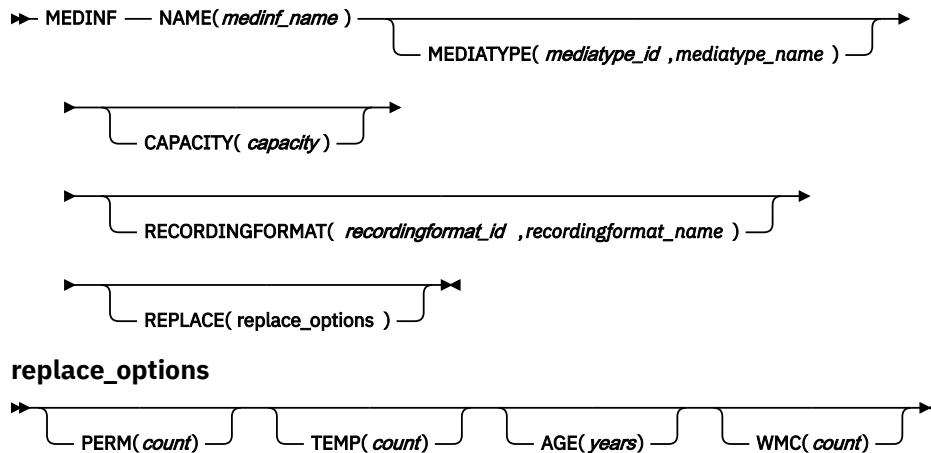


Figure 63. Parmlib member EDGRMMxx MEDINF command syntax

MEDINF command operands

NAME(*medinf_name*)

Specifies a name for the media information. Specify a value between 1 and 8 alphanumeric characters. This value is used to match the recorded volume information to the MEDINF installation defined media information. The values defined for MEDIATYPE, RECORDINGFORMAT, and CAPACITY are taken if the assigned media information for a volume matches *medinf_name*.

Default: None.

MEDIATYPE(*mediatype_id*,*mediatype_name*)

Use this operand to identify the internally-used *mediatype_id* to be converted to the externally-used *mediatype_name* in reports and in the output of the DFSMSrmm TSO subcommands. Also, use this operand to identify the externally-used *mediatype_name* to be converted to the internally-used *mediatype_id* that is recorded in the control data set when used as input to RMM TSO subcommands.

When you specify multiple MEDINF entries that use the same *mediatype_id*, but with different *mediatype_name* values, the first such MEDINF entry is used for the internal-to-external conversion. The remaining values are synonyms used for external-to-internal conversions.

mediatype_id is a number in the range from 1 to 255.

mediatype_name is 1 to 8 alphanumeric or national characters.

Default: None. When you do not specify the MEDIATYPE operand, DFSMSrmm displays a *mediatype_id* of 0 and *mediatype_name* of *.

RECORDINGFORMAT(*recordingformat_id*,*recordingformat_name*)

Use this operand to identify the internally-used *recordingformat_id* to be converted to the externally-used *recordingformat_name* in reports and in the output of the DFSMSrmm TSO subcommands. Also, use this operand to identify the externally-used *recordingformat_name* to be converted to the internally-used *recordingformat_id* that is recorded in the control data set when used as input to RMM TSO subcommands.

When you specify multiple MEDINF entries that use the same *recordingformat_id*, but with different *recordingformat_name* values, the first such MEDINF entry is used for the internal-to-external conversion. The remaining values are synonyms used for external-to-internal conversions.

recordingformat_id is a number in the range from 1 to 255.

recordingformat_name is 1 to 8 alphanumeric or national characters.

Default: None. When you do not specify the RECORDINGFORMAT operand, DFSMSrmm displays a *recordingformat_id* of 0 and *recordingformat_name* of *.

CAPACITY(capacity)

Specifies the media capacity in MB available on the non-IBM media that has a media type *mediatype_id* and a recording format *recordingformat_id*. Specify a value from 0 to 4294967295.

When you specify a capacity value, DFSMSrmm uses that value to set the capacity of a volume when the volume is added or when the recording format or media type of the volume is manually changed. When new data is written on a volume, DFSMSrmm uses your specified capacity only if the tape drive does not return the volume capacity.

Do not specify the CAPACITY operand on a MEDINF command that specifies the same MEDINF NAME and MEDIATYPE *mediatype_id* and RECORDINGFORMAT *recordingformat_id* as an earlier MEDINF command. If you do, an information message EDG0243I is issued at start-up time, the CAPACITY operand is ignored, and the capacity from the earlier entry used instead. When you use synonyms, use the CAPACITY operand only when the combination of MEDINF NAME, MEDIATYPE *mediatype_id*, and RECORDINGFORMAT *recordingformat_id* are unique.

Default: When you do not specify the CAPACITY operand, DFSMSrmm displays a capacity of 0.

REPLACE(PERM | TEMP | AGE | WMC)

Use this operand to identify the policies for replacement of volumes based on such values as read and write errors, age, and numbers of times written. Volumes are identified for replacement when one or more of the policy values are met or exceeded.

Do not specify the REPLACE operand on a MEDINF command that specifies the same MEDINF NAME and MEDIATYPE *mediatype_id* and the RECORDINGFORMAT *recordingformat_id* as an earlier MEDINF command. If you do, an information message EDG0243I is issued at start-up time, the REPLACE operand is ignored, and the replace policy from the earlier entry is used instead. When you use synonyms, only use the REPLACE operand if the combination of MEDINF NAME, MEDIATYPE *mediatype_id* and RECORDINGFORMAT *recordingformat_id* are unique.

When you add or modify a volume replacement policy, DFSMSrmm does not implement it retrospectively. Volumes that are already set with the REPLACE release action are not reprocessed and the action reset. In order to have DFSMSrmm reconsider the policy for all non-pending release volumes, you first have to manually change the REPLACE release action to SCRATCH, and then run EXPROC processing.

When you do not specify replacement policies with MEDINF, DFSMSrmm uses the built-in value of REPLACE(PERM(1)).

You can specify a global REPLACE based on MEDINF NAME. For example:

```
MEDINF NAME(VEND1) REPLACE(PERM(1) AGE(20))
```

This policy is applied to all other MEDINF commands that specify the same NAME(VEND1), but do not specify the REPLACE operand. This is enough to override the built-in value of PERM(1) for all media with a MEDNINF name of VEND1.

For IBM media, the hard-coded default of PERM(1) can be overridden by coding a single MEDINF in parmlib. For example:

```
MEDINF NAME(IBM) REPLACE(PERM(0))
```

This example disables replacement for IBM media. To implement your own chosen replacement policies, customize the command, and then add it to DFSMSrmm parmlib.

You can also specify a media-level policy for all media of a certain type. You do this by specifying the MEDIATYPE operand without the RECORDINGFORMAT operand, such as the following example:

```
MEDINF NAME(IBM) MEDIATYPE(1,CST) REPLACE(AGE(30))
```

This example sets a default value to be used for all media with a *mediatype_id* of 1.

You can override the global and media level policies by specifying the REPLACE operand for a specific media type and recording format combination.

No matter how you define the MEDINF commands in parmlib, DFSMSrmm LISTCONTROL always lists a REPLACE policy for each MEDINF entry. The way DFSMSrmm determines the values is:

1. REPLACE is specified on the MEDINF command.
2. A media level REPLACE policy was specified.
3. A global REPLACE policy was specified.
4. The default of REPLACE(PERM(1)) is used.

When inventory management determines the policy to use, it does so as follows:

1. An exact match to the combination of MEDINF NAME, MEDIATYPE *mediatype_id*, and RECORDINGFORMAT *recordingformat_id*.
2. A match on the combination of MEDINF NAME and MEDIATYPE *mediatype_id* to a media-level policy.
3. A match to a global policy based on MEDINF NAME alone.
4. The default of REPLACE(PERM(1)) is used.

PERM(count)

Use this operand when your policy is to be based on permanent I/O errors. DFSMSrmm compares your value with the sum of permanent read and write errors over the life of the volume.

The value range is 0 to 99999. Specify 0 to disable replacement based on permanent errors.

There is no default value.

TEMP(count)

Use this operand when your policy is to be based on temporary I/O errors. DFSMSrmm compares your value with the sum of temporary read and write errors over the life of the volume.

The value range is 0 to 99999. Specify 0 to disable replacement based on temporary errors.

There is no default value.

AGE(years)

Use this operand when your policy is to be based on the age of the volume. DFSMSrmm compares your value with the volume creation, and current date and time.

The value range is 0 to 99999. Specify 0 to disable replacement based on age.

There is no default value.

WMC(count)

Use this operand when your policy is to be based on how many times the volume is mounted for output and written to. DFSMSrmm compares your value with the write mount count for the volume.

The valid value range is 0 to 65535. Specify 0 to disable replacement based on volume write mount count. Values in the range 65536 to 99999 are accepted, but have the same effect as specifying 0 (replacement based on volume write mount count is disabled).

There is no default value.

Using MEDINF REPLACE commands to implement volume replacement policies

DFSMSrmm automatically works with the tape drive SARS MIM alerts to track volume errors and volumes that need replacement. In addition, built-in policies for IBM media are used to determine when volumes should be replaced. You can create your own customized replacement policies using the MEDINF REPLACE values based on media type. You can create your replacement policy by specifying one or more of the following MEDINF REPLACE values:

AGE

Use this operand when your policy is to be based on your known usage characteristics and the expected life of the technology, not the expected failure of the tape media.

Properly cared for magnetic media should have a life expectancy equal to or greater than that of the recording technology concerned. When you select Age as one of the criteria for replacement of media, you are really deciding based on your known usage characteristics and the expected life of the technology at what future time you plan to replace the technology, not the expected failure of the tape media. Many experts believe that you should recopy data every 10 to 20 years to ensure that you have the technology required to read the data when required. For further reading on this subject, see *Care and Handling of Computer Magnetic Storage Media*, by S. G. Geller, National Bureau of Standards Special Publication 500-101, for sale by the Superintendent of Documents, U. S. Government Printing Office, Washington, D.C. 20402.

PERM

Use this operand when your policy is to be based on permanent I/O errors. DFSMSrmm compares your value with the sum of permanent read and write errors over the life of the volume. PERM(1) is the default value.

TEMP

Use this operand when your policy is to be based on temporary i/o errors. DFSMSrmm compares your value with the sum of temporary read and write errors over the life of the volume.

WMC

Use this operand when your policy is to be based on how many times the volume is mounted for output and written to. DFSMSrmm compares your value with the write mount count for the volume.

Once policies are implemented, you can regularly report on volumes to be replaced. Volumes that have already expired, and have the REPLACE action, can either be replaced and the action confirmed, or they can be deleted. Volumes with the REPLACE release action should be considered as candidates for copying so that any bad media can be replaced. If the volume is owned by an application that tracks the volume serial number, such as DFSMSHsm and OAM-owned tape volumes, you can use the built-in functions of that application to recycle, or move the data to a new volume. For all other data, you can use a product such as IBM Tivoli Tape Optimizer to copy data onto new volumes and recatalog the data, where necessary.

You can manually maintain the volumes to be replaced by disabling the built-in policy management provided by DFSMSrmm. To check for media errors, do one of the following:

- Use reports based on the extract file with the DFSMSrmm report generator.
- Use the EREP System Exception report.
- Contact your IBM Service representative to check the Service Agent reports.

For volumes that need to be replaced, set the release action to REPLACE and allow DFSMSrmm to manage the replacement of the volume.

See [“Defining media information: MEDINF” on page 148](#) for information about defining media information.

These sample reports, shipped in the DFSMSrmm Report Generator, can help you identify the volumes to be replaced:

- EDGGREPL - This sample report can help you identify the volumes that should be replaced. Volumes are selected for this report if the volume is pending release with the REPLACE action, or if the release action is set to REPLACE.
- EDGGREPV - This sample report can help you define your own customizable Volume Replacement Policies. You can use the report to determine for which volumes you will manually set the REPLACE release action. The sample report is set up to select volumes if one or more of the following is detected:
 - Write mount count > 99
 - > 25 years old and > 50% used
 - Temporary write errors > 20

- Permanent write errors > 1

See [z/OS DFSMSrmm Reporting](#) for information about the DFSMSrmm Report Generator.

Example of using the MEDINF STK command

Here is an example of tasks involved in assigning installation-defined media information (StorageTek, in this case) to a volume:

1. Ensure that volumes can be displayed, updated by commands, read from and written to, and scratched. The following steps assume that you have a StorageTek library attached and that HSC/MVS is running. Both physical and virtual testing can be performed.
2. Next, code the MEDINF commands in the EDGRMMxx parmlib member. The external values used for media type and recording format must match those that HSC returns for volumes. Check that the values in the sample are what you want to use. If you wish to use different values, you can add additional entries to exploit the alias capability for external values. The first external value for each internal value is used for the LISTVOLUME command; all other values allow aliases either by the user or from the HSC or CSC PGMI. You might not want to use all the sample MEDINF commands, but a complete set of media types and recording format combinations is better than just having the minimum necessary for your installation. This ensures that the internal numeric values used can be converted in the future. Do not set a MEDINF(STK) for any volumes yet. Issue the Modify command to DFRMM started procedure to refresh the parmlib member to be the one you have updated. Repeat the testing from task 1 as necessary.
3. Now, set one or two volumes that you plan to use, but not all of the volumes, to have MEDINF(STK). You can use the RMM CV *volser* MEDINF(STK) command. The existing media type and recording format will display using the STK external values, rather than the IBM values. The resultant displayed information probably will not make sense to you, but should not cause problems.

Select a volume and use the RMM CV *volser* MEDINF(STK) MEDIATYPE(*stkvalue*) RECORDINGFORMAT(*stkvalue*) command. The media type and recording format will now change for this volume. Once a volume has MEDINF STK, DFSMSrmm should no longer change the media type/format for this volume, unless the EDG_EXIT300 installation exit is in use, or you use additional DFSMSrmm commands to change this information.

4. If you are using the EDG_EXIT300 installation exit, this exit will work with HSC/MVS or CSC/MVS software and will try to record the correct values for your volumes.

Repeat these steps as necessary and, as the volumes are read or written, the correct media types and recording formats are recorded.

See [“Using the EDG_EXIT300 installation exit” on page 305](#) for information about the EDG_EXIT300 installation exit.

Defining mount and fetch messages: MNTMSG

Use the MNTMSG command to tailor mount and fetch messages so they display the volume serial number, rack number, and pooling decision. The pooling decision can be a pool prefix, pool name, or storage group name. The operator can use this information to pull and mount volumes.

MNTMSG is an optional parmlib command. Specify a MNTMSG command for any message you want DFSMSrmm to update. When you specify the MNTMSG command, you must include all the MNTMSG operands.

DFSMSrmm inserts the required information within the message text (if the message has no continuation lines) or at the end of the message based on the value you use with the RACK operand. For nonspecific mount requests, if you have defined a pool name with the VLPOOL command, DFSMSrmm provides a pool name or storage group name rather than a pool prefix in the message. DFSMSrmm updates mount messages with a pool name only when DFSMSrmm updates messages at the end of the message text.

Use the VOLUME operand to define the position of the volume serial number in a message. [Figure 64 on page 154](#) shows examples of the parmlib member EDGRMMxx MNTMSG command specified with the

VOLUME operand defined for 4-digit device numbers. If your definitions specify a volume serial number offset 1 less than the value shown in the examples, your definitions are from z/OS supported 3-digit device numbers and should be updated to match the position of the volume serial number when 4-digit device numbers are used.

```

/* MNTMSG - Add RACK= or POOL= at end of WTOs */
MNTMSG MSGID(IEF233A) ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID(IEF233D) ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEF234E K') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEF234E R') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEF234E D') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEF455D') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID(IEC501A) ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEC502E K') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEC502E D') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEC502E R ') ID(1) VOLUME(16) RACK(999)
MNTMSG MSGID('IEC502E RD') ID(1) VOLUME(17) RACK(999)
MNTMSG MSGID('IEC502E RK') ID(1) VOLUME(17) RACK(999)
MNTMSG MSGID(IAT5110) ID(1) VOLUME(44) RACK(999)
MNTMSG MSGID(IAT5210) ID(1) VOLUME(50) RACK(999)
MNTMSG MSGID(IAT5410) ID(1) VOLUME(20) RACK(999)

```

Figure 64. Parmlib member EDGRMMxx MNTMSG command examples for 4-digit devices

JES3 Considerations: When you use the JES3 IATUX71 exit, use the RACK operand to determine if you want the rack number to replace the volume serial number in the message or appended to the end of the message. If the RACK operand value indicates a position within the length of the message, DFSMSrmm replaces the volume serial number in the message with the required information. If the RACK operand value exceeds the message length, DFSMSrmm appends the information to the end of the message.

When the JES3 IATUX71 exit is not used, use the TYPE=MCS keyword on at least one CONSOLE statement in the JES3 parmlib. This activates JES3 MCS console support so JES3 issues messages as WTO messages rather than directly to JES3 consoles.

MNTMSG command syntax

Figure 65 on page 154 shows the syntax of the MNTMSG command:

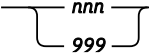
➤ MNTMSG — MSGID(nnnnnnnnnnn) — ID(nnn) — RACK() — VOLUME(nnn) ➤

Figure 65. Parmlib member EDGRMMxx MNTMSG command syntax

DFSMSrmm supplies a sample set of mount messages for your use. You can define additional MNTMSG commands to include other IBM operator messages and messages produced by your installation.

For example, if you define:

```
MNTMSG MSGID(IEF233A) ID(1) VOLUME(16) RACK(999)
```

your mount message displays:

```
IEF233A M 0480,999003,,J1400001,S0300 - RACK = T14103
```

If you specify:

```
MNTMSG MSGID(IEF233A) ID(1) VOLUME(16) RACK(16)
```

your mount message displays:

```
IEF233A M 0480,T14103,,J1400001,S0300
```

If you have a non-specific mount request where you have defined scratch pool AB* for that system, and you specify:

```
MNTMSG MSGID(IEF233A) ID(1) VOLUME(16) RACK(999)
```

your mount message displays:

```
IEF233A M 0480,PRIVAT,,J1400001,S0300 - POOL = AB****
```

If you specify:

```
MNTMSG MSGID(IEF233A) ID(1) VOLUME(16) RACK(16)
```

your mount message displays:

```
IEF233A M 0480,AB****,,J1400001,S0300
```

MNTMSG command operands

ID(*nnn*)

Specifies the starting position of the message identifier. Specify a value between 1 and 128.

Default: None. You must specify ID when you define a MNTMSG command in parmlib.

MSGID(*nnnnnnnnnnnnnnnn*)

Specifies message text 1 to 12 characters long. Normally the text is the message number, but it can include additional characters and blanks. You must enclose the MSGID value in quotation marks if you use blanks or special characters.

Default: None. You must specify MSGID when you define a MNTMSG command in parmlib.

RACK(*nnn|999*)

Specifies the position to insert the rack number, pool prefix, pool name, or storage group in the message. Specify a value between 1 and 128 to insert the value within the message text.

Use 999 if you want to add the value to the end of the message.

If the message has continuation lines, DFSMSrmm adds the required information to the end of the message regardless of the value specified with the RACK operand.

DFSMSrmm adds either RACK=rack_number or POOL=pool_value to the end of the message if there is enough space for the additional information. If you specify RACK(999), DFSMSrmm adds '- POOL=pool_value' if a pool name or storage group is substituted for a non-specific volume mount. For a rack number, DFSMSrmm writes *nnnnnn*. For a pool prefix, DFSMSrmm writes *nnnnn**. For a pool name or storage group, DFSMSrmm writes *nnnnnnnn*.

- If you specify RACK(999), DFSMSrmm adds - RACK=*nnnnnn* if a rack number is added:

```
IEF233A M 0480,999003,,J1400001,S0300 - RACK=T14103
```

- If you specify RACK(999), DFSMSrmm adds - POOL=*ppp** if a pool prefix is substituted for a non-specific volume mount:

```
IEF233A M 0480,PRIVAT,,J1400001,S0300 - POOL=AB****
```

- If you specify RACK(999), DFSMSrmm adds - POOL=pool_value if a pool name is substituted for a non-specific volume mount.

```
IEF233A M 0480,PRIVAT,,J1400001,S0300 - POOL=SCRTCH00
```

If there is not enough space to add the information, DFSMSrmm overlays the end of the message text with the 6 to 8 character rack number or pool value preceded by a -.

To display the pool prefix in the message, you must specify the SYSID operand on the VLPOOL command as described in [“Defining pools: VLPOOL” on page 209](#).

Default: None. You must specify RACK when you define a MNTMSG command in parmlib.

VOLUME(*nnn*)

Specifies the position of the volume serial number in the message. Specify a value between 1 and 128.

Default: None. You must specify VOLUME when you define a MNTMSG command in parmlib.

ONLYIF

ONLYIF allows to specify parmlib options that are only valid for a certain system. This makes it possible to use one common PARMLIB member to define the RMM setup for different LPARs. The command that follows ONLYIF is disregarded on all systems except the system specified in the ONLYIF parameter.

The OPTION command can be specified more than once to allow separate options, valid for all systems, from options for a specific system. ONLYIF can also be specified before the DEFAULT command in the EDGDEFxx parmlib member. For more information, see [Chapter 21, “Using the parmlib member EDGDEFxx,” on page 483](#).

The ONLYIF system option makes it easier to share RMM parmlib members among multiple instances of RMM in a SYSPLEX. The ONLYIF operand causes the parameter that follows the ONLYIF statement to be run only on the system that includes the same system name as specified in the **ONLYIF** command.

All other systems disregard the statements that follow the **ONLYIF** command. If duplicate operands are specified, DFSMSrmm uses the last value that was specified.

Multiple operands are possible with each ONLYIF statement. On the ONLYIF statement, no continuation character is needed. However, the last operand on each line, except the last line, needs a continuation character when multiple operands are used inside an ONLYIF statement [Figure 66 on page 156](#)).

```
ONLYIF SYSNAME(ZOS24)
OPTION JOURNALFULL(70) -
DEFTABLE(00)
ONLYIF SYSNAME(PRD24)
OPTION JOURNALFULL(75) -
DEFTABLE(00)
ONLYIF SYSNAME(PRD23)
LOCDEF LOCATION(OFFSITE2) TYPE(STORAGE) MEDIANAME(3480) -
MANAGEMENTTYPE(NOBSINS) AUTOMOVE(YES)
```

Figure 66. ONLYIF statement

Note: When DFSMSrmm is setting a default value for a parameter that is under the OPTION statement, the ONLYIF system options must be after the global OPTION statements; otherwise, the global defaults might override the statements in the ONLYIF. DFSMSrmm uses the last assignment of a value.

ONLYIF command syntax

[Figure 67 on page 156](#) shows the syntax of the ONLYIF command:

➤ ONLYIF — SYSNAME(*system_name*) ➤

Figure 67. Parmlib member EDGRMMxx ONLYIF command syntax

Value range: 1-8 of these characters: A-Z, 0-9, \$, @, #.

Defaults: None

SYSNAME(system_name) is the required parameter to specify the system id for which the command following should be executed. System name can be specified either in IEASYMxx parmlib member, the IEASYSxx parmlib member, or, by the operator, in response to write to operator with reply (WTOR) message IEA101A SPECIFY SYSTEM PARAMETERS. ONLYIF is valid before any command, including LOCDEF, MEDINF, MNTMSG, OPENRULE, OPTION, PRITITION, REJECT, SECCLS, VLPOOL.

Controlling the use of tape volumes: OPENRULE

Use the OPENRULE command to identify special processing for DFSMSrmm to perform during OPEN processing. It applies equally to both system-managed and non-system managed volumes.

In a shared CDS RMMplex, these commands allow you to control sharing of volumes between systems while still maintaining a single CDS. OPENRULE allows you to specify what cannot be used as well as what can be used.

The processing provided is based on volume sets, and these are identified by OPENRULE entries that you create using OPENRULE commands. When a volume is processed during OPEN processing, the volume is matched to an OPENRULE entry as follows:

- The TYPE(RMM|NORMM) is determined. For additional details, see the TYPE operand in [“OPENRULE command operands”](#) on page 158.
- DFSMSrmm matches the volume serial number to a volume set from most specific volume set to least specific volume set within TYPE.
- This command uses the VOLUMERANGE and VOLUME values including the volume prefix based on the set scope. DFSMSrmm OPENRULE entries are sorted in ascending EBCDIC collating sequence in most specific to least specific matching order within type. VOLUME(*) is considered to be the least specific.

Note: During OPEN processing, the first processing performed is the OPENRULE IGNORE action checking. Next, the PRITITION processing occurs before OPENRULE ACCEPT | REJECT processing, so that a volume identified as TYPE(NORMM) by PRITITION processing can be TYPE(RMM) at OPENRULE time. This occurs when an action of ACCEPT is set under PRITITION, and DFSMSrmm processing automatically adds the volume to the control data set.

If there are any OPENRULE or PRITITION commands found in EDGRMMxx parmlib, or there are no REJECT commands, default entries are created from the following command to cover any volumes for which you do not define an OPENRULE command:

```
OPENRULE VOLUME(*) TYPE(ALL) ANYUSE(ACCEPT)
```

You can list all the OPENRULE entries used by DFSMSrmm using the RMM LISTCONTROL OPENRULE command; which lists all entries including any created from the default command.

OPENRULE command syntax

[Figure 68 on page 158](#) shows the syntax of the OPENRULE Command.

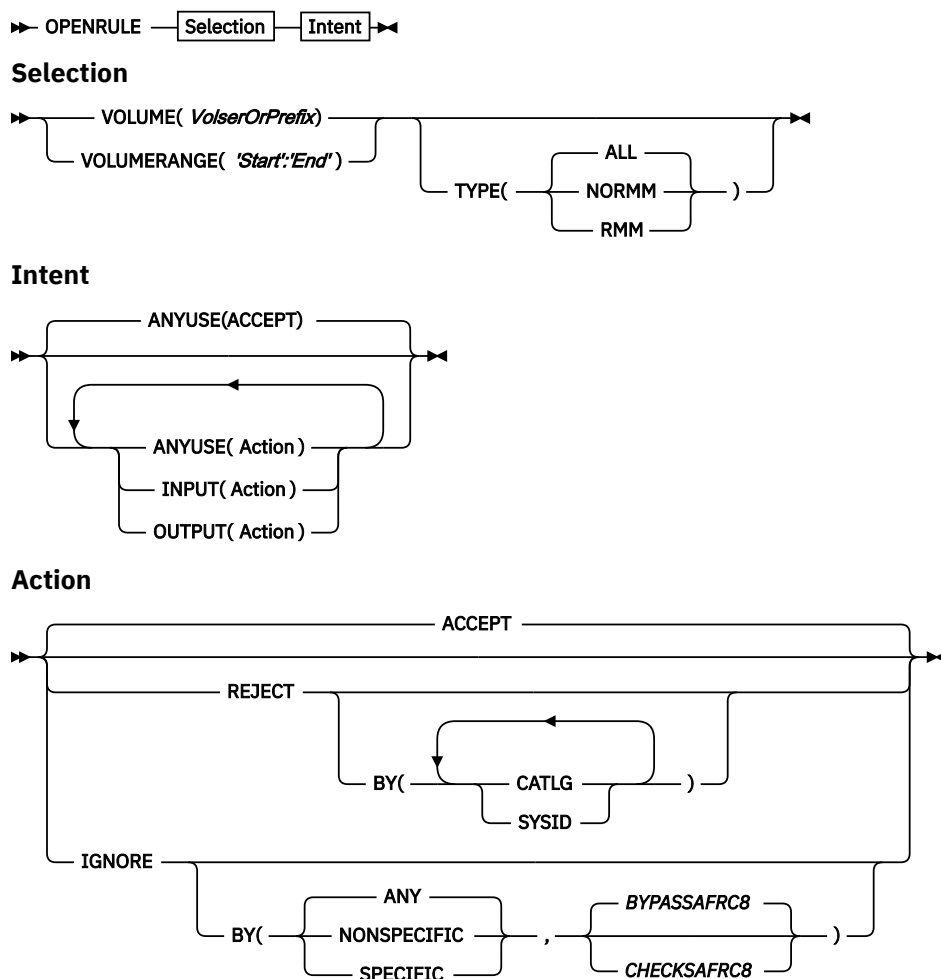


Figure 68. Parmlib member EDGRMMxx OPENRULE command

OPENRULE command operands

ACCEPT | IGNORE | REJECT

Use this operand to identify the action to be taken by DFSMSrmm. The action applies to OPEN processing.

ACCEPT

DFSMSrmm processes the volume. This attempt to open a file on the tape volume is also subject to DFSMSrmm open-time volume validation, and if allowed, the use of the volume and file is recorded by DFSMSrmm. ACCEPT means that the volume is accepted provided that the validation performed by DFSMSrmm at OPEN time allows the volume to be used. See the following for additional information:

- [“What tape usage does DFSMSrmm support?” on page 17](#)
- [“How does DFSMSrmm validate tape mounts?” on page 17](#)
- [“Why does DFSMSrmm reject tape volumes?” on page 18](#)

ACCEPT is the default value.

IGNORE

DFSMSrmm does not process the volume. When the mounted volume matches the requested volume, the use of the volume is ignored by DFSMSrmm. No validation of the volume is performed, and there is no recording of the volume or file. DFSMSrmm processing is as if the EDG_EXIT100 exit requested the volume be ignored. Volume ignore processing is attempted if either IGNORE is specified or the EDG_EXIT100 exit requests ignore. For ignore processing to be successful, the

user must be authorized to ignore the volume. The ignore processing is just as if you coded EXPDT=98000 in the JCL and were using the sample EDGUX100 exit module shipped with DFSMSrmm.

The use of action IGNORE enables you to ignore any specific or non-specific volume requests, including all those for system-managed volumes. However, you must use caution when using the IGNORE action with non-specific requests for system-managed volumes of type NORMM. Ensure that the PRTITION entries for the ignored NORMM volumes also specify IGNORE. If the PRTITION entry does not specify IGNORE, the DFSMSrmm processing of the CUA request (scratch change to private) causes the volume to be added automatically to the DFSMSrmm CDS, however the Open/Close/End-of-Volume (O/C/EOV) processing is not recorded.

Note: For specific volume requests, DFSMSrmm uses the requested volume to determine whether the volume is defined to DFSMSrmm, and uses the VOL1 label volser only to validate duplicate volumes. The requested volume is matched to the correct OPENRULE, and in this way correctly handles the identification and ignore action for duplicate volumes.

REJECT

DFSMSrmm must prevent OPEN processing from allowing use of the volume. For a specific volume, this results in the OPEN request failing. For a non-specific volume, the volume is dismounted by the system and another mount request issued.

ANYUSE | INPUT | OUTPUT (action)

Use this operand to identify the type of OPEN request issued by the application. It reflects the applications intent towards the data.

ANYUSE

The application is attempting either to read from or write to the tape volume. ANYUSE is the default operand. When you specify the INPUT or OUTPUT operands, they override the default.

INPUT

The application is attempting to read from the tape data set.

OUTPUT

The application is attempting to write to the tape data set.

BY(SYSID | CATLG)

Use this operand for volumes of TYPE RMM to specify that the REJECT action only applies if the volume is defined to DFSMSrmm, the request is for a specific volume, and the BY condition is not met. Otherwise, the ACCEPT action is used.

When specified for TYPE(NORMM), the operand is parsed successfully, but is then ignored and never used by DFSMSrmm because volumes of TYPE(NORMM) cannot have existing tape data set records and are not defined to DFSMSrmm.

You can specify one or more of the values: SYSID and CATLG. Use this operand to specify that the REJECT action applies if either applied by SYSID or by CATLG. Otherwise, the ACCEPT action is used. There is no default value.

SYSID

Use this operand to specify that for non-scratch volumes use of a volume is to be rejected if the creating SYSID of the first file does not match the current SYSID. This operand might help you if you have a common scratch pool across multiple systems, but once a volume is used, you only want the volume used or referenced on the system from which the data was created. When the creating SYSID matches the current SYSID, the ACCEPT action is used.

CATLG

Use this operand to specify that existing tape data sets must be referenced by their catalog entry. When the data set is referenced by its catalog entry, the ACCEPT action is used.

DFSMSrmm checks to see if there is an existing data set record defined to RMM and applies this rule if the data set record exists regardless of the disposition specified for the data set. In cases where only the first file is being recorded by DFSMSrmm, a data set record does not exist for file 2 onwards, but DFSMSrmm processing assumes that they exist.

BY(SPECIFIC | NONSPECIFIC | ANY)

Use this operand to identify the requests to which the IGNORE action applies.

SPECIFIC

The IGNORE action only applies if a specific volume is requested. Otherwise, the ACCEPT action is used.

NONSPECIFIC

The IGNORE action only applies if a non-specific request is being processed. The volser in the mount message is either PRIVAT or SCRTCH. Otherwise, the ACCEPT action is used.

ANY

The IGNORE action applies to all types of request.

ANY is the default value.

CHECKSAFRC8/ BYPASSAFRC8

Use this operand to identify the SAF Return code check for which the IGNORE action applies.

CHECKSAFRC8

The IGNORE action only applies if a user is authorized to access the requested tape volume/data set.

Otherwise, RMM rejects the use of this volume.

BYPASSAFRC8

The IGNORE action applies if user is authorized /not authorized to access the requested tape volume/data set(Bypassing the SAF return code).

BYPASSAFRC8 is the default value.

TYPE(ALL | RMM | NORMM)

Use to identify the type of volumes selected by the OPENRULE command. The volser and other values determined during OPEN are used to determine the TYPE, as follows:

- For OPEN processing, the same rules as volume ignore processing are used (see [“Ignoring duplicate or undefined volume serial numbers”](#) on page 275), and the TYPE that is assigned is based on the following:

- TYPE(RMM)

1. The volume serial number is defined in the control data set, and there is no HDR1 tape label for the volume.
2. The volume serial number is defined in the control data set, and no labels are used (NL, NSL, or BLP with an NL tape).
3. The 17 characters read from the HDR1 label of the mounted volume match the last 17 characters of the data set name in the control data set.

- TYPE(NORMM)

1. The volume serial number is not defined in the control data set.
2. The volume is defined, but the 17 characters read from the HDR1 label of the mounted volume do not match the last 17 characters of the data set name in the control data set.

For each set of volumes, you can code one command with RMM and another with NORMM, but only a single command if ALL is used.

RMM

The volume is defined to DFSMSrmm.

NORMM

The volume is not defined to DFSMSrmm.

ALL

Applies to all volumes regardless of whether they are defined to DFSMSrmm.

ALL is the default value.

VOLUME | VOLUMERANGE

Use these operands to select volumes that are to be managed by this command. You must specify either VOLUME or VOLUMERANGE, and each command defines a set of one or more volumes. Sets cannot overlap within a TYPE, but a set can be a subset of another and such a subset is more specific. When you code a set with TYPE(RMM) and repeat that set with TYPE(NORMM), you can specify different operands for each set. If you want to use the same operands for the sets, you can do so by coding a type of ALL.

VOLUME

You can specify the volume as fully qualified or a volser prefix ending in *. A fully qualified volume is one to six alphanumeric, national or special characters. A volser prefix is zero to five alphanumeric, national, or special characters ending in an asterisk. Single quotation marks are required for special characters, and the first character must not be blank. Any value ending in *, even if enclosed in quotation marks, is considered to be a volser prefix.

VOLUMERANGE

Use to select a subset of volumes based on starting and ending volser. Specify each volser value as one to six alphanumeric, national, or special characters. Single quotation marks are required for each value regardless of the use of special characters, and the first character must not be blank. The end of range must not be lower than the start of the range.

Note: Masks cannot be used in VOLUMERANGE, as * and other special characters are recognized as a valid volume serial character. To use a mask for OPEN or PRTITION rules, use the VOLUME keyword.

The following example will match all volumes between ABC000 and ABC999
VOLUMERANGE('ABC000':'ABC999')

The following example will match all volumes starting with ABC, including ebcdic and special characters in positions 4 through 6

VOLUME(ABC*)

Defining system options: OPTION

Use the OPTION command to define the installation options for DFSMSrmm. [Figure 69 on page 162](#) shows an example of the OPTION command and the operands that you can code in the parmlib member EDGRMMxx.

```

OPTION  OPMODE(P)                /* protect mode */ -
ACCOUNTING(J)                   /* Account information */ -
BACKUPPROC(RMMBKUP)            /* backup procedure */ -
BLP(RMM)                       /* bypass label process */ -
CATRETPD(12)                   /* catalog retention */ -
CATSYSID(SYSTEM1,SYSTEM2,SYSTEM3,SYSTEM4,SYSTEM5,
SYSTEM6,SYSTEM7,SYSTEM8) /* CATALOG SYS */ -
CDSID(MVS2)                    /* control data set ID */ -
COMMANDAUTH(OWNER)            /* security check */ -
DATEFORM(E)                   /* European dates */ -
DISPDDNAME(DISPDD)            /* DD card name */ -
DISPMSGID(EDG4054I)           /* WTO message number */ -
DSNAME(RMM.CONTROL.DSET)       /* control data set */ -
EXPDTDROP(PERCENT(10),INFO) /* EXPDT maximum dropped */ -
GDG(CYCLEBY(GENERATION) /* -
DUPLICATE(BUMP)) /* VRSEL GDG options */ -
HSKP(LIMITED)                 /* -
/* Housekeeping functionality switch */ -
IPLDATE(NO)                   /* ipl date */ -
JOURNALFULL(75)               /* journal percentage */ -
/* threshold */ -
JRNLNNAME(RMM.JOURNAL.DSET) /* journal */ -
JRNLTRAN(NO)                  /* only updates are */ -
/* journalled */ -
LINECOUNT(60)                /* lines per page */ -
LOCALTASKS(10)                /* number of tasks */ -
MASTEROVERWRITE(LAST)        /* overwrite default */ -
MAXHOLD(100)                  /* number of records */ -
MAXRETPD(NOLIMIT)             /* maximum retention */ -
MCATTR(VRSELXDI)              /* management class use */ -
MEDIANAME(3480)               /* media name */ -
MEMBER(&SYSCLONE)              /* system specific member */ -
MOVEBY(VOLUME)                /* movement processing */ -
MSG(M)                        /* message mixed case */ -
NOTIFY(N)                     /* no notification */ -
PDA(ON)                       /* PDA trace enabled */ -
PDABLKCT(255)                 /* PDA block count */ -
PDABLKSZ(12)                  /* PDA blocksize */ -
PDALOG(OFF)                   /* Disable trace logging */ -
PREACS(NO)                    /* Preacs option */ -
RETAINBY(VOLUME)              /* retention processing */ -
RETENTIONMETHOD(EXPDT(RETAINBY(SET),LASTREF(24))) /* -
/* EXPDT retain process */ -
RETPD(5)                      /* default retention */ -
REUSEBIN(CONFIRMMOVE)         /* reuse bin */ -
SCRATCHPROC(RMMSCR)          /* scratch procedure */ -
SMFAUD(YES)                   /* SMF records */ -
SMFSEC(YES)                   /* SMF records */ -
SMSACS(NO)                    /* SMSACS option */ -
SMSTAPE(UPDATE(EXITS,SCRATCH,COMMAND),PURGE(YES)) /* -
/* TCDB update and purge */ -
SYSID(&SYSNAME.)              /* use system name symbol */ -
TPRACF(AUTOMATIC)            /* automatic */ -
TVEXTPURGE(RELEASE)          /* EDGTVEXT action */ -
UNCATALOG(Y)                  /* catalog option */ -
VRSCHANGE(VERIFY)            /* VRS change information */ -
VRSEL(NEW)                    /* VRS processing */ -
VRSJOBNAME(2)                 /* retention by job name */ -
VRSDROP(PERCENT(10),INFO)     /* VRS maximum dropped */ -
VRSMIN(1,FAIL)                /* VRS minimum */ -
VRSRETAIN(PERCENT(80),INFO) /* VRS minimum retained */ -

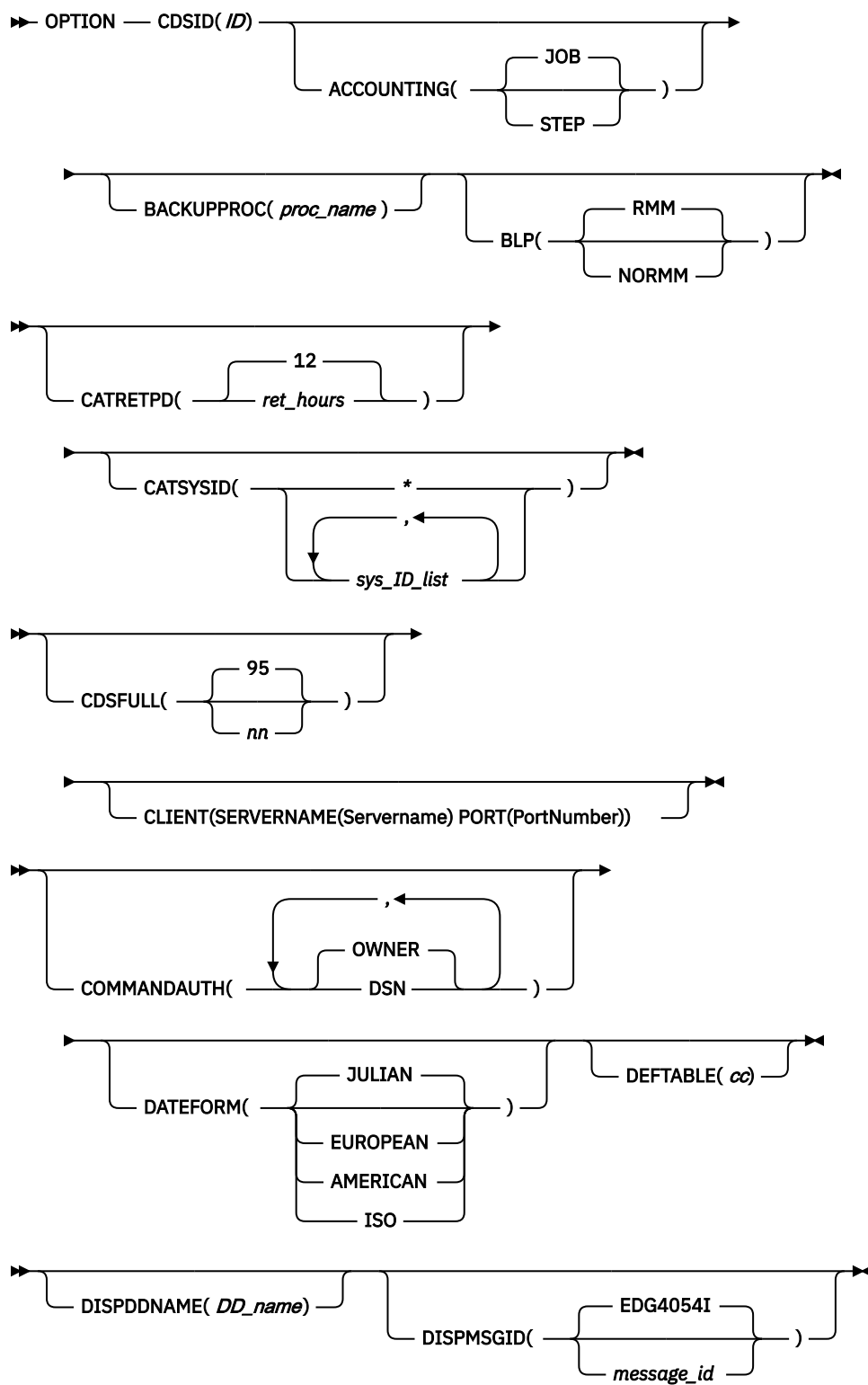
```

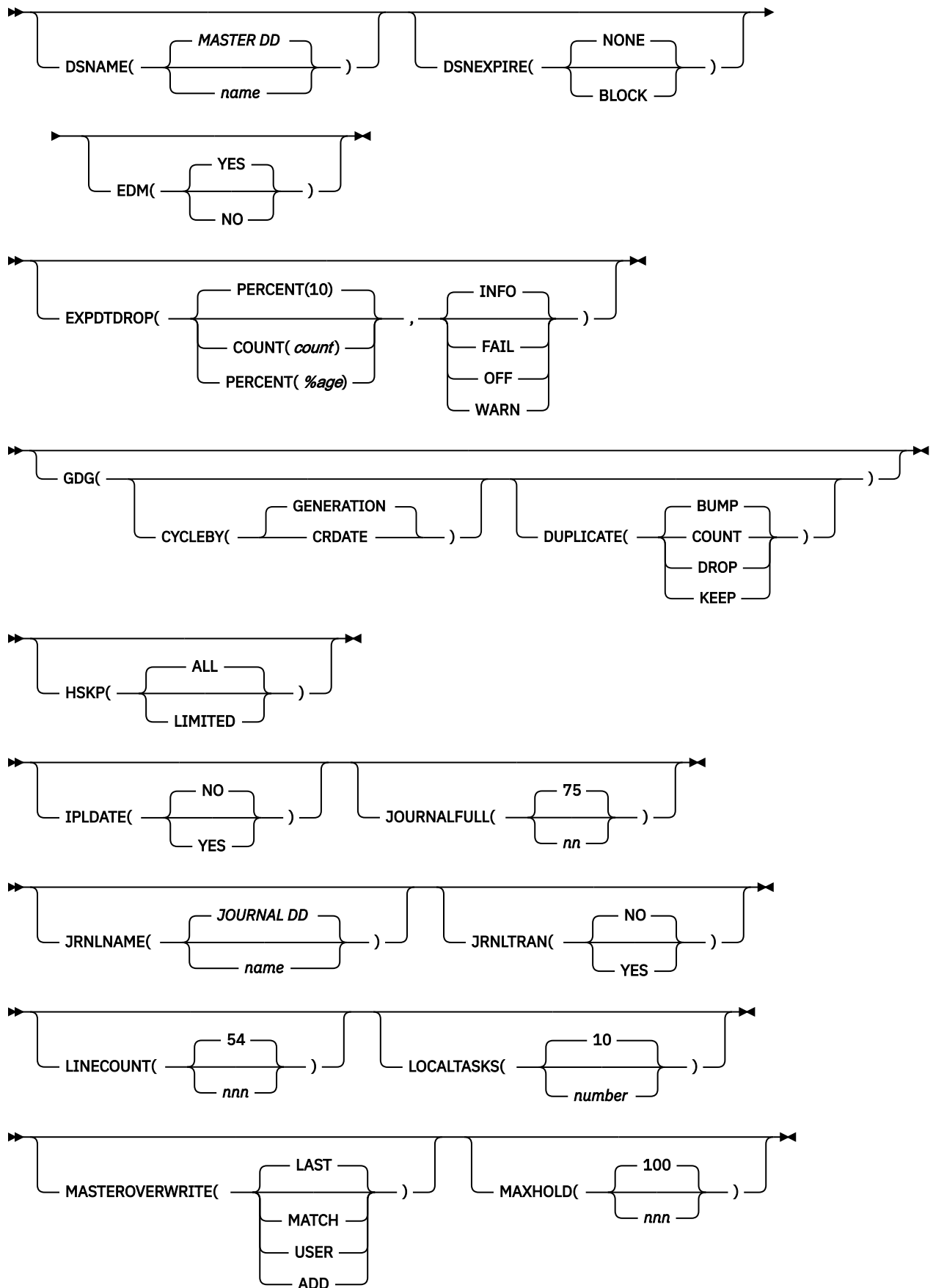
Figure 69. Parmlib member EDGRMMxx OPTION command examples

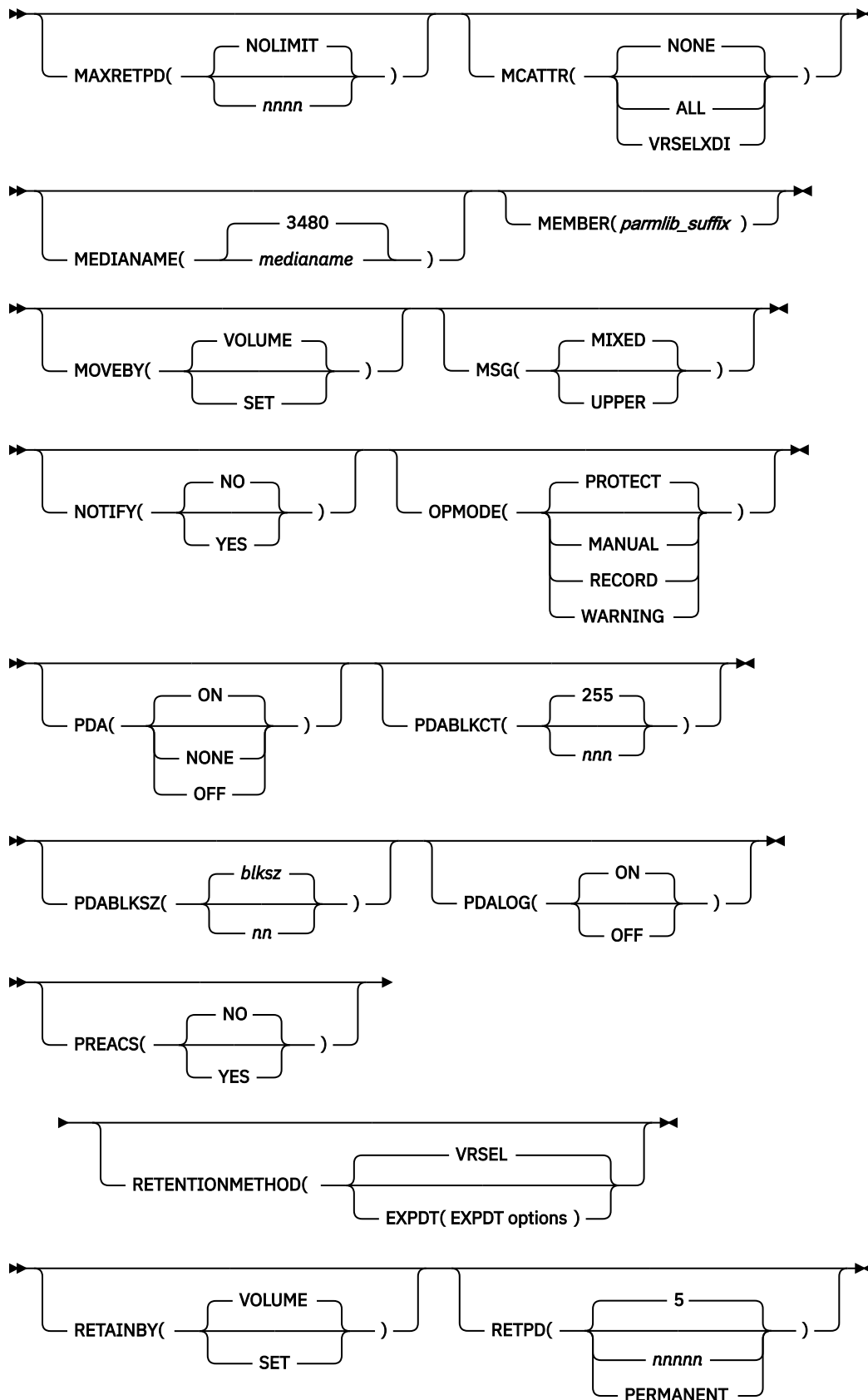
OPTION command syntax

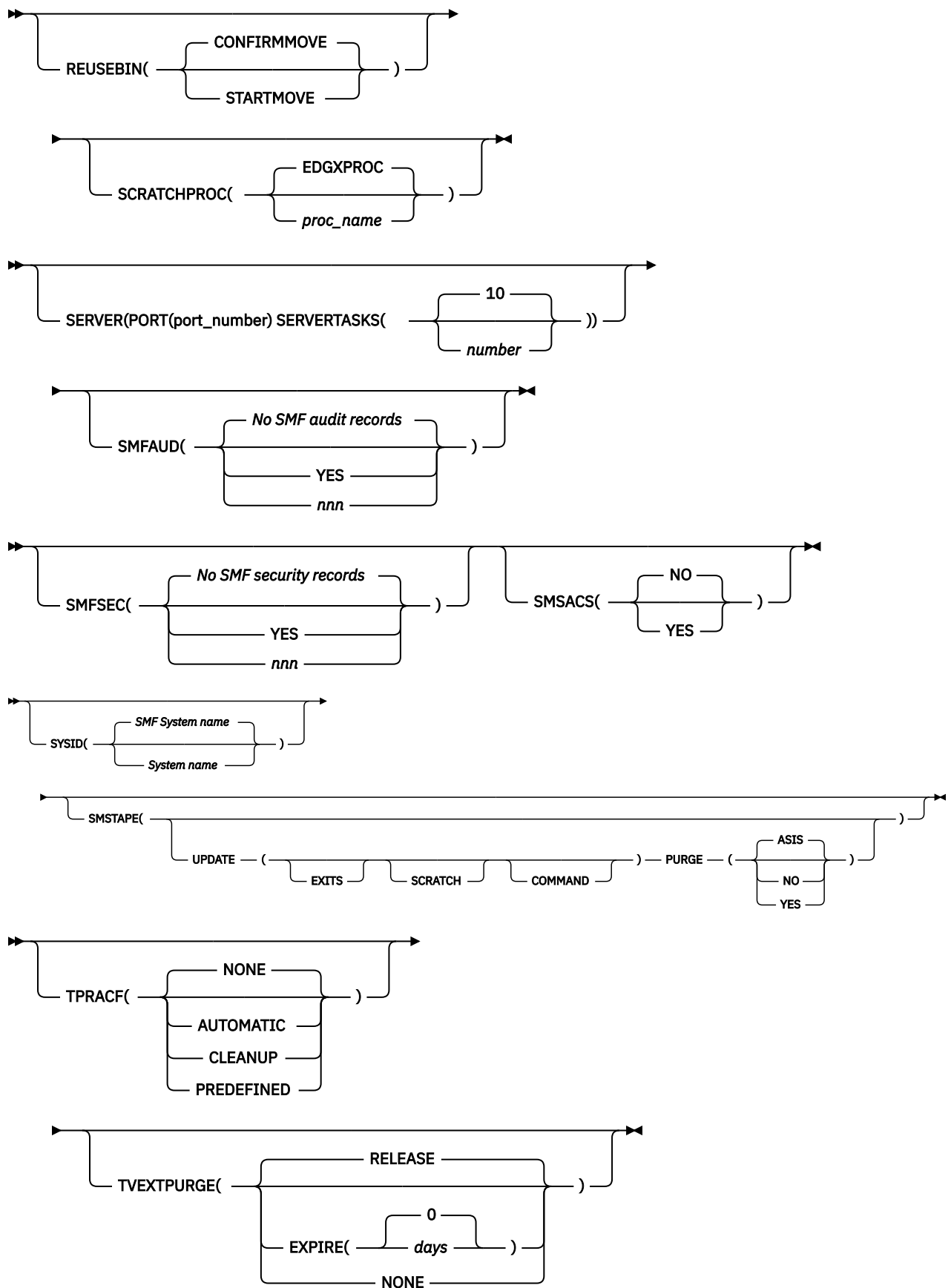
The following syntax diagram shows the syntax of the OPTION command:

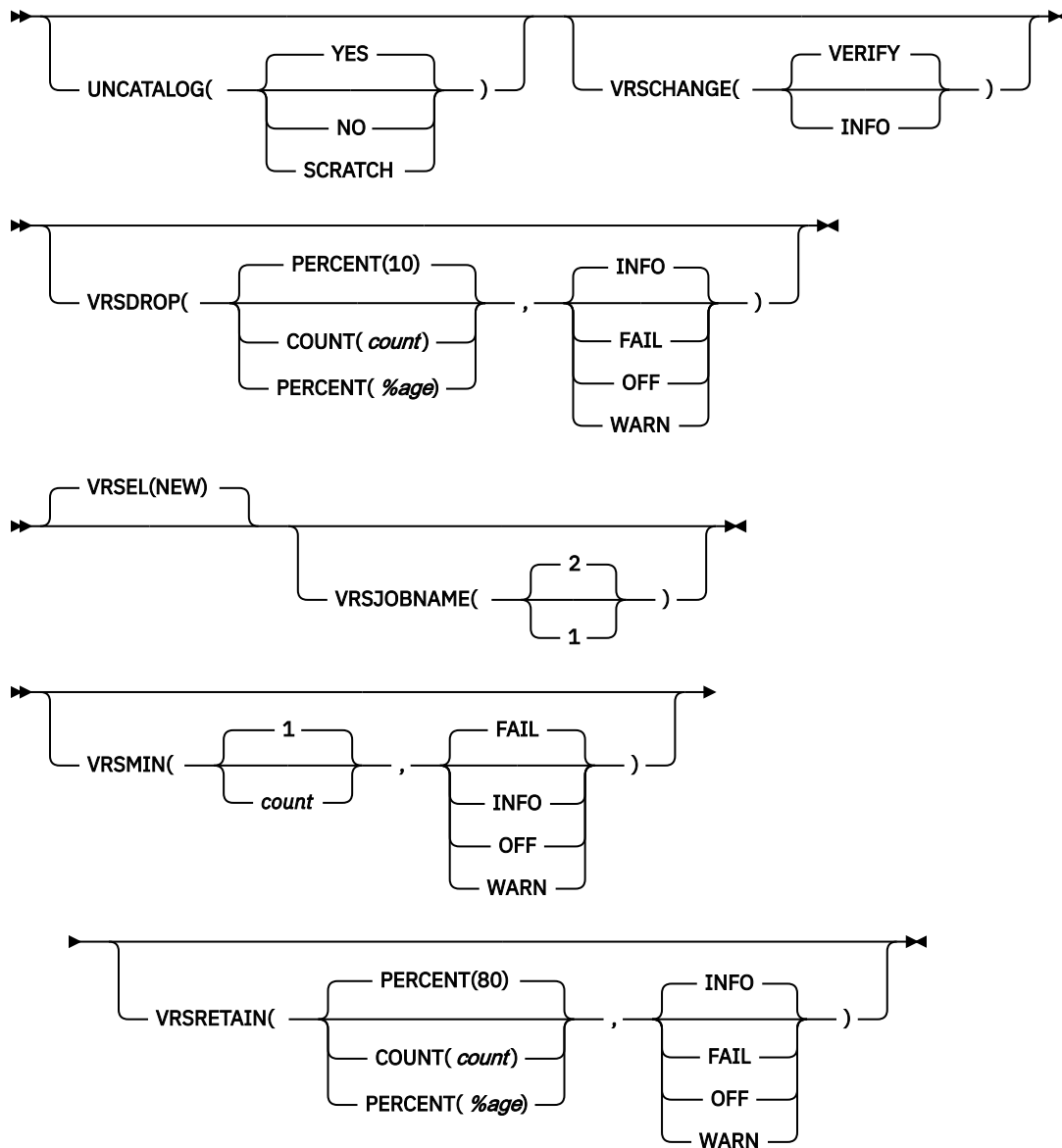
Figure 70. Parmlib member EDGRMMxx OPTION command syntax



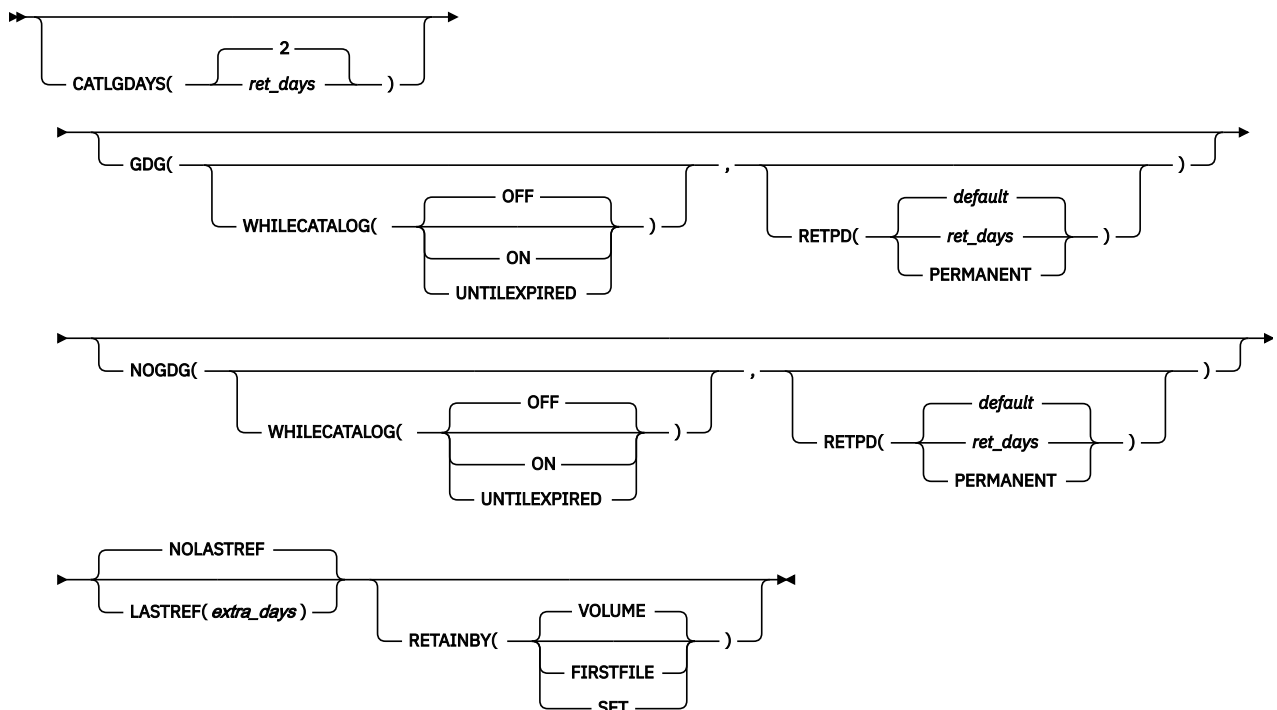








EXPDT options



OPTION command operands

CDSID(ID)

Specifies the identifier of the control data set that must be used on this system. Specify a value one to eight characters long. Ensure that each DFSMSrmm control data set has a unique CDSID ID.

When you start DFSMSrmm, the CDSID ID is compared to the ID in the control data set control record. If the IDs match, DFSMSrmm startup continues. If the control data set does not have an ID, DFSMSrmm creates the ID in the control record from the CDSID. If the IDs do not match, DFSMSrmm startup fails and DFSMSrmm issues a message to the operator to select another parmlib member.

If you do not specify a value for CDSID, you cannot start DFSMSrmm (on toleration systems, a warning message is issued and startup continues if the DFSMSrmm control data set does not have a CDSID). See [“Creating or updating the control data set control record” on page 409](#) for information about how the DFSMSrmm EDGUTIL utility sets the control data set ID.

Default: None.

This operand is required.

ACCOUNTING(JOB|STEP)

Specifies whether DFSMSrmm records JOB or STEP accounting information along with volume information.

JOB

DFSMSrmm records the accounting information from the JOB statement of the JCL.

STEP

DFSMSrmm records the accounting information from the EXEC statement of the JCL. If you specify STEP and there is no accounting information in the EXEC statement, DFSMSrmm records the JOB statement accounting information.

Default: ACCOUNTING(JOB).

BACKUPPROC

Specifies the name of the procedure that you want to be started automatically when the journal percentage full threshold is reached.

Specify a valid alphanumeric procedure name from 1 to 8 characters. If no name is specified, then no automatic start command is issued. See [“Steps for automating control data set backup and journal clearing”](#) on page 374 for more information.

Default: None. DFSMSrmm ignores BACKUPPROC on the client system.

BLP(RMM|NORMM)

Specifies how DFSMSrmm controls bypass label processing (BLP).

Authorization to perform BLP is still dependent on the ICHBLP resource in the RACF FACILITY class, and the ability to use BLP can still be controlled by JES.

Note: DFSMSrmm allows BLP processing to continue if a volume is mounted with a VOL1 header label that matches the volume serial number that is specified in JCL or if the volume has no label. When a labeled volume is mounted and the volume serial number does not match the requested volume, DFSMSrmm prevents processing when either volume is defined to DFSMSrmm. To circumvent this DFSMSrmm processing, you can request that DFSMSrmm ignores the volume serial number as described in [“Ignoring duplicate or undefined volume serial numbers”](#) on page 275.

RMM

You can use BLP for input from and output to volumes in user status and for input from volumes in controller status.

NORMM

You can use BLP under the normal system controls and DFSMSrmm records the activities that you perform on tapes. You can also use BLP for input from and output to volumes in user and controller status, and for output to scratch tapes. BLP can be used for reading and writing of controller and user status tapes and for output to scratch tapes. BLP read of scratch tapes is not supported.

For scratch tapes written using BLP, DFSMSrmm changes the volume to controller status and sets the initialize release action so that the tape is correctly labeled on return to scratch. DFSMSrmm also overrides the logical volume serial number that is generated by OPEN for BLP output to scratch tapes, so that the correct volume serial number is used for cataloging of data sets.

DFSMSrmm does not allow no label (NL) tapes to be mounted in response to a scratch request. However, you can use BLP to create NL tapes during scratch processing.

Default: BLP(RMM)

CATRETPD(*ret_hours*)

Specifies the number of hours from creation that a data set should be retained if it has not been cataloged and matches a vital record specification with the WHILECATALOG operand.

DFSMSrmm retains the data set for the catalog retention period if the data set has never been cataloged. DFSMSrmm does not retain the data set if DFSMSrmm detected that the data set was cataloged and then uncataloged during the catalog retention period.

CATRETPD can also be used to ensure that new data sets using the EXPDT retention method are not expired before the catalog signal is received when the default WHILECATALOG setting is not OFF.

ret_hours can be 0 to 9999 hours. For example, CATRETPD(24) keeps data sets for 24 hours. Set *ret_hours* to 0 to request that DFSMSrmm does not perform this processing.

Default: CATRETPD(12).

CATSYSID(*|*sys_ID_list*)

Specify CATSYSID to enable DFSMSrmm catalog synchronization. Use the CATSYSID operand to the user catalogs you want tracked when you run the DFSMSrmm EDGHSKP utility with the CATSYNCH operand to use catalog status tracking. All the systems that you identify that identify must have the code that supports catalog synchronization. See [“Running DFSMSrmm catalog synchronization”](#) on page 365 for more information.

CATSYSID(*) means that all catalogs are fully shared. You must specify an * to specify that catalogs are fully shared so that any data set can be processed by DFSMSrmm on any DFSMSrmm subsystem.

CATSYSID(sys_ID_list) provides a list of DFSMSrmm system IDs that share tape data set user catalogs with this DFSMSrmm subsystem. You can identify up to 16 system IDs for DFSMSrmm subsystems. You must specify a list of system IDs when user catalogs are not shared and must ensure they are synchronized with the DFSMSrmm control data set before you run inventory management vital record processing or expiration processing. Be sure to include IDs for systems that you no longer use but for which you are still retaining tape data sets.

Default: None.

CDSFULL(nn)

Specify CDSFULL to define a percentage full threshold for the CDS data set. When DFSMSrmm detects that the CDS has reached this threshold, DFSMSrmm issues message EDG2120W. DFSMSrmm also issues message EDG2120W at DFSMSrmm startup if the CDS has already reached the threshold specified. If you specify a value of 0, DFSMSrmm issues no warnings on that system.

Note that it can be possible to add new records into the CDS even after it reaches 100% utilization. Still, the High Used RBA/High Available RBA ratio used for the CDS Utilization value is the best indicator of possible shortage of space in a VSAM data set such as the RMM CDS.

Specify a value in the range 0-99.

Default: 95.

CLIENT(SERVERNAME(ServerName) PORT(PortNumber))

Specifies the type of system you want to set up. CLIENT is mutually exclusive with SERVER. If neither client nor server are specified, DFSMSrmm starts as a standard system

SERVERNAME(servername)

SERVERNAME is a required operand when you specify CLIENT. The *servername* can either be an IP address, a fully qualified domain name, or a server hostname.

An IP address can be specified either as a hexadecimal IPv6 address or a numeric IPv4 address. Specify an IPv6 IP address only when the server is running z/OS V1R12 or later release and TCP/IP on the server system has IPv6 dual mode IP stack. For example, CLIENT(SERVERNAME(1080:0:0:0:8:800:200C:417A)) specifies an IPv6 IP address. See [RFC 3513 \(tools.ietf.org/html/rfc3513\)](https://tools.ietf.org/html/rfc3513) for the syntax of an IPv6 IP address.

When a domain name or a host name is specified, DFSMSrmm uses the domain name system (DNS) to resolve that name into an IP address. The host name can be a maximum of 63 characters. The host name must contain one or more tokens separated by a period. Each token must be larger than one character. The first character in each token must start with a letter. The remaining characters in each token can be a letter, number, or hyphen. For example, CLIENT(SERVERNAME(RMMPLX1.MAINZ.IBM.COM) PORT(1950)) tells DFSMSrmm to start as a client without direct DASD access and to share the tape inventory in access by the RMM server with the host name RMMPLX1 using network IP protocol port 1950.

PORT(PortNumber)

Use this operand to specify the port number to be used for IP communication. The PORT operand is required. Specify a value from 1024 to 65535. Port numbers 1 to 1023 are reserved. Also, the client port number and server port number must match for the systems to communicate.

Default: None.

COMMANDAUTH(OWNER DSN)

Specifies the type of authorization that DFSMSrmm is to check. Specify OWNER when you expect the owners of volume information and data set information to be able to update their own data sets and volumes using RMM TSO subcommands. Specify DSN when you expect changes to volume and data set information to be authorized using the RACF DATASET class and TAPEVOL class.

You can set up authorization so that DFSMSrmm checks for authorization by owner first, and then checks for authorization using the DATASET class and TAPEVOL class. Set up this type of checking by specifying both the OWNER operand and the DSN operand separated by a comma. When the RACF name-hiding function is enabled in the system, this overrides the DFSMSrmm COMMANDAUTH

processing. DFSMSrmm processing, when the name-hiding function is enabled, is the same as COMMANDAUTH(DSN).

Default: COMMANDAUTH(OWNER)

DATEFORM(AMERICAN | EUROPEAN | ISO | JULIAN)

Specifies the date format for messages and reports. See [“EXEC parameters for EDGHSKP” on page 337](#) for information about setting different date formats for reports.

Value	Language	Format	Example
A	American	mm/dd/yyyy	12/15/2013
E	European	dd/mm/yyyy	15/12/2013
I	ISO	yyyy/mm/dd	2013/12/15
J	Julian	yyyy/ddd	2013/349

Default: DATEFORM(JULIAN)

DEFTABLE(cc)

Specifies the two-letter suffix for the defaults table, located in the EDGDEFxx member of the parmlib. See [Chapter 21, “Using the parmlib member EDGDEFxx,” on page 483](#) for more information about the defaults table.

Default: None

DISPDDNAME(DD_name)

Specifies the name of the DD card which identifies the data set that contains disposition control statements that DFSMSrmm processes during CLOSE or EOVS processing. The data set must be a sequential file and must be defined with LRECL 80. The data set can be a member of a partitioned data set. When you specify the DISPDDNAME operand, you are requesting that DFSMSrmm performs disposition processing during CLOSE or EOVS processing.

If you code DISPDDNAME, you must provide the name of the DD card that identifies the data set containing the disposition control statements optionally included in your JCL.

For information about DFSMSrmm disposition processing, see [Chapter 18, “Setting up DFSMSrmm disposition processing,” on page 467](#).

Default: None.

DISPMSGID(message_id)

Specifies the message number that DFSMSrmm uses for write-to-operator messages specified in the disposition control file.

EDG4054I

The message text provides the device number, volume serial number, volume sequence, the location where the volume is to move, and any message text you defined in the disposition control file.

message_id

You can define any alphanumeric value of up to eight characters.

For information about DFSMSrmm disposition processing, see [Chapter 18, “Setting up DFSMSrmm disposition processing,” on page 467](#).

Default: Message EDG4054I.

DSNAME(name)

Specifies the name of the DFSMSrmm control data set. Specify a name up to 44 characters long.

If you do not specify DSNAME, you must specify the data set name in the MASTER DD statement in the DFSMSrmm started procedure. If you specify a name both for DSNAME and MASTER DD, DFSMSrmm ignores the MASTER DD statement.

DFSMSrmm ignores DSNAME on the client system.

DSNEXPIRE(BLOCK | NONE)

The DSNEXPIRE option allows for blocking access to data sets that are expired.

NONE

Data sets that are no longer retained and which have an expiry date in the past are still accessible until the volume on which they reside is scratched. Data sets modified using the RMM CHANGEDATASET subcommand with the FORCEEXPIRE operand are not accessible even with DSNEXPIRE(NONE).

BLOCK

RMM blocks access to data sets that are expired and not retained by a Vital Record Specification. Newly created data sets that are not VRSELEXCLUDED are not blocked until VRSEL housekeeping has a chance to retain them.

Data sets that are retained for the following reasons are not blocked, even if their expiration date is in the past: the data set resides on a volume that has HOLD set to YES, or EDM set to YES, or the data set is cataloged and has WHILECATALOG set to ON.

For NL (not labeled) tapes, a data set that is expired is blocked even if the JCL has the wrong data set name, as long as the sequence number of the data set matches that stored in the RMM CDS. BLP (bypass label processing) cannot be used to read or write expired data sets when DSNEXPIRE(BLOCK) is set.

Note: Instead of using DSNEXPIRE(BLOCK), you can manually block selected data sets using the RMM CHANGEDATASET subcommand with the FORCEEXPIRE operand. Such data sets will show "(ForceExpire)" next to their expiration date in the RMM LISTDATASET subcommand output.

EDM(NO | YES)

Specifies this operand to control whether the dynamic setting of the volume EDM attribute occurs.

NO

Specify NO to avoid dynamic setting of the volume EDM attribute at OPEN time when a volume is written by the programs ANZSRVR, ANRSERV, ARCCTL, ARCWCTL, CBROAM, DSMSEVR unless a non-permanent expiration date is specified in the DD statement.

All other EDM functions (ineligibility for expiration, the dynamic resetting of the volume EDM attribute, possibility to set and reset the attribute via RMM CHANGEVOLUME, disability to release a EDM-managed volume) are in effect despite the operand value.

YES

Specify YES to allow dynamic setting of the volume EDM attribute. It occurs at OPEN time when a volume is written by the programs ANZSRVR, ANRSERV, ARCCTL, ARCWCTL, CBROAM, DSMSEVR unless a non-permanent expiration date is specified in the DD statement.

Default: EDM(YES).

EXPDTDROP(COUNT(count)|PERCENT(%age),action)

Use EXPDTDROP to specify a maximum number or percentage of existing expiration date retained volumes that can be dropped from retention and the action to be taken by DFSMSrmm. DFSMSrmm counts the number of EXPDT-retained physical and logical volumes at the start of inventory management expiration processing and the number of these to be set to pending release. An EXPDT-retained volume is one that is not VRS-retained and is not newly assigned. When you specify count, this is an absolute maximum number of volumes that can be released by a single run of EDGHSKP EXPROC processing. When you specify a percentage, this is a maximum percentage of the existing EXPDT-retained volumes that can be released by a single run of EDGHSKP EXPROC processing. This processing occurs each time that you run inventory management EXPROC processing.

When VRSEL and EXPROC are run together in a single EDGHSKP run, volumes that are dropped by VRSEL processing are counted only towards the VRSDROP limit, not to the EXPDTDROP limit.

EXPDTDROP processing counts all processed volumes that are retained only by volume expiration date at the start of the expiration processing run and also which of these are set to pending release. When the EXPROC SYSIN command causes a subset of volumes to be processed, only those volumes are counted.

EXPDTDROP processing is intended to provide limited checking for volumes that are not VRS-retained. The volumes would previously have been through VRSRETAIN limit checking and if retained by VRS, the VRSDROP limit checking as well. It considers how many of the EXPDT-retained volumes expire during the EXPROC run. Those volumes that are not set pending release will be considered by EXPDTDROP limit processing on the next run of EXPROC.

The total number of volumes set pending release during EXPROC processing includes:

- Those considered by VRSRETAIN limit checking, but not VRS-retained.
- Those considered by VRSDROP limit checking that were dropped from VRS retention.
- Those considered by EXPDTDROP limit checking.

count can be 0 to 2,147,483,647. *%age* can be 0 to 100.

Specify *action* to control the action DFSMSrmm takes during processing and when the value is exceeded. *action* can be FAIL, INFO, OFF, or WARN.

If You Specify	DFSMSrmm
FAIL	Issues messages EDG2427I and EDG2428I to the MESSAGE file. When the value is exceeded, DFSMSrmm stops expiration processing prior to making any updates to volume records and in addition, message EDG2310I is issued, report extract is run if requested, and any other inventory management, processing ends with return code 12. Updates to data set records might have been made by VRSEL processing.
INFO	Issues messages EDG2427I and EDG2428I to the MESSAGE file and processing continues.
OFF	Processing of this function is turned off.
WARN	Issues messages EDG2427I and EDG2428I to the MESSAGE file. When the value is exceeded, DFSMSrmm additionally issues message EDG2321W, sets a minimum return code of 4 and processing continues.

The default is EXPDTDROP(PERCENT(10),INFO).

To aid analysis of the results of the EXPDTDROP limit checking (when the action is not OFF) you can use the contents of the ACTIVITY file and extended records from the extract file. The EDGJACTP sample generates a detailed report and a summary report of expiration date retained volumes showing why they are set to pending release. See the contents of the EXPDROP and EXPDROPS files.

GDG

Use the GDG option to specify how generation data groups are handled for cycle retention by VRSEL processing. Cycle retention includes both the CYCLES and the BYDAYSCYCLE retention types. The correct sequence for determining the retention can be either by using the generation number or the creation order. You can also specify how duplicate generations (generation data sets) are handled and have the flexibility to include or exclude duplicate generations from the cycles count as required by your application processing.

The GDG option has two operands, CYCLEBY and DUPLICATE.

Use the CYCLEBY operand to specify whether retention is to be based on generation number or on creation date:

CYCLEBY(GENERATION)

Specifies that retention is to be based on the generation number. DFSMSrmm determines the generation number by applying a similar algorithm to that used by Catalog processing. Both the creation order, and the generation number from the data set name are considered, allowing wraps in generation number to be correctly handled.

CYCLEBY(CRDATE)

Specifies that only the creation sequence is to be used to determine the retention.

Default: CYCLEBY(GENERATION)

Use the DUPLICATE operand to specify how VRSEL processing handles duplicate generations. You can specify one of:

- Count duplicate generations
- Keep duplicate generations, but do not count them
- Bump duplicate generations from the current subchain
- Drop duplicate generations from VRS retention

Duplicate generations are determined within a single VRS sub chain and only if the generation and the generation it duplicates are not already dropped for another reason. For GENERATION based cycles, the duplicate generations are determined in generation (data set name), then creation order. For creation date based cycles, the duplicate generations are determined only in creation order, so they can be detected only when they are created consecutively.

Duplicates are processed within the context of the matching VRS chain or sub chain depending on the DUPLICATE option. DROP is within the context of the VRS chain and all others are within the context of the sub chain. A duplicate generation is considered a duplicate for cycles retention only if the generation it duplicates is retained by the current VRS sub chain.

DUPLICATE(BUMP)

Specifies that a duplicate generation is to be bumped by the current VRS sub chain and considered for retention by a subsequent VRS sub chain.

DUPLICATE(COUNT)

Specifies that a duplicate generation is to be treated like a non-duplicate generation regarding CYCLE and BYDAYSCYLE processing.

DUPLICATE(DROP)

Specifies that a duplicate generation is to be dropped from VRS retention without further consideration.

DUPLICATE(KEEP)

Specifies that a duplicate generation is considered as either the same CYCLE or the same BYDAYSCYLE depending on the VRS retention type and regardless of the duplicate generation creation date.

Default: DUPLICATE(BUMP)

HSKP(ALL|LIMITED)

This parameter specifies whether housekeeping would be having full or restricted functionality

ALL

If ALL is set, then it will allow to execute/run all functions under housekeeping job

LIMITED

If LIMITED is set, then inventory management functions CATSYNC, DSTORE, EXPROC and VRSEL are restricted as the CDS is not allowed to be updated. Following functions are allowed VRSEL(Verify), CATSYNC(VERIFY), RPTEXT and BACKUP

IPLDATE

IPLDATE(NO | YES) specifies whether IPL date checking is required. Specify YES to request IPL date checking, or NO to bypass it.

If you specify YES, DFSMSrmm issues a write-to-operator message during startup only if the date of the last run of expiration processing did not occur within two days of the current date. The message prompts the operator to enter the current date and day of the week. Initialization does not proceed until DFSMSrmm receives a valid reply. If you specify NO for this operand value, DFSMSrmm does not issue a message.

The IPLDATE operand helps prevent you from running expiration processing with an incorrect system date that can result in expiration of unexpired volumes.

Under normal conditions, the operator message requesting the current date is only issued once per IPL of z/OS. If you restart DFSMSrmm, it does not recheck the date.

Additionally, if you run expiration processing daily, a message is not issued. As a result, it is possible under normal conditions to run for a long time and never have the operator prompted to confirm the system date.

Default: IPLDATE(NO)

JOURNALFULL(nn)

Specify JOURNALFULL to define a percentage full threshold for the journal data set. When DFSMSrmm detects that the journal has reached this threshold, DFSMSrmm issues message EDG2107E.

DFSMSrmm also issues message EDG2107E at DFSMSrmm startup if the journal has already reached the threshold specified. DFSMSrmm issues message EDG2108E as a reminder until the backup completes and the journal is reset. If you specify a backup procedure name on the BACKUPPROC operand, the procedure is started automatically. If you specify a value of 0, DFSMSrmm issues no warnings on that system. You can specify different threshold values for sharing systems. See [“Steps for automating control data set backup and journal clearing” on page 374](#) for additional information.

Specify a value in the range 0-99.

Default: 75.

DFSMSrmm ignores JOURNALFULL on the client system.

JRNLLNAME(name)

Specifies the name of the journal. Specify a name up to 44 characters long.

If you do not specify JRNLLNAME in EDGRMMxx, you can specify a name in the JOURNAL DD statement in the DFSMSrmm started procedure. If you do not specify a journal name in either EDGRMMxx or the started procedure, DFSMSrmm does not provide journaling. If you specify a name in both, DFSMSrmm uses the JRNLLNAME value and ignores the JOURNAL DD statement.

DFSMSrmm ignores JRNLLNAME on the client system.

JRNLTRAN(NO | YES)

Use the JRNLTRAN operand to specify whether the unchanged copy of a CDS record is journaled, as well as the updated copy.

NO

Specifies that only the updated copy of the CDS record is to be journaled.

YES

Specifies that the pre-update copy of the CDS record being updated is to be journaled, in addition to the updated copy of the CDS record. Set this option only on a test or recovery system when you plan to use the EDGUPDT utility to duplicate CDS record updates back in the production CDS. As a result of using this option you should plan on providing up to 33% more journal data set space to accommodate the additional records.

Default: JRNLTRAN(NO).

LINECOUNT(nnn)

Specifies the default number of lines per page for reports, including heading and trailer lines. Specify a value between 10 and 999.

You can override this value when producing individual reports by specifying a parameter to the report program or report utility as described in [z/OS DFSMSrmm Reporting](#).

Default: LINECOUNT(54)

LOCALTASKS(number)

Use this operand to set the number of tasks available on each system for processing locally initiated requests. You can optionally specify a value for local tasks on each and every instance of the DFSMSrmm subsystem; client system, server system, or standard system. On a client system, LOCALTASKS is also the maximum number of tasks that can make a socket connection to the server. Specify a value from 1 to 999.

Recommendation: Specify or accept the default value of 10 local tasks on all systems. Most of these tasks are rarely used by DFSMSrmm.

The number of local and server tasks you can use and still successfully start DFSMSrmm is limited by the size of the private region above and below 16MB. To start with more tasks, you will require a larger REGION size.

Default: LOCALTASKS(10)

MASTEROVERWRITE(ADD | LAST | MATCH | USER)

Specify to control how DFSMSrmm allows the overwriting of a volume. You can use one of these values:

ADD

Specify this value so new data can be created and no existing data can be destroyed. No existing file on a volume can be re-created, but the last file can have new data added to it. When adding data to the last file, DFSMSrmm checks that the data set name used must match the existing data set name. Select this option when you want the last file on the volume to be extended or a new file added to the volume.

Note: DFSMSrmm enforces the MASTEROVERWRITE(ADD) option on a WORM tape that is in master status. This is done to ensure that you see a message from DFSMSrmm rather than one of a number of symptoms as a result of the tape drive preventing overwrites.

LAST

Specify this value to ensure that when an existing file on a master volume is being written to that only the last file on the volume can be used. The data set name used must match the existing data set name. Select this option when you want the last file on the volume to be used for output.

MATCH

Specify this value to ensure that when an existing file on a master volume is being used for output that exactly the same data set name must be used. Select this option when you want any existing file on the volume to be re-created regardless of whether it is the last file on the volume as long as the same data set name is used.

When you use an existing tape file for output all the files that are higher in sequence are destroyed.

USER

Specify this value to allow any existing file on a master volume to be used for output regardless of the data set names being used and its relative file position on the volume. Select this option when you want validation of master volumes to be just the same as for user status volumes.

When you use an existing tape file for output all the files that are higher in sequence are destroyed.

Default: MASTEROVERWRITE(LAST)

MAXHOLD(nnn)

Specifies the maximum number of activities DFSMSrmm performs before the reserve is released and reacquired. For example, if you specify MAXHOLD(100) and you issue the RMM ADDRACK command with COUNT(1000), DFSMSrmm adds 100 racks before the reserve is released and reacquired. Specify MAXHOLD to minimize the impact that long operations, such as RMM TSO ADD subcommands with large COUNT values, or large searches, have to other users.

Even if the DASD where the control data set resides is not shared, consider specifying MAXHOLD because it controls how often users on the same system can gain access to the data set when a long task is running. MAXHOLD also influences the amount of virtual storage that DFSMSrmm uses. Increasing the value might improve the performance of individual DFSMSrmm functions, but other users are locked out from the control data set for a longer time. In a shared environment, the device is also reserved for a longer time, possibly impacting users of other data on the volume.

You do not normally need to alter this value. If your system is storage constrained, you might lower the value from 100 to reduce the amount of virtual storage that DFSMSrmm needs, and therefore its demand on real storage.

In a shared DASD environment in which you are using global resource serialization to convert the DFSMSrmm control data set reserve, you could increase the value to reduce the frequency of the release/reserve, which reduces the traffic on the global resource serialization ring. However, this action causes more virtual storage to be used and increases the time between releases of the control data set. The result is that other users wait longer before gaining access to the control data set.

Specify a value between 10 and 500. Under normal conditions, use the default value.

Default: MAXHOLD(100)

MAXRETPD(NOLIMIT | nnnnn)

Specifies the maximum retention period that a user can request for data sets on volumes. Specify NOLIMIT or a value between 0 and 93000 days. When a value between 0 and 93000 days is specified, the value is added to the current date to determine the maximum allowed expiration date. Specify NOLIMIT to use the dates 99365 or 99366 which mean to never expire. If the calculated date is 31 December 1999, the expiration date 1 January 2000 is used.

MAXRETPD is always used to determine the volume expiration date. The volume expiration date can be ignored when EXPIRYDATEIGNORE is specified on all vital record specifications. MAXRETPD is important if the vital record specification specifies UNTILEXPIRED and the decision is based on the volume expiration date, or if the volume is managed by the EXPDT retention method. For the VRSEL retention method, the preferred method is to put the retention requirements all into the vital record specifications and make MAXRETPD=RETPD. If users are allowed to specify expiration and retention period to override vital record specifications, select a MAXRETPD that covers the maximum retention period they would like to enforce. If this forces 99365 to be reduced, define vital record specifications for any data that should be permanently retained, like DFSMSHsm tape data.

Use MAXRETPD to set limits on the values that can be specified for EXPDT and RETPD. If the retention period or expiration date specified in the JCL or management class exceeds the MAXRETPD value, DFSMSrmm overrides it and uses the value in MAXRETPD to determine the expiration date. For data sets on volumes that use the EXPDT retention method, if the expiration date determined with the LASTREF data set attribute exceeds the MAXRETPD value, DFSMSrmm overrides it and uses the value in MAXRETPD to determine the expiration date. In either case, the job passes without warning, and no message comes up on a log to indicate the override. The volume label has the JCL-specified value because DFSMSrmm does not change tape labels or system control blocks. The control data set contains the JCL-specified value as well as the expiration date calculated from MAXRETPD. You can display the JCL-specified value for information only.

If the DFSMSrmm ISPF dialog or RMM TSO subcommands are used to specify a retention period or expiration date that exceeds the MAXRETPD value, DFSMSrmm fails the subcommand or panel request.

For more information about how to automatically handle expiration date-protected tapes, see [“Defining pools: VLPOOL” on page 209](#) for information on EXPDTCHECK.

Default: MAXRETPD(NOLIMIT)

MCATTR(NONE|ALL|VRSELXDI)

Specifies whether DFSMSrmm should be enabled to use of DFSMS management class (MC) expiration attributes that apply to tape management. The management class expiration attributes processed by DFSMSrmm are:

- *Expire after days Non-usage*, which is equivalent to LASTREF(*extra days*)
- *Expire after Date/Days*, which is equivalent to expiration date / retention period
- *Retention Method*, which sets the retention method on the volume level
- *Volume Set Management Level*, which relates to how RMM manages multivolume sets
- *Retain While Catalogued*, which is equivalent to the WHILECATALOG attribute
- *Exclude from VRSEL*, which flags volumes to be excluded from RMM VRSEL processing

ALL

The use of all applicable management class attributes is enabled. The MC attributes are exploited as they are appropriate, depending on the retention method. The MC attribute Retention Limit is not applied.

NONE

No management class attributes are used.

VRSELXDI

The use of management class attributes is enabled only for volumes managed by the EXPDT retention method. The MC attributes are exploited as they are appropriate, depending on the retention method, with the exception of the MC attribute *Expire after Date/Days*, which is ignored if the data set is on a volume managed by the VRSEL retention method. The MC attribute Retention Limit is not applied. VRSELXDI is recommended if it is desired that the processing of VRSEL managed volumes is the same as in prior DFSMSrmm releases.

Default: MCATTR(NONE)

MEDIANAME(3480 | *medianame*)

Specify to set a default medianame value that DFSMSrmm uses when you do not specify a media name for a volume. The media name is used when you add volumes, define pools in your installation, define a default pool for your installation, and when the EDGINERS utility selects volumes for automatic processing.

Specify a one to eight character name. Here are examples of MEDIANAME that you might define: CART, ROUND, SQUARE, 3420, 3480, TAPE, OPTICAL, and CASSETTE. You can use any name for a media name because DFSMSrmm does not check that the media name you define is a device type that has been defined to z/OS. Use MEDIANAME to identify different types of physical shelf space for different media or to distinguish different media characteristics such as Cartridge Tape and Enhanced Capacity Cartridge System Tape.

Prior to setting or changing the default media name, check for any VLPOOL commands that are using the media name that you plan to change. If you change the default media name that is used for a VLPOOL command for an existing pool of volumes, you must consider changing the media names for those existing volumes. Refer to “[Changing pool definitions](#)” on page 79 which describes how to use RMM CHANGEVOLUME *volser* MEDIANAME subcommand to change the volume media name to match the value in the VLPOOL command. Also check your jobs that run EDGINERS to ensure that MEDIANAME, if coded in the execution parameters, is still consistent with the values you are using.

Default: MEDIANAME(3480)

MEMBER(*parmlib_suffix*)

Use the MEMBER operand in the primary DFSMSrmm parmlib to identify a second parmlib member that contains overriding or additional parmlib options. Some information in the EDGRMMxx parmlib might need to be specific to a subset of your systems. For example, the PRTITION, OPENRULE, or VLPOOL entries might need to be different.

Parmlib_suffix must be any 2 characters used as the suffix of the EDGRMMxx parmlib member name. You can use system symbols (for example, &SYSC clone) to enable easier sharing of the EDGRMMxx parmlib member. The system symbol must resolve to 2 characters used as the suffix of the EDGRMMxx parmlib member name. For example:

OPTION	DSNAME (DFRMM.PROD.MASTER)
	JRNLIB (DFRMM.PROD.JOURNAL)
	MEMBER (S5)

DFSMSrmm ignores the MEMBER operand specified in the second parmlib member.

Default: None.

MOVEBY(SET | VOLUME)

Specify the MOVEBY operand to move volumes that are retained by DFSMSrmm vital record specifications as a set of volumes or as individual volumes.

SET

Specify this value when you want volumes moved as a set. DFSMSrmm moves all volumes in the same multivolume set that are retained by vital record specifications. All the volumes are retained in the same location selected by the vital record specification priority or location priority for the volume.

VOLUME

Specify this value when you want volumes moved as individual volumes. DFSMSrmm moves the volumes without considering the location of the other volumes in the multivolume set.

Default: MOVEBY(VOLUME)

MSG(MIXED | UPPER)

Specify to control the case for message text. You can use:

MIXED

Specify MIXED when you want messages to be displayed exactly as they appear in EDGMTAB. This includes mixed case message text.

UPPER

Specify UPPER to ensure that all messages are displayed in upper case only. Any mixed case messages in EDGMTAB are translated to upper case before they are displayed.

This is the value used when DFSMSrmm is inactive and utilities need to issue messages.

Default: MSG(MIXED)

NOTIFY(NO | YES)

Specifies whether DFSMSrmm should automatically notify volume owners when the volumes they own become eligible for release or when software product volumes are added. Specify YES for automatically notify owners, or NO for notification. You must run DFSMSrmm under the JES2 or JES3 subsystem to use the DFSMSrmm NOTIFY function.

Specify YES and DFSMSrmm notifies the owner using the TSO TRANSMIT command to send an electronic message. The owner information in the DFSMSrmm control data set must include either a valid user ID and node name, or a valid Internet e-mail address, for the message to be sent, and the volume record must specify Notify Owner as a release action. DFSMSrmm uses the Internet e-mail address, if it exists, rather than the user ID and node name.

When you use e-mail addresses for owner notification, DFSMSrmm is dependent on you having an existing SMTP server address space configured to support the sending of e-mails. Refer to [*z/OS Communications Server: IP Configuration Guide*](#) for additional information. The SMTP server can be on the same system as the DFSMSrmm started task, or on another system known through NJE.

The Owner 'SMTP' is now a reserved owner name value that you can use to configure the Node name and SMTP server address space or machine name. You use the NODE operand to identify the node that runs the SMTP server, and the USER operand to identify the SMTP server. You must specify both values. When you do not have the Owner SMTP defined, DFSMSrmm uses the JES node name of the running system and SMTP as the SMTP server address space name.

If there is a chance that notify messages might not be delivered, you must ensure that for:

- e-mail messages, the SMTP server is configured to handle undelivered mail.
- NJE messages, you have a process to identify messages and clean up the queues on the JES spool.

Specify NO for expired volumes with a release action of notify and the librarian performs the notification. The librarian issues the RMM CHANGEVOLUME subcommand with the CONFIRMRELEASE operand to indicate that the notification has been performed.

Default: NOTIFY(NO).

OPMODE(MANUAL | RECORD | WARNING | PROTECT)

Specifies the running mode of DFSMSrmm. The running mode affects:

- Whether DFSMSrmm records tape activity

- How DFSMSrmm interacts with system-managed tape support
- How DFSMSrmm enforces the tape mount validation rules described in [“How does DFSMSrmm validate tape mounts?”](#) on page 17.

Protect mode is the only running mode that provides complete validation of volumes and rejection of volumes that do not adhere to DFSMSrmm tape mount validation rules. The running mode also affects other actions and defaults used by DFSMSrmm.

- **MANUAL** — Manual mode. When DFSMSrmm is running in manual mode:
 - DFSMSrmm does not record tape volume usage or validate volumes.
 - DFSMSrmm does not participate in system managed tape activity; that is, DFSMSrmm does **not**:
 - Record system managed tape activity
 - Control change of use for system managed volumes
 - Provide information to OAM during entry or eject, or during volume not in library activities
 - Does not apply the PRTITION partition rules
 - You can use RMM TSO subcommands, the DFSMSrmm ISPF dialog, and inventory management functions for all types of media. Updates to the TCDB for system managed tape volumes are controlled by the SMSTAPE(UPDATE(...)) parmlib option and affect these functions:
 - UPDATE (SCRATCH): Volume status updates if you run inventory management processing.
 - UPDATE (COMMAND): Any TSO command change that affects the information in the TCDB, such as EJECT, status change, and media information.
- **RECORD** — Record-only mode. When DFSMSrmm is running in record mode, DFSMSrmm performs the actions described for manual mode and also:
 - DFSMSrmm records information about tape volumes used on the system, including details about volume owners and data set names.
 - DFSMSrmm does not validate or reject volumes during OPEN processing.
 - DFSMSrmm actions for system managed tape activity are limited to only recording the system managed tape activity.
 - You can use RMM TSO subcommands, the DFSMSrmm ISPF dialog, and inventory management functions for all types of media. Updates to the TCDB for system managed tape volumes are controlled by the SMSTAPE(UPDATE(...)) parmlib option and affect these functions.
 - UPDATE (SCRATCH): Volume status updates if you run inventory management processing.
 - UPDATE (COMMAND): Any TSO command change that affects the information in the TCDB, such as EJECT, status change, and media information.
 - UPDATE (EXITS): Providing information to OAM during entry or eject activities. Controlling the disposition of TCDB volume records at EJECT time.
- **WARNING** — Warning mode. When DFSMSrmm is running in warning mode, DFSMSrmm performs the actions described for manual and record-only mode. DFSMSrmm issues warning messages when an action would have failed if DFSMSrmm was running in protect mode.
 - DFSMSrmm validates, but does not reject, volumes during OPEN processing.
 - DFSMSrmm actions for system managed tape activity are limited to only recording the system managed tape activity.
 - You can use RMM TSO subcommands, the DFSMSrmm ISPF dialog, and inventory management functions for all types of media. Updates to the TCDB for system managed tape volumes are controlled by the SMSTAPE(UPDATE(...)) parmlib option and affect these functions:
 - UPDATE (SCRATCH): Volume status updates if you run inventory management processing.
 - UPDATE (COMMAND): Any TSO command change that affects the information in the TCDB, such as EJECT, status change, and media information.

- UPDATE (EXITS) : Providing information to OAM during entry or eject activities. Controlling the disposition of TCDB volume records at EJECT time.
- **PROTECT** — Protect mode. In protect mode, DFSMSrmm is fully operational. In addition to performing the actions described for manual and record-only mode, DFSMSrmm also:
 - Validates all magnetic tape requests and rejects magnetic tape volume mounts under certain conditions, discussed in [“How does DFSMSrmm validate tape mounts?”](#) on page 17.
 - Sets the UNCATALOG operand default to YES. UNCATALOG(YES) specifies that DFSMSrmm should uncatalog data sets under conditions described in the UNCATALOG operand description.
 - Fully participates in system managed tape activity:
 - DFSMSrmm records all system managed tape activity.
 - During OAM exit processing, DFSMSrmm validates all changes to the TCDB and fails those that are not allowed.
 - During cartridge entry processing, DFSMSrmm information overrides information provided by OAM.
 - DFSMSrmm always updates the TCDB when DFSMSrmm information is updated, either by command or by DFSMSrmm processing.
 - During eject processing DFSMSrmm can optionally control the disposition of the TCDB tape volume record. Use SMSTAPE(PURGE) to control the purging or keeping of records in the TCDB during eject processing.

Note: DFSMSrmm processes some parmlib options you have set even when DFSMSrmm does not validate or reject volumes. For example, if you set the TPRACF or UNCATALOG operands described in this topic, DFSMSrmm honors these operands when running in any mode. For example if you specify UNCATALOG(YES) and OPMODE(RECORD), DFSMSrmm uncatalogs data sets even though DFSMSrmm does not validate or reject volumes. When you specify OPMODE(WARNING) or OPMODE(PROTECT), DFSMSrmm also honors the setting of the VLPOOL EXPDTCHECK options. To obtain the results you desire, you should review the values you select for these additional options.

[Table 16 on page 181](#) and [Table 17 on page 181](#) provide information about the options affected by the OPMODE value.

Table 16. How OPMODE honors the settings of various options

Option	Manual	Record Only	Warning	Protect
EXPDTCHECK	N	N	Y	Y
UNCATALOG	Y	Y	Y	Y
TPRACF	Y	Y	Y	Y
SMSTAPE(UPDATE)	Y	Y	Y	N
SMSTAPE(PURGE)	N	Y	Y	Y

Table 17. How OPMODE value affects system-managed tape library support

Main Area of Activity	Manual	Record Only	Warning	Protect
Command Processing	OPT	OPT	OPT	YES
Expiration Processing	OPT	OPT	OPT	YES
Support for CBRUXxxx Exits	N/A	OPT	OPT	YES
Purging TCDB Records During Eject Processing	N/A	OPT	OPT	YES

Table 17. How OPMODE value affects system-managed tape library support (continued)

Main Area of Activity	Manual	Record Only	Warning	Protect
Legend:				
N/A Function not available				
OPT Function can be controlled by SMSTAPE option				
YES Function is always provided				
Default: OPMODE(PROTECT).				
PDA(ON NONE OFF) Specify to enable or disable the PDA trace facility.				
NONE Specify to prevent DFSMSrmm from enabling the PDA facility. If NONE is specified at startup, DFSMSrmm does not obtain storage for the trace buffer. If NONE is specified when DFSMSrmm is refreshed, it is equivalent to specifying PDA(OFF) and PDALOG(OFF).				
OFF Specify to disable the trace facility after DFSMSrmm initialization. When the trace facility is disabled, trace data is not accumulated.				
ON Specify to enable the trace facility.				
Default: PDA(ON)				
PDABLKCT(nnn) Specifies the number of blocks or buffers that make up the in-storage trace wrap table. Each block or buffer is the size specified by the PDABLSZ operand. <i>nnn</i> is a minimum of 3 and a maximum of 255.				
Default: PDABLKCT(255)				
PDABLSZ(blksz) Specifies the DASD blocksize for the DFSMSrmm PDA trace facility data sets, EDGPDOX and EDGPDOY. <i>nn</i> is the number of kilobytes in each DASD block and is a minimum of 1 and a maximum of 31.				
Default: PDABLSZ(27) for 3390 DASD devices, PDABLSZ(22) for 3380 DASD devices, and PDABLSZ(12) for all other DASD devices				
PDALOG(ON OFF) Specify to enable or disable output to the PDA trace data sets.				
OFF Specify to disable output to the PDA trace data sets.				
ON Specify to enable output to the PDA trace data sets.				
Default: PDALOG(ON)				
PREACS(NO YES) Specify this operand to control whether DFSMSrmm-supplied and EDG_EXIT100 installation exit-supplied values are input to SMS Pre-ACS processing.				
NO Specify NO to avoid DFSMSrmm Pre-ACS processing using the EDG_EXIT100 installation exit.				
YES Specify YES to enable DFSMSrmm Pre-ACS processing using the EDG_EXIT100 installation exit.				
Default: PREACS(NO)				

RETAINBY(SET|VOLUME)

Use the RETAINBY option for volumes managed by the VRSEL retention method to specify whether DFSMSrmm retains multivolume sets as a set or as individual volumes.

Note:

1. For volumes managed by the EXPDT retention method, use the RETAINBY suboption of the RETENTIONMETHOD(EXPDT) option to obtain a similar function.
2. SET should be used for most multi-volume files unless some files on the same volume chain will be expired before others. VOLUME should be used for DFSMSHsm migration and backups as well as any duplexed volumes along with EDM(YES) option which protects DFSMSHsm volumes, preventing tapes from being accidentally released by users they do not belong to. If you use choose to use RETAINBY(SET), you can use DEFTABLE to specify DFSMSHsm primary and duplex migration and backup data set names or masks to RETAINBY(VOLUME).

SET

When you retain by set, if any volume in a set is retained by a vital record specification, all volumes in the set are retained as vital records. DFSMSrmm uses highest retention date of all volumes in the set as the retention date for all volumes retained as vital records in a set. If no volume in a set is retained by a vital record specification, DFSMSrmm performs expiration processing by set. DFSMSrmm does not expire volumes in a set if at least one volume in a set is still not ready to expire because it has not reached its expiration date and you have not specified that you want the expiration date ignored.

VOLUME

When you retain by volume, DFSMSrmm retains a volume based on vital record specifications and on the volume expiration date. DFSMSrmm does not consider other volumes in the set.

DFSMSrmm sets an indicator 'retained by set' in the volume information when a volume is vital record specification retained or not expired only because it is a member of a set.

The location where volumes are retained is determined by the MOVEBY option.

Default: RETAINBY(VOLUME).

RETENTIONMETHOD(EXPDT | VRSEL)

Use this operand to set the system-wide retention method default for new tape volume sets. New tape volume sets may be created during Open/Close/End-of-Volume (O/C/EOV) processing, or through DFSMSrmm commands. A tape volume set may be a multivolume set, or a single tape volume. RETENTIONMETHOD can be abbreviated as RM.

EXPDT

Specify EXPDT to set the default retention method for new tape volume sets to be based on EXPDT. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management. When you specify the EXPDT retention method the DFSMSrmm inventory management EXPROC processing always attempts to return volumes to scratch on the same run as the volume is released (this is as if the SCRATCHIMMEDIATE attribute is set for the volume). DFSMSrmm maintains a consistent expiration date and time for all data set records of a multivolume data set, unless the volume set is retained by first file.

EXPDT can be specified either as EXPDT or EXPDT(*options*). The available options are:

CATLGDAYS(*ret_days*)

Specifies that when a data set specified with WHILECATALOG(ON) or WHILECATALOG(UNTILEXPIRED) becomes uncataloged, it will be kept for at least *ret_days* more days.

ret_days can be 0 to 93000 days. For example, CATLGDAYS(24) keeps data sets for 24 days. Set *ret_days* to 0 to have data sets expire the moment they become uncataloged.

Default: CATLGDAYS(2).

GDG(options)

Specifies the default options for new GDG data sets on volumes managed by the EXPDT retention method. The options that can be specified are:

RETPD(*ret_days*)

Specifies the default retention period in days for GDG data sets on volumes managed by the EXPDT retention method.

- *ret_days* can be a number in the range 0 to 93000 or the keyword PERMANENT.
- RETPD(0) specifies that the default expiration date is the same as the creation date. GDG data sets with both RETPD(0) and WHILECATALOG(ON) specified are managed only by their catalog status.
- RETPD(PERMANENT) specifies that the default expiration date for new GDG data sets is to be 1999/365, which means that the data set will be retained permanently. This is useful in conjunction with the WHILECATALOG(UNTILEXPIRED) operand to create data sets that are only kept by their catalog status unless RETPD or EXPDT is specified in the JCL or the data class.
- If RETPD is not specified on the GDG operand, then the default retention period specified in the global RETPD parmlib option (which is used for both retention methods) is used.

WHILECATALOG(OFF|ON|UNTILEXPIRED)

Specifies the default WHILECATALOG value for GDG data sets on volumes managed by the EXPDT retention method.

WHILECATALOG(OFF)

Specifies that new GDG data sets residing on volumes managed by the EXPDT retention method will by default have the WHILECATALOG attribute set to OFF. As a result, retention of these data sets depends only on their expiration date and not on their catalog status.

WHILECATALOG(ON)

Specifies that new GDG data sets residing on volumes managed by the EXPDT retention method by default have the WHILECATALOG attribute set to ON. As a result, these data sets are not be expired as long as they are cataloged, nor are they expired prior to their expiration date. The expiration date of the new data sets is set to the greater of the EXPDT value or (*creation date* + CATRETPD *hours*). The CATRETPD option is used to ensure that new data sets are not prematurely expired before the catalog signal is received, when EDGHSKP housekeeping functions are running at the same time the tape is being created.

Tip: WHILECATALOG(ON) is a useful default option for GDG data sets, because old generations of GDG data sets can be uncataloged automatically.

WHILECATALOG(UNTILEXPIRED)

Specifies that new GDG data sets residing on volumes managed by the EXPDT retention method will by default have the WHILECATALOG attribute set to UNTILEXPIRED. As a result, these data sets will expire after they are uncataloged or if the expiration date is reached. When DFSMSrmm determines that a data set is no longer cataloged, the DFSMSrmm resets the expiration date of the data set to a date equal to the date the data set was uncataloged plus the number of days specified by the CATLGDAY option. WHILECATALOG(UNTILEXPIRED) has no effect on data sets that are never cataloged.

Default: WHILECATALOG(OFF)

LASTREF(*extra_days*)

LASTREF specifies the default for the data set record LASTREF attribute. The LASTREF attribute specifies the number of days that the data set will be retained after the data set was last referenced by a read or write operation. LASTREF applies only to data sets on volumes managed by the EXPDT retention method.

extra_days is a decimal number between 0 and 93000. The value must not exceed the maximum retention period (MAXRETPD) specified in the EDGRMMxx parmlib member. An *extra_days* value of 0 has the same effect as the NOLASTREF operand.

When a data set is added to DFSMSrmm on a volume managed by the EXPDT retention method and neither LASTREF nor NOLASTREF are specified for the data set, then DFSMSrmm uses the default LASTREF value.

DFSMSrmm uses the data set LASTREF value to determine the data set expiration date. The extra days are added to the date of last reference. The data set expiration date is set to the maximum of the date calculated using data set LASTREF value and the date resulting from applying the EXPDT, RETPD, or default RETPD. Any reference to the data set by a read or write operation will change the expiration date.

Default: If neither LASTREF nor NOLASTREF are specified in parmlib, NOLASTREF is used by default.

NOGDG(options)

Specifies the default options for non-GDG data sets on volumes managed by the EXPDT retention method. The options that can be specified are:

RETPD(ret_days)

Specifies the default retention period in days for non-GDG data sets on volumes managed by the EXPDT retention method.

- *ret_days* can be a number in the range 0 to 93000 or the keyword PERMANENT.
- RETPD(0) specifies that the default expiration date is the same as the creation date. Non-GDG data sets with both RETPD(0) and WHILECATALOG(ON) specified are managed only by their catalog status.
- RETPD(PERMANENT) specifies that the default expiration date for new non-GDG data sets is to be 1999/365, which means that the data set will be retained permanently. This is useful in conjunction with the WHILECATALOG(UNTILEXPIRED) operand to create data sets that are only kept by their catalog status unless RETPD or EXPDT is specified in the JCL or the data class.
- If RETPD is not specified on the NOGDG operand, then the default retention period specified in the global RETPD parmlib option (which is used for both retention methods) is used.

WHILECATALOG(options)

Specifies the default WHILECATALOG value for non-GDG data sets on volumes managed by the EXPDT retention method.

WHILECATALOG(OFF)

Specifies that new non-GDG data sets residing on volumes managed by the EXPDT retention method will by default have the WHILECATALOG attribute set to OFF. As a result, retention of these data sets depends only on their expiration date and not on their catalog status.

WHILECATALOG(ON)

Specifies that new non-GDG data sets residing on volumes managed by the EXPDT retention method will by default have the WHILECATALOG attribute set to ON. As a result, these data sets will not be expired as long as they are cataloged, nor will they be expired prior to their expiration date. The expiration date of the new data sets will be set to the greater of the EXPDT value or (*creation date* + CATRETPD hours). The CATRETPD option is used to ensure that new data sets are not prematurely expired before the catalog signal is received, when EDGHSKP housekeeping functions are running at the same time the tape is being created.

WHILECATALOG(UNTILEXPIRED)

Specifies that new non-GDG data sets residing on volumes managed by the EXPDT retention method will by default have the WHILECATALOG attribute set to UNTILEXPIRED. As a result, these data sets will expire after they are uncataloged or

when the expiration date is reached. When DFSMSrmm determines that a data set is no longer cataloged, the DFSMSrmm resets the expiration date of the data set to a date equal to the date the data set was uncataloged plus the number of days specified by the CATLGDAYS option. WHILECATALOG(UNTILEXPIRED) has no effect on data sets that are never cataloged.

Tip: WHILECATALOG(UNTILEXPIRED) could be a helpful default option for non-GDG data sets, because this default would ensure that a cataloged dataset that is never going to be automatically uncataloged would still expire on the expiration date specified in the JCL.

Default: WHILECATALOG(OFF)

NOLASTREF

NOLASTREF is the default setting for the data set record LASTREF attribute. NOLASTREF specifies that DFSMSrmm does not consider the data set last reference date when determining the data set expiration date. NOLASTREF applies only to data sets on volumes managed by the EXPDT retention method.

RETAINBY(FIRSTFILE | SET | VOLUME)

RETAINBY specifies how DFSMSrmm is to retain volumes or multivolume sets that are managed by the EXPDT retention method:

FIRSTFILE

The expiration date of the first file is used to set the expiration date of the volume or multivolume set. All volumes in a set will have the exact same expiration date and will be released to scratch in the same run of DFSMSrmm inventory management.

SET

The expiration date is the maximum of all data set expiration dates of all the volumes in the set. All volumes in the set will have the exact same expiration date. Any file on any volume of the set can increment the volume expiration date. All files of a multivolume data set have the same expiration date.

VOLUME

The expiration date is determined separately for each volume in the set. Unless defined differently, the expiration date is the maximum of all data set expiration dates on the volume. Each file on a volume can increment the volume expiration date. All files of a multivolume data set have the same expiration date.

Default: RETAINBY(VOLUME).

Note:

1. In a multivolume set, RETAINBY is assigned only to the first volume in a multivolume sequence. All other volumes added to the set assume the same RETAINBY.
2. For volumes managed by the VRSEL retention method, use the RETAINBY option to obtain a similar function.
3. FIRSTFILE is not recommended in most cases. SET should be used for most multi-volume files unless some files on the same volume chain will be expired before others. VOLUME should be used for DFSMSHsm migration and backups as well as any duplexed volumes along with EDM(YES) option which protects DFSMSHsm volumes, preventing tapes from being accidentally released by users they do not belong to. If you use choose to use RETAINBY(SET), you can use DEFTABLE to specify DFSMSHsm primary and duplex migration and backup names or masks to RETAINBY(VOLUME).

VRSEL

Specify VRSEL to set the default retention method for new tape volume sets to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volume is to be retained by VRS. The VRSEL retention method is controlled by all the other VRS related options in parmlib including OPTION RETAINBY MOVEBY.

Default: RETENTIONMETHOD(VRSEL)

RETPD(nnnnn / PERMANENT)

Specifies the default retention period for all new data sets on volumes. Specify a value between 0 and 93000 days. The specified value is added to the current date to determine the expiration date. Select a default retention for parmlib RETPD that is a small value to ensure that all tape data created outside the service levels is released as soon as possible. The MAXRETPD value you specify in the parmlib limits the calculated expiration date.

RETPD(PERMANENT) specifies that the default expiration date for new data sets is to be 1999/365, so the data set will be retained permanently.

Note: The GDG and NOGDG options can also be used to specify separate RETPD defaults for new GDG and non-GDG data sets managed by the EXPDT retention method.

DFSMSrmm sets a default retention period as follows:

- If you specify RETPD or EXPDT, the value is used as the data set's new expiration date.
- If you do not specify RETPD or EXPDT, then:
 1. DFSMSrmm uses the EXPDT or RETPD allocation attribute of a data class, if all these are true:
 - The data set is associated with a data class, either explicitly by the DATACLAS keyword on the JCL or implicitly by an automatic class selection routine
 - The data class has an EXPDT or RETPD allocation attribute
 - The Storage Management Subsystem is active.
 2. DFSMSrmm uses the management class Expire after Date/Days as the data set's new expiration date, if all these are true:
 - You do not specify RETPD or EXPDT in the JCL or data class
 - The data set is assigned to a management class
 - The management class Expire after Date/Days is set
 - The EDGRMMxx parmlib option MCATTR allows the exploitation of the management class for this type of volume
 - The Storage Management Subsystem is active.
 3. DFSMSrmm uses the default retention period set in EDGRMMxx, if RETPD or EXPDT are not otherwise specified.

Whenever a new data set is written to tape, DFSMSrmm checks whether the volume's expiration date should be updated, based on whether the new data set has a longer expiration date than the volume on which it is written. DFSMSrmm gets the expiration date for a data set from the job file control block (JFCB) or from the management class at open time. If there is a date in the JFCB or management class, DFSMSrmm compares this date to the current expiration date for the volume. If the date in the JFCB allows the volume to be retained longer, DFSMSrmm uses that date to update the volume's expiration date.

If there is no expiration date in the JFCB or management class, DFSMSrmm uses the EDGRMMxx RETPD value to calculate the new expiration date. If the RETPD value allows the volume to be retained longer, DFSMSrmm uses that date to update the volume's expiration date.

You can set the date in the JFCB in several ways, including:

- RETPD and EXPDT keywords in the JCL
- Data class when the Storage Management Subsystem is active and the volume is system-managed
- A user program, using the RDJFCB macro and the OPEN TYPE=J after modifying the JFCB
- Installation exits in use on your particular system

If the use of the management class attributes is enabled by the EDGRMMxx parmlib OPTION MCATTR, you can use the management class Expiration Attributes ('Expire after Date/Days' and 'Expire after Days Non-usage') to set the data set EXPDT and LASTREF attributes, where appropriate, to define more specific default retention periods.

When you use the VRSEL retention method, you can also use vital record specifications to define more specific default retention periods for users by using a data set name prefix. For example, specify:

```
RMM ADDVRS DSNAME('RICK.**') DAYS COUNT(30)
```

to keep all data sets for ID RICK for 30 days. For more information about using vital record specifications, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

Default: RETPD(5)

REUSEBIN(CONFIRMMOVE | STARTMOVE)

Use the REUSEBIN operand to control how DFSMSrmm reuses bins when a volume is moving from a bin.

CONFIRMMOVE

When a volume moves out of a bin, DFSMSrmm does not reuse this bin until the volume move has been confirmed.

STARTMOVE

A bin can be reused as soon as a volume starts moving out of a bin. Extended bin support must be enabled before you can use this operand. See [“Enabling extended bin support” on page 417](#) to enable extended bin support.

Default: REUSEBIN(CONFIRMMOVE).

SCRATCHPROC(proc_name)

Specifies the name of the procedure DFSMSrmm starts to replenish scratch volumes in an automated tape library. Specify a procedure name one to eight characters long.

You must run DFSMSrmm with a scratch procedure. You can modify or replace the DFSMSrmm-supplied sample, EDGXPROC, to support your location procedures. You can use the scratch procedure to take any action you would like. For example, you can code the procedure to trigger the required inventory management expiration processing job, to run inventory management, or to take no action.

When an automated tape library detects a low-on-scratch condition, OAM issues write-to-operator messages CBR3660A, CBR3792E, and CBR3794A. DFSMSrmm intercepts these messages and starts the SCRATCHPROC procedure. This procedure runs DFSMSrmm expiration processing to replenish the automated tape library's scratch volumes.

Default: SCRATCHPROC(EDGXPROC)

See [“Replenishing scratch volumes in a system-managed library” on page 458](#) for information about EDGXPROC the DFSMSrmm default procedure.

SERVER(PORT(PortNumber) SERVERTASKS(number))

Specifies the type of system you want to set up. SERVER is mutually exclusive with CLIENT. Neither SERVER nor CLIENT must be specified when DFSMSrmm is used as a standard system.

PORT(PortNumber)

Use this operand to specify the port number to be used for IP communication. The PORT operand is required. Specify a value from 1024 to 65535. Port numbers 1 to 1023 are reserved. The port number must be the same for the client system and the server system to establish a network connection.

Default: None.

SERVERTASKS(number)

Use this operand to specify how many DFSMSrmm tasks should be available on the server to handle socket connections from client systems. DFSMSrmm uses this number to determine how many tasks are to be started for processing all client requests on this server. Specify a value from 1 to 999.

Recommendation: Specify or accept the default value of 10 server tasks on each server system. Depending on the number of CLIENT systems, this value should be increased to allow 3 tasks per

CLIENT. Most of the local tasks are rarely used by DFSMSrmm. You do not need more server tasks than the sum of local tasks across your CLIENT systems.

The number of local and server tasks you can use and still successfully start DFSMSrmm is limited by the size of the private region above and below 16MB. To start with more tasks, you will require a larger REGION size.

Default: 10.

SMFAUD(YES | *nnn*)

Specifies the SMF record type to be used for audit records. Specify YES or a number between 128 and 255 that is different from the value for SMFSEC. The value must conform to standard SMF conventions.

IBM recommends that you do not use an SMF record type number *nnn*, but instead use the IBM assigned record number by specifying YES. The IBM assigned record number is type 42, and the subtype is 22. You cannot mix SMF record types. For example, you cannot specify YES for SMFAUD and a record type for SMFSEC.

If you do not specify either YES or a number, DFSMSrmm does not produce audit records.

Default: No audit records

DFSMSrmm ignores SMFAUD on the client system.

SMFSEC(YES | *nnn*)

Specifies the SMF record type to be used for security records. Specify YES or a number between 128 and 255 that is different from the value for SMFAUD. The value must conform to standard SMF conventions.

IBM recommends that you do not use an SMF record type number *nnn*, but instead use the IBM assigned record number by specifying YES. The IBM assigned record number is type 42, and the subtype is 23. You cannot mix SMF record types. For example, you cannot specify YES for SMFAUD and a record type for SMFSEC.

If you do not specify either YES or a number, DFSMSrmm does not produce security records.

Default: No security records

SMSACS(NO | YES)

Specify this operand to control whether DFSMSrmm calls SMS ACS processing to enable use of storage group and management class values with DFSMSrmm for non-system managed data.

NO

Specify NO to prevent DFSMSrmm from calling the SMS ACS processing to obtain management class and storage group names. DFSMSrmm system-based scratch pooling, and scratch pooling and VRS management values based on the EDG_EXIT100 installation exit are used.

YES

Specify YES to enable DFSMSrmm calls to the SMS ACS processing to obtain management class and storage group names. If values are returned by the SMS ACS routines the values are used instead of the DFSMSrmm and EDG_EXIT100 decisions.

Default: SMSACS(NO)

SMSTAPE(UPDATE PURGE)

Use SMSTAPE to specify how DFSMSrmm updates the TCDB and controls system-managed tape processing.

UPDATE

Use UPDATE to select the system-managed tape functions DFSMSrmm provides. The UPDATE operand has 3 subparameters: EXITS, SCRATCH, and COMMAND. You can specify one or more of the subparameters. When DFSMSrmm is running in PROTECT mode, DFSMSrmm ignores the UPDATE operand and performs processing as if you specified EXITS, SCRATCH, and COMMAND. When DFSMSrmm is running in WARNING or RECORD mode, DFSMSrmm does not update TCDB

information unless you request the update. You can specify one or more of the values. When you specify a value, DFSMSrmm performs the updates to the TCDB.

EXITS

Specify EXITS when you want DFSMSrmm volume status information to override the OAM volume status during entry processing, and you want to use the DFSMSrmm VNL exit.

SCRATCH

Specify SCRATCH when you want DFSMSrmm to update the volume status in the TCDB during expiration processing. This can be:

- When volumes are returned to scratch status.
- When scratch volumes are changed to master status during expiration volume replacement processing.

COMMAND

Specify COMMAND when you want to use the RMM TSO subcommands or the DFSMSrmm API to update the TCDB. This controls change of status, TDSI and owner information, eject processing, and manual cartridge entry processing.

Default: None, if you are running in MANUAL, RECORD, or WARNING mode, but DFSMSrmm ignores the update options and forces them to UPDATE(EXITS,SCRATCH,COMMAND) when DFSMSrmm is running in PROTECT mode. See the description of the [OPTMODE parameter](#) in this topic for more information on how OPMODE relates to and affects System-Managed Tape Library Support.

PURGE

Use PURGE to control how DFSMSrmm affects the TCDB volume records during EJECT processing. The default is PURGE(ASIS) in all operating modes except MANUAL mode. In manual mode, DFSMSrmm provides no support for eject processing.

ASIS

Specify ASIS when you do not want DFSMSrmm to determine the TCDB volume record dispositions at eject time. Specifying ASIS allows the eject requestor or Library defaults to control the TCDB volume record disposition

NO

Specify NO to request that DFSMSrmm prevent TCDB records being deleted.

YES

Specify YES when you want DFSMSrmm to force the TCDB volume records to be purged at eject time.

Default: ASIS.

SYSID(system_name)

Specifies the name of the system on which DFSMSrmm is running. Specify a unique system name one-to-eight characters long for each system.

If you are running multiple z/OS systems and sharing the control data set and journal, specify a unique SYSID for each system.

If you have unshared catalogs, you can specify a list of system IDs using the CATSYSID operand. Select the SYSID values from the list of IDs which include current IDs and previously used IDs.

Default: DFSMSrmm uses the system's SMF identification.

TPRACF(NONE | AUTOMATIC | CLEANUP | PREDEFINED)

Specifies the type of RACF tape support that DFSMSrmm provides. Use this operand when you want DFSMSrmm to maintain the security profiles that protect tape volumes. You can define RACF tape support for pools of volumes within your installation by using the VLPOOL RACF command described in [“Defining pools: VLPOOL”](#) on page 209. RACF tape support you define can be overridden by RACF tape support you define with VLPOOL.

The TPRACF(NONE) option is assumed for any volume serial number containing special characters. To protect tape volumes that use special characters in the volume serial number, use RACF generic TAPEVOL profiles that are outside of DFSMSrmm control.

DFSMSrmm honors all other TPRACF options for volume serial numbers that are alphanumeric including national characters. You can also use RACF generic DATASET profiles to protect data created on tape volume. DFSMSrmm honors the TPRACF setting when running in all modes.

If you set TPRACF(PREDEFINED) or TPRACF(AUTOMATIC), DFSMSrmm ensures that all nonscratch tapes are protected by a discrete RACF TAPEVOL profile by checking that a RACF profile exists whenever a data set is written on a tape. If a profile does not exist, DFSMSrmm creates one. Therefore you do not need to use RACF installation exits to set the JCL PROTECT=YES option or specify PROTECT=YES in your JCL. You can use generic data set profiles for all tape data sets without changes to JCL or installation procedures, if you used the VLPOOL command with the RACF(Y) operand, because DFSMSrmm creates a TVTOC when you use the RACF TAPEDSN option.

For both TPRACF(PREDEFINED) and TPRACF(AUTOMATIC), DFSMSrmm ensures TAPEVOL profiles are always deleted during recycling of scratch volumes, if they exist. When you use a volume for output and there is no RACF protection for the volume when you close the data set, DFSMSrmm creates a profile to protect the volume. The owner of the volume is given RACF access to the profile. If TAPEVOL and TAPEDSN are active, DFSMSrmm creates a TVTOC and adds the first file data set entry.

For TPRACF(CLEANUP), DFSMSrmm ensures that TAPEVOL profiles and discrete tape DATASET profiles are deleted during recycling of scratch volumes and existing TAPEVOL profiles are deleted when volumes are deleted from the DFSMSrmm CDS. When you use this option, DFSMSrmm never creates any RACF profiles for you. This processing is only provided for VLPOOLS with RACF(Y).

TPRACF(CLEANUP) is intended to be used when you are changing how tape data sets are protected. For example, you no longer wish to use TAPEVOL profiles and are enabling the use of DATASET profiles, TPRACF(CLEANUP) can be used for this occasion.

When you specify TPRACF(CLEANUP), DFSMSrmm deletes RACF tape profiles for any volumes in your installation based on these VLPOOL values:

- VLPOOL RACF(N), DFSMSrmm does no processing of tape profiles for volumes in the pool at any time.
- VLPOOL RACF(Y), RACF tape profiles are deleted when RMM CHANGEVOLUME, or DELETEVOLUME subcommands are issued. DFSMSrmm deletes TAPEVOL and discrete tape data set profiles during recycling of scratch tapes if the profiles exist.

For more information on RACF processing and security options, see [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,”](#) on page 215.

- When you specify TPRACF(NONE), DFSMSrmm does not create RACF tape profiles for any volumes in your installation.

If you also specify one of these VLPOOL values:

- VLPOOL RACF(N)

DFSMSrmm does not create or delete tape profiles or TVTOCS for volumes in the pool that you are defining with the VLPOOL command.

- VLPOOL RACF(Y)

DFSMSrmm does not create or delete tape profiles or TVTOCS for volumes in the pool that you are defining with the VLPOOL command.

- When you specify TPRACF(PREDEFINED) or TPRACF(P), DFSMSrmm creates predefined RACF tape profiles for any volumes in your installation.

If you also specify one of these VLPOOL values:

- VLPOOL RACF(N)

DFSMSrmm does not or delete create tape profiles or TVTOCS for volumes in the pool that you are defining with the VLPOOL command.

- VLPOOL RACF(Y)

DFSMSrmm creates RACF tape profiles for volumes in the pool that you are defining with the VLPOOL command. The RACF tape profiles are created using RACF options that you have specified and the RACF tape profiles that DFSMSrmm creates when RMM ADDVOLUME, CHANGEVOLUME, or DELETEVOLUME subcommands are issued.

DFSMSrmm deletes TAPEVOL profiles during recycling of scratch tapes if the profiles exist.

DFSMSrmm creates a profile when a volume that is used for output does not have a RACF tape profile defined.

The owner of the volume is given RACF access to the profile.

If TAPEVOL and TAPEDSN are active, DFSMSrmm creates a TVTOC and adds the first file data set entry.

- When you specify TPRACF(AUTOMATIC) or TPRACF(A), DFSMSrmm creates tape profiles for any volume in your installation.

If you also specify one of these VLPOOL values:

- VLPOOL RACF(N)

DFSMSrmm does not create or delete tape profiles or TVTOCS for volumes in the pool that you are defining with the VLPOOL command.

- VLPOOL RACF(Y)

DFSMSrmm creates or delete RACF tape profiles for volumes in the pool. Table 29 on page 235 shows what DFSMSrmm processing takes place when you specify RACF(Y) for a pool.

The processing is the same as TPRACF(PREDEFINED) with these exceptions:

- Scratch tape volumes do not have TAPEVOL profiles created.
- IF TAPEVOL and TAPEDSN are active, RACF automatically creates TAPEVOL and data set profiles for the ADSP and PROTECT=YES users.
- TPRACF(AUTOMATIC) also supports the environment where an installation uses RACF installation exits to create TAPEVOL profiles.

Default: TPRACF(NONE)

TVEXTPURGE(EXPIRE(*days*)|NONE|RELEASE)

Specifies how DFSMSrmm processes volumes to be purged by callers of EDGTVEXT or EDGDFHSM.

- EXPIRE(*days*) — Use the EXPIRE(*days*) option to set the volume expiration date to the current date plus *days* for volumes to be purged. Use the EXPIRE(*days*) parameter when you use the EXPDT retention method, using *days* as a way to delay expiration of the volume. When using the VRSEL retention method, you can optionally use this operand in combination with vital record specifications that use the UNTILEXPIRED retention type. First, the EXPIRE(*days*) is applied to set a new volume EXPDT. Next, by running VRSEL you can then optionally extend retention using the extra days retention type. For example, specify EXPIRE(0) when using a VRS with extra days retention or you can use a non-zero EXPIRE(*days*) value and avoid using an extra days retention VRS.

days is the number of days for which DFSMSrmm retains the volume before considering it for release. The value is a one to four digit decimal number and is added to today's date and time to calculate the new expiration date and time. If the value exceeds the maximum retention period (MAXRETPD) it is reduced to the MAXRETPD value. The default value for *days* is 0. That is, TVEXTPURGE(EXPIRE) is the same as TVEXTPURGE(EXPIRE(0)).

- NONE — DFSMSrmm take no action for volumes to be purged.
- RELEASE — DFSMSrmm releases volume to be purged according to the release actions set for the volume. You must run expiration processing to return a volume to scratch status.

Default: TVEXTPURGE(RELEASE)

UNCATALOG(YES | NO | SCRATCH)

Specifies the type of catalog support to provide:

- **N** — Do not use DFSMSrmm catalog processing. DFSMSrmm does not uncatalog data sets under any circumstances.
- **S** — Only uncatalog data sets when the volume on which they reside is returned to scratch status.
- **Y** — always uncatalog data sets. Use DFSMSrmm catalog processing. DFSMSrmm uncatalogs data sets when:
 - A volume is returned to scratch status, DFSMSrmm uncatalogs all the data sets on the volume.
 - The RMM DELETEVOLUME FORCE subcommand is issued for a volume, DFSMSrmm uncatalogs all the data sets on the volume.
 - The RMM CHANGEVOLUME DSNNAME subcommand is issued for a volume, DFSMSrmm uncatalogs all the data sets on the volume. If the data set name specified on the RMM CHANGEVOLUME subcommand command matches the data set name on the volume, then DFSMSrmm only uncatalogs subsequent data sets.
 - The RMM DELETEDATASET subcommand is issued for a data set, DFSMSrmm uncatalogs the data set. Also, DFSMSrmm uncatalogs all data sets recorded on the same volume with higher data set sequence numbers.
 - A tape data set is overwritten, DFSMSrmm uncatalogs the data set. Also, all data sets recorded on the same volume with higher data set sequence numbers are uncataloged.

If you set UNCATALOG(S) or UNCATALOG(Y), DFSMSrmm uncatalogs data sets even when DFSMSrmm is running in manual mode, record mode, or warning mode.

You should use the UNCATALOG(N) option during early implementation of DFSMSrmm on your system, when DFSMSrmm is running at the same time as your existing management software.

If you use the EDG_EXIT100 exit to request suppression of data set recording for a volume, you should ensure that any data sets that are cataloged, but not recorded by DFSMSrmm are uncataloged by some other mechanism. To leave nonexistent data sets cataloged could lead to later processing problems.

If you use the UNCATALOG(N) option to prevent DFSMSrmm from uncataloging tape data sets, you should ensure that data sets are uncataloged by some other mechanism. To leave nonexistent data sets cataloged could lead to later processing problems.

Ensure Integrated catalog facility catalogs are shared if catalog control is required or if you specify the UNCATALOG(Y) or (S) operand.

Default: The default is UNCATALOG(N) unless DFSMSrmm is running in protect mode. DFSMSrmm uses the UNCATALOG default of Y when DFSMSrmm is running in protect mode.

VRSCCHANGE(INFO | VERIFY)

Use VRSCCHANGE to specify the action that DFSMSrmm should take during inventory management if you have made any changes to vital record specifications using RMM ADDVRS or RMM DELETEVRS subcommands.

If You Specify	Then
INFO	No additional processing or actions are required when vital record specification changes occur.
VERIFY	Any changes made to vital record specifications must be verified by running EDGHSKP vital record processing using the VERIFY parameter. DFSMSrmm must issue return code zero before EDGHSKP vital record processing can be performed to update the control data set.

Default: VRSCCHANGE(VERIFY)

VRSDROP(COUNT(count)|PERCENT(%age), action)

Use VRSDROP to specify a maximum number or percentage of existing VRS-retained volumes that can be dropped from vital records retention and the action to be taken by DFSMSrmm. DFSMSrmm counts the number of VRS-retained physical and logical volumes at the start of vital record selection processing and the number of these dropped by vital record processing. When you specify COUNT, this is an absolute maximum number of volumes that can be dropped by a single run of EDGHSKP VRSEL processing. When you specify PERCENT, this is the maximum percentage of the existing VRS-retained volumes that can be dropped by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.

VRSDROP processing is intended to provide limited checking for volumes that are already VRS-retained. It considers how many of these existing VRS-retained volumes are removed from VRS retention. Those volumes dropped from VRS-retention can be set pending release, depending on VRS release options and the volume expiration date. They can also become EXPDT retained and on the next run of EXPROC, they will be considered by EXPDTRDROP limit processing.

count can be 0 to 2,147,483,647. *%age* can be 0 to 100.

Specify *action* to control the action DFSMSrmm takes during processing and when the value is exceeded. *Action* can be FAIL, INFO, OFF, or WARN.

If You Specify	DFSMSrmm
FAIL	Issues messages EDG2242I and EDG2244I to the MESSAGE file. When the value is exceeded, DFSMSrmm stops VRSEL processing prior to making CDS updates and in addition, message EDG2310I is issued, report extract is run if requested, and any other inventory management processing ends with return code 12.
INFO	Issues messages EDG2242I and EDG2244I to the MESSAGE file and processing continues.
OFF	Processing of this function is turned off.
WARN	Issues messages EDG2242I and EDG2244I to the MESSAGE file. When the value is exceeded, DFSMSrmm additionally issues message EDG2321W, sets a minimum return code of 4 and processing continues.

The default is VRSDROP(PERCENT(10),INFO).

To aid analysis of the results of the VRSDROP limit checking you can use the contents of the ACTIVITY file. The EDGJACTP sample generates a detailed report and a summary report of data sets dropped from VRS retention showing the reasons why. See the contents of the VRS and VRSS files.

VRSEL

VRSEL controls how DFSMSrmm inventory management vital record processing uses retention and movement information that are defined in vital record specifications.

Note: The VRSEL(OLD) option is no longer supported. VRSEL(NEW) is the only valid value for the VRSEL option.

Default: VRSEL(NEW)

VRJOBNAME

Use VRJOBNAME to select how DFSMSrmm uses a job name in a vital record specification to match a data set to a movement and retention policy. DFSMSrmm records the data set name and job name for new tape files you create. You can define the name of the job that created a data set by using the RMM ADDDATASET subcommand or the CHANGEDATASET subcommand for existing data sets. You can specify a job name and data set name in a vital record specification so that data sets that match the data set name and job name are moved and retained by the policy defined in the vital record specification.

If You Specify	DFSMSrmm Uses	And if There Is No Match
VRSJOBNAME(1)	The data set and job name to match a data set to a vital record specification. Job name is the primary value used to match the data set to a vital record specification.	A match by data set name only is acceptable.
VRSJOBNAME(2)	The data set name to match a data set to a vital record specification. If a data set matches multiple vital record specifications with the same data set name, then DFSMSrmm uses a job name to further qualify the data set name.	DFSMSrmm does not apply a policy to the data set.

Default: VRSJOBNAME(2).

VRSMIN(count,action)

Use VRSMIN to specify a minimum number of defined vital record specifications required by inventory management vital record processing, and the action to be taken by DFSMSrmm when you do not have enough vital record specifications defined. Vital record specifications that have reached their deletion date are not counted towards the VRSMIN limit. The VRS deletion date is set by the DELETEDATE parameter of the ADDVRS subcommand, as described in *z/OS DFSMSrmm Managing and Using Removable Media*.

VRSMIN is intended to prevent the unintended scratching of tapes in case of accidental deletion of vital record specifications.

count can be 0 to 2,147,483,647. Specify a count value of 0 to disable this checking.

The default value is 1,FAIL.

Specify *action* to control the action DFSMSrmm takes when the *count* is not reached. *Action* can be FAIL, INFO, OFF, or WARN.

If You Specify	DFSMSrmm
FAIL	Issues message EDG2229I to the MESSAGE file and stops inventory management processing. Processing ends with return code 8.
INFO	Issues message EDG2229I to the MESSAGE file and processing continues.
OFF	Processing of this function is turned off.
WARN	Issues message EDG2229I and EDG2321W to the MESSAGE file and sets a minimum return code of 4. Processing continues.

VRSRETAIN(COUNT(count)|PERCENT(%age), action)

Use VRSRETAIN to specify a minimum number or percentage of newly assigned volumes that are to be retained by vital records retention and the action to be taken by DFSMSrmm. A newly assigned volume is one that has a volume assignment date and time that is higher than the run date and time of the previous VRSEL processing and that is not VRS-retained. DFSMSrmm counts the number of newly assigned physical and logical volumes at the start of vital record selection processing and the number of these that become VRS retained by vital record processing. When you specify count, this is an absolute minimum number of volumes that are to be retained by a single run of EDGHSKP VRSEL processing. When you specify a percentage, this is the minimum percentage of the newly assigned volumes that are to be retained by a single run of EDGHSKP VRSEL processing. This processing occurs each time that you run inventory management VRSEL processing.

VRSRETAIN processing is intended to provide limited checking for volumes containing newly created data sets that have not yet been processed by inventory management VRSEL. The data sets on the volumes have been created since the start of the last completed VRSEL run. It considers how many of these volumes become VRS-retained during the new VRSEL run. Those volumes that become VRS-retained will be considered by the VRSDROP limit checking in future VRSEL runs. Those volumes

that are not retained by VRS can be set pending release, depending on VRS release options and the volume expiration date. They can also become EXPDT retained and on the next run of EXPROC, they will be considered by EXPDTRDOP limit processing.

count can be 0 to 2,147,483,647. *%age* can be 0 to 100.

Specify *action* to control the action DFSMSrmm takes during processing and when the value is exceeded. *Action* can be FAIL, INFO, OFF, or WARN.

If You Specify	DFSMSrmm
FAIL	Issues messages EDG2243I and EDG2245I to the MESSAGE file. When the value is below the required threshold, DFSMSrmm stops VRSEL processing prior to making CDS updates and, in addition, message EDG2310I is issued, report extract is run if requested, and any other inventory management processing ends with return code 12.
INFO	Issues messages EDG2243I and EDG2245I to the MESSAGE file and processing continues.
OFF	Processing of this function is turned off.
WARN	Issues messages EDG2243I and EDG2245I to the MESSAGE file. When the value is not reached, DFSMSrmm additionally issues message EDG2321W, sets a minimum return code of 4 and processing continues.

The default is VRSRETAIN(PERCENT(80),INFO).

To aid analysis of the results of the VRSRETAIN limit checking (when the action is not OFF) you can use the contents of the ACTIVITY file and extended records from the extract file. The EDGJACTP sample generates a detailed report and a summary report of newly assigned volumes showing how they are processed. See the contents of the VRSRETN and VRSRETNS files.

Using the OPTION command to switch from SMF record types to IBM-assigned SMF record types

IBM recommends that you switch from using SMF record types in the user-written range to using IBM-assigned SMF record types. Do the following steps:

1. Run the DFSMSrmm EDGAUD reports on z/OS V1R10 or higher release.
2. Update the SMF record collection jobs to select SMF record type 42 subtypes 22 and 23, as well as the existing SMF record types used for DFSMSrmm.
3. Update the SMFPRMxx parmlib member to ensure that SMF records are collected when generated by DFSMSrmm. Ensure that the SMF record type 42 subtypes 22 and 23 are included in the parmlib member.
4. Update the EDGRMMxx parmlib member to change the OPTION SMFSEC(*nnn*) SMFAUD(*nnn*) commands to be OPTION SMFSEC(YES) SMFAUD(YES).
5. Refresh the DFRMM started procedure: F DFRMM,M=xx

Partitioning tape volumes: PRTITION

Use the PRTITION command to identify special processing for DFSMSrmm to perform during library entry, export/import, eject, and CUA processing, during OPEN processing, and EXPROC processing. The PRTITION commands apply to all volumes, not only system-managed volumes. However, only system-managed volumes benefit for the functions such as library insert, CUA, and eject.

The processing provided is based on volume sets identified by PRTITION entries that you create using PRTITION commands. When a volume is processed during library entry, insert, import, eject, or export, or CUA processing, during OPEN processing, or during EXPROC processing the volume is matched to a PRTITION entry as follows:

- The TYPE(RMM|NORMM) is determined. For additional details, see the TYPE operand in “[PRTITION command operands](#)” on page 197.
- DFSMSrmm matches the volume serial number to a volume set from the most specific volume set to the least specific volume set within TYPE. This uses the VOLUMERANGE and VOLUME values including the volume prefix based on the set scope. DFSMSrmm PRTITION entries are sorted in ascending EBCDIC collating sequence from most specific to least specific matching order within type. VOLUME(*) is least specific.

The DFSMSrmm partitioning support based on the PRTITION commands requires either that you use the CBRUXENT exit shipped with DFSMSrmm or that your customized CBRUXENT calls EDGLCSUX and uses the decisions that EDGLCSUX makes.

Note:

When DFSMSrmm running mode is MANUAL (OPMODE=M), the PRTITION partition rules are not applied. In addition, DFSMSrmm does not participate in system managed tape activity; that is, DFSMSrmm does not:

- DFSMSrmm does not participate in system managed tape activity; that is, DFSMSrmm does not:
- Record system managed tape activity
- Control change of use for system managed volumes

PRTITION command syntax

Figure 71 on page 197 shows the syntax of the PRTITION Command.

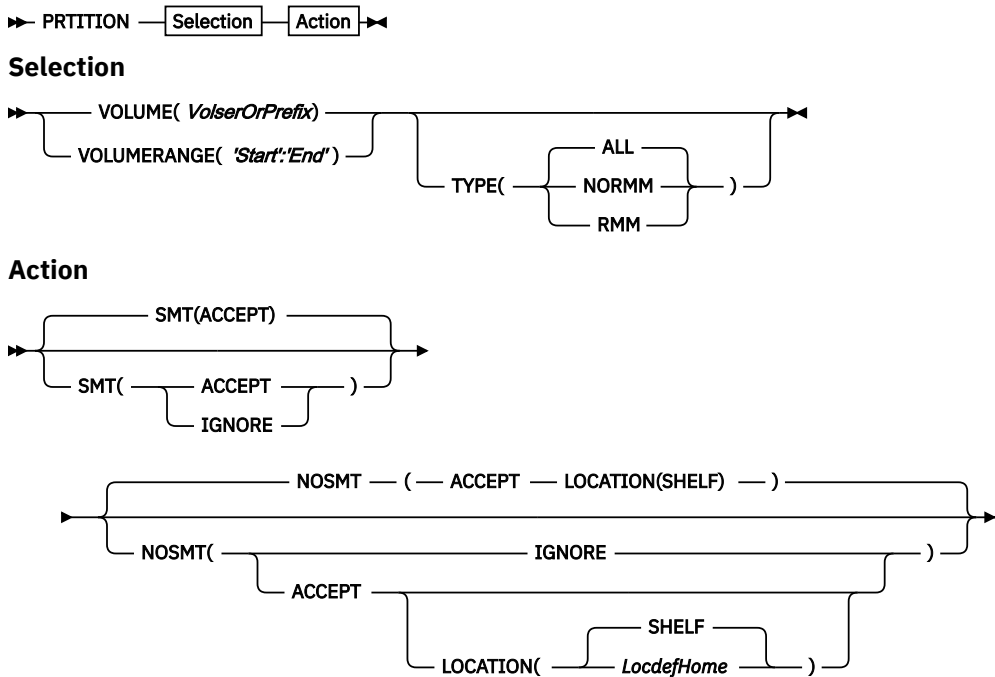


Figure 71. Parmlib member EDGRMMxx PRTITION command

PRTITION command operands

VOLUME | VOLUMERANGE

Use these operands to select volumes that are to be managed by this command. You must specify either VOLUME or VOLUMERANGE, and each command defines a set of one or more volumes. Sets cannot overlap within a TYPE, but a set can be a subset of another and such a subset is more specific. When you code a set with TYPE(RMM) and repeat that set with TYPE(NORMM), you can specify different operands for each set. If you want to use the same operands for the sets, you can do so by coding a type of ALL.

VOLUME

You can specify the volume as fully qualified or a volser prefix ending in *. A fully qualified volume is one to six alphanumeric, national or special characters. A volser prefix is zero to five alphanumeric, national, or special characters ending in an asterisk. Quotation marks are required for special characters, and the first character must not be blank. Any value ending in *, even if enclosed in quotation marks, is considered to be a volser prefix.

VOLUMERANGE

Use to select a subset of volumes based on starting and ending volser. Specify each volser value as one to six alphanumeric, national, or special characters. Quotation marks are required for each value regardless of the use of special characters, and the first character must not be blank. The end of range must not be lower than the start of the range.

TYPE(ALL | RMM | NORMM)

Use to identify the type of volumes selected by the PRTITION command. The volser is used to determine whether the volume is defined in the RMM CDS, and, depending on the function, the VOL1 label and HDR1 can be used. Therefore, DFSMSrmm cannot always determine if the volume is an existing one defined to DFSMSrmm. TYPE is determined as follows:

- For system-managed library functions such as import/export, insert/entry, CUA, and eject, only the volser is used to determine whether the volume is defined in the RMM CDS.
- For OPEN processing, the same rules as volume ignore processing are used (see [“Ignoring duplicate or undefined volume serial numbers” on page 275](#)), and the TYPE that is assigned is based on the following:
 - TYPE(RMM)
 1. The volume serial number is defined in the control data set, and there is no HDR1 tape label for the volume.
 2. The volume serial number is defined in the control data set, and no labels are used (NL, NSL, or BLP with an NL tape).
 3. The 17 characters read from the HDR1 label of the mounted volume match the last 17 characters of the data set name in the control data set.
 - TYPE(NORMM)
 1. The volume serial number is not defined in the control data set.
 2. The volume is defined, but the 17 characters read from the HDR1 label of the mounted volume do not match the last 17 characters of the data set name in the control data set.
- For EXPROC processing, TYPE(RMM) always applies because EXPROC is driven only by volumes defined to DFSMSrmm.

For each set of volumes, you can code one command with RMM and another with NORMM, but only a single command if ALL is used.

RMM

The volume must already be defined to DFSMSrmm.

NORMM

The volume is not defined to DFSMSrmm.

ALL

Applies to all volumes regardless of whether they are defined to DFSMSrmm.

ALL is the default value.

SMT | NOSMT(action)

Use the SMT or NOSMT operands to select the action you want based on whether the volume is system-managed or not. You can specify a different action for NOSMT and SMT volumes.

System managed volumes are identified as follows:

- Any request made through the EDGLCSUX programming interface that is used from the OAM installation exits: CBRUXCUA, CBRUXENT, CBRUXEJC, and CBRUXVNL.

- During OPEN processing, the tape drive is identified as being in a system-managed library.
- During EXPROC processing, the volume is in the TCDB, or the current location, or the home location is a system managed library.
- In any other case, the volume is treated as non-system managed.

ACCEPT | IGNORE

Use this operand to identify the action to be taken by DFSMSrmm. The action applies to library entry/insert/import, eject/export, and CUA processing, OPEN processing, and to EXPROC processing. During OPEN processing, once the decision is taken about IGNOREing the volume, the PRTITION checking occurs first, and the action taken before the OPENRULE REJECT/ACCEPT processing occurs. As a result, the action taken by PRTITION can influence the TYPE processing by OPENRULE.

ACCEPT

DFSMSrmm processes the volume. During entry/insert import/export, CUA, and OPEN processing, the volume is added to the DFSMSrmm CDS if not yet defined. For NOSMT volumes, the location from the LOCATION operand is used to set the volume current and home locations. During EXPROC processing, the volume is processed for return to scratch. Note that a volume identified as TYPE(NORMM) with an action of ACCEPT, that is added to the CDS, now becomes a TYPE(RMM) volume for subsequent processing including OPENRULE processing.

By default, all volumes are processed during EXPROC, and all volumes are accepted during library entry/insert/import or OPEN processing. During OPEN processing and during library entry/insert/import processing, any undefined volumes are automatically added to the DFSMSrmm CDS using the library default volume entry status.

During OPEN processing, volumes are added to the CDS only if the current OPMODE is either WARN or PROTECT.

IGNORE

DFSMSrmm does not process the volume. During entry/insert/import processing, DFSMSrmm requests that OAM ignore the volume and leave it in the 'insert' category. During CUA and eject/export processing, the request for the volume is ignored. During EXPROC processing, the return to scratch processing is skipped. During OPEN processing:

- For NORMM volumes, the volume is not added to the CDS
- For RMM volumes, processing depends on the OPENRULE entries.
- For NORMM volumes, CUA can be accomplished for an Ignored volume using EDGSPLCS, CBRSPCLCS, or ISMF ALTER of the Use Attribute.

ACCEPT is the default value.

For system-managed volumes, DFSMSrmm EXPROC processing normally skips volumes that are not defined in the current TCDB or volumes in libraries that are not online and available. This means that you do not need to create commands for these cases.

During EXPROC processing, DFSMSrmm inventory management considers your PRTITION commands as well as any EXPROC control commands specified in the SYSIN file.

LOCATION(SHELF | *LocdefHome*)

Use this operand for NOSMT volumes that are of TYPE(NORMM) with action ACCEPT. DFSMSrmm's add volume processing uses the defined location value as the volume's home location and current location.

You can specify either SHELF or the name of a LOCDEF defined location that has a specified TYPE(STORAGE,HOME). DFSMSrmm validates this value against the LOCDEF commands.

Although the operand is only used for NORMM volumes, DFSMSrmm does not prevent you from specifying it on a PRTITION command for volumes of TYPE RMM or ALL.

The default value is SHELF.

Implementing PRTITION and OPENRULE parmlib commands

Implementation depends on whether you already use REJECT commands.

If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. See [“Converting REJECT commands to PRTITION and OPENRULE commands”](#) on page 204 for information about converting from the use of REJECT commands.

If you do not use REJECT commands or PRTITION and OPENRULE commands, DFSMSrmm processing uses the default PRTITION and OPENRULE commands. These default commands implement almost all of the default processing for releases prior to z/OS V1R10 DFSMSrmm. In addition, they offer the ability to dynamically add non-system managed volumes to the CDS as volumes are used. The OPENRULE and PRTITION commands can be used on z/OS V1R10 and later releases. Releases prior to z/OS V1R10 use REJECT commands, if they are defined in parmlib.

To start implementing PRTITION and OPENRULE commands, do the following:

1. Review the customization of your installation exits, and identify the OPENRULE and PRTITION commands that can be used instead. See [“Reviewing installation exits”](#) on page 200 for additional information.
2. Identify the global PRTITION and OPENRULE commands that can be used. See [“Identifying basic rules”](#) on page 200 for additional information.
3. Review your requirements across all systems images, and identify the sets of volumes that are to be used on each system. Identify the PRTITION and OPENRULE commands required for each system to minimize parmlib maintenance as your storage requirements change.
4. Code the parmlib commands. DFSMSrmm has an option to have a shared or common parmlib member and then a system specific parmlib member for each system. The global PRTITION and OPENRULE commands are good candidates for the common parmlib member, and the more specific volume sets related commands are best placed in the system specific parmlib member.
5. Create new parmlib members that are ready for implementation, leaving the existing parmlib member as a fallback.
6. Refresh the DFRMM started procedure to use the new parmlib members (F DFRMM,M=xx).
7. At a later time, remove any unnecessary exit customization, and remove the old parmlib member that is no longer required for fallback.

Reviewing installation exits

Look at your planned or existing use of EDG_EXIT100, EDG_EXIT200, and CBRUXENT installation exits to determine whether customization can be avoided. The function provided by OPENRULE and PRTITION should allow almost any customization related to partitioning, rejecting volumes, ignoring volumes, to be avoided.

Your exit customization does not have to be removed immediately, but can be run alongside the new function, and removed only when you are confident that it is no longer needed. Any existing customization of the CBRUXENT and EDG_EXIT200 installation exits can very likely be removed and replaced with use of PRTITION parmlib commands. Any existing function in the EDG_EXIT100 installation exit that controls the use of a volume, that might fail open or reject a volume, or enable volumes to be ignored without coding EXPDT=98000, can likely be removed and replaced by use of the OPENRULE parmlib commands.

The DFSMSrmm partitioning support based on the PRTITION commands requires either that you use the CBRUXENT exit shipped with DFSMSrmm or that your customized CBRUXENT calls EDGLCSUX and uses the decisions that EDGLCSUX makes.

Identifying basic rules

To start implementing PRTITION and OPENRULE commands, you should first list the basic rules you want to implement. For example:

- For partitioning, all non-defined volumes are to be ignored.

- For open rules, only volumes defined to DFSMSrmm can be used for both input and output.

After you have the basic rules identified, you have to identify the sets of volumes and how each is to be treated or managed. For example, the only volumes to be used on a system or partition might be all those volumes with the 'A' prefix. You can create a command that uses VOLUME(A*) and then select the TYPE based on whether you will always pre-define the volumes or allow them to be defined during cartridge entry, OPEN processing, or both.

For the PRTITION commands, you must consider the system-managed and non-system managed volumes separately. Be careful not to specify TYPE(RMM|ALL) with NOSMT(IGNORE) unless you really want EXPROC processing to skip the volumes. However, TYPE(RMM|ALL) with SMT(IGNORE) makes sense because DFSMSrmm causes OAM to ignore the volume and leave for another system or partition, and it also causes DFSMSrmm to skip EXPROC without checking if the volume is in the TCDB.

Consider the function provided by PRTITION and OPENRULE commands and whether it makes sense to exploit it. For example:

- You can have separate commands based on whether the volume is defined to DFSMSrmm.
- OPENRULE provides an automated way to ignore volumes at open time without the need to customize EDGUX100 or use EXPDT=98000 in JCL.
- OPENRULE allows you to control the use of existing data sets, such as enforcing cataloging and reference only from the creating system.

Examples of OPENRULE and PRTITION commands

These examples show how OPENRULE and PRTITION commands can be used.

The sample commands in [Figure 72 on page 201](#) specify volumes defined to DFSMSrmm in the "A*" volume set that must not be used and must be partitioned.

```
OPENRULE VOLUME(A*) TYPE(RMM) ANYUSE(REJECT)
PRTITION VOLUME(A*) TYPE(ALL) SMT(IGNORE) NOSMT(IGNORE)
```

Figure 72. Volumes not used and partitioned

The sample commands in [Figure 73 on page 201](#) specify volumes defined to DFSMSrmm in the "prefix" volume set that cannot be used for output on this system.

```
OPENRULE VOLUME(prefix) TYPE(RMM) -
OUTPUT(REJECT) INPUT(ACCEPT)
```

Figure 73. Volumes not used for output on this system

The sample commands in [Figure 74 on page 201](#) specify volumes not defined to DFSMSrmm, but are partitioned and must not be used on this system.

```
OPENRULE VOLUME(*) TYPE(NORMM) ANYUSE(REJECT)
PRTITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)
```

Figure 74. Volumes partitioned and not used on system

The sample commands in [Figure 75 on page 202](#) specify volumes not defined to DFSMSrmm and not allowed to be written on, but are allowed to be read. The partitioned rule does not add the volume to the DFSMSrmm CDS.

On occasion, you might need to read tapes not defined to DFSMSrmm on a stand-alone tape drive, where there is a possibility that an incorrect tape volume is mounted. In this situation, you would want the opportunity to get another chance to mount a correct tape. Using OPENRULE with OUTPUT(IGNORE) allows you to issue another mount, if necessary. If OPENRULE with INPUT(IGNORE) or ANYUSE(IGNORE) is used, then the job will fail with EDG4061I and IEC149I 813 04,IFG0195H, because DFSMSrmm ignores the volume. Mounting of an incorrect tape volume is unlikely to happen in an automated tape library.

```

OPENRULE VOLUME(*) TYPE(NORMM) OUTPUT(IGNORE)
PRITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)

```

Figure 75. Volumes partitioned, not used on system, but allowed to be read

The sample commands in [Figure 76 on page 202](#) specify this situation:

- Three systems, SYSA, SYSB, and SYSC, that are each allowed to use only a single range of system-managed volumes (JT, JP, JX).
- Each system has its own CDS and TCDB.
- System-managed volumes are not to be automatically defined to DFSMSrmm.
- Private volumes can be shared.
- Undefined non-system-managed volumes can be used for input.
- The ignoring of private system-managed volumes from other partitions and the ignoring of undefined non-system-managed volumes is required.

Note: You must use IDCAMS CREATE VOLENT to manually add private volumes temporarily to the another TCDB in order to mount volumes from another partition.

```

/* SYSA Example */
/* Allow read of undefined volumes */
OPENRULE VOLUME(*) TYPE(NORMM) -
INPUT(ACCEPT) OUTPUT(REJECT)
/* Ignore for input selected system managed volumes if authorized */
OPENRULE VOLUME(JP*) TYPE(NORMM) -
INPUT(IGNORE) OUTPUT(REJECT)
OPENRULE VOLUME(JX*) TYPE(NORMM) -
INPUT(IGNORE) OUTPUT(REJECT)
/* Global partition rules - ignore all volumes */
/* Allow EXPROC for NOSMT */
PRITION VOLUME(*) TYPE(RMM) SMT(IGNORE) NOSMT(ACCEPT)
PRITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)
/* This partition owns JT* volumes if predefined */
PRITION VOLUME(JT*) TYPE(RMM) SMT(ACCEPT)

```

Figure 76. Three systems using a range of system-managed volumes

Defining tapes not available on systems: REJECT

Note: While the REJECT command is still supported by DFSMSrmm, you should use the OPENRULE (described in [“Controlling the use of tape volumes: OPENRULE” on page 157](#)) and PRITION (described in [“Partitioning tape volumes: PRITION” on page 196](#)) commands to prevent a range of tapes from being used on a particular system. If you are already using REJECT commands, you should consider converting them to OPENRULE and PRITION commands. See [“Converting REJECT commands to PRITION and OPENRULE commands” on page 204](#).

Use the REJECT command to prevent a range of tapes from being used on a particular system. For example, you can reject production tapes on a development system.

This command is useful under these conditions:

- Using a shared RACF data set and the security profiles for a volume allow the volume to be used on all systems.
- To partition tape libraries as described in [“Partitioning system-managed tape libraries” on page 125](#).
- Preventing a stacked volume from being used outside a VTS.

The REJECT command helps you limit tape usage at the system level. [Figure 77 on page 203](#) shows an example of the REJECT command and the operands you can code in the parmlib member EDGRMMxx.

```
REJECT OUTPUT(CC12*)
REJECT ANYUSE(VM1*), OUTPUT(VM*)
REJECT ANYUSE(DD0*)
REJECT ANYUSE(*)
```

Figure 77. Parmlib member EDGRMMxx REJECT command examples

REJECT command syntax

Figure 78 on page 203 shows the syntax of the REJECT command:

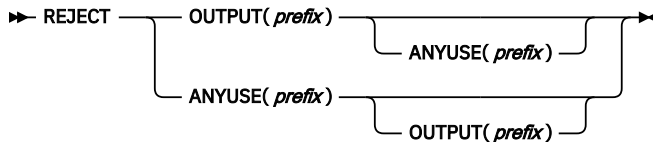


Figure 78. Parmlib member EDGRMMxx REJECT command syntax

You can specify multiple REJECT commands to define different ranges of tapes. However, use only one ANYUSE operand and one OUTPUT operand for each REJECT command, as shown in [Figure 77 on page 203](#).

DFSMSrmm allows all volume mounts. DFSMSrmm checks volume validity only when a data set on the volume is opened. When a data set on a volume is opened, DFSMSrmm checks the volume's shelf location against the prefixes specified in the REJECT command. If a volume has no library shelf location, DFSMSrmm checks the volume serial number against the prefixes specified in the REJECT command. DFSMSrmm recognizes the most specific prefix match. If you specify both ANYUSE and OUTPUT for the same prefix, ANYUSE overrides OUTPUT. Normally, REJECT prefixes match those that are used on VLPOOL commands, but it is not mandatory.

At OPEN time, DFSMSrmm can reject a range of tapes if you define the range of volumes to DFSMSrmm. DFSMSrmm cannot reject a range of tapes that are not defined to it.

You can use the REJECT ANYUSE command with an automated tape library to prevent volumes from being automatically defined in the TCDB when they are entered in an automated tape library. If the volume matches the REJECT ANYUSE command, the volume is not defined to the TCDB. Another system sharing the automated tape library can take the volume to define it in its TCDB only. You can also prevent a volume from being defined in the TCDB by specifying other than the USE(MVS) attribute when defining the volume to DFSMSrmm.

If you use REJECT OUTPUT commands, the volume is defined in the TCDB and can be entered into an automated tape library.

Use caution when using the REJECT command with an automated tape library as it is possible to continue needless rejecting of volumes.

REJECT command operands

ANYUSE(prefix)

Specifies the shelf locations of volumes not available for read or write processing. *prefix* is a generic shelf location that consists of one-to-five alphanumeric, national, or special characters that are followed by an asterisk (*). Specify a single asterisk if, at OPEN time, you want to reject all volumes not defined to DFSMSrmm. For example, ANYUSE(*) prevents the use of foreign tapes, unless you use the DFSMSrmm installation exit EDGUX100 to request that DFSMSrmm ignore such tapes. You can also use the REJECT ANYUSE command to partition system-managed tape libraries. See [“Partitioning system-managed tape libraries” on page 125](#) for more information.

Default: None.

OUTPUT(prefix)

Specifies the shelf locations of volumes not available for write processing. *prefix* is a generic shelf location that consists of one-to-five alphanumeric, national, or special characters that are followed by

an asterisk (*). Specify a single asterisk if, at OPEN time, you want to prohibit writing to all volumes that are not defined to DFSMSrmm.

Default: None.

Converting REJECT commands to PRTITION and OPENRULE commands

If you use REJECT commands, you have to convert from the use of REJECT commands in order to use the PRTITION and OPENRULE commands. Review your existing REJECT commands across all system images and identify the sets of volumes to be used on each system. Identify the PRTITION and OPENRULE commands required for each system to minimize parmlib maintenance as your storage requirements change. Until you define one or more PRTITION or OPENRULE commands, both partitioning and rejecting of volumes are controlled by REJECT commands. Any REJECT commands that specify ANYUSE are used for partitioning of undefined volumes, but all REJECT commands are used for rejecting volumes at OPEN time.

When you use either PRTITION or OPENRULE commands, the REJECT commands are no longer used so you must start using both PRTITION and OPENRULE at the same time to avoid loss of function. You have to remove the REJECT commands from parmlib, because they will fail when any PRTITION or OPENRULE commands are defined. When parmlib is processed, DFSMSrmm issues message EDG0239E, followed by message EDG0215D to provide an option to ignore the error or fail and then retry with message EDG0107D with another parmlib member. When you have created your new commands, remove the REJECT commands. The PRTITION and OPENRULE commands can only be used on z/OS V1R10 and later releases. Lower level releases continue to use the REJECT commands.

To give you more choice and flexibility, conversion from REJECT commands should not be done one to one and should not be automated. When each REJECT command is converted strictly to an equivalent PRTITION and OPENRULE command, you can end up with too much complexity and duplication. The best approach is to start from scratch and list the basic rules you want to implement. See [“Identifying basic rules” on page 200](#) for additional information. In addition to understanding the basic rules, you must also consider the changed function and whether you are affected by it:

- For volumes not defined to DFSMSrmm, the REJECT command with ANYUSE causes CUA requests to fail. With the PRTITION command, you can choose between the actions of either ACCEPT or IGNORE. You no longer have the option to fail a CUA request. The ACCEPT action results in the volume being added to DFSMSrmm and the CUA continues. The IGNORE action causes DFSMSrmm to take no action for the CUA request. However, it is possible to affect CUA via EDGSPCLS, CBRSPCLS, or ISMF ALTER of the Use Attribute.
- For non-system-managed volumes that are not defined to DFSMSrmm, the default processing during OPEN did not add the volume to the CDS. The default processing for the PRTITION command is that non-system-managed volumes are added to the CDS.

Examine any existing EDG_EXIT100, EDG_EXIT200, and CBRUXENT installation exits to determine whether they contain customization that is no longer needed. The PRTITION and OPENRULE functions should allow the removal of almost any customization related to partitioning, rejecting volumes, and ignoring volumes. The existing installation exit customization need not be removed immediately, but can be used alongside the new function, and removed only when it is certain that they are no longer needed. Any existing customization of CBRUXENT and EDG_EXIT200 can very likely be replaced by the use of PRTITION parmlib commands. Any existing EDGUX100 function that controls use of a volume, that might fail the open of a tape data set, or reject a volume, or enable volumes to be ignored without coding EXPDT=98000, can likely be replaced by the use of OPENRULE parmlib commands.

Follow these steps:

1. Review customization of installation exits and identify the OPENRULE and PRTITION commands that can be used instead
2. Identify your global PRTITION and OPENRULE commands
3. Review your requirements across all systems images and identify which sets of volumes should be used on each system. Identify the PRTITION and OPENRULE commands required for each system to minimize parmlib maintenance as your storage requirements change

4. Code the new parmlib commands.

DFSMSrmm provides an option to have a common parmlib member and then a system-specific parmlib member for each system. The global PRTITION and OPENRULE commands are good candidates for the common parmlib member and the more specific volume sets related commands are best placed in the system specific parmlib member.

5. Create new parmlib members ready for implementation, leaving the existing parmlib member as a fallback

6. Refresh DFRMM started procedure to use the new parmlib members (F DFRMM,M=xx)

7. Plan to remove at a later time exit customization and remove the old parmlib member that is no longer required for fallback.

Examples of converting REJECT commands to OPENRULE and PRTITION commands

Table 18 on page 205 provides simple examples that show the conversion of REJECT commands to OPENRULE and PRTITION commands.

Note: For simplicity, the examples in Table 18 on page 205 only show how OPENRULE and PRTITION commands can be constructed from existing REJECT commands, considered one command at a time. While that approach will work, it might lead to unnecessary effort and complexity. A better method is to simplify the task by first determining a global policy and then adding to a parmlib only the OPENRULE and PRTITION commands required to implement that policy.

Table 18. Converting REJECT commands to OPENRULE and PRTITION commands		
Example	REJECT Command	OPENRULE and PRTITION Commands
1	REJECT ANYUSE(<i>prefix</i>)	OPENRULE VOLUME(<i>prefix</i>) TYPE(RMM) ANYUSE(REJECT) PRTITION VOLUME(<i>prefix</i>) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)
In example 1, the existing REJECT ANYUSE command is being used to control partitioning and to reject volumes at OPEN time. The right most column shows the equivalent PRTITION and OPENRULE commands. Remember that a REJECT with a prefix, when used at OPEN time, applies only to volumes defined to RMM, so TYPE(RMM) is used for OPENRULE with an action of ANYUSE(REJECT). However, for partitioning, REJECT ANYUSE applies only to volumes not defined to RMM, so TYPE(NORMM) with ACTION(IGNORE) is used for PRTITION.		
2	REJECT OUTPUT(<i>prefix</i>)	OPENRULE VOLUME(<i>prefix</i>) TYPE(RMM) OUTPUT(REJECT) INPUT(ACCEPT)
The sample commands in example 2 show REJECT is used only for OPEN time rejects. A REJECT with a prefix, when used at OPEN time, applies only to volumes defined to RMM so TYPE(RMM) is used for OPENRULE. The REJECT OUTPUT allows input processing, so the INPUT(ACCEPT) option is coded on the OPENRULE for completeness, although INPUT(ACCEPT) is the default.		
3	REJECT ANYUSE(*)	OPENRULE VOLUME(*) TYPE(NORMM) ANYUSE(REJECT) PRTITION VOLUME(*) TYPE(NORMM) SMT(IGNORE) NOSMT(IGNORE)
The sample commands in example 3 show REJECT is used by both partitioning and for OPEN time rejects. A REJECT with PREFIX(*) applies only to volumes not defined to RMM, so TYPE(NORMM) is used for both OPENRULE and for PRTITION.		
4	/* Example for SYSA only */ REJECT ANYUSE(JP*) REJECT ANYUSE(JX*) REJECT ANYUSE(*)	/* Allow read of undefined volumes */ OPENRULE VOLUME(*) TYPE(NORMM) - OUTPUT(REJECT) /* Ignore for input selected system managed volumes if authorized */ OPENRULE VOLUME(JP*) TYPE(NORMM) - INPUT(IGNORE) OUTPUT(REJECT) OPENRULE VOLUME(JX*) TYPE(NORMM) - INPUT(IGNORE) OUTPUT(REJECT) /* Global partition rule - ignore all smt volumes */ PRTITION VOLUME(*) TYPE(RMM) - SMT(IGNORE) NOSMT(ACCEPT) PRTITION VOLUME(*) TYPE(NORMM) - SMT(IGNORE) NOSMT(IGNORE) /* This partition owns JT* volumes if predefined */ PRTITION VOLUME(JT*) TYPE(RMM) SMT(ACCEPT)

Table 18. Converting REJECT commands to OPENRULE and PRTITION commands (continued)

Example	REJECT Command	OPENRULE and PRTITION Commands
The sample commands in example 4 shows the following: • Three systems, SYSA, SYSB, and SYSC, that are each allowed to use only a single range of system-managed volumes (JT, JP, JX). The example shows the commands for SYSA only. • Each system has its own CDS and TCDB. • System-managed volumes are not to be automatically defined to DFSMSrmm. • Private volumes can be shared. • Undefined non-system-managed volumes can be used for input. • EDGUX100 is customized to automate the ignoring of private system-managed volumes from other partitions, and the ignoring of undefined non-system-managed volumes. This is based on the RMM API checking if a volume is defined to RMM. The following RACF FACILITY class profiles are also required: <pre>STGADMIN.EDG.IGNORE.TAPE.NORMM.JP* STGADMIN.EDG.IGNORE.TAPE.NORMM.JX*</pre> The customization in the EDG_EXIT100 installation exit is no longer needed and can be removed. The OPENRULE with INPUT(IGNORE) provides the same function.		

Defining security classes: SECCLS

Use the SECCLS command to define security classes for data sets and volumes. These security classes appear in reports and in output for the RMM TSO subcommands. DFSMSrmm records these security classes in the DFSMSrmm control data set only; it does not make RACF aware of them. There is no connection between these definitions and any similar definitions or function provided in RACF, but you can use similar values for overall consistency.

DFSMSrmm determines the security classification of a tape volume with multiple data sets by the highest classification found for a single data set.

Data sets that do not match any of the masks specified in SECCLS definitions are assigned no security classification. DFSMSrmm uses a number 0 to indicate no security classification. Whenever you create a tape data set, DFSMSrmm uses the SECCLS masks to classify data sets and volumes. [Figure 79 on page 207](#) shows an example of the SECCLS command and the operands you can code in the parmlib member EDGRMMxx.

If you remove a security class that is assigned to a volume, DFSMSrmm issues an error message when a data set on the volume is opened. DFSMSrmm also treats the volume as having the lowest defined security level. The default security class is defined by a mask of '***'.

```

SECCLS  NUMBER(01)           -
        NAME(UNCLASS)       -
        DESCRIPTION('UNCLASSIFIED') -
        MASK('**')          -
        SMF(N)               -
        MESSAGE(N)           -
        ERASE(N)             -
SECCLS  NUMBER(02)           -
        NAME(U)              -
        DESCRIPTION('UNCLASS') -
        MASK('STSGAM.**')     -
        SMF(N)               -
        MESSAGE(N)           -
        ERASE(N)             -
SECCLS  NUMBER(10)           -
        NAME(IU0)            -
        DESCRIPTION('INTERNAL USE ONLY') -
        SMF(N)               -
        MESSAGE(N)           -
        ERASE(N)             -
        MASK('SYS1.IU0.**')   -
SECCLS  NUMBER(30)           -
        NAME(CC)             -
        DESCRIPTION('CONFIDENTIAL') -
        MASK('PAYROLL.**')    -
        SMF(Y)               -
        MESSAGE(N)           -
        ERASE(Y)             -
SECCLS  NUMBER(100)          -
        NAME(IC)             -
        DESCRIPTION('CONFIDENTIAL') -
        MASK(+
          '**.IC.**',+
          '**.VERTR.**',+
          '**.CONFI.**'+
        )
        SMF(Y)               -
        MESSAGE(N)           -
        ERASE(Y)             -

```

Figure 79. Parmlib member EDGRMMxx SECCLS command examples

SECCLS command syntax

Figure 80 on page 207 shows the syntax of the SECCLS command:

► SECCLS — NUMBER(— *nnn* —) — DESCRIPTION('*cc.....cc*') ►

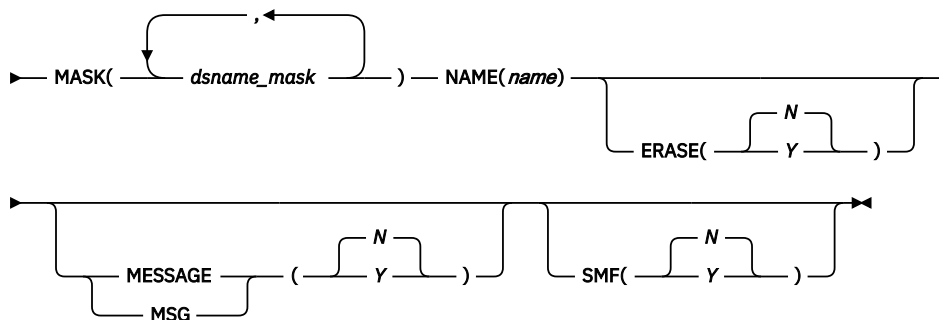


Figure 80. Parmlib member EDGRMMxx SECCLS command syntax

To make the default security class the highest level, specify these operands with the SECCLS command:

```
SECCLS NUMBER(255) MASK('**')
```

If you do not specify any SECCLS statements, DFSMSrmm defaults to a classification of blanks.

SECCLS command operands

DESCRIPTION('cc.....cc')

Describes the security class. DFSMSrmm uses the description you provide as the security class name. Specify a description between 1 and 32 characters.

Default: None. You must specify this operand.

ERASE(N|Y)

Specifies whether volumes must be erased on release. *Y* indicates volumes must be erased on released, and *N* indicates that they do not have to be.

You can use the DFSMSrmm utility EDGINERS to automatically erase and reinitialize tapes, or you can use your own method to erase them.

Default: ERASE(N)

Note: DFSMSrmm supports the use of the RACF 'erase on scratch' setting in DATASET profiles instead of using the SECCLS ERASE option. The support is only provided when TAPEAUTHDSN=YES is set in DEVSUPxx and the DATASET class is active.

MASK(, 'dsname_mask')

Specifies a data set name mask used for assigning a security classification. You can specify multiple data set name masks separated by a comma. In addition to standard data set naming conventions, you can use these masking characters to create the data set name mask:

A single * represents a single qualifier of any number of characters.

a single * when used within a qualifier represents zero or more characters.

more than one single * can be used within a qualifier as long as a character precedes or follows the *.

.** represents zero or more qualifiers. At the end of the mask, it indicates to ignore any remaining characters.

** indicates to select all data sets.

% (percent_sign)

A place holder for a single character.

For example, you can specify MASK('USERID.**.CONF.**').

Default: You are required to specify this operand.

MESSAGE|MSG(N|Y)

Specifies whether to issue operator confirmation messages when a data set on a volume in this security class is opened. Specify *Y* to issue operator confirmation messages for this class, or *N* to suppress the messages. The message number is EDG4008A.

You can use this opportunity to perform any local operation procedures, such as adding a security sticker to a volume.

Default: MSG(N)

NAME(name)

Specifies a name for the security class. The name is used for reports and communication with users. Specify a value between 1 and 8 characters.

Default: None. You are required to specify this operand.

NUMBER(nnn)

Specifies a security classification number between 1 and 255.

The security classification numbers indicate relative levels of importance. Higher numbers mean higher levels of security. Your installation can assign more specific meanings to the numbers.

Default: None. You are required to specify this operand.

SMF(N|Y)

Specifies whether to write an SMF security record each time a tape data set in this class is opened. *Y* means to write an SMF record, and *N* means not to write it. See [z/OS DFSMSrmm Reporting](#) for information on using EDGAUD to produce a report for these SMF records.

Default: SMF(N)

Defining pools: VLPOOL

Use the VLPOOL command to define pools and to set release actions for all volumes in the pool. See Chapter 4, “Organizing the removable media library,” on page 75 for information about pooling. When you add a new volume to the library, DFSMSrmm assigns it a shelf location from the specified pool. [Figure 81](#) on page 209 shows an example of the VLPOOL command and the operands you can code in the parmlib member EDGRMMxx.

```
VLPOOL PREFIX(Y2*) TYPE(R) DESCRIPTION('CUSTOMER REEL TAPES') -  
    MEDIANAME(3420) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(Y3*) TYPE(R) DESCRIPTION('CUSTOMER 3480 CARTRIDGES') -  
    MEDIANAME(3480) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(P0*) TYPE(R) DESCRIPTION('SOFTWARE PRODUCT REELS') -  
    MEDIANAME(3420) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(P2*) TYPE(R) DESCRIPTION('SOFTWARE CARTRIDGES') -  
    MEDIANAME(3480) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(C*) TYPE(R) DESCRIPTION('MVS AND VM BACKUP TAPES') -  
    MEDIANAME(3480) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(X8*) TYPE(R) DESCRIPTION('IN TAPES') -  
    MEDIANAME(3480) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(M*) TYPE(S) DESCRIPTION('SCRATCH POOL') -  
    MEDIANAME(3480) RACF(Y) EXPDTCHECK(Y) SYSID(DG4)  
VLPOOL PREFIX(*) TYPE(R) DESCRIPTION('EVERYTHING ELSE') -  
    MEDIANAME(3480) RACF(Y) EXPDTCHECK(Y)  
VLPOOL PREFIX(D*) NAME(DEFAULT) TYPE(S) SYSID(SYS1) -  
    DESCRIPTION('Default pool')  
VLPOOL PREFIX(S*) NAME(SPECIAL) TYPE(R) -  
    DESCRIPTION('EDGUX100 set pool')
```

Figure 81. Parmlib member EDGRMMxx VLPOOL command examples

VLPOOL command syntax

[Figure 82](#) on page 210 shows the syntax of the VLPOOL command:

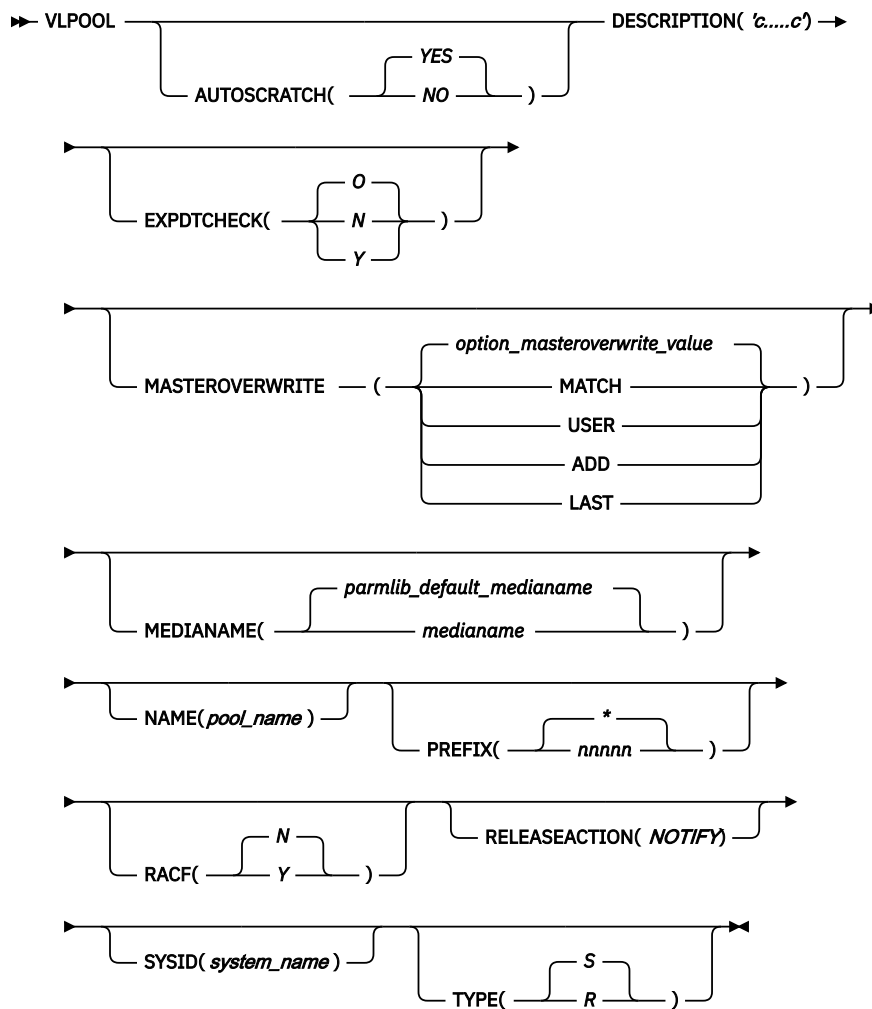


Figure 82. Parmlib member EDGRMMxx VLPOOL command syntax

Each VLPOOL command defines a pool. Each generic shelf location defined by PREFIX must be unique. If you do not specify any VLPOOL commands, or do not specify a VLPOOL command for POOL(*), DFSMSrmm automatically creates a default pool definition that includes all the shelf locations and volumes in them. The defaults for this pool are:

```
VLPOOL RACF(N) TYPE(S) EXPDTCHECK(O) MEDIANAME(parmlib_default_medianame) -
DESCRIPTION('DEFAULT POOL') PREFIX(*)
```

VLPOOL command operands

AUTOSCRATCH(YES|NO)

Use this operand to override the scratch processing release action for volumes in this pool. By default, AUTOSCRATCH(YES) return to scratch is performed automatically when expiration processing is running on a system that has access to the catalogs, TCDB, and library for the volume. If you need to take any special actions not performed by DFSMSrmm, perhaps on another system, specify AUTOSCRATCH(NO).

When you specify AUTOSCRATCH(YES) and do not manually confirm the scratch release action, DFSMSrmm performs return to scratch processing, including:

- UNCATALOG parmlib option.
- TPRACF parmlib option.
- SMSTAPE(UPDATE(SCRATCH)).

When you specify AUTOSCRATCH(NO) or you manually confirm the scratch release action for any volume, DFSMSrmm does not perform this part of the return to scratch processing. DFSMSrmm assumes that you have manually performed the cleanup already.

When you specify AUTOSCRATCH(NO), DFSMSrmm does not return the volume to scratch status until you have manually confirmed the volume release action and run expiration processing.

DFSMSrmm checks the AUTOSCRATCH setting and the scratch release action for the volume when the volume is returning to scratch. If the scratch release action is set and the volume is in a pool with AUTOSCRATCH(NO), DFSMSrmm leaves the volume in pending release status. You must perform whatever actions or cleanup activity you require and confirm that the scratch release action has been performed using the subcommand: RMM CV volser CRLSE(SCRATCH). Run expiration processing on any DFSMSrmm system to return the volume to scratch.

Default: AUTOSCRATCH(YES)

DESCRIPTION('c.....c')

Describes the pool. Specify a description between 1 and 32 characters. You must enclose the value in quotation marks if you use blanks or special characters.

Default: None. You are required to specify this operand.

EXPDTCHECK(Y|N|O)

Specifies to automate the processing of unexpired tape data sets at the pool level.

EXPDTCHECK(N) indicates that DFSMSrmm is not to check or validate the expiration date of data sets on a tape. It allows the tape to be overwritten after the system has checked the validity of the tape. DFSMSrmm does not overwrite or replace the expiration date on the tape. This situation happens when a volume is released early, or when someone changes its expiration date or retention period by using the RMM TSO subcommands.

Recommendation: Specify N when running a DFSMSrmm-managed scratch tape pool to ensure that tapes are reused only when the information on them is no longer required. DFSMSrmm can easily automate reuse of tapes if they are still expiration protected, for example, when they are reused as scratch tapes.

EXPDTCHECK(Y) indicates that DFSMSrmm ensures that all unexpired tape data sets are never overwritten. It fails specific mount requests and requests a remount if the tape request is nonspecific. If an expiration date or retention period was coded in the JCL when the date was originally written to the volume, the tape label is expiration date protected, and DFSMSrmm records this as the original expiration date for the volume. Setting EXPDTCHECK(Y) ensures that the original expiration dates specified in the JCL are honored, even if the tape has been released early, or its expiration date or retention period has been changed using the RMM TSO subcommands.

When you specify Y and a tape is expiration protected, reinitialize it before using it again. You can determine what volumes you need to reinitialize by looking at the original expiration date recorded in the DFSMSrmm control data set. To find the date, see the extract data set or use the RMM LISTVOLUME subcommand.

EXPDTCHECK(O) indicates that DFSMSrmm takes no action but allows the operator or automated software such as NetView to reply as necessary to any write-to-operator messages (IEC507D).

DFSMSrmm ignores the EXPDTCHECK operand and requires an operator reply to the write-to-operator with reply messages when:

- DFSMSrmm is running in record-only mode. DFSMSrmm does not have enough information to base a decision on. Your installation's procedures help the operator to decide how to reply to messages.
- DFSMSrmm is running in warning mode and the volume is rejected for any reason. DFSMSrmm decides that the volume should not be used, but allows its use because in warning mode it does not reject volumes. Your operator or current operating procedures determine whether to reuse the volume and override the expiration protection.

Note: When you use the EDG_EXIT100 exit to tell DFSMSrmm to ignore a volume, DFSMSrmm does not reply to the IEC507D message, because the volume is not DFSMSrmm-managed.

Default: EXPDTCHECK(O)

MASTEROVERWRITE(ADD|LAST|MATCH|USER)

Specify the VLPOOL MASTEROVERWRITE operand to control how DFSMSrmm allows the overwriting of a volume. You can specify if you want to allow data to be appended to the end of a volume or overwritten, or to allow new files to be added to a volume. The MASTEROVERWRITE value applies to all volumes that reside in a pool. If you do not specify a MASTEROVERWRITE value, DFSMSrmm uses the MASTEROVERWRITE value that you set using the EDGRMMxx parmlib member OPTION MASTEROVERWRITE operand, as described in [“Defining system options: OPTION” on page 161](#).

ADD

Specify this value so new data can be created and no existing data can be destroyed. No existing file on a volume can be recreated, but the last file can have new data added to it. When adding data to the last file, DFSMSrmm checks that the data set name used must match the existing data set name. Select this option when you want the last file on the volume to be extended or a new file added to the volume.

Note: DFSMSrmm enforces the MASTEROVERWRITE(ADD) option on a WORM tape that is in master status. This is done to ensure that you see a message from DFSMSrmm rather than one of a number of symptoms as a result of the tape drive preventing overwrites.

LAST

Specify this value to ensure that when an existing file on a master volume is being written to that only the last file on the volume can be used. The data set name used must match the existing data set name. Select this option when you want the last file on the volume to be used for output.

MATCH

Specify this value to ensure that when an existing file on a master volume is being used for output that exactly the same data set name must be used. Select this option when you want any existing file on the volume to be recreated regardless of whether it is the last file on the volume as long as the same data set name is used.

When you use an existing tape file for output all the files that are higher in sequence are destroyed.

USER

Specify this value to allow any existing file on a master volume to be used for output regardless of the data set names being used and its relative file position on the volume. Select this option when you want validation of master volumes to be just the same as for user status volumes.

When you use an existing tape file for output all the files that are higher in sequence are destroyed.

Default: EDGRMMxx parmlib member OPTION MASTEROVERWRITE operand.

MEDIANAME(media_name)

Specifies a media name for all volumes in this pool. Specify a one-to-eight character name. If you do not specify a MEDIANAME, DFSMSrmm uses the medianame that you set using the EDGRMMxx parmlib member OPTION MEDIANAME operand, as described in [“Defining system options: OPTION” on page 161](#).

You can specify any name because DFSMSrmm does not check whether the device type has been defined to z/OS. Some examples of MEDIANAME that you might define include: CART, ROUND, SQUARE, 3420, 3480, TAPE, OPTICAL, CASSETTE. DFSMSrmm accepts media names that have not been generated on the z/OS system that is running DFSMSrmm. Use of MEDIANAMES that describe size or shape can give you more flexibility in the media that can reside in a pool. Use MEDIANAME to identify different types of physical shelf space for different media, or to distinguish different media characteristics such as Cartridge System Tape and Enhanced Capacity Cartridge System Tape. See [“Defining storage locations: LOCDEF” on page 143](#) for information about using media names when defining storage locations.

If you change a media name for a VLPOOL command for an existing pool of volumes, or add a new VLPOOL command that has a different media name than existing volumes that are covered by that VLPOOL command, you must consider changing the media names for those existing volumes. Refer

to [“Changing pool definitions” on page 79](#) which describes how to use RMM CHANGEVOLUME *volser* MEDIANAME to make the volume media name match the value in the VLPOOL command.

Default: MEDIANAME(*parmlib_default_medianame*)

NAME(*pool_name*)

Specifies a pool name that DFSMSrmm uses to update operator messages and tape drive displays for nonspecific tape mount requests. *pool_name* can be a 1 to 8 character name. The first character must be an alphanumeric or national character. Blank, comma, or semicolon cannot be used. Specify a BTLS category name to use DFSMSrmm pooling support for volumes managed by BTLS. *pool_name* can be a tape storage group name defined in your active SMS configuration. NAME is optional and is used on tape drive displays. To use the pool name in messages, you must specify RACK(999) on the MNTMSG command so that the message is updated at the end of the message text.

If you specify a NAME value that is a valid storage group name, DFSMSrmm uses the NAME value as the default storage group name for all the volumes added into this pool range. By naming a specific storage group with the RMM ADDVOLUME subcommand, you can select a pool for a volume at the individual volume level.

You can use the same NAME value on multiple VLPOOL commands enabling you to group multiple prefix ranges into a single logical pool without the need to use the RMM ADDVOLUME subcommand to specify storage group names. You can use RMM ADDVOLUME or RMM CHANGEVOLUME subcommand with the STORAGEGROUP operand to add or remove individual volumes in a logical pool.

Default: None.

PREFIX(*nnnnn)**

Specifies a generic shelf location that is used in operator messages, RMM TSO subcommands, and the DFSMSrmm ISPF dialog. A pool prefix is one to five alphanumeric, national, or special characters followed by an asterisk. Use a single asterisk to specify the default volume pool that contains all volumes not specifically included in another pool.

If a volume's shelf location falls into a number of possible pools, DFSMSrmm chooses the most specific pool. For example, you defined AB* and ABC* as pools. If volume ABC123 is from shelf location ABC123, it belongs in pool ABC*, not AB*. DFSMSrmm prevents duplicate pool prefixes.

DFSMSrmm uses the prefix value to assign a newly defined volume to a scratch pool. For all volumes, DFSMSrmm uses the:

- Storage group value specified with the RMM ADDVOLUME subcommand issued when the volume is defined to DFSMSrmm.
- PARMLIB VLPOOL NAME operand value to assign the storage group when NAME specifies a valid tape storage group.
- Pool prefix to assign a pool when no NAME operand value or storage group value is specified.

Default: PREFIX(*)

RACF(Y|N)

Specifies the type of RACF tape support that DFSMSrmm should provide for volumes in the pool you are defining. When you are defining RACF tape support for volumes in a pool, you must look at the RACF tape support you have defined for your installation with the OPTION TPRACF command described in [“Defining system options: OPTION” on page 161](#). Specify Y if you want DFSMSrmm to create RACF tape profiles for the volumes in the pool. Ensure that OPTION TPRACF (*AUTOMATIC* or *PREDEFINED* or *CLEANUP*) is specified. If OPTION TPRACF(*NONE*) is specified, DFSMSrmm will not create the RACF tape profiles.

Only specify Y for tape pools because TAPEVOL profiles are not required for optical volumes.

Specify N if you do not want DFSMSrmm to create RACF tape profiles for the volumes in the pool. If you specify N, DFSMSrmm plays no part in creating or deleting RACF tape profiles, regardless of the system-wide option TPRACF.

Default: RACF(N)

RELEASEACTION(NOTIFY)

Use this operand with the NOTIFY value to automatically set the NOTIFY release action for all volumes in this pool. If you have an e-mail address for the owner of the volume, DFSMSrmm sends the owner notification that the volume is pending release. By default, DFSMSrmm does not set the NOTIFY release action. The VLPOOL NOTIFY value is checked at the time the volume is set pending release. If NOTIFY is set for the volume, DFSMSrmm sets the NOTIFY release action.

Recommendation: You could use this option to build in a delay or a checkpoint in scratch processing to ensure that volumes are ready to return to scratch. If parmlib OPTION NOTIFY(NO) is set, or you do not have an e-mail address for the owner of the volume, you must notify the user and later confirm the notify action to DFSMSrmm.

Default: None.

Note: When RMM DV volser RELEASE is issued or when API EDGTVEXT/EDGDFHSM is used by HSM, it releases the tapes which effectively uses the same process as RMM DV volser RELEASE and the VLPOOL RELEASEACTION(NOTIFY) parameter is not used.

SYSID(system_name)

Associates the scratch pool you are defining with a particular system. Specify a value one to eight characters long. DFSMSrmm matches the value with the SYSID operand of the OPTION command.

DFSMSrmm enforces a match on SYSID when validating nonspecific tape mounts. Only scratch volumes from a pool associated with a specific system can satisfy nonspecific mount requests for that system. DFSMSrmm rejects volumes from other pools on that system.

When you specify a SYSID, and you have also activated MNTMSG, all nonspecific mount messages are updated to include the pool prefix to identify the pool to be used to the operator.

You can use the EDG_EXIT100 installation exit to assign multiple scratch pools to be used for nonspecific tape volume requests for each system.

If there is no scratch pool defined for the current system, and the EDG_EXIT100 exit does not select a specific scratch pool, DFSMSrmm does not update the mount message. DFSMSrmm selects the first pool in the collating sequence with the correct SYSID to update mount messages. However, the operator can mount a volume from any eligible pool. DFSMSrmm ignores the SYSID value when you use ACS routines to select a storage group pool for a nonspecific tape request.

Default: The pool is not associated with any particular system.

TYPE(S|R)

Specifies the type of pool. In DFSMSrmm, there are two categories of pools: rack and scratch. Specify *R* for a rack pool, and *S* for a scratch pool.

A *rack pool* is shelf space that can be assigned to hold any volumes that are generally read-only and that enter and leave your installation on an ad hoc basis. These volumes are typically software product volumes and customer volumes and do not adhere to your installation's naming conventions. Although you can add scratch volumes to rack pools, you cannot normally use these volumes to satisfy nonspecific mount requests unless EDGUX100 selects a pool or a storage group scratch pool is selected.

A *scratch pool* is shelf space assigned to hold volumes for use with DFSMSrmm system based scratch pooling. The volumes assigned to this shelf space can be used to satisfy scratch requests as long as the volumes are in scratch status. Once the volume has been written to, it becomes a volume with MASTER status until the data is no longer required by the installation. The volume remains in the same DFSMSrmm scratch pool in that it occupies the same shelf space regardless of status.

Default: TYPE(S)

Chapter 9. Authorizing DFSMSrmm users and ensuring security

To protect DFSMSrmm functions, you need to use an external security product, such as RACF. Invocations to the product are made by DFSMSrmm through the System Authorization Facility (SAF). If RACF is not installed, you must provide equivalent function with the SAF interface described in [“Using the SAF interface” on page 241](#).

This topic explains how you can define RACF profiles to protect DFSMSrmm functions. Define RACF profiles and authorize DFSMSrmm users to various levels of access, either CONTROL, READ, ALTER, or UPDATE. These access levels vary depending on the security requirements of your installation. DFSMSrmm supports a role-based authorization model where you can allow a storage administrator to perform all tasks, or you can delegate regular tasks to others. See [“Setting the level of access for the DFSMSrmm resources” on page 219](#) for additional details.

In addition, [“Controlling RACF tape profile processing” on page 233](#) describes how to use the DFSMSrmm parmlib OPTION TPRACF command and VLPOOL command and your installation's RACF options to control the actions that DFSMSrmm takes on RACF tape profiles.

This topic also explains how to ensure security:

- [“Creating audit trails” on page 232](#)
- [“Using security classification processing” on page 232](#)
- [“Preventing the use of IEHINITT” on page 233](#)

Protecting DFSMSrmm resources with RACF profiles

The DFSMSrmm resources you protect with RACF profiles in the FACILITY class each have an entity name prefixed with STGADMIN.EDG. [Table 19 on page 215](#) lists the DFSMSrmm resources.

For optimal security, define RACF profiles to control access to DFSMSrmm functions that are protected by the DFSMSrmm resource. If there is a RACF profile, specific or generic, that matches a DFSMSrmm resource, the resource is treated as if it has been defined. DFSMSrmm provides control for some resources, as described in [“Setting the level of access for the DFSMSrmm resources” on page 219](#), even when you do not protect DFSMSrmm resources with RACF or another equivalent security product.

The TSO commands and utilities are authorized by DFSMSrmm on the system on which you run them.

Table 19. Resources you protect with RACF profiles

Define the Profile	To Control the
STGADMIN.EDG.ACTIONS.action ¹	Setting of the release action.
STGADMIN.EDG.AV.status.volser ²	Adding of volumes.
STGADMIN.EDG.CD.COPYFROM.dsname ⁴	Use of CHANGEDATASET COPYFROM subcommand to copy the data set attributes from one data set dsname to another data set.
STGADMIN.EDG.CD.VX ⁴	Overriding of DFSMSrmm VRSEL processing for a data set.
STGADMIN.EDG.CMOVE.location.destination	Confirmation of moves and ejects.
STGADMIN.EDG.CRLSE.action ¹	Confirmation of the release action.
STGADMIN.EDG.CV.[HOLD NOHOLD].volser ³	Setting and resetting the volume HOLD attribute

Table 19. Resources you protect with RACF profiles (continued)

Define the Profile	To Control the
STGADMIN.EDG.CV.RM ³	Use of the RMM CHANGEVOLUME RETENTIONMETHOD subcommand to update the retention method for a volume. Use of the RMM CHANGEVOLUME RETAINBY subcommand to update the retain by attribute of a volume managed by the EXPDT retention method.
STGADMIN.EDG.DV.SCRATCH.volser	Deleting of scratch volumes.
STGADMIN.EDG.FORCE	Changing of information recorded by DFSMSrmm during O/C/EOV processing. Adding or deleting data sets on volumes or to use the DELETEVOLUME command.
STGADMIN.EDG.FORCEEXPIRE	Ability to search for data sets that were blocked using the CHANGEDATASET FORCEEXPIRE DFSMSrmm subcommand. Ability to undo FORCEEXPIRE using the CHANGEDATASET REMOVEEXPIRE DFSMSrmm subcommand - however it is also possible to use REMOVEEXPIRE without access to STGADMIN.EDG.FORCEEXPIRE if the FORCE option is specified.
STGADMIN.EDG.EDGUPDT.UPDATE	Use of the EDGUPDT utility UPDATE function.
STGADMIN.EDG.HOUSEKEEP	Use of DFSMSrmm inventory management functions.
STGADMIN.EDG.HOUSEKEEP.RPTEXT	Use of DFSMSrmm inventory management extract function
STGADMIN.EDG.IGNORE.TAPE.volser	Use of volume serial numbers that are not defined to DFSMSrmm and use of duplicate volume serial numbers to allow a volume to be ignored. If you are authorized to ignore use of a tape volume, DFSMSrmm also overrides the SAF authorization (Unless CHECKSAFRC8 is specified in OPENRULE parmlib or PL100_SET_NOTBYPASS_SAFRC8 flag is enabled in EDGUX100) for you to access data on the tape when the data is not defined to RACF and when the user is not authorized to the data. Recommendation: Do not assign an access level to the STGADMIN.EDG.IGNORE.TAPE.volser resource to any specific user group. When a tape volume that must be ignored by DFSMSrmm is identified, grant the user or user group the needed access level. Once the volume is no longer needed, delete the resource.

Table 19. Resources you protect with RACF profiles (continued)

Define the Profile	To Control the
STGADMIN.EDG.IGNORE.TAPE.RMM.volser	Use of duplicate volume serial numbers and to allow a volume to be ignored. If you are authorized to ignore use of a tape volume, DFSMSrmm also overrides the SAF authorization (Unless CHECKSAFRC8 is specified in OPENRULE parmlib or PL100_SET_NOTBYPASS_SAFRC8 FLAG is enabled in EDGUX100) for you to access data on the tape when the data is not defined to RACF and when the user is not authorized to the data. Recommendation: Specify UACC(NONE) to the STGADMIN.EDG.IGNORE.TAPE.RMM.volser resource. Grant a user or user group the needed access level only when access is needed. When the volume is no longer needed, delete the resource.
STGADMIN.EDG.IGNORE.TAPE.NORMM.volser	Use of volume serial numbers that are not defined to DFSMSrmm to allow a volume to be ignored. If you are authorized to ignore use of a tape volume, DFSMSrmm also overrides the SAF authorization (Unless CHECKSAFRC8 is specified in OPENRULE parmlib or PL100_SET_NOTBYPASS_SAFRC8 FLAG is enabled in EDGUX100) for you to access data on the tape when the data is not defined to RACF and when the user is not authorized to the data. Recommendation: Specify UACC(NONE) to the STGADMIN.EDG.IGNORE.TAPE.NORMM.volser resource. Grant the needed access level to a user or user group only when access is needed. When the volume is no longer needed, delete the resource.
STGADMIN.EDG.INIT	Setting of the INIT action.
STGADMIN.EDG.LABEL.volser	Creation of standard tape labels. The variable <i>volser</i> can be specified as a specific volume serial number or a generic volume serial number. For example, A12345 is a specific volume serial number and AB* is a generic volume serial number. If you use generic profiles you can use these functions in a subset of your volumes. If the volume serial numbers and rack numbers match, you can control relabeling at the pool level. For example you could have a pool using rack number prefix AB*. If you want to create an AL tape and your installation has an SL scratch pool, you need ALTER access to STGADMIN.EDG.LABEL.volser. The <i>volser</i> can be specified as the pool prefix of the scratch pool. If you want to switch to an AL tape from either an SL or NL tape that has already been assigned to you, UPDATE access to STGADMIN.EDG.LABEL.volser is required.

Table 19. Resources you protect with RACF profiles (continued)

Define the Profile	To Control the
STGADMIN.EDG.LIST	List and search DFSMSrmm resources.
STGADMIN.EDG.LISTCONTROL	Use of the RMM LISTCONTROL subcommand to display DFSMSrmm control data set control record information and EDGRMMxx parmlib settings.
STGADMIN.EDG.MASTER	Access to information in the DFSMSrmm control data set. Assign the control data set a universal access of NONE so that DFSMSrmm grants access to various functions through STGADMIN.EDG.MASTER.
STGADMIN.EDG.MOVES. <i>location.destination</i>	Initiation of moves and ejects.
STGADMIN.EDG.NOLABEL. <i>volser</i>	Creation of tapes without labels.
STGADMIN.EDG.OPERATOR	Use of the initialize, erase, and scan functions.
STGADMIN.EDG.OWNER. <i>userid</i>	Access to owned resources. DFSMSrmm checks this entity only if the command issuer is not the owner of the resource and does not have CONTROL access to STGADMIN.EDG.MASTER. Use of the RMM CHANGEVOLUME subcommand to update information based on the owner. Using STGADMIN.EDG.OWNER.<i>userid</i>, individual owners can permit other users to access owned volumes. An owner can be a group or department as well as an individual. Define owner resources only for those owners who will allow their volumes to be managed by another user.
STGADMIN.EDG.RELEASE	Use of the RMM DELETEVOLUME RELEASE subcommand to process any release actions specified for a volume.
STGADMIN.EDG.RESET.SSI	Use of the RESET facility for removing DFSMSrmm from the system. You can use the facility without defining this resource when you have no security product installed.
STGADMIN.EDG.VRS	Use of the RMM LISTVRS and SEARCHVRS subcommands to obtain information about vital record specifications. Use of the RMM ADDVRS and DELETEVRS subcommands to define or remove vital record specifications.
STGADMIN.EDG.INERS.WRONGLABEL	Processing for volumes mounted with the wrong label.

Note:

1. Action can be either SCRATCH, RETURN, REPLACE, NOTIFY, ERASE, or INIT.
2. Status can be either SCRATCH, USER, MASTER, or VOLCAT.
3. If you use a generic profile, the minimum non-generic profile name checked for by DFSMSrmm is 'STGADMIN.EDG.CV.'
4. If you use a generic profile, the minimum non-generic profile name checked for by DFSMSrmm is 'STGADMIN.EDG.CD.'

Creating profiles

To protect an DFSMSrmm resource, you can create a RACF profile as shown in [Figure 83 on page 219](#).

```
RDEFINE FACILITY STGADMIN.EDG.OWNER.HSMATH0 UAC(ALTER)
```

Figure 83. Creating a RACF profile

For the most complete records, create profiles with auditing options that are set to record attempted activities. For example, to ensure that RACF logs all successful and failed attempts to change vital record specifications, create profiles as shown in [Figure 84 on page 219](#).

```
RDEFINE FACILITY STGADMIN.EDG.VRS UACC(NONE)
RALT FACILITY STGADMIN.EDG.VRS GLOBALAUDIT(ALL(UPDATE))
```

Figure 84. Creating a RACF profile with audit options

Setting the level of access for the DFSMSrmm resources

When you define the DFSMSrmm resources, you need to authorize levels of access to these resources. DFSMSrmm checks the resource and the level of access to ensure that users are authorized to request certain tasks. For example, if you attempt to change an owned volume, DFSMSrmm checks to ensure that you have at least UPDATE access to resource STGADMIN.EDG.MASTER.

When checking authorization to use RMM subcommands and operands, DFSMSrmm checks in this sequence:

1. CONTROL access to STGADMIN.EDG.MASTER. If the user is authorized, no further checking is performed.
2. Next, DFSMSrmm checks for specific subcommand operands and for each operand that requires specific authorization checks for the required access. If the resource is not protected, authorization continues with the next step. If the resource is protected, but the user is not authorized, the subcommand fails.
3. Finally, DFSMSrmm continues with ownership checks and RELEASE and FORCE checking, if required.

Because of the way authorization is checked, it is not necessary to have CONTROL access to STGADMIN.EDG.MASTER to perform many of the regular administrative tasks.

[Table 20 on page 219](#) shows the access that is required to perform DFSMSrmm functions.

Table 20. Authorized functions		
When You Define	With Access	Then
STGADMIN.EDG.ACTIONS.action ^{1,5}	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to set the specific release action with the TSO DFSMSrmm subcommand CHANGEVOLUME with option RELEASEACTION.
STGADMIN.EDG.AV.status.volser ⁶	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to add volumes to the DFSMSrmm tape inventory with the TSO DFSMSrmm subcommand ADDVOLUME with option STATUS(status).

Table 20. Authorized functions (continued)

When You Define	With Access	Then
STGADMIN.EDG.CD.COPYFROM.dsname	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	READ	You are permitted to copy attributes and update retention for identically named data sets.
	UPDATE	You are permitted to copy attributes and update retention for any two data set records.
STGADMIN.EDG.CD.VX	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	Allows any data set to be updated.
STGADMIN.EDG.CMOVE.location.destination	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to confirm that the move or eject has occurred for either a single volume or globally ³ with the TSO DFSMSrmm subcommand CHANGEVOLUME with option CONFIRMMOVE, as well as to reverse a previous move confirmation with option NOCONFIRMMOVE.
STGADMIN.EDG.CRLSE.action ^{1,5}	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to confirm that the specific release action has been performed either for a single volume or globally with the TSO DFSMSrmm subcommand CHANGEVOLUME with option CONFIRMRELEASE, as well as to reverse a previous release action confirmation with option NOCONFIRMRELEASE. In addition, it enables the DELETEVOLUME REPLACE subcommand to be specified for a volume waiting to be replaced.
STGADMIN.EDG.CV.[HOLD NOHOLD].volser	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are permitted to set and reset the volume HOLD attribute.
STGADMIN.EDG.CV.RM	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	Allows any volume to be updated.
STGADMIN.EDG.DV.SCRATCH.volser	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to remove scratch volumes from the DFSMSrmm tape inventory with the TSO DFSMSrmm subcommand DELETEVOLUME with option REMOVE.
STGADMIN.EDG. EDGUPDT.UPDATE	Entity not defined	Same as UPDATE access.
	NONE	No authority is granted to use EDGUPDT UPDATE function.
	UPDATE	You can use the EDGUPDT UPDATE function.

Table 20. Authorized functions (continued)		
When You Define	With Access	Then
STGADMIN.EDG.FORCE	Entity not defined	Information previously recorded by DFSMSrmm cannot be changed.
	UPDATE	Information previously recorded by DFSMSrmm can be changed based on access to STGADMIN.EDG.MASTER.
STGADMIN.EDG.FORCEEXPIRE	Entity not defined	Not authorized to issue RMM SEARCHDATASET FORCEEXPIRE and RMM SEARCHDATASET NOFORCEEXPIRE. Not authorized to issue RMM CHANGEDATASET REMOVEEXPIRE without the FORCE parameter.
	UPDATE	The following RMM subcommands can be issued based on access to STGADMIN.EDG.MASTER: RMM SEARCHDATASET FORCEEXPIRE RMM SEARCHDATASET NOFORCEEXPIRE RMM CHANGEDATASET REMOVEEXPIRE (without the FORCE parameter specified).
STGADMIN.EDG.HOUSEKEEP	Entity not defined	Same as READ access
	READ	Any of the inventory management facilities can be invoked.
STGADMIN.EDG.HOUSEKEEP.RPTEXT	Entity not defined	Same as READ access
	READ	RPTEXT inventory management function can be invoked.
STGADMIN.EDG.IGNORE.TAPE.volser	Entity not defined	Volumes cannot be ignored using the DFSMSrmm installation exit.
	READ	Volumes that are to be ignored by DFSMSrmm for input requests are allowed to be opened.
	UPDATE	Volumes that are to be ignored by DFSMSrmm for output requests are allowed to be opened.
STGADMIN.EDG.IGNORE.TAPE.RMM.volser	Entity not defined	Access is based on the STGADMIN.EDG.IGNORE.TAPE.volser setting. Use of the STGADMIN.EDG.IGNORE.TAPE.RMM.volser profile allows volumes that are defined to DFSMSrmm to be ignored.
	READ	Volumes that are defined to DFSMSrmm can be ignored by DFSMSrmm for input requests are allowed to be opened.
	UPDATE	Volumes that are defined to DFSMSrmm can be ignored by DFSMSrmm for output requests are allowed to be opened.

Table 20. Authorized functions (continued)

When You Define	With Access	Then
STGADMIN.EDG.IGNORE.TAPE.NORMM.volser	Entity not defined	Access is based on the STGADMIN.EDG.IGNORE.TAPE.volser setting. Use of the STGADMIN.EDG.IGNORE.TAPE.NORMM.volser profile allows volumes that are not defined to DFSMSrmm to be ignored.
	READ	Volumes not defined to DFSMSrmm that are to be ignored for input requests are allowed to be opened.
	UPDATE	Volumes not defined to DFSMSrmm that are to be ignored for output requests are allowed to be opened.
STGADMIN.EDG.INIT	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to specify whether a volume should be initialized or not. You can specify INITIALIZE(YES) to indicate that a volume requires initialization, and INITIALIZE(NO) to indicate that the volume does not need to be initialized.
STGADMIN.EDG.LABEL.volser	Entity not defined	A volume must be in user status to switch from NL or to change to label types at OPEN time.
	UPDATE	Standard labels can be created on a non-scratch volume for an AL or SL output request.
	ALTER	Standard labels can be created on a scratch volume during a nonspecific volume request for AL or SL.
STGADMIN.EDG.LIST	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	CONTROL	You are allowed to list and search resources defined in the DFSMSrmm inventory. This option can be used to replace CONTROL access to STGADMIN.EDG.MASTER in a name-hiding environment or when COMMANDAUTH(DSN) is in use.
STGADMIN.EDG.LISTCONTROL	Entity not defined	The nearest specific profile definition. ⁸
	CONTROL	You are allowed to use the RMM LISTCONTROL subcommand.

Table 20. Authorized functions (continued)

When You Define	With Access	Then
STGADMIN.EDG.MASTER	Entity not defined	Same as CONTROL access.
	READ	These functions can be performed: - List all control data set information except vital record specifications and control information - Search for all control data set information except vital record specifications - Update your own owner ID details - Request a scratch volume for yourself - Release an owned volume when the STGADMIN.EDG.RELEASE resource is not protected
	UPDATE	Same as READ access, plus: Some non-restricted fields can be updated for owned volumes and data sets based on user ID. See z/OS DFSMSrmm Managing and Using Removable Media , RMM CHANGEVOLUME subcommand information, for a list of the non-restricted fields. See “Using RACF options for authorizing RMM TSO subcommands” on page 241 for information about changing information using DFSMSrmm command authorization by data set name.
	CONTROL	Same as UPDATE access, plus: You can - Define, change, and delete any control data set entries except vital record specifications - List control information when the STGADMIN.EDG.LISTCONTROL resource is not protected
STGADMIN.EDG.MOVES. <i>location.destination</i>	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	UPDATE	You are allowed to initiate the move with the TSO DFSMSrmm subcommand CHANGEVOLUME with either option LOC(<i>destination</i>) ⁴ or LOANLOC(<i>destination</i>) ^{2,4} , as well as to initiate the eject of a volume to a previously specified destination with option EJECT ^{4,7} .
STGADMIN.EDG.NOLABEL. <i>volser</i>	Entity not defined	A volume must be in user status to switch to NL at OPEN time.
	UPDATE	You are allowed to destroy labels on a non-scratch volume for a no label output request.
	ALTER	You are allowed to destroy labels on a scratch volume during a nonspecific volume request for no labels.

Table 20. Authorized functions (continued)

When You Define	With Access	Then
STGADMIN.EDG.OPERATOR	Entity not defined	Same as UPDATE access.
	NONE	No authority is granted to use EDGINERS to initialize, scan, and erase volumes.
	READ	EDGINERS can be used to scan volumes.
	UPDATE	Same as READ access, plus EDGINERS can be used to initialize and erase volumes.
STGADMIN.EDG.OWNER.userid	Entity not defined	No authority is granted to update volume information except based on STGADMIN.EDG.MASTER access.
	NONE	Based on STGADMIN.EDG.MASTER access.
	UPDATE	Some non-restricted fields for volumes and data sets owned by <i>userid</i> can be updated. Also a volume owned by <i>userid</i> can be released. See <i>z/OS DFSMSrmm Managing and Using Removable Media</i> , RMM CHANGEVOLUME subcommand information, for a list of the non-restricted fields. See “Using RACF options for authorizing RMM TSO subcommands” on page 241 for information about changing information using DFSMSrmm command authorization by data set name.
STGADMIN.EDG.RELEASE	Entity not defined	Based on STGADMIN.EDG.MASTER access.
	READ	You are allowed to use the RMM DELETEVOLUME RELEASE subcommand to release an owned volume.
STGADMIN.EDG.RESET.SSI	Entity not defined	You cannot use the EDGRESET utility to remove DFSMSrmm from the system. If you have no security product installed, you can use EDGRESET to remove DFSMSrmm from the system.
	ALTER	You are allowed to withdraw DFSMSrmm from the system during error recovery or problem during implementation.
STGADMIN.EDG.VRS	Entity not defined	The nearest specific profile definition. ⁸
	READ	You are allowed to list and search for all vital record specifications.
	CONTROL	Same as READ access, plus: You can define and delete vital record specifications.
STGADMIN.EDG.INERS.WRONGLABEL	Entity not defined	Use of the EDGINERS EXEC statement PARM IGNORE and RMMPROMPT parameters is denied.
	UPDATE	You are allowed to use the EDGINERS EXEC statement PARM RMMPROMPT parameter.
	CONTROL	You are allowed to use the EDGINERS EXEC statement PARM IGNORE parameter.

Table 20. Authorized functions (continued)

When You Define	With Access	Then
Note: - Action can be either SCRATCH, RETURN, REPLACE, NOTIFY, ERASE, or INIT. - To set a loan, the entity STGADMIN.EDG.MOVES.<i>current location.loan location</i> is used. - To confirm a global move with the TSO DFSMSrmm subcommand CV CMOVE (ALL,ALL), the RACF entity STGADMIN.EDG.CMOVE.ALL.ALL is checked. - When the destination is not set or blank, for example, when you issue the CHANGEVOLUME command with either the operand LOCATION or LOANLOC with a blank location, or when you eject a volume that has no destination set, the entity STGADMIN.EDG.MOVES.<i>locationA.locationA</i> is used. <i>locationA</i> is the current location of the volume. - To grant access to a list of actions, for example, when you issue the CHANGEVOLUME subcommand CV CRLSE(INIT,NOTIFY,ERASE), every single action resource is checked, and access is granted only if all single actions are granted. - Status can be either SCRATCH, USER, MASTER, or VOLCAT. - For an EJECT, the same entity is checked that is used to check if the user is allowed to start the move. - The "nearest specific profile definition" relates to wild carded profiles. For Example: Assume - STGADMIN.** with ACCESS(ALTER) - STGADMIN.EDG.** with ACCESS(CONTROL) - STGADMIN. ** with ACCESS(NONE) - STGADMIN.EDG.** with ACCESS(READ) If A & B are defined on a system, B is a more specific profile that will be applied to the noted RACF profile, granting ACCESS(CONTROL) to the resources. If C & D are defined on a system, D is a more specific profile that will be applied to the noted RACF profile, granting ACCESS(NONE) to the resources. It is recommended for the noted profiles to define them explicitly with the ACCESS level desired.		

Authorizing resources

Table 21 on page 225 and Table 22 on page 226 provide suggestions for authorizing different types of users. Implementing these suggestions depends on your organization's structure, administrative procedures, and security requirements. For storage administrator and tape librarian roles, DFSMSrmm provides you the choice of whether they are authorized to use almost any DFSMSrmm subcommand, or whether you limit the scope of regular administrative tasks they can perform. For example, you might want a particular librarian to only confirm moves and actions, and another librarian to only be able to remove scratch volumes and volumes waiting to be replaced. There are two tables. Table 21 on page 225 shows how you can set up the administrative roles to limit the scope of tasks that can be performed and shows a storage administrator with authority to all tasks, and a librarian limited to certain tasks. Table 22 on page 226 shows how different users might be authorized without consideration to limiting administrative tasks.

Table 21. Suggested resource access to limit scope of tasks

Resource	General User	Storage Administrator	System Programmer	Librarian	Inventory Management Functions	Operator
STGADMIN.EDG.ACTIONS. <i>action</i>	-	-	U	U	-	-
STGADMIN.EDG.AV. <i>status.volser</i>	-	-	-	U	-	-

Table 21. Suggested resource access to limit scope of tasks (continued)

Resource	General User	Storage Administrator	System Programmer	Librarian	Inventory Management Functions	Operator
STGADMIN.EDG.CMOVE. <i>location.destination</i>	-	-	-	U	U	U
STGADMIN.EDG.CV.COPYFROM	-	-	U	U	-	-
STGADMIN.EDG.CV.HOLD	-	-	U	U	-	-
STGADMIN.EDG.CV.RM	-	-	U	U	-	-
STGADMIN.EDG.CV.VX	-	-	U	U	-	-
STGADMIN.EDG.CRLSE. <i>action</i>	-	-	-	U	U	U
STGADMIN.EDG.DV.SCRATCH. <i>volser</i>	-	-	-	U	-	-
STGADMIN.EDG.INIT	-	-	U	U	-	-
STGADMIN.EDG.LIST	-	C ¹	-	C ¹	-	-
STGADMIN.EDG.MASTER	R	C	U	U	U	R
STGADMIN.EDG.MOVES. <i>location.destination</i>	-	-	U	U	-	-

Access: R—Read, U—Update, C—Control, A—Alter

Note: Access to STGADMIN.EDG.LIST should be granted only in a RACF name-hiding environment, or when OPTION COMMANDAUTH(DSN) is in effect, and only if the user has no CONTROL access to STGADMIN.EDG.MASTER.

Table 22. Suggested resource access without limited tasks

Resource	General User	Storage Administrator	System Programmer	Librarian	Inventory Management Functions	Operator
STGADMIN.EDG.FORCE	-	U	-	U	-	-
STGADMIN.EDG.FORCEEXPIRE						

Table 22. Suggested resource access without limited tasks (continued)

Resource	General User	Storage Administrator	System Programmer	Librarian	Inventory Management Functions	Operator
	-	-	-	-	-	-
STGADMIN.EDG.HOUSEKEEP						
	-	-	-	-	R	-
STGADMIN.EDG.HOUSEKEEP.RPTEXT						
	-	R	R	R	R	R
STGADMIN.EDG.IGNORE.TAPE.volser						
	-	-	-	-	-	-
STGADMIN.EDG.IGNORE.TAPE.RMM.volser						
	-	-	-	-	-	-
STGADMIN.EDG.IGNORE.TAPE.NORMM.volser						
	-	-	-	-	-	-
STGADMIN.EDG.LABEL.volser						
	U	A	A	A	A	A
STGADMIN.EDG.LISTCONTROL						
	C	C	C	C	-	C
STGADMIN.EDG.MASTER						
	R	C	C	C	-	C
STGADMIN.EDG.NOLABEL.volser						
	U	A	A	A	A	A
STGADMIN.EDG.OPERATOR						
	-	-	U	U	-	U
STGADMIN.EDG.OWNER.userid						
	-	U	-	-	-	-
STGADMIN.EDG.RELEASE						
	R	-	-	-	-	-
STGADMIN.EDG.RESET.SSI						
	-	-	-	-	-	A
STGADMIN.EDG.VRS						
	R	C	C	C	-	-
STGADMIN.EDG.INERS.WRONGLABEL						
	-	-	-	U	-	-
Access: R—Read, U—Update, C—Control, A—Alter						

These topics describe the functions available to DFSMSrmm users that are based on the recommendations in [Table 21 on page 225](#).

General user functions

[Table 23 on page 228](#) describes general user functions.

Table 23. General user functions

With	The General User Can
UPDATE access to STGADMIN.EDG.LABEL.volser	Create standard labels on a non-scratch volume for an AL or SL output request.
CONTROL access to STGADMIN.EDG.LISTCONTROL	Display control information and installation options and rules.
READ access to STGADMIN.EDG.MASTER	Display all control data set details except vital record specifications and control information. Search for data sets, software products, shelf locations, and volumes. Update their owner ID details. Request a scratch volume.
UPDATE access to STGADMIN.EDG.NOLABEL.volser	Erase labels on a non-scratch volume for a no label output request.
READ access to STGADMIN.EDG.RELEASE	Release volumes they own.
READ access to STGADMIN.EDG.VRS	Search for any vital record specification and display the details that DFSMSrmm records.

Storage administrator functions

[Table 24 on page 228](#) describes storage administrator functions.

Table 24. Storage administrator functions

With	The Storage Administrator Can
ALTER access to STGADMIN.EDG.LABEL.volser	Create standard labels on a scratch volume during a non-specific volume request for an AL or SL volume.
CONTROL access to STGADMIN.EDG.LISTCONTROL	Display control information and installation options and rules.
CONTROL access to STGADMIN.EDG.MASTER	Display all control data set details. Request scratch volumes. Release volumes. Update volume, data set, owner, and software product details. Add and delete information about volumes, data sets, shelf locations, owner IDs, and software products.
ALTER access to STGADMIN.EDG.NOLABEL.volser	Erase labels on a scratch volume during a non-specific volume request for a no label volume.

Table 24. Storage administrator functions (continued)

With	The Storage Administrator Can
UPDATE access to STGADMIN.EDG.OWNER.userid.	Update details that DFSMSrmm records for volumes that <i>userid</i> owns and release volumes that <i>userid</i> owns. The storage administrator might use this level of authorization rather than CONTROL access to STGADMIN.EDG.MASTER for administrators responsible for specific production applications.
CONTROL access to STGADMIN.EDG.VRS	Create and delete vital record specifications.
READ access to STGADMIN.EDG.HOUSEKEEP.RPTEXT	Can obtain information from the control data set that can be used as input for creating reports.

System programmer functions

Table 25 on page 229 describes system programmer functions.

Table 25. System programmer functions

With	The System Programmer Can
ALTER access to STGADMIN.EDG.LABEL.volser	Create standard labels on a scratch volume during a non-specific volume request for an AL or SL volume.
CONTROL access to STGADMIN.EDG.LISTCONTROL	Display control information and installation options and rules
CONTROL access to STGADMIN.EDG.MASTER	Display all control data set details. Request scratch volumes. Release volumes. Update volume, data set, owner, and software product details. Add and delete information about volumes, data sets, shelf locations, owner IDs, and software products.
ALTER access to STGADMIN.EDG.NOLABEL.volser	Erase labels on a scratch volume during a non-specific volume request for a no label volume.
UPDATE access to STGADMIN.EDG.OPERATOR	Can initialize and erase volumes.
CONTROL access to STGADMIN.EDG.VRS	Create, display, and delete vital record specifications.
READ access to STGADMIN.EDG.HOUSEKEEP.RPTEXT	Can obtain information from the control data set that can be used as input for creating reports.

Librarian functions

Table 26 on page 229 describes librarian functions.

Table 26. Librarian functions

With	The Librarian Can
ALTER access to STGADMIN.EDG.LABEL.volser	Create standard labels on a scratch volume during a non-specific volume request for an AL or SL volume.
CONTROL access to STGADMIN.EDG.LISTCONTROL	Display control information and installation options and rules.

Table 26. Librarian functions (continued)

With	The Librarian Can
CONTROL access to STGADMIN.EDG.MASTER	Display all control data set details. Request scratch volumes. Release volumes. Update volume, data set, owner, and software product details. Add and delete information about volumes, data sets, shelf locations, owner IDs, and software products.
ALTER access to STGADMIN.EDG.NOLABEL.volser	Erase labels on a scratch volume during a non-specific volume request for a no label volume.
UPDATE access to STGADMIN.EDG.OPERATOR	Initialize and erase volumes.
CONTROL access to STGADMIN.EDG.VRS	Search, display, add, and delete vital record specifications.
READ access to STGADMIN.EDG.HOUSEKEEP.RPTEXT	Can obtain information from the control data set that can be used as input for creating reports.

Inventory management functions

[Table 27 on page 230](#) describes inventory management functions.

Table 27. Inventory management functions

With	The Person Performing Inventory Management Can
ALTER access to STGADMIN.EDG.LABEL.volser	Create standard labels on a scratch volume during a non-specific volume request for an AL or SL volume.
READ access to STGADMIN.EDG.HOUSEKEEP	Invoke the DFSMSrmm inventory management facilities to perform these functions: • Create an extract of the control data set. • Select volumes to be retained and moved as vital records or for disaster recovery. • Perform expiration processing. • Perform storage location management processing. • Back up the control data set.
UPDATE access to STGADMIN.EDG.EDGUPDT.UPDATE	Update the control data sets with journal processed at test or recovery site.
ALTER access to STGADMIN.EDG.NOLABEL.volser	Erase labels on a scratch volume during a non-specific volume request for a no label volume.
READ access to STGADMIN.EDG.HOUSEKEEP.RPTEXT	Obtain information from the control data set that can be used as input for creating reports.

Operator functions

[Table 28 on page 231](#) describes operator functions.

Table 28. Operator functions

With	The Operator Can
ALTER access to STGADMIN.EDG.LABEL.volser	Create standard labels on a scratch volume during a non-specific volume request for an AL or SL volume.
CONTROL access to STGADMIN.EDG.LISTCONTROL	Display control information and installation options and rules
CONTROL access to STGADMIN.EDG.MASTER	Display all control data set details. Request scratch volumes. Release volumes. Update volume, data set, owner, and software product details. Add and delete information about volumes, data sets, shelf locations, owner IDs, and software products.
ALTER access to STGADMIN.EDG.NOLABEL.volser	Erase labels on a scratch volume during a non-specific volume request for a no label volume.
UPDATE access to STGADMIN.EDG.OPERATOR	Initialize and erase volumes.
ALTER access to STGADMIN.EDG.RESET.SSI	Remove DFSMSrmm from the system.
READ access to STGADMIN.EDG.HOUSEKEEP.RPTEXT	Can obtain information from the control data set that can be used as input for creating reports.

Normally, the operator performs these functions by using prepared procedures or batch jobs. In these cases, the user ID performing the procedure requires the relevant access; not the operator's user ID. For example, the EDGRESET function is invoked by starting the DFSMSrmm procedure with OPT=RESET. The user ID that the DFSMSrmm procedure uses needs the access to STGADMIN.EDG.RESET.SSI.

DFSMSrmm supports the issuing of the RMM TSO subcommands from an operator console. The operator must be authorized for each command issued and must be logged onto the operator console using their user ID with the required subcommand authority level.

Using the tape relabeling resources

DFSMSrmm supports three basic label types: NL, SL, and AL. You can switch between any of these basic types by specifying the new label type in your JCL when creating the first file on a volume.

Using EDGINERS to relabel tape volumes requires a separate mount of the volume and must be performed prior to the use of the volume.

Changing volume labels at OPEN time when creating the first file does not require EDGINERS or the INIT release action. Tape volume labels can be created or destroyed at any time regardless of volume status when the user has the required access to a security resource. Scratch volumes can only be used for nonspecific output requests.

To enable this processing, use the STGADMIN.EDG.LABEL.volser and STGADMIN.EDG.NOLABEL.volser profiles in the FACILITY class. These profiles are described in Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215. If you do not create the appropriate security profiles in FACILITY class, the volume must be in user status.

When DFSMSrmm allows the tape labels to be created or destroyed, it automatically replies to the operator WTOR to provide the required information. In addition, for any volume with the return to scratch release action, which has a NL label type at release time, DFSMSrmm sets the INIT release action to ensure that only standard label tapes are returned to the scratch pool.

Your installation can bypass this processing for any volume by using the EDG_EXIT100 installation exit to request that DFSMSrmm ignore the volume. EDG_EXIT100 is called before any tape label conflicts are resolved. See [“Using the DFSMSrmm EDG_EXIT100 installation exit”](#) on page 267.

Creating audit trails

There are several ways you can create audit information with DFSMSrmm. You can use these sources:

- Control data set information
- SMF audit records
- RACF audit information

See [z/OS DFSMSrmm Reporting](#) for information about creating reports that use DFSMSrmm control data set information and SMF audit records as input.

Control data set information

Use control data set information as input to inventory reports and movement reports to keep track of volumes in your removable media library.

You can also obtain control data set information by using RMM TSO subcommands and the DFSMSrmm ISPF dialog. Here is some of the information available:

- User ID, system, date, and time stamp of the last update before the audit record was created.
- Create date, time, user ID, and system.
- Storage location data and bin numbers.
- Original expiration date and the current expiration date. The original expiration date is the expiration date coded in the JCL when the data was originally written to the volume.
- Current location of a volume.
- Release actions for a volume.

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information on using the RMM TSO subcommands.

SMF audit information

Using the DFSMSrmm audit facility, you can record in SMF records all the updates your users make to the control data set. To specify that you want auditing, use the SMFAUD installation parameter in the EDGRMMxx parmlib member. You can collect records through your installation's normal SMF processing and use them as input to the DFSMSrmm program EDGAUD. An SMF audit record can capture any change to the DFSMSrmm inventory.

DFSMSrmm provides several mapping macros for DFSMSrmm SMF audit and security records. See [z/OS DFSMSrmm Reporting](#) for the layout of these macros. See [“Updating SMFPRMxx \(optional\)”](#) on page 27 for information on how identify records to be collected. See [“Defining security classes: SECCLS”](#) on page 206 for information on defining which SMF record numbers to use.

RACF audit information

Using standard RACF facilities, you can keep an audit trail of any command and authorization violations that occur in RMM TSO subcommands or utilities. Use RACF AUDIT and GLOBALAUDIT options to get RACF to create this information.

Using security classification processing

Certain classifications of data might require special processing, or operator action, or both. You can control these actions by using DFSMSrmm security classifications, defined with the SECCLS command in the EDGRMMxx parmlib member. Use this command to specify that DFSMSrmm:

- Issues messages when certain data sets are opened so the operator can take any required installation action before confirming the use of the volume
- Creates a special SMF record for each access to secure volumes
- Automatically flags these volumes to be erased when they are released, before they can be reused

Preventing the use of IEHINITT

You no longer need to use IEHINITT, because the DFSMSrmm utility EDGINERS labels tapes and validates mounted volumes. Use EDGINERS to process new volumes, volumes where old and new labels are known to DFSMSrmm, or old volumes that have been degaussed. EDGINERS defines volumes to the DFSMSrmm control data set that have not been defined previously. For existing volumes, EDGINERS updates the volume record to show that volumes have been initialized. See [Chapter 15, “Initializing, erasing, and scanning tape volumes,” on page 423](#) for information about using EDGINERS.

To protect IEHINITT from being used or to limit usage to specific individuals, add RACF program resource profiles and create limited access lists as shown in [Figure 85 on page 233](#):

```
RDEFINE PROGRAM IEHINITT ADDMEM('SYS1.LINKLIB'//NOPADCHK) UACC(NONE)
PE IEHINITT CLASS(PROGRAM) RESET(STANDARD)
```

Figure 85. Limiting the use of IEHINITT

If program control is not active, use the RACF command SETROPTS WHEN(PROGRAM) to activate it. If program control is already active, specify the RACF command SETROPTS WHEN(PROGRAM) REFRESH to activate use of the new profile.

Controlling RACF tape profile processing

You can control the actions that DFSMSrmm takes on RACF tape profiles through the DFSMSrmm OPTION TPRACF command and your installation's RACF options. For more information on OPTION TPRACF, see [“Defining system options: OPTION” on page 161](#).

If you are running DFSMSrmm with DFSMSHsm, see [“Securing tapes when running DFSMSHsm and DFSMSrmm” on page 325](#). See [z/OS Security Server RACF System Programmer's Guide](#) for information about RACF tape security. See [z/OS MVS Initialization and Tuning Guide](#) for information about DEVSUPxx tape security.

RACF provides these tape protection options:

- No protection
- TAPEVOL class
- TAPEDSN option
- TAPEVOL and TAPEDSN

DFSMS, with DEVSUPxx options, provides additional ways for you to select how tape data sets are protected. This includes:

- TAPEAUTHDSN
- TAPEAUTHF1

When you use TAPEAUTHDSN=YES, you override the RACF settings, and the DFSMSrmm TPRACF AUTOMATIC and PREDEFINED are not likely to be beneficial to you.

Independent of the TAPEAUTHDSN setting, DFSMSrmm TPRACF processing considers just the RACF tape protection options. Using those RACF options to protect volumes, you can specify one of these actions:

No protection

If neither TAPEVOL nor TAPEDSN is active then DFSMSrmm takes no action, regardless of the setting of the TPRACF option.

TAPEVOL

TPRACF(N)—no action.

TPRACF(P)—a TAPEVOL profile is created or deleted only as a result of issuing a DFSMSrmm ADD, CHANGE, or DELETE subcommand for a private volume from a RACF-protected pool. This includes scratch tapes that are assigned by the user or librarian by using the RMM GETVOLUME subcommand.

If you require tape volume protection for volumes that are used for nonspecific tape requests, you can either:

1. Specify the JCL PROTECT=YES option, and DFSMSdftp processing protects the volume.

Or

2. Leave DFSMSrmm to protect the volume when a data set on the volume is closed, by creating a TAPEVOL profile.

DFSMSrmm automatically deletes any TAPEVOL profile for recycled scratch tapes when they return to scratch. If you use nonspecific mounts and scratch pools, no TAPEVOL profile exists while the volume is in scratch status.

TPRACF(A)—The same as TPRACF(P).

TPRACF(C)—DFSMSrmm processing is limited to cleaning up the existing TAPEVOL profile when volumes are deleted from a RACF-protected pool and when recycled scratch tapes return to scratch.

TAPEDSN

TPRACF(N)—no action.

TPRACF(P), TPRACF(C), and TPRACF(A)—The processing is the same for all options. DFSMSrmm does not cause any RACF profiles to be created at any time. During recycling or releasing of tapes, DFSMSrmm checks for discrete RACF data set profiles for data sets known to be on the volume, and automatically deletes them.

TAPEVOL and TAPEDSN

TPRACF(N)—no action.

TPRACF(A)—TAPEVOL profiles are created for identified, RACF- controlled, pools of non-scratch tapes when you use the RMM ADD, CHANGE and DELETE subcommands. No TVTOC is created in these circumstances.

DFSMSrmm assumes that correct DFSMSdftp and RACF processing, when a data set on a volume is opened, result in both TAPEVOL and discrete DATASET profiles being created. DFSMSrmm automatically deletes any such profiles for recycled scratch tapes when they return to scratch.

If a volume is unprotected when a data set on the volume is closed, DFSMSrmm automatically protects the volume with a TAPEVOL profile. If it is the first file of an IBM standard label tape being processed, DFSMSrmm creates a TVTOC containing an entry for the first file. This enables the installation to use RACF generic data set profiles to control access to the tape data sets, even when the JCL does not include the PROTECT=YES option.

TPRACF(P) processing is the same as TPRACF(A) except that all scratch tapes, whether defined by RMM TSO subcommand or returned to scratch by expiration processing, are protected by predefined RACF TAPEVOL profiles. The TAPEVOL profile includes an empty TVTOC so that RACF considers the volume to be scratch.

TPRACF(C)—DFSMSrmm processing is limited to cleaning up the existing TAPEVOL and discrete DATASET profile when volumes are deleted from a RACF-protected pool and when recycled scratch tapes return to scratch.

Table 29 on page 235 shows DFSMSrmm processing when RACF is active and OPTION TPRACF is set for volume pools identified as DFSMSrmm and RACF-managed. The processing is dependent on the

combinations of RACF TAPEVOL class and TAPEDSN option, and the TPRACF value. The DFSMSrmm EDGRMMxx parmlib VLPOOL RACF(Y) is in effect.

Table 29. RACF processing performed by DFSMSrmm

Command or Function	TAPEVOL	TAPEDSN	TAPEVOL and TAPEDSN
ADDVOLUME MASTER	For TPRACF(A/P): • Create TAPEVOL profile • Add access list built using the owner, user, and access information	No processing	As for TAPEVOL
ADDVOLUME USER	For TPRACF(A/P): • Create TAPEVOL profile • Add access list built using the owner, user, and access information	No processing	As for TAPEVOL
ADDVOLUME MASTER PREVVOL	For TPRACF(A/P): • Add to tape volume set	No processing	As for TAPEVOL
ADDVOLUME USER PREVVOL	For TPRACF(A/P): • Add to tape volume set	No processing	As for TAPEVOL
ADDVOLUME SCRATCH	No processing	No processing	If TPRACF(P) • Create TAPEVOL profile • Add a TVTOC if SL or AL.
DELETEVOLUME FORCE / REMOVE	Delete TAPEVOL profile	If data set records, delete tape data set profiles	• Delete TAPEVOL profile • If data set records, delete tape data set profiles
DELETEVOLUME RELEASE	No processing	No processing	No processing
CHANGEVOLUME RACK Change of pool—RACF(Y) to RACF(N)	Delete TAPEVOL profile	If data set records, delete tape data set profiles	• Delete TAPEVOL profile • If data set records, delete tape data set profiles
CHANGEVOLUME RACK Change of pool—RACF(N) to RACF(Y)	For TPRACF(A/P): • Create TAPEVOL profile • Add access list built using the owner, user, and access information	No processing	As for TAPEVOL
CHANGEVOLUME OWNER OWNERACC PROT USERS	For TPRACF(A/P): • Replace access list	No processing	As for TAPEVOL
CHANGEVOLUME USER / MASTER (not from SCRATCH)	No processing	No processing	No processing

Table 29. RACF processing performed by DFSMSrmm (continued)

Command or Function	TAPEVOL	TAPEDSN	TAPEVOL and TAPEDSN
CHANGEVOLUME USER / MASTER (from SCRATCH) GETVOLUME	For TPRACF(A/P): - Delete TAPEVOL profile - Create TAPEVOL profile - Add access list	If data set records, delete tape data set profiles	As for TAPEVOL
CHANGEVOLUME PREVVOL	For TPRACF(A/P): - Delete TAPEVOL profile - Add volume to tape volume set	No processing	As for TAPEVOL
SCRATCH mount processing	For TPRACF(A/P): At close end-of-volume create a TAPEVOL profile if one does not exist and add OWNER as accessor	No processing	For TPRACF(A/P): At close/end-of-volume create a TAPEVOL profile if one does not exist and create a TVTOC containing first file data set and add OWNER as accessor
MASTER / USER mount processing	No processing	No processing	No processing
Release processing When volume returns to scratch	Delete TAPEVOL profile	If data set records, delete tape data set profiles	- Delete TAPEVOL profile - For TPRACF(P) - Create TAPEVOL profile - Add a TVTOC if SL or AL.
DELETEDATASET	No processing	Delete tape data set profile	As for TAPEDSN
DELETEOWNER NEWOWNER	As for CHANGEVOLUME OWNER	As for CHANGEVOLUME OWNER	As for CHANGEVOLUME OWNER

Recommendations for tape security

For optimum tape security, exploiting the capabilities of DFSMSrmm, DFSMSdfp, and RACF, it is recommended that you use of these:

- In DEVSUPxx:
 - TAPEAUTHDSN=YES
 - TAPEAUTHF1=YES
 - TAPEAUTHRC4=FAIL
 - TAPEAUTHRC8=FAIL
- In EDGRMMxx:
 - OPTION TPRACF(N)
- In RACF:
 - SETROPTS NOTAPEDSN NOCLASSACT(TAPEVOL)

The combination of DFSMSrmm, DFSMSdftp, and RACF ensures:

- Full 44 character data set name validation.
- Validation that the correct volume is mounted.
- Control the overwriting of existing tape data sets.
- Management of tape data set retention.
- Control over the creation and destruction of tape volume labels.
- No limitations caused by RACF TAPEVOL profile sizes and TVTOC limitations.
- All tape data sets on a volume have a common authorization.
- Use of generic DATASET profiles, enabling common authorization with DASD data sets.
- Authorization for all tape data sets regardless of the tape label type.
- Authorization for the use of bypass label processing (BLP).
- Exploitation of RACF 'erase on scratch' support.
- Use of DFSMSrmm FACILITY class profiles for data sets unprotected by RACF. Your authorization to use a volume outside of DFSMSrmm control with 'ignore' processing also enables authorization to the data sets on that volume.

To aid migration to this recommended environment, DFSMSrmm provides the TPRACF(CLEANUP) option, and DEVSPxx provides TAPEAUTHRC8(WARN) and TAPEAUTHRC4(ALLOW).

Recommendations for using RACF tape profile processing

When you do not use the DEVSPxx TAPEAUTHxxx options to control tape data set security, these steps are recommended. For optimal security, make TAPEVOL and TAPEDSN active with either TPRACF(P) or TPRACF(A) to:

- Obtain full protection at the volume and the data set level
- Avoid the need to predefine volumes to individual users
- Avoid the need to use ADSP or PROTECT=YES which eliminates exit code that is not standard
- Use generic tape data set profiles

Note:

1. The maximum number of entries for data sets that a TVTOC can contain is 500.



Attention:

Processing that creates large numbers of TVTOC entries and large access lists, for example, could result in an attempt to exceed the maximum profile size.

2. The maximum number of volumes that any data set on the tape with an entry in the TVTOC can span is 42.
3. The maximum number of volumes that any data set on tape without a TVTOC can span is limited only by the maximum profile size.

When both TAPEDSN and TAPEVOL are active, RACF can create two different types of TVTOC profiles:

- An automatic TVTOC tape volume profile.
- A nonautomatic TVTOC tape volume profile.
- The NOSET option on the DELDSD command can be used to remove a discrete tape data set profile without deleting the tape volume profile. For more information, see [*z/OS Security Server RACF Command Language Reference*](#).

Although we discourage this, it is possible to have no tape protection, or to use only TAPEVOL profiles. If you have no tape security, you cannot control the creation and use of data on tape. If you use only TAPEVOL profiles to protect tape volumes, you must maintain the access lists in the TAPEVOL profiles to

allow access to data. Consider the use of either DEVSUPxx TAPEAUTHDSN or TAPEDSN so that access to tape data is covered by your normal, existing, generic data set profiles.

There is a potential security exposure with scratch volumes that have no RACF profile, but with DFSMSrmm active in protect mode, you can prevent reading of scratch tapes.

You can use either DEVSUPxx TAPEAUTHDSN or TAPEDSN on its own to provide data set level security. RACF cannot, however, guarantee full data set name integrity (only the last 17 characters of the data set name that are recorded in the tape label). Run DFSMSrmm in protect mode to ensure that full 44-character data set names are validated. With TAPEDSN only, you lack control of access to tape volumes at the volume level. If you do not use protect mode, your system security could be circumvented and tape data could be accessed.

You can prevent volume usage on individual systems by using the DFSMSrmm REJECT command in the EDGRMMxx parmlib member. You can reject volumes that are defined to DFSMSrmm based on your chosen pool prefix using the REJECT command in parmlib. See [“Implementing PRITITION and OPENRULE parmlib commands” on page 200](#) for additional information on the PRITITION and OPENRULE commands.

Rejecting volumes on specific systems in a system complex

In a system complex where all systems share the RACF data set, the RACF profile provides the same protection for all systems. If you want to prevent a volume from being used on one system, you must prevent its use on all systems. However, you can prevent volume usage on individual systems by using the DFSMSrmm REJECT command in the EDGRMMxx parmlib member. You can reject volumes that are defined to DFSMSrmm based on your chosen pool prefix using the REJECT command in parmlib. See [“Implementing PRITITION and OPENRULE parmlib commands” on page 200](#) for additional information on the PRITITION and OPENRULE commands.

To protect several volumes across all systems in the complex, you can use generic RACF profiles.

Example: Create a profile that prevents any tape starting with a volume serial number prefix AB from being used on all systems in a system complex.

```
RDEFINE TAPEVOL AB* UACC(NONE)
```

To prevent a pool of volumes from being used on one system in a multisystem complex, do not define generic RACF TAPEVOL profiles. Use the DFSMSrmm REJECT parmlib option as shown in [Figure 86 on page 238](#).

```
REJECT ANYUSE(AB*)
```

Figure 86. Using the REJECT parmlib option

Protect volumes by using a combination of generic TAPEVOL profiles that you create, the discrete TAPEVOL profiles that DFSMSrmm creates, and the DFSMSrmm REJECT commands that you define.

Maintaining the user access list

For DFSMSrmm to maintain the access list when TPRACF(A) or TPRACF(P) is in use, use the RMM TSO subcommands to change access lists, rather than RACF. You can use RACF commands to add users and owners if the times when DFSMSrmm updates, deletes, or creates the TAPEVOL profiles are well defined, and during the time a volume is not scratch. The RACF profile is updated only if the DFSMSrmm volume access list is updated.

If you use TPRACF(A) or TPRACF(P), DFSMSrmm ensures that your tapes are protected by RACF. DFSMSrmm ensures that all non-scratch tapes are protected by a discrete RACF TAPEVOL profile. DFSMSrmm checks that a RACF profile exists whenever a data set is written on a tape. If a profile does not exist, DFSMSrmm creates one. Therefore you do not need to use RACF installation exits to set the JCL PROTECT=YES option or specify PROTECT=YES in your JCL. Additionally, because DFSMSrmm creates a

TVTOC when the RACF TAPEDSN option is used, you can use generic data set profiles for all tape data sets without changes to JCL or installation procedures.

Be careful about using RACF profiles to maintain a list of users who own and can access volumes. When you use the RMM ADDVOLUME and CHANGEVOLUME subcommands, you can maintain up to 12 users and owners for each volume. If the volume is in a RACF-controlled pool and RACF TAPEVOL class is active, the TAPEVOL profile access list is maintained with the list of users who can access the volume and the owner's user ID. If you change the RACF TAPEVOL profile access list using RACF commands, the DFSMSrmm control data set does not reflect the changes. The next time that DFSMSrmm updates the RACF TAPEVOL profile, it creates the access list from the volume information, but does not include any users or owners you added using RACF commands.

Using RACF with DFSMSrmm

Despite the varied tape security support that RACF and DFSMS DEVSUPxx TAPEAUTHDSN provides, many installations use RACF exits to control access to tape volumes and to tape data sets. Although there are likely to be many different implementations, these are some of the commonly implemented functions:

- Use of the RACHECK exit or the RACDEF exit to test or set the PROTECT=YES option
- Use of the ability to model a TAPEVOL profile on another existing profile; either a model or data set profile
- Use of RACHECK post-processing exit to prevent use of tapes that are not protected by RACF (PROTECTALL for tape)

DFSMSrmm RACF tape security support

The objective for DFSMSrmm RACF tape security support is to provide a complete interface with RACF for tapes so you can use any combination of valid DEVSUPxx or RACF options, including use of any RACF installation exits you still have. You can tell DFSMSrmm not to provide any RACF support by specifying the DFSMSrmm EDGRMMxx parmlib OPTION TPRACF(N) command. You can set up the type of processing wanted by requesting automatic TPRACF(A), predefined TPRACF(P), or TPRACF(C) RACF profiles. You can also use the DFSMSrmm EDGRMMxx parmlib VLPOOL RACF command to provide control of RACF at the pool level.

DFSMSrmm provides support for using the system TAPEAUTHDSN and TAPEAUTHF1 options and the RACF standard tape volume security protection with any combination of RACF TAPEVOL and TAPEDSN options.

DFSMSrmm automatic tape security support processing

DFSMSrmm automatic tape security processing assumes that when a tape is mounted for output as a scratch tape it will not be RACF protected. During the open processing for the tape data set either DFSMSdfp or your installation exits will cause some RACF profiles to be created. DFSMSrmm predefined processing also ensures that, when TAPEVOL and TAPEDSN are active, RACF predefined TAPEVOL profiles with an empty TVTOC are created for scratch volumes. To enforce a DFSMSrmm standard, that all tapes are RACF protected when the data set is closed, DFSMSrmm checks that a RACF security profile exists for the volume. If a RACF security profile does not exist, DFSMSrmm creates one and places the owner of the tape in the access list for the TAPEVOL profile with ALTER authority. The process ensures that your current tape security mechanism should continue to work as long as it is based on TAPEVOL profiles. If you do not want DFSMSrmm to create and maintain RACF TAPEVOL profiles (for example, you are using RACF TAPEDSN or DEVSUPxx TAPEAUTHDSN), specify TPRACF(NONE).

Data set profile processing implications

If you only use RACF SETROPT TAPEDSN and RACF DATASET data set profiles for tape data sets, you need to consider the implications described in this topic. If you use TAPAUTHDSN=YES in DEVSUPxx to support tape data set security, you do not need to consider this information.

The objective should be to get to standard RACF tape security without using installation exits. However, the introduction of the TAPEDSN option or use of TAPEDSN only can cause some complications. When TAPEDSN is in effect and a TAPEVOL profile with no TVTOC exists, only the owner can access the data set. This happens because RACF does not use the data set profiles unless the TAPEVOL profile contains a TVTOC. When no security profile has been created by DFSMSdftp or RACF installation exits, DFSMSrmm creates a TAPEVOL profile with a TVTOC, a UACC(NONE) and the volume owner in the access list. The first file is added to the TVTOC using RACFIND=NO so that any generic data set profiles your installation has can be used with tape data sets. Once the first file entry is created RACF will maintain the TVTOC for any future tape activity on the same volume.

RACF installation exit conversion

Some installations use only data set profiles to protect tape data. They rely on the tape management system to ensure that only valid scratch tapes are used for output. If they use TAPEVOL class without TAPEDSN option in effect, they have RACF exits that change the RACHECK for TAPEVOL class to a RACHECK for the data set being processed. Consequently, they prevent RACDEF in the TAPEVOL class. If your RACDEF exit is used to dummy out the DFSMSdftp RACDEFs that would normally create TAPEVOL profiles, they will also prevent DFSMSrmm from defining TAPEVOL profiles. If this applies to your installation you will have to make changes to your RACF exits before making use of the DFSMSrmm RACF support.

This topic contains examples for converting RACF installation exits.

TAPEVOL class active. PROTECT=YES JCL option used

Currently the installation uses RACF exits to model the creation of the RACF TAPEVOL profile on an existing RACF data set profile. Access to tape data is effectively based on data set profiles.

When the DFSMSrmm TPRACF option is activated, volumes that are not covered by the PROTECT=YES option and the TAPEVOL profiles created using the data set modeling are protected by a TAPEVOL profile created by DFSMSrmm. Access to these data sets is based on the TAPEVOL profile which allows owner access only.

If the RACF exits modeling function is turned off, the volumes that were previously protected by a modeled profile are now only protected for the DFSMSrmm created profiles. Most probably, users lose access to tape data.

The suggested approach is to activate the TAPEDSN RACF option. For those volumes where PROTECT=YES is specified, the TAPEVOL profiles have TVTOCs created and data access is through data set profiles that currently exist. The creation of the TAPEVOL profiles by DFSMSrmm is also affected, and DFSMSrmm ensures that a TVTOC is created. The protection for tape volumes protected by DFSMSrmm created profiles is also through data set profiles.

TAPEVOL class active. exit requested PROTECT=YES option used

If your RACF installation exits are used to 'turn on' the PROTECT=YES option, and those exits are removed, no DFSMSdftp requests are made to RACF to protect tape data, even if TAPEDSN is activated. All volumes get protected at data set CLOSE time when DFSMSrmm creates TAPEVOL profiles.

Whether or not you choose to use the RACF TAPEDSN option, the results in creating a security environment for your installation are the same before and after you remove your installation exits.

TAPEVOL active. RACF exits using DATASET not TAPEVOL profiles

Conversion to standard options to remove the exits involves deactivating the TAPEVOL class and activating the TAPEDSN option. Use the PROTECTALL option to ensure complete security for tape data. When the TAPEVOL class is inactive, DFSMSrmm does not create TAPEVOL profiles for volumes and cleans up any discrete data set profiles that exist when a tape returns to scratch status. You can rely on DFSMSrmm to validate volumes that are mounted and prevent overwrites, validate 44 character data set names and manage data set and volume expiration.

Using RACF options for authorizing RMM TSO subcommands

Related reading: See [z/OS DFSMSrmm Managing and Using Removable Media](#) for the authorization for specific RMM TSO subcommands.

DFSMSrmm allows you to set up authorization for the TSO subcommands that you use with DFSMSrmm. In addition to using STGADMIN.EDG resources in the FACILITY class, you can use ownership and RACF DATASET and TAPEVOL class resources to protect resources. You can use the DFSMSrmm parmlib OPTION COMMANDAUTH command to control how ownership and DATASET and TAPEVOL resources are used. You can also use the RACF name-hiding function that is provided by RACF. Use the RACF SETROPTS MLNAMES command to activate the name-hiding function. See [“Defining system options: OPTION” on page 161](#) for information about the DFSMSrmm parmlib OPTION COMMANDAUTH operand.

Set up the DFSMSrmm authorization and security to control access to the information in the DFSMSrmm control data set. You do not have to perform additional setup tasks for librarians and storage administrators because they typically have CONTROL access to STGADMIN.EDG.MASTER, which allows them to access all resources. To allow general users to access data sets and volumes that they do not own, set up authorization for the general user.

In general, when you use DATASET and TAPEVOL authorization instead of or as well as ownership you do not need to define additional resource profiles because the existing DATASET and TAPEVOL profiles are used. Using command authorization by data set name provides an additional check to determine who is authorized to access information about data sets and volumes in DFSMSrmm. DFSMSrmm checks OWNER profiles or DATASET or TAPEVOL profiles to determine ownership of the data and authorization to access the information.

Using the SAF interface

DFSMSrmm does not provide any security functions itself but relies on installed security products to process requests. For example, DFSMSrmm relies on the installed security product to confirm that a user is authorized to perform a particular function using an RMM TSO subcommand.

DFSMSrmm uses the z/OS SAF interface to perform authorization checks and other security processing, as described in [“SAF calls for authorization checking” on page 241](#). DFSMSrmm issues RACROUTE requests that RACF, or a functionally equivalent security product, can process.

[“Protecting DFSMSrmm resources with RACF profiles” on page 215](#) describes security profile names and class name. If you have not installed a security product, you could write a SAF router exit to handle the calls that DFSMSrmm makes to the interface.

DFSMSrmm also uses the SAF interface to create, update, and delete tape-related security profiles and access lists as described in [“SAF and RACF calls for creating, updating and deleting security profiles” on page 244](#). When you update the volume access list in the control data set, DFSMSrmm uses the RACF ICHEINTY macro to delete the entire access list and allows you to use the SAF interface to add the required access list. If your system does not support this function, do not use or update the DFSMSrmm volume access lists contained in the control data set.

SAF calls for authorization checking

DFSMSrmm issues RACROUTE calls to determine if a user is authorized to perform a DFSMSrmm function. The calls are issued in the address space and under the task of the command or utility user. Most SAF calls in the FACILITY class are issued regardless of the state of RACF, the SAF interface, or the FACILITY class.

DFSMSrmm prevents RACF users with the OPERATIONS and PRIVILEGED attributes from gaining authorization to the DFSMSrmm resources in the FACILITY class. Any user attempting to use DFSMSrmm functions must be authorized through the resource access list or through universal access. For all authorization checks, except for EDGRESET, DFSMSrmm issues the RACROUTE request with an ACEE address that identifies an ACEE that has had these attributes removed.

Figure 87 on page 242 shows the RACROUTE call that DFSMSrmm issues to create an ACEE for a user that is defined to RACF. DFSMSrmm issues this call in the address space of the command issuer or batch utility.

```
RACROUTE REQUEST=VERIFY,ENVIR=CREATE,RELEASE=1.9,  
        USERID=ACEEUSER,GROUP=ACEEGRP,PASSCHK=NO,SUBPOOL=(3)
```

Figure 87. Creating an ACEE for a user defined to RACF

Figure 88 on page 242 shows the RACROUTE call that DFSMSrmm issues to create an ACEE for a user that is not defined to RACF. DFSMSrmm issues the call using a blank USERID value. DFSMSrmm issues this call in the address space of the command issuer or batch utility.

```
RACROUTE REQUEST=VERIFY,ENVIR=CREATE,RELEASE=1.9,  
        USERID=ACEEUSER,PASSCHK=NO,SUBPOOL=(3)
```

Figure 88. Creating an ACEE for a user not defined to RACF

Examples: Checking authorization for issuers of RMM TSO subcommands

Example 1: This example shows authorization checking for issuers of RMM TSO subcommands and users of the utilities when a single RACROUTE is issued. The request is issued in the address space of the command issuer or batch utility.

```
RACROUTE REQUEST=AUTH,CLASS=class,ATTR=level,ENTITY=resource,  
        LOG=ASIS
```

Example 2: This example shows authorization checking for issuers of RMM TSO subcommands and users of the utilities when multiple RACROUTEs are issued. The request is issued in the address space of the command issuer or batch utility.

```
RACROUTE REQUEST=AUTH,CLASS=class,ATTR=level,ENTITY=resource,  
        LOG=NOFAIL
```

The variables in the examples have these meanings:

class - FACILITY

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

level - READ, UPDATE, CONTROL

Access level as described in [Table 20 on page 219](#).

resource

This field is 39 characters and contains the name of the resource to which access is being checked.

Examples: Checking for authorization when additional security is in use

Before you begin: See [z/OS Security Server RACF Security Administrator's Guide](#) for information about how to use the SETROPTS command.

Example 1: This example shows how authorization checking for RMM TSO subcommands is set. The RACROUTE command is issued in the DATASET class. The request can be issued in the address space where the command is issued or in the DFSMSrmm subsystem address space. When you issue the command in the DFSMSrmm subsystem address space, a third-party RACROUTE is issued.

```
RACROUTE REQUEST=AUTH,CLASS=DATASET,ATTR=level,ENTITY=resource,  
        LOG=ASIS,DSNTYPE=T,FILESEQ=n
```

When a subcommand is issued against a data set name or a volume containing data sets, the RACROUTE is issued in the DATASET class with DSNTYPE=T. When there is no data set information for a MASTER or USER volume, the RACROUTE is issued in the TAPEVOL class. When DSNTYPE=T is coded, the authorization checking that is performed depends on the security product settings such as SETROPTS TAPEDSN, and whether the TAPEVOL class is active.

The variables in the examples have these meanings:

level

Is READ or UPDATE depending on the subcommand issued.

resource

Is the data set name to be processed. When the subcommand is issued against a volume, the data set name is the name of the first file on the volume.

n

Is the file sequence number of the data set on the volume.

Example 2: This example shows how authorization checking for RMM TSO subcommands is set. The RACROUTE command is issued in the TAPEVOL class. The request can be issued in the address space where the command is issued or in the DFSMSrmm subsystem address space. When issued in the DFSMSrmm subsystem address space, a third-party RACROUTE is issued.

```
RACROUTE REQUEST=AUTH,CLASS=TAPEVOL,ATTR=level,ENTITY=resource,  
LOG=ASIS
```

The variables in the examples have these meanings:

level

is READ or UPDATE depending on the subcommand issued.

resource

Is the volume to be processed.

Example: Checking authorization to ignore volumes

This example shows the command that you can use to check whether the current user is authorized in order to have DFSMSrmm ignore a volume.

```
RACROUTE REQUEST=AUTH,ENTITY=resource, ACCESS=value,LOG=ASIS
```

The variables in the examples have these meanings:

resource

resource can be: STGADMIN.EDG.IGNORE.TAPE.*volser*, STGADMIN.EDG.IGNORE.TAPE.RMM.*volser*, or STGADMIN.EDG.IGNORE.TAPE.NORMM.*volser* as described in [Table 20 on page 219](#).

value - READ,UPDATE

One of these values is used, as described in [Table 20 on page 219](#).

volser

This field is the volume serial number of the current mounted volume or the requested volume. The default value is the mounted volume *volser*. The DFSMSrmm installation exit EDGUX100 can select whether the mounted or the requested volume serial number is used.

Example: Checking for authorization to create label and no label volumes

This example is the RACROUTE call for the STGADMIN.EDG.LABEL.*volser* and STGADMIN.EDG.NOLABEL.*volser* profiles.

```
RACROUTE REQUEST=AUTH,ENTITY=STGADMIN.EDG..volslabeller,  
ACCESS=value,LOG=ASIS
```

The variables in the examples have these meanings:

label

The label is either LABEL or NOLABEL.

value - UPDATE,ALTER

One of these values is used, as described in [Table 20 on page 219](#).

volser

This field is the volume serial number of the current mounted volume.

Example: Checking authorization to remove DFSMSrmm from the system

When you do not have a security product installed, you can use the EDGRESET utility to remove DFSMSrmm from the system. To use the EDGRESET utility, you must use the SAF router exit.

This example is used to check that EDGRESET utility can be used. This request is issued in the address space of the EDGRESET utility.

```
RACROUTE REQUEST=AUTH,CLASS=class,ATTR=ATTR,ENTITY=resource,
        RELEASE=1.8
```

The variables in the examples have these meanings:

class - FACILITY

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource - STGADMIN.EDG.RESET.SSI

This field is 39 characters and contains the name of the resource to which access is being checked.

SAF and RACF calls for creating, updating and deleting security profiles

These are the SAF calls that DFSMSrmm issues to maintain the security protection of tape resources in your installation. They are not used to perform authorization checking. Authorization checking for access to tape data and tape volumes is still the responsibility of the OPEN macro and RACF. All these requests are issued from the DFSMSrmm started procedure address space.

When the requests are issued, if the RMM started task is not already in either privileged or trusted mode, the RACF ACEE control block for the DFSMSrmm started procedure is modified to set the privileged bit on (ACEEPRIV), so that DFSMSrmm requests are honored without authorization checking being performed. Modifying the ACEE control block causes the IRR421I message to be issued when the RACF ACEECHK class is activated. To avoid receiving this message, ensure that the STARTED profile for the DFSMSrmm started task is defined using either the PRIVILEGED(YES) or the TRUSTED(YES) attributes.

Calls for TAPEVOL class are only issued if RACF is active and the TAPEVOL class is active. Calls for DATASET class are only issued if RACF is active and the TAPEDSN option is active.

For additional information on SAF calls, refer to [Table 29 on page 235](#).

Example: Checking for DATASET class resource

```
RACROUTE REQUEST=AUTH,CLASS=class,ENTITY=(resource,PRIVATE),LOG=NONE,
        RELEASE=1.8,DSTYPE=T,FILESEQ=seq,VOLSER=vol
```

The variables in the examples have these meanings:

class - DATASET

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 44 characters and contains the name of the data set for which the protecting resource is requested to be returned.

seq

The file sequence number for this data set.

vol

The volume on which the data set resides.

Example: Checking for TAPEVOL class resource

```
RACROUTE REQUEST=AUTH,CLASS=class,ENTITY=(resource,PRIVATE),
LOG=NONE,RELEASE=1.8
```

The variables in the examples have these meanings:

class - TAPEVOL

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 6 characters and contains the name of the volume for which the protecting resource is requested to be returned.

Example: Defining a TAPEVOL class

```
RACROUTE REQUEST=DEFINE,CLASS=class,ENTITY=resource,TYPE=DEFINE,
RELEASE=1.8,UACC=NONE
```

The variables in the examples have these meanings:

class - TAPEVOL

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 6 characters and contains the name of the volume for which the resource is being defined.

Example: Defining a TAPEVOL class resource when TAPEDSN is active

This example shows how to add the first entry to the TVTOC of the discrete TAPEVOL profile just created if TAPEDSN option is active.

```
RACROUTE REQUEST=DEFINE,CLASS=class,ENTITY=resource,TYPE=DEFINE,
RELEASE=1.8,UACC=NONE,RACFIND=NO,VOLSER=vol,
DSTYPE=T,FILESEQ=1,TAPELBL=STD
```

The variables in the examples have these meanings:

class - DATASET

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 44 characters and contains the name of the data set for which the resource is being defined.

vol

The volume on which the data set resides.

Example: Adding a tape volume

This example is used to add a tape volume to an existing tape volume set.

```
RACROUTE REQUEST=DEFINE,CLASS=class,ENTITY=resource,TYPE=ADDVOL,
RELEASE=1.8,VOLSER=vol
```

The variables in the examples have these meanings:

class - TAPEVOL

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 6 characters and contains the name of the volume for which the resource is being defined.

vol

The previous volume in the tape volume set.

Example: Deleting a TAPEVOL profile

This example is used to delete a discrete TAPEVOL profile.

```
RACROUTE REQUEST=DEFINE, CLASS=class, ENTITY=resource, TYPE=DELETE,
        RELEASE=1.8
```

The variables in the examples have these meanings:

class - TAPEVOL

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 6 characters and contains the name of the volume for which the resource is being deleted.

Example: Deleting a DATASET profile

This example is used to delete a discrete DATASET profile.

```
RACROUTE REQUEST=DEFINE, CLASS=class, ENTITY=resource, TYPE=DELETE,
        RELEASE=1.8, VOLSER=vol
```

The variables in the examples have these meanings:

class - DATASET

This field names a 9 character variable. The first byte contains the length of the class name, which follows in the next 8 bytes.

resource

This field is 44 characters and contains the name of the data set for which the resource is being deleted.

vol

The volume on which the data set resided.

Example: Checking for authorization

Use the commands in [Figure 89 on page 247](#) to perform these tasks:

- Create or update the volume access list
- Give the volume a TVTOC
- Rebuild a tape volume set when a volume is removed from within, rather than from the end of the set, when a TVTOC is used.

```

RACROUTE REQUEST=EXTRACT,CLASS=class,ENTITY=resource,TYPE=EXTRACT,
        RELEASE=1.8,SUBPOOL=0,SEGMENT=BASE,FIELDS=fields
ICHEINTY ALTER,TYPE='GEN',CLASS=class,ACTIONS=D,ENTRY=resource,
        RELEASE=1.8,OPTIONS=FLDEF
ICHEACTN FIELD=ACLCNT,FLDATA='DEL',GROUP=YES,
        RELEASE=1.8
RACROUTE REQUEST=EXTRACT,CLASS=class,ENTITY=resource,TYPE=REPLACE,
        RELEASE=1.8,SEGDATA=data,SEGMENT=BASE,FIELDS=fields

```

Figure 89. Checking authorization

The variables in the examples have these meanings:

***class* - TAPEVOL**

This field is 8 characters and contains the class name.

resource

This field is 6 characters and contains the name of the volume whose profile is being processed.

fields

Defines which fields are being extracted and updated. The field names used include: TVTOCCNT,VOLCNT,ACLCNT,VOLSER,OWNER,UACC,ACL.

data

data defines the fields that are being replaced. The field names used include: OWNER,UACC,ACL.

Chapter 10. Using DFSMSrmm programming interfaces

DFSMSrmm provides programming interfaces EDGLCSUX, EDGMSGEX, and EDG3X71 to use Object Access Method (OAM), DFSMSdfp, and JES3 installation exits. DFSMSrmm uses the installation exits in [Table 30 on page 249](#). Before implementing DFSMSrmm, ensure that there are no conflicts between how DFSMSrmm uses these exits and how your installation currently uses them.

DFSMSrmm provides programming interface EDGTVEXT for use from DFSMSHsm or Tivoli Storage Manager. DFSMSrmm provides programming interfaces EDGTVEXT and EDGDFHSM for use by applications that want to release tape volumes.

Table 30. Installation exits used by DFSMSrmm

Exit	DFSMSrmm Programming interface	DFSMSrmm's Use of the Exit
CBRUXCUA	EDGLCSUX	DFSMSrmm uses OAM's change use attribute exit to manage the volumes in system-managed tape libraries.
CBRUXEJC	EDGLCSUX	DFSMSrmm uses OAM's cartridge eject exit to manage the volumes in system-managed tape libraries.
CBRUXENT	EDGLCSUX	DFSMSrmm uses OAM's cartridge entry exit to manage the volumes in system-managed tape libraries.
CBRUXVNL	EDGLCSUX	DFSMSrmm uses OAM's volume-not-in-library installation exit to process tape volumes that are not resident in system-managed tape libraries but that are needed for processing to continue.
IGXMSGEX	EDGMSGEX	DFSMSrmm uses the DFSMSdfp MSGDISP exit to update tape drive displays and control the use of cartridge loaders.
IATUX71	EDG3X71	DFSMSrmm uses the JES3 exit to determine the processing needed for JES3 messages and tape drive display processing.

When you are converting from another tape management product to DFSMSrmm, you might run DFSMSrmm in parallel with the other tape management system for some time before you complete your conversion. Some installation exits must be used by both systems. DFSMSrmm supplies exits that help you run the CBRUXCUA exit, the CBRUXEJC exit, and the IGXMSGEX exit from an existing tape management product and DFSMSrmm in parallel. When exits are used by both systems, the tape management system that is in control makes the tape management decisions and the second system records the activities. See [“Setting up parallel processing” on page 264](#) for additional information.

Releasing tapes: EDGTVEXT

DFSMSrmm provides the programming interface EDGTVEXT that is used from DFSMSHsm, OAM, or any other APF-authorized program that needs to obtain the same services as the DFSMSHsm ARCTVEXT exit.

If you have a product with similar requirements for releasing tapes as DFSMSHsm, you can use the EDGTVEXT program interface. You can also use the EDGDFHSM interface. The difference between EDGTVEXT and EDGDFHSM is that EDGTVEXT accepts the ARCTVEXT parameter list and the EDGDFHSM interface accepts only a single volume at a time in the parameter list.

Any caller of EDGTVEXT must be defined to RACF. If the caller is a started task, define the user ID with the STARTED class. Your application must also be authorized to release its own tape volumes.

Related reading: Refer to [Chapter 12, “Running DFSMSrmm with DFSMSHsm,”](#) on page 311 for information about setting up DFSMSHsm with DFSMSrmm. You can use this information as an example for setting up other applications, including OAM, that manage tape. Refer to [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,”](#) on page 215 for information about the authorization support available with DFSMSrmm.

You must also consider how volumes are retained until the application calls the EDGTVEXT exit to release the volumes. You could retain tapes by defining vital record specifications like the ones shown in these examples. The examples define policies to retain all the data until a volume is released by the application.

```
RMM ADDVRS DSN('**') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
RMM ADDVRS DSN('ABEND') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
RMM ADDVRS DSN('DELETED') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
RMM ADDVRS DSN('OPEN') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
```

You can use DFSMSrmm parmlib option TVEXTPURGE to control the processing that EDGTVEXT performs. You can release volumes or set the volume expiration date either to the current date or based on a number of days from the current date. The effect of setting the expiration date depends on the retention method (EXPDT or VRSEL) already specified for the volume. Refer to TVEXTPURGE in [“Defining system options: OPTION”](#) on page 161.

Invocation

Invoke EDGTVEXT from: LOAD and CALL macros or the LINK macro.

Input

The input is a parameter list that describes the volumes DFSMSHsm is releasing and the actions required. The parameter list is identical to the one DFSMSHsm passes to ARCTVEXT.

On entry, register 1 contains a pointer to the ARCTVEXT parameter list. See [z/OS DFSMS Installation Exits](#) for information on ARCTVEXT.

Output

EDGTVEXT issues messages when errors are encountered. EDGTVEXT sets the parameter list return code value to zero. EDGTVEXT does not always pass a zero return code in register 15 back to the caller. EDGTVEXT issues a non-zero return code in the register 15 return code from subsystem requests attempted by EDGTVEXT. You can obtain information about the return codes in [z/OS MVS Using the Subsystem Interface](#).

Processing

EDGTVEXT ensures that DFSMSrmm is in use on your system before continuing with DFSMSrmm processing. If DFSMSrmm is not in use, EDGTVEXT sets return code zero and returns to the caller. If DFSMSrmm is in use or should be in use, EDGTVEXT calls EDGDFHSM to process the volumes passed to it in the ARCTVEXT parameter list.

Environment

EDGTVEXT must be link edited in an APF-authorized library. EDGTVEXT runs in AMODE(31) RMODE(ANY).

Managing DFSMSHsm tapes: EDGDFHSM

DFSMSrmm uses the EDGDFHSM programming interface to release DFSMSHsm tape volumes. The interface between DFSMSHsm and DFSMSrmm is automatically in place when you install DFSMS, so you do not need to take any action to activate it or to call it.

Any caller of EDGDFHSM must be defined to RACF. If the caller is a started task, define the user ID with the STARTED class. Authorize your application to release its own tape volumes.

You must also consider how volumes are retained until the application calls the EDGDFHSM exit to release the volumes. You can use this program interface from programs other than DFSMSHsm to release tape volumes. Refer to Chapter 12, “Running DFSMSrmm with DFSMSHsm,” on page 311 for information about setting up DFSMSHsm with DFSMSrmm. You can use this information as an example for setting other applications that manage tape. You could retain tapes by defining vital record specifications like the ones shown in this example.

Example: Define policies to retain all the data until a volume is released by the application.

```
RMM ADDVRS DSN('**') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
RMM ADDVRS DSN('ABEND') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
RMM ADDVRS DSN('DELETED') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
RMM ADDVRS DSN('OPEN') JOBNAME(jobname) LOCATION(CURRENT) DAYS COUNT(99999)
```

Another way to retain tapes is with the EXPDT retention method.

You can use DFSMSrmm parmlib OPTION TVEXTPURGE operand to control the processing that EDGDFHSM performs. You can release volumes or set the volume expiration date either to the current date or based on a number of days from the current date. The effect of setting the expiration date depends on the retention method (EXPDT or VRSEL) already specified for the volume. Refer to TVEXTPURGE described in “Defining system options: OPTION” on page 161.

Callers of EDGDFHSM can request the processing of volumes only when their RACF user ID is authorized as defined by the RACF FACILITY class profiles for DFSMSrmm.

Invocation

Invoke EDGDFHSM from: LOAD and CALL macros or the LINK macro.

Input

The input is a parameter list that describes the volume DFSMSHsm is releasing and the actions required. The parameter list is similar to the one DFSMSHsm passes to ARCTVEXT, except that DFSMSrmm's parameter list handles only a single volume at a time.

On entry, register 1 contains a pointer to an 8-byte field. The first 6 bytes are the volume serial number to be processed and the last 2 bytes are flag bytes. The contents of the flag bytes are the flag bytes DFSMSHsm passes to the ARCTVEXT exit. See *z/OS DFSMS Installation Exits* for information on ARCTVEXT.

On entry, register 13 contains the address of a standard 18 word save area and register 14 contains the return address.

Output

A non-zero return code is the register 15 return code from the subsystem request attempted by EDGDFHSM. You can obtain information about the return codes in *z/OS MVS Using the Subsystem Interface*.

Processing

EDGDFHSM issues messages when it encounters errors, but it does not always pass a non-zero return code back to the caller.

Environment

EDGDFHSM must be link edited in an APF-authorized library. It runs in AMODE(31) RMODE(ANY).

Managing system-managed tape library volumes: EDGLCSUX

DFSMSrmm uses the OAM installation exits CBRUXCUA, CBRUXEJC, CBRUXENT, and CBRUXVNL as described in [Table 31 on page 252](#) to manage the tape volumes defined in system-managed tape

libraries. DFSMSrmm supplies Assembler source code for these installation exits, which is installed on your system as modules CBRUXCUA, CBRUXEJC, CBRUXENT, and CBRUXVNL. The sample CBRUXEJC, CBRUXENT, and CBRUXVNL exits show how to call the DFSMSrmm API. The use of the API is optional and must be enabled by the customer. The customer must also write code to process the results of the call to the API. The sample code shows how to list the volume being processed by OAM using the DFSMSrmm API. You can modify the source code to include your own function or merge with another product's supplied code.

DFSMSrmm uses EDGLCSUX to support these functions:

- Tracking TCDB changes by updating the DFSMSrmm control data set.
- Updating the CBRUXxxx parameter list based on DFSMSrmm information.
- Partitioning a library.
- Handling volume-not-in-library conditions.
- Controlling the purging of TCDB volume entries at eject time.

You can use the DFSMSrmm EDGRMMxx OPTION SMSTAPE operand to control the support that DFSMSrmm provides. Refer to [“Defining system options: OPTION” on page 161](#) for information about the SMSTAPE operand. See [Appendix A, “DFSMSrmm mapping macros,” on page 511](#) for mapping macros you can use with the installation exits.

Table 31. OAM installation exits

Exit	Description
CBRUXCUA	The CBRUXCUA exit controls the return to scratch status and updates the DFSMSrmm control data set with information from the TCDB.
CBRUXEJC	The CBRUXEJC exit controls the ejection of a cartridge from a library and updates information in the DFSMSrmm control data set. You can run this exit in parallel with another copy of the exit. See “Setting up parallel processing” on page 264 for more information.
CBRUXENT	The CBRUXENT exit controls the entry of a cartridge into a library, updates information in the DFSMSrmm control data set, and updates DFSMSrmm information in the TCDB. You can run this exit in parallel with another copy of the exit. See “Setting up parallel processing” on page 264 for more information.
CBRUXVNL	The CBRUXVNL exit retrieves information from the DFSMSrmm control data set when a volume that is not in a tape library is needed for processing to continue.

The OAM exits that DFSMSrmm supplies call EDGLCSUX, the DFSMSrmm program that supports the OAM interface. EDGLCSUX is a callable program interface to DFSMSrmm that you can use only from the OAM installation exits.

If you do not have a license for DFSMSrmm, the DFSMSrmm OAM exits set a return code of 16, telling OAM to never call the exit again. For information on licensing, see [“Enabling DFSMSrmm” on page 42](#). If you are licensed for DFSMSrmm, and you have performed some of the installation steps, but have not added EDGSSSI to the IEFSSNxx member of parmlib, or have not started the DFSMSrmm procedure, DFSMSrmm sets return code zero, allowing all OAM requests to be accepted. Once you have performed the installation steps, but do not start the DFSMSrmm subsystem, the OAM exits set a return code 8 and fail all requests.

Input

The input is a parameter list mapped by the macro EDGLCSUP, shown in [“OAM interface: EDGLCSUP” on page 511](#). The parameter list describes the OAM installation exit calling EDGLCSUX and provides the address of the information passed to the exit.

In the EDGLCSUP macro, the field LCSUP_LCSPL must contain the pointer to the OAM installation exit parameter list, which is the value in register 1 on entry to the OAM exit. The OAM installation exit parameter list macros are used to map this data. See *z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries* for more information about CBRUXCPL, CBRUXEPL, CBRUXJPL, and CBRUXNPL.

Complete the LCSUP_FUNCTION field each time EDGLCSUX is called to indicate which OAM exit is calling it. Select from LCSUP_CUA, LCSUP_EJC, LCSUP_ENT, LCSUP_VNL, and LCSUP_ACTVNL.

The parameter list EDGLCSUP includes output fields for the OAM return code LCSUP_LCSRC and the DFSMSrmm reason code LCSUP_LCSRS. The fields LCSUP_LCSRC and LCSUP_LCSRS contain a reason code from DFSMSrmm and a return code to pass back to OAM. The LCSUP_LCSRC field contains the value that is recommended by DFSMSrmm to be returned for this OAM request. The supplied DFSMSrmm OAM installation exits pass back these values. For example, if the LCSUP_LCSRC value is 4, DFSMSrmm has updated the OAM parameter list during processing. You can also use the return and reason codes set in register 15 and register 0 after the call to EDGLCSUX to determine the processing you might want to perform.

On entry, register 13 contains the address of a standard 18 word save area and register 14 contains the return address.

Output

The DFSMSrmm control data set and OAM parameter list are updated depending on the type of processing performed.

Register 15 contains the return code which indicates what processing should be performed next. Register 0 contains the reason code. Use the return code and the reason code to determine the processing that DFSMSrmm has been able to perform. [Table 32 on page 253](#) shows the return and reason codes that EDGLCSUX sets in register 15 and register 0.

Table 32. EDGLCSUX return and reason codes returned in register 15 and register 0

Return Code	Reason Code	Description
0	0	Processing successful. LCSUP_LCSRC contains the return code for OAM. LCSUP_LCSRS contains a reason code that provides information about DFSMSrmm processing. The reason codes are described by variables LCSUP_RS_xxxx listed in the EDGLCSUP macro.
0	1	Processing successful. LCSUP_LCSRC contains the return code for OAM.
0	2	Processing successful. DFSMSrmm is not licensed for use on this processor. LCSUP_LCSRC contains the return code 16 for OAM.
4	0	Processing not performed because the DFSMSrmm subsystem was not available. LCSUP_LCSRC contains the return code for OAM.
8	0	Processing not performed because there was a logic error during processing. LCSUP_LCSRC contains the return code for OAM.
12	Various reason codes set	Processing is unsuccessful. The reason codes are described by variables LCSUP_RS_xxxx listed in the EDGLCSUP macro. The supplied exits set RC 8 for OAM to fail requests.

[Table 33 on page 254](#) defines the return codes generated for each OAM function based on the reason codes set by DFSMSrmm in field LCSUP_LCSRS. The return code variables used in the table are defined in the OAM macros, CBRUXCPL, CBRUXEPL, CBRUXJPL, and CBRUXVNL. If the DFSMSrmm parmlib OPTION SMSTAPE(UPDATE(EXITS)) operand is not set or if DFSMSrmm is running in warning or record-only mode, the OAM return codes UXxFAIL are changed to UXxNOCHG.

Table 33. EDGLCSUX return and reason codes based on DFSMSrmm reason code setting

Reason Code	Description	Related Message Number	Change Use Return Code	Volume Entry Return Code	Volume Eject Return Code	Volume- Not in- Library
0	DFSMSrmm accepted request	n/a	UXCNOCHG or UXCCHG “1” on page 255	UXENOCCHG or UXECHG “1” on page 255	UXJNOCHG or UXJCHG “1” on page 255	UXNNORML or UXNCHG “1” on page 255
LCSUP_RS_DEB	Specified destination is not current library	EDG8192I	n/a	UXEFAIL	n/a	n/a
LCSUP_RS_DUPLV	Volume duplicates an existing logical volume	EDG8183I	n/a	UXEFAIL	n/a	n/a
LCSUP_RS_DUPPV	Volume duplicates an existing physical volume	EDG8182I	n/a	UXEFAIL	n/a	n/a
LCSUP_RS_DUPSV	The volume duplicates an existing stacked volume.	EDG8181I	UXCFAIL	UXEFAIL	UXJFAIL	n/a
LCSUP_RS_IRK	Volume rack number inconsistent	EDG8189I	UXCFAIL	UXEFAIL	UXJNOCHG	n/a
LCSUP_RS_IVU	User ID not valid for DFSMSrmm	EDG8195I	UXCFAIL	UXEFAIL	UXJNOCHG	n/a
LCSUP_RS_NMV	Volume not to be used with z/OS	EDG8191I	UXCFAIL	UXEIGNOR	UXJFAIL	n/a
LCSUP_RS_NOTEXP	Imported volume is not exported	EDG8183I	n/a	UXEFAIL	n/a	n/a
LCSUP_RS_NRM	Volume not defined in a manual tape library	n/a	UXCNOCHG	UXENOCCHG	UXJNOCHG	UXNNORML
LCSUP_RS_PBD	Inconsistent parameter list	EDG8190I	UXCFAIL	UXEFAIL	UXJFAIL	UXNFAIL
LCSUP_RS_PNI	Volume matches to PRTITION, type NORMM, action IGNORE	n/a	UXCNOCHG	UXEIGNOR	Eject: UXJNOCHG, Export: UXJIGNOR	n/a
LCSUP_RS_PRI	Volume matches to PRTITION, type RMM, action IGNORE	n/a	UXCNOCHG	UXEIGNOR	Eject: UXJNOCHG, Export: UXJIGNOR	n/a
LCSUP_RS_RIU	Rack to match volser not available	EDG8198I	UXCFAIL	UXEFAIL	UXJNOCHG	n/a
LCSUP_RS_RJP	Undefined volume rejected by reject prefix	EDG8193I “2” on page 255	UXCFAIL	UXEIGNOR	Eject: n/a, Export: UXJIGNOR	n/a

Table 33. EDGLCSUX return and reason codes based on DFSMSrmm reason code setting (continued)

Reason Code	Description	Related Message Number	Change Use Return Code	Volume Entry Return Code	Volume Eject Return Code	Volume- Not in- Library
LCSUP_RS_RPX	Retention period exceeds installation maximum	EDG8196I	UXCFAIL	UXEFAIL	UXJNOCHG	n/a
LCSUP_RS_SCR	Private to scratch status not permitted	EDG8194I	UXCFAIL	UXEFAIL	UXJFAIL	n/a
LCSUP_RS_SMM	Entry status and DFSMSrmm status of volume mismatch	EDG8180I	n/a	UXENOCCHG	n/a	n/a

Notes:

- When these conditions are true:
 - Status is private and no expiration date is supplied
 - Status is private and last read or write dates are lower than the DFSMSrmm equivalent dates
 - Status is scratch and owner information exists; the first 8 bytes are set to blanks
 - Volume entered into a system-managed tape library and DFSMSrmm has values for either storage group name, owner, or tape device selection information
- Not issued during entry processing.

Processing

For information about how DFSMSrmm works with system-managed tape libraries, see [Chapter 5, “Running DFSMSrmm with system-managed tape libraries,”](#) on page 99. For information about how DFSMSrmm running mode affects system-managed tape library support, see [“Defining system options: OPTION”](#) on page 161.

DFSMSrmm processing for OAM support

DFSMSrmm processing is dependent on these conditions:

- The OPMODE processing mode specified in the DFSMSrmm EDGRMMxx parmlib member. When DFSMSrmm is running in manual mode, no DFSMSrmm processing is performed. When DFSMSrmm is running in warning mode, DFSMSrmm issues messages but does not fail any requests.
- The TCDB purge option specified in the DFSMSrmm EDGRMMxx parmlib member. Use the SMSTAPE(PURGE) option to control the processing that DFSMSrmm performs when a volume is ejected. DFSMSrmm can always keep the TCDB record, always purge the record, or accept the requestor's decision. If the requestor did not make a decision, DFSMSrmm uses the ISMF default value. The ISMF default can be set at the library level. If the TCDB record is kept, DFSMSrmm adds the destination location and bin information to the OAM shelf location information. This avoids the WTOR to the operator prompting for shelf information. If the shelf location is still set to 'DEST=' during entry processing, DFSMSrmm clears the field. Library partitioning driven by REJECT ANYUSE(*prefix*) and volume not for use on z/OS, is performed by DFSMSrmm in all modes except manual mode.
- DFSMSrmm performing library partitioning driven by REJECT ANYUSE(ANYUSE(*prefix*)) and volume not for use on z/OS when DFSMSrmm is running in all modes except manual mode.
- The DFSMSrmm EDGRMMxx parmlib OPTION SMSTAPE(UPDATE(EXITS)) operand. Information provided by OAM to the exits is used to enhance the information recorded by DFSMSrmm. You can decide whether the OAM information overrides the information already defined to DFSMSrmm. Use the SMSTAPE(UPDATE(EXITS)) option to use DFSMSrmm information to override other information and to activate the volume-not-library processing. When DFSMSrmm is running in protect mode,

the DFSMSrmm information overrides the information from OAM and performs volume-not-library processing

See “Defining system options: OPTION” on page 161 for information about the options.

Table 34 on page 256 describes DFSMSrmm processing for OAM support.

<i>Table 34. Processing for the change use attribute, cartridge entry and cartridge eject parameter list</i>			
Variable	Field Name	Input	Output
Checkpoint volume indicator	UXCCHKPT UXECHKPT UXJCHKPT	Not used	Not used
Installation exit information	UXCEXITI UXEEXITI UXJEXITI	DFSMSrmm uses this for communication between different parts of DFSMSrmm.	Input only
Last entry or ejection date	UXCENTEJ UXEENTEJ UXJENTEJ	DFSMSrmm updates the control data set with this value if no date is recorded or if the input date is later than the existing date. DFSMSrmm uses this to set the movement tracking date.	Input only
Last mounted date	UXCMOUNT UXEMOUNT UXJMOUNT	DFSMSrmm records this date if no date exists or if this date is more current than the existing date.	DFSMSrmm returns a date in the parameter list if the date is less than the date already recorded by DFSMSrmm. DFSMSrmm sets return code UXyCHG to indicate that the parameter list has been changed.
Last written date	UXCWRITE UXEWRITE UXJWRITE	DFSMSrmm records this date if no date exists or if this date is later than the existing date.	If the date is less than the existing date, DFSMSrmm returns the last written date and sets return code UXyCHG to show that the parameter list has been changed.
Library console name	UXCLCON UXELCON UXJLCON	All error messages issued by DFSMSrmm to support the OAM functions are issued to the named console and to consoles using the correct routing codes.	Input only
Library description	UXCLDESC UXELDESC UXJLDESC	Not used	Input only
Library device type	UXCLDEV UXELDEV UXJLDEV	Not used	Input only

Table 34. Processing for the change use attribute, cartridge entry and cartridge eject parameter list (continued)

Variable	Field Name	Input	Output
Library logical type	UXCLTYP UXELTYP UXJLTYP	If the volume is being defined to DFSMSrmm resides in a system-managed tape library, the location type identifies if the volume resides in an automated tape library or manual tape library. For a volume that is already defined to DFSMSrmm, DFSMSrmm updates the location type with this value. When a volume residing in a system-managed tape library is not defined to DFSMSrmm, DFSMSrmm creates a volume record. The latest information is recorded by DFSMSrmm when volumes are ejected from the system-managed tape library. This ensures information will be available if the volume is ever entered into any other library controlled by the same control data set.	Input only
Library name	UXCLIB UXELIB UXJLIB	Location name • If the volume is being defined for the first time, DFSMSrmm records this value as the home location and location name for the volume. • If the volume is already defined to DFSMSrmm, DFSMSrmm will update the volume location name to this value if the existing location name is different. If the rack number for the volume is not the same as the volume serial number, then DFSMSrmm sets a return code and reason code and issues message EDG8189I. If the rack number is not available, DFSMSrmm sets a return code and reason code and issues message EDG8198I.	Input only

Table 34. Processing for the change use attribute, cartridge entry and cartridge eject parameter list (continued)

Variable	Field Name	Input	Output
New use attribute	UXCUSEA UXEUSEA UXJUSEA	The value can be: S for scratch For change use attribute processing, volumes that are scratch candidates are returned to scratch by any OAM CBRUXCUA request, such as those generated by the EDGSPCLS utility or the ISMF mountable tape volume list processing. Attempts to use OAM CBRUXCUA requests to change the status of a non-scratch-candidate volume to a scratch volume fail with message EDG8194I. For cartridge eject processing, if the volume currently is defined to DFSMSrmm as a master or user volume, then this request is rejected and DFSMSrmm does not change the use attribute. DFSMSrmm sets a reason code and return code and issues message EDG8194I. For cartridge entry processing, if the volume is currently defined as a master or user volume, then DFSMSrmm returns the volume status recorded in the control data set to the caller. P for private The volume information is changed or added to DFSMSrmm regardless of its current status.	This value is input only for change use attribute and cartridge eject processing. DFSMSrmm updates this information during cartridge entry processing if the volume is defined to DFSMSrmm and the status recorded in control data set is different.
Notification call	UXJNCALL	DFSMSrmm uses this field to avoid rejecting or failing an export of a logical volume.	Input only
Shelf location	UXCSHLOC UXESHLOC UXJSHLOC	Not used	Not used Cleared if the location name starts with DEST= Set to DEST=destination name, bin number, and medianame if TCDB record is kept.
Stacked volume	UXJSTKVS	DFSMSrmm uses this field as the 'in container' value.	Input only
Storage group name	UXCGROUP UXEGROUP UXJGROUP	DFSMSrmm updates the DFSMSrmm control data set with the storage group name, except during cartridge entry processing when the storage group name is already set.	DFSMSrmm updates this value during cartridge entry processing if the storage group name is already set.
Tape drive selection information	UXCTDSI UXETDSI UXJTDSI	DFSMSrmm uses this value to replace tape drive selection information during change use attribute and cartridge eject processing. Tape drive selection information corresponds to the DFSMSrmm recording format, media type, compaction, and special attributes.	DFSMSrmm uses this value to update tape drive selection information during cartridge entry processing. DFSMSrmm merges the known information from OAM with the information in the control data set to produce tape drive selection information.

Table 34. Processing for the change use attribute, cartridge entry and cartridge eject parameter list (continued)

Variable	Field Name	Input	Output
Volume attribute	UXEVATTR	Volume type. DFSMSrmm uses the physical volume to process the volume as a real volume. DFSMSrmm uses the logical volume and imported logical volume attributes to identify the volume type as a logical volume. DFSMSrmm checks to see if logical volumes have been correctly imported or entered to control entry and import processing. For imported volumes, the volumes must not already be defined to DFSMSrmm or must be defined as an exported logical volumes. If the volumes are not correctly defined, DFSMSrmm issues message EDG8182I or EDG8184I and appropriate reason codes.	Input only
Volume expiration date	UXCEXPIR UXEEXPIR UXJEXPIR	DFSMSrmm records this value when a new private volume is added or when a volume is changed from scratch to master status. If the volume is already defined as a master volume, DFSMSrmm ignores this value. If the date exceeds the maximum retention period then DFSMSrmm fails the request and issues message EDG8196I and sets return code UXCFAIL.	If the input date is less than the existing date, DFSMSrmm returns the most recent date in the parameter list and sets return code UXyCHG to show that the parameter list has been changed. Note that this is an output field when changing the status to PRIVATE.
Volume location code	UXCLOC UXELOC UXJLOC	Values can be: S for SHELF The volume was ejected from the system-managed tape library and is in transit. L for LIBRARY The volume is resident in the system-managed tape library.	Input only

Table 34. Processing for the change use attribute, cartridge entry and cartridge eject parameter list (continued)

Variable	Field Name	Input	Output
Volume owner information	UXCOWNER UXEOWNER UXJOWNER	The first 8 characters are used to identify the DFSMSrmm owner. If the volume is changing from scratch to master status or is being added as master then DFSMSrmm checks the owner information. If this is a valid DFSMSrmm owner then DFSMSrmm adds or updates the volume owner information in the control data set. If the owner is not a valid owner name, DFSMSrmm sets a default, which is the DFSMSrmm user ID or its step name if the DFSMSrmm ACEE cannot be located. If DFSMSrmm is running in protect mode during change use attribute processing, DFSMSrmm rejects the request and issues message EDG8195I and sets the CBRUXCUA return code UXCFAIL. If the volume is a master volume, DFSMSrmm examines the first 8 characters of the owner information. If the owner information is valid but is different from the current DFSMSrmm owner then DFSMSrmm uses the information to update the owner information for the volume. If the owner information is not valid, DFSMSrmm sets a return code and issues message EDG8195I. During cartridge entry processing, or when owner information does not exist, DFSMSrmm returns owner information in the parameter list.	For volumes to be added as scratch volumes, there should be no owner information. If the first 8 characters of owner information are not null or blank, DFSMSrmm set a null owner name and sets CBRUXCUA return code to UXCCHG. During cartridge entry processing, or if the input owner information was not provided, DFSMSrmm returns the first 8 bytes of owner information.
Volume record creation date	UXCCREAT UXEcreat UXJcreat	Not used	Input only
Volume serial	UXCVOLSR UXEVOLSR UXJVOLSR	This value is used to identify the volume to be processed.	Input only
Write protection status	UXCWPROT UXEWPROT UXJWPROT	Not used	Not used

Change use attribute specific processing

Table 35 on page 260 describes change use attribute processing.

Table 35. Processing for the change use attribute parameter list

Variable	Field Name	Input	Output
Current use attribute	UXCCUSEA	Not used	Input only

Cartridge entry specific processing

Table 36 on page 261 describes cartridge entry processing.

Table 36. Processing for the cartridge entry parameter list			
Variable	Field Name	Input	Output
		There are no cartridge entry specific parameter list fields for which DFSMSrmm does processing.	

Cartridge eject specific processing

Table 37 on page 261 describes cartridge eject processing.

Table 37. Processing for the cartridge eject parameter list			
Variable	Field Name	Input	Output
Volume serial	UXJVDISP	This is a volume record disposition which can be: K for KEEP Specifies that the volume record should be kept P for PURGE Specifies that the volume record should be purged	DFSMSrmm processing depends on the DFSMSrmm EDGRMMxx parmlib OPTION SMSTAPE(PURGE) operand value specified. If ASIS is specified, DFSMSrmm does not change the output field. If YES, we set to P, if NO we set to K.

Volume-not-in-library specific processing

The DFSMSrmm EDGLCSUX programming interface returns information for a volume, indicating whether the volume is a logical exported volume and the stacked volume on which it is exported.

Table 38 on page 261 describes volume-not-in-library processing.

Table 38. Processing for the volume-not-in-library parameter list			
Variable	Field Name	Input	Output
Library name	UXNLIB	Contains library name into which the volume should be entered, or blanks. DFSMSrmm adds this library name to message EDG8121D so that the operator can enter the volume into the correct library. A value of blanks is displayed as the value *ANY*.	Input only

With volume-not-in-library processing, you can set flags to enable DFSMSrmm to retrieve volume information and to request that DFSMSrmm uses WTOR processing to communicate with the operator. When EDGLCSUX is called from CBRUXVNL, you can set either LCSUP_VNL flag or LCSUP_ACTVNL flag. You must first specify LCSUP_VNL to enable DFSMSrmm to retrieve volume information. Your second request, which is optional, must specify LCSUP_ACTVNL to request that DFSMSrmm uses WTOR processing to communicate with the operator. Between calls to EDGLCSUX from CBRUXVNL, do not modify any of the data returned by DFSMSrmm because the information is used in the messages sent to the operator.

When DFSMSrmm is first called from CBRUXVNL, DFSMSrmm retrieves information about the subject volume from the DFSMSrmm control data set if the volume is defined to DFSMSrmm. DFSMSrmm passes back volume location, movement, and status information in output fields in the EDGLCSUP parameter list. When DFSMSrmm is called a second time, and you have set the LCSUP_ACTVNL flag, DFSMSrmm issues message EDG8124I - VOLUME *req_volser* RACK *rack_number* LOCATION *loc_name* BIN *bin_number* HOME LOCATION *home* - NOT IN LIBRARY *lib_name*, followed by either message EDG8121D, EDG8122D, or EDG8123D which prompt the operator to enter the volume into the identified library. If the LCSUP_ACTVNL flag is not set, DFSMSrmm does not issue these WTORs for the operator to move the volume into the system-managed library.

- **EDG8121D** ENTER VOLUME *req_volser* INTO LIBRARY *lib_name* AND REPLY "RETRY", OTHERWISE REPLY "CANCEL" OR CONTINUE
- **EDG8122D** ENTER VOLUME *req_volser* INTO LIBRARY *lib_name* AND REPLY "RETRY", OTHERWISE REPLY "CANCEL"

For volumes residing in a IBM TotalStorage Peer-to-Peer Virtual Tape Server (PtP VTS), DFSMSrmm issues the message EDG8123D.

- **EDG8123D** IMPORT VOLUME *req_volser* TO LIBRARY *lib_name* AND REPLY "RETRY", OTHERWISE REPLY "CANCEL"

Based on the reply, the return code for OAM, is set in field LCSUP_LCSRC ready to be passed back direct to OAM by the CBRUXVNL exit. The LCSUP_LCSRC field returns these values.

Reply

Return Code

CONTINUE

0 - UXNNORML

RETRY

4 - UXNRETRY

CANCEL

8 - UXNFAIL in PROTECT mode, UXNNORML in other modes.

Table 39 on page 262 defines the OAM return codes generated when the EDGLCSUX return code is not zero. The return code is the value in register 15 on return from EDGLCSUX.

Table 39. DFSMSrmm OAM return codes from EDGLCSUX register 15

Return Code	Description	Related Message Number	OAM Return Code
LCSUP_RC_OK R0 is LCSUP_RS_OK	Processing successful	none	See reason code in LCSUP_LCSRS field
LCSUP_RC_OK R0 is LCSUP_RS_NOACTION	Subsystem not required by installation	none	UXxNOCHG "1" on page 263, "2" on page 263 UXNNORMAL "3" on page 263
LCSUP_RC_OK R0 is LCSUP_RS_DONT	DFSMSrmm not licensed by installation	none	UXxDONT "1" on page 263
LCSUP_RC_SSNA	Subsystem not available	EDG8102D EDG8108D EDG8110D	UXxFAIL "1" on page 263
LCSUP_RC_LERR	Logic error in DFSMSrmm processing	EDG8105I EDG8106I EDG8107I	UXxFAIL "1" on page 263
LCSUP_RC_ENV	Environment error detected	EDG8101I EDG8111I EDG8112I	UXxFAIL "1" on page 263

Table 39. DFSMSrmm OAM return codes from EDGLCSUX register 15 (continued)

Return Code	Description	Related Message Number	OAM Return Code
Notes:			
1. x can be:			
• C for change use return codes • E for volume entry • J for volume eject return codes • N for not in volume return codes			
2. UXxNOCHG is issued for change use, volume entry and volume eject			
3. UXNNORML is issued for not in library only			

Environment

EDGLCSUX executes in the same environment as the OAM exits, but in AMODE(31) RMODE(ANY).

Processing fetch and mount messages: EDGMSGEX

The system uses DFSMSdfp MSGDISP to update the display screens on tape drives. DFSMSrmm supplies Assembler source code for IGXMSGEX, the MSGDISP installation exit, that you can modify to include your own functions or you can enable to run with another product's supplied exit code.

IGXMSGEX calls EDGMSGEX, the program that supports the MSGDISP interface. EDGMSGEX is a callable program interface to DFSMSrmm that you can use from the MSGDISP exit. You can run this exit in parallel with another copy of the exit. See [“Setting up parallel processing” on page 264](#) for more information. This information describes how to use the interface.

Input

The invocation environment must be identical to that provided at entry to the MSGDISP exit IGXMSGEX.

Output

The output is MSGDISP parameters that are updated as appropriate. Register 15 is always zero.

Processing

For nonspecific mount requests, where a specific scratch pool is required, DFSMSrmm updates the message text with the pool details. If you use the EDG_EXIT100 installation exit to select a specific pool, DFSMSrmm updates the message text and checks the cartridge loader status. If you requested the loader be disabled for a specific pool, DFSMSrmm sets bit 7 of the format control byte to zero.

For specific mount requests, DFSMSrmm checks the volume and uses the volume serial number to determine the rack number and updates the message text with the rack number. If you use the EDG_EXIT100 installation exit to specify a rack number or an external volume serial number for a volume that DFSMSrmm should ignore, DFSMSrmm uses the value you specify to update the message with the rack number.

Environment

EDGMSGEX runs only from the MSGDISP exit IGXMSGEX. It runs in AMODE(31) RMODE(31).

Processing JES3 messages: EDG3X71

The DFSMSrmm-supplied IATUX71 USERMOD calls EDG3X71, which is the program that supports JES3 message processing. EDG3X71 is a callable program interface to DFSMSrmm that you can use from the IATUX71 exit. This information describes how to use the interface.

Input

The invocation environment must be identical to that provided at entry to the exit IATUX71.

Output

R15 on exit is determined by DFSMSrmm processing. The possible values for the return code are the same codes described for IATUX71 return code. The intention is that you pass the EDG3X71 return code to JES3 as the IATUX71 return code.

0

DFSMSrmm sets this return code when the volume is not managed by DFSMSrmm or when DFSMSrmm is not currently in use.

4

DFSMSrmm sets this return code when the volume serial number is to be replaced by the rack number or pool prefix. MSGDISP text is also provided. DFSMSrmm sets this return code when you have coded the MNTMSG definitions to specify that the rack number is to be placed within the JES3 message text.

8

DFSMSrmm sets this return code when the rack number or pool prefix is to be added to the end of the JES3 message. MSGDISP text is also provided. DFSMSrmm sets this return code when you have:

- Coded MNTMSG definitions so that DFSMSrmm appends the rack number after the JES3 message text.
- Have not coded a MNTMSG definition for this message.

16

DFSMSrmm never sets this return code.

Processing

EDG3X71 can only be called from the JES3 exit IATUX71. If EDG3X71 is called when DFSMSrmm is not started or in use, EDG3X71 sets return code 0 for IATUX71. EDG3X71 determines whether information should be inserted or appended for JES3 messages and determines the value to be used for MSGDISP tape drive display processing.

To check that the IATUX71 exit modification is installed correctly, you can run some batch jobs that use both specific and nonspecific tape requests. You should check that messages displayed on the consoles, in SYSLOG, in DLOG and MLOG, and in JESMSG contains the expected DFSMSrmm updates.

Environment

EDG3X71 runs under a JES3 subtask in the JES3 address space on the JES3 global. EDG3X71 must be link edited in an APF-authorized library. It runs in AMODE(31) RMODE(31).

Setting up parallel processing

When you are converting to DFSMSrmm, you might want to run two versions of your installation exits at the same time. The CBRUXENT exit, the CBRUXEJC exit, and the IGXMSGEX exit provide support that enables your existing exits to control processing before DFSMSrmm gains control. You can set up parallel processing using an SMP/E USERMOD or outside of SMP/E.

The CBRUXENT exit and the CBRUXEJC exit samples shipped with DFSMSrmm can ensure that the DFSMSrmm processing is run after your existing exits are run and that DFSMSrmm records any

information modified by your existing exits. The tape management decisions made by your existing exits are used instead of any tape management decisions that DFSMSrmm might make.

The IGXMSGEX exit includes support that enables your existing exit to perform processing before DFSMSrmm gains control. You use the IGXMSGEX exit to update tape drive displays. You only need either one system or the other to be updating the drive display. As long as you use the same scratch pool names on both systems, either your existing exit or DFSMSrmm can be used to update the drive display.

Setting up parallel processing using SMP/E

Create and install an SMP/E USERMOD as shown in [Figure 90 on page 265](#) to set up parallel running capability.

```
//RMMSTUFF JOB , 'SLIP IT IN',MSGCLASS=H,MSGLEVEL=(1,1)
//STEP1 EXEC PGM=GIMSMP,REGION=6120K
//SMPCNTL DD *
SET BDY(GLOBAL) .
RECEIVE .
/*
//SMPPTFIN DD DATA,DLM=###
++USERMOD (VMRMM03) REWORK(2000287) .
++VER (Z038) FMID(HDZ11D0) PRE(UWxxxxx) /*
Change FMID as needed.
Edit PRE as needed or use the BYPASS(PRE)
operand on the APPLY command */ .
++SRC(CBRUXENT) TXLIB(AEDGSR1) DISTLIB(AEDGSR1) /*
AEDGSR1 points to latest source level
assumption is that PTFs are already accepted
if not, use SMPSTS as the TXLIB */ .
++SRC(CBRUXEJC) TXLIB(AEDGSR1) DISTLIB(AEDGSR1) .
++SRC(IGXMSGEX) TXLIB(AEDGSR1) DISTLIB(AEDGSR1) .
###
//STEP2 EXEC PGM=GIMSMP,REGION=6120K
//SMPCNTL DD *
SET BDY(GLOBAL) .
UCLIN .
ADD UTILITY(ASMSYSP) NAME(ASMA90)
PARM(XREF(FULL),NOOBJECT,DECK,RENT,SYSPARM(YES)) .
ADD OPTIONS(ASMSYSP)
ASM(ASMSYSP) .
ENDUCL .
/*
//STEP3 EXEC PGM=GIMSMP,REGION=6120K
//SMPCNTL DD *
SET BDY(TGT1) OPTIONS(ASMSYSP) .
APPLY SELECT(VMRMM03) .
/*
```

Figure 90. Sample USERMOD for setting up running exits in parallel

To set up parallel processing using SMP/E:

1. Copy the DFSMSrmm-supplied IGXMSGEX exit to a source library and modify the exit if necessary
2. Copy the source of the IGXMSGEX exit from your existing tape management system after the end of the DFSMSrmm-supplied source and change the name of the CSECT to ANOMSGEX or any other meaningful name.
3. Install the SMP/E USERMOD. You install both the DFSMSrmm-supplied exit and your exit from your existing tape management system.
4. To implement changes to the CBRUXENT exit and the CBRUXEJC exit, you do not need to IPL. You can copy the updated load modules into LINKLIB and refresh LLA.
5. To implement changes to the IGXMSGEX exit, you must copy IGX00030 into LPALIB and re-IPL with CLPA.

Recommendation: Because you might not want to IPL at the time you switch DFSMSrmm to production, careful planning of the change to IGX00030 is required. Replace the parallel running IGXMSGEX exit while you continue running in parallel when you are sure that DFSMSrmm is providing the expected scratch pooling support. DFSMSrmm should be running in warning mode and should not issue any error messages about volumes from incorrect scratch pools.

6. When you no longer need to run both exits, remove the USERMOD and install the standard load modules again.

To edit the source code to set your own selected alternate exit load module names, copy the latest DFSMSrmm source code from either SMPSTS or AEDGSR1, and alter the TXTLIB to point to your source library that contains your modified source code.

Setting up parallel processing outside of SMP/E

To set up parallel processing for the OAM tape exits CBRUXENT and CBRUXEJC, perform these steps:

1. Rename the existing exit load module names as follows:
 - CBRUXENT to ANOUXENT
 - CBRUXEJC to ANOUXEJC
2. Assemble the DFSMSrmm-supplied exits with PARM='SYSPARM(YES)'.
3. Link the DFSMSrmm-supplied exits.

To set up the parallel processing for the message display exit IGXMSGEX, follow these steps:

1. Assemble the DFSMSrmm-supplied exits with PARM='SYSPARM(YES)'.
2. Link the DFSMSrmm-supplied exit together with the renamed IGXMSGEX exit that is provided by the existing tape management system as shown in [Figure 91 on page 266](#). The first step of the link-edit is to extract the IGXMSGEX exit that is provided by the existing tape management system from IGX00030 and rename it to ANOMSGEX. The second step of the link-edit is to update the existing load module with the DFSMSrmm-supplied IGXMSGEX exit and to add in the ANOMSGEX section.

```
//LINK1 EXEC PGM=IEWL,REGION=2M,
// PARM='LET,LIST,XREF,NCAL,RENT,NCAL'
//LPALIB DD DISP=SHR,DSN=SYS1.LPALIB
//*
//SYSLMOD DD DISP=SHR,DSN=DFRMM1.NEW.LOAD
//AEDGMOD1 DD DISP=SHR,DSN=SYS1.AEDGMOD1
//*
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD UNIT=SYSDA,SPACE=(CYL,(1,1))
//SYSLIN DD *
REPLACE EDGMSGEX
REPLACE IGXMSG01
REPLACE IGX00030
RENAME IGXMSGEX,ANOMSGEX
CHANGE IGXMSGEX(ANOMSGEX)
INCLUDE LPALIB(IGX00030)
NAME ANOMSGEX(R) RC=4
INCLUDE AEDGMOD1(IGXMSGEX)
INCLUDE LPALIB(IGX00030)
INCLUDE SYSLMOD(ANOMSGEX)
ORDER IGX00030,IGXMSG01
ORDER IGXMSGEX,EDGMSGEX
MODE RMODE(ANY)
ENTRY IGX00030
NAME IGX00030(R) RC=4
```

Figure 91. Sample JCL for setting up running exits in parallel

You can also edit the DFSMSrmm-supplied exits as shown in [Figure 92 on page 266](#). The example shows the CBRUXENT exit to set the &ANOEXIT variable with the new name of the existing exit and the &PARALLEL variable to YES. Then you can assemble and link-edit the updated DFSMSrmm-supplied exits.

```
CBRUXENT TITLE 'DFSMSRMM CBRUXENT SAMPLE USER EXIT'
&ANOEXIT SETC 'ANOUXENT'      Replace ANOUXENT with required name
&PARALLEL SETC '&SYSPARM'      Replace &SYSPARM with YES if req'd
      AIF ('&PARALLEL' EQ '').SETNO
      AIF ('&PARALLEL' EQ 'YES').SETYES
      .SETNO ANOP
&PARALLEL SETC 'NO'
      .SETYES ANOP
```

Figure 92. Example JCL for Setting Up Running Exits in Parallel

Chapter 11. Using DFSMSrmm installation exits

DFSMSrmm provides these installation exits to allow you to customize selected DFSMSrmm functions:

EDG_EXIT100

Described in [“Using the DFSMSrmm EDG_EXIT100 installation exit” on page 267.](#)

EDG_EXIT200

Described in [“Using the EDG_EXIT200 installation exit” on page 300.](#)

EDG_EXIT300

Described in [“Using the EDG_EXIT300 installation exit” on page 305.](#)

When you assemble DFSMSrmm installation exits, the IBM-supplied default assembler options should be used, unless noted otherwise. Ensure that the ALIGN option is in effect.

DFSMSrmm uses the services of the z/OS Dynamic Exit Facility for all of its installation exits. This permits the use of multiple installation exit routines for each DFSMSrmm exits and enables you to control these exits and their exit routines with the EXIT statement of the PROGxx parmlib member, the SET PROG=xx operator command, the SETPROG EXIT operator command, or the CSVDYNEX macro.

DFSMSrmm uses Dynamic Exit Services to load and activate the default exit module, if found in the LINKLIST, for each exit at initialization time. The default exit module is activated in first position. If you do not use the default exit module, you must specify your own exit module with PROGxx in z/OS parmlib, or the SETPROG operator command, or the CSVDYNEX macro.

If a problem occurs during definition of the dynamic exits or activation of the default exits, DFSMSrmm startup provides the operator with a choice of actions; CANCEL, RETRY, or CONTINUE.

You can use the ADDABENDNUM and ABENDCONSEC parameters on the CSVDYNEX REQUEST=ADD macro or the ABENDNUM parameter of the SETPROG EXIT operator command to limit the number of times the exit routine abnormally ends before it becomes inactive. An abend is counted when both of the following conditions exist:

- The exit routine does not provide recovery, or the exit routine does provide recovery but percolates the error.
- The system allows a retry; that is, the recovery routine is entered with bit SDWACLUP off.

By default, the system disables the exit routines for EDG_EXIT300 after one abend. By default, the system does not disable the exit routines for EDG_EXIT100 and EDG_EXIT200.

DFSMSrmm installation exit modules can be loaded from any APF authorized library. The default is the LINKLST.

For information about the:

- Dynamic Exit Facility, see [z/OS MVS Installation Exits](#).
- CSVDYNEX macro, see [z/OS MVS Programming: Authorized Assembler Services Guide](#).
- PROGxx parmlib member, see [z/OS MVS Initialization and Tuning Guide](#).
- D PROG, SETPROG EXIT, and SET PROG=xx commands, see [z/OS MVS System Commands](#).

Using the DFSMSrmm EDG_EXIT100 installation exit

Use the DFSMSrmm EDG_EXIT100 installation exit to perform these tasks:

- Plan for scratch pools as described in [“Planning to manage scratch pools with EDG_EXIT100 ” on page 268](#)
- Perform pooling management as described in [“Managing scratch pools” on page 269](#) to:
 - Manage scratch pools based on job name and data set name.

- Select a pool to use for a nonspecific tape volume request.
- Select a specific pool to use for a nonspecific tape volume request and request that the tape drive cartridge loader is disabled.
- Ignore foreign or duplicate volumes as described in [“Ignoring duplicate or undefined volume serial numbers”](#) on page 275 and optionally provide an external volume serial number for use in messages intercepted and updated by DFSMSrmm.
- Override the system default retention method when you create a new tape volume set or rewrite an existing set from the first file, as described in [“Specifying the retention method to be used for new tape data sets”](#) on page 271.
- Override DFSMSrmm VRSEL processing for specific data sets as they are created or rewritten, as described in [“Excluding specific data sets from VRSEL processing as they are created or rewritten”](#) on page 274.
- Use JCL special expiration dates to manage volumes by setting vital record specification management values to retain data sets as described in [“Using vital record specification management values to retain tape volumes”](#) on page 279.
- Pass pooling decisions to pre-ACS processing so that they can be used in ACS routines to assign storage group and management class as described in [“Using the EDG_EXIT100 installation exit from pre-ACS processing”](#) on page 281.
- Set any expiration date, including a zero value, for a data set as described in [“Assigning expiration dates”](#) on page 289.
- Obtain information that is described in [“Creating sticky labels”](#) on page 281 to modify input from DFSMSrmm disposition processing.
- Request the recording of only the first file on a multifile volume as described in [“Controlling tape volume data set recording”](#) on page 285.
- Change the location for a volume as described in [“Changing location information with EDG_EXIT100”](#) on page 287.
- Support the use of the storage group name for pooling for volumes that reside in manual tape libraries as described in [“Using storage group for manual tape library pooling”](#) on page 292.

Planning to manage scratch pools with EDG_EXIT100

You define pools to DFSMSrmm in parmlib that the EDG_EXIT100 installation exit selects for new tape data sets that are to be created on nonspecific volumes. During OPEN processing DFSMSrmm uses the pool that you specify to validate that a scratch volume has been mounted from the selected pool. You can define each pool with different attributes and each pool can either be a scratch pool or a rack pool. You can use DFSMSrmm ACS storage group support instead of the EDG_EXIT100 exit selected pooling. Refer to [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82 for information about using the DFSMSrmm storage group support.

If you plan to use EDG_EXIT100 to select volumes from specific scratch pools with DFSMSrmm, you need to consider:

Selecting pool types

The pools that you select using the EDG_EXIT100 installation exit must match to a VLPOOL rack number prefix defined in the DFSMSrmm parmlib member as described in [“Defining pools: VLPOOL”](#) on page 209.

You can use EDG_EXIT100 to set up a specific pool for a particular user of your systems. You can define a scratch or rack pool and can define any type of volume in that pool. You do not need to have two pools for a user; one for scratch volumes and one for private volumes. For example, define some volumes that will cycle between scratch and master status throughout their life, and you could also define specific, private volumes that are never to be used as scratch.

You use scratch pools with DFSMSrmm system based scratch pooling. You can define scratch volumes in rack pools and use the RMM GETVOLUME subcommand to claim them or assign them to a user. You can also use EDG_EXIT100 to use rack pools or scratch pools for nonspecific tape output requests.

When you define pools for use with EDG_EXIT100, you also might need to prevent the pools from being used at the wrong times. For example, if your EDG_EXIT100 exit does not make a pool selection you might want to ensure that DFSMSrmm accepts only scratch volumes that are not in your specific pools.

Here are three ways you can prevent pools from being used at the wrong time:

1. Ensure that your EDG_EXIT100 exit always provides a specific pool. Specify a default pool prefix of '*' in the exit module to use the DFSMSrmm default pool for all requests that have no specific pool identified.
2. Use only rack pools for your EDG_EXIT100 scratch pools. You can also have DFSMSrmm scratch pools or a default scratch pool for use with those requests where your exit does not provide a specific pool.
3. Define all the specific scratch pools that use a SYSID value that does not match any of your systems. This ensures that, if the exit does not supply a specific scratch pool and DFSMSrmm uses its own pooling, all the specific pools would be ignored.

Controlling pool selection

The sample EDGUX100 exit module provides a working solution for application type scratch pools based on job names and data set names.

Tip: The exit module does not validate the field contents; follow the job naming conventions or your entry will never match real job names.

Managing tape drive availability

You can use the EDG_EXIT100 exit to request DFSMSrmm to disable the tape drive cartridge loader to prevent specific scratch pool requests from emptying the loaders through volume rejection. Alternatively, you can direct requests to the correct drives or run the loaders in a mode that prevents them being automatically indexed when a mount is received.

Defining operator messages for tape drive display

To use ACS routines or exit selected scratch pooling, you must define messages IEC501A, IEF233A, IAT5110, and IAT5210 using the DFSMSrmm MNTMSG commands in parmlib as described in “Defining mount and fetch messages: MNTMSG” on page 153. These messages provide the pool identifiers to the operator, and the job name and data set name information to the EDG_EXIT100 exit.

The sample EDGUX100 exit module obtains the job name and data set name for a nonspecific volume request from the text of the message. This exit module includes the data set name only if the z/OS MONITOR DSNAME option is in effect. If you plan to use the sample EDGUX100 exit module, you must activate the z/OS MONITOR DSNAME option through an operator command. See [z/OS MVS System Commands](#) for more information about the z/OS MONITOR command.

Managing scratch pools

You define pools to DFSMSrmm in parmlib that the EDG_EXIT100 installation exit can select for new tape data sets. The new tape data sets are to be created on nonspecific volumes. During OPEN processing DFSMSrmm uses the pool that you specify to validate that a scratch volume has been mounted from the selected pool. Each pool can be defined with different attributes, and can either be a scratch pool or a rack pool.

You can specify whether you want DFSMSrmm to disable the autoloader when the EDG_EXIT100 installation exit selects a scratch pool. By using the exit module to disable the loader, you can prevent the loader from being emptied when it contains volumes from another pool. If you are using multiple scratch pools and a tape drive is allocated for a request that requires a pool that has not been pre-loaded, DFSMSrmm rejects pre-loaded volumes from other pools.

You can pass the EDG_EXIT100 exit selected scratch pool prefix to your ACS routines for making allocation decisions and for assigning storage group names for system-managed tape data sets. You can do this using the pre-ACS call to the EDG_EXIT100 installation exit. See [“Using the EDG_EXIT100 installation exit from pre-ACS processing” on page 281](#) for information about using the EDG_EXIT100

installation exit from pre-ACS processing. You can replace the EDG_EXIT100 processing by using the DFSMSrmm calls to SMS ACS routines requesting a storage group name. Refer to [“Using SMS tape storage groups for DFSMSrmm scratch pooling”](#) on page 82 for information about DFSMSrmm ACS support for storage group.

Step 1: Define the pools

For each pool, you need to decide whether this pool is to be used only for specifically identified purposes or whether it can be also selected based on system scratch pools. The easiest way to prevent a pool from being used as a system-based scratch pool is to use only rack pools for your exit driven scratch pooling. If you do not do this, or you do not use one of the other methods described in [“Planning to manage scratch pools with EDG_EXIT100”](#) on page 268, you must use the EDG_EXIT100 installation exit to always set a scratch pool to prevent an incorrect scratch pool from being accepted by DFSMSrmm.

DFSMSrmm identifies pools by using a pool prefix, which is a one-to-five character name followed by an asterisk. See [“Defining pools: VLPOOL”](#) on page 209 for more information.

For example, if you want to define a pool with a prefix of ABC* for use by a specific application, define the pool to DFSMSrmm in parmlib using one of the statements shown in [Figure 93](#) on page 270.

```
VLPOOL PREFIX(ABC*) TYPE(R) RACF(Y) EXPDTCHECK(N) -  
    DESCRIPTION('Application XyZ')  
VLPOOL PREFIX(ABC*) TYPE(S) RACF(Y) EXPDTCHECK(N) -  
    SYSID(NONE) DESCRIPTION('Application XyZ')
```

Figure 93. Defining pools for a specific application

The parameter SYSID(NONE) coded in the second statement in the example, prevents the pool from being used by the DFSMSrmm system scratch pooling as long as no DFSMSrmm system is defined with SYSID(NONE).

For each pool, decide if you will run with scratch volumes from this pool loaded in a cartridge loader. For those pools that are not loaded, consider using the EDG_EXIT100 installation exit to disable the cartridge loader whenever those pools are requested.

Step 2: Tailor the sample EDGUX100 exit module

Update the sample EDGUX100 exit module based on the pools you define using the VLPOOL command in parmlib. The supplied sample EDGUX100 exit module supports pooling based on job name and data set name. Modify the supplied exit if you set up pools that are based on other criteria.

To use the exit module, you must follow these steps:

1. Copy the sample EDGUX100 exit module. If you are already using an EDGUX100 exit module to support another function, you can either:
 - Use the existing exit module as your starting point, or
 - Use the sample exit module after merging in any changes that you have previously made.
2. Update the exit module, bearing in mind this information:
 - Only perform your processing when the PL100_CAN_POOL bit is set to B'1'.
 - Your decisions on pooling must be possible whether the exit is called for MNTMSG processing, pre-ACS processing, or for OPEN processing. Your pooling decisions must work when a WTO address is provided during MNTMSG processing, an ACERO is provided during pre-ACS processing, or a JFCB address is provided during OPEN processing.
 - Based on your pooling decisions, optionally select a pool for the current request and set the PL100_POOL field. You must also set the PL100_SET_POOL flag to B'1' for DFSMSrmm to use the PL100_POOL field. You can also optionally set the PL100_SET_ACLOFF flag to B'1' to request that the tape drive cartridge loader is not indexed for the request. See [“Supplying a scratch pool name”](#) on page 290 and [“Using the system name to select a scratch pool”](#) on page 291 for additional information.

Step 3: Activate the EDG_EXIT100 installation exit

See “Installing the EDGUX100 default exit routine” on page 293 for information about building an SMP/E USERMOD to apply the updated source code for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109 on page 294](#).

Step 4: Define MNTMSG parmlib options

The sample MNTMSG parmlib options includes all the messages that are required to successfully run the scratch pooling function provided by the sample exit. However, refer to “Operator messages and tape drive displays” on page 77 for information about the MNTMSG requirements so that you can ensure your system has the required options already in use.

Step 5: Set up cartridge loaders

Set cartridge loaders that are used with DFSMSrmm to system mode or manual mode.

Step 6: Updating JES3 code (Optional)

This step is only required if you have JES3 managed tape devices and do not use deferred tape mounting. With deferred tape mounting, either allocation processing issues or OPEN processing issues the mount requests and the tape drive displays are correct.

JES3 updates the drive display before it issues the IAT5210 mount message, so the drive display might not contain the pool that you select in your exit.

DFSMSrmm provides three methods for updating the IAT5210 message and the tape drive display. Using the DFSMSrmm supplied USERMOD EDGUX71 to IATUX71 is the preferred method for updating the IAT5210 message with the correct exit selected pool. The exit selected pool is used to update the tape drive displays when selection is based on the IAT5210 message. See [Chapter 19, “Running DFSMSrmm with JES3,” on page 473](#) for information about installing and using DFSMSrmm-supplied JES3 USERMODs.

Step 7: Activate MONITOR DSNAME

The sample EDGUX100 exit module depends on the system mount messages containing data set name information. If you plan to use data set name based pooling, issue the z/OS MONITOR DSNAME command.

For testing purposes, issue this command at an operator terminal. However, for production implementation, ensure that the command is always issued by including it as a command in the COMMNDxx member of the parmlib. Any other method that results in the monitor option being active before any tape requests are issued is also acceptable.

When a non-specific volume request is received, use the exit module to determine the correct pool to be used.

Specifying the retention method to be used for new tape data sets

You can use the EDG_EXIT100 installation exit to set the retention method to be used for new tape data sets. When you create a new tape volume set, or rewrite an existing set from the first file you can override the system default retention method.

1. Define the system default retention method. Refer to Parmlib Member EDGRMMxx OPTION Command
2. Define the DFSMSrmm default and maximum retention periods. OPTION RETPD MAXRETPD
3. Update the sample EDGUX100 exit module based on your retention requirements:
 - a. Copy the sample EDGUX100 exit module and use the copy as a base for your exit module.
 - b. Update the exit module. Perform your processing only when the PL100_CAN_RETMET bit is set to B'1'. Set PL100_RETENTIONMETHOD to the value required, and ensure the PL100_SET_RETMET bit is set to B'1'. If you do not set a retention method, the system default retention method is used.

c. Make any other changes required, such as setting or clearing the EXPDT.

The sample EDGUX100 exit module includes an example of setting the retention method. To use the sample EDGUX100 exit module for this function, modify the table as shown in Figure 94 on page 272. The order in which the table entries are listed is important because the exit scans the table until it finds the first entry where the job name, data set name and program name masks match the current request. You can change the priority of matching by changing the order of the table entries.

RMTAB	DS	0F	START OF RM TABLE
	SPACE	1	
	DC	CL8 '*'	JOBNAME
	DC	CL44'RMMUSER.RMVRSEL.*'	DATA SET NAME
	DC	CL8 '*'	PROGRAM NAME
	DC	AL1(PL100_RM_VRSEL)	RETENTION METHOD VRSEL
	DC	XL3'00'	RESERVED
	SPACE	1	
	DC	CL8 '*'	JOBNAME
	DC	CL44'RMMUSER.RMEXPDT.*'	DATA SET NAME
	DC	CL8 '*'	PROGRAM NAME
	DC	AL1(PL100_RM_EXPDT)	RETENTION METHOD EXPDT
	DC	XL3'00'	RESERVED
	SPACE	1	
	DC	CL8'RM END'	END OF RM TABLE MARKER

Figure 94. Sample retention method selection table

The RMTAB table contains:

jobname

One-to-eight alphanumeric or national characters, including % and *.

- The character % can be used to ignore a positional character in the job name.
- The character * can be used to ignore all remaining characters in the job name. A jobname of * means that the entry applies to all jobs.

data set name

Can be up to forty-four characters, following z/OS data set naming conventions, including % and *.

- The character % can be used to ignore a positional character in the data set name.
- The character * can be used to ignore all remaining characters in the data set name. A data set name of * means that the entry applies to all data sets. The use of the character * is not the same as in the generic data set names supported by DFSMSrmm for vital records specifications and search data set masks. Here the * works like the characters *.* might in a generic data set name mask.

program name

A value up to eight alphanumeric characters, including % and *.

- The character % can be used to ignore a positional character in the program name.
- The character * can be used to ignore all remaining characters in the program name. A program name of * means that the entry applies to all programs.

retention method

One of:

- PL100_RM_VRSEL to assign the retention method VRSEL
- PL100_RM_EXPDT to assign the retention method EXPDT

DFSMSrmm provides a second sample for EDGUX100. This sample is called EDGCVRSX. It is different from the other EDGUX100 sample in that special date, retention method, VRSELEXCLUDE, and pooling function are table driven and you can change the table dynamically. Refer to SAMPLIB member EDGCMM01 and the IBM Redbook, *Converting to Removable Media Manager: A Practical Guide* (SG24-4998) for documentation on using EDGCVRSX for EDGUX100.

4. Activate the sample EDGUX100 exit module. See [“Installing the EDGUX100 default exit routine”](#) on page 293 for information about building an SMP/E USERMOD to apply the updated source code

for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109](#) on page 294.

5. When you write new tape data sets, DFSMSrmm calls your installation exit and saves the resultant retention method into the volume record and propagates it to all future volumes added to the volume set. Use LISTVOLUME to verify that the correct retention method is recorded. Run inventory management to verify that the applied retention method is used for your new data.

Copying data set attributes in a tape copy application

A tape copy application can use the EDG_EXIT100 installation exit to copy the data set attributes from the original to the copy during OPEN processing. In the EDG_EXIT100 installation exit you can specify the data set details for the source data set from which attributes should be copied.

1. Modify your tape copy application to exploit use of EDG_EXIT100:
 - a. Verify that EDG_EXIT100 is active
 - b. Activate your exit module
 - c. Copy the tape data sets and communicate your processing to your exit module
 - d. When all copies are completed or as processing progresses successfully you can update the source data sets and volumes to set specific retention methods, expiration dates, or release volumes that are no longer required.
 - e. Deactivate your exit module
2. Create your EDG_EXIT100 exit module.

You must perform these tasks:

- a. Ensure your exit module only performs your required processing when your copy application is running and data set attributes are to be copied.
- b. Only perform processing during OPEN when PL100_CAN_COPYFROM is B'1'. The following EDGPL100 parameter list fields must be set:

```
PL100_SET_COPYFROM
PL100_COPYFROM_DSN
PL100_COPYFROM_VOLSER
PL100_COPYFROM_DSEQ
PL100_COPYFROM_OWNER This is the only optional field.
```

The specified owner is used for the first file on the first volume of a multivolume set, and will be propagated by DFSMSrmm during EOV to any additional volumes in the set. The owner is optional. If not specified, DFSMSrmm uses the RACF user ID of the copy application at the time the tape data set is copied.

- c. Make any other changes required such as setting or clearing the EXPDT You can consider also exploitation of the ability of the exit to select either VRSEL or EXPDT retention method for the target volume and whether to exclude the target data sets from VRSEL processing. If you do not set the retention method for the target volume in the EDG_EXIT100 installation exit, DFSMSrmm uses the default retention method specified by the current parmlib OPTION RETENTIONMETHOD.
3. Update your copy application packaging to ship updated and new parts
 4. No activation within DFSMSrmm is required, as all required processing, such as use of CSVDYNEX to activate your exit module, is performed by the tape copy application.
 5. Run the tape copy application.

Note: It is your responsibility to provide the correct source data set details. If you pass the wrong source COPYFROM data set details, one of the follow applies:

- a. DFSMSrmm ignores your request if you do not provide valid values for:

PL100_COPYFROM_DSN,
PL100_COPYFROM_VOLSER, and
PL100_COPYFROM_DSEQ

- b. If the data set record is not already defined in the DFSMSrmm CDS, processing continues, but WTO EDG4063I is issued to the job log and the system log.
- c. If the data set record is found, all relevant attributes are copied. In all cases, you can always complete processing or correct processing using the CHANGEDATASET subcommand with COPYFROM specifying the correct source data set record.

If the tape copy application uses this interface, there is no need to use the CHANGEDATASET COPYFROM subcommand. However, at the end of the copy, the application must consider and handle any appropriate retention changes for the source data sets and volumes.

For best results you should ensure that the retention method of the new tape volume set matches that of the original. When data set attributes are copied all existing VRSEL related attributes are copied and so, as long as the new data set is on a volume set with the VRSEL retention method you can expect the same results as for the original data set. When both original and new data is VRSEL managed the VRSELEXCLUDE attribute is copied.

If you plan to switch from VRSEL managed original data sets to EXPDT managed new data sets, you must ensure that the expiration date or retention period is set appropriately for each new data set.

Excluding specific data sets from VRSEL processing as they are created or rewritten

You can use the EDG_EXIT100 installation exit to exclude specific data sets from DFSMSrmm VRSEL processing as they are created or rewritten. You can specify this for any data set, but DFSMSrmm ignores the request unless the data set is on a volume that is managed by the VRSEL retention method. The data set VRSELEXCLUDE attribute is set for all data sets on volumes managed by the EXPDT retention method and is not affected by this support.

When DFSMSrmm excludes a data set from VRSEL processing, it ensures that the data set vital record attribute is reset and the retention date is set to current date. The matching VRS information is left unchanged.

You can use the EDG_EXIT100 installation exit to set the VRSELEXCLUDE attribute for VRSEL managed data sets at OPEN time when you create a new tape data set or rewrite existing data sets.

1. Update the sample EDGUX100 exit module based on your retention requirements:
 - a. Copy the sample EDGUX100 exit module and use the copy as a base for your exit module.
 - b. Update the exit module. Perform your processing only when the PL100_CAN_VRSELEXCLUDE bit is set to B'1'.
 - b. Set the PL100_SET_VRSELEXCLUDE bit to B'1' for data sets. If you do not request VRSELEXCLUDE, the default for the retention method will be used. If the installation exit sets PL100_SET_VRSELEXCLUDE, any VRS management value set in PL100_VRS is ignored.
 - c. Make any other changes required such as setting the retention method when creating the first file on the tape, or clearing the EXPDT.

The sample EDGUX100 exit module includes an example of setting the VRSELEXCLUDE attribute. To use the sample EDGUX100 exit module for this function, modify the table as shown in Figure 95 on page 275. The order in which the table entries are listed is important because the exit scans the table until it finds the first entry where the job name, data set name and program name masks match the current request. You can change the priority of matching by changing the order of the table entries.

VXTAB	DS	0F	START OF VRSELEXCLUDE TABLE
	SPACE	1	
	DC	CL8 '*'	JOBNAME
	DC	CL44 'RMMUSER.VX.*'	DATA SET NAME
	DC	CL8 '*'	PROGRAM NAME
	SPACE	1	
	DC	CL8 'VX END'	END OF VX TABLE MARKER

Figure 95. Sample VRSELEXCLUDE selection table

The VXTAB table contains:

jobname

One-to-eight alphanumeric or national characters, including % and *.

- The character % can be used to ignore a positional character in the job name.
- The character * can be used to ignore all remaining characters in the job name. A jobname of * means that the entry applies to all jobs.

data set name

Can be up to forty-four characters, following z/OS data set naming conventions, including % and *.

- The character % can be used to ignore a positional character in the data set name.
- The character * can be used to ignore all remaining characters in the data set name. A data set name of * means that the entry applies to all data sets. The use of the character * is not the same as in the generic data set names supported by DFSMSrmm for vital records specifications and search data set masks. Here the * works like the characters *.* might in a generic data set name mask.

program name

A value up to eight alphanumeric characters, including % and *.

- The character % can be used to ignore a positional character in the program name.
- The character * can be used to ignore all remaining characters in the program name. A program name of * means that the entry applies to all programs.

DFSMSrmm provides a second sample for EDGUX100. This sample is called EDGCVRSX. It is different from the other EDGUX100 sample in that special date, retention method, VRSELEXCLUDE, and pooling function are table driven and you can change the table dynamically. Refer to SAMPLIB member EDGCMM01 and the IBM Redbook, *Converting to Removable Media Manager: A Practical Guide* (SG24-4998) for documentation on using EDGCVRSX for EDGUX100.

2. Activate the sample EDGUX100 exit module. See [“Installing the EDGUX100 default exit routine” on page 293](#) for information about building an SMP/E USERMOD to apply the updated source code for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109 on page 294](#).
3. When you write new tape data sets, DFSMSrmm calls your new installation exit module and saves the VRSELEXCLUDE attribute into the data set. Use LISTDATASET to verify that the correct information is recorded. Run inventory management to verify that the data sets are excluded from VRSEL.

Ignoring duplicate or undefined volume serial numbers

You do not need to use the EDG_EXIT100 installation exit to ignore tape volumes because you can automate ignore processing using the OPENRULE command in parmlib with an action of IGNORE.

There is a difference between using OPENRULE with an action of IGNORE and the EDG_EXIT100 installation exit ignore processing related to scratch volumes. The OPENRULE action of IGNORE for a non-specific request can automate the ignore request being made so that the scratch volume mounted and the scratch request is neither validated nor recorded by DFSMSrmm. For EDG_EXIT100 ignore processing for a non-specific request, the exit is only offered the opportunity to ignore a volume when the non-specific request is for a non-system managed drive. For system-managed libraries, you cannot use EDG_EXIT100 to ignore non-specific (scratch) requests. However, you can use OPENRULE with the IGNORE action to support this processing.

When DFSMSrmm ignores a volume, DFSMSrmm does not perform these management functions for the volume:

- Record information about the volume in the DFSMSrmm control data set.

When EDG_EXIT100 requests ignore, DFSMSrmm validates that the correct volume has been mounted in all cases, including bypass label processing (BLP). The only exception is for the case when both the mounted and the requested volumes are with no label (NL). If the wrong volume has been inserted, EDG4064D is issued to ask the operator whether RMM should ignore the volume or reject it.

For volumes that are defined to DFSMSrmm and that are ignored, DFSMSrmm performs inventory management based on previously recorded information.

For a specific volume request, DFSMSrmm normally ignores volumes that are not defined to DFSMSrmm. For nonspecific requests, DFSMSrmm ensures that a DFSMSrmm-managed scratch volume is mounted in response to the request unless you use OPENRULE with action IGNORE, or the tape drive is not system-managed and you use EDG_EXIT100 to request that the volume is ignored.

To request that DFSMSrmm ignore a volume, perform these steps:

1. Tailor the sample EDGUX100 exit module to use undefined volumes or duplicate volumes. DFSMSrmm invokes EDG_EXIT100 each time a volume is opened and might offer the exit module the opportunity to ignore the request. The sample exit module checks that PL100_CAN_IGNORE is set and that the JCL-specified EXPDT value for the special date 98000 or for the ACCODE value xCANORES. If the 98000 special date or the ACCODE value xCANORES is found, the sample EDGUX100 exit module uses the installation exit parameter list to request that DFSMSrmm ignore the mounted volume.
2. Activate the exit module.
3. If you want to distinguish between volumes that are managed by DFSMSrmm and volumes that are not managed by DFSMSrmm, use STGADMIN.EDG.IGNORE.TAPE.RMM.volser and STGADMIN.EDG.IGNORE.TAPE.NORMM.volser to check user authorization. The SAF authorization checking is performed by DFSMSrmm at the first attempt to use these entity names and only if they are not protected by an existing profile is the RACF FACILITY class entity is STGADMIN.EDG.IGNORE.TAPE.volser required. Use STGADMIN.EDG.IGNORE.TAPE.volser only if you do not have the STGADMIN.EDG.IGNORE.TAPE.RMM.volser and STGADMIN.EDG.IGNORE.TAPE.NORMM.volser entities protected.
4. Authorize users to the STGADMIN.EDG.IGNORE.TAPE.volser, STGADMIN.EDG.IGNORE.TAPE.RMM.volser, and STGADMIN.EDG.IGNORE.TAPE.NORMM.volser resources that you have defined.

Use the EDG_EXIT100 installation exit to have DFSMSrmm ignore a volume when any of these conditions exist:

- You want to ignore a volume that has not been ignored using the parmlib OPENRULE command with an action of IGNORE.
- You specify the parmlib REJECT ANYUSE(*) or OPENRULE TYPE(NORMM) ANYUSE(REJECT) commands to reject an undefined volume.
- You use a volume in an automated tape library and you want to prevent it from being automatically defined to DFSMSrmm.
- You want to ignore a volume used for a non-specific, non-system managed output request.

If these conditions are not present, DFSMSrmm automatically ignores all volumes that are not defined to it without using the DFSMSrmm installation exit EDG_EXIT100 and without any need for authorization using FACILITY class profiles with the STGADMIN.EDG.IGNORE prefix.

Before DFSMSrmm ignores the volume, it ensures that the user who opened the volume is authorized to request this function. DFSMSrmm uses a security resource in RACF FACILITY class, and issues a SAF RACROUTE TYPE=AUTH request with one of these entity names.

- STGADMIN.EDG.IGNORE.TAPE.RMM.volser
- STGADMIN.EDG.IGNORE.TAPE.NORMM.volser

- STGADMIN.EDG.IGNORE.TAPE.*volser*

Where *volser* is the volume serial number of the mounted volume or requested tape volume.

When deciding which entity name to use, DFSMSrmm must determine which *volser* (mounted volume or requested volume) to use and whether to treat the volume as RMM or NORMM.

Determining *volser*

DFSMSrmm uses the mounted volume *volser* by default. The mounted volume *volser* is taken from the VOL1 label if it is successfully read from the volume. If you have DEVSUPxx VOLNSNS set and the VOL1 label cannot be read, OPEN could have obtained the mounted volume *volser* from the VOLID mark. In that case and any other case where there is no VOL1 label, DFSMSrmm uses the mounted volume *volser* provided by OPEN. In order to correctly support duplicate volumes (that are defined to DFSMSrmm) and support OPENRULE with IGNORE, for OPENRULE IGNORE processing only, DFSMSrmm first uses the system managed library vision system *volser*, then when the mounted volume and requested volume are different DFSMSrmm uses the requested volume *volser* to check if the volume is defined to DFSMSrmm and that the VOL1 label *volser* is correctly defined for that volume.

Alternatively, you can customize the EDGUX100 sample exit to use the requested *volser*, by setting the PL100_SET_IGNORE_REQUESTED bit, as described in “[EDG_EXIT100 parameter list](#)” on page 295. The requested *volser* is obtained either from JCL or from the dynamic allocation request.

Determining RMM versus NORMM

DFSMSrmm uses this procedure to determine whether to use the RMM or NORMM entity name:

1. If a VOL1 label exists for the mounted tape, DFSMSrmm uses it to determine whether the label type is AL, NL, or SL. If the label type cannot be determined, the label type is assumed to be NL.
2. If the *volser* is not defined to DFSMSrmm, it is handled as NORMM.
3. If the mounted and DFSMSrmm recorded label types do not match, it is handled as NORMM.
4. If the label types match and are NL, it is handled as RMM.
5. If the label types match and are AL or SL, and the HDR1 data set name information matches the data set name recorded for the volume, it is handled as RMM.
6. If none of the previous cases are true, it is handled as NORMM.

Note: Even if you specify BLP, DFSMSrmm uses the VOL1 and the HDR1 label from the mounted volume as part of determining whether to use RMM or NORMM text in the entity name.

DFSMSrmm uses STGADMIN.EDG.IGNORE.TAPE.RMM.*volser* only for volumes that are to be handled as RMM. .

DFSMSrmm uses STGADMIN.EDG.IGNORE.TAPE.NORMM.*volser* only for volumes that are to be handled as NORMM.

The third entity name, STGADMIN.EDG.IGNORE.TAPE.*volser*, is used only when the first SAF authorization check for either RMM/NORMM determines that the resource is not protected.

For example, if you want DFSMSrmm to ignore volume A00001 that is defined to DFSMSrmm, you could define the STGADMIN.EDG.IGNORE.TAPE.** UAC(NONE) profile, which protects both STGADMIN.EDG.IGNORE.TAPE.RMM.*volser* and STGADMIN.EDG.IGNORE.TAPE.NORMM.*volser* resources. When DFSMSrmm calls SAF to check the authorization using the entity name STGADMIN.EDG.IGNORE.TAPE.RMM.A00001, it will be authorized or not based on the access list you set up for the STGADMIN.EDG.IGNORE.TAPE.** profile.

When DFSMSrmm ignores a volume as requested by the EDG_EXIT100 installation exit and the user is authorized to ignore processing, this overrides any decision taken by RACF as a result of authorization checks issued by OPEN processing. You can tailor the EDGUX100 exit module to prevent DFSMSrmm from overriding the decision in the case of SAF return code 8 (not authorized).

Step 1: Tailor the sample EDGUX100 exit module

Based on the decisions you have made about how to manage duplicate or undefined volume serial numbers, tailor the supplied sample EDGUX100 exit module as follows:

1. Make a copy of the sample exit module to use as a base for your exit module.
2. Update the exit module. Only perform your processing when the PL100_CAN_IGNORE bit is set to B'1'. Decide whether to support the pre-ACS call to obtain a VRS management value or scratch pool prefix for use as the MSPOLICY and MSPPOOL variables in ACS processing. You must ignore the call if the volume is to be ignored. During the pre-ACS call, an ACERO is passed instead of a JFCB. See *z/OS DFSMS Installation Exits* for information about the ACERO. See [“Assigning expiration dates”](#) on page 289 for additional processing information.
 - Use the value in the JFCB expiration date field, JFCBXPDT, to determine if a special date has been specified. You could also use the PL100_ACCODE field to check if the special ACCODE value has been supplied.
 - Set one of these flags to B'1' so that the volume is ignored if the special date 98000 or the special ACCODE=xCANORES value has been specified. The flags are
 - PL100_SET_IGNORE
 - PL100_SET_IGNORE_MOUNTED
 - PL100_SET_IGNORE_REQUESTED
 - Set PL100_SET_NOTBYPASS_SAFRC8 bit to B'1' to turn off the function that uses the successful IGNORE authorization to change SAF return code 8 (not authorized) to 0. The flag is evaluated only in conjunction with PL100_SET_IGNORE, PL100_SET_IGNORE_MOUNTED, and PL100_SET_IGNORE_REQUESTED.
 - Update the WTO messages with a rack number. Place the rack number value in the PL100_RACKNO field
 - Clear the expiration date in the JFCB copy that DFSMSrmm uses to prevent DFSMSrmm from using the 98000 date as a volume expiration date. You do not need to update the JFCB. The information is not passed on to DFSMSrmm. Because it is a copy of the JFCB used solely by DFSMSrmm, no other component can make use of your changes. However, because the EDG_EXIT100 installation exit is called for system-managed volumes, but nonspecific system-managed volumes cannot be ignored, the DFSMSrmm sample exit module clears the expiration date.

Step 2: Activate the sample EDGUX100 exit module

See [“Installing the EDGUX100 default exit routine”](#) on page 293 for information about building an SMP/E USERMOD to apply the updated source code for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109](#) on page 294.

Step 3: Define a RACF FACILITY class entity to check authorization

Define a RACF FACILITY class entity to protect volumes. DFSMSrmm checks that the user trying to use a duplicate or undefined volume is authorized to request that DFSMSrmm ignore the volume. The RACF FACILITY class entities are:

- STGADMIN.EDG.IGNORE.TAPE.volser
- STGADMIN.EDG.IGNORE.TAPE.RMM.volser
- STGADMIN.EDG.IGNORE.TAPE.NORMM.volser

Only users with the correct authorization to the entities can request that DFSMSrmm ignore volumes.

Define multiple resources, as required, for individual volumes or groups of volumes by using RACF generic resource names. GLOBALAUDIT(SUCCESS) ensures that RACF maintains an audit trail of all successful uses of this facility.

```
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.X* UACC(NONE)
RALTER FACILITY STGADMIN.EDG.IGNORE.TAPE.X* GLOBALAUDIT(SUCCESS)
```

```
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.810* UACC(NONE)
RALTER FACILITY STGADMIN.EDG.IGNORE.TAPE.810* GLOBALAUDIT(SUCCESS)
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.Y* UACC(NONE)
RALTER FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.Y* GLOBALAUDIT(SUCCESS)
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.710* UACC(NONE)
RALTER FACILITY STGADMIN.EDG.IGNORE.TAPE.RMM.710* GLOBALAUDIT(SUCCESS)
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.Z* UACC(NONE)
RALTER FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.Z* GLOBALAUDIT(SUCCESS)
RDEFINE FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.910* UACC(NONE)
RALTER FACILITY STGADMIN.EDG.IGNORE.TAPE.NORMM.910* GLOBALAUDIT(SUCCESS)
```

Step 4: Authorize users

Permit those users authorized in your installation to the appropriate FACILITY CLASS resources.

```
PE STGADMIN.EDG.IGNORE.TAPE.X* CLASS(FACILITY) ID(user1) ACCESS(level)
PE STGADMIN.EDG.IGNORE.TAPE.810* CLASS(FACILITY) ID(user2) ACCESS(level)
PE STGADMIN.EDG.IGNORE.TAPE.RMM.Y* CLASS(FACILITY) ID(user1) ACCESS(level)
PE STGADMIN.EDG.IGNORE.TAPE.RMM.710* CLASS(FACILITY) ID(user2) ACCESS(level)
PE STGADMIN.EDG.IGNORE.TAPE.NORMM.Z* CLASS(FACILITY) ID(user1) ACCESS(level)
PE STGADMIN.EDG.IGNORE.TAPE.NORMM.910* CLASS(FACILITY) ID(user2) ACCESS(level)
```

In the example, the values that are specified have these meanings:

user1, user2

The user ID of the user given access.

level

The granted access level; one of READ or UPDATE.

READ access is required for a volume to be opened for input requests. UPDATE access is required for a volume to be opened for output requests.

Using vital record specification management values to retain tape volumes

You can use the EDG_EXIT100 installation exit to assign vital record specification management values to new tape data sets. When a volume is opened, DFSMSrmm records the vital record specification management value you assign to new tape data sets. You then define vital record specifications using data set name masks that match the vital record specification management values you define for special dates. These vital record specifications define the management policies for volumes with special dates and are considered for matching during VRSEL processing for volumes that use the VRSEL retention method.

You can use vital record specification management values to be the basis for assigning management class for system-managed tape. DFSMSrmm calls EDG_EXIT100 to obtain a vital record specification management value. During pre-ACS processing, the vital record specification management value is passed to your ACS routine so that allocation decisions can be made. During OPEN processing for new tape data sets, this call is optional and is not made for a non-system-managed tape data set if you have assigned a management class during the RMMVRS ACS call.

Step 1: Define vital record specification management values

A vital record specification management value is a single qualifier name that you define in a vital record specification to tell DFSMSrmm how to manage and retain your tape data sets. The vital record specification management value can be up to eight alphanumeric characters, and must begin with an alphabetic character. DFSMSrmm matches a data set to a vital record specification management value during vital records processing when the data set does not match a vital record specification with a data set name mask and optionally a job name. This matching is done only if the data set is on a volume that has retention method VRSEL and the data set attribute VRSELEXCLUDE is not set.

Use the RMM ADDVRS subcommand with the DSNAME operand or the DFSMSrmm Add Data Set VRS panel in the DFSMSrmm ISPF dialog to define data set vital record specifications. Use the data set name masks that match the vital record specification management values you have defined. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for more information on defining vital record specifications.

For example, you can use the vital record specification management value D99000, for the special date 99000 and define a vital record specification using the data set name, D99000. [Figure 96 on page 280](#) defines a vital record specification for managing the special date 99000. You could also set the ACCODE=xCACATLG value in the DD statement to manage the special date 99000.

```
RMM ADDVRS DSNAME('D99000') WHILECATALOG
```

Figure 96. Managing special date 99000 with vital record management value

You could define a vital record specification data set name mask that matches multiple vital record specification management values. For example you could define a data set name mask of D9900% as shown in [Figure 97 on page 280](#) to cover several vital record specification management values. With this data set name mask you could manage special dates in the range 99001 through 99009.

```
RMM ADDVRS DSNAME('D9900%')
```

Figure 97. Specifying data set masks for vital record management values

Step 2: Tailor the sample EDGUX100 exit module

Update the sample EDGUX100 exit module based on vital record specification management values you define and perform these tasks:

1. Copy the sample EDGUX100 exit module and use the copy as a base for your exit module.
2. Update the exit module. Only perform your processing when the PL100_CAN_VRS bit is set to B'1'.
 - If an ACERO is passed to the exit, use the ACEROEXP and ACERORTP fields to decide if a special date is specified. If a JFCB is passed to the exit, use the value in the JFCB expiration date field, JFCBXPDT to determine if a special date has been specified. Test the JFCBEXP flag to determine if the user specified an expiration date or a retention period. You can also test to see if the user wants the date used as real expiration date by checking for the existence of the dummy file NOTKEYD8.

The sample EDGUX100 exit module includes code for using this method specifying the DD name NOTKEYD8. You can also use the PL100_ACCODE field to check if the special ACCODE value has been supplied. Specifying ACCODE=xCACATLG is the same as using EXPDT=99000.

- Based on the special date like 99000 or the ACCODE value, select the chosen vital record specification management value and use it to set the field PL100_VRS.
- If you want DFSMSrmm to use a true expiration date, rather than the special date from the JCL, update the JFCBXPDT field with the date or with zero. If you update the field with a zero, DFSMSrmm uses the default retention period to calculate an expiration date. See [“Assigning expiration dates” on page 289](#) for additional information. If you use in JCL only the ACCODE value, you do not need to alter the JCL expiration date.

The sample EDGUX100 exit module provides support for catalog control special dates during pre-ACS and OPEN time and provides the checking necessary for true expiration dates. Modify the sample exit module if you wish to support other special dates.

Tip: DFSMSrmm provides a second sample for EDGUX100. This sample is called EDGCVRSX. It is different from the other EDGUX100 sample in that special date, retention method, VRSELEXCLUDE, and pooling function are table driven and you can change the table dynamically. Refer to SAMPLIB member EDGCMM01 and the IBM Redbook, *Converting to Removable Media Manager: A Practical Guide* (SG24-4998) for documentation on using EDGCVRSX for EDGUX100.

3. Make any changes required for using vital record management values before building the USERMOD.

Step 3: Activate the sample EDGUX100 exit module

See [“Installing the EDGUX100 default exit routine” on page 293](#) for information about building an SMP/E USERMOD to apply the updated source code for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109 on page 294](#).

To apply vital record specification management values to a new tape data set, you can either specify the special date 99000 in the EXPDT keyword or use corresponding special values in the ACCODE keyword of the DD statement. The EDGUX100 sample exit module checks the ACCODE value for xCACATLG. If there is a matching value, DFSMSrmm performs the same actions that it performs when it processes 99000. If both ACCODE and EXPDT are used to specify special values, then DFSMSrmm uses the ACCODE value.

Using the ACCODE keyword, you can use the xCAUSER and xCAPERM special values. The sample EDGUX100 exit module treats these two values the same as the never expire date 99365 specified in the EXPDT parameter. A vital record specification management value D99365 is assigned to the data set which ensures that the data set is retained indefinitely.

Step 4: Run DFSMSrmm inventory management vital record processing

Run DFSMSrmm inventory management vital record processing to identify which volumes should be retained based on the vital record specification management values you define. The data set is considered for vital record processing only if the data set is on a volume that has retention method VRSEL and the data set attribute VRSELEXCLUDE is not set. DFSMSrmm uses the vital record specification management value assigned to the data set to select the appropriate vital record specification, if inventory management vital record processing does not find a match on the data set mask. See [“How vital record processing works”](#) on page 352 for information about vital record processing.

Using the EDG_EXIT100 installation exit from pre-ACS processing

You can use ACS routines to automatically determine the target storage group and assign data classes, storage classes, and management classes to SMS-managed data sets. You can use the pre-ACS interface to provide additional information like vault destination or pool information to the ACS routines. You can use the EDG_EXIT100 installation exit for pre-ACS processing. See [“Defining system options: OPTION”](#) on page 161 for information about the DFSMSrmm EDGRMMxx PARMLIB OPTION PREACS operand and the OPTION SMSACS operand that you can use to control how storage group and management class values are assigned. When called during pre-ACS processing, the values selected by the exit are used as input to the ACS routine. Then, when the EDG_EXIT100 installation exit is called at OPEN or when a volume is mounted, the exit can provide the same values it passed during the pre-ACS processing or provide different vital record specification management values and pool values.

The pre-ACS routine passes the address of the ACERO to the EDG_EXIT100 installation exit. The ACERO is mapped by the IGDACERO macro and is the input to the pre-ACS installation exit IGDACSXT. You can use any of the values in the ACERO as input to the EDG_EXIT100 installation exit. The sample EDGUX100 exit module uses these values:

- ACEROJOB, which is the job name.
- ACERODSN, which is the data set name.
- ACEROEXP, which is the expiration date. The expiration date is used only if the retention period is not set.
- ACERORTP, which is the retention period.

The values are used to perform PL100_CAN_IGNORE, PL100_CAN_VRS, and PL100_CAN_POOL processing. All other functions are performed only when the EDG_EXIT100 is not called by the pre_ACS routine.

If your installation's version of the exit module provides values for scratch pooling (PL100_POOL) or vital record specification management value (PL100_VRS), DFSMSrmm updates the ACS input values for MSPPOOL and MSPOLICY when they have not already been set by the IGDACSXT pre-ACS installation exit.

To perform pre-ACS processing, check the PL100_ACEROPTR field for the address of the ACERO.

Creating sticky labels

You can create sticky labels using DFSMSrmm disposition processing. You can choose to either drive the sticky labels using one or more disposition control files, or you can choose to drive the sticky label

using the EDG_EXIT100 installation exit. You can combine these methods as required to meet your requirements.

If you use DFSMSrmm disposition processing, you do not need to use EDG_EXIT100 to implement sticky label support because DFSMSrmm provides some default sticky label support as part of disposition control file processing. However, you can optionally use EDG_EXIT100 to modify the default DFSMSrmm processing. Refer to [Chapter 18, “Setting up DFSMSrmm disposition processing,”](#) on page 467 for more information. If you do not use disposition control file processing, you can still use the EDG_EXIT100 installation exit to implement sticky label support. You can exploit EDG_EXIT100 in these ways:

- By using the default label created by DFSMSrmm disposition processing, determine that you would like the sticky label created in any case, and set off PL100_SET_NOLABEL.
- By using information provided to the installation exit as input to a sticky label routine you have written. Just ignore any sticky label passed to EDG_EXIT100, ensure that PL100_SET_NOLABEL flag is set, and perform your own processing. The steps for tailoring the installation exit are described in [“Step 1: Tailor the sample EDGUX100 exit module”](#) on page 282.
- For modifying the default labels produced by DFSMSrmm disposition processing described in [“Modifying DFSMSrmm label output”](#) on page 284.
- For suppressing the default labels produced by DFSMSrmm disposition processing described in [“Modifying DFSMSrmm label output”](#) on page 284.
- For tailoring how location names are processed, you can decide whether a location is treated as a loan location, or storage location, and how the movement is to be confirmed.

Step 1: Tailor the sample EDGUX100 exit module

You can create sticky labels using the EDG_EXIT100 installation exit and can use either the DFSMSrmm built-in function, disposition control, or you can use a sticky label routine you have written. DFSMSrmm calls the EDG_EXIT100 installation exit whenever a tape data set is closed or reaches end of volume. In any case, you can use either the DFSMSrmm built-in function, your own routine, or any combination. When the EDG_EXIT100 installation exit is called to enable sticky label processing, the PL100_ITS_CLOSE option is set. When the exit is called with PL100_ITS_CLOSE set, there is information in the EDGPL100 parameter list for a default sticky label that DFSMSrmm has prepared and also information that allows you to create or customize sticky labels. See [“Installation exit mapping macro: EDGPL100”](#) on page 517.

If there is an DFSMSrmm prepared label, the PL100_LABPTR is non-zero. In addition, you can determine if this label was created because of disposition control file processing, or as a default label. PL100_INFO_DISPLAB indicates that the disposition control file requested the label. [Figure 98 on page 282](#) shows where the sample exit determines if a label is available and why it was created.

```

JFCBP00L EQU      *
*****
* Now we'll check if we come from CLOSE/END OF VOLUME.
*****
      TM      PL100_VALID,PL100_ITS_CLOSE Is it a call from C/EOV
      BZ      NOTCLOSE                No, continue
      ...
      L        R3,PL100_LABINFO      Address Label info block
      USING    PL100_LABDS,R3        Addressability
      TM      PL100_OFLAG,PL100_FOUT Is it OUTPUT ?
      BZ      NOTCLOSE                No, continue
      DS      DS
      STM      R14,R10,DSPCNTR      STORE REGISTER
      CLI      PL100_VERN0,X'02'     REQUIRED VERSION NUMBER 2
      BL      DSPCNTE                NO, CONTINUE
      L        R5,PL100_LABPTR      Do we have a prepared label?
      LTR      R5,R5
      BZ      DSPCNT2                NO, CONTINUE
      TM      PL100_INFO,PL100_INFO_DISPLAB
      BZ      DSPCNT_OPT2            NO, CONTINUE with option 2

```

Figure 98. Sample EDGUX100 exit module sticky label support

For each time that the code is entered at label DSPCNT, you can choose to do one of these:

- If a disposition control file provided user data, it is copied to the prepared label. See label '=1' in [Figure 99 on page 283](#).
- You can customize the prepared label. See label '=2' in [Figure 99 on page 283](#).
- If the label is created because of disposition control file processing, you can suppress the label. See label '=3' in [Figure 99 on page 283](#) for where you can suppress the label.
- If you are not using a disposition control file and would like to use the DFSMSrmm default label, you add your decision making code at label '=4' in [Figure 99 on page 283](#). This is a much easier option than using your custom routine. To enable the production of the prepared label, uncomment the code at label '=5' in [Figure 99 on page 283](#). By default, DFSMSrmm does not produce the label, but has created it for you.
- In any case, whether you produce a label or not, you can set a location name in PL100_LOCATION, specify if this is a loan location or a storage location, and determine how moves should be confirmed. See label '=6' in [Figure 100 on page 283](#).

```

*-----
* sticky label option 1                                @17A
*-----
* COPY USER DATA INTO STICKY LABEL
*-----
          TM      PL100_INFO,PL100_INFO_USERDATA      @K4
          BZ      DSPCNT0      NO, CONTINUE           =1
          MVC      SLABCUSR,PL100_LAB_USERDATA        @K4
*-----
* CREATE CUSTOM STICKY LABEL
* Here is where you can customize the prepared label
*-----
DSPCNT0  DS      0H                                  =2
...
*-----
* SUPPRESS STICKY LABEL OUTPUT
* Remove comments in the following code to suppress the label @17A
*-----
DSPCNT1  DS      0H                                  @K4
*       OI      PL100_FUNCTION,PL100_SET_NOLABEL      =3
*       B       DSPCNT2  continue with LOCATION handling @17
*-----
* Consider sticky label option 2                        @17A
*-----
DSPCNT_OPT2 DS 0H                                  @17
* Add here code to decide when the prepared labels should be @17
* created.                                              =4
*-----
* For Option 2 we must reset the PL100_SET_NOLABEL flag @17A
* Remove comments in the following code to produce the label @17A
*-----
*       NI      PL100_FUNCTION,255-PL100_SET_NOLABEL  =5

```

Figure 99. Sample EDGUX100 exit module sticky label support

```

DSPCNT2  DS      0H                                  @K4A
*-----
* ASSIGN LOCATION
* Remove comments in the following code to activate or customize@17A*
*-----
*       MVC     PL100_LOCATION,=CL8'SAMPLE '          =6
*       MVI     PL100_LOCTYPE,PL100_LOC_LOAN          @K4A
*       NI      PL100_FUNCTION2,255-PL100_SET_NOCMOVE @K4A
*       OI      PL100_FUNCTION2,PL100_SET_CM MOVE     @K4A

```

Figure 100. Sample EDGUX100 exit module sticky label support location

To create your own sticky labels, you can add your sticky label routine in one of these ways:

- Create a new exit routine to perform sticky label processing. Use dynamic exit services to add your sticky labels routine as a separate exit module to the EDG_EXIT100 exit.
- Add your sticky label routine directly to the exit module or add a LOAD or LINK statement to call your sticky label routine externally. [Figure 101 on page 284](#) shows where your routine for creating sticky labels can be called from EDGUX100. See label '=7' in [Figure 101 on page 284](#).

Be sure that your code sets the PL100_SET_NOLABEL to B'1' so that DFSMSrmm does not also produce a label.

```

DSPCNTE DS    0H                                @L6A
*****@L6M
* To create your own sticky label support, add your code right @L6M
* before the statement 'DSPEND EQU *'. @L6M
*      WTO  'Sticky label routine called . . . . ' @04A
* @04A
* Refer to PL100_LABDS DSECT in EDGPL100 macro for an explanation @04A
* of all the fields you may need for sticky label processing. @04A
* @04A
*****=7
DSPEND DS    0H                                @K4A

```

Figure 101. Sample EDGUX100 exit module sticky label support own labels

Step 2: Activate the sample EDGUX100 exit module

See “Installing the EDGUX100 default exit routine” on page 293 for information about building an SMP/E USERMOD to apply the updated source code for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109 on page 294](#).

Modifying DFSMSrmm label output

Use the EDG_EXIT100 installation exit to modify the label that DFSMSrmm produces as part of disposition processing. You can set new values for the number of rows, the length of each row in the label. You cannot exceed the maximum label area size of 2000 characters. You can set an LRECL other than the default LRECL of 80. The LRECL you set must be the same length or less than the number of columns in each row in the label. You can change the values by modifying the label area passed to the EDG_EXIT100 exit based on the PL100_LABDATA field. PL100_LABPTR is the address of the label DFSMSrmm has prepared. You use the EDGLAB mapping macro to map the label area. The label area starts with control information as shown in [Figure 102 on page 284](#). You set the LRECL to your selected output label LRECL, and decide how many columns and rows you will have in your label. The number of columns can be less than the LRECL

SLABENT DS	0D	** BEGIN OF SLAB	**
SLABID DS	CL8'EDGLAB'	** SLAB EYECATCHER	**
SLABSPL DS	XL1	** SLAB SUBPOOL NUMBER	**
SLABSIZE DS	XL3	** SLAB TOTAL SIZE	**
SLABKEY DS	XL1	** SLAB PROTECTION KEY	**
SLABVER DS	XL1	** SLAB VERSION	**
SLABLRECL DS	XL1	** SLAB OUTPUT FILE LRECL	**
SLABTYPE DS	CL1	** SLAB LABEL TYPE	**
SLABTYPE_CART EQU	X'80'	** SLAB CARTRIDGE LABEL BUILT	**
SLABTYPE_REEL EQU	X'40'	** SLAB REEL LABEL BUILT	**
SLABCOL DS	XL1	** SLAB NUMBER OF COLUMNS	**
SLABROW DS	XL1	** SLAB NUMBER OF ROWS	**
SLABLAB DS	0F	** SLAB STICKY LABEL	**
ORG SLABLAB		** SLAB MAXIMUM LABEL LAYOUT	**
SLABMAX DS	CL2000	**	**

Figure 102. Sticky label area

Once you have decided on your label size, you must decide on the layout of the data fields. Each row in the label can have none, one, or more data fields mapped to them. EDGLAB shows you how DFSMSrmm lays out the labels for cartridges and tape reels. The sample EDGUX100 exit module also shows you how to map your own labels over the SLABLAB label area. You can easily customize the label area by adapting the sample label area in EDGUX100. See [Figure 103 on page 285](#) for how the label area can be addressed and updated.

```

L      R5,PL100_LABPTR      Do we have a prepared label?      @17M
LTR    R5,R5                @17M
BZ     DSPCNT2              NO, CONTINUE                      @17M
TM     PL100_INFO,PL100_INFO_DISPLAB                      @17C
BZ     DSPCNT_OPT2         NO, CONTINUE with option 2        @17C
USING  SLAB,R5              @L6A

...

*-----*
* CREATE CUSTOM STICKY LABEL *
* Remove comments in the following code to activate or customize@17A*
*-----*
DSPCNT0 DS    0H                @K4A
*      LR     R1,R3              @K4A
*      LA     R2,SLABMAX          CLEAR PREPARED STICKY LABEL @K4A
*      LA     R3,2000            @K4A
*      LA     R6,@CC00506        @K4A
*      ICM    R7,15,@CB07614     @K4A
*      MVCL   R2,R6              @K4A
*      LR     R3,R1              @K4A
*      MVI    SLABCOL,80          SET ROW/COLUMN             @K4A
*      MVI    SLABLRECL,80        @K4A
*      MVI    SLABROW,10          @K4A
*      MVC    SLABUUSR,PL100_LAB_USERDATA @K4A
*      MVC    SLABUDSN,PL100_DSN   @K4A
*      MVC    SLABUJBN,PL100_JOBNAME @K4A
*      MVC    SLABUVSL,PL100_VOLSER @K4A

```

Figure 103. Addressing the sticky label area

Also, see Figure 104 on page 285 for how you can overlay the label mapping over the SLABLAB area. First, define the rows/records for the labels, and then for each row, overlay the data fields you want to use. Refer to the sample EDGUX100 exit module for the complete sample solution.

```

EDGSLAB LIST=YES,DSECT=YES      @K4A
ORG    SLABLAB                  @L6A
SLABULN1 DS    CL80              RECORD 1      @L6A
SLABULN2 DS    CL80              RECORD 2      @L6A
SLABULN3 DS    CL80              RECORD 3      @L6A
SLABULN4 DS    CL80              RECORD 4      @L6A
SLABULN5 DS    CL80              RECORD 5      @L6A
SLABULN6 DS    CL80              RECORD 6      @L6A
SLABULN7 DS    CL80              RECORD 7      @L6A
SLABULN8 DS    CL80              RECORD 8      @L6A
SLABULN9 DS    CL80              RECORD 9      @L6A
SLABULNA DS    CL80              RECORD 10     @L6A
ORG    SLABULN1                 LAYOUT RECORD1 @L6A
SLABUDSN DS    CL44              SLAB USER LABEL DSNAME @L6A
ORG    SLABULN2                 LAYOUT RECORD2 @L6A
SLABUUSR DS    CL69              SLAB USER USER DATA  @K4C
ORG    SLABULN3                 LAYOUT RECORD3 @L6A

...

```

Figure 104. Mapping a custom sticky label

You can also suppress the production of labels by setting the PL100_SET_NOLABEL bit to B'1'. DFSMSrmm checks for this bit at CLOSE time. See [“Selecting the method used for label processing”](#) on page 470 for information about modifying labels.

Controlling tape volume data set recording

Use the EDG_EXIT100 installation exit to request that DFSMSrmm records information about the first data set only and keeps track of the statistics at the volume level. DFSMSrmm then performs data set name checking on the first data set only. DFSMSrmm does not keep track of the number of data sets on the volume when data set recording is turned off for a volume. DFSMSrmm keeps track of some items like the use count to reflect the number of times the volume is read or written. You must request that DFSMSrmm records only the first data set each time you rewrite the first data set. Otherwise DFSMSrmm resumes recording information about all the data sets on the volume.

You might want to limit DFSMSrmm recording of tape data set information for these reasons:

- A tape volume is assigned for use by a specific application and the application tracks the tape contents.

- A tape volume contains a large number of files and you do not need to know details of all the files.

If you have applications that create multiple cataloged data sets on tape, and you suppress the DFSMSrmm recording of other than the first data set on the volume, ensure that the data sets other than the first data set are uncataloged. When the recording of data set information is suppressed, DFSMSrmm does not uncatalog these data sets during inventory management processing when the parmlib option UNCATALOG is specified as S or Y. Since DFSMSrmm cannot uncatalog the data sets, another method must be used to uncatalog them. Leaving non-existent data sets cataloged could lead to processing problems later.

Step 1: Tailor the sample EDGUX100 exit module

During the OPEN processing for the first file on a volume, the sample EDGUX100 exit module retrieves the data set name, job name and job step program name from system control blocks. The sample EDGUX100 exit module then scans the EDMTAB table for a match. If there is a match, the sample EDGUX100 exit module sets the PL100_SET_IGNORE_FILE2_TON bit to request DFSMSrmm only record data set details for the first file.

To use the sample EDGUX100 exit module for this function, define a table as shown in [Figure 105 on page 286](#). The order in which the table entries are listed is important because the exit scans the table until it finds the first entry where the job name, data set name and program name masks match the current request. You can change the priority of matching by changing the order of the table entries.

EDMTAB	DS	0F	start of table
*		jobname	data set name
	DC	CL8 '*'	',CL44'BACKUP*'
	DC	CL8 'ABC*'	program name
*			
	DC	CL8 'STSGWD*'	',CL44'*'
	DC	CL8 'A*'	program name
*			
	DC	CL8 'STSG%%*'	',CL44'STSG%%.BACKUP.*'
	DC	CL8 'DEF%MAIN'	program name
*			
	DC	CL8 'STSGDPW '	',CL44'DAVE.TOOMUCH.DATA'
	DC	CL8 'AB999*'	program name
*			
	DC	CL8 'EDMEND'	end of table marker

Figure 105. Sample table for controlling data set recording

The table contains:

jobname

One-to-eight alphanumeric or national characters including % and *.

% can be used to ignore a positional character in the job name.

* can be used to ignore all remaining characters in the job name. A jobname of * means that the entry applies to all jobs.

data set name

Can be up to forty-four characters, following z/OS data set naming conventions, including % and *.

- The character % can be used to ignore a positional character in the data set name.
- The character * can be used to ignore all remaining characters in the data set name. A data set name of * means that the entry applies to all data sets.

The use of the character * is not the same as in the generic data set names supported by DFSMSrmm for vital records specifications and search data set masks. Here the * works like the characters *.* might in a generic data set name mask.

Program name

A value up to eight alphanumeric character including % and *.

% can be used to ignore a positional character in the program name.

* can be used to ignore all remaining characters in the program name. A program name of * means that the entry applies to all programs.

Step 2: Activate the sample EDGUX100 exit module

See “Installing the EDGUX100 default exit routine” on page 293 for information about building an SMP/E USERMOD to apply the updated source code for EDGUX100 so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109 on page 294](#).

Changing location information with EDG_EXIT100

You can use the EDG_EXIT100 installation exit to perform these tasks:

- Specify a new location name or location type for a volume.
- Clear the location name to prevent a volume from being assigned to a location.
- Add a location name when one was not originally assigned in the disposition control file.

You can change the contents of the PL100_NAME and PL100_LOCTYPE fields to change location or name information. On input, the fields contain values set from the disposition control file.

DFSMSrmm checks the storage location name and type that you specify against the list of storage locations defined in the DFSMSrmm EDGRMMxx parmlib LOCDEF command. If you specified a storage location that is not defined to DFSMSrmm, DFSMSrmm treats the location as a loan location.

Use the PL100_INFO_CMOVE field to determine if a volume move is to be confirmed by checking for the bit that indicates that a later confirm move is required or is marked as already completed. Set PL100_SET_CMOVE or PL100_SET_NOCMOVE for move confirmation action. PL100_SET_CMOVE forces a later move. PL100_SET_NOCMOVE marks the move as already completed.

EDG_EXIT100 exit routine processing

EDG_EXIT100 gets called at several points if DFSMSrmm does not already have the required information. For example, if the required information for the current data set and volume combination was provided in a previous call of the exit, EDG_EXIT100 is not called again. The points that EDG_EXIT100 can be called are as follows:

- At pre-ACS processing time for new allocations, a pointer to the ACERO is passed in PL100_ACEROPTR.
- At OPEN time in the address space of the program issuing the OPEN request. During OPEN processing the address of a copy of the JFCB is provided in PL100_JFCBPTR.

EDG_EXIT100 can also be called at CLOSE or EOVS if the information that DFSMSrmm requires for processing is not available. For example, EDG_EXIT100 can be called if DFSMSrmm is restarted while tape volumes are in use.

- At CLOSE and EOVS time in the address space of the program processing the tape file. PL100_ITS_CLOSE is set and PL100_LABINFO provides information useful for creating labels.
- When DFSMSrmm updates a write to operator message defined by the EDGRMMxx parmlib MNTMSG command.

You use the MNTMSG operands to identify the write to operator messages that you want DFSMSrmm to update with external volume serial number and pool information. Messages are processed for both specific and non-specific volume serial numbers. If the JES3 IATUX71 exit is used with the DFSMSrmm EDG3X71 exit, EDG_EXIT100 is called during JES3 fetch and mount message processing.

EDG_EXIT100 can also be called from MSGDISP processing if the information that DFSMSrmm requires for processing is not available. For example, EDG_EXIT100 can be called if DFSMSrmm is restarted while tape volumes are in use.

DFSMSrmm has not yet checked if the volume is defined in its control data set, if the volume will be rejected, or if the volume is subject to any other DFSMSrmm processing.

After the installation exit has been called, DFSMSrmm performs this processing:

- If the expiration date is updated by the exit, DFSMSrmm updates the date in the copy of the JFCB for the current request. DFSMSrmm does not update the real JFCB control block.
- If a vital record specification management value is returned, DFSMSrmm records this value together with other details for the data set in the DFSMSrmm control data set, for use during inventory management.
- During pre-ACS processing if a pool or vital record specification management value is returned, DFSMSrmm sets the MSPPOOL and MSPOLICY variables into the ACERO if the pre-ACS installation exit IGDACSXT has not already done so.
- If a specific pool name is returned and it meets the pool naming conventions, DFSMSrmm uses the value to update messages intercepted for MNTMSG processing, to update tape drive displays through MSGDISP processing and to perform pool validation of scratch volumes at OPEN time. If JES3 is used and IATUX71 is in use, the pool is also passed to JES3 for use in JES3 fetch and mount messages. If you request that DFSMSrmm prevents the tape drive cartridge loader from being indexed, DFSMSrmm resets the automatic cartridge load flag during MSGDISP processing. If the selected pool does not meet the naming conventions, DFSMSrmm uses the pool it has already selected.
- If the volume must be ignored, DFSMSrmm ensures that the current user is authorized to request that DFSMSrmm ignores this volume. A SAF request is used to check authorization:

```
RACROUTE REQUEST=AUTH,ENTITY=STGADMIN.EDG.IGNORE.TAPE.volser,
ACCESS=value,LOG=ASIS
```

where:

volser

Is the current volume serial number of the mounted volume

value

Is either READ or UPDATE

If the user is authorized, DFSMSrmm ignores all further activity for this volume until end-of-volume processing or until a data set on the volume is closed. This means that DFSMSrmm does not validate that the correct volume is mounted, does not record volume usage in its control data set and cannot provide any management functions for the volume based on the data about to be created. However, if the user is authorized to ignore processing, this overrides any decision taken by RACF because of authorization in the DATASET class.(TAPEAUTHDSN=YES, or SETROPTS TAPEDSN). During OPEN processing for specific volume requests, DFSMSrmm overrides RACROUTE return codes of 4 or 8 so that they are return code 0. EDGUX100 can be tailored to prevent overriding of the RACROUTE return code 8, if PL100_SET_NOTBYPASS_SAFRC8 is set on.

If authorization fails and DFSMSrmm is running in protect mode, DFSMSrmm rejects the volume. For non-specific mount requests, another volume is requested. For specific volume requests the OPEN request fails.

If the resource does not exist, the request is treated as not authorized.

If authorization fails and DFSMSrmm is running in record-only mode, DFSMSrmm ignores the volume and issues the information message, EDG4047I.

If authorization fails and DFSMSrmm is running in warning mode, DFSMSrmm ignores the volume but issues error and warning messages.

- If a rack number is returned, DFSMSrmm uses this value as if it were a normal DFSMSrmm rack number for use in messages that are updated as defined by the parmlib MNTMSG definitions.
- DFSMSrmm validates the information you set in the parameter list as follows:

PL100_VRS

Must meet the z/OS data set naming standards for a single data set qualifier.

PL100_RACKNO

For specific volume requests, this value must be uppercase, alphanumeric, national, or special characters.

PL100_POOL

For non-specific volume requests the pool prefix must be uppercase, one to five alphanumeric, national, or special characters ending in *.

PL100_LOCATION

If the location type is set to store or library, DFSMSrmm uses the EDGRMMxx LOCDEF definitions to validate the location. If the location name is not valid, DFSMSrmm treats the location name as a loan location.

- If you request no data set recording for a volume, DFSMSrmm updates the volume record during OPEN processing to identify that the volume does not record all data sets.
- If you specify the DFSMSrmm EDGRMMxx OPTION DISPDDNAME operand, DFSMSrmm uses the location, location type, and volume move confirmation values from the PL100 parameter list to update the volume record location and destination fields. If you request "no confirm move", DFSMSrmm sets the location, otherwise DFSMSrmm sets the destination for the volume. If the location type is a loan location, DFSMSrmm only records the loan location.
- When any exit module sets a non-zero return code, DFSMSrmm issues message EDG0313I. When any exit module abends, DFSMSrmm issues message EDG0314I. If an error occurs in CSVDYNEX at any point in processing, DFSMSrmm issues message EDG0312I. In all cases, mount message, SMS pre-ACS, and O/C/EOV processing continues.

Assigning expiration dates

When a JFCB address is supplied, the sample EDGUX100 exit module checks to see whether an expiration date, rather than a retention period, is specified by the user. If an expiration date was specified:

- If the JCL expiration date is 98000, the sample EDGUX100 exit module clears the expiration date so that DFSMSrmm uses the DFSMSrmm parmlib default retention period to calculate the expiration date. This prevents DFSMSrmm from treating a special expiration date as an actual expiration date. If the exit parameter list indicates that the exit can request the volume is ignored, the sample EDGUX100 exit module requests that DFSMSrmm ignores the volume.
- If the JCL expiration date is 99000, the sample EDGUX100 exit module clears the expiration date field, so that DFSMSrmm uses the DFSMSrmm parmlib default retention period to calculate an expiration date. This prevents DFSMSrmm from treating a special expiration date as an actual expiration date. If the exit parameter list indicates that the exit can specify a vital record specification management value, the sample EDGUX100 exit module returns a vital record specification management value of D99000.
- Before checking for any other expiration date values, the sample EDGUX100 exit module checks to see if you want the expiration dates to be used as actual expiration dates or special dates. The sample EDGUX100 exit module checks for the existence of a dummy DD statement with the name NOTKEYD8.

```
//NOTKEYD8 DD DUMMY
```

Code a NOTKEYD8 dummy DD statement to indicate that the date is an actual expiration date. When you do not code the NOTKEYD8 DD statement in the current job step or do not code it as a dummy DD statement, the expiration date is treated as a special date.

If you want to use a DD name other than NOTKEYD8, change the DD name that is coded in the sample EDGUX100 exit module. Ensure that you specify a DD name padded to 8 characters with trailing blanks.

The sample EDGUX100 exit module does not perform any further special date processing. However, if you want to add checks for additional special dates you can do so.

Note: Expiration dates derived from Model DSCB (DCB=model.dsname) or LIKE=dsname are not evaluated as KEYDATE by EDGUX100, DEFTABLE or SMS Management Class ACS routine. Only EXPDT=yyddd specified in JCL, DATACLAS or dynamic allocation are evaluated as KEYDATE. However, EXPDT derived from Model DSCB will be stored in DFRMM Volume and Data set records and will be used as valid expiration dates if no further matches occur from DEFTABLE or VRS.

Supplying a scratch pool name

When the JFCB address, ACERO address, or the WTO address is supplied, the sample EDGUX100 exit module can supply a scratch pool name under these conditions:

- If the supplied ACERO address is nonzero, the jobname and data set name are taken from the ACERO.
- If the supplied JFCB address is non-zero, the job name and data set name are extracted from system control blocks.
- If the supplied WTO address is non-zero, the job name and data set name are extracted from the WTO message text.

To use the sample EDGUX100 exit module for pool selection, enter the job names and data set names into a table in the assembler source code as shown in [Figure 106 on page 290](#).

EDGUX100 scans a scratch pool table for the extracted job name and data set name to determine if this request requires a specific scratch pool. If a specific scratch pool is required, the exit module requests that DFSMSrmm use the pool selected by the exit module. If the cartridge loader is not to be used for the selected pool the exit module requests that DFSMSrmm prevent the cartridge loader being indexed.

Specify the entries in the table in the order that you want them to be matched by the exit module. The exit module scans the table until it finds an entry where both the job name and data set name masks match the current request. You can change the priority of matching by changing the order of the table entries.

POOLTAB	DS	0F	start of pool table
*		jobname	data set name
	DC	CL8 '*'	',CL44'BACKUP*'
	DC	CL6 'A02*'	pool name
	DC	AL1(PL100_SET_ACLOFF)	do not use loader
*			
	DC	CL8 'STSGWD*'	',CL44'*
	DC	CL6 'A*'	pool name
	DC	AL1(0)	use loader
*			
	DC	CL8 'STSG%%*'	',CL44'STSG%.BACKUP.*'
	DC	CL6 '12*'	pool name
	DC	AL1(0)	use loader
*			
	DC	CL8 'STSGDPW'	',CL44'DAVE.POOL1.DATA'
	DC	CL6 'AB999*'	pool name
	DC	AL1(PL100_SET_ACLOFF)	do not use loader
*			
	DC	CL8 'POOLEND'	end of pool table marker

Figure 106. Sample EDGUX100 pool selection table

Each table entry consists of:

jobname

An 8-byte field that can contain up to 8 alphanumeric or national characters including % and *.

The character % to ignore a positional character in the job name.

The character * to ignore all remaining characters in the job name. A job name of * means that the entry applies to all jobs.

Examples of job names in the table are: STSGWD*, STSG%%*, STSGDPW.

data set name

A forty-four byte field that can contain up to forty four characters, following z/OS data set naming conventions, including % and *.

- The character % to ignore a positional character in the data set name.
- The character * to ignore all remaining characters in the data set name. A data set name of * means that the entry applies to all data sets.

The use of the character* is not the same as in the generic data set names supported by DFSMSrmm for vital records specifications and search data set masks. The * works like the characters *.* might in a generic data set name mask.

Examples of data sets in the table are: STSGWD*, STSG%%.BACKUP*, DAVE.POOL1.DATA.

pool name

A 6-byte field that can contain up to five alphanumeric characters ending in *. If the pool name that you specify does not match the pool naming conventions, DFSMSrmm ignores the exit selected pool and uses the DFSMSrmm selected pool.

If your exit selects a pool that does not match a VLPOOL prefix value, all volumes that are mounted are rejected.

Examples of pool names in the table are: A*, 12*, AB999*.

loader status

A 1-byte field that contains a flag indicating whether the tape drive cartridge loader is to be disabled for the named pool.

The sample EDGUX100 installation module ORs this flag byte into the EDG_EXIT100 parameter list PL100_FUNCTION field.

The valid values are either X'00' or X'01'. You can set the correct value using one of these assembler statements:

```
DC    AL1(0)
DC    AL1(PL100_SET_ACLOFF)
```

You can use EDG_EXIT100 exit selected pools for your non-system-managed volumes including volumes managed using BTLS. See [“Using the EDG_EXIT100 installation exit from pre-ACS processing” on page 281](#) for information about using the EDG_EXIT100 exit to manage non-system-managed volumes using pre-ACS processing.

For system-managed volumes, use ACS routines to assign SMS constructs to new tape data sets at allocation time. DFSMSrmm does not perform pool validation for system managed volumes. As a result, if the exit selects a pool for a system-managed allocation, DFSMSrmm ignores the selection during volume validation.

If the operator mounts a volume that is not from the pool the exit specifies, the volume is rejected, and DFSMSrmm issues message EDG4021I.

```
EDG4021I VOLUME volser REJECTED. IT IS NOT IN AN ACCEPTABLE SCRATCH POOL
```

Using the system name to select a scratch pool

You can specify multiple scratch pool selection tables in your EDG_EXIT100 installation exit routine. This allows you to select scratch pools based on the name of the running system.

The sample installation exit uses the SYSNAME value from the IEASYSxx parmlib member to determine the system name. This is the CVTSNAME field.

[Figure 107 on page 292](#) is an example showing how you can add system names to the existing system name table in the sample EDGUX100 installation module. You can add system names in any order. If you want to remove a system from the list, delete the entry or set the system name to blanks.

```

SYSTAB DS 0F start of SYSTEM table
SYSCOUNT DC A(SYSTABL/SYSTENTL) count of entries
SYSENT1 DS 0F DO NOT CHANGE THIS LINE
          DC CL8' ',A(PTBLSYS1) Add the names of the
          DC CL8' ',A(PTBLSYS2) systems you want to
          DC CL8'W98MVS1 ',A(PTBLSYS3) use into the
          DC CL8' ',A(PTBLSYS4) prepared entries on
          DC CL8' ',A(PTBLSYS5) the left. There is no
          DC CL8'SYSA ',A(PTBLSYS6) need to change table
          DC CL8' ',A(PTBLSYS7) names on the right.
          DC CL8' ',A(PTBLSYS8)
* To add more system names to the table, just repeat the last
* table entry, specify a new system name, and the name of
* new pool table. For example:
* DC CL8'ANOTHER ',A(PTBLSYS9)
* Then build the new table by copying how one of the existing
* PTBLSYSx tables are defined.
SYSEND DS 0F end of table
SYSTABL EQU SYSEND-SYSENT1 length of table

```

Figure 107. Sample EDGUX100 exit module system name table

You can define a pool selection table for each system name that contains data set names, pool names, and automatic cartridge loader controls as shown in [Figure 106 on page 290](#). If there is no match found in the system table for the current system, DFSMSrmm uses the default selection pool POOLTAB. [Figure 108 on page 292](#) shows a selection table for a specific system.

```

*****
* POOL TABLE For 3rd System
*****
PTBLSYS3 DS 0F start of pool table
*          jobname data set name
          DC CL8'WOODMW* ',CL44'WOODMW.SYS3.* '
          DC CL6'S03*' pool name
          DC AL1(PL100_SET_ACLOFF) bypass acl load flag
*
          DC CL8'POOLEND' end of pool table marker

```

Figure 108. Pool selection table for system 3

Using storage group for manual tape library pooling

The sample EDGUX100 exit module is written to prevent the use of storage groups for manual tape library scratch pools. Use the sample EDGUX100 exit module as-is for manual tape library volumes to be pooled based on DFSMSrmm system-based pooling or on exit-selected pooling.

To use the storage group assigned at allocation time for pooling decisions, modify the sample EDGUX100 exit module. Remove the code that overrides using storage group for pooling decisions. Do not set the PL100_SET_IGNORE_SGNAME or PL100_SET_POOL parameters when the request is for a tape drive in a manual tape library.

Using the DFSMSrmm API from an EDG_EXIT100 exit module

You can use any DFSMSrmm TSO subcommand through the DFSMSrmm application programming interface.

The sample installation exit module shows how to use the API call to list a volume record from the DFSMSrmm control data set.

The use of the API call is optional, and must be enabled by the customer. The customer must also write the code to process the results of the call to the API.

See “Installing the EDGUX100 default exit routine” on [page 293](#) for information about building an SMP/E USERMOD to apply the updated source code for the EDGUX100 exit module, so that it supersedes any old EDGUX100 USERMODs. Include the necessary JCLIN statements as shown in [Figure 109 on page 294](#).

Setting up the EDG_EXIT100 routine environment

To activate or reactivate the exit routine, you can do one of the following:

- Use dynamic exit services by operator command.
- Use the CSVDYNEX macro.
- Start DFRMM for the first time after an IPL.

Note: Stopping and restarting, or refreshing, the DFSMSrmm procedure does not reload or reactivate your exit routine.

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information.

Installing the EDGUX100 default exit routine

Perform these actions to update or replace the default exit module:

1. Build and install an SMP/E USERMOD to apply the updated source code for the EDGUX100 exit module. Include the necessary JCLIN statements to get the EDGUX100 load module added to the LINKLIB target library.

You can apply the exit using an SMP/E USERMOD as shown in [Figure 109 on page 294](#). Modify the FMID and PRE to reflect the release you are running.

- a. Allocate a user SAMPLIB data set. In [Figure 109 on page 294](#) the user SAMPLIB data set is defined as MY.RMM.SRCLIB and allocated to DD card SRCLIB.
- b. Copy the EDGUX100 source from SAMPLIB to the user SAMPLIB, and copy EDGAPISR from SMPSTS to the user SAMPLIB and modify them, as needed, for your installation.
- c. SMP/E RECEIVE the USERMOD.
- d. SMP/E APPLY the USERMOD. Ensure that a DD card exists for the user SAMPLIB in the APPLY job, or as a DDDEF to SMP/E in the target zone.

After performing these steps, the modified version of the EDGUX100 exit module resides in both the user SAMPLIB and SYS1.SAMPLIB. IBM's original copy is only in the distribution libraries at this point. If you accept the USERMOD, only the modified version of the exit module exists. The SMP/E target zone reflects RMID indicators of VMRMM01 for all of these records:

```
SAMP EDGUX100 RMID=VMRMM01 SYSLIB=SAMPLIB
SRC  EDGUX100 RMID=VMRMM01 SYSLIB=SAMPLIB
MOD  EDGUX100 RMID=VMRMM01 LMOD=EDGUX100
SRC  EDGAPISR RMID=VMRMM01 SYSLIB=SMPSTS
MOD  EDGAPISR RMID=VMRMM01 LMOD=EDGUX100, CBRUXVNL, CBRUXEJC, CBRUXENT
LMOD EDGUX100          SYSLIB=LINKLIB
```

The RMID of VMRMM01 for the SAMP record prevents IBM service from being installed. This results in an ID search and notification to you that IBM is the servicing exit.

```

/* RECEIVE Step */
//RMMEXAM JOB , 'EDGUX100',MSGCLASS=H,MSGLEVEL=(1,1)
//SMPE EXEC PGM=GIMSMP,
// PARM='PROCESS=WAIT',
// DYNAMNBR=120
//SMPCSI DD DISP=SHR,DSN=your.CSI.dataset
//SMPHOLD DD DISP=SHR,DSN=your.HOLDDATA.dataset
//SMPCTL DD *
SET BDY(GLOBAL) .
RECEIVE .
/*
//SMPPTFIN DD DATA,DLM=###
++USERMOD (VMRMM01) REWORK(2008082) .
++VER (Z038) FMID(HDZ1190) .
++JCLIN .
//EDGUX100 EXEC PGM=IEWL,PARM='LET,NCAL,RENT,REUS,REFR,LIST,XREF'
//SYSLMOD DD DISP=SHR,DSN=SYS1.LINKLIB
//SRCLIB DD DISP=SHR,DSN=MY.RMM.SRCLIB
//AEDGMOD1 DD DISP=SHR,DSN=SYS1.AEDGMOD1
//SYSPRINT DD SYSOUT=*
//SYSLIN DD *
INCLUDE AEDGMOD1(EDGUX100)
INCLUDE AEDGMOD1(EDGAPISR)
ENTRY EDGUX100
NAME EDGUX100(R)
++SRC(EDGUX100) TXLIB(SRCLIB) DISTLIB(ASAMPLIB) .
++SRC(EDGAPISR) TXLIB(SRCLIB) DISTLIB(AEDGSR1) .
++SAMP(EDGUX100) TXLIB(SRCLIB) DISTLIB(ASAMPLIB) .
###
/*
/* APPLY Step */
//SMPAPPLC JOB ('T,H,IOM,,',SYSPROG),'***IBMUSER***',
// MSGLEVEL=(1,1),MSGCLASS=X,CLASS=A,REGION=0M,
// NOTIFY=&SYSUID
//S1 EXEC PGM=GIMSMP,
// PARM='PROCESS=WAIT',
// DYNAMNBR=120
//SMPCSI DD DISP=SHR,DSN=MAZRM3.GLOBAL.CSI
//SRCLIB DD DISP=SHR,DSN=RMSTST.MAZ.SOURCE
//SMPCTL DD *
SET BOUNDARY (MVSTZN)
.
APPLY
JCLINREPORT
SELECT (VMRMM01)
.

```

Figure 109. Building an SMP/E USERMOD to apply the updated EDGUX100 exit module

2. Copy the new exit load module into the LNKLST library.
3. Refresh LLA.
4. Refresh the exit module by using MVS operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

When you use the default exit module, EDGUX100, for the installation exit EDG_EXIT100, it is loaded and activated as an exit routine, if it is not already active, by DFSMSrmm when DFSMSrmm is started and stays loaded and active for the life of the IPL. It can be refreshed or deleted at any time by using one of the MVS system operator commands.

You can use any load module name for your exit module, because you use PROGxx in z/OS parmlib, SETPROG, or CSVDYNEX macro to associate the exit module with the exit. Therefore, you do not have to use EDGUX100 as the load module name. However, using the default exit module name simplifies your implementation, because DFSMSrmm itself ensures that the exit module is loaded and activated.

Deleting the EDG_EXIT100 exit routines

To delete, or selectively delete, the exit routines for the EDG_EXIT100 exit from the system, you can use MVS operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

Writing an exit routine for the EDG_EXIT100 exit

EDG_EXIT100 exit routines run in SYSTEM KEY 0 or 5 AMODE(31) RMODE(ANY) in the user's address space. KEY 0 is used when a WTO address or an ACERO address is provided, and KEY 5 is used when a JFCB address is provided. Exit modules can be contained in any APF authorized LNKLIST library or an APF authorized library named on CSVDYNEX macro invocation.

You can write your exit module so that it can issue commands or call other programs to update external inventories. You can also code the exit module to module issue a WTOR if needed. Do not issue messages that are contained in the MNTMSG table, or the EDG_EXIT100 installation exit could be called recursively.

Registers on entry to an EDG_EXIT100 exit routine

Register

Contents

0

Not applicable

1

Address of a parameter list mapped by the macro EDGPL100

2-12

Not applicable

13

Address of register save area

14

Caller's return address

15

Address of exit module entry point

EDG_EXIT100 parameter list

All communication is done using the parameter list. The parameter list is mapped by the macro EDGPL100 as shown in [“Installation exit mapping macro: EDGPL100” on page 517](#).

The same parameter list area is passed to each exit module associated with the exit. Each exit module can update the output area of the parameter list to indicate what processing is required by DFSMSrmm. Depending on the order of the exit routines, your exit module could receive the parameter list before any other exit module has processed it, or it could already have been processed. Before taking a decision in your exit routine, you should check to see if an earlier exit routine has already taken a decision. You should be prepared for your decision to be negated by other exit routine processing. For example, if you set a VRS management value, but another exit routine requests that the volume be ignored, DFSMSrmm ignores the volume.

If your exit routine is to build a custom sticky label, do not assume that the label built in the parameter list area is the default label created by DFSMSrmm. Instead of modifying the pre-built label, consider building the complete label.

The parameter list input values are:

PL100_VALID

This field defines functions you can request during this call of the installation exit.

PL100_CAN_IGNORE

If set to B'1', you can request that DFSMSrmm ignore the volume. To see if OPENRULE with an action of IGNORE has already made an ignore request, check the PL100_INFO_IGNORE_REQUEST_BYRULE bit.

PL100_CAN_VRS

If set to B'1', you can provide a vital record specification management value for DFSMSrmm to use for this data set.

PL100_CAN_RACKNO

If set to B'1', you can provide an external volume serial number or rack number for DFSMSrmm to use for this volume in any WTO messages DFSMSrmm updates.

PL100_CAN_IGNORE_FILE2_TON

If set to B'1', you can request that DFSMSrmm record the data set details only for the first file on a tape volume.

PL100_CAN_COPYFROM

If set to B'1', you can request to notify DFSMSrmm that the data set being created is being copied from another.

PL100_CAN_POOL

If set to B'1', you can provide a specific pool name for DFSMSrmm to use for this volume in any WTO messages DFSMSrmm updates. Also the pool name is used to validate that a correct scratch volume is mounted for a request.

PL100_ITS_CLOSE

If set to B'1', this indicates that DFSMSrmm called EDG_EXIT100 because a tape data set was closed or an end-of-volume condition occurred.

PL100_CAN_RETMET

If set to B'1', you can request to set the volume retention method when the first file on the volume is written.

PL100_VALID2

This field defines functions you can request during this call of the installation exit.

PL100_CAN_VRSELEXCLUDE

If set to B'1', you can request to set the VRSELEXCLUDE attribute for a data set.

PL100_REQ_VOLSER

This field contains one of: PRIVAT, SCRTCH or a volume serial number. For a nonspecific request PL100_REQ_VOLSER can contain either PRIVAT or SCRTCH. For a specific request PL100_REQ_VOLSER contains the actual volume serial number requested.

PL100_MOUNT_VOLSER

This field contains the volume serial number of the volume mounted to satisfy this request. This volume serial number is only available when a data set on the volume is opened or closed.

PL100_WTOPTR

This field contains the address of the WTO message that has been intercepted by MNTMSG processing. The address is zero during OPEN processing. See the PL100_WTOPTR field for information on the operator message being updated.

PL100_JFCBPTR

This field can contain the address of the JFCB copy or is set to zero. You can use it to locate the expiration date field JFCBXPDT. You can determine if the user specified an expiration date rather than a retention period by checking the JFCB for the JFCBEXP flag. If the exit is called during OPEN processing, the JFCB address is provided. During MNTMSG processing the JFCB address is set to zero and no JFCB is available.

PL100_POOL

When PL100_JFCBPTR is zero and PL100_CAN_POOL is set to B'1', this field contains the scratch pool that DFSMSrmm has already determined should be used for this request. It is determined using the VLPOOL definitions that you specify in the DFSMSrmm parmlib member. If you do not provide a replacement value and set the PL100_SET_POOL flag, this is the pool that DFSMSrmm uses.

PL100_LABINFO

When PL100_ITS_CLOSE is set to B'1', this field points to a data area mapped by PL100_LABDS. The data area contains information about the volume and file being processed.

PL100_ACCODE

This field contains either the value of the JCL specified ACCODE parameter or blanks if the ACCODE is specified in the reduced form 'ACCODE=' or if the JCL does not contain ACCODE. The field can be 1 to eight characters as described in [z/OS MVS JCL Reference](#). The first character is the ISO/ANSI accessibility code. The remaining characters can be any characters you choose. If the ACCODE value

specifies a special value, then your exit module can process the ACCODE value rather than the special date in the EXPDT keyword if it exists.

PL100_INFO

This field defines additional information.

PL100_INFO_IGNORE

If set to B'1' The volume is to be ignored by the EDG_EXIT100 installation exit.

PL100_INFO_IGNORE_REQUEST_BYRULE

If set to B'1' The volume is to be ignored based on OPENRULE with an action of IGNORE.

PL100_INFO_NOTRMM

If set to B'1', the volume is not defined to DFSMSrmm. This is only set at CLOSE or EOVS time.

PL100_INFO_DISPDD

If set to B'1', a disposition file was found and has been processed for this DD name.

PL100_INFO_DISPLAB

If set to B'1', the prepared label passed to the exit was requested by an entry in a disposition control file.

PL100_INFO_CMOVE

If set to B'1', a confirm must be performed and the location requested is set as the volume's destination, not the location.

PL100_INFO_USERDATA

If set to B'1', userdata was provided from the disposition file. The user data has been included in the default label created by DFSMSrmm processing.

PL100_INFO_MTL

If set to B'1', indicates that the allocated tape drive is a manual tape library tape drive and a storage group has been set by SMS ACS processing. DFSMSrmm uses the storage group name as the specific scratch pool unless you select a specific scratch pool or request that DFSMSrmm ignore the storage group name.

PL100_ACEROPTR

This field contains the address of the ACERO system control block during pre-ACS processing.

PL100_LAB_USERDATA

This field contains the user data from the disposition processing file for use in sticky labels.

PL100_LABPTR

This field contains the address of the sticky label prepared by DFSMSrmm.

PL100_LOCATION

This field contains the name of the location specified in the disposition file.

PL100_LOCTYPE

This field contains the type of the location.

The parameter list can be updated as follows:

PL100_FUNCTION

Use this field to request functions of DFSMSrmm

PL100_SET_IGNORE

Set the PL100_SET_IGNORE bit to indicate that you want this volume ignored until the current file reaches end-of-volume or is closed.

PL100_SET_IGNORE_MOUNTED

Set the PL100_SET_IGNORE_MOUNTED bit to indicate that you want this volume ignored based on the mounted volser until the current file reaches end-of-volume or when the current volume is closed.

PL100_SET_IGNORE_REQUESTED

Set the PL100_SET_IGNORE_REQUESTED bit to indicate that you want this volume ignored, based on the requested volser until the current file reaches end-of-volume or when the current volume is closed.

PL100_SET_IGNORE_FILE2_TON

Set the PL100_SET_IGNORE_FILE2_TON bit to indicate that DFSMSrmm is only to record data set details for the first file on the tape volume.

PL100_SET_POOL

Set the PL100_SET_POOL bit to indicate that you want to use a specific pool for the current non-specific volume request. When you set PL100_SET_POOL and provide a pool prefix for a tape drive in a manual tape library, DFSMSrmm does not use the storage group assigned by SMS ACS processing.

PL100_SET_ACLOFF

Set the PL100_SET_ACLOFF bit to indicate that you want DFSMSrmm to disable the cartridge loader for this request.

PL100_SET_NOLABEL

- Set the PL100_SET_NOLABEL bit to B'1' to suppress a label requested by a disposition control file. See PL100_INFO_DISPLAB.
- Set the PL100_SET_NOLABEL bit to B'0' when DFSMSrmm has created a default label other than through a disposition control file, and you would like the label to be produced.

The other functions you can request are determined by the presence of data in the output fields.

PL100_FUNCTION2

You can use this field to change some of the processing decisions that were made during disposition control processing. DFSMSrmm uses the PL100_LOCATION value to set the destination location or the current location. When DFSMSrmm sets the destination location, you must confirm the volume move later using the RMM CHANGEVOLUME subcommand. See [Chapter 18, “Setting up DFSMSrmm disposition processing,”](#) on page 467 for more information about disposition control.

PL100_SET_CMOVE

Set PL100_SET_CMOVE to B'1' when you want to confirm the volume move at a later time. DFSMSrmm uses the PL100_LOCATION value to set the volume destination and not the current location. If the destination is bin-managed, DFSMSrmm sets the required location. DFSMSrmm assigns a bin number to the volume during storage location management processing.

PL100_SET_NOCMOVE

Set PL100_SET_NOCMOVE to B'1' and DFSMSrmm confirms the volume move immediately. DFSMSrmm uses the PL100_LOCATION value to set the current location when it does not need to assign a bin number for the volume.

PL100_SET_IGNORE_SGNAME

Set this field to B'1' when you want to use DFSMSrmm system-based pooling instead of the storage group name set by SMS ACS processing when the tape drive is in a manual tape library.

PL100_SET_NOTBYPASS_SAFRC8

Set this bit to B'1' to turn off the DFSMSrmm function that uses the successful IGNORE authorization to change SAF return code 8 to 0. Set this bit on only if PL100_SET_IGNORE or PL100_SET_IGNORE_MOUNTED or PL100_SET_IGNORE_REQUESTED is set on.

PL100_SET_RETMET

Set this bit to B'1' to indicate that the volume's retention method has been set to the value defined in PL100_RETENTIONMETHOD.

PL100_SET_COPYFROM

Set this bit to B'1' to indicate that the data set being created is being copied from another.

PL100_SET_VRSELEXCLUDE

Set this bit to B'1' to indicate that the data set being created has the attribute VRSELEXCLUDE set.

PL100_JFCBPTR

If a JFCB address was provided as input, set the JFCB expiration date field to the new expiration date value you want DFSMSrmm to use for this data set. You can update the JFCB expiration date even if you do not provide a vital record specification management value.

Recommendation: Zero the expiration date field in the JFCB copy to allow DFSMSrmm to calculate a default expiration date.

PL100_VRS

Set the DFSMSrmm vital record specification management value you have selected. The values you use in this field should correspond to the RMM ADDVRS DSNAMES subcommands you have used. The vital record specification management value you specify is only used during inventory management vital records processing once it has been established that use of the data set name has not produced a match.

Only update this field if the PL100_CAN_VRS flag is set, as it is only when this flag is set that DFSMSrmm makes use of the value you specify. DFSMSrmm records the vital record specification management value you specify in the control data set when:

- A new data set is created
- An existing data set is rewritten with DISP=OLD
- A data set not yet recorded by DFSMSrmm is read

When an existing data set is extended, and the data set already has a vital record specification management value, DFSMSrmm ignores the new value you specify and propagates the existing vital record specification management value.

In all other cases, the vital record specification management value you specify is not used by DFSMSrmm.

PL100_RACKNO

You can provide a 6 character value for DFSMSrmm to use as the rack number or an external volume serial number for adding to the mount messages that DFSMSrmm intercepts and updates. Provide this value when requesting that DFSMSrmm ignore the volume. This helps your operators retrieve the correct volume.

Only update this field if the PL100_CAN_RACKNO flag is set, as it is only when this flag is set that DFSMSrmm makes use of the value you specify.

PL100_POOL

You can use this field to provide a specific pool name to be used with a non-specific output request.

Provide a 6 character pool identifier for DFSMSrmm to use for adding to the mount messages and tape drive display requests that DFSMSrmm intercepts and updates and for use during volume validation. Provide this value when requesting that DFSMSrmm use a specific scratch pool. This helps your operators retrieve the correct volume.

Only update this field if the PL100_CAN_POOL flag is set. Also set PL100_SET_POOL to B'1', as it is only when this flag is set that DFSMSrmm makes use of the value you specify.

If your exit module selects a pool that does not meet the pool naming conventions, DFSMSrmm uses the DFSMSrmm selected pool, and ignores the setting of the PL100_SET_ACLOFF flag.

If your exit module selects a pool that does not match a VLPOOL prefix value, all volumes that are mounted are rejected.

PL100_LOCATION

The value is set by DFSMSrmm depending on the keyword LOC=, or OUT= used in the disposition control file. You can change the value during EDG_EXIT100 processing and DFSMSrmm validates it on return from the exit.

PL100_RETENTIONMETHOD

Set this field to 0 or 1 for:

PL100_RM_VRSEL EQU 0 RETENTION METHOD VRSEL
PL100_RM_EXPDT EQU 1 RETENTION METHOD EXPDT

PL100_COPYFROM_DSN

Set this field to the name of the source data set.

PL100_COPYFROM_VOLSER

Set this field to the source volume number.

PL100_COPYFROM_DSEQ

Set this field to the source data set sequence number.

PL100_COPYFROM_OWNER

Set this field if you want to specify the owner of the volume being written to. It can be applied only when the first file on the first volume of a multivolume set is written, and will be propagated by DFSMSrmm during EOV to any additional volumes in the set. This field is optional for the COPYFROM function.

Registers on return from a EDG_EXIT100 exit routine
Register
Contents

0

Not applicable

1-14

Restored to contents at entry

15

Return code

EDG_EXIT100 installation exit routine return codes

Table 40 on page 300 shows the contents of register 15 upon return from each exit module.

If any exit module returns return code 0, DFSMSrmm processes the updated parameter list. To avoid excessive calls to the exit, DFSMSrmm tracks which requests for a particular event have been passed to the exit. When any exit module fails, either because it abends or because it returns any return code other than 0, DFSMSrmm resets the tracking information so that any future calls will be made to the exit, regardless of the error.

Table 40. EDG_EXIT100 installation routine exit return codes

Return Code	Description
0	Processing was successful. The parameter list might have been updated by the exit module.
16	The exit module determined that the parameter list passed to it did not conform to the correct specifications.

Using the EDG_EXIT200 installation exit

Use the EDG_EXIT200 installation exit to perform these tasks:

- Return a volume to scratch status in an external inventory
- Prevent a volume from returning to scratch status
- Request that DFSMSrmm ignores data set information recorded for a volume

EDG_EXIT200 exit routine processing

The DFSMSrmm sample EDGUX200 exit module performs these functions:

- Validates the parameter list
- Returns immediately for a system-managed volume

The EDG_EXIT200 installation exit is called during inventory management expiration processing in the DFSMSrmm started procedure address space. It is called by DFSMSrmm each time a volume is identified

for the return to scratch action. The volume has not yet been returned to scratch in either the TCDB or the DFSMSrmm control data set. If the exit requests that the volume is not returned to scratch status, DFSMSrmm leaves the volume in pending release status. If the exit requests that DFSMSrmm ignore the data set name information, all data set information for the volume is removed from the DFSMSrmm control data set and the volume is returned to scratch status.

The EDG_EXIT200 installation exit is not called for volumes that are under manual scratch control until the scratch release action has been confirmed because of VLPOOL AUTOSCRATCH(NO). Use the EDG_EXIT200 installation exit to implement control of return to scratch without using the VLPOOL AUTOSCRATCH(NO) feature. For example, you can use EDG_EXIT200 to check if the scratch action has been confirmed before you allow EXPROC to scratch the volume. When MVFLGE.MVRETSCR is set, the scratch action has not been confirmed. If you want to manually cleanup volumes on another system, use this flag and set the PL200_SET_NOSCRATCH to prevent return to scratch. When MVFLGE.MVRETSCR is off, manual actions are performed and confirmed, and you can allow the volume to return to scratch.

When any exit module detects an incorrect parameter list and sets return code 16, DFSMSrmm skips return to scratch processing for the volume and issues message EDG0313I the first occurrence and EDG2448I at the end of processing. When any exit module abends, DFSMSrmm disables return to scratch processing for the remainder of the EXPROC run, and issues message EDG0314I is issued for the first occurrence, followed by EDG2447I.

Setting up the EDG_EXIT200 routine environment

To activate or reactivate the exit routine, you can do one of the following:

- Use dynamic exit services by operator command.
- Use the CSVDPYNEX macro.
- Start DFRMM for the first time after an IPL.

Note: Stopping and restarting, or refreshing, the DFSMSrmm procedure does not reload or reactivate your exit routine.

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information.

Installing the EDGUX200 default exit routine

Perform these actions to update or replace the default exit module:

1. Build and install an SMP/E USERMOD to apply the updated source code for the EDGUX200 exit module.

Include the necessary JCLIN statements to get the EDGUX200 load module added to the LINKLIB target library.

You can apply the exit using an SMP/E USERMOD as shown in [Figure 110 on page 302](#). Modify the FMID and PRE to reflect the release you are running.

- a. Allocate a user SAMPLIB data set. In [Figure 110 on page 302](#) the user SAMPLIB data set is defined as MY.RMM.SRCLIB and allocated to DD card SRCLIB.
- b. Copy the shipped EDGUX200 source from SAMPLIB to the user SAMPLIB and modify as needed for your installation.
- c. SMP/E RECEIVE the USERMOD.
- d. SMP/E APPLY the USERMOD. Ensure that a DD card exists for the user SAMPLIB in the APPLY job or as a DDDEF to SMP/E in the target zone.

After performing these steps, the modified version of the EDGUX200 exit module resides in both the user SAMPLIB and SYS1.SAMPLIB. IBM's original copy is only in the distribution libraries at this point. If you accept the USERMOD, only the modified version of the exit module exists. The SMP/E target zone reflects RMID indicators of VMRRMM02 for all of these records:

```
SAMP EDGUX200 RMID=VMRRMM02 SYSLIB=SAMPLIB
SRC   EDGUX200 RMID=VMRRMM02 SYSLIB=SAMPLIB
```

```
MOD  EDGUX200 RMID=VMRMM02 LMOD=EDGUX200
LMOD EDGUX200              SYSLIB=LINKLIB
```

The RMID of VMRMM02 for the SAMP record prevents IBM service from being installed. This results in an ID search and notification to you that IBM is the servicing exit.

```
//RMMSTUFF JOB  , 'SLIP IT IN', MSGCLASS=H, MSGLEVEL=(1,1)
//SMPE     EXEC PGM=GIMSMP,
//          PARM= 'PROCESS=WAIT',
//          DYNAMNBR=120
//SMPCSI   DD DISP=SHR, DSN=MAZRRM3.GLOBAL.CSI
//SMPHOLD  DD DISP=SHR, DSN=SAP1.DUMMY.HOLDDATA
//SMPCNTL  DD *
//          SET BDY(GLOBAL) .
//          RECEIVE .
/*
//SMPPTFIN DD  DATA, DLM=##
++USERMOD (VMRMM02) REWORK(2007082) .
++VER (Z038) FMID(HDZ11D0) .
++JCLIN .
//EDGUX200 EXEC PGM=IEWL, PARM= 'LET, NCAL, RENT, REUS, REFR, LIST, XREF'
//SYSLMOD  DD  DISP=SHR, DSN=SYS1.LINKLIB
//SRCLIB   DD  DISP=SHR, DSN=MY.RMM.SRCLIB
//AEDGMOD1 DD  DISP=SHR, DSN=SYS1.AEDGMOD1
//SYSPRINT DD  SYSOUT=*
//SYSLIN   DD  *
//          INCLUDE AEDGMOD1(EDGUX200)
//          ENTRY  EDGUX200
//          NAME   EDGUX200(R)
++SRC(EDGUX200) TXLIB(SRCLIB) DISTLIB(ASAMPLIB) .
++SAMP(EDGUX200) TXLIB(SRCLIB) DISTLIB(ASAMPLIB) .
###
/*
//SMPAPPLC JOB ('T,H,IOM,,', SYSPROG), '***IBMUSER***',
//          MSGLEVEL=(1,1), MSGCLASS=X, CLASS=A, REGION=0M,
//          NOTIFY=&SYSUID
//S1        EXEC PGM=GIMSMP,
//          PARM= 'PROCESS=WAIT',
//          DYNAMNBR=120
//SMPCSI   DD DISP=SHR, DSN=MAZRRM3.GLOBAL.CSI
//SRCLIB   DD DISP=SHR, DSN=RMMTST.MAZ.SOURCE
//SMPCNTL  DD *
//          SET  BOUNDARY (MVSTZN)
//          .
//          APPLY
//          JCLINREPORT
//          SELECT (VMRMM02)
//          .
```

Figure 110. Building an SMP/E USERMOD to apply the updated EDGUX200 exit module

2. Copy the new exit load module into the LNKST library.
3. Refresh LLA.
4. Refresh the exit module by using MVS system operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

When you use the default exit module, EDGUX200, for the installation exit EDG_EXIT200, it is loaded and activated as an exit routine, if it is not already active, by DFSMSrmm when DFSMSrmm is started and stays loaded and active for the life of the IPL. It can be refreshed or deleted at any time by using one of the MVS system operator commands.

You can use any load module name for your exit module, because you use PROGxx in z/OS parmlib, SETPROG, or CSVDYNEX macro to associate the exit module with the exit. Therefore, you do not have to use EDGUX200 as the load module name. However, using the default exit module name simplifies your implementation, because DFSMSrmm itself ensures that the exit module is loaded and activated.

Deleting the EDG_EXIT200 exit routines

To delete, or selectively delete, the exit routines for the EDG_EXIT200 exit from the system, you can use MVS operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

Writing an exit routine for the EDG_EXIT200 exit

EDG_EXIT200 exit routines run in PROBLEM KEY, SUPERVISOR STATE AMODE(31) RMODE(ANY) in the address space of the DFSMSrmm started procedure. Exit modules can be contained in any APF authorized LNKST library or an APF authorized library named on CSVDYNEX macro invocation.

You can write your exit module to issue commands or call other programs to get external inventories updated.

Registers on entry to an EDG_EXIT200 exit routine

Register

Contents

0

Not applicable

1

Address of a parameter list mapped by the macro EDGPL200

2-12

Not applicable

13

Address of register save area

14

The caller's return address

15

Address of the exit module entry point

EDG_EXIT200 parameter list

All communication is done using the parameter lists fields. The parameter list is mapped by the macro EDGPL200 as shown in [“Installation exit mapping macro: EDGPL200” on page 524](#).

The same parameter list area is passed to each exit module associated with the exit. Each exit module can update the output area of the parameter list to indicate what processing is required by DFSMSrmm. Depending on the order of the exit routines, your exit module could receive the parameter list before any other exit module has processed it, or it might already have been processed. Before taking a decision in your exit routine, you should check to see if an earlier exit routine has already taken a decision. You should be prepared for your decision to be negated by other exit routine processing.

The parameter list input values are:

PL200_VALID

This field defines which functions you can request during this call of the installation exit.

PL200_CAN_SCRATCH

If set to B'1' DFSMSrmm is returning the volume to scratch and you can request DFSMSrmm not to do this or can request DFSMSrmm to ignore data set name information for the volume.

PL200_VOLSER

This field contains the volume serial number of the volume being returned to scratch.

PL200_RACK_NUMBER

This field contains the rack number of the volume being returned to scratch.

PL200_MEDIA_NAME

This field contains the media name used for the volume being returned to scratch.

PL200_LOCATION

This field contains the location name used for the volume being returned to scratch. It can be any value that is valid for volumes in the installation. It can be any valid location name, including storage locations defined as Home locations, but not a regular storage location.

PL200_DSNAME

This field contains the name of the first data set on the volume. There might be other data sets on the volume, but this information is not available to the exit.

PL200_VOLUME_FLAGS

This flag byte is used to give you information about the volume.

PL200_SMS_VOL

If set to B'1' this volume is a system-managed volume. For system-managed volumes DFSMSrmm dynamically updates the TCDB so you do not need to.

PL200_HOME_LOCDEF

This flag byte is used to give you information that the volume is in the storage location defined as home.

PL200_MANUAL_SCRATCH

This flag byte is used to give you information that the volume is in VLPOOL with AUTOSCRATCH(NO).

PL200_EDGSVREC_ADDR

This flag byte is used to give you information about the address of the volume.

PL200_CATSYSID

This flag byte is used to give you information about the CATSYSID list for the running system.

PL200_DESCRIPTION

This field contains descriptive information about the volume that is returning to scratch status.

PL200_OWNER

This field contains the owner ID of the volume owner.

The parameter list can be updated as follows:

PL200_FUNCTION

This field describes functions that can be updated.

PL200_SET_NOSCRTCH

Set the PL200_SET_NOSCRTCH bit if you do not want DFSMSrmm to return the volume to scratch status at this time. You can control return to scratch processing each time inventory management expiration processing is run.

PL200_SET_IGNORE_DSN

Set the PL200_SET_IGNORE_DSN bit if you do not want DFSMSrmm to use the data set information for this volume. Setting this flag allows you to control the validation that DFSMSrmm performs at OPEN time. If DFSMSrmm cannot use the data set name information it cannot ensure that the volume has not changed since it was last used. DFSMSrmm will still use the internal volume label for validation.

Registers on return from the EDG_EXIT200 exit routine**Register****Contents****0**

Not applicable

1-14

Restored to contents at entry

15

Return code

EDG_EXIT200 installation exit routine return codes

Table 41 on page 305 shows the contents of register 15 upon return from each exit module.

If all exit modules return return code 0, DFSMSrmm processes the updated parameter list and return to scratch processing continues. When any exit module fails, either because it abends or because it returns any return code other than 0, return to scratch for the volume is skipped. When any exit module abends, return to scratch processing is disabled for the remainder of the EXPROC run.

Table 41. EDG_EXIT200 installation exit routine return codes

Return Code	Description
0	Processing was successful. The parameter list might have been updated by the exit module.
16	The exit module determined that the parameter list passed to it did not conform to the correct specifications.

Using the EDG_EXIT300 installation exit

You must use the EDG_EXIT300 installation exit to dynamically set media information for volumes that have a media information name (MEDINF operand) other than IBM. If you do not use the EDG_EXIT300 installation exit, only the volumes with an IBM MEDINF name are maintained dynamically. Use the EDG_EXIT300 installation exit to supply media information for a specific volume for non-IBM media using the vendor API. The exit is called during O/C/EOV processing if the installation-defined media information for this volume is not IBM, and you have MEDINF definitions.

EDG_EXIT300 exit routine processing

Each time that the EDG_EXIT300 installation exit is invoked, your code should perform the following tasks:

- Checks that the function is to gather media information. PL300_MEDINF flag is B'1'.
- Uses the information provided as input to the EDG_EXIT300 exit routine to determine if you need to provide media information for this volume. You can check the information provided by O/C/EOV in the IFGTEP area to verify what type of device is being used, media information provided by the drive, and so on. Check the information provided about the volume, such as MEDINF name, MEDIATYPE, and RECORDINGFORMAT. PL300_LSTMDFN contains the media information name.
- If required, calls any API provided by your software vendor to retrieve details of the media.
- If the media information available to DFSMSrmm is incorrect, sets the correct values in the EDGPL300 output area. For example, to set the external values for the media type and recording format copy the values to the PL300_LSTMDTX and PL300_LSTMDRX fields.
- Sets the related flags to indicate the values to be used. When you have set the PL300_LSTMDTX and PL300_LSTMDRX fields, also set the PL300_LSTOFMV and PL300_LSTOFFX flags to B'1' to indicate you have set the output values and that they are to be used by DFSMSrmm. PL300_LSTOFFX indicates that you have set the external values. If you choose to instead set the internal values into PL300_LSTOTDSI, you must ensure that PL300_LSTOFFX is b'0'.

You might not have to make any changes if the information is already correct, or perhaps, the media type is correct and only the recording format has been changed.

The EDG_EXIT300 exit could be extended at any time by IBM to add additional OEM media and library-related functionality. Because of this fact, ensure that you always check that the function is required of the exit and then only perform that required function.

When any exit module sets a non-zero return code, DFSMSrmm issues message EDG0313I. When any exit module abends, DFSMSrmm issues message EDG0314I. If an error in CSVDYNEX occurs at any point in processing, DFSMSrmm issues message EDG0312I. In all cases, O/C/EOV processing continues.

Setting up the EDG_EXIT300 routine environment

To activate or reactivate the exit routine, you can do one of the following:

- Use dynamic exit services by operator command.
- Use the CSVDYNEX macro.
- Start DFRMM for the first time after an IPL.

Note: Stopping and restarting, or refreshing, the DFSMSrmm procedure does not reload or reactivate your exit routine.

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information.

Installing the EDGUX300 default exit routine

Perform these actions to update or replace the default exit module:

1. Build and install an SMP/E USERMOD to apply the updated source code for the EDGUX300 exit module.

Include the necessary JCLIN statements to get the EDGUX300 load module added to the LINKLIB target library.

You can apply the exit module using an SMP/E USERMOD as shown in [Figure 111 on page 307](#). Modify the FMID and PRE to reflect the release you are running.

Note: The sample EDGUX300 includes code to show you where to insert your own code to communicate with your non-IBM tape library software. You must insert into the sample EDGUX300 exit module the code to retrieve the media type, recording format, and, optionally, capacity.

- a. Allocate a user SAMPLIB data set. In [Figure 111 on page 307](#) the user SAMPLIB data set is defined as MY.RMM.SRCLIB and allocated to DD card SRCLIB.
- b. Copy the shipped EDGUX300 source from SAMPLIB to the user SAMPLIB and modify, as needed, for your installation.
- c. SMP/E RECEIVE the USERMOD.
- d. SMP/E APPLY the USERMOD. Ensure that a DD card exists for the user SAMPLIB in the APPLY job, or as a DDDEF to SMP/E in the target zone.

After performing these steps, the modified version of the EDGUX300 exit module resides in both the user SAMPLIB and SYS1.SAMPLIB. IBM's original copy is only in the distribution libraries at this point. If you accept the USERMOD, only the modified version of the exit module exists. The SMP/E target zone reflects RMID indicators of VMRMM03 for all of these records:

```
SAMP EDGUX300 RMID=VMRMM03 SYSLIB=SAMPLIB
SRC  EDGUX300 RMID=VMRMM03 SYSLIB=SAMPLIB
MOD  EDGUX300 RMID=VMRMM03 LMOD=EDGUX300
LMOD EDGUX300              SYSLIB=LINKLIB
```

The RMID of VMRMM03 for the SAMP record prevents IBM service from being installed. This results in an ID search and notification to you that IBM is the servicing exit.

```

//RMMSTUFF JOB  , 'SLIP IT IN',MSGCLASS=H,MSGLEVEL=(1,1)
//SMPE     EXEC PGM=GIMSMP,
//          PARM='PROCESS=WAIT',
//          DYNAMNBR=120
//SMPCSI   DD DISP=SHR,DSN=MAZRMM3.GLOBAL.CSI
//SMPHOLD  DD DISP=SHR,DSN=SAP1.DUMMY.HOLDDATA
//SMPCNTL  DD *
//          SET BDY(GLOBAL) .
//          RECEIVE .
/*
//SMPPTFIN DD DATA,DLM=##
++USERMOD (VMRMM03) REWORK(2007082) .
++VER (Z038) FMID(HDZ1180) .
++JCLIN .
//EDGUX300 EXEC PGM=IEWL,PARM='LET,NCAL,RENT,REUS,REFR,LIST,XREF'
//SYSLMOD  DD DISP=SHR,DSN=SYS1.LINKLIB
//SRCLIB   DD DISP=SHR,DSN=MY.RMM.SRCLIB
//AEDGMOD1 DD DISP=SHR,DSN=SYS1.AEDGMOD1
//SYSPRINT DD SYSOUT=*
//SYSLIN   DD *
//          INCLUDE AEDGMOD1(EDGUX300)
//          ENTRY  EDGUX300
//          NAME   EDGUX300(R)
++SRC(EDGUX300) TXLIB(SRCLIB) DISTLIB(ASAMPLIB) .
++SAMP(EDGUX300) TXLIB(SRCLIB) DISTLIB(ASAMPLIB) .
###
/*
//SMPAPPLC JOB ('T,H,IOM,,',SYSPROG),'***IBMUSER***',
//          MSGLEVEL=(1,1),MSGCLASS=X,CLASS=A,REGION=0M,
//          NOTIFY=&SYSUID
//S1       EXEC PGM=GIMSMP,
//          PARM='PROCESS=WAIT',
//          DYNAMNBR=120
//SMPCSI   DD DISP=SHR,DSN=MAZRMM3.GLOBAL.CSI
//SRCLIB   DD DISP=SHR,DSN=RMMTST.MAZ.SOURCE
//SMPCNTL  DD *
//          SET BOUNDARY (MVSTZN)
//          .
//          APPLY
//          JCLINREPORT
//          SELECT (VMRMM03)
//          .

```

Figure 111. Building an SMP/E USERMOD to apply the updated EDGUX300 exit module

2. Copy the new exit load module into the LNKLST library.
3. Refresh LLA.
4. Refresh the exit module by using MVS operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

When you use the default exit module, EDGUX300, for the installation exit EDG_EXIT300, it is loaded and activated as an exit routine, if it is not already active, by DFSMSrmm when DFSMSrmm is started and stays loaded and active for the life of the IPL. It can be refreshed or deleted at any time by using one of the MVS system operator commands.

You can use any load module name for your exit module, because you use PROGxx in z/OS parmlib, SETPROG, or CSVDYNEX macro to associate the exit module with the exit. Therefore, you do not have to use EDGUX300 as the load module name. However, using the default exit module name simplifies your implementation, because DFSMSrmm itself ensures that the exit module is loaded and activated.

Deleting the EDG_EXIT300 exit routines

To delete, or selectively delete, the exit routines for the EDG_EXIT300 exit from the system, you can use MVS operator commands.

If DFSMSrmm is running in a shared environment, you must repeat this step on each system.

Writing an exit routine for the EDG_EXIT300 exit

EDG_EXIT300 exit routines run in SUPERVISOR STATE KEY 5 AMODE(31) RMODE(ANY) in the user's address space. Exit modules can be contained in any APF authorized LNKST library or an APF authorized library named on CSVDYNEX macro invocation.

You can write your installation exit so that it can issue commands or call other programs to update external inventories. You can also code the exit module to issue a WTOR if needed.

Registers on entry to an EDG_EXIT300 exit routine

Register

Contents

0

Not applicable

1

Address of a parameter list mapped by the macro EDGPL300

2-12

Not applicable

13

Address of register save area

14

The caller's return address

15

Address of the exit module entry point

EDG_EXIT300 parameter list

All communication is done using the parameter lists fields. The parameter list is mapped by the macro EDGPL300 as shown in [“Installation exit mapping macro: EDGPL300” on page 526](#).

The same parameter list area is passed to each exit module associated with the exit. Each exit module can update the output area of the parameter list to indicate what processing is required by DFSMSrmm. Depending on the order of the exit routines, your exit module might receive the parameter list before any other exit module has processed it, or it might already have been processed. Before taking a decision in your exit routine, you should check to see if an earlier exit routine has already taken a decision. You should be prepared for your decision to be negated by other exit routine processing.

The parameter list input values are:

PL300_HDR

This field contains the control block header.

PL300_IDENT

This field contains the control block ID.

PL300_VERNO

This field contains the control block version number.

PL300_REVNO

This field contains the control block reversion number.

PL300_SUBPOOL

This field contains the control block subpool number.

PL300_LENGTH

This field contains the control block length.

PL300_FUNCTION

This field contains the requested function.

PL300_MEDINF

This field contains gathered media information.

PL300_LSTTEP

This field contains the address of IFGTEP tape exit parameters.

PL300_LSTMEDINF

This field contains the address of the media information.

PL300_LSTMDFN

This field contains the volume media information name.

PL300_LSTOFLGS

This field contains medinf processing flags.

PL300_LSTOFMV

This field contains volume is non-IBM media.

PL300_LSTOFFX

This field contains returned data external format.

PL300_LSTRC

This field contains vendor API return code.

PL300_LSTRS

This field contains vendor API reason code.

PL300_LSTOTDSI

This field contains tape data set info.

PL300_LSTMDTX

This field contains media type external format.

PL300_LSTMDRX

This field contains recording format external format.

PL300_LSTMCAP

This field contains medium capacity.

Registers on return from the EDG_EXIT300 exit routine

Register

Contents

0

Not applicable

1-14

Restored to contents at entry

15

Return code

EDG_EXIT300 installation exit return codes

Table 42 on page 309 shows the contents of register 15 upon return from each exit module.

If any exit module returns return code 0, DFSMSrmm processes the updated parameter list.

Table 42. EDG_EXIT300 installation exit return codes

Return Code	Description
0	Processing was successful. The parameter list might have been updated by the exit.
16	The exit determined that the parameter list passed to it did not conform to the correct specifications.

Chapter 12. Running DFSMSRmm with DFSMSHsm

DFSMSRmm can provide enhanced management functions for the tape volumes that DFSMSHsm uses for each of its tape functions. The way the two products work together depends on how you are using each of them in your installation. Run DFSMSRmm with DFSMSHsm to enhance the management of the volumes DFSMSHsm uses. For example, DFSMSRmm can manage the movement of tapes that must be sent out of the library for disaster recovery. Using DFSMSRmm and DFSMSHsm together, you can use the scratch tape pool rather than a DFSMSHsm tape pool.

DFSMSRmm provides the EDGTVEXT, and EDGDFHSM programming interfaces that can be used by products like DFSMSHsm, OAM, and Tivoli Storage Manager. Use these programming interfaces for DFSMSRmm tape management so that you can maintain correct volume status. DFSMSRmm treats DFSMSHsm like any other tape volume user and retains DFSMSHsm volumes based on vital record specifications and retention period. DFSMSHsm automatically calls EDGTVEXT so you do not need to perform any special set up for DFSMSHsm to communicate with DFSMSRmm. You can use the TVEXTPURGE parmlib option in the DFSMSRmm EDGRMMxx parmlib member to control the action DFSMSRmm takes when DFSMSHsm calls EDGTVEXT. A benefit of this interaction is that DFSMSRmm can prevent DFSMSHsm from overwriting its own control data set backup, automatic dump, ABARS, and copies of backup or migration tapes. Although DFSMSHsm checks its own migration and its own backup tapes, DFSMSRmm checks them as well. For more information on these programming interfaces, see [“Managing DFSMSHsm tapes: EDGDFHSM” on page 250](#) and [“Releasing tapes: EDGTVEXT” on page 249](#).

Defining DFSMSHsm to RACF

To run DFSMSHsm with DFSMSRmm, define DFSMSHsm to RACF. Define the DFSMSHsm user ID with the STARTED class. See [z/OS DFSMSHsm Implementation and Customization Guide](#).

DFSMSHsm issues the ARC0516I error message if DFSMSHsm cannot successfully load the EDGTVEXT exit or if an ABEND occurs. The error message will not scroll off the screen until the operator responds to the message. If an ABEND occurs, the EDGTVEXT exit becomes disabled so that you can correct the problem. Issue the RELEASE RMM command to reactivate the DFSMSHsm invocation of EDGTVEXT.

Authorizing DFSMSHsm to DFSMSRmm resources

Before you can use DFSMSRmm with DFSMSHsm, you are required to authorize DFSMSHsm to STGADMIN.EDG.MASTER, STGADMIN.EDG.OWNER.user, and STGADMIN.EDG.RELEASE.

If you have multiple DFSMSHsm USER IDs, for example in a multi-system or multi-host environment, and any DFSMSHsm ID can create tapes or return tapes to scratch status or return tapes to the DFSMSHsm tape pool, you must authorize each DFSMSHsm USER ID. Define STGADMIN.EDG.OWNER.hsmid for each DFSMSHsm USER ID and give the other DFSMSHsm USER IDs UPDATE access to it. Either STGADMIN.EDG.OWNER.*, STGADMIN.EDG.OWNER.**, or both must be defined with the universal access of READ. That secures users not having their own STGADMIN.EDG.OWNER.userid profiles from releasing their tapes by other users.

See [Chapter 9, “Authorizing DFSMSRmm users and ensuring security,” on page 215](#) for information about authorizing resources.

[Table 43 on page 311](#) shows the authorization required to use scratch tapes with DFSMSHsm.

Table 43. Authorization required to use scratch tapes with DFSMSHsm

Resource	Access Required
STGADMIN.EDG.RELEASE	READ
STGADMIN.EDG.MASTER	READ

Table 43. Authorization required to use scratch tapes with DFSMSHsm (continued)

Resource	Access Required
STGADMIN.EDG.OWNER.hsmid	UPDATE

Table 44 on page 312 shows the authorization required to use DFSMSHsm with a DFSMSHsm-managed scratch tape pool.

Table 44. Authorization required to use DFSMSHsm with a DFSMSHsm scratch pool

Resource	Access Required
STGADMIN.EDG.MASTER	UPDATE
STGADMIN.EDG.OWNER.hsmid	UPDATE

Authorizing ABARS to DFSMSrmm resources

To use DFSMSrmm with DFSMSHsm ABARS, you must assign ABARS IDs the correct levels of authorization to STGADMIN.EDG.MASTER, STGADMIN.EDG.OWNER.user, and STGADMIN.EDG.RELEASE.

If you have multiple ABARS USER IDs, for example in a multi-system environment, and any ABARS ID can return tapes to scratch status, you must authorize each ABARS USER ID. Define STGADMIN.EDG.OWNER.abarsid for each ABARS USER ID and give the other ABARS USER IDs UPDATE access to it. This allows one ABARS ID to release the tapes initially obtained from scratch by the other ABARS ID. Either STGADMIN.EDG.OWNER.*, STGADMIN.EDG.OWNER.**, or both must be defined with the universal access of READ. That secures users not having their own STGADMIN.EDG.OWNER.userid profiles from releasing their tapes by other users.

See Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215 for information about authorizing resources.

Table 45 on page 312 shows the authorization required to use DFSMSrmm with ABARS.

Table 45. Authorization required to use DFSMSrmm with ABARS

Resource	Access Required
STGADMIN.EDG.RELEASE	READ
STGADMIN.EDG.MASTER	READ
STGADMIN.EDG.OWNER.abarsid	UPDATE

Setting DFSMSrmm options when using DFSMSHsm

You use the DFSMSrmm parmlib EDGRMMxx to specify the installation options for DFSMSrmm as described in “Defining system options: OPTION” on page 161. If you are using expiration dates to manage tapes, you should consider the values you specify for the parmlib OPTION command MAXRETPD, RETENTIONMETHOD, and TVEXTPURGE operands and the VLPOOL command EXPDTCHECK operand.

- The MAXRETPD operand specifies the maximum retention period that a user can request for data sets on volumes and is described in “Defining system options: OPTION” on page 161.
- The RETENTIONMETHOD operand specifies the retention method (EXPDT or VRSEL) that DFSMSrmm is to use for DFSMSHsm volumes and is described in “Defining system options: OPTION” on page 161.

When the EXPDT retention method is used, the DFSMSHsm volume can be retained by VOLUME, SET, or FIRSTFILE.

- The TVEXTPURGE operand specifies how you want to handle the release of DFSMSHsm tape volumes and is described in “Defining system options: OPTION” on page 161.

- The VLPOOL command EXPDTCHECK operand described in [“Defining pools: VLPOOL”](#) on page 209 tells DFSMSrmm how to manage a volume based on the expiration date field in the volume label. See [“A pooling example”](#) on page 84 for information about setting up pools.

You can use the EXPDT retention method to avoid the processing of DFSMSShsm volumes on each inventory management VRSEL run. This retention method requires that you use expiration date protection for DFSMSShsm tape volumes, namely: EXPDT=99365

When DFSMSShsm sets 99365 as the expiration date to manage its tapes, 99365 means permanent retention or to never expire. If you choose to use DFSMSShsm expiration date protected tape volumes, DFSMSShsm sets the date 99365 to prevent DFSMSrmm from considering the volumes for release at any time. You must specify the MAXRETPD(NOLIMIT) operand to ensure that DFSMSrmm honors the 99365 date.

You also use the DFSMSrmm parmlib MAXRETPD operand value to reduce the expiration date for all volumes including DFSMSShsm volumes. If you want to reduce the 99365 permanent retention expiration date, specify the MAXRETPD with a value between 0 and 9999 days.

Recommendation: If you want to avoid the overhead of VRSEL processing for tape volumes managed by DFSMSShsm (and similar applications), use the EXPDT retention method. If, however, you require DFSMSrmm to manage the movement of DFSMSShsm volumes, use DFSMSrmm vital record specifications instead of using the 99365 permanent retention date to retain DFSMSShsm volumes. See [“Using the VRSEL retention method to manage DFSMSShsm tapes”](#) on page 315 for information on setting up vital record specifications.

If you use expiration dates, see [“Retaining DFSMSShsm tapes using expiration dates”](#) on page 314 for details on how DFSMSrmm provides facilities to avoid reinitializing tapes before reuse or having the operator reply to IEC507D messages.

Setting DFSMSShsm options when using DFSMSrmm

As DFSMSShsm uses a tape volume, DFSMSrmm records information about data sets and multivolume data sets at OPEN time. DFSMSrmm can use this information to manage the volumes based on the DFSMSrmm policies you define.

DFSMSShsm uses the DCB Open/EOV volume security and verification exit to ensure that an acceptable volume is mounted for DFSMSShsm's use. DFSMSShsm uses this exit to reject unacceptable volumes. For example, DFSMSShsm rejects volumes already in use by DFSMSShsm and volumes that are not authorized for use by DFSMSShsm. DFSMSrmm records information only for those volumes not rejected by DFSMSShsm.

DFSMSrmm provides facilities so that DFSMSShsm can tell DFSMSrmm when it no longer requires a tape volume or when a tape volume changes status. The benefit is that DFSMSShsm cannot mistakenly overwrite one of its own tape volumes if an operator mounts a tape volume in response to a request for a non-specific tape volume.

Setting DFSMSShsm system options

Example: The DFSMSShsm system options that relate to using a tape management system with global or private scratch pools are shown in this example:

```
SETSYS EXITON(ARCTVEXT)/EXITOFF(ARCTVEXT) -
      SELECTVOLUME -
      TAPEDELETION -
      TAPESECURITY -
      PARTIALTAPE
```

Setting DFSMSShsm dump definitions

Example: The DFSMSShsm dump definitions are shown in this example:

```
DEFINE DUMPCCLASS(class -
      AUTOREUSE -
```

```
TAPEEXPIRATIONDATE(date) -  
RETENTIONPERIOD(day))
```

DFSMSrmm support for DFSMSHsm naming conventions

DFSMSrmm supports all DFSMSHsm options and any of the naming conventions for DFSMSHsm tape data sets except for password security on tape volumes. See [“Recommendations for using DFSMSrmm and DFSMSHsm” on page 325](#) for information about the DFSMSHsm options to use with DFSMSrmm.

DFSMSrmm support for retention and pooling

With DFSMSHsm, you can use DFSMSrmm system-based scratch tape pools, exit-selected scratch pools, or DFSMSHsm-managed tape pools.

Recommendation: Use a DFSMSrmm scratch pool. Use the DFSMSHsm-managed pool only when necessary. For example, use the DFSMSHsm-managed pool if you want to keep DFSMSHsm-managed pools for Enhanced Capacity Cartridge System Tapes, as DFSMSHsm fully uses a tape's capacity.

You must let DFSMSHsm decide whether a tape volume contains valid data and whether it should return to the DFSMSrmm scratch pool or DFSMSHsm-managed tape pool.

Define vital record specifications to retain tape volumes until DFSMSHsm finishes with them. DFSMSrmm uses the retention period determined by the vital record specification to extend any expiration date or retention period previously set for the volume. Additionally, you can use vital record specifications to identify volumes that should be moved out of the installation media library for safe keeping, or moved from an automated to manual library. For DFSMSHsm-managed tapes, you do not have to respond to the IEC507D messages that are issued for expiration data protected tapes because DFSMSHsm can override expiration for its own tapes. If you choose to use DFSMSHsm expiration date protected tape volumes, DFSMSHsm sets the expiration date 99365 which means permanent retention to prevent DFSMSrmm from considering the volumes for release at any time.

Retaining DFSMSHsm tapes using expiration dates

Example: To use DFSMSHsm expiration date protection, specify the DFSMSHsm startup option as shown in this example:

```
SETSYS TAPESECURITY(EXPIRATION)
```

If you use a system scratch tape pool for DFSMSHsm tapes, you need a way to manage tapes protected with expiration dates that are set by DFSMSHsm. To help you manage this situation, DFSMSrmm lets you automate the responses to expiration date protection messages for scratch pool tape volumes. Use the parmlib member VLPOOL command to set up this automation. Set the VLPOOL EXPDTCHECK operand to EXPDTCHECK(N) as described in [“Defining pools: VLPOOL” on page 209](#). DFSMSrmm automatically lets your users reuse the volumes in the pool without operator intervention and without creating data integrity exposures.

If you use a DFSMSHsm-managed tape pool, DFSMSHsm validates and overrides expiration dates on its emptied, previously used tapes.

Using the EXPDT retention method to manage DFSMSHsm tapes

To use the EXPDT retention method to manage DFSMSHsm tapes, you must specify RETENTIONMETHOD(EXPDT) for DFSMSHsm tape volumes and a RETAINBY value. With the EXPDT retention method, volumes and volume sets can be retained as individual volumes, as volume sets, or based on the expiration date of the first file.

You must also specify a expiration date or retention period for each volume.

You can specify that volumes purged from DFSMSHsm are to be retained a few extra days to be sure that purged volumes do not contain any data that might still be needed. DFSMSHsm migration and backup volumes can be retained by EXPDT=99365 and you can optionally set EXPDT to the current date

or a future date when the volume is purged from DFSMSHsm with EDGTVEXT. See the parmlib option TVEXTPURGE with the EXPIRE(days) subparameter.

See the "Retention Methods" topic in *z/OS DFSMSrmm Managing and Using Removable Media* for considerations on using the EXPDT retention method.

Using the VRSEL retention method to manage DFSMSHsm tapes

To use the VRSEL retention method to manage DFSMSHsm tapes, you must define movement and retention policies with vital record specifications by specifying data set names or volume serial numbers. To define vital record specifications, use the RMM ADDVRS subcommand. See *z/OS DFSMSrmm Managing and Using Removable Media* for information about the RMM ADDVRS subcommand and the operands you can specify.

When specifying the RMM ADDVRS command operands, it might be helpful to think about the operands in three categories:

- Operands to define retention policies, including COUNT and CYCLES, that are used in these examples
- Operands to define movement policies, including DELAY, LOCATION, and STORENUMBER, that are used in these examples
- Operands to manage the vital record specification itself, including DELETEDATE and OWNER, that are defaults in the RMM ADDVRS subcommand
 - DELETEDATE(1999/365) specifies the date when the vital record specification no longer applies. The default value is 1999/365 which means the vital record specification is permanent. It can only be manually deleted if it is no longer appropriate.
 - OWNER(owner) specifies the user ID that owns the vital record specification.

As shown in the examples that follow, you can specify data set name masks in vital record specifications to manage migration, backup, dumps, TAPECOPY, DUPLEX tape feature, tapes written by ABARS, ABARS accompany tapes, and control data set version backups. You can tailor the examples to define policies for your DFSMSHsm tapes. As you gain more experience defining vital record specifications, you will see that there might be several ways to define the retention and movement policies you desire.

The examples use the current DFSMSHsm data set naming formats. Some old name formats were in use prior to APAR OY20664. APAR OY20664 required you to use PATCH commands to use the new format. The new format is standard in DFSMSHsm. If you have occurrences of the old naming formats, you might need to define vital record specifications that use both naming formats. Delete the vital record specifications with the old naming format once the old names no longer exist. See *z/OS DFSMSHsm Implementation and Customization Guide* for current DFSMSHsm data set naming formats.

Retaining all DFSMSHsm tapes

You can retain all DFSMSHsm tapes that require no movement, with the exception of tapes that are written by ABARS, as shown in Figure 112 on page 315 and Figure 113 on page 316. See “Retaining and moving tapes written by ABARS” on page 321 and “Retaining and moving ABARS accompany tapes” on page 321 for examples for moving and retaining ABARS tapes.

```
RMM ADDVRS DSNAME('mprefix.**') COUNT(99999) CYCLES
RMM ADDVRS DSNAME('bprefix.**') COUNT(99999) CYCLES
RMM ADDVRS DSNAME('authid.**') COUNT(99999) CYCLES
```

Figure 112. Retaining DFSMSHsm tapes that require no movement

DSNAME('mprefix.')**

Specifies the DFSMSHsm-defined migrated data set prefix.

DSNAME('bprefix.')**

Specifies the DFSMSHsm-defined backup and dump data set prefix.

DSNAME('authid.')**

Specifies the DFSMSShsm prefix used for control data set backups.

COUNT(99999)

Specifies to retain all data sets forever or until DFSMSShsm releases the volume by notifying DFSMSrmm through the EDGTVEXT installation exit.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

Retaining open data sets

You can define vital record specifications to control the retention of data sets that might have been left open by a system failure or that might be open during DFSMSrmm inventory management. Using the OPEN or ABEND data set name masks in a vital record specification allows you to define specific policies for these data sets so that they are not retained like normal data sets.

Specify the JOBNAME operand with the DFSMSShsm and ABARS procedure names in the vital record specifications that you define to manage open data sets. This ensures that DFSMSShsm and ABARS volumes are always retained permanently under DFSMSrmm until DFSMSShsm releases them.

```
RMM ADDVRS DSNAME('ABEND') JOBNAME(hsm_proc) COUNT(99999) CYCLES
RMM ADDVRS DSNAME('ABEND') JOBNAME(abars_proc) COUNT(99999) CYCLES
RMM ADDVRS DSNAME('OPEN') JOBNAME(hsm_proc) COUNT(99999) CYCLES
RMM ADDVRS DSNAME('OPEN') JOBNAME(abars_proc) COUNT(99999) CYCLES
```

Figure 113. Retaining DFSMSShsm tapes with the DFSMSShsm and ABARS procedure name

DSNAME('ABEND')

A reserved data set name mask to manage all data sets closed as a result of an abnormal end in a task.

DSNAME('OPEN')

A reserved data set name mask to manage all data sets open when inventory management vital record processing is run or open when the system failed.

JOBNAME(hsm_proc)

DFSMSShsm procedure name.

JOBNAME(abars_proc)

ABARS procedure name.

COUNT(99999)

Specifies to retain all data sets forever or until DFSMSShsm releases the volume by notifying DFSMSrmm through the EDGTVEXT installation exit.

CYCLES

Specifies that DFSMSrmm should retain data sets based on cycles or copies of a data set.

Retaining single file format migration tapes

Figure 114 on page 316 shows how to retain all single file format tapes created as original tapes by DFSMSShsm migration until DFSMSShsm releases the tapes.

```
RMM ADDVRS DSNAME('mprefix.HMIGTAPE.DATASET') -
COUNT(99999) CYCLES
```

Figure 114. Keeping all single file format migration tapes

mprefix

Specifies the DFSMSShsm-defined migrated data set prefix.

COUNT(99999)

Specifies to retain all data sets forever or until DFSMSShsm releases the volume by notifying DFSMSrmm through the EDGTVEXT installation exit.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

Retaining multifile format migration tapes

Figure 115 on page 317 shows how to keep multifile format migration tapes. Multifile format applies to reels and not to cartridges.

```
RMM ADDVRS DSNAME('mprefix.HMIG.T%%%%%%%%.**') -
COUNT(1) CYCLES
```

Figure 115. Keeping multifile format migration tapes

mprefix

Specifies the DFSMSShsm-defined migrated data set prefix.

%

Represents one character of a data set name.

Represents zero or more qualifiers of a data set name.

COUNT(1)

Specifies to keep a single cycle. There should never be more than one cycle as DFSMSShsm generates the data set names using dates and times. COUNT(99999) can be specified and provide the same results.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

Retaining single file format backup tapes

Figure 116 on page 317 shows how to retain all single file format tapes created as originals by DFSMSShsm backup until DFSMSShsm releases the tapes.

```
RMM ADDVRS DSNAME('bprefix.BACKTAPE.DATASET') -
COUNT(99999) CYCLES
```

Figure 116. Keeping single file format backup tapes

bprefix

Specifies the DFSMSShsm-defined backup and dump data set prefix.

COUNT(99999)

Specifies to retain all data sets forever or until DFSMSShsm releases the volume by notifying DFSMSrmm through the EDGTVEXT installation exit.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

Retaining multifile format backup tapes

The example in Figure 117 on page 317 shows how to keep multifile format backup tapes. Multifile format applies to reels and not to cartridges.

```
RMM ADDVRS DSNAME('bprefix.BACK.T%%%%%%%%.**') -
COUNT(1) CYCLES
```

Figure 117. Keeping multifile format backup tapes

bprefix

Specifies the DFSMSShsm-defined backup and dump data set prefix.

%

Represents one character of a data set name.

Represents zero or more qualifiers of a data set name.

COUNT(1)

Specifies to keep a single cycle. There should never be more than one cycle as DFSMSShsm generates the data set names by using dates and times. COUNT(99999) can be specified and provide the same results.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

Retaining and moving TAPECOPY tapes or DUPLEX tapes

The DFSMSShsm TAPECOPY command and the DUPLEX tape feature create tapes that are called alternate tapes. Both the TAPECOPY command and the DUPLEX tape feature use the same naming convention for the alternate tapes.

To retain alternate tapes and to move them to another location, you can define two vital record specifications, one for backup copies and one for migration copies as shown in [Figure 118 on page 318](#).

```
RMM ADDVRS DSNAME('mprefix.COPY.HMIGTAPE.DATASET') -
COUNT(99999) CYCLES LOCATION(REMOTE) STORENUMBER(99999)

RMM ADDVRS DSNAME('bprefix.COPY.BACKTAPE.DATASET') -
COUNT(99999) CYCLES LOCATION(REMOTE) STORENUMBER(99999)
```

Figure 118. Keeping tapes created by the DFSMSShsm TAPECOPY command and DUPLEX tape feature

mprefix

Specifies the DFSMSShsm-defined migration data set prefix.

bprefix

Specifies the DFSMSShsm-defined backup and dump data set prefix.

COUNT(99999)

Specifies to retain all data sets forever or until DFSMSShsm releases the volume by notifying DFSMSrmm through the EDGTVEXT installation exit.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

LOCATION(REMOTE)

Specifies the storage location called REMOTE to which the number of volumes for the data sets specified in STORENUMBER should be moved.

STORENUMBER(99999)

Specifies that all copies should be kept in the specified storage location until they are returned to scratch by DFSMSShsm. Specifying STORENUMBER as 99999 in the two examples ensures that when DFSMSShsm release the original volumes, it also returns the alternate tapes.

As you create copies of migration and backup tapes with the DFSMSShsm TAPECOPY command, DFSMSrmm recognizes the tapes, retains them, and identifies them for movement to the named storage location. During DFSMSShsm RECYCLE processing, DFSMSShsm releases an alternate tape when it releases the original tape. DFSMSrmm identifies that these volumes should return to the library after DFSMSShsm releases the volume. You can use this mechanism to get copy tapes ejected from an automated tape library after creation and returned to the library after RECYCLE.

Use of DFSMSrmm to manage movement is only available for those alternate volumes you have created since implementing the new DFSMSShsm TAPECOPY data set name format. The old naming format used for alternate volumes was the same as that used for the base volumes. If you have copies which were created with the old naming convention we suggest that you use DFSMSShsm RECYCLE against the base

volumes or take a new copy of the base volume. See [“Disaster recovery using DFSMSHsm alternate tapes with DFSMSrmm”](#) on page 324 for information on use of alternate tapes during recovery.

Retaining and moving dump tapes

When defining vital record specifications for retaining dump tapes, consider these conditions:

- The dump class definitions because the dump classes and the vital record specifications you define must work together.
- The dump class and DASD volume serial numbers when you define the data set names you use in the data set name masks.

The data set name format used by DFSMSHsm for dump tapes includes the dump class and the DASD volume serial number. These are the two most likely variables in the data set name on which you will base your retention and movement policies.

- The dump classes that are managed the same.
- The dump classes that require separate management.
- The dump classes that need to be stored off site.
- The need to define additional vital record specifications to support the old data set naming conventions for a period of time.

Figure 119 on page 319 is the minimum you are required to specify in a vital record specification for DFSMSrmm to keep dump tapes and to prevent DFSMSrmm from releasing DFSMSHsm volumes.

```
RMM ADDVRS DSNAME('bprefix.DMP.**') -  
COUNT(1) CYCLES
```

Figure 119. Keeping tapes used for dump

bprefix

Specifies the DFSMSHsm-defined backup and dump data set prefix.

Represents zero or more qualifiers of a data set name.

COUNT(1)

Specifies to retain a single cycle. In the example, the COUNT(1) is used to retain a single dump cycle. Since DFSMSHsm generates a unique name for each dump cycle, COUNT(1) retains all dump cycles.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

If you want to be more specific and manage cycles of dumps, using differing policies you can extend the data set name mask as shown in [Figure 120 on page 319](#).

```
RMM ADDVRS DSNAME('bprefix.DMP.class.V%%%%%%%%.**') -  
COUNT(1) CYCLES
```

Figure 120. Managing cycles of dumps

bprefix

Specifies the DFSMSHsm-defined backup and dump data set prefix.

class

Specifies the DUMPCCLASS you have defined to DFSMSHsm.

V%%%%%%%%

The six character volume serial number.

Represents zero or more qualifiers of a data set name.

COUNT(1)

Specifies to retain a single data set that matches the filter mask.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

You can also define vital record specifications to manage different types of dump classes having different cycles. Figure 121 on page 320 shows the use of a pseudo-GDG data set name to identify dump cycles for movement to storage locations for disaster recovery. A DFSMSrmm pseudo-GDG is a collection of data sets, using the same data set name, that DFSMSrmm manages like a GDG. A pseudo-GDG data set name contains the `-` as a placeholder for the characters in the pattern that change with each generation.

In the example, `-` is used to mask the DFSMSShsm generated date and time in the data set names so that all generations of a dump can be logically managed together.

```
RMM ADDVRS DSNAME('bprefix.DMP.class.V%/%/%/%.T-----.D-----') -  
COUNT(100) CYCLES LOCATION(REMOTE) STORENUMBER(2) -  
DELAY(1)
```

Figure 121. Retaining and moving volumes by cycles

bprefix

Specifies the DFSMSShsm-defined backup and dump data set prefix.

V%/%/%/%

The six character volume serial number.

`-`

Is a place holder for a single character in a data set name mask for a pseudo-GDG data set name. When `-` is used in a data set name mask, DFSMSrmm manages the data sets matching the data set name mask like a generation data group.

COUNT(100)

Specifies to retain all dump cycles managed by DFSMSShsm. 100 is the limit for the number of dump cycles.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

LOCATION(REMOTE)

Specifies the storage location called REMOTE to which the number of volumes for the data sets specified in STORENUMBER should be moved.

STORENUMBER(2)

Two cycles are kept in the storage location REMOTE. All other cycles up to the COUNT(100) value are retained in the HOME location.

DELAY(1)

The current cycle is kept in the library for one day before being removed to the storage location.

In Figure 121 on page 320 all 100 cycles of a dump taken for a specific volume and class combination are kept if produced by DFSMSShsm. For each matching vital record group:

- The current cycle is kept in the library for one day before being removed to the storage location.
- Two cycles are kept in the storage location.
- All other cycles are returned to and kept in the library until DFSMSShsm releases them.
- You can use any combination of LOCATION, STORENUMBER, NEXTVRS, and DELAY to provide the required level of service.
- COUNT(100) is coded so DFSMSrmm keeps all cycles that DFSMSShsm produces. You can tailor the example by using different values for the class name.

You might need to define additional vital record specifications to support the old data set naming conventions for a period of time.

Retaining and moving tapes written by ABARS

Figure 122 on page 321 shows how to keep 10 tapes written by ABARS that were created under DFSMS, and provide storage location management for them based on cycles as if they are a generation data group definition.

When you define a vital record specification as shown in Figure 122 on page 321, DFSMSrmm keeps 10 versions of each of the aggregate backup output data sets and any copies ABARS created. DFSMSrmm retains the latest 10 versions in the REMOTE location and any others in the home location. If there is only one copy of each aggregate backup version produced, and there are 10 versions of each aggregate backup, DFSMSrmm keeps 10 versions of each aggregate backup. If there are 2 copies of each, DFSMSrmm keeps 10 versions of each copy of each aggregate backup.

In the example, the COUNT and STORENUMBER are the same. You could use different values where STORENUMBER is less than or equal to COUNT when you want to store a number of tapes offsite.

```
RMM ADDVRS DSNAME('outputdatasetprefix.%C%%V-') -  
COUNT(10) CYCLES -  
LOCATION(REMOTE) STORENUMBER(10) -
```

Figure 122. Keeping ABARS tapes

outputdatasetprefix

Specifies the same value as entered in the ISMF aggregate processing application as the prefix to be used for output data set.

%

Represents the characters D, C, O, and I that are used by DFSMSShsm ABARS processing.

C%%

Represents the copy number.

V-

Represents the version number DFSMSShsm maintains as a pseudo-generation data group.

COUNT(10)

Specifies the number of versions of the aggregate backup to retain.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

LOCATION(REMOTE)

Specifies the storage location called REMOTE to which the number of volumes for the data sets specified in STORENUMBER should be moved.

STORENUMBER(10)

Represents a number of versions to be kept in a storage location.

Retaining and moving ABARS accompany tapes

Retaining ABARS accompany tapes applies equally to old ABARS tapes and ABARS tapes created under DFSMS. You can retain and manage ABARS accompany tapes based on the naming conventions your installation uses. Modify the example shown in Figure 123 on page 321 with data set names, COUNT, and STORENUMBER values for your installation. In the example shown in Figure 123 on page 321, we assume that all accompany tapes end with 'COPY'.

To keep ABARS accompany tapes, specify this command:

```
RMM ADDVRS DSNAME('app11.**.COPY') COUNT(10) CYCLES -  
LOCATION(REMOTE) STORENUMBER(10) DELAY(1)
```

Figure 123. Keeping ABARS accompany tapes

app11..COPY**

Specifies the data set name mask that identifies the application data set names to retain. DFSMSrmm keeps all data sets that begin with prefix app11 and end with COPY. You can use any data set name

you choose and can select vital record specification options to best match those selected for your aggregate groups or other retention and movement policies.

COUNT(10)

Specifies the number of versions of the data set to retain.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

LOCATION(REMOTE)

Specifies the storage location called REMOTE to which the number of volumes for the data sets specified in STORENUMBER should be moved.

STORENUMBER(10)

Represents a number of versions to be kept in a storage location.

Retaining DFSMShsm control data set backup tapes

To keep all cycles of DFSMShsm control data set and journal backup tapes until DFSMShsm releases them, issue RMM ADDVRS subcommands as shown in [Figure 124 on page 322](#):

```
RMM ADDVRS DSNAME('authid.%CDS.BACKUP.V-') -  
COUNT(99999) CYCLES  
RMM ADDVRS DSNAME('authid.JRNL.BACKUP.V-') -  
COUNT(99999) CYCLES
```

Figure 124. Keeping DFSMShsm control data set and journal backup tapes

authid.**

Specifies the DFSMShsm prefix used for control data set backups.

%

Represents the characters M, B, or O used by DFSMShsm for each of its control data sets.

V-

Represents the version number DFSMShsm maintains as a pseudo-generation data group.

COUNT(99999)

Specifies to retain all data sets forever or until DFSMShsm releases the volume by notifying DFSMSrmm through the EDGTVEXT installation exit.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

All versions of each of the control data set and journal backups are retained until released when DFSMShsm calls DFSMSrmm to release the tape volume. You can optionally use any of the RMM ADDVRS subcommand operands to meet your movement and retention requirements. For example, you can use the LOCATION and STORENUMBER operands to identify that movement is required.

Retaining cycles of dump tapes

You can move dump cycles to storage locations for disaster recovery. Use additional options for pseudo-generation data groups, to retain and move by CYCLES using the old naming format as shown in [Figure 125 on page 322](#).

```
RMM ADDVRS -  
DSNAME('bprefix.DMP.T-..class.D-..V%') -  
COUNT(100) CYCLES LOCATION(REMOTE) STORENUMBER(2) -  
DELAY(1)
```

Figure 125. Moving dumps to storage locations

bprefix

Specifies the data set name mask that identifies the application.

T*

Is the time mask to mask the time the dump was created.

class

Specifies the DUMPCCLASS you have defined to DFSMSHsm.

D*

Is the date mask to mask the date the dump was created.

V%%%%%%%%

The six character volume serial number.

COUNT(100)

Specifies to retain all dump cycles managed by DFSMSHsm. 100 is the limit for the number of dump cycles.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

LOCATION(REMOTE)

Specifies the storage location called REMOTE to which the number of volumes for the data sets specified in STORENUMBER should be moved.

STORENUMBER(2)

Represents a number less than the total number of versions to be kept. All other cycles are retained, up to the COUNT value in the HOME location.

DELAY(1)

The current cycle is kept in the library for one day before being removed to the storage location.

You might need to use both types of dump vital record specifications until you have replaced all old retained volumes.

Retaining ABARS backup tapes

For ABARS backups taken prior to DFSMS, the naming convention was different and used true GDG names. [Figure 126 on page 323](#) shows an example for keeping 10 ABARS backup tapes using GDG names.

```
RMM ADDVRS DSNAME('outputdatasetprefix.%.') GDG -
COUNT(10) CYCLES LOCATION(REMOTE) STORENUMBER(10) -
DELAY(1)
```

Figure 126. Keeping ABARS backup tapes using GDG names

outputdatasetprefix

Specifies the same value as entered in the ISMF aggregate processing application as the prefix to be used for output data set.

%

Represents the characters D, C, O, and I used by DFHSM ABARS processing.

GDG

Specifies that the data set name is a generation data group name.

COUNT(10)

Specifies the number of versions of the aggregate backup to retain.

CYCLES

Specifies that DFSMSrmm retain data sets based on cycles or copies of a data set.

LOCATION(REMOTE)

Specifies the storage location called REMOTE to which the number of volumes for the data sets specified in STORENUMBER should be moved.

STORENUMBER(10)

Represents a number of versions to be kept in a storage location.

DELAY(1)

The current cycle is kept in the library for one day before being removed to the storage location.

Retaining DFSMShsm tapes with extra days retention

You can use DFSMSrmm to release DFSMShsm tapes that are requested to be purged by DFSMShsm. You can also specify that DFSMSrmm retain a tape for a few days after its expiration date has been reached. By default, the expiration date protection for DFSMShsm tapes is done by DFSMShsm. DFSMShsm uses 1999/365 as the expiration date for permanent retention. To enable extra days retention for purged DFSMShsm tape volumes, you need to either, depending on the retention method used, use the TVEXTPURGE(EXPIRE(*days*)) option or set up retention options in the vital record specifications that are used to retain the tape volumes:

- When you use either the EXPDT retention method or the VRSEL retention method, you can specify the number of extra days in OPTION TVEXTPURGE(EXPIRE(*days*)).
- When you use the VRSEL retention method, you can use OPTION TVEXTPURGE(EXPIRE(0)) and combine an UNTILEXPIRED VRS with an EXTRADAYS VRS:
 1. Ensure that the DFSMSrmm EDGRMMxx parmlib OPTION MAXRETPD operand is set to NOLIMIT to prevent DFSMSrmm from reducing the expiration date used for the DFSMShsm tape volumes. See “Defining system options: OPTION” on page 161 for information about the MAXRETPD and TVEXTPURGE operands.
 2. Specify the DFSMSrmm EDGRMMxx parmlib OPTION TVEXTPURGE(EXPIRE) operand.
 3. Define a name vital record specification that specifies the EXTRADAYS retention type and chain the name vital record specification to the vital record specifications used to retain DFSMShsm tape volumes. Also use the COUNT operand to specify the number of days you would like the tape volumes to be retained.
 4. Include the UNTILEXPIRED retention type in the vital record specifications you use to retain DFSMShsm tape volumes and chain these vital record specifications to the name vital record specifications that include the EXTRADAYS retention type.

```
RMM ADDVRS DSNAME('mprefix.**') UNTILEXPIRED NEXTVRS(HSMEXT)
RMM ADDVRS DSNAME('bprefix.**') UNTILEXPIRED NEXTVRS(HSMEXT)
RMM ADDVRS DSNAME('authid.**') UNTILEXPIRED NEXTVRS(HSMEXT)
RMM ADDVRS NAME(HSMEXT) EXTRADAYS COUNT(N)
```

Disaster recovery using DFSMShsm alternate tapes with DFSMSrmm

DFSMShsm supports the creation of a duplicate cartridge tape, called an alternate, for each migration tape or each backup tape. The major use of this is to support remote storage of the alternate so that it can easily be used in case of a disaster.

The recommended process includes these steps related to tape usage:

1. Have a copy of the DFSMShsm control data sets at the disaster site.
2. Perform a TAPERREPL by specifying the DISASTERALTERNATE parameter. This flags each existing alternate tape as a disaster alternate.
3. Place DFSMShsm in DISASTER mode. When in disaster mode DFSMShsm dynamically checks before mounting an input tape whether the needed data resides on a tape having a disaster alternate. If it does, the disaster alternate is requested.

DFSMSrmm recognizes when DFSMShsm is opening tape data sets and tolerates the data set names that DFSMShsm uses as long as the last 17 characters of the data set name match.

When you perform a true replacement by using the TAPERREPL command, without the DISASTERALTERNATE keyword, DFSMShsm uses the data set name it uses for the original tapes.

Securing tapes when running DFSMSHsm and DFSMSrmm

How you use the DFSMSHsm SETSYS option TAPESECURITY influences the TPRACF value in the DFSMSrmm parmlib member EDGRMMxx. DFSMSHsm can optionally use RACF TAPEVOL profiles to secure its volumes. You can combine the DFSMSHsm method for securing volumes with DFSMSrmm RACF options as described in [“Defining system options: OPTION” on page 161](#) to ensure complete security for your tape data.

Select the appropriate combinations for your environment from these options:

- Use RACF with TAPEVOL and TAPEDSN.
- Use DFSMSHsm TAPESECURITY with RACF or EXPIRATIONINCLUDE.
- Use RACF or RACFINCLUDE when RACF TAPEVOL profiles are used.
- Use EXPIRATION or EXPIRATIONINCLUDE when RACF TAPEDSN or DEVSUPxx TAPEAUTHDSN is used.
- Use DFSMSrmm TPRACF with A, P, or N and VLPOOL RACF with Y or N.

Recommendations for using DFSMSrmm and DFSMSHsm

Related reading: See [z/OS DFSMSHsm Implementation and Customization Guide](#) for details on DFSMSHsm data set naming formats.

Follow these guidelines when running DFSMSrmm with DFSMSHsm:

- Run DFSMSHsm with scratch tapes that DFSMSrmm manages so you can have a single scratch pool for all users of tape and so you can gain any benefits available from pre-mounting of scratch tapes in cartridge loaders.
- Use RACF TAPEVOL or DATASET profiles to ensure tape data security on your system.
- Set these suggested DFSMSHsm system option parameters.

```
SETSYS EXITOFF(ARCTVEXT) -  
      SELECTVOLUME(SCRATCH) -  
      TAPEDELETION(SCRATCH) -  
      TAPESECURITY(RACF) -  
      PARTIALTAPE(MARKFULL)
```

Specify EXITOFF(ARCTVEXT) if DFSMSrmm is your only tape management product because DFSMSHsm always calls the DFSMSrmm programming interface EDGTVEXT.

Specify SELECTVOLUME(SCRATCH) to cause DFSMSHsm to issue a nonspecific mount request for output tapes which can be satisfied by any empty tape acceptable to both DFSMSHsm and DFSMSrmm.

Specify PARTIALTAPE(MARKFULL):

- If you are using automatic cartridge loaders so DFSMSHsm marks partially used volumes as full. This allows the first output tape of each task to be a nonspecific mount, which means that you get faster tape mounts. You can specify migration and backup options separately.
 - If you want to get migration and backup data duplicated and moved to a storage location in a timely manner. Marking the tapes full prevents a task's last tape, that is only partially filled, from remaining in the library an additional 24 hours before being duplicated the next day for disaster protection.
 - If you use TAPECOPY or the DUPLEX tape feature to duplicate tapes and ship them to a storage location, use MARKFULL to end one day's cycle and begin another day's cycle. TAPECOPY only processes full volumes which means that the original tapes have no copy until they are full. This means that a copy might not be available at the storage location as quickly as expected. Using the DUPLEX tape feature means that both the original and copy tapes are created at the same time. DFSMSrmm ships the DUPLEX copy to the storage location as quickly as possible. This might create problems if DFSMSHsm expects to continue the writing to the tape the next day. Marking the DUPLEX tapes full allows DFSMSrmm to process both TAPECOPY copies and DUPLEX tapes in the same way.
- Set these suggested dump definitions:

```
DEFINE DUMPCLASS(class -  
    AUTOREUSE -  
    RETENTIONPERIOD(days))
```

Use RETENTIONPERIOD instead of TAPEEXPIRATIONDATE to allow the tapes to be reused or written to by any other program without their needing to be reinitialized after DFSMSHsm expires them.

Use AUTOREUSE to have the tapes returned to a scratch pool as soon as the data on the tape are invalidated. This option is necessary for DFSMSHsm to call DFSMSrmm to release the tape volume. Without AUTOREUSE, a DELVOL command must be issued for each tape after it is returned to the locale of the tape drives.

Do not use AUTOREUSE for a DFSMSHsm-managed pool when also removing tapes from the vicinity of the tape drives. This is because DFSMSHsm might select a remotely located tape before DFSMSrmm is able to cause the tape's physical return to the locale of the tape drives.

Chapter 13. Performing inventory management

DFSMSrmm samples provided in SAMPLIB

- EDGJHSKP Sample JCL for Using the EDGHSKP Utility
- EDGJHKPA Sample IBM Tivoli Workload Scheduler for z/OS Job for Allocating the Data Sets Required for Inventory Management

DFSMSrmm provides the EDGHSKP utility to help you perform inventory management. You can run inventory management functions in a single job step or can split requests into multiple steps and jobs if required. Running inventory management as a single step allows DFSMSrmm to optimize processing. The default processing for EDGHSKP is to run all inventory management functions in sequence as described in “EXEC parameters for EDGHSKP” on page 337. Use this topic to set up and run inventory management activities and to schedule all DFSMSrmm utilities.

You use the EDGHSKP utility to request that DFSMSrmm perform inventory management processing. The utility validates parameters, checks that the correct files are allocated and can be used, and requests that the DFSMSrmm subsystem performs the functions you request. The utility uses the EDGBKUP utility to perform backup processing.

During DFSMSrmm subsystem inventory management processing, you can see the address space executing EDGHSKP waiting for the subsystem processing to complete. Use the RMM LISTCONTROL subcommand with the CNTL operand to see which inventory management functions are active and when the individual functions were last successfully processed. If you cancel a batch job running DFSMSrmm processing (such as EDGHSKP), the processing in the DFSMSrmm subsystem is interrupted and the function ends early.

When starting the DFSMSrmm subsystem, you can use the HSKP parameter to allow or to restrict (limit) the housekeeping functions that are run. If the HSKP parameter is set to "LIMITED", the following housekeeping functions (CATSYNC, DSTORE, EXPROC and VRSEL) are restricted and cannot be run.

During inventory management the DFSMSrmm subsystem issues messages that describe processing. Messages are in the MESSAGE data set. During expiration processing, DFSMSrmm identifies data sets that might be open and puts them in a list in the MESSAGE file. See [Figure 142 on page 364](#) for an example of this output.

Scheduling DFSMSrmm utilities

You can schedule each DFSMSrmm utility to run on its own, or schedule many activities to run together in a sequence. You can schedule activities such as vital records processing to be performed daily. Other activities can be scheduled less frequently, for example on a weekly basis. You can use a job scheduling product like the IBM Tivoli Workload Scheduler for z/OS to run the utilities. See [Chapter 20, “Running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS,” on page 477](#) for information about using the IBM Tivoli Workload Scheduler for z/OS for scheduling DFSMSrmm utilities.

[Table 46 on page 327](#) suggests how frequently to run DFSMSrmm utilities. References are listed to help you locate the information about the utilities.

Table 46. Scheduling DFSMSrmm utilities

Activity	Frequency	Reference
Processing vital records	Daily	“JCL for EDGHSKP” on page 337 and “Running vital record processing” on page 345

Table 46. Scheduling DFSMSrmm utilities (continued)

Activity	Frequency	Reference
Performing expiration processing	Daily	“JCL for EDGHSKP” on page 337 and “Running expiration processing” on page 359
Performing storage location management	Weekly	“JCL for EDGHSKP” on page 337 and “Running storage location management processing” on page 357
Creating an extract data set	Daily	“JCL for EDGHSKP” on page 337
Creating volume movement and inventory reports	Weekly	See z/OS DFSMSrmm Reporting
Creating security and audit reports	Monthly	See z/OS DFSMSrmm Reporting
Backing up the DFSMSrmm control data set	Daily	“JCL for EDGHSKP” on page 337 , “Backing up the control data set” on page 370 , and “Backing up the control data set” on page 386
Backing up the DFSMSrmm journal	Daily	“JCL for EDGHSKP” on page 337 , “Backing up the control data set” on page 370 , and “Backing up the DFSMSrmm control data set and journal” on page 387
Verifying the control data set	Monthly	“Using EDGUTIL for tasks such as creating and verifying the control data set” on page 401
Initializing and erasing volumes	Weekly	Chapter 15, “Initializing, erasing, and scanning tape volumes,” on page 423

Running inventory management

If you decide to run inventory management in multiple steps, we suggest that you run inventory management as follows:

1. Back up the control data set and journal and keep multiple backup generations to aid recovery.
2. Run vital record processing first before expiration and storage location management processing. This is to identify which volumes to retain and where volumes should be moved, based on vital record specifications.
3. Run expiration processing to identify those volumes not required for vital records that are ready to expire. During expiration processing, release actions for volumes are noted.

4. IBM recommends that you verify that volumes removed from VRS retention, or that have otherwise expired, are the volumes that are expected to expire. Consider adding a job step that reviews the lists of volumes that are no longer under VRS control or have expired and prevent further automated processing if necessary. DFSMSrmm provides VLPOOL RELEASEACTION(NOTIFY) to help with this task. You can use the automated setting of this release action to ensure that you have to confirm all volumes before they can be returned to scratch.
5. Run storage location management to set a destination location for a volume. Optionally run storage location management to assign shelf locations in storage locations for volumes that are being sent out of or returned to the removable media library. You must run storage location management processing after vital record processing has been successfully run, but not necessarily in the same run of EDGHSKP.
6. Run an EDGINERS job after expiration processing completes to ensure that volumes that are waiting to be released and waiting to be initialized are returned to scratch. Use automatic processing to request EDGINERS to initialize and erase any volumes flagged in the control data set. Run EDGINERS before and after each run of EDGHSKP.
7. Create an extract data set to use as input to the DFSMSrmm report utility, EDGRPTD, and create a report that shows the new volume movements required.

DFSMSrmm prevents expiration processing from releasing volumes that have been updated since the last run of vital records processing. You normally need a minimum of two EDGHSKP runs to process an expired volume and return it to scratch, which involves at least a 24 hour delay if you are running expiration processing daily. During this time, you can reclaim an expired volume from pending release status by setting a new expiration date. You can also reclaim an expired volume from scratch status by using the RMM CHANGEVOLUME subcommand to change the volume status to master or user. When you use DFSMSrmm to reclaim a volume that resides in a system-managed tape library to master or user status, the volume is changed to PRIVATE in the TCDB.

Note: If the DFSMSrmm subsystem is started with the HSKP parameter set to "LIMITED" then the following housekeeping functions are not run (CATSYNC, DSTORE, EXPROC and VRSEL). Functions like VERIFY, RPTTEXT and BACKUP continue to be allowed. If you want to continue to allow all housekeeping functions to be able to run, you can set the HSKP parameter to "ALL" which is also the default.

Inventory management considerations for EXPROC and VRSEL processing

- You do not need to run VRSEL processing unless any volumes are defined with the VRSEL retention method. Only EXPROC processing is required to handle expiration of all volumes managed by the EXPDT retention method.
- EXPROC processing provides a summary of volumes by retention method.
- The expiration date of volumes is set during OPEN processing, so for volumes managed by the EXPDT retention method, no special considerations exist for open data sets; they are managed based on the volume EXPDT.
- For volumes managed by the EXPDT retention method, no special considerations exist for data sets closed by ABEND processing or that are DELETED; they are managed based on the volume EXPDT.
- Volumes managed by the EXPDT retention method are included only in the EXPDTDROP limit. VRSRETAIN and VRSDROP limits apply only to volumes managed by VRSEL retention method.
- For disaster recovery testing, ensure that expiration processing only selects the subset of volumes needed for testing and does not include production volumes.

Managing exceptions in retention

DFSMSrmm provides several parmlib options to help you ensure that data is retained as expected and that the correct retention policies are in place. Use these options to customize how you expect DFSMSrmm to detect and handle exceptions to normal retention processing.

VRSMIN

Use this to ensure that sufficient VRSEs are defined to support your retention policies.

VRSCCHANGE

Use this to detect when a VRS might have been added, changed, or deleted, which could cause the number of data sets and volumes being retained to differ than expected.

VRSDROP

Use this to establish your normal expectations for numbers or percentage of volumes being removed from VRS retention.

VRSRETAIN

Use this to establish your normal expectations of the numbers or percentages of newly assigned volumes to be retained by VRS.

EXPDTDROP

Use this to establish your normal expectations for the numbers or percentages of volumes dropped from expiration date retention.

You can use the actions for each of these options to allow DFSMSrmm processing to continue after an information message is issued or you can have the message issued and either a warning or an error return code set. You can build recovery actions into your production batch job to take the correct actions when one of these alerts occurs based on the job step return code.

All of these options, except for VRSCCHANGE, provide you with a way to disable the checking that DFSMSrmm performs. When an option is disabled, DFSMSrmm does not count the resources and does not check if the actions are to be taken. To disable an option set the action to OFF.

To help you establish the settings for these parmlib options, each time you run VRSEL processing, and you have not disabled the limit checking, DFSMSrmm counts and calculates how many volumes are affected and lists information in the MESSAGE file, as shown in this example. You can see how many volumes and what percentage of these are affected. In particular, look at messages:

EDG2242I

Shows how many volumes are VRS retained as VRSEL starts

EDG2243I

Shows how many volumes are newly assigned as VRSEL starts

EDG2244I

Shows number of volumes dropped from VRS retention and the percent of the initially VRS retained volumes

EDG2245I

Shows number of newly assigned volumes retained for VRS and the percent of the initially newly assigned volumes

EDG2246I

Shows how many data sets are excluded from VRSEL processing

EDG2427I

Shows how many volumes are expiration date retained at the start of expiration processing

EDG2428I

Shows number of volumes dropped from EXPDT retention and the percent of the initially expiration date retained volumes

```

1EDG6001I INVENTORY MANAGEMENT STARTING ON 01/11/2010 AT 04:24:44 - PARAMETERS IN USE ARE
DATEFORM(A),VRSEL,CATSYNCH,DSTORE(LOCATION(*:*)),EXPROC
EDG6013I THE SYSDN OPTIONS CURRENTLY IN USE ARE
EXPROC
EDGSPLCS(YES)
VOLUMES(A039*)
EDG2309I THE PARMLIB OPTIONS CURRENTLY IN USE ARE
VRSEL(NEW)
VRSJOBNAME(2)
VRSMIN(1,FAIL)
VRSCCHANGE(INFO)
VRSDROP(PERCENT(10),INFO) VRSRETAIN(PERCENT(80),INFO) EXPDTDROP(PERCENT(10),INFO)
SMSTAPE(PURGE(ASIS) UPDATE(EXITS,SCRATCH,COMMAND))
CATRETPD(12)
UNCATALOG(S)
TPRACF(N)
NOTIFY(Y)
SYSID(EZU0000)
CATSYSID(*)

```

```

        RETAINBY(VOLUME)
        MOVEBY(VOLUME)
        GDG(CYCLEBY(GENERATION),DUPLICATE(BUMP))
EDG2308I CHANGES HAVE BEEN MADE TO VRS POLICIES SINCE THE PREVIOUS INVENTORY MANAGEMENT RUN
EDG2229I NUMBER OF VRS RECORDS READ IS 8
EDG2238I NUMBER OF UNUSED VRS RECORDS IS 2
EDG2246I NUMBER OF DATA SET RECORDS EXCLUDED FROM VRSEL =          1 25%
EDG2242I INITIAL NUMBER OF VRS RETAINED VOLUMES           =          0  0%
EDG2244I NUMBER OF VRS RETAINED VOLUMES TO BE DROPPED      =          0  0%
EDG2243I INITIAL NUMBER OF NEWLY ASSIGNED VOLUMES          =         18 100%
EDG2245I NUMBER OF NEWLY ASSIGNED VOLUMES TO BE RETAINED=          6 33%
EDG2234I DFSMSrmm CDS CATALOG STATUS UNKNOWN FOR RMMUSER.CATCDSN VOLUME A03971 FILE 1 SYNCHRONIZED TO CATALOG STATUS
YES
EDG2234I DFSMSrmm CDS CATALOG STATUS UNKNOWN FOR RMMUSER.CATCDSY VOLUME A03972 FILE 1 SYNCHRONIZED TO CATALOG STATUS
YES
EDG2307I INVENTORY MANAGEMENT TASK CATSYNCH COMPLETED SUCCESSFULLY
EDG2444I EXIT PROCESSING DISABLED FOR THIS EXPROC RUN - NO ACTIVE EXIT MODULE FOR EXIT EDG_EXIT200
EDG2427I INITIAL NUMBER OF EXPDT RETAINED VOLUMES           =         14 44%
EDG2428I NUMBER OF EXPDT RETAINED VOLUMES TO BE DROPPED      =         14 100%
EDG2420I PHYSICAL VOLUMES READ                               =         21 54%
EDG2420I LOGICAL VOLUMES READ                                =         11 28%
EDG2420I STACKED VOLUMES READ                                =          7 18%
EDG2420I RM_VRSEL VOLUMES READ                               =         22 56%
EDG2420I RM_EXPDT VOLUMES READ                               =         17 44%
EDG2420I TOTAL VOLUMES READ                                   =         39 100%
EDG2421I PHYSICAL VOLUMES UPDATED                            =         21 100%
EDG2421I LOGICAL VOLUMES UPDATED                             =         11 100%
EDG2421I RM_VRSEL VOLUMES UPDATED                            =         18 82%
EDG2421I RM_EXPDT VOLUMES UPDATED                            =         14 82%
EDG2421I TOTAL VOLUMES UPDATED                               =         32 82%
EDG2422I PHYSICAL VOLUMES, THIS RUN, KEPT FOR VRS           =          6 29%
EDG2422I TOTAL VOLUMES, THIS RUN, KEPT FOR VRS              =          6 15%
EDG2423I PHYSICAL VOLUMES, THIS RUN, ASSIGNED TO STORES     =          1  5%
EDG2423I TOTAL VOLUMES, THIS RUN, ASSIGNED TO STORES        =          1  3%
EDG2435I PHYSICAL VOLUMES SELECTED FOR EXPROC                =         21 100%
EDG2435I LOGICAL VOLUMES SELECTED FOR EXPROC                 =         11 100%
EDG2435I RM_VRSEL VOLUMES SELECTED FOR EXPROC                =         18 82%
EDG2435I RM_EXPDT VOLUMES SELECTED FOR EXPROC                =         14 82%
EDG2435I TOTAL VOLUMES SELECTED FOR EXPROC                   =         32 82%
EDG2424I PHYSICAL VOLUMES SET PENDING RELEASE                =         15 71%
EDG2424I LOGICAL VOLUMES SET PENDING RELEASE                 =         11 100%
EDG2424I RM_VRSEL VOLUMES SET PENDING RELEASE                =         12 67%
EDG2424I RM_EXPDT VOLUMES SET PENDING RELEASE                =         14 100%
EDG2424I TOTAL VOLUMES SET PENDING RELEASE                   =         26 81%
EDG2425I PHYSICAL VOLUMES RETURNED TO SCRATCH                =          7 33%
EDG2425I LOGICAL VOLUMES RETURNED TO SCRATCH                 =          5 45%
EDG2425I RM_EXPDT VOLUMES RETURNED TO SCRATCH                =         12 86%
EDG2425I TOTAL VOLUMES RETURNED TO SCRATCH                   =         12 38%
EDG2426I PHYSICAL VOLUMES - SCRATCH RECORDS WRITTEN          =          2 10%
EDG2426I RM_EXPDT VOLUMES - SCRATCH RECORDS WRITTEN          =          2 14%
EDG2426I TOTAL VOLUMES - SCRATCH RECORDS WRITTEN             =          2  6%
EDG2429I MAIN INVENTORY MANAGEMENT UPDATES HAVE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK VRSEL COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK DSTORE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK EXPROC COMPLETED SUCCESSFULLY
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0

```

Through analysis of several runs, you can determine what your normal expectations are and use these to set the thresholds, limits and actions you require.

The ACTIVITY file contains detailed information about data set and volume related changes to the control data set made by DFSMSrmm during inventory management. EDGJACTP sample JCL provides reports from a combination of ACTIVITY file and the report extract file data set and volume records to help you analyze the results of processing.

For the best retention limit reporting you need the ACTIVITY file and the extended records from the report extract file (XREPTXT or REPTXT DD) from the same EDGHSKP run. This could be one of the following:

- VRSEL, VERIFY, RPTEXT
- VRSEL, RPTEXT
- VRSEL, EXPROC, RPTEXT
- EXPROC, RPTEXT

Other supported EDGHSKP parameters might also have been requested.

The sample retention limit reports require that the date formats used for the ACTIVITY file and report extract file are the same and for correct processing require either ISO or Julian date format.

If you did not include RPTEXT, you should create a report extract as soon as possible and definitely before any other inventory management functions are used to update the control data set. The longer time you

leave between the creation of the ACTIVITY file and the report extract the less accurate the retention limit reporting.

When you use a trial run, or the retention limit trigger action is FAIL, no updates are made to the CDS by inventory management, so the report extract reflects the CDS contents at the start of the run, plus any tape activity during the run. In all other cases, the report extract also reflects the updates made by the inventory management run. Regardless of this, EDGJACTP VRSRETN, VRSRETNS, EXPDROP, and EXPDROPS reports provide a good analysis of the processing performed or that would have been performed.

Inventory management considerations

You can perform some inventory management activities only under specific conditions:

- In an environment where the DFSMSrmm control data set is shared, and you use system-managed tape, you must run inventory management on a system that has access to the TCDB. If the TCDB is not shared by each system that is sharing the control data set, and there are multiple TCDBs, run inventory management at least once against each TCDB on a system that has access to that TCDB.
- When either of these conditions exist, you must run inventory management on a system that has access to the catalogs.
 - The DFSMSrmm control data set is shared, the DFSMSrmm control data set and catalogs are not synchronized, and you want to retain data sets that are based on the catalog information.
 - The DFSMSrmm parmlib OPTION command UNCATALOG is coded with Y or S to indicate that catalog processing is required.
- In an unshared user catalog environment, you must specify the DFSMSrmm EDGRMMxx parmlib OPTION command CATSYSID operand to use DFSMSrmm. When you use unshared catalogs, ensure that the DFSMSrmm control data set and the user catalogs are synchronized. You must run inventory management expiration processing on each different catalog environment.
- When DFSMSrmm is active, you cannot reorganize the control data set.
- The DFSMSrmm subsystem address space performs all inventory management functions, except backing up the control data set and backing up the journal. During inventory management processing, the address space running EDGHSKP waits until processing completes. Once the subsystem processing completes, EDGHSKP performs backup processing in its home address space; this can include the backup of the DFSMSrmm control data set and the journal.
- When inventory management functions are active, you can continue using RMM TSO subcommands and DFSMSrmm continues to record tape usage.
- You must define at least one vital record specification in order to run DFSMSrmm vital record processing. The EDGRMMxx parmlib OPTION command VRSMIN operand provides control over this processing.

Example: Use the RMM ADDVRS subcommand to define a vital record specification.

```
RMM ADDVRS DSN('**') WHILECATALOG
```

DFSMSrmm inventory management considerations when client/server support is enabled

This topic describes utilities that provide restricted functions when run on a client or server system.

DFSMSrmm Utility	Considerations	Where to Find More Information
EDGHSKP	- You can run all inventory management functions except BACKUP on a client system. However, because there is no direct connection to the control data set, the elapsed times will be longer when run on a client system. We recommend you run all inventory management functions other than CATSYNCH and EXPROC on either the server or a standard DFSMSrmm system. - When you attempt backup on a client system, the utility ends with RC16 and DFSMSrmm issues message EDG6137I stating that it cannot be run on a client system	Chapter 13, “Performing inventory management,” on page 327

Allocating data sets for inventory management

DFSMSrmm samples provided in SAMPLIB

- EDGJHKPA Sample JCL for Allocating the Data Sets Required for Inventory Management
- EDGPHKPA Sample OPC for Allocating the Data Sets Required for Inventory Management

Before running EDGHSKP, you must define several data sets. The data sets that are used by both the EDGHSKP utility and the DFSMSrmm subsystem address space must be pre-allocated and cataloged as shown in [Table 47 on page 333](#). These data sets can be directed to EAS by exploiting DC attributes, SMS ACS routines, and JCL keywords.

Table 47. DFSMSrmm EDGHSKP data sets

DD Statement	Preallocated, Cataloged, DASD Data Set	Description
ACTIVITY	Yes	Contains detailed information about data set related changes DFSMSrmm makes to the control data set during inventory management. This data set is required when you specify the VERIFY parameter.
BACKUP	No	Contains the backup copy of the DFSMSrmm control data set. Specify this data set to run backup processing for the control data set. You can back up directly to tape when you specify the BACKUP(DSS) parameter even when DFSMSdss concurrent copy is not available. BACKUP DD is optional when you specify BACKUP(COPY) and your environment is SMS managed.
DSSOPT	No	Contains COPY, DUMP, or RESTORE command options used by DFSMSdss during backup processing. See “Customizing the DSSOPT DD statement” on page 384 for information about changing the commands.
EDGSPLCS	Yes	Contains statements to be used with the EDGSPLCS utility. They are created by EXPROC when you use the SYSIN EXPROC option EDGSPLCS(YES).

Table 47. DFSMSrmm EDGHSKP data sets (continued)

DD Statement	Preallocated, Cataloged, DASD Data Set	Description
JRNLBKUP	No	Contains the backup copy of the DFSMSrmm journal. Specify this data set to run backup processing for the journal. DFSMSrmm uses IDCAMS to back up the journal when you specify the BACKUP(AMS) or BACKUP(DSS) parameter. You can back up directly to tape when you specify the BACKUP(DSS) parameter even when DFSMSdss concurrent copy is not available. Although the journal is initialized as RECFM=VB and LRECL=32756, when running EDGHSKP or EDGBKUP PARM='BACKUP BACKUP(DSS)' you can specify RECFM=VB,LRECL=9248 for JRNLBKUP DD to optimize dasd space.
MESSAGE	Yes	Lists the messages the DFSMSrmm subsystem issues during inventory management. This data set is required.
REPORT	Yes	Contains a detailed report of DFSMSrmm vital record specification processing. Specify if you want a report when you have specified the VRSEL parameter.
REPTXT	Yes	Contains the extract copy of the DFSMSrmm control data set. The extract copy is called the extract data set. You must specify either the REPTXT DD statement or the XREPTXT DD statement when you use the EDGHSKP RPTXT parameter. If the DD name is XREPTXT, RMM creates only extended records. If the DD name is REPTXT, RMM creates all records except extended records.
SYSIN	No	An optional file that can contain commands to tailer inventory management processing. See “SYSIN file for the EDGHSKP utility” on page 342 .
SYSRINT	No	Contains the utility program messages that IDCAMS and ADRDSSU issue when backing up the DFSMSrmm control data set. The SYSRINT data set is required when you specify the BACKUP parameter. This data set can be a SYSOUT file.
XREPTXT	Yes	Contains the extract copy of the DFSMSrmm control data set. The extract copy is called the extract data set. You must specify either the REPTXT DD statement or the XREPTXT DD statement when you use the EDGHSKP RPTXT parameter.

To avoid enqueue contention, consider these conditions:

- These data sets cannot be created in the same job as they are used. If you plan to retain multiple versions of these data sets, consider using a subsequent job step to copy the data sets to a new GDG generation.
- The JCL used to run EDGHSKP should specify DISP=SHR for these data sets.

Recommendation: For your regularly scheduled inventory management, use a data set to serialize the jobs that run EDGHSKP to ensure that only one copy of EDGHSKP is running on a system. You can run EDGHSKP with RPTXT, EXPROC, or CATSYNCH options at the same time on different systems. You can also run EDGHSKP with the RPTXT option at the same time on the same system. To run each of these tasks in parallel, for example, to run EXPROC on each system in a complex at the same time, use a different preallocated and cataloged data set for each job instead of including the ENQDSN DD statement.

To avoid problems that might occur when multiple EDGHSKP jobs are running together, add a DD statement to the job step that runs EDGHSKP as shown in [Figure 127 on page 335](#):

```
//ENQDSN DD DISP=OLD,DSN=RMM.HSKP.ENQ
```

Figure 127. JCL for adding a DD statement to the EDGHSKP job step

Alternatively you can use the sample RMM OPC application which uses OPC special resources to prevent multiple jobs running together as described in [Chapter 20, “Running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS,”](#) on page 477.

To protect the data sets that must be preallocated, cataloged, and on DASD, ensure that the RACF user ID associated with the DFSMSrmm subsystem has authority to write to the data sets.

Recommendation: Always run DFSMSrmm with the optional files ACTIVITY and REPORT. Maintain these files, as well as the MESSAGE, BACKUP, JRNLBKUP, SYSPRINT, REPTTEXT, and XREPTTEXT files, as GDGs or copy them to GDGs. You might find that multiple versions of the files are helpful for use in diagnosing problems or for creating reports.

Creating an extract data set

You can create an extract data set to use as input for creating reports. You can select the records to be extracted from the DFSMSrmm control data set. When you specify the RPTEXT command in the EDGHSKP SYSIN file, you can be selective about the records to be extracted. When you do not use the RPTEXT command in SYSIN, you can use the DD name to determine if extended records are created. When you specify the XREPTTEXT DD statement, the extract contains only extended records that are a combination of data set information and volume information that can be useful as input to DFSMSrmm reporting tools. When you code only the REPTTEXT DD, all records other than the extended records are created.

DFSMSrmm reads sequentially through its control data set, and creates a record in the extract data set for each shelf location, volume, data set, software product, owner and vital record specification record that is selected. DFSMSrmm can convert information like the date information into a format and time zone offset that you specify. The extract data set is a point-in-time copy of the control data set contents. Use the RMM TSO SEARCH and LIST subcommands to obtain the most current information.

You can use the extract data set as input to any of these programs:

- DFSMSrmm Report Generator
- DFSMSrmm report utility EDGRPTD. This utility does not process extended records.
- DFSMSrmm-supplied exec EDGRRPTE. This exec processes only extended records.

Refer to [z/OS DFSMSrmm Reporting](#) for information about using the DFSMSrmm Report Generator, EDGRPTD, and EDGRRPTE.

Calculating DASD space and placement for the extract data set

Table 48 on page 335 helps you calculate DASD space requirements for the extract data set specified by the REPTTEXT DD statement or the XREPTTEXT DD statement.

Table 48. DFSMSrmm extract data set DASD space requirements

Extract Data Set Content	DASD Space
Data sets	• 500 KB for every 1000 data sets when data set records are selected. • 1564 KB for every 1000 data sets when extended records are selected.
Shelf locations in the library that do not contain volumes	11 KB for every 1000 shelf locations
Shelf locations in storage locations	11 KB for every 1000 shelf locations
Owners	35 KB for every 1000 volumes

Table 48. DFSMSrmm extract data set DASD space requirements (continued)

Extract Data Set Content	DASD Space
Software products	17 KB for every 1000 software products
Volumes	1000 KB for every 1000 volumes when volume records are selected.
Vital record specification	20 KB for every 1000 vital record specifications

Convert the final figure into a space allocation. The extract data set has a record format of variable-length blocked records. Choose either a system-determined block size, or a block size suitable for the device your installation uses.

To calculate the space required:

1. The number of KB is the number to use in the space allocation.
2. Use a fraction of this total space as secondary space if you want to have an extract data set that extends to multiple extents.

You can use JCL as shown in [Figure 128 on page 336](#) to create the extract data set prior to running EDGHSKP.

```
//REPTXT DD DISP=(NEW,CATLG),DSN=RMM.EXTRACT.DSET,
//      RECFM=VB,BLKSIZE=0,
//      UNIT=SYSALLDA,AVGREC=U,SPACE=(1024,(pp,ss))
```

Figure 128. JCL for specifying the extract data set

where:

pp

Specifies the calculated primary space.

ss

Specifies a fraction used for secondary space.

Placing the extract data set

You can place the extract data set on any volume.

JCL for creating an extract data set

To create an extract data set, specify the RPTXT parameter. You can submit a job with JCL as shown in [Figure 129 on page 336](#). To create an extract data set that contains extended records, uncomment the XREPTXT DD statement or code the RPTXT command with the RECORDS(X) operand in the SYSIN file. When you specify both XREPTXT and RPTXT, DFSMSrmm uses the XREPTXT DD statement.

```
//HSPK EXEC PGM=EDGHSKP,
//      PARM='RPTXT,DATEFORM(E)'
//MESSAGE DD DISP=SHR,DSN=HSPK.MESSAGES
//REPTXT DD DISP=SHR,DSN=MASTER.EXTRACT
//*XREPTXT DD DISP=SHR,DSN=MASTER.EXTENDED.EXTRACT
//SYSIN DD *
RPTXT
/*
```

Figure 129. JCL for creating an extract data set

Use the DATEFORM parameter to specify the format for date fields in the report extract data set. The DATEFORM parameter can take any of these values:

Value	Language	Format	Example
A	American	mm/dd/yyyy	12/15/2013

Value	Language	Format	Example
E	European	dd/mm/yyyy	15/12/2013
I	ISO	yyyy/mm/dd	2013/12/15
J	Julian	yyyy/ddd	2013/349
D	Default	Installation's default in EDGRMMxx	Initially set to Julian

DFSMSrmm provides the format of the records in the extract data set in mapping macros. See [z/OS DFSMSrmm Reporting](#) for layouts of the macros. You can use DFSORT to sort the extract data set records to create many types of reports.

For example, you could select the extract records that show volumes with temporary-read errors. Sort the resulting list by descending number of errors. Use this list to identify the damaged volumes that should be replaced. Then, you could use the RMM CHANGEVOLUME subcommand with the RELEASEACTION(REPLACE) operand to update DFSMSrmm with the required action.

JCL for EDGHSKP

Figure 130 on page 337 shows example JCL for EDGHSKP.

```
//HSKP EXEC PGM=EDGHSKP,
// PARM='VRSEL,EXPROC,DSTORE,BACKUP(DSS),RPTTEXT,DATEFORM(E)'
//SYSPRINT DD SYSOUT=A
//DSSOPT DD *
//          CONCURRENT OPTIMIZE(1) VALIDATE
//          *
//BACKUP DD DSN=BACKUP.FILE.NAME,DISP=(NEW,CATLG),
// UNIT=SYSALLDA,AVGREC=U,SPACE=(4096,(1000,500)),
// LRECL=9216,BLKSIZE=0,RECFM=U
//JRNLBKUP DD DSN=JOURNAL.BACKUP.FILE.NAME,DISP=(NEW,CATLG),
// UNIT=SYSALLDA,AVGREC=U,SPACE=(4096,(1000,500)),
// LRECL=9248,BLKSIZE=0,RECFM=VB
//MESSAGE DD DSN=MESSAGE.FILE.NAME,DISP=SHR
//REPORT DD DSN=REPORT.FILE.NAME,DISP=SHR
//ACTIVITY DD DSN=ACTIVITY.FILE.NAME,DISP=SHR
//*RPTTEXT DD DSN=REPORT.EXTRACT.FILE.NAME,DISP=SHR
//XRPTTEXT DD DSN=REPORT.EXTENDED.EXTRACT.FILE.NAME,DISP=SHR
//EDGSPLCS DD DSN=EDGSPLCS.FILE.NAME,DISP=SHR
```

Figure 130. Example of JCL for EDGHSKP

EXEC parameters for EDGHSKP

Figure 131 on page 338 shows the EXEC parameters for EDGHSKP. DFSMSrmm uses all the parameters except CATSYNCH, DATE, and VERIFY if you do not specify PARM on the EXEC statement.

You can specify the EXEC parameters in any order, but DFSMSrmm processes them in this sequence:

1. CATSYNCH processing
2. VRSEL vital record processing
3. DSTORE storage location management processing
4. EXPROC expiration processing
5. RPTTEXT extract data set creation
6. BACKUP control data set and journal back up processing

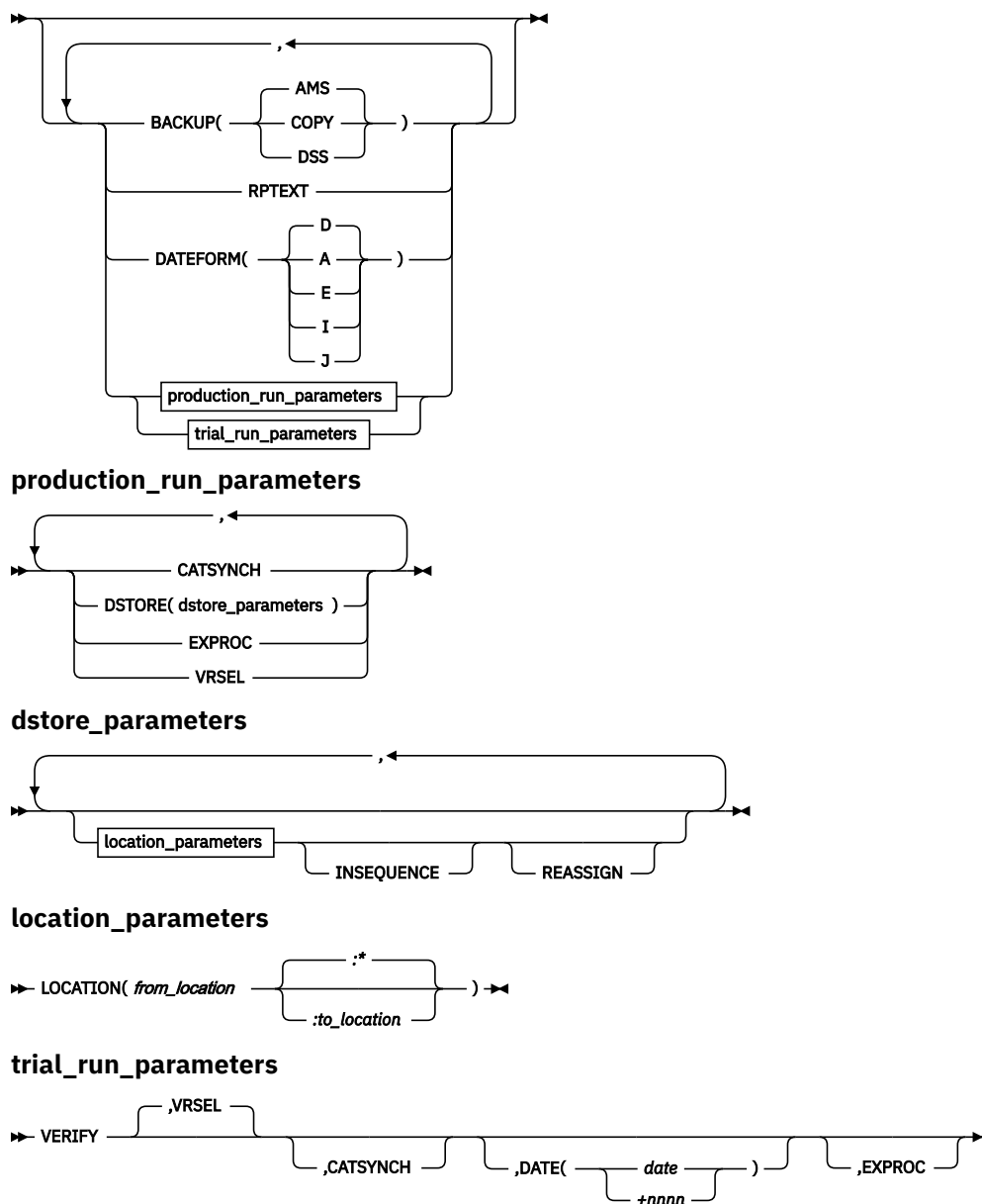


Figure 131. EDGHSKP EXEC parameters

The parameters on the EXEC statement are:

BACKUP(AMS | DSS | COPY)

Specify BACKUP to back up or copy the control data set, to back up the journal or to back up/copy both the control data set and the journal. When you specify BACKUP on the EXEC statement, you must also specify a DD statement. Specify BACKUP DD when you back up or copy the control data set and JRNLBKUP DD when you back up the journal. You can also specify both DD statements in your EDGHSKP JCL.

BACKUP(AMS) is the default. DFSMSrmm uses the access method services REPRO command to perform backup processing. Specify BACKUP(AMS) to prevent DFSMSrmm from updating the control data set during control data set backup processing. Backup cannot be directly to tape.

Specify BACKUP(DSS) to create a portable backup of the CDS. Use backup with the DFSMSdss concurrent copy environment set up to permit the update of the DFSMSrmm control data set during backup processing. This ensures that all tape activities can continue during backup processing. You can specify BACKUP(DSS) without setting up the concurrent copy environment, but CDS updates are prevented. If you specify BACKUP(DSS), backup can be direct to tape.

Specify BACKUP(COPY) to use DFSMSdss to create a copy of the CDS and optionally, a backup of the journal. The COPY uses DFSMSdss logical data set copy and enables you to use fast replication services that enables CDS updates to be made while the copy is created. COPY works just as the DSS option, but uses the COPY command instead of the DUMP command with DFSMSdss. The BACKUP DD is optional for COPY. Specify it if you want to direct the copy to a specific volume, or your environment is not SMS-managed.

For journal backup, DFSMSrmm uses IDCAMS to back up the journal even when BACKUP(DSS) is specified. When using EDGHSKP to perform back up, the journal can be cleared. Refer to [“Steps for automating control data set backup and journal clearing”](#) on page 374 for information.

CATSYNCH

Specify CATSYNCH to update the DFSMSrmm control data set with information from available user catalogs. Synchronizing the DFSMSrmm control data set with the catalogs is normally a one-time setup action. See [“Running DFSMSrmm catalog synchronization”](#) on page 365 for implementation details. Before you can synchronize the DFSMSrmm control data set with user catalogs, define system IDs by using the EDGRMMxx parmlib OPTION CATSYSID operand as described in [“Defining system options: OPTION”](#) on page 161.

Specify the EDGRMMxx parmlib OPTION CATSYSID(*) to indicate that catalogs are fully shared and that any data set can be processed by DFSMSrmm on any DFSMSrmm system. DFSMSrmm checks that the control data set has been synchronized prior to performing vital record processing or expiration processing. DFSMSrmm dynamically adds the CATSYNCH execution option when the control data set and catalogs are not synchronized and the EDGHSKP VRSEL or EXPROC parameters are specified.

If CATSYSID is specified with specific system IDs, you cannot run vital record processing until the control data set is synchronized with all user catalogs and you have run EDGUTIL with the SYSIN statement CONTROL CATSYNCH(YES). See [“Creating or updating the control data set control record”](#) on page 409 for information about marking the control data set for synchronization.

Prior to synchronizing the DFSMSrmm control data set with available user catalogs, you can specify the VERIFY parameter with the CATSYNCH parameter to find differences between the DFSMSrmm control data set and the user catalogs. When you run CATSYNCH with VERIFY prior to implementation, you do not need to add the CATSYSID operand to parmlib. DFSMSrmm reports all the differences so you can make updates, if required, to catalog information before running CATSYNCH without the VERIFY parameter.

When you run the CATSYNCH parameter with the VERIFY parameter on a DFSMSrmm control data set that is already synchronized, and there are differences found between the catalog and the DFSMSrmm information, the differences are reported, but the DFSMSrmm control data set synchronization is unchanged. You should review the differences and make any changes to the catalogs as required and then use EDGUTIL UPDATE to mark the DFSMSrmm control data set as not synchronized. The next time you run the CATSYNCH parameter without the VERIFY parameter or when CATSYNCH is added automatically by EDGHSKP, the DFSMSrmm control data set and catalogs are synchronized again.

DFSMSrmm adds information about the data sets that are synchronized in the ACTIVITY report. It is expected that CATSYNCH is setup for unshared catalogs in a client server environment.

DATE(date|+nnnn)

Specify DATE to set the date used for VERIFY processing.

To specify a date with *date*, supply the year and day in one of two forms:

- yyddd, where yy is the last two-digit number for the year and ddd is the three-digit number for the day of the year, such as 93001.
- yyyy/ddd, where yyyy is the four-digit number for the year and ddd is the three-digit number for the day of the year, such as 1993/001. The slash is required.

For dates in the year 2000 and or in the 21st century or higher, you can only use the yyyy/ddd format. If you use the yyddd format, DFSMSrmm defaults to the 20th century.

To specify a number of days to add to the current date, supply *+nnnn* to determine the actual date to be used for VERIFY processing. You specify the value as a plus sign and the number of days, for example, to use the date in 7 days time specify DATE(+7).

When specifying DATE, DFSMSrmm performs housekeeping as if it is run at the current time of day but on the specified date. The current time will be taken into account by expiration processing when evaluating the expiration time of a volume.

The current date is the default.

DATEFORM (A|E|I|J|D)

Specify DATEFORM to set the date format for records that are written to the extract data set, records written to the ACTIVITY file, and any messages issued during inventory management.

Value	Language	Format	Example
A	American	mm/dd/yyyy	12/15/2013
E	European	dd/mm/yyyy	15/12/2013
I	ISO	yyyy/mm/dd	2013/12/15
J	Julian	yyyy/ddd	2013/349
D	Default	Installation's default in EDGRMMxx	Initially set to Julian

The default date format for all date fields is the value specified in the parmlib member EDGRMMxx. The value is initially set to J for Julian. To change the date format for each run of EDGHSKP, use the DATEFORM parameter.

DSTORE

Specify DSTORE for storage location management processing.

LOCATION(*from_location*:*to_location*, ...)

Specify this parameter if you want to perform storage location management processing by location.

Specify a pair of location names separated by a colon. You can specify 1 to 8 pairs of *from_location*:*to_location* names.

If you omit this parameter, DFSMSrmm performs storage location management processing for all volumes that are required to move.

The *from_location* is the name of the location from which the volume should move. The *to_location* is the name of the destination for the volume.

These location names can be specified in one of these ways:

- Specify a specific location using 1 to 8 character names.
- Specify all locations using a single asterisk (*).
- Specify all locations that begin or end with specific characters, such as ATL* or *DR.
- Use the percent sign % in the location name to replace a single character. You can specify up to eight % in a location name mask.

DFSMSrmm does not validate the location names that you specify against the DFSMSrmm LOCDEF entries or the names of SMS libraries.

INSEQUENCE

Specify INSEQUENCE for volumes that are required to move from a non-bin-managed location to a bin-managed location.

Storage location management processing assigns volumes to available bins in volume sequence and bin sequence, starting with the lowest volume serial number and the lowest bin number.

All bins that become available in a single run of storage location management processing can be reused for other volumes. Bins can become available during storage location management processing under these conditions:

- Global confirm move processing.
- Volumes starting a move out of the bin with parmlib option REUSEBIN(STARTMOVE) specified.

If REASSIGN is also specified, the volumes that restart their move are merged in sequence with those volumes that have just started their move. Freed bins are merged with empty bins.

Bins are best utilized, if INSEQUENCE and REASSIGN are specified and parmlib option REUSEBIN(STARTMOVE) is also specified. Fewer empty bins need to be defined.

When you do not specify INSEQUENCE, the DFSMSrmm DSTORE processing assigns volumes to bins in the sequence that volumes are processed. Each time a volume is assigned to a bin, the lowest empty bin number is used. The exact bin depends on the setting of the REUSEBIN option.

REASSIGN

Specify REASSIGN for volumes that are already moving from a storage location that is not bin-managed storage location and the required location is either a bin-managed storage location or is different from the destination. When you specify REASSIGN, you are canceling the move for these volumes and requesting that the move for the volumes is restarted so that DFSMSrmm could assign these volumes to other locations or bins.

If the LOCATION parameter is specified, DFSMSrmm reassigns a volume when at least one of the LOCATION subparameter pairs matches the volume's current location name and destination name.

EXPROC

Specify EXPROC for expiration processing. When you specify the EXPROC execution parameter, you can also optionally specify the EXPROC command in SYSIN to tailor expiration processing. See [“SYSIN file for the EDGHSKP utility” on page 342](#) for details. To perform a trial run of expiration processing, you can also specify the VERIFY parameter and optionally the DATE parameter. Performing a trial run allows you to see how expiration processing changes affect volume information before the control data set is updated.

RPTEXT

Specify RPTEXT to create an extract data set. When you specify RPTEXT as the only execution parameter you enable DFSMSrmm to create your extract at the same time that other RPTEXT or inventory management requests are being processed. When there are multiple requests, you must provide a MESSAGE file and a REPTTEXT file or an XREPTTEXT file for each extract request.

VERIFY

Specify VERIFY to request that DFSMSrmm performs a trial run of selected processing. You can run a trial run for vital record processing and catalog synchronization, or expiration processing. When you specify VERIFY, DFSMSrmm performs the requested processing, but does not update the DFSMSrmm control data set as a result of the processing.

You could specify VERIFY to perform a trial run to test new vital record specifications that you define to DFSMSrmm. Run VERIFY to confirm that your new and changed retention and movement policies achieve the expected results. If all you want to perform is a trial run of vital record processing, you can specify VERIFY with or without the VRSEL parameter. You can use the DATE parameter with VERIFY to set a date to perform VERIFY processing. When you specify VERIFY, you can use all the inventory management parameters except DSTORE. The ACTIVITY file is required when you select the VERIFY option for VRSEL or CATSYNCH.

The DFSMSrmm parmlib OPTION VRSCCHANGE operand default of VERIFY prevents inventory management vital record processing when there are policy changes that have not been tested. You only need one successful run of VERIFY before you can continue with inventory management processing. If VERIFY completes successfully, but you do not obtain the vital record specification results you expected, you must continue to modify policies and run VERIFY until you obtain acceptable results. If VERIFY is specified together with EXPROC, there will be a message added

to the MESSAGE file about each volume that would be set to pending release or scratch status during actual expiration processing. Analyze whether these results are acceptable.

VRSEL

Specify this parameter for vital record processing. To perform a trial run of vital record processing, you can also specify the VERIFY parameter and optionally the DATE parameter. Performing a trial run allows you to see how DFSMSrmm vital record processing changes affect data set and volume information before the control data set is updated.

SYSIN file for the EDGHSKP utility

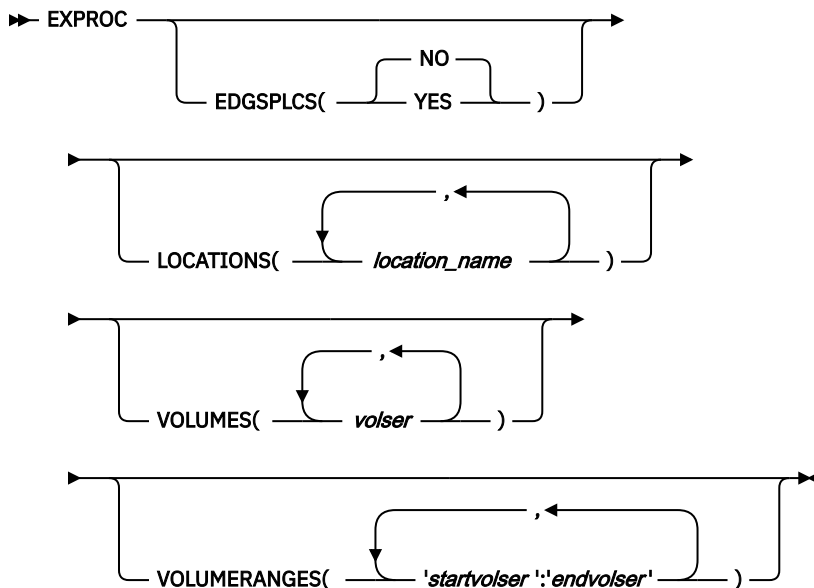
The EDGHSKP utility supports the use of an optional SYSIN file to enable the processing performed by the EXPROC and RPTEXT parameters to be customized. The commands supported in the SYSIN file match the execution parameter. You can specify EXPROC command in the SYSIN file with EXPROC parameter and the RPTEXT command in SYSIN with the RPTEXT parameter.

The SYSIN file is processed by EDGHSKP only when either of the EXPROC or RPTEXT execution parameters are specified. Processing of the SYSIN file allows any supported and correctly specified SYSIN command to be accepted, but ignored if not required. Only a single EXPROC command is supported, and only a single RPTEXT command is supported.

EXPROC command

When you specify the EXPROC execution parameter, an optional SYSIN file EXPROC command allows you to select the subset of available locations and volume entries to be processed during expiration. By default, all volumes in all eligible locations are processed. You do not need to specify a subset of system-managed libraries and volumes simply because some libraries or volumes are not managed on the current system. DFSMSrmm automatically detects the system-managed libraries that are available and skips volumes that cannot be successfully returned to scratch on the current system.

The SYSIN file can be any data set including a JCL in-stream data set.



EXPROC

Use this command to specify the selection criteria for EXPROC processing. The selection criteria you specify selects volumes for processing by EXPROC that includes these aspects of expiration processing:

- Releasing expired volumes.
- Setting and processing individual release actions.
- Returning volumes to scratch status.

If you specified the EXPROC parameter and SYSIN command EXPROC, but neither the VRSEL nor the DSTORE parameters were specified, global confirmed actions and moves are not processed.

If this run of EDGHSKP includes parameters other than EXPROC:

- DFSMSrmm processes all volumes, but only the selected volumes are subject to EXPROC.
- Global confirmed actions and moves are completed if the DSTORE or VRSEL parameters are specified.

By default, all volumes are processed. Only one EXPROC command can be specified for the parameter EXPROC.

These optional operands can be specified:

EDGSPLCS

EDGSPLCS(YES|NO) specifies whether DFSMSrmm should return system-managed volumes to scratch during EXPROC processing or to write volume scratch statements in the EDGSPLCS file to be used with the EDGSPLCS utility.

When you specify EDGSPLCS(YES), EDGHSKP writes a scratch (S) statement to the EDGSPLCS file for each volume for which DFSMSrmm processing would normally have issued a CUA request to OAM to return the volume to scratch. The statements always include the 'Q' verify request option to enable serialization of the return to scratch processing performed by EDGSPLCS. The EDGSPLCS utility can use these scratch statements to perform asynchronous processing, such as concurrent updating of multiple libraries and parallel processing within a library. If the EDGSPLCS DD name is missing, EDGHSKP issues message EDG6101E and ends with return code 12.

When you specify EDGSPLCS(NO), each system-managed volume to be returned to scratch is returned to scratch during EXPROC using a CUA request to OAM.

The default value is NO.

LOCATIONS

LOCATIONS(*location_name*) specifies a subset of the available volumes based on the volume's current location for processing. The location names specified should be a storage location defined with LOCDEF, an available system-managed library on the current system, or shelf. Volumes to be released can be in any location, but volumes can only be returned to scratch while not in transit, and resident in a system managed library, shelf, or their home location. A *location_name* is one-to-eight characters and can be a location name mask. Each *location_name* can be specified in one of these ways:

- Specify a specific location using one-to-eight character names.
- Specify all locations using a single asterisk (*).
- Specify all locations that begin or end with specific characters, such as ATL* or *DR, or multiple locations by using * within a location name.
- Use % (percent sign) in the location name to replace a single character. You can specify up to eight % in a location name mask.

DFSMSrmm does not validate the specified location names against the DFSMSrmm LOCDEF entries or the names of the SMS libraries.

You can specify a list of up to eight values.

VOLUMES

VOLUMES(*volser*) specifies a list of volumes to be processed. You can specify the volumes as fully qualified or as a *volser* prefix ending in *. A fully qualified volume is one-to-six alphanumeric, national or special characters, but the first character must not be blank. Quotation marks are required for special characters. Any value ending in *, even if it is enclosed in quotation marks, is considered to be a *volser* prefix. You can specify a list of up to eight values.

VOLUMERANGES

VOLUMERANGES(*startvolser:endvolser*) specifies a subset of volumes based on the starting and ending *volser* to be processed. The *volser* must be one-to-six alphanumeric, national, or special

characters, but the first character must not be blank. Quotation marks are required for each value regardless of the use of special characters. The end of range must not be lower than the start of the range. You can specify a list of up to eight values.

RPTEXT command

Use the RPTEXT command in the SYSIN file to select the records you want to be written to the report extract file. The extracted records are written to the XREPTTEXT DD, if it exists. If XREPTTEXT DD does not exist, the REPTEXT DD is used.

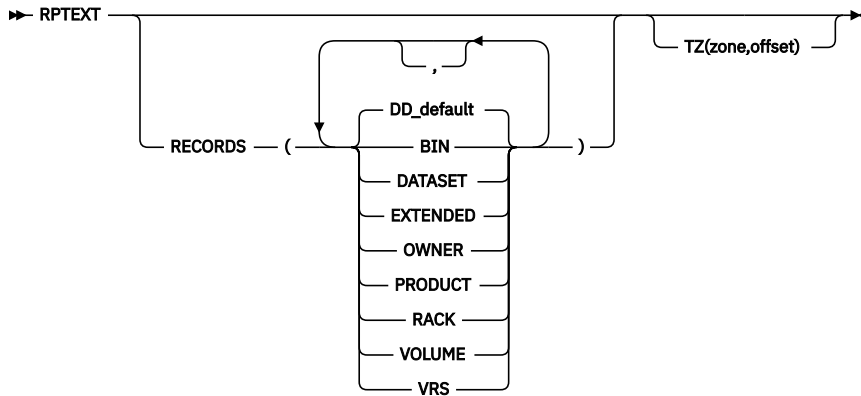


Figure 132. RPTEXT command

RPTEXT

Use the RPTEXT command in the SYSIN file to select the records you want to be written to the report extract file.

These optional operands can be specified:

RECORDS

Use this operand to select the records you want written by report extract processing. You can specify one or more of the following values:

BIN

May be abbreviated as B

DATASET

May be abbreviated as D

EXTENDED

May be abbreviated as X

OWNER

May be abbreviated as O

PRODUCT

May be abbreviated as P

RACK

May be abbreviated as R

VOLUME

May be abbreviated as V

VRS

May be abbreviated as S

. Acceptable aliases are shown in the syntax diagram.

The default is that the records written depend on the extract file DD being used. This is the same default used when the RPTEXT command is not specified in SYSIN.

- If the DD name is XREPTTEXT, DFSMSrmm creates only extended records.

- If the DD name is REPTTEXT, DFSMSrmm creates all records except extended records.

Note: The report extract header record is always written.

TZ

Use the TZ operand to request that the dates and times in the extracted records are converted to your requested time zone offset. TZ is specified as TZ(zone, offset {+|-}hh[:mm[:ss]]), where:

zone

This is a one to four character string that represents the time zone to you.

offset

This is a time zone offset in the format *ohh:mm:ss*, where:

- o can be either + or - to represent the offset direction. Specify + to indicate that the offset is East of the zero median (UT). Specify - to indicate that the offset is West of the zero median (UT). The offset direction is required.
- hh is hours.
- mm is minutes. Minutes are optional. Specify if required.
- ss is seconds. Seconds are optional. If specified, you must also specify minutes.

An optional colon (:) separates hours from optional minutes and optional seconds.

You can specify a time in the range between 00:00:00 to 15:00:00 for *hh:mm:ss*. MM and SS value range is 00 to 59. For example, to select MST, you might enter: TZ(MST,-07).

The zone name and the offset from GMT are included in the report extract header record. By default, DFSMSrmm extract processing converts dates and times to the systems local time. The offset and the zone name, if specified, are set into the report extract header record for use by reporting utilities.

EDGSPLCS file for the EDGHSKP utility

The EDGSPLCS file is a pre-allocated and cataloged file written to during EDGHSKP EXPROC processing when selected with the SYSIN commands.

Figure 133 on page 345 shows JCL for allocating the EDGSPLCS file prior to EDGHSKP processing:

```
//EDGSPLCS DD DISP=(,CATLG),UNIT=SYSALLDA,SPACE=(TRK,(1,1)),LRECL=80,RECFM=FB ,
//          DSN=MY.SPLCS.DATA(+1)
```

Figure 133. Sample JCL for allocating the EDGSPLCS file prior to EDGHSKP processing

Running vital record processing

Vital records processing identifies the volumes that should be retained and how they should be moved based on the vital record specifications you define. Only the volumes with retention method VRSEL, and the data sets on these volumes that do not have the VRSELEXCLUDE attribute on, are considered for vital record processing. Use the EDGRMMxx parmlib OPTION VRSMIN operand to specify the minimum number of vital record specifications that must be defined in order to run inventory management vital record processing. The default number of vital record specifications is 1. You can also set other parmlib OPTION operands to control how DFSMSrmm performs vital record processing as described in “Defining system options: OPTION” on page 161. You can retain data sets and volumes in several ways. For example, you can retain volumes by generic volume serial number, data sets for as long as they are cataloged, or by the job name that created the data set. For more information about moving and retaining volumes with vital record specifications, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

DFSMSrmm can create a report describing data sets and volumes being retained and the location where the data set or volume resides. See “Using the vital records retention report” on page 348 for an example of this report.

You can also request an ACTIVITY file as described in “Using the inventory management ACTIVITY file” on page 352. DFSMSrmm can write details to the ACTIVITY file about changes to data set information

made during vital record processing. The ACTIVITY file is optional except when the VERIFY EXEC parameter is used to request that DFSMSrmm performs a trial run of vital record processing.

To help you establish the settings for these EDGRMMxx parmlib options when you run VRSEL processing and have not disabled the limit checking, DFSMSrmm counts and calculates how many volumes are affected and lists this information in the MESSAGE file. You can see how many volumes and what percentage of these volumes are affected. In particular, look at messages:

EDG2242I

This message shows how many volumes are VRS-retained as VRSEL starts.

EDG2243I

This message shows how many volumes are newly assigned as VRSEL starts.

EDG2244I

This message shows the number of volumes to be dropped from VRS retention and the percentage of the initially VRS-retained volumes.

EDG2245I

This message shows the number of newly assigned volumes to be retained for VRS and the percentage of the initially newly assigned volumes.

EDG2246I

This message shows how many data sets are excluded from VRSEL processing

EDG2427I

This message shows how many volumes are expiration date retained at the start of expiration processing.

EDG2428I

This message shows the number of volumes to be dropped from EXPDT retention and the percentage of the initially expiration date retained volumes.

Analyze several runs to determine your normal expectations and then use these expectations to set the thresholds, the limits, and the actions that you require.

```

1EDG6001I INVENTORY MANAGEMENT STARTING ON 01/11/2010 AT 04:24:44 - PARAMETERS IN USE ARE
DATEFORM(A),VRSEL,CATSYNCH,DSTORE(LOCATION(*:)),EXPROC
EDG6013I THE SYSIN OPTIONS CURRENTLY IN USE ARE
EXPROC
EDGSPLCS(YES)
VOLUMES(A039*)
EDG2309I THE PARMLIB OPTIONS CURRENTLY IN USE ARE
VRSEL(NEW)
VRSJOBNAME(2)
VRSMIN(1,FAIL)
VRSCHANGE(INFO)
VRSDROP(PERCENT(10),INFO) VRSRETAIN(PERCENT(80),INFO) EXPDTPROP(PERCENT(10),INFO)
SMSTAPE(PURGE(ASIS) UPDATE(EXITS,SCRATCH,COMMAND))
CATRETPD(12)
UNCATALOG(S)
TPRACF(N)
NOTIFY(Y)
SYSID(EZU0000)
CATSYSID(*)
RETAINBY(VOLUME)
MOVEBY(VOLUME)
GDG(CYCLEBY(GENERATION),DUPLICATE(BUMP))
EDG2308I CHANGES HAVE BEEN MADE TO VRS POLICIES SINCE THE PREVIOUS INVENTORY MANAGEMENT RUN
EDG2229I NUMBER OF VRS RECORDS READ IS 8
EDG2238I NUMBER OF UNUSED VRS RECORDS IS 2
EDG2246I NUMBER OF DATA SET RECORDS EXCLUDED FROM VRSEL = 1 25%
EDG2242I INITIAL NUMBER OF VRS RETAINED VOLUMES = 0 0%
EDG2244I NUMBER OF VRS RETAINED VOLUMES TO BE DROPPED = 0 0%
EDG2243I INITIAL NUMBER OF NEWLY ASSIGNED VOLUMES = 18 100%
EDG2245I NUMBER OF NEWLY ASSIGNED VOLUMES TO BE RETAINED= 6 33%
EDG2234I DFSMSzmm CDS CATALOG STATUS UNKNOWN FOR RMMUSER.CATCDSN VOLUME A03971 FILE 1 SYNCHRONIZED TO CATALOG STATUS
YES
EDG2234I DFSMSzmm CDS CATALOG STATUS UNKNOWN FOR RMMUSER.CATCDSY VOLUME A03972 FILE 1 SYNCHRONIZED TO CATALOG STATUS
YES
EDG2307I INVENTORY MANAGEMENT TASK CATSYNCH COMPLETED SUCCESSFULLY
EDG2444I EXIT PROCESSING DISABLED FOR THIS EXPROC RUN - NO ACTIVE EXIT MODULE FOR EXIT EDG_EXIT200
EDG2427I INITIAL NUMBER OF EXPDT RETAINED VOLUMES = 14 44%
EDG2428I NUMBER OF EXPDT RETAINED VOLUMES TO BE DROPPED = 14 100%
EDG2420I PHYSICAL VOLUMES READ = 21 54%
EDG2420I LOGICAL VOLUMES READ = 11 28%
EDG2420I STACKED VOLUMES READ = 7 18%
EDG2420I RM_VRSEL VOLUMES READ = 22 56%
EDG2420I RM_EXPDT VOLUMES READ = 17 44%
EDG2420I TOTAL VOLUMES READ = 39 100%
EDG2421I PHYSICAL VOLUMES UPDATED = 21 100%
EDG2421I LOGICAL VOLUMES UPDATED = 11 100%
EDG2421I RM_VRSEL VOLUMES UPDATED = 18 82%
EDG2421I RM_EXPDT VOLUMES UPDATED = 14 82%
EDG2421I TOTAL VOLUMES UPDATED = 32 82%
EDG2422I PHYSICAL VOLUMES, THIS RUN, KEPT FOR VRS = 6 29%
EDG2422I TOTAL VOLUMES, THIS RUN, KEPT FOR VRS = 6 15%
EDG2423I PHYSICAL VOLUMES, THIS RUN, ASSIGNED TO STORES = 1 5%
EDG2423I TOTAL VOLUMES, THIS RUN, ASSIGNED TO STORES = 1 3%
EDG2435I PHYSICAL VOLUMES SELECTED FOR EXPROC = 21 100%
EDG2435I LOGICAL VOLUMES SELECTED FOR EXPROC = 11 100%
EDG2435I RM_VRSEL VOLUMES SELECTED FOR EXPROC = 18 82%
EDG2435I RM_EXPDT VOLUMES SELECTED FOR EXPROC = 14 82%
EDG2435I TOTAL VOLUMES SELECTED FOR EXPROC = 32 82%
EDG2424I PHYSICAL VOLUMES SET PENDING RELEASE = 15 71%
EDG2424I LOGICAL VOLUMES SET PENDING RELEASE = 11 100%
EDG2424I RM_VRSEL VOLUMES SET PENDING RELEASE = 12 67%
EDG2424I RM_EXPDT VOLUMES SET PENDING RELEASE = 14 100%
EDG2424I TOTAL VOLUMES SET PENDING RELEASE = 26 81%
EDG2425I PHYSICAL VOLUMES RETURNED TO SCRATCH = 7 33%
EDG2425I LOGICAL VOLUMES RETURNED TO SCRATCH = 5 45%
EDG2425I RM_EXPDT VOLUMES RETURNED TO SCRATCH = 12 86%
EDG2425I TOTAL VOLUMES RETURNED TO SCRATCH = 12 38%
EDG2426I PHYSICAL VOLUMES - SCRATCH RECORDS WRITTEN = 2 10%
EDG2426I RM_EXPDT VOLUMES - SCRATCH RECORDS WRITTEN = 2 14%
EDG2426I TOTAL VOLUMES - SCRATCH RECORDS WRITTEN = 2 6%
EDG2429I MAIN INVENTORY MANAGEMENT UPDATES HAVE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK VRSEL COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK DSTORE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK EXPROC COMPLETED SUCCESSFULLY
EDG6901I UTILITY EDGHSKP COMPLETED WITH RETURN CODE 0

```

JCL for vital record processing

To request vital record processing, specify the VRSEL parameter. You can submit a job with JCL as shown in [Figure 134](#) on page 347:

```

//HSKP      EXEC PGM=EDGHSKP,
//          PARM='VRSEL'
//MESSAGE   DD DISP=SHR,DSN=HSKP.MESSAGES
//REPORT    DD DISP=SHR,DSN=HSKP.VRS.REPORT
//ACTIVITY  DD DISP=SHR,DSN=HSKP.ACTIVITY

```

Figure 134. Example of JCL for vital record specification processing

To run a trial run of vital record processing, you can submit a job with JCL as shown in [Figure 135](#) on page 348.

```
// HSKP      EXEC PGM=EDGHSKP,
//          PARM='VRSEL,VERIFY'
// MESSAGE DD DISP=SHR,DSN=HSKP.MESSAGES
// REPORT  DD DISP=SHR,DSN=HSKP.VRS.REPORT
// ACTIVITY DD DISP=SHR,DSN=HSKP.ACTIVITY
```

Figure 135. Example of JCL for trial run vital record specification processing

You can request that DFSMSrmm create a REPORT file shown in [“Using the vital records retention report” on page 348](#). DFSMSrmm creates a report that contains data sets and volumes being retained, the vital record specification that is retaining the data set or volume, the required location for the data set or volume, and lists of the unused vital record specification chains.

You can specify the LRECL for the REPORT file. The LRECL can be a value from 133 to 255. If you specify an LRECL that is 148 or higher, only a single line is displayed for a retained data set. If you specify a value less than 148, DFSMSrmm uses two lines to display data set information. The page, date, and time fields in the report are right aligned assuming a record length of 133 characters.

The minimum LRECL must include one byte for the ASA print control character. If you use variable length record format for the REPORT file, then you must add four bytes to the minimum record length to hold the descriptor word. This means that if your REPORT file contains variable length records with a data length of 132 bytes, then you must define the minimum LRECL as 137 bytes.

You can also request an ACTIVITY file as described in [“Using the inventory management ACTIVITY file” on page 352](#). The ACTIVITY file is optional except when the VERIFY EXEC parameter is used to request that DFSMSrmm performs a trial run of vital record processing.

Using the vital records retention report

Use the Vital Records Retention Report to check the vital record specifications that match to data sets and to see which versions of the data sets are being retained. The report file lists the required location for each data set and volume. Use the EDGRPTD movement report to identify the destination selected for each volume after VRSEL processing is completed and DSTORE and RPTEXT have been run.

Here is a sample of the report produced by vital record processing. You can use the information in the Vital Records Retention Report and the last reference date in the vital record specification extract records to manage your vital record specifications and delete any that are no longer required.

REMOVABLE MEDIA MANAGER Copyright IBM CORPORATION 1993,2006				VITAL RECORDS RETENTION REPORT										PAGE 120 TIME 11:21:02 DATE 03/22/2006		
JOB MASK	DATA SET OR VOLUME MASK (CONTINUED)				OWNER	TYPE	RETN	C	X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF
.GDG..ICRMT.**				X002	DSN	CYCLES	Y	N	12/31/1999	0	99999	99999	LOCAL			03/22/2001
JOB NAME	DATA SET NAME (CONTINUED)				2ndVRS	2ndNAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDATE	RETNAME
X015IJC	DT04.GDG.DSNDB06.ICRMT.SYSSTR.G0001V00					12	12	L01699	1	X015	LOCAL	LOCAL	1	1	300	WHILECATLG *
NUMBER OF DATA SETS RETAINED (GROUP STORE) =																
X015IJC	DT04.GDG.DSNDB06.ICRMT.SYSUSER.G0001V00					11	11	L01699	1	X015	LOCAL	LOCAL	1	1	300	WHILECATLG *
NUMBER OF DATA SETS RETAINED (GROUP STORE) =																
X015IJC	DT04.GDG.DSNDB06.ICRMT.SYSVIEWS.G0001V00					13	13	L01699	1	X015	LOCAL	LOCAL	1	1	300	WHILECATLG *
NUMBER OF DATA SETS RETAINED (GROUP STORE) =																
JOB MASK	DATA SET OR VOLUME MASK				OWNER	TYPE	RETN	C	X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF
.GDG..IMGCPY2.**				X002	DSN	CYCLES	Y	N	12/31/1999	0	99999	99999	LOCAL			03/22/2001
JOB NAME	DATA SET NAME				2ndVRS	2ndNAME	FSEQ	DSEQ	VOLUME	VSEQ	OWNER	CURRENT	REQUIRED	PRTY	RETDATE	RETNAME
QDP0900D	DB2Q1.GDG.DSNDB01.IMGCPY2.DBD01.G0889V00					1	1	L00504	1	OPCESA	LIB1	LOCAL		300	WHILECATLG *	
QDP0900D	DB2Q1.GDG.DSNDB01.IMGCPY2.DBD01.G0888V00					1	1	L00358	1	OPCESA	LIB1	LOCAL		300	WHILECATLG *	
QDP0900D	DB2Q1.GDG.DSNDB01.IMGCPY2.DBD01.G0887V00					1	1	L01761	1	OPCESA	LIB1	LOCAL		300	WHILECATLG *	
QDP0900D	DB2Q1.GDG.DSNDB01.IMGCPY2.DBD01.G0886V00					1	1	L01659	1	OPCESA	LIB1	LOCAL		300	WHILECATLG *	
NUMBER OF DATA SETS RETAINED (GROUP STORE) =																
REMOVABLE MEDIA MANAGER Copyright IBM CORPORATION 1993,2006				UNUSED VRS CHAINS REPORT										PAGE 121 TIME 11:21:02 DATE 03/22/2006		
JOB MASK	DATA SET OR VOLUME MASK				OWNER	TYPE	RETN	C	X	DELETE	DLY	COUNT	STNUM	LOCATION	RLSE	LASTREF
*.BACKUP.**				LIB	DSN	CYCLES	Y	N	12/31/1999	0	99999	99999	LOCAL			01/23/1992
*.IMGCPY.IMSDB.**				LIB	NAME	DAYS	N	Y	12/31/1999	0	99999	99999	HOME			12/12/2004
VITAL.**				LIB	DSN	CYCLES	Y	N	12/31/1999	0	99999	99999	CURRENT	IX		05/27/2003

The data columns in the vital records retention report provide this information:

JOB MASK

The job name mask defined for the vital record specification.

DATA SET OR VOLUME MASK

The data set name mask or generic volume serial number defined for the vital record specification. In addition to the data set name mask or the volume serial number, this field can contain descriptive text about a vital record specification chain.

OWNER

Owner ID of the vital record specification owner.

TYPE

Type of vital record specification: one of AND, DSN, GDG, NEXT, PGDG, or VOL.

- AND designates a NAME vital record specification that is pointed to by a vital record specification with the ANDVRS operand.
- DSN for data set vital record specification.
- GDG designates a data set vital record specification with the GDG operand specified.
- NEXT designates a NAME vital record specification that is pointed to by a vital record specification with the NEXTVRS operand.
- PGDG designates a data set vital record specification with a pseudo-generation data set name mask.
- VOL for volume vital record specification.

RETN

Type of retention for data set vital record specifications and name vital record specifications:

BYDAYC

Retention type is BYDAYSCYCLE.

CYCLES

Retention type is CYCLES.

DAYS

Retention type is DAYS since creation.

LRDAYS

Retention type is LASTREFERENCEDAYS.

XDAYS

Retention type is EXTRADAYS.

C

Y if the WHILECATALOG option applies. N if the WHILECATALOG option does not apply.

X

Y if the UNTILEXPIRED option applies. N if the UNTILEXPIRED option does not apply.

DELETE

The date DFSMSrmm deletes the vital record specification.

DLY

The number of days that DFSMSrmm delays the latest copy of a data set or volume before sending it to the named location

COUNT

Total number of retention days or cycles for a data set, total number of retention days for a single volume, total number of volumes for generic volume. The number of days, cycles, or volumes is listed for each vital record specification in the chain that contains retention information. The report lists the total number of data sets retained in the current group at the end of each group of retained data sets.

STNUM

Lists the vital record specification store number value. The store number value is the number of days to retain a data set or volume, the number of data set cycles to retain, or the number of volumes to retain. STNUM is blank for AND vital record specifications. The report lists the total number of data sets in the current group that are requested to be retained outside of their home location.

LOCATION

The location in which the data set or volume should be retained. LOCATION is blank for AND vital record specifications.

RLSE

Lists the vital record specification release options in the data set vital record specification. RLSE can be IX SI, IX, or SI. IX is EXPIRYDATEIGNORE and SI is SCRATCHIMMEDIATE.

LASTREF

Lists the last reference date DFSMSrmm maintains for the vital record specification. The last reference date is the value from the vital record specification records at the start of VRSEL processing. DFSMSrmm only updates the last reference date in a vital record specification after both the REPORT file and VRSEL processing has completed.

JOB NAME

The job that created the data set.

DATA SET NAME OR VOLUME MASK

Data set name. For data set names that exceed 30 characters and that match to a primary vital record specification and a secondary vital record specification, DFSMSrmm splits the output line to display all the data set information.

2ndVRS

Lists the management value vital record specification or management class vital record specification when a data set matches to both a primary and secondary vital record specification. If the management class vital record specification or management value vital record specification is currently retaining the data set, DFSMSrmm puts the name of the vital record specification that is retaining the data set in the 2nd NAME field.

2ndNAME

Lists the name of the first vital record specification in the secondary vital record specification chain that retains the data set. This field is always filled when a data set is retained by a secondary vital record specification. When vital record specifications are chained using AND, DFSMSrmm puts the name of the first vital record specification in this field. This field is blank when a data set is retained by a primary vital record specification.

FSEQ

Physical file sequence number.

DSEQ

Data set sequence number on the named volume.

VOLUME

Volume serial number.

VSEQ

Volume sequence number.

OWNER

Owner ID of the volume and data set owner.

CURRENT

Current location of the volume.

REQUIRED

Destination of the volume based on the matching vital record specification. When there are multiple data sets on a volume, DFSMSrmm displays the destination for the volume using the priority of each of the required locations identified for the volume. The volume's destination is calculated based on the location priorities described in [Table 49 on page 356](#). If there are multiple data sets on a volume or the volume is retained using multiple vital record specifications, the required location listed in the report might be different from the destination to which DFSMSrmm attempts to move the volume.

When the RETDATE is CATRETPD, this field contains the volume's current location.

PRTY

Defined or defaulted relative priority of the location.

RETDATE

Lists information for individual data sets and volumes. This field contains the retention date calculated by vital record processing. The meaning of the retention date depends on the retention type of the vital record specification. See *z/OS DFSMSrmm Managing and Using Removable Media* for information about how DFSMSrmm calculates the retention date.

BYDAYSCYCLE

For the BYDAYSCYCLE retention type, RETDATE is displayed as a special date format CYCL/cccc. ccccc is the COUNT value for cycles used in the vital record specification.

CYCLES

For the CYCLES retention type, RETDATE is displayed as a special cycles date format, CYCL/cccc, where ccccc is the COUNT value for cycles used in the vital record specification.

DAYS

For the DAYS retention type, RETDATE is displayed as the date calculated by vital record selection processing. This is the COUNT value added to the data set creation date or the volume assigned date.

EXTRADAYS

For the EXTRADAYS retention type, RETDATE is a date calculated by inventory management vital record processing. The RETDATE is the COUNT value in the vital record specification chain added to the date when the subchain started to retain the data set.

LASTREFERENCEDAYS

For the LASTREFERENCEDAYS retention type, RETDATE is displayed as the date calculated by the vital record selection processing. This is the COUNT value added to the date the data set or volume was last referenced.

UNTILEXPIRED

For the UNTILEXPIRED retention type, DFSMSrmm calculates the RETDATE based on the presence of a secondary vital record specification. When a secondary vital record specification retains a data set, the UNTILEXPIRED retention is based on the retention type in the secondary vital record specification. The retention type can be any of the date formats specified for CYCLES, DAYS, LASTREFERENCEDAYS, WHILECATALOG, BYDAYSCYCLE, and EXTRADAYS. If the secondary vital record specification contains WHILECATALOG, the retention date is WHILECATLG as long as the data set is cataloged.

WHILECATALOG

Retention date for WHILECATALOG can be the special catalog date format, WHILECATLG or CATRETPD. When the RETDATE is WHILECATLG, the data set is retained by a vital record specification with the WHILECATALOG retention type and the data set is cataloged. CATRETPD is used when the data set is not cataloged and is created within the period defined by the parmlib OPTION CATRETPD operand. DFSMSrmm retains the data set for the catalog retention period if the data set has never been cataloged. DFSMSrmm does not retain the data set if DFSMSrmm detected that the data set was cataloged and then uncataloged during the catalog retention period. DFSMSrmm sets the REQUIRED field to the volume's current location and does not include the data set in the cycle count.

RETNAME

RETNAME displays the name of the current vital record specification in the primary vital record specification chain that retains the data set as follows:

- The first vital record specification in the subchain for ANDVRS groups
- An * if the retaining vital record specification is a data set name vital record specification in a primary vital record specification
- A blank for when a secondary vital record specification retains a data set

NUMBER OF DATA SETS RETAINED (GROUP STORE)

Lists two numbers. The first is the number of data sets or volumes retained by the vital record specification. The second is the number of data sets or volumes retained in a storage location.

Using the inventory management ACTIVITY file

DFSMSrmm provides the VERIFY function to perform a trial run of vital record processing and synchronize catalog processing so you can see the results of processing on a production run.

The ACTIVITY file is optional except during VERIFY processing. During VERIFY processing, the ACTIVITY file is required so that you can analyze processing results before they are actually performed. The ACTIVITY file is a pre-allocated DASD data set, like the REPORT file. The ACTIVITY file is a variable blocked file with the record length set to the largest record created by DFSMSrmm. The block size is determined by the system. The ACTIVITY file is not intended to be a report, but to contain detailed information about changes made to data sets and volumes during DFSMSrmm inventory management processing. All data set record changes, but only a subset of volume changes, are reported in the ACTIVITY file:

- Newly assigned volumes that are retained only by a volume VRS or that are retained only because they are in a volume set and another volume in the set is VRS retained. This information is needed to aid analysis of the VRSRETAIN retention limit processing and the records are produced only when the VRSRETAIN action is not OFF.
- EXPDT retained volumes set to pending release because of the expiration date. This information is needed to aid analysis of the EXPDTRDROP retention limit processing and the records are produced only when the EXPDTRDROP action is not OFF.

You can use the ACTRC_DSN_CHANGE field of the data set ACTIVITY record to identify the reason for the change. You can use the ACTRC_VOL_CHANGE field of the volume ACTIVITY record to identify the reason for the change. During processing, if an ACTIVITY file is allocated, DFSMSrmm writes information about changes in the data set and volume information, such as expiration, matching vital record specification, vital record status, retention date and catalog status to the ACTIVITY file. If VERIFY processing is being run, or the retention limit checking action is FAIL and the limit is triggered, the changes are not actually made.

The DFSMSrmm supplied sample EDGJHKPA shows the JCL to allocate the ACTIVITY file.

You can view the ACTIVITY file on-line. To print the ACTIVITY file, use a product like DFSORT or DFSORT ICETOOL to selectively format and print fields. DFSMSrmm provides a sample job EDGJACTP in SAMPLIB to print the ACTIVITY report. See [z/OS DFSMSrmm Reporting](#) for information about the reports you can produce with the sample job.

How vital record processing works

This topic describes how vital record processing works.

1. Only volumes with VRSEL retention method are included in vital record processing.
2. Data sets that have the VRSELEXCLUDE attribute set to on are excluded from vital record processing.
3. You control how vital record processing works by using the EDGRMMxx parmlib OPTION command options CATRETPD, EXPDTRDROP, MOVEBY, RETAINBY, RETENTIONMETHOD, VRSDROP, VRSJOBNAME, VRSCHANGE, VRSMIN, and VRSRETAIN, as described in [“Defining system options: OPTION” on page 161](#).
4. You must run vital record processing when you add or change a vital record specification for DFSMSrmm to apply the policy defined by the vital record specification. You should reclaim any volumes that are pending release or ready to return to scratch to avoid data loss. Use the RMM CHANGEVOLUME subcommand to change the status of these volumes to reclaim them.
5. As part of vital record processing, DFSMSrmm checks that data set name masks and job name masks used in vital record specifications are specified according to DFSMSrmm naming conventions. [z/OS DFSMSrmm Managing and Using Removable Media](#) provides information on defining vital record specifications. DFSMSrmm compares the data set, job name, and volume information recorded in the control data set with information in the vital record specifications to determine which data sets and volumes to retain and the processing required. This includes any volumes with special JCL specified expiration dates used by your installation. See [“Managing volumes with special dates” on page 91](#).

6. DFSMSRmm deletes any vital record specifications that have reached their automatic deletion date from the control data set. When a vital record specification is deleted, no data set or volume information is changed. DFSMSRmm uses only the remaining vital record specifications to apply policies. If the data set or volume matches to another remaining vital record specification, DFSMSRmm applies those policies.
7. When stacked volume support is enabled, DFSMSRmm retains exported stacked volumes based on how the contained volumes are retained. The stacked volume is retained as a vital record if any contained volume is retained on a volume basis or data set basis. DFSMSRmm sets the stacked volume's retention date to the highest retention date of all contained volumes. DFSMSRmm sets the required location of the stacked volume using the required location priority of each contained logical volume. Retention and movement of copy exported stacked volumes is based only on the volume VRs that matched to the stacked volume volser.
8. During vital record processing, DFSMSRmm records information about the matching vital record specification name and retention date that DFSMSRmm sets for the data set or volume. If the data set or volume does not match to any vital record specifications, and is no longer retained by a vital record specification, the data sets are eligible for expiration processing.
9. DFSMSRmm determines the release options for each data set and sets them at the volume level. Release options can be determined regardless of whether a data set is retained as a vital record. When there are multiple data sets on a volume, the results for release option processing are such that:
 - If any data set on a volume is or has been retained by a vital record specification, the release options for the volume are set only from data sets that are retained by a vital record specification.
 - If no data sets on a volume are vital record specification retained and none of them have yet been retained by a vital record specification, and the volume is not yet retained by a volume vital record specification, the release options are taken from any data sets that match to a vital record specification. Both primary and secondary vital record specification matches are considered.
10. DFSMSRmm saves movement and retention information in each volume record. DFSMSRmm also saves retention information in each data set record.
 - When a data set is retained by a vital record specification, DFSMSRmm sets the data set retention date to the date calculated during vital record processing.
 - When a data set is no longer retained by a vital record specification, DFSMSRmm sets the data set retention date to the date vital record processing started.
 - When a volume is pending release or scratch, or is no longer retained by a vital record specification, DFSMSRmm does not change the retention date that was already set.
 - DFSMSRmm uses the highest data set retention date on the volume to set the volume retention date.
11. See [“Confirming global volume movement”](#) on page 369 for information about the global move confirmation processing that takes place during vital record processing.
12. Any volume no longer retained by a vital record specification is updated to indicate that it is not retained by a vital record specification and the retention date is set to the date that inventory management is run.

How DFSMSRmm processes vital record specification chains

DFSMSRmm validates vital record specification chains by checking for the existence of the next link in a vital record specification chain. If the next link in the vital record specification chain is missing, DFSMSRmm issues message EDG2230I.

Any data sets that match to an incomplete chain are retained by a special broken vital record specification. The special broken vital record specification uses the name **broken** and is listed in the REPORT and ACTIVITY files and in the data set matching vital record specification information. The broken vital record specification uses a permanent retention date.

You can define policies using individual vital record specifications and vital record specification groups or subchains that DFSMSrmm will use during inventory management. When a data set matches to a vital record specification, DFSMSrmm uses the retention information to retain the data set. Once the retention criteria has been met, the next vital record specification in the chain is used, and so on, until the end of the chain is reached. Each time you run vital record processing, DFSMSrmm processes from the start of the chain in case any of the earlier retention criteria apply again.

How DFSMSrmm processes primary and secondary vital record specifications

You can define both a primary and secondary vital record specification to retain a data set. DFSMSrmm matches the data set to the vital record specification based on data set name, jobname, management class, vital record specification management value, the ABEND, DELETED, and OPEN reserved names, or the '*' data set name mask. DFSMSrmm matches the data set to the secondary vital record specification based on management class or vital record specification management value if the primary match is on data set name, but not '*'. If the data set is the subject of special ABEND, DELETED, or OPEN retention, then the secondary vital record specification will also be the subject of the special ABEND, DELETED or OPEN retention, respectively.

DFSMSrmm uses the following criteria to retain the data set. A data set is kept in retention as long as it is retained by either the primary or the secondary VRS, the primary VRS having a higher priority when determining location and release options:

- Retained by the primary data set name vital record specification

Location is taken from the primary data set name vital record specification. Retention date is calculated based on the values set in the current subchain of the primary vital record specification. Release actions are determined by values that are set in the primary vital record specification.

- Retained by the primary data set name vital record specification including the UNTILEXPIRED retention type in the current subchain

Location is taken from the primary data set name vital record specification. Retention date is calculated based on the values set in both the primary vital record specification and the secondary vital record specification. Release actions are determined by values that are set in the primary vital record specification. The retention date is the earliest date of the two current vital record specifications. This is the retention date calculated for the primary vital record specification. After it has passed, the data set can still be retained by the secondary vital record specification alone, as explained in the next paragraph. As such, UNTILEXPIRED causes the retention of the primary vital record specification to be limited by the retention of the secondary vital record specification, or the volume expiration date if there is no secondary vital record specification. However, UNTILEXPIRED does not decrease the retention of the secondary vital record specification.

- Retained by the secondary vital record specification and not the primary vital record specification

Location is taken from the secondary vital record specification. Retention date is the calculated based on the value set in the secondary vital record specification. Release actions are determined by values that are set in the secondary vital record specification.

- Not retained by any vital record specification

Combining retention types

You can define vital record specifications so that DFSMSrmm retains the data set or volume by combining the retention information from two vital record specifications. DFSMSrmm processes both primary and secondary vital record specifications to retain a data set. DFSMSrmm can combine retention information from the two vital record specifications and apply the combined retention to a data set. Use the UNTILEXPIRED retention type in the primary vital record specifications to combine its retention with the retention in the secondary vital record specification. When UNTILEXPIRED is specified in a primary vital record specification, DFSMSrmm looks at the secondary vital record specification to determine if UNTILEXPIRED is true or not. If DFSMSrmm finds no secondary vital record specification that matches a data set or if UNTILEXPIRED is specified in the secondary vital record specification, then DFSMSrmm uses the volume expiration date to determine if UNTILEXPIRED is true or not. UNTILEXPIRED can be used to

have the location taken from the primary VRS while ensuring that the Primary VRS does not retain the data set longer than it would be retained by the Secondary VRS.

How DFSMSrmm selects retention and movement policies

DFSMSrmm retains data sets and volumes using policies you define in vital record specifications by matching data sets and volumes to vital record specifications. DFSMSrmm matches data sets with data set names and job names specified in the vital record specification. Data sets that have the VRSELEXCLUDE attribute set to "on" are excluded from vital record processing.

You can define policies for specific data sets and volumes. You can also define system-wide policies for all data sets not covered by a specific policy.

The data set name masks ** and primary vital record specification *** match to all data sets not covered by a more specific vital record specification. DFSMSrmm matches a vital record specification with the data set name mask *** to a data set before matching to a vital record specification with a vital record specification management value. DFSMSrmm uses the vital record specification with the data set name mask ** to cover any data sets that do not match to any other vital record specifications. You can use these data set name masks to define a system-wide retention policy.

DFSMSrmm matches a data set to a secondary vital record specification when the data set first matches to a vital record specification with a data set name mask, except **.

You can define special OPEN, ABEND, and DELETED policies to manage data sets that are left open, closed as a result of an abnormal end in a task, or closed with normal disposition DELETE.

You can select how DFSMSrmm uses data set name and job name in matching with the parmlib OPTION VRSJOBNAME operand described in “Defining system options: OPTION” on page 161. [z/OS DFSMSrmm Managing and Using Removable Media](#) provides information on defining vital record specifications and how to retain and move data sets and volumes.

Considerations for retaining data sets and volumes

Any volumes that are currently open or left open are listed in message EDG2404W. If you do not want to retain those volumes that are left open from processing errors or abends, you can use the RMM DELETEVOLUME *volser* RELEASE command to release those volumes.

You can also define vital record specifications with the reserved data set name mask or jobname mask OPEN to manage volumes that are left open. You can use the reserved data set name mask or jobname mask ABEND in a vital record specification to specify policies for data sets closed as a result of an abnormal end in a task. You can use the reserved data set name mask or jobname mask DELETED in a vital record specification to specify policies for data sets closed with normal disposition DELETE. This method applies only when the VRSEL retention method is used and the data set does not have the VRSELEXCLUDE attribute set on. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information on defining vital record specifications.

Data sets and volumes that have reached or passed their expiration date and that are not to be retained are subject to normal expiration and release processing. You can control when DFSMSrmm releases volumes by defining vital record specifications that ignore the expiration date, so that volumes are released before their expiration date. See “How expiration processing works” on page 360 for more information.

Moving volumes

During inventory management, DFSMSrmm identifies the destination for a volume. DFSMSrmm uses the location priority value to resolve movement conflicts when a volume is destined for more than one location. Priority values as shown in [Table 49 on page 356](#) are purely relative and do not have any further significance. If more than one destination is identified, and the locations are not the same, the DFSMSrmm default priorities are based on these conditions:

- If none of the locations are storage locations, DFSMSrmm selects the most automated location that is based on the location priority.

- If one or more storage locations is specified for a single volume, DFSMSrmm stores the volume in the location based on the DFSMSrmm default location priority, the priority set in the LOCDEF command for the location, or the priority specified in a vital record specification. The lower priority numbers take precedence. For example, a volume would move to a location with a priority value 100 before moving to a location with a priority value of 200. You can set a priority value with the PRIORITY operand on the LOCDEF parmlib command to define the relative importance of locations.

You can define PRIORITY on RMM ADDVRS subcommands to override default or assigned priorities at the data set level. When you do not set a priority value, DFSMSrmm uses the priority shown in [Table 49](#) on page 356.

Table 49. DFSMSrmm movement priority default values

Priority Number	Location Name or Location Type
100	REMOTE DFSMSrmm built-in storage location name
200	DISTANT DFSMSrmm built-in storage location name
300	LOCAL DFSMSrmm built-in storage location
2000	Installation defined storage locations
4800	AUTO automated tape libraries
4900	MANUAL manual tape libraries
5000	SHELF location name

For stacked volumes, DFSMSrmm determines the required location using the required location of each of the volumes in the same container that is retained by a vital record specification. Conflicts in locations are resolved using movement priority.

Volumes requiring movement are identified for storage location management processing. Storage location management processing sets the destination for the volume and allocates shelf locations, based on the destination specified in the vital record specification. See [“How storage location management processing works”](#) on page 357 for more information.

You can identify locations that are not applicable for automated movement by using the LOCDEF operand AUTOMOVE(NO) in the EDGRMMxx parmlib member. When the volume’s current location is defined with AUTOMOVE(NO), DSTORE processing will not change the destination from the required location.

You can request that DFSMSrmm retain a volume in its existing location instead of specifying a storage location or the home location. You define the location in a vital record specification by using the CURRENT reserved location name. When you run vital record processing, DFSMSrmm keeps the volume in the current location rather than scheduling the volume for a move.

Inventory management trial run

You can run a trial run of inventory management vital record processing before DFSMSrmm makes the changes. The trial run lets you see how the vital record specifications you defined would be processed in a production run except that DFSMSrmm does not make any changes to the control data set.

1. Allocate the ACTIVITY file that DFSMSrmm uses to record data set changes that would result during inventory management vital record processing.
2. Submit an inventory management job using JCL as shown in [Figure 135 on page 348](#) to request a trial run of vital record processing. Perform step 3, 4, and 5 until you are satisfied that the correct policies are in place.
3. Analyze the ACTIVITY file to see what changes would have occurred based on the vital record specifications you defined.
4. Change vital record specifications by deleting and adding vital record specifications as needed to correct the policies you want in place.

5. Check the EDGRMMxx member OPTION VRSCCHANGE operand value. Make sure VRSCCHANGE(VERIFY) is specified so that DFSMSrmm issues message EDG2308I if any changes are made since the last run of inventory management.
6. Submit an inventory management job requesting a production run of vital record processing after you have completed making changes to the vital record specifications.

Running storage location management processing

Request storage location management processing when you are using vital record specifications to identify data sets and volumes to be moved between locations or using RMM TSO CHANGEVOLUME subcommands to move volumes between system-managed libraries and between storage locations. Manual moves using the RMM CHANGEVOLUME subcommand could be used to support dynamic shelf-management. Vital record processing sets the required location where a volume should be stored. Storage location management sets the destination for the volume and optionally assigns the exact shelf location to be used for the volume while it is in the storage location.

JCL for storage location management processing

To request storage location management processing, specify the DSTORE parameter. You can submit a job with JCL similar to [Figure 136 on page 357](#).

```
//HSPK      EXEC PGM=EDGHSKP,  
//          PARM='DSTORE'  
//MESSAGE  DD DISP=SHR,DSN=HSPK.MESSAGES
```

Figure 136. Example of JCL for storage location management processing

To move all volumes from one location to another location using the INSEQUENCE and REASSIGN parameters, you can submit a job with JCL that is similar to [Figure 137 on page 357](#).

```
//HSPK EXEC PGM=EDGHSKP,  
// PARM='DSTORE (LOCATION (SHELF:VAULT) , INSEQUENCE , REASSIGN)  
//MESSAGE DD DISP=SHR,DSN=HSPK.MESSAGES
```

Figure 137. Example of JCL for storage location management processing using the LOCATION, INSEQUENCE, and REASSIGN parameters

How storage location management processing works

Storage location management processing assigns volume destinations and sets required volume destinations that are based on the results of other inventory management functions. Storage location management must be run after vital record processing because the volume destinations are determined by vital record processing. See [“DFSMSrmm support for stacked volumes when stacked volume support is enabled” on page 112](#) and [“DFSMSrmm support for stacked volumes when stacked volume support is not enabled” on page 118](#) for information about the processing sequence you must use when a VTS is in use. See [“Confirming global volume movement” on page 369](#) for information about the global move confirmation processing that takes place during vital record processing.

Storage location management processing assigns a shelf location, known as the bin number, to volumes moving to and among shelf-managed storage locations and sets a destination storage location. If the volume destination is a system-managed tape library, DFSMSrmm does not assign a bin number.

When you have enabled stacked volume support, storage location management is performed for stacked volumes, not for contained volumes. For contained volumes, the location of the stacked volume identifies the contained volume location. DFSMSrmm sets the required location for the stacked volume based on the required location defined for all volumes contained in the stacked volume. At the completion of export processing, the stacked volume has a required location. The required location is set during export based on the required location and priority of each exported volume. At the completion of export processing, the required location is used to set a destination if the location is not a shelf-managed storage location. If the destination location is shelf-managed, DFSMSrmm sets the destination during storage location

management and assigns a bin number. See [“Managing stacked volumes” on page 110](#) for details of the actions performed for copy export.

When you do not enable stacked volume support, DFSMSrmm does not consider the stacked volume when performing storage location management.

DFSMSrmm marks volumes that are not residing in a system-managed library as 'intransit'. DFSMSrmm marks volumes that are residing in system-managed libraries as 'intransit' only after you eject them.

Recommendation: After successful storage location processing, schedule a job or job step to eject all volumes to be moved using the bulk output station. To automatically eject physical volumes, add the job step shown in [Figure 138 on page 358](#) to the job that runs EDGHSKP vital record processing and storage location management processing.

```
// EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
RMM SEARCHVOLUME VOLUME(*) OWNER(*) LIMIT(*) -
  LOCATION(atlname) DESTINATION(storename) -
  INTRANSIT(N) CLIST('RMM CHANGEVOLUME ',' EJECT(BULK)')
EXEC EXEC.RMM.CLIST
/*
```

Figure 138. Automatically ejecting volumes from system-managed libraries

The job step shown in [Figure 138 on page 358](#) runs the batch TMP and uses DFSMSrmm TSO subcommands to produce a list of volumes to be ejected and then uses the list to initiate the ejects. To eject stacked volumes, you can use the same DFSMSrmm TSO subcommands that you used for physical volumes to generate a list to be ejected, but you must use the list to enter the requests at the Library Manager console.

You can override automatic vital record specification location selection by using the RMM CHANGEVOLUME subcommand. See [“Moving volumes manually” on page 138](#) for more information.

To select a suitable bin number for a volume moving to a storage location, DFSMSrmm uses the volume media name and the LOCDEF media names. If the media name is specifically defined on LOCDEF, DFSMSrmm looks for a bin number that is empty and uses the same media name. If the specific media name is not found, DFSMSrmm checks if '*' is defined on the LOCDEF command. If it is, then DFSMSrmm looks for a bin with a media name of '*'. If neither the specific media name nor '*' is found on the LOCDEF command, then the volume cannot be sent to that location, and DFSMSrmm issues message EDG2412E.

To determine if a volume move is required, DFSMSrmm uses vital record processing information. If a move to a shelf-managed storage location is required, DFSMSrmm selects an empty bin in the target location and assigns it to the volume based on media name and LOCDEF information.

If there are no empty bin numbers available in a storage location, DFSMSrmm issues message EDG2403E. Inventory management continues and DFSMSrmm does not change the status of the volumes destined for a storage location which has no empty bin numbers. The volumes are moved in subsequent inventory management runs after more empty bin numbers are added, or when existing bin numbers are freed by other volumes being moved out of the storage location. After you have corrected the problem, request storage location management processing again.

If you move volumes by set, DFSMSrmm sets the same required location for all the volumes in the set that are retained by vital record specifications. DFSMSrmm sets the location based on the priority of each volume's required location. See [“Defining system options: OPTION” on page 161](#) for information about implementing volume movement by set by using the DFSMSrmm EDGRMMxx parmlib OPTION command MOVEBY(SET) operand.

When you run storage location management processing without specifying a location, DFSMSrmm processes all volumes. You can request that DFSMSrmm only process volumes in specific locations when you run the EDGHSKP exec using the LOCATION parameter to specify from and to locations. See [“EXEC parameters for EDGHSKP” on page 337](#) for a detailed description. If you use the INSEQUENCE parameter, DFSMSrmm assigns volumes to bins in sequential volume serial number order and bin number order. If you use the REASSIGN parameter, DFSMSrmm reassigns volumes to bins during processing. To maximize

reuse of bins, use both the INSEQUENCE and REASSIGN parameters and specify the DFSMSrmm parmlib OPTION command REUSEBIN(STARTMOVE) operand.

Running expiration processing

During expiration processing, DFSMSrmm performs these functions:

- Identifies volumes not required for vital records that not held by the HOLD attribute or not having the EDM attribute set on, that are ready to expire by checking the expiration date and seeing if the volume holds any data sets that are kept by catalog status.
- Optionally, if an EXPDTRDOP limit is set, ensures that the user's expectations for the number or percentage of volumes dropped from expiration date retention are met.
- Determines the type of release action, if any, needed for the volumes that have reached their expiration date.
- Manages the release actions for each volume in preparation for return to scratch.
- Handles return to the scratch pool for all volumes. For system managed, this can either be by synchronous processing or by EDGSPLCS.
- Uncatalogs all data sets for volumes returning to scratch status if you have specified UNCATALOG(Y or S) in parmlib member EDGRMMxx.
- Updates the catalog entry to remove the volume for the part of the DFSMSrmm control data set on the volume returning to scratch for a multivolume data set.
- Optionally, updates RACF tape security profiles for volumes returning to scratch status.
- Ensures all volumes in a set are not expired if any one volume does not expire when the EDGRMMxx parmlib option RETAINBY(SET) is in use.
- Determines if a volume has to be replaced by checking against volume replacement policies.

JCL for expiration processing

To request expiration processing, specify the EXPROC parameter. You can submit a job with JCL similar to [Figure 139 on page 359](#).

```
//HSCP      EXEC PGM=EDGHSKP,
//          PARM='EXPROC'
//MESSAGE   DD DISP=SHR,DSN=HSCP.MESSAGES
```

Figure 139. Example of JCL for expiration processing

To run a trial run of expiration processing, you can submit a job with JCL as shown in [Figure 140 on page 359](#).

```
//HSCP      EXEC PGM=EDGHSKP,
//          PARM='EXPROC,VERIFY'
//MESSAGE   DD DISP=SHR,DSN=HSCP.MESSAGES
```

Figure 140. Example of JCL for trial run expiration processing

To request expiration processing on a subset of volumes or to request asynchronous return to scratch of system-managed volumes, specify the EXPROC parameter, and use the EXPROC command in SYSIN. You can submit a job with JCL similar to [Figure 141 on page 359](#).

```
//HSCP      EXEC PGM=EDGHSKP,
//          PARM='EXPROC'
//MESSAGE   DD DISP=SHR,DSN=HSCP.MESSAGES
//EDGSPLCS  DD DSN=RMMTST.Z19MTU.SPLCSHSP,DISP=SHR
//SYSIN     DD *
EXPROC VOLUMES(RFA01*,RFA11*,A035*) EDGSPLCS(YES)
```

Figure 141. Example of JCL for expiration processing

Figure 141 on page 359 requests that scratch (S) statements are generated for the EDGSPLCS utility and that only the volumes in the subsets RFA01*, RFA11*, and A035* are processed.

How expiration processing works

Expiration processing can be either on all volumes or a subset based on the EXPROC command in the SYSIN file. DFSMSrmm checks the expiration date for volumes not retained by a vital record specification not held by the HOLD attribute and not having the EDM attribute set on. If the expiration date has been reached, the volume does not contain any data sets that are kept by their catalog status, and the EXPDTPROP limit and action settings do not require the volume to be retained, DFSMSrmm changes the volume status to "pending release". Data sets that are cataloged and have WHILECATALOG set to ON will prevent a volume from expiring even if its expiration date has been reached. DFSMSrmm checks for any actions that should be taken before the volume can be released. DFSMSrmm defers most release actions for later processing. However, DFSMSrmm processes the "notify owner" release action immediately if an electronic address is provided for the owner and the DFSMSrmm system option NOTIFY is in use. If the volume has the retention method EXPDT and there are no outstanding actions then DFSMSrmm tries to release the volume in the same run.

You can use parmli option EXPDTPROP to control the results of expiration processing. EXPDTPROP specifies a limit on the number of expiring volumes to be processed and on the action to be taken when the limit is reached. If the EXPDTPROP limit has been reached and the action is FAIL, then the volume status is **not** changed. Otherwise, if the INFO or WARN action is specified, DFSMSrmm changes the volume status to 'pending release' and the appropriate message is issued.

When a data set is no longer retained by a vital record specification, DFSMSrmm releases the volume on which the data set resides only if no data set on the volume is retained by a vital record specification the volume is not held by the HOLD attribute and not having the EDM attribute set on. If you use the RMM ADDVRS RELEASE(EXPIRYDATEIGNORE) operand, DFSMSrmm ignores the volume expiration date and uses information in a vital record specification to control retention.

If the volume has the EXPDT retention method, expiration processing always attempts to return volumes to scratch on the same run as the volume is released. This is as if the SCRATCHIMMEDIATE attribute is set for the volume with the VRSEL retention method.

If the volume is retained by the VRSEL retention method, DFSMSrmm does not immediately return a volume to scratch status or to its owner when a volume reaches its expiration date and is not retained by a vital record specification. You must run expiration processing two times to return a volume to scratch status or to its owner. The first run of expiration processing sets the volume status to pending release. The second run of expiration processing completes the return. DFSMSrmm does not change the volume status during the first run of expiration processing. DFSMSrmm marks the volume as pending release during the first run of expiration processing in case you want to reclaim the volume. Running expiration processing two times give you time to make changes to the volume status before the volume is released.

To reclaim from pending release and set the hold attribute for the volume, issue the subcommand RMM CV volser RETPD(1) HOLD. To reclaim from scratch status and set the hold attribute for the volume, issue the subcommand RMM CV volser STATUS(MASTER) RETPD(1) HOLD. Authorization requires either CONTROL access to STGADMIN.EDG.MASTER, or UPDATE access to STGADMIN.EDG.CV.HOLD.volser.

If the volume is retained by the VRSEL retention method and you do not need to run expiration processing in two runs, specify the RMM ADDVRS RELEASE(SCRATCHIMMEDIATE) operand. This enables you to return volumes to scratch in a single run of expiration processing. Sometimes DFSMSrmm cannot make the return in a single run, for example there could be other release actions required. Also, when all catalogs are not fully shared or when TCDBs for partitioned tape libraries are not shared, you might need to run expiration processing more than one time to return volumes to scratch status.

See [“Confirming global volume movement” on page 369](#) for information about the global move confirmation processing that takes place during vital record processing.

If you retain volumes by set, DFSMSrmm retains all the volumes in the set if any volume in the set is retained. DFSMSrmm sets the retention date for all the volumes in the set to the highest retention date for the volumes in the set. See [“Defining system options: OPTION” on page 161](#) for information about

implementing volume retention by set by using the DFSMSrmm EDGRMMxx parmlib OPTION command RETAINBY(SET) operand.

Returning volumes to scratch status

The way in which volumes are returned to scratch depends on whether they are retained by the EXPDT retention method or the VRSEL retention method:

- For volumes that use the EXPDT retention method, DFSMSrmm returns volumes to scratch when their expiration date and time has been reached and they require no other action than returning to scratch.
- For volumes that use the VRSEL retention method, DFSMSrmm returns volumes to scratch that are already in pending release status, that can be returned to scratch on the running system, and require no other action than returning to scratch. These volumes are available to satisfy scratch mount processing or RMM TSO GETVOLUME subcommand requests.

To determine if a volume can return to scratch on the running system, DFSMSrmm considers if the scratch action has been manually confirmed, the TCDB, and catalog sharing. If the scratch action has been manually confirmed, the return to scratch can be processed on any system; otherwise, these conditions must be considered: The volume must be in the TCDB and the system-managed library must be defined. When catalogs are not fully shared, the volume must be returned to scratch on a system with access to the catalogs where the first file was created.

Return to scratch processing for system-managed volumes can be either synchronous or asynchronous with EXPROC processing. You use the SYSIN EXPROC command EDGSPLCS operand to select the type of processing you want. Asynchronous processing is handled by the EDGSPLCS utility.

When DFSMSrmm returns a volume to scratch status, DFSMSrmm clears this information from the control data set:

- Volume description.
- Jobname.
- Accounting information.
- Access list.
- Volume owner.
- Owner access.
- Software product details. DFSMSrmm removes the volume from the product's list of volumes.

DFSMSrmm updates the volume information to indicate that the volume is no longer retained by a vital record specification. When a volume has been released manually using the RMM DELETEVOLUME subcommand, but is still retained by a vital record specification, DFSMSrmm sets the retention date to the current date and updates the volume information to indicate that the volume is no longer retained by a vital record specification.

When a volume returns to scratch status, DFSMSrmm also updates the related rack number record in the control data set. For volumes where the rack number matches the volume serial number, you can avoid this processing by removing the rack numbers. You can use the RMM CV *volser* NORACK subcommand followed by the RMM DR *volser* subcommand to remove the rack numbers from the control data set.

During inventory management expiration processing, DFSMSrmm calls the EDG_EXIT200 installation exit. The exit is called every time a volume is identified as ready to return to scratch. See [“Using the EDG_EXIT200 installation exit” on page 300](#) for information on the EDG_EXIT200 installation exit processing.

In addition, DFSMSrmm requests that all data sets recorded in the control data set for the volume are uncataloged. DFSMSrmm performs RACF processing as described in [Table 29 on page 235](#).

Volume replacement policies

DFSMSrmm supports policies for volume replacement. You define them in the MEDINF commands in parmlib. This enables you to define different replacement policies based on media type and recording

format. The replacement policies are implemented for all types of volumes, other than logical volumes, and for both private and scratch volumes, but not for those volumes pending release. Until you define one or more MEDINF commands in parmlib with the REPLACE operand, DFSMSrmm processing uses a hard-coded value of PERM(1).

Volume replacement policies are implemented during inventory management expiration processing for stacked and physical volumes. This enables the release action for private volumes that meet or exceed one or more of the threshold values to be changed from SCRATCH to REPLACE (and from RETURN to REPLACE for WORM volumes) so that the volumes are identified for replacement. Later, if and when the data expires and the volume is released, the volume pending actions are set from the release actions. It also enables scratch volumes to be processed by the replacement policies and if any threshold is met, the release action is set to REPLACE, and the scratch volume is changed to master status pending release.

When inventory management sets the release action, the volume last change user ID is set to '*HKP' and if the volume is scratch, the volume owner is set to 'REPLACE'. Any other subcommand processing sets the last change user ID as normal and any other fields based on the subcommand operands and the command issuers ID.

DFSMSrmm open-time volume validation checks if a volume opened for output processing has the REPLACE release action and rejects the volume. This prevents volumes that need to be replaced from being used for new data sets. Existing processing rejects volumes pending release.

You can report on volumes that need to be replaced:

- Use the RMM SEARCHVOLUME subcommand with the ACTION(REPLACE) operand. This lists the volumes that are ready to be replaced. You can physically replace them and confirm the replace action with the command:

```
RMM CV volser CRLSE(REPLACE)
```

or, you can delete them from DFSMSrmm with the command:

```
RMM DV volser REPLACE
```

and destroy the volumes.

Alternatively, if you do not want to replace or destroy a volume marked for replacement, you can turn off the replace action with the command:

```
RMM CV volser REPLACE(NO)
```

To have the replacement policy re-applied, you can reclaim the volume from pending release with the command:

```
RMM CV volser RETPD(1) RELEASEACTION(SCRATCH)
```

When the volume is next processed by EXPROC, the REPLACE policy is used to determine if the REPLACE action is required.

- Use the SEARCHVOLUME subcommand with the RELEASEACTION(REPLACE) operand. This lists the volumes that should be replaced, but contain data that has not yet expired.
 - For physical volumes, if the data does not expire soon, you should consider copying the data to a new volume and then release the volume so that it can be replaced or destroyed. You can use a product such as IBM Tivoli Tape Optimizer to copy the data to a newer volume.
 - For stacked volumes, check how the stacked volumes is being used by the virtual tape system. For exported volumes, consider importing the logical volumes back to a library enabling the stacked volume to be removed from service. For stacked volumes in use inside a virtual tape system, consult the appropriate documentation for how to remove a volume from service.
- DFSMSrmm also provides sample reports in the Report Generator.

EREP and the IBM Service Agent provide reporting capability based on LOGREC and SMF records and can generate a report of volumes that should be replaced. There is no current link between this report

and DFSMSrmm. DFSMSrmm tracks volume usage and I/O errors and now provides support for volume replacement policies. You could manually use EREP or Service Agent (with IBM Service support) reports as a base for setting the REPLACE release action.

IBM also has a function called Statistical Analysis & Recording System (SARS) that is used by current IBM tape drives for tracking media statistics to determine or predict media failure. DFSMSrmm now uses the alerts from SARS Media Information Messages (MIM) to drive the setting of the REPLACE release action. This ensures that the best decisions about replacement are taken because the IBM tape hardware uses many different factors in considering whether a tape volume is still in good shape. MIM alerts occur at the time a volume is used and are intercepted and handled on z/OS by device services and an eventual action message issued to the console. The message number is IEA486E. WTO message ID IEA486E is now intercepted by the DFSMSrmm subsystem and the MIM message code used to determine the action that needs to be taken. Any message code (mc) that is one of 60-62 or 64 is used to drive volume replacement.

An example of message IEA486E:

```
IEA486E dddd, TVOL, severity ALERT, VOLUME=valid, S=s, MC=F0, E=e, REF=mcee-mmmm-tt, REPEATED.
```

Where:

severity

Is one of the following:

ACUTE

indicates that tape directory errors occurred

MODERATE

indicates that high temporary read or write errors occurred

SERIOUS

indicates that permanent read or write errors occurred

SERVICE

indicates cleaner cartridge

mc

MIM message code. Values DFSMSrmm cares about are 60, 61, 62, and 64; these cause the REPLACE release action to be set in place of the SCRATCH release action (and from RETURN to REPLACE for WORM volumes), if not already set.

SARS driven actions override DFSMSrmm policies by causing the volume release action to be set to REPLACE when the existing release action is SCRATCH. For scratch volumes, the release action is set to REPLACE, the volume is changed to master status, and then released.

When interception of a SARS MIM message sets the release action, the volume last change user ID is set to *mim and, if the volume is scratch at the time the SARS MIM message driven processing occurs, the volume owner is set to 'SARSMIM'. Any other subcommand or subsystem processing sets the last change user ID as normal and any other fields based on the subcommand operands and the command issuers ID, therefore the last change user ID is likely to change. However, you can use the EDGAUD audit report with a SYSIN command of SELECT USERS(*mim) to report on volumes updated as a result of SARS MIM driven processing even if the volume has later been updated.

For active volumes (those that are used regularly), the SARS functionality is the best way to identify volumes that need to be replaced based on problems that SARS detects with the media. For volumes that are not used regularly, perhaps because they are scratch or are subject to long term retention, either on-site or off-site, the DFSMSrmm replacement policies provide a good way to manage the life of the volume because the policies are used regularly by expiration processing. For IBM virtual tape libraries, the SARS alerts for stacked volumes are handled internally to the library and volumes removed from use as required. There is no automatic way for the DFSMSrmm policies to be used other than for AGE.

Tracking volumes with permanent errors

DFSMSrmm keeps track of permanent I/O errors for a volume. During each run of expiration processing, DFSMSrmm sets the release actions for volumes that are not pending release. During the run volumes

that are marked with the return to scratch release action, that have permanent I/O errors will be marked for replacement. DFSMSrmm only identifies those volumes with the 'return to scratch' release action, not those with the 'return to owner' action.

Use the RMM SEARCHVOLUME subcommand to obtain information about volumes that might need replacement. You can use the extract data set or the DFSMSrmm ISPF dialog to check for volumes with temporary errors. See [“JCL for creating an extract data set” on page 336](#) for information about using the extract data set to identify volumes with temporary errors.

Managing open data sets

DFSMSrmm produces a report in the MESSAGE file as shown in [Figure 142 on page 364](#) that lists tape data sets that might be open as well as messages describing expiration processing.

```
EDG2404W VOLUME 111020 FOR JOB STSGGWC1 IS OPEN - VOLUME HAS EXPIRATION DATE 16/03/1993
EDG2404W VOLUME 111023 FOR JOB STSGGWC1 IS OPEN - VOLUME HAS EXPIRATION DATE 16/03/1993
EDG2229I NUMBER OF VRS RECORDS READ IS 4
EDG2238I NUMBER OF UNUSED VRS RECORDS IS 2
EDG2427I INITIAL NUMBER OF EXPDT RETAINED VOLUMES = 728 69%
EDG2428I NUMBER OF EXPDT RETAINED VOLUMES TO BE DROPPED = 0 0%
EDG2420I PHYSICAL VOLUMES READ = 1041 97%
EDG2420I LOGICAL VOLUMES READ = 20 2%
EDG2420I STACKED VOLUMES READ = 10 1%
EDG2420I TOTAL VOLUMES READ = 1071 100%
EDG2421I PHYSICAL VOLUMES UPDATED = 303 29%
EDG2421I LOGICAL VOLUMES UPDATED = 10 50%
EDG2421I STACKED VOLUMES UPDATED = 10 100%
EDG2421I TOTAL VOLUMES UPDATED = 323 30%
EDG2424I TOTAL VOLUMES, THIS RUN, SET PENDING RELEASE = 110 10%
EDG2425I TOTAL VOLUMES RETURNED TO SCRATCH = 203 19%
EDG2426I TOTAL NUMBER OF SCRATCH RECORDS WRITTEN = 0 0%
EDG2429I MAIN INVENTORY MANAGEMENT UPDATES HAVE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK VRSEL COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK DSTORE COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK EXPROC COMPLETED SUCCESSFULLY
EDG2307I INVENTORY MANAGEMENT TASK RPTXT COMPLETED SUCCESSFULLY
EDG6401I MASTER FILE BACKUP SUCCESSFUL
```

Figure 142. Expiration processing report

These might be valid cases, or jobs that are currently active. However, some data sets on the list might be produced by a job that failed so the data set was never closed. Review this list and make the necessary corrections. The expiration date in the message, shows when the volume will be automatically released. During vital record processing, DFSMSrmm matches open data sets to vital record specifications using the OPEN reserved data set name mask or to vital record specifications defined with data set name masks.

Running multiple copies of the EDGHSKP EXPROC utility

The EDGHSKP EXPROC utility can be run in parallel on multiple systems, but only one copy per system can be run regardless of the volumes that are processed. An optional EXPROC command in the SYSIN file can be used to:

- Request that only a subset of volumes are processed by EXPROC.
- Request that system-managed volumes are not returned to scratch during expiration processing. Instead, for each system-managed volume ready to return to scratch, a control statement is generated for use with the EDGSPLCS utility so that volumes can be scratched later. The control statements are written to the EDGSPLCS DD.

Expiration processing trial run

You can run a trial run of inventory management expiration processing before DFSMSrmm makes the changes. The trial run lets you see how your volumes would be processed in an production run except that DFSMSrmm does not make any changes to the control data set. Perform step 1, 2 and 3 until you are satisfied with the trial run results:

1. Submit an inventory management job using JCL as shown in [Figure 140 on page 359](#) to request a trial run of expiration processing. (You can specify the DATE parameter as needed with the date of the assumed expiration processing run. By default, current date is used.)

2. Review the MESSAGE file to see if there are messages EDG2470I or EDG2471I saying what exact volumes would be returned to scratch or set to pending release at the date of the assumed production run.
3. Verify you are satisfied with the trial run results. If not, consider what changes can be done to avoid or postpone setting some or all of the reported volumes to scratch or pending release, and do the proper changes.
4. If you are satisfied with the trial run results, and the current data is the date of the planned production UN of expiration processing, submit the corresponding inventory management job requesting a production run of expiration processing.

Running DFSMSrmm catalog synchronization

DFSMSrmm always tracks tape data set catalog activity independently of the CATSYSID parameter. When you execute catalog synchronization, DFSMSrmm synchronizes the DFSMSrmm control data set and available user catalogs. Running catalog synchronization is normally a one-time task. You enable catalog synchronization by specifying the DFSMSrmm EDGRMMxx OPTION CATSYSID operand described in [“Defining system options: OPTION” on page 161](#). When you specify the CATSYSID operand, DFSMSrmm uses the catalog search interface to obtain catalog information used for DFSMSrmm vital record processing. DFSMSrmm performs checking to determine that the catalog search interface catalog environment does not have any errors that might prevent successful processing. DFSMSrmm issues messages EDG2235E, EDG2236I, or EDG2237E when an error is encountered. If you do not specify the DFSMSrmm EDGRMMxx OPTION CATSYSID operand, DFSMSrmm vital record processing uses catalog locates to determine if data sets are still cataloged.

To disable catalog synchronization and revert to using catalog locates, remove the CATSYSID operand from the EDGRMMxx parmlib member and restart DFSMSrmm, or issue the following command to reinitialize DFSMSrmm. For example, if your DFSMSrmm task is called DFRMM:

F DFRMM,M=xx, where xx is the suffix for EDGRMMxx parmlib member.

Then, use EDGUTIL with the UPDATE parameter and with the SYSIN command CONTROL CDSID(cdsid) CATSYNCH(NO).

For more information, see [“JCL for updating the control data set” on page 402](#).

You must resynchronize the DFSMSrmm control data set and user catalogs under these conditions:

- You ran EDGHSKP with CATSYNCH and VERIFY, and differences in catalog status were found that you believe should be corrected. After you have made any corrections to the catalogs, either run CATSYNCH without VERIFY or use the EDGUTIL utility with the UPDATE option to reset the catalog synchronization date. EDGUTIL clears the catalog synchronization date and time to force you to run CATSYNCH.
- DFSMSrmm is not active when catalog activity for tape data sets is taking place. DFSMSrmm issues messages EDG8200E and EDG8201E when DFSMSrmm cannot track catalog updates.
- A DFSMSrmm failure occurs during catalog processing. DFSMSrmm issues EDG8200E and EDG8201E when an error is detected in processing.
- You connect or disconnect user catalogs, that contain entries for tape data sets, to the master catalog.

When you use IDCAMS REPRO MERGECAT for catalog maintenance, you do not need to run a job to resynchronize the DFSMSrmm control data set and catalogs.

Recommendation: Use EDGUTIL with the UPDATE parameter and with the SYSIN command CONTROL CATSYNCH(NO) parameter when you think the DFSMSrmm control data set and the catalogs are no longer synchronized. This prevents DFSMSrmm from relying on the catalog information in the control data set. Later you can resynchronize the information or DFSMSrmm will synchronize the information automatically at inventory management time when the catalogs are fully shared. You can automate the response to the DFSMSrmm messages EDG8200E and EDG8201E to force resynchronization before any inventory management functions dependent on catalog information are run. Run EDGUTIL UPDATE with the SYSIN command CONTROL CATSYNCH(NO) parameter to set up this automation.

There are some limitations to using DFSMSrmm catalog synchronization. Some of the limitations include these conditions:

- You must define data sets to DFSMSrmm before you create catalog entries for them.
- If either of these statements applies to you, do not rely on DFSMSrmm for maintaining catalog synchronization. However, you can exploit the status tracking DFSMSrmm performs, by always specifying the EDGHSKP CATSYNCH option when running expiration processing or vital record processing functions.
 - If you use the DFSMSrmm TSO subcommands to add data sets to the DFSMSrmm control data set, you should do so before cataloging the data sets. If you have to add cataloged data sets you can run EDGHSKP with CATSYNCH to synchronize the catalogs with DFSMSrmm.
 - If you create a cataloged tape data set by writing to the tape volume, but do so without defining the data set in the catalog using catalog services (because, for example, a catalog entry for another data set with the same name already exists), then the catalog status of the data set in the DFSMSrmm data set record is set to Unknown. In this case, DFSMSrmm cannot track the catalog status because it relies on intercepting the catalog and uncatalog activity to set the catalog status.

Control how DFSMSrmm performs catalog synchronization by using the EDGRMMxx parmlib OPTION CATSYSID operand. If you specify the CATSYSID(*) operand, you must ensure that the catalog environment is shared. If you specify CATSYSID(*) without ensuring that catalogs are shared, you can lose data. If you specify the CATSYSID operand with system IDS, you cannot run DFSMSrmm inventory management until the DFSMSrmm control data set and user catalogs are synchronized.

If catalogs are not fully shared, and you specify the EDGRMMxx parmlib OPTION UNCATALOG(Y/S) operand, you must run expiration processing on each system. This ensures that DFSMSrmm uncatalogs data sets in the correct catalogs on return to scratch. You must run expiration processing on each system to avoid running low on scratch volumes because DFSMSrmm only returns volumes to scratch when processing in the correct catalog environment.

To ensure that DFSMSrmm processes the return to scratch, you must also specify the EDGRMMxx parmlib OPTION CATSYSID operand with the system IDs on which the volumes were created. DFSMSrmm uses the CATSYSID operand to determine the systems to which volumes can be returned to scratch status. DFSMSrmm checks the system creation ID for the first file on the volume with the list of IDs that are specified for the CATSYSID operand. If there is a match, DFSMSrmm processes the return to scratch.

DFSMSrmm catalog processing

When you run DFSMSrmm with user catalogs and the DFSMSrmm control data set unsynchronized, DFSMSrmm VRSEL processing issues catalog locates as required to check if data sets are cataloged. Catalog locates use the standard catalog search to find if a data set is cataloged.

When you run DFSMSrmm with user catalogs and the DFSMSrmm control data set synchronized, DFSMSrmm VRSEL processing does not need to issue catalog locates to find if a data set is cataloged. The catalog status tracked by DFSMSrmm in the control data set is used to determine if a data set is cataloged. During catalog synchronization DFSMSrmm uses the Catalog Search Interface (CSI) to retrieve data set catalog information. CSI returns catalog information for all data sets in all catalogs that are in or connected to the master catalog. Because of this, DFSMSrmm can detect a data set is cataloged even if it cannot be found using the standard catalog search.

During CATSYNCH processing, DFSMSrmm matches the retrieved catalog information for tape data sets with the tape data set information in the DFSMSrmm control data set. If there is a difference in catalog status between the catalog and the DFSMSrmm control data set, a message is issued, and the status is corrected as follows:

- Status is set to YES when a catalog entry is found and the status in the DFSMSrmm control data set is not YES.
- Status is set to NO when a catalog entry is not found and the status in the DFSMSrmm control data set is YES.

- The status is unchanged when a catalog entry is not found and the status in the DFSMSrmm control data set is either NO or UNKNOWN.

Normal processing for catalog status tracking is to initially set catalog status to UNKNOWN, and then to change status to YES when the catalog entry is created, and to change status to NO when the catalog entry is deleted. Data sets with UNKNOWN status have not yet been cataloged and might never become cataloged; DFSMSrmm applies the CATRETPD value to these data sets.

JCL for catalog synchronization

To request catalog synchronization, specify the CATSYNCH parameter. You can submit a job with JCL similar to [Figure 143 on page 367](#).

```
//HSCP      EXEC PGM=EDGHSKP,
//          PARM='CATSYNCH'
//MESSAGE   DD DSN=MESSAGE.FILE.NAME,DISP=SHR
//ACTIVITY  DD DSN=ACTIVITY.FILE.NAME,DISP=SHR
```

Figure 143. Example of JCL for catalog synchronization processing

Specify the CATSYNCH and the VERIFY parameter to perform a trial run of catalog synchronization. You can submit a job with JCL similar to [Figure 144 on page 367](#).

```
//HSCP      EXEC PGM=EDGHSKP,
//          PARM='CATSYNCH,VERIFY'
//MESSAGE   DD DSN=MESSAGE.FILE.NAME,DISP=SHR
//ACTIVITY  DD DSN=ACTIVITY.FILE.NAME,DISP=SHR
```

Figure 144. Example of JCL for trial run catalog synchronization processing

IBM recommends that you run CATSYNCH with VERIFY to perform a trial run early in your implementation so that you can be sure your catalogs are free from errors before you get started. A trial run prior to implementation does not need the CATSYSID operand specified in parmlib.

Synchronizing the DFSMSrmm control data set with user catalogs in a fully shared catalog environment

You can use shared user catalogs whether or not the control data set and user catalogs are synchronized. Synchronize the DFSMSrmm control data set and user catalogs so that DFSMSrmm VRSEL processing does not need to locate catalog information for every data set that matches to a WHILECATALOG vital record specification. To synchronize the DFSMSrmm control data set with user catalogs:

1. Install the DFSMSrmm level of code that performs catalog tracking and catalog synchronization on all systems that are sharing the control data set. When you have the correct level of DFSMSrmm code installed, DFSMSrmm dynamically records all catalog updates for tape data sets in the DFSMSrmm control data set.
2. Run the DFSMSrmm EDGHSKP utility with the PARM=CATSYNCH,VERIFY parameter as a trial run to find all the differences between the DFSMSrmm control data set and the catalogs and to ensure that the catalogs can be used successfully. Before you run CATSYNCH, you should verify that your MESSAGE data set is large enough to contain all the messages issued during the verify processing. For example, when you run CATSYNCH for the first time, perhaps as part of DFSMSrmm implementation, or when you run CATSYNCH with VERIFY, you should expect messages for each data set where the status is different between the catalogs and the DFSMSrmm control data set.
3. Specify the DFSMSrmm EDGRMMxx parmlib OPTION CATSYSID(*). When the CATSYSID operand is specified in the EDGRMMxx parmlib member, you cannot run VRSEL or EXPROC processing unless the control data set is synchronized or CATSYNCH is specified in the same processing run. When running EDGHSKP expiration processing or vital record processing, if CATSYSID(*) is specified in the EDGRMMxx parmlib member and the control data set is not synchronized with the catalogs, DFSMSrmm initiates catalog synchronization even if you have not specified it.

4. Set up automatic synchronization by automating the responses to DFSMSrmm messages EDG8200E and EDG8201E. Run the EDGUTIL UPDATE with the SYSIN command CONTROL CATSYNCH(NO).
5. Run the DFSMSrmm EDGHSKP utility with the PARM=CATSYNCH parameter to synchronize all data set records in the control data set with the status from the catalog. Before you run CATSYNCH you should verify that your MESSAGE data set is large enough to contain all the messages issued during the catalog synchronization process. For example, when you run CATSYNCH for the first time, perhaps as part of DFSMSrmm implementation, or you run CATSYNCH with VERIFY you should expect messages for each data set that must be updated or the status is different between the catalogs and the DFSMSrmm control data set.

Synchronizing the DFSMSrmm control data set with user catalogs when catalogs are not fully shared

When catalogs are not fully shared, you can have non-GDG and GDG data sets with the same name that are cataloged but in different, unshared catalogs. If you are using RETENTIONMETHOD(VRSEL) and want to retain all copies of all data sets, define a retention policy that uses COUNT(99999) with DAYS rather than with CYCLES. Using CYCLES can produce unpredictable results for example, the premature expiration of data sets.

If you are using RETENTIONMETHOD(VRSEL) and need to ensure that cycles are grouped together, use the job name to further qualify the data sets, specifying a different job name for each set of cataloged data sets. For example, use JOBNAME(xjzA) for data sets created on system A, and JOBNAME(xjzB) for data sets created on system B so that the cycle retention is based on the scope of the creation system.

The DFSMSrmm control data set and user catalogs must be synchronized for DFSMSrmm to support unshared user catalogs. To synchronize the DFSMSrmm control data set with user catalogs, follow these procedures:

1. Ensure that all systems that share the control data set are running the DFSMSrmm level of code that supports catalog tracking and catalog synchronization. When you have the correct level of DFSMSrmm code installed, DFSMSrmm dynamically records all catalog updates for tape data sets in the control data set.
2. Run the DFSMSrmm EDGHSKP utility with the PARM=CATSYNCH,VERIFY parameter as a trial run to find all the differences between the DFSMSrmm control data set and the catalogs and to ensure that the catalogs can be used successfully. Before you run CATSYNCH, you should verify that your MESSAGE data set is large enough to contain all the messages that are issued during the verify processing. For example, when you run CATSYNCH for the first time, perhaps as part of DFSMSrmm implementation, or when you run CATSYNCH with VERIFY, you should expect messages for each data set where the status is different between the catalogs and the DFSMSrmm control data set.
3. Specify the DFSMSrmm EDGRMMxx parmlib OPTION CATSYSID operand with the system IDs that are common to the set of user catalogs on the system. Specify current IDs and also IDs for systems that you no longer use but for which you are still retaining tape data sets.
4. Set up automatic synchronization by automating the responses to DFSMSrmm messages EDG8200E and EDG8201E running the EDGUTIL UPDATE with the SYSIN command CONTROL CATSYNCH(NO).
5. Run the DFSMSrmm EDGHSKP utility with the PARM=CATSYNCH parameter to partially synchronize all data set records in the control data set with the status from the accessible catalogs. Run CATSYNCH on other systems that share the control data set until all the catalogs have been synchronized. Before you run CATSYNCH, verify that your MESSAGE data set is large enough to contain all the messages issued during the catalog synchronization process. For example, when you run CATSYNCH for the first time, perhaps as part of DFSMSrmm implementation, or you run CATSYNCH with VERIFY you should expect messages for each data set that must be updated or the status is different between the catalogs and the DFSMSrmm control data set.
6. Run EDGUTIL with PARM=UPDATE parameter and CONTROL CATSYNCH(YES) to set the 'last catalog synchronization date and time' in the DFSMSrmm control data set control record.
7. Run inventory management functions other than expiration processing on any system once if you are using retention method VRSEL. However, if you run expiration processing to return volumes to scratch

status, DFSMSRmm only processes a subset of volumes if the first file created system ID matched to one of the systems that are specified in CATSYSID(sysid_list).

8. To ensure that on returning volumes to scratch DFSMSRmm will uncatalog data sets in the correct catalogs you must run EDGHSKP expiration processing on multiple systems if you specify the EDGRMMxx parmlib OPTION UNCATALOG(Y or S). You should run expiration processing once for each set of user catalogs on a system with access to those user catalogs as specified in CATSYSID(sysid_list).

Confirming global volume movement

Vital record processing determines the volume movements required and storage location management processing assigns the volume destinations. You must ensure that volume movements are completed and confirmed to DFSMSRmm.

You use the RMM subcommands and DFSMSRmm ISPF dialog to confirm that volume movements have taken place. You use the RMM CHANGEVOLUME * CONFIRMMOVE subcommand to change the status of a volume move from PENDING to CONFIRMED. You can perform confirmation for a single volume or for multiple volumes. Performing confirmation for multiple volumes is called global confirmation. In DFSMSRmm, confirmation means to issue a command to notify DFSMSRmm that a volume has moved to a location or that a release action defined for a volume has been completed. If you perform confirmation for a single volume, DFSMSRmm processes the confirmation immediately. If you issue a request for a global confirmation, DFSMSRmm processes the global confirmation during inventory management processing.

Global move confirmation is performed as part of DFSMSRmm inventory management. DFSMSRmm performs move confirmation for a volume, prior to making any other updates to a volume, as part of these inventory management functions:

- Vital record processing
- Storage location management processing
- Expiration processing. Only if all volumes are processed.

If a volume move is outstanding, and it has been globally confirmed, DFSMSRmm performs confirmation of the move for that volume. DFSMSRmm confirms moves in progress prior to starting any new moves that might be set by the current inventory management run. If each of the inventory management functions is in a separate job step, then global move confirmation is performed once for each step. If the inventory management functions are run in a single job step, DFSMSRmm performs global move confirmation once.

The timing of the inventory management runs is very important. Here are some things to consider:

- If you requested global move confirmation and want to change it, you can undo the request prior to the next run of inventory management. You can use the RMM CHANGEVOLUME subcommand to undo global confirmation of volume moves. After the next run of inventory management, DFSMSRmm will have confirmed the volume moves making undoing the confirmation of the moves difficult.
- When you request an extract data set to be used for generating movement or inventory reports, consider the timing of inventory management functions. An extract data set produced after a run of vital record processing and storage location management processing, contains the information of new volume movement for movement reports. An extract data set produced after a global confirm move and a run of any of the listed inventory management functions, contains information about volumes after moves have been confirmed.
- To make scratch volumes available after a move and when you use global move confirmation, make sure you run expiration processing to complete the global move confirmation and return the moved volumes to scratch status.

Confirming global release actions

If the RMM CHANGEVOLUME subcommand with * CONFIRMRELEASE operands is issued to confirm global release actions, for any volume with the action outstanding, DFSMSRmm marks the corresponding action complete. Global release action confirmation takes place during expiration processing, but only

if all volumes are processed, just prior to when DFSMSrmm performs a check for outstanding global confirmation or if the volume expiration date has been reached.

When you use the NOTIFY release action, the NOTIFY release action must be confirmed in order for any other release action to be processed. DFSMSrmm can confirm this automatically by sending a notify message, or you can confirm it manually. See [“OPTION command operands” on page 168](#) for more information about the NOTIFY parmlib OPTION.

If the ‘return to owner’ action is confirmed, the volume is deleted from the DFSMSrmm control data set. If the erase or initialize release actions are confirmed, DFSMSrmm deletes the data set information for those volumes with the actions outstanding. If erase is confirmed, DFSMSrmm sets the initialize action as outstanding so that the correct volume labels can be written to the erased volume.

When you use global confirmation for the ERASE release action, you process the volume outside the control of DFSMSrmm. Possibly, you could have degaussed the volume or used a bulk degaussing machine. Remember that you should only degauss certain types of tape volumes.

Note: Do not degauss IBM 3592, 3590, or similar media that are pre-formatted with servo tracks. Use of a degausser renders the volume unusable.

If the RMM DELETEVOLUME subcommand with the RELEASE operand was issued, DFSMSrmm changes the volume status to ‘pending release’ during subcommand processing. Use expiration processing to complete the release process. These volumes are returned to scratch during the first EDGHSKP run, if no other release actions are required.

Backing up the control data set

You can use EDGHSKP or EDGBKUP described in [Chapter 14, “Maintaining the control data set,” on page 377](#) to back up the control data set and the journal. Choose the appropriate utility based on your needs. You can start the back up of the control data set or journal at any time. But you cannot start backup if backup is already in progress. You need to schedule backing up and clearing of the journal before it exceeds its threshold using the EDGHSKP utility.

You can request that DFSMSrmm backup processing use DFSMSdss COPY services instead of using DUMP services. Use of COPY services enables the system to exploit the fast replication services of the DASD subsystems, such as FlashCopy V2 data set level flashcopy. COPY services enable an almost instant copy of the DFSMSrmm CDS to be created, which is ready to use without the need to first restore a backup copy.

Note: When you backup the CDS, you have the choice to either create a portable backup of the CDS or to create a copy of the CDS that is usable by DFSMSrmm without first being restored.

- Use EDGHSKP to back up both the control data set and the journal when the DFSMSrmm subsystem is active by specifying the BACKUP DD statement and the JRNLBKUP DD statement.

Use the EDGHSKP utility to back up the control data set and to clear the journal data set.

- Use EDGBKUP to back up the control data set and journal when the DFSMSrmm subsystem is not active.

Use the EDGBKUP utility to back up the control data set and journal when DFSMSrmm is active. EDGBKUP does not clear the journal data set.

Schedule regular backups for both the control data set and journal. Backing up the DFSMSrmm control data set and the journal as the first step in inventory management is a way for you to create a restore point for the control data set. Schedule back ups of the journal when it reaches the threshold that you define. Use the DFSMSrmm parmlib OPTION command JOURNALFULL operand, described in [“Defining system options: OPTION” on page 161](#). Clearing out the journal data set regularly prevents the journal from becoming full, and thus reduces the risk of losing the updates to the control data set. You can automate the process of backing up the control data set and clearing the journal as described in [“Steps for automating control data set backup and journal clearing” on page 374](#).

You should plan to keep multiple backup copies, so that, should a backup fail, you still have a previous, successful backup from which to recover. Use the current journal with the most recent control data set and journal backups to forward recover to the point of failure.

You can request that DFSMSrmm back up the control data set and the journal, using the access method services (AMS) REPRO command, the DFSMSdss COPY command, or DFSMSdss DUMP command. When you use the access method services REPRO command or DFSMSdss without concurrent copy or without fast replication for backup, DFSMSrmm does not allow updates to the control data set during control data set backup. Backup using DFSMSdss DUMP enables output directly to tape. DFSMSrmm resets the journal data set and discards journal records if the back up completes successfully. It is not necessary to use DFSMSdss concurrent copy when you specify BACKUP(DSS).

When DFSMSrmm is performing an intrusive backup, some processing might wait until the backup completes or some command processing can be interrupted. DFSMSrmm fails RMM TSO subcommand requests that update the DFSMSrmm control data set. For example, if you start backup using the AMS REPRO backup method, command processing can be interrupted. DFSMSrmm functions such as the DFSMSrmm TSO SEARCH subcommands, that only read the control data set, continue uninterrupted.

Example: Issue the RMM ADDRACK subcommand to add 2000 new shelf locations.

```
RMM ADDRACK RK0000 COUNT(2000)
```

Example: If backup processing starts before the request completes, DFSMSrmm issues these messages.

```
EDG3212E REQUEST REJECTED - DFSMSrmm BACKUP  
      CURRENTLY IN PROGRESS  
EDG3017I THE ERROR OCCURRED WHILE ADDING RACK NUMBER RK0700  
EDG3018I 700 RACK NUMBER(S) ADDED
```

Example: When backup completes, issue the ADDRACK subcommand to add the remainder of the shelves.

```
RMM ADDRACK RK0700 COUNT(1300)
```

When a tape is used during inventory management, DFSMSrmm updates the control data set. If a job that opens or closes a tape data set tries to update the control data set during AMS REPRO backup processing, DFSMSrmm waits for up to five minutes. If backup is still in progress, DFSMSrmm issues a write-to-operator message EDG4010D that prompts the operator to retry or cancel each job. If the operator specifies retry, DFSMSrmm retries the job five more times at minute intervals before again issuing a write-to-operator if backup processing has not been completed.

When you use the DFSMSdss DUMP command to request a back up, plan to keep backup copies for both the control data set and the journal. For any backup being restored, a journal backup is also required for forward recovery. For the latest control data set backup, both the latest journal backup and the current journal must be used for forward recovery.

To use DFSMSdss concurrent copy, you must authorize the DFSMSrmm batch inventory management backup job to the RACF profile STGADMIN.ADR.DUMP.CNCURRNT. See [z/OS DFSMSdss Storage Administration](#) for information about protecting DFSMSdss keywords with RACF.

When you use DFSMSdss to back up the control data set and you use concurrent copy or fast replication, DFSMSrmm resets the DFSMSrmm journal only if you also back up the journal data set. The journal and the journal backup are required to forward recover to the latest point in time.

When you use DFSMSdss concurrent copy or fast replication for backup, DFSMSrmm continues to process commands and update the control data set. As an alternative to use of concurrent copy capable hardware, you can use snapshot capable hardware. When you request backup using DFSMSdss, DFSMSdss uses CC-compatible snapshot instead of concurrent copy.

Ensuring a consistent copy of the DFSMSrmm CDS

When creating a backup copy of the DFSMSrmm control data set, care must be taken to avoid creating a fuzzy backup. A fuzzy backup means that the control data set was in one state when the backup started, but in another state by the time the backup finished. As a result, the backup copy could represent

an inconsistent state for the control data set and could be useless, because a restore would yield inconsistent CDS records.

You can create a consistent copy of the DFSMSrmm control data set using any of the following methods:

1. EDGHSKP/EDGBKUP BACKUP

The target of the BACKUP request is a VSAM KSDS (AMS REPRO is used). You can optionally use forward recovery to a more recent level of the CDS by means of journal or DFSMSrmm SMF records. If BACKUP(AMS), or BACKUP(DSS) without concurrent copy, is used, the backup processing prevents updates to the DFSMSrmm control data set until the backup completes.

2. EDGHSKP/EDGBKUP followed by RESTORE (AMS or BACKUP(DSS) backup can be used)

BACKUP(DSS) creates a portable data set by means of the DFSMSdss DUMP command. You can optionally use forward recovery to a more recent level of the CDS by means of journal or DFSMSrmm SMF records. If BACKUP(AMS), or BACKUP(DSS) without concurrent copy, is used, the backup processing prevents updates to the DFSMSrmm control data set until the backup completes.

3. EDGHSKP/EDGBKUP with the BACKUP(COPY) parameter This ensures that a consistent, non-fuzzy, copy is created under DFSMSrmm control. No forward recovery is needed unless a more recent point in time of the CDS is required.

4. XRC, PPRC, Metro Mirror, Global Mirror

These options use DASD mirroring technologies to create a copy of the CDS. Forward recovery from journal records is required to ensure the copy is not fuzzy. Although the mirroring technologies create copies that are consistent with the source data, because of the way DFSMSrmm updates its CDS, the CDS records themselves might not be consistent. The journal records are written before the CDS is updated, so the most recent complete set of journal records can be used to de-fuzz the CDS.

5. FlashCopy

RMM must be quiesced or stopped to ensure that a CDS copy is not “fuzzy”. For Volume or Data Set Level (FlashCopy V2), the DASD subsystem must be a FlashCopy capable disk subsystem. Forward recovery from journal records is required to ensure the copy is not fuzzy.

JCL for using backup to create a CDS copy

Use the DSSOPT DD to override the default COPY command options used by DFSMSrmm. See “Customizing the DSSOPT DD statement” on page 384 for details of the DSSOPT file. The BACKUP DD is optional when you request BACKUP(COPY). When you specify the BACKUP DD, DFSMSrmm builds the COPY command with the OUTDD(BACKUP) keyword to direct where the copy is created. Otherwise, your SMS managed environment determines where the copy is created.

```
//EDGHSKP EXEC PGM=EDGHSKP,PARM=' BACKUP(COPY) '  
//MESSAGE DD DISP=SHR,DSN=RMM.MESSAGE  
//SYSPRINT DD SYSOUT=*  
//BACKUP DD DISP=SHR,UNIT=SYSDA,VOL=SER=myvol  
//DSSOPT DD *  
FR(PREF) CONCURRENT(ANYREQUIRED) -  
RENAMEU((*.CDS.**,*.COPYCDS.**)) REPLACEU -  
DEBUG(FRMSG(DETAILED))  
/*
```

Figure 145. Example of JCL for using backup to create a CDS copy

JCL for backing up the control data set and journal

DFSMSrmm obtains the name of the control data set and journal from the running DFSMSrmm subsystem.

Rule: Do not specify the data set names in the JCL. If you do, the job fails.

To create a backup copy using EDGHSKP, specify the BACKUP parameter.

1. Submit a job to back up the control data set to tape with JCL as shown in [Figure 146 on page 373](#).

```
//EDGHSKP EXEC PGM=EDGHSKP,PARM='BACKUP(DSS)'
//MESSAGE DD DISP=SHR,DSN=RMM.MESSAGE
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.CDS(+1),
// LABEL=(,SL)
// AVGREC=U,LRECL=9216,BLKSIZE=0,RECFM=U
//JRNLBKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.JRNL(+1),
// DCB=(RECFM=VB,BLKSIZE=0,LRECL=9248),
// LABEL=(2,SL),VOL=REF=*.BACKUP
//DSSOPT DD *
//CONCURRENT OPTIMIZE(1) VALIDATE
/*
```

Figure 146. Example of JCL for backing up the control data set and the journal to tape

2. Specify the JRNLBKUP DD statement only if you want to back up the journal. The DSSOPT DD statement is optional and allows you to customize the DFSMSdss DUMP and DFSMSdss RESTORE commands. See “Customizing the DSSOPT DD statement” on page 384 for information about changing the DSSOPT DD statement.
3. Specify BACKUP(DSS) when you want to use DFSMSdss to perform the back up. You do not need to use concurrent copy to specify BACKUP(DSS). Using DFSMSdss concurrent copy permits the update of the control data set during backup processing so that all tape activities can continue during backup processing. To use DFSMSdss concurrent copy, you must have a concurrent copy environment set up. If you specify BACKUP(DSS) without concurrent copy, you can still direct the output to tape. All other updates to the control data set will wait until the control data set backup completes. If you specify BACKUP(DSS) and you use concurrent copy, DFSMSrmm clears the journal only if you also back up the journal.

You can submit a job to back up the control data set to DASD with JCL as shown in [Figure 147 on page 373](#).

```
//BACKUP EXEC PGM=EDGHSKP,PARM='BACKUP(AMS)'
//SYSPRINT DD SYSOUT=*
//MESSAGE DD DSN=RMM.MESSAGE(0),
// DISP=SHR
//BACKUP DD DSN=RMM.BACKUP(+1),
// DISP=(,CATLG,DELETE),
// UNIT=SYSALLDA,
// AVGREC=U,SPACE=(4096,(1000,500),RLSE),
// LRECL=9216,BLKSIZE=0,RECFM=VB,
// BUFNO=30,
// DCB=RMM.GDGMODEL
//JRNLBKUP DD DSN=RMM.JRNLBKUP(+1),
// DISP=(,CATLG,DELETE),
// UNIT=SYSALLDA,
// AVGREC=U,SPACE=(4096,(1000,500),RLSE),
// LRECL=9248,BLKSIZE=0,RECFM=VB,
// BUFNO=30,
// DCB=RMM.GDGMODEL
```

Figure 147. Example of JCL for backing up the control data set and journal to DASD

Backing up the journal

You can backup and clear the journal without backing up the control data set. DFSMSrmm allows updates to the control data set during journal backup so that the impact of journal backup on other tasks is low. Use EDGHSKP when you want to backup and clear the journal. Use EDGBKUP when you only want to back up the journal and not clear it. Use the RMM LISTCONTROL TSO subcommand to find the time of the last journal back up.

When the journal reaches its threshold, schedule a backup of just the journal to minimize the time needed to clear the journal. When you restore the control data set, ensure that the correct journal backups are used for forward recovery.

JCL for backing up the journal

DFSMSrmm obtains the name of the control data set and journal from the running DFSMSrmm subsystem.

Rule: Do not specify the data set names in the JCL. If you do, the job fails.

You can back up and clear the journal without taking a backup of the control data set. To back up the journal using EDGHSKP, specify the BACKUP parameter as shown in [Figure 148 on page 374](#).

```
//EDGBKP EXEC PGM=EDGHSKP,PARM='BACKUP'  
//SYSPRINT DD SYSOUT=*  
//MESSAGE DD DISP=SHR,DSN=RMM.MESSAGE  
//JRNLBKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.JRNL(+1),  
// LRECL=9248,BLKSIZE=0,RECFM=VB,  
// BUFNO=30,  
// LABEL=(1,SL)
```

Figure 148. Example of JCL for backing up and clearing the journal

Steps for automating control data set backup and journal clearing

You need to ensure that the journal does not fill up because a full journal can impact tape usage on your system. You can run regular backups as part of inventory management in order to clear the journal. Automating control data set backup and clearing the journal help to ensure that the journal does not fill up.

To automate the clearing of the journal, follow these steps:

1. Specify a value for JOURNALFULL operand on the OPTION command in parmlib as described in [“Defining system options: OPTION” on page 161](#). If you do not specify a value, DFSMSrmm uses a value of 75%. Use only a single system to trigger backup using the journal threshold. If you have multiple systems and the journal threshold is reached, each system can start the backup procedure at the same time if you use the same threshold number. To avoid this situation if you have multiple systems, you can specify a different threshold on each system. For example, specify a threshold of 75% on the main system where DFSMSrmm is active and then specify thresholds of 80% and 85% for your other systems. You can also disable threshold processing on a system by specifying a zero value. Ensure that the systems you select have DFSMSrmm active and have a high chance of processing DFSMSrmm requests. A system that processes no or few DFSMSrmm requests is a bad choice for automatic backup because DFSMSrmm only checks the journal threshold when a request is processed.
2. Create a backup procedure in the system procedure library. The backup procedure that you write should use EDGHSKP to backup the control data set and journal because EDGHSKP also clears the journal. To automate the clearing of the journal without backing up the control data set, write your backup procedure to backup just the journal. See [“Automating backup” on page 459](#) for an example of a procedure that runs backup as part of inventory management. Alternatively, you might want to use the procedure to submit a batch job to perform the backup or to inform your job scheduler to submit the job. DFSMSrmm will not start the procedure if backup is already in progress. See [“Event triggered tracking” on page 482](#) for information about using the IBM Tivoli Workload Scheduler for z/OS.
3. Specify the procedure name for the BACKUPPROC operand of the OPTION command in parmlib as described in [“Defining system options: OPTION” on page 161](#). DFSMSrmm starts this procedure when the journal threshold is reached. If you do not specify a procedure name, you cannot automate backup using DFSMSrmm. However, you could still use the EDG2107E message as a trigger for your site automation processes.

Return codes for EDGHSKP

EDGHSKP can issue the return codes shown in [Table 50 on page 374](#).

Table 50. EDGHSKP return codes

Return Code	Explanation
0	All requested functions completed successfully.

Table 50. EDGHSKP return codes (continued)

Return Code	Explanation
4	DFSMSrmm encountered a minor error during processing. It issues a warning message and processing continues.
8	DFSMSrmm has stopped at least one requested function. Processing continues with the next requested function.
12	A severe error occurred during processing of one of the requested functions. For example, the operator has cancelled the inventory management run. DFSMSrmm stops the utility.
16	A severe error occurred during a required communication with the DFSMSrmm subsystem. DFSMSrmm stops the utility.

Chapter 14. Maintaining the control data set

DFSMSrmm samples provided in SAMPLIB

- EDGJBKUP Sample JCL for Using the Backup Program
- EDGJUTIL Sample JCL for Initializing the Control Data Set
- EDGJMFAL Sample JCL for Allocating the Control Data Set
- EDGJNLAL Sample JCL for Allocating the Journal

Before you begin: DFSMSrmm provides you with the EDGHSKP utility, the EDGBKUP utility, the EDGUTIL utility, the EDGSPLCS utility, and the EDGUPDT utility that you can use to maintain the control data set. EDGHSKP and EDGBKUP provide functions that you can use to back up the DFSMSrmm control data set. Review these descriptions to determine which utility you should use for back up.

Use the DFSMSrmm utility EDGHSKP described in [“Backing up the control data set” on page 370](#) to perform these tasks:

- Back up the DFSMSrmm control data set and journal when DFSMSrmm is active.
- Clear the journal data set. Clear the journal only after back up is completed successfully to avoid losing changes that are made since the last backup. Without these latest changes, a forward recovery can only recover the control data set up to the last backup before the journal was cleared.

Use the DFSMSrmm utility EDGBKUP described in [“Backing up the control data set” on page 386](#) to perform these tasks:

- Back up the DFSMSrmm control data set and journal when DFSMSrmm is active.
- Back up or restore the DFSMSrmm control data set when DFSMSrmm is stopped or quiesced. You can use EDGBKUP independently of DFSMSrmm to back up, restore, and reorganize the DFSMSrmm control data set and to back up the journal.

Use the DFSMSrmm utility EDGUPDT described in [“Updating the active control data set” on page 387](#) to update the active DFSMSrmm CDS with record updates created during testing or recovery.

Perform these tasks to maintain the DFSMSrmm control data set:

- Create the control data set control record as described in [“Creating or updating the control data set control record” on page 409](#).
- Back up the control data set and the journal when DFSMSrmm is active using either EDGHSKP as described in [“Backing up the control data set” on page 370](#) or EDGBKUP as described in [“Backing up the control data set” on page 386](#).
- Back up the control data set and the journal when DFSMSrmm is inactive using EDGBKUP as described in [“Backing up the control data set” on page 386](#).
- Back up the journal when DFSMSrmm is active or inactive using either EDGHSKP as described in [“Backing up the journal” on page 373](#) or EDGBKUP as described in [“Backing up the control data set” on page 386](#).
- Clear the journal by using EDGHSKP when DFSMSrmm is active as described in [“JCL for backing up the journal” on page 373](#).
- Restore the control data set, and optionally forward recover when DFSMSrmm is stopped or quiesced using EDGBKUP as described in [“Restoring the control data set with forward recovery” on page 390](#).
- Restore the control data set by using non-DFSMSrmm products such as IDCAMS and DFSMSdss as described in [“Using non-dfsmsrmm utilities to restore the control data set” on page 392](#).
- Forward recover the control data set by using EDGBKUP as described in [“Forward recovering the control data set” on page 391](#).

- Reorganize the control data set using EDGBKUP as described in [“Reorganizing the control data set”](#) on page 393.
- Move the control data set by using EDGHSKP as described in [“Moving the control data set and journal to a different device”](#) on page 397.
- Move the control data set by using non-DFSMSrmm utilities such as IDCAMS and products such as DFSMSdss as described in [“Steps for moving the control data set using non-dfsmsrmm utilities”](#) on page 400.
- Move the journal as described in [“Moving the journal using DFSMSrmm utilities”](#) on page 399.
- Recover from control data set update failures by using EDGBKUP as described in [“Recovering from control data set update failures”](#) on page 395.
- Verify the contents of the control data set by using EDGUTIL as described in [“Verifying the contents of the control data set”](#) on page 411.
- Repair the control data set by using EDGUTIL as described in [“Mending the control data set”](#) on page 414.
- Regularly test any procedures or jobs you create to handle recovery of the DFSMSrmm control data set. This ensures that they work and that you and operations are familiar with how they work and what is required.

This topic contains information for using the DFSMSrmm EDGBKUP utility and the EDGUTIL utility:

- [“Using EDGBKUP”](#) on page 379
- [“Using EDGUTIL for tasks such as creating and verifying the control data set”](#) on page 401

Use the DFSMSrmm utility EDGSPLCS, described in [“Using EDGSPLCS to issue commands to OAM for system-managed volumes”](#) on page 418, to issue supported commands to OAM for system-managed volumes.

DFSMSrmm considerations when client/server support is enabled

This topic describes utilities that provide restricted functions when run on a client or server system.

DFSMSrmm Utility	Considerations	Where to Find More Information
EDGUTIL	- When you run EDGUTIL on a client system, the utility can only process a DFSMSrmm control data set that is not in use by DFSMSrmm. Because there is no control data set for a client system, you cannot run with the active control data set. However, you could recover a backup copy of the DFSMSrmm control data set to the client system to run VERIFY(VOLCAT), VERIFY(SMSTAPE), and MEND(SMSTAPE). - When you run EDGUTIL on a server system, you cannot access the TCDB or library for tape libraries that you can only access from the client system. - When you run EDGUTIL on a client system and do not specify the MASTER DD, DFSMSrmm issues message EDG6101E and the utility ends with return code 12.	“Using EDGUTIL for tasks such as creating and verifying the control data set” on page 401
EDGBKUP	You cannot process an active DFSMSrmm control data set on a client system. All functions are available on a client system as long as you provide the DD statements for the control data set and journal. This enables you to use the BACKUP and RESTORE options independent of the DFSMSrmm subsystem. When you run EDGBKUP on a client system and do not specify the MASTER DD or JOURNAL DD, DFSMSrmm issues message EDG6101E and the utility ends with return code 12.	“Backing up the control data set” on page 370

Using EDGBKUP

This topic describes the JCL, EXEC parameters, DD statements, and return codes associated with using the EDGBKUP utility. In addition, this topic describes how to customize the DSSOPT DD statement.

JCL for EDGBKUP

This topic describes the EXEC parameters for EDGBKUP and provides JCL examples to back up the control data set and journal, restore the control data set, and reorganize the control data set. [Figure 149 on page 380](#) shows the JCL for EDGBKUP.

```
//EDGBKUP EXEC PGM=EDGBKUP,PARM=' BACKUP(DSS) '
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.CDS
//JOURNAL DD DISP=SHR,DSN=RMM.JOURNAL
//BACKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.CDS(+1),
// LABEL=(,SL)
//JRNLBKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.JRNL(+1),
// LABEL=(2,SL),VOL=REF=*.BACKUP,LRECL=9248,
// BLKSIZE=0,RECFM=VB
```

Figure 149. JCL example for EDGBKUP

EXEC parameters for EDGBKUP

Figure 150 on page 380 shows the EXEC parameters for EDGBKUP.

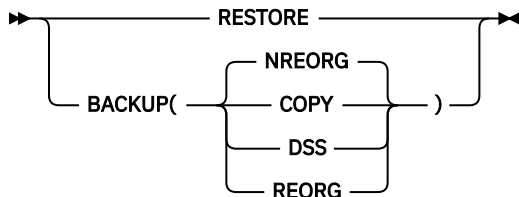


Figure 150. EDGBKUP EXEC parameters

RESTORE

Specify RESTORE to restore and forward recover the control data set from a backup copy. When you specify the BACKUP DD statement, DFSMSrmm restores the control data set. When you specify the JOURNAL DD or SMFIN DD statements, forward recovery is attempted. You can specify both the BACKUP DD statement and the JOURNAL or SMFIN DD statement in the same job step to restore and forward recover the control data set from journal backups and the journal data set or from SMF records.

RESTORE allows you to customize the DFSMSdss options that DFSMSrmm uses. See [“Customizing the DSSOPT DD statement”](#) on page 384 for additional information.

Forward recovery using SMF records is only possible when you use the IBM SMF record type by means of parmlib option SMFAUD(YES). You cannot mix JOURNAL and SMFIN records for forward recovery in a single execution of EDGBKUP. Message EDG6138E is issued if you try this. When you have some journal records and some SMF records, run EDGBKUP multiple times with either SMFIN or JOURNAL specified in the correct sequence from the oldest records to the newest to the recovery target or latest time. If JOURNAL records are available for forward recovery, IBM recommends that you use journal records instead of SMF records.

When you forward recover from SMF records, it might not be possible to recover to an exact time that matches with the current DFSMSrmm JOURNAL. On restart of DFRMM, WTOR EDG2106D can be expected.

Note: When you plan using SMF records for forward recovery it is necessary to use the START and END parameters on the IFASMF DL and IFASMF DP jobs so that all systems supply SMF records for the same time frame. We would recommend a START time of a couple of minutes before the CDS backup was taken and an END time a couple of minutes after the CDS has been noticed to be broken (perhaps EDG2100I has been issued for example). If these parameters are not used for dumping the SMF data on all sharing systems then error message EDG6428E can be expected in the job performing the forward recovery.

BACKUP(COPY | DSS | NREORG | REORG)

Specify BACKUP to control the way DFSMSrmm backs up, copies, and optionally reorganizes the DFSMSrmm control data set.

Restriction: You cannot RESTORE or REORG an active DFSMSrmm control data set. You must stop or quiesce DFSMSrmm or ensure that the control data set is not in use by DFSMSrmm.

COPY

Specify BACKUP(COPY) to use DFSMSDss to create a logical data set copy of the control data set and optionally, a backup of the journal. DFSMSrmm uses DFSMSDss to create the copy of the control data set and IDCAMS to back up the journal. To allow updates to the control data set during backup processing, set up the DFSMSDss concurrent copy environment, virtual concurrent copy environment, or a fast replication with virtual concurrent copy environment. You must have the hardware and software required to establish a concurrent copy session, a virtual concurrent copy session, or fast replication with concurrent copy or virtual concurrent copy.

Restriction: If you specify BACKUP(COPY) without establishing a non-intrusive copy environment, DFSMSrmm performs back up processing, but DFSMSrmm does not allow updates to the control data set until DFSMSDss notifies DFSMSrmm that the control data set copy or copy initialization completes. A non-intrusive copy environment is guaranteed by using the FR(PREFERRED) together with CONCURRENT(ANYREQUIRED) keywords. When you use FR(REQUIRED), there is no notification to DFSMSrmm until all DFSMSDss processing is completed. Although this might be a very short time, any CDS updates attempted during that period will be processed as if an intrusive backup is in progress. IBM recommends that you use FR(PREFERRED) with CONCURRENT.

DSSOPT is an optional DD statement that is for use with BACKUP(COPY). BACKUP allows you to customize the DFSMSDss COPY options that DFSMSrmm uses. See [“Customizing the DSSOPT DD statement” on page 384](#) for additional information.

The CDS copy that is created is ready for use with DFSMSrmm. No restore is required. However, you can optionally use the EDGBKUP utility with RESTORE and MASTER, and JOURNAL DD statements to perform forward recovery to your selected point in time.

DSS

Specify BACKUP(DSS) to use DFSMSDss to back up the control data set. DFSMSrmm uses DFSMSDss to back up the control data set and IDCAMS to back up the journal. To allow updates to the control data set during backup processing, set up the DFSMSDss concurrent copy environment or virtual concurrent copy environment. You must have the hardware and software required to establish a concurrent copy session or a virtual concurrent copy session.

Restriction: If you specify BACKUP(DSS) without establishing a concurrent copy session, DFSMSrmm performs back up processing but DFSMSrmm does not allow updates to the control data set until the control data set backup completes. If DFSMSrmm is active and you specify BACKUP(DSS), DFSMSrmm allows backup directly to tape when the journal is not full or locked.

DSSOPT is an optional DD statement that is for use with BACKUP(DSS). BACKUP allows you to customize the DFSMSDss options that DFSMSrmm uses. See [“Customizing the DSSOPT DD statement” on page 384](#) for additional information.

You can use the EDGBKUP utility or DFSMSDss to restore backups of the control data set that are created using DFSMSDss. If you use DFSMSDss to restore the backup, you must use the EDGBKUP utility to perform forward recovery to reset the control information in the restored control data set. DFSMSrmm issues message EDG0123D when you start DFSMSrmm and have not reset the control information in the control data set.

NREORG

Specify BACKUP(NREORG) to use IDCAMS to back up the control data set and to back up the journal data set.

NREORG is the default.

REORG

Specify BACKUP(REORG) to reorganize the control data set during backup processing. DFSMSrmm uses IDCAMS to back up the control data set, to optionally back up the journal, and to restore the control data set from the backup.

DD statements for EDGBKUP

The data sets that are used by the EDGBKUP utility are described in [Table 51 on page 382](#).

Table 51. DFSMSrmm EDGBKUP data sets

DD Statement	Description
BACKUP	Contains the backup copy of the DFSMSrmm control data set. This data set is optional. When you run backup, specify the BACKUP DD statement, the JRNLBKUP DD statement, or both statements. You can back up directly to tape when you specify the BACKUP(DSS) parameter on the EXEC statement even when you have not set up DFSMSdss concurrent copy when DFSMSrmm is active. The BACKUP DD statement is optional for BACKUP(COPY) when you create a copy of the CDS and the target environment is SMS-managed.
DSSOPT	Contains DUMP, COPY, or RESTORE command options used by DFSMSdss during processing. This data set is optional. See “Customizing the DSSOPT DD statement” on page 384 for information about changing the commands.
JOURNAL	Identifies the DFSMSrmm journal to be used during backup processing. Identifies the journal backups and journal to be used during restore processing. JOURNAL is mutually exclusive with SMFIN for RESTORE.
JRNLBKUP	Contains the backup copy of the DFSMSrmm journal. This data set is optional. When you run backup, specify the BACKUP DD statement, the JRNLBKUP DD statement, or both statements. DFSMSrmm uses IDCAMS to back up the journal when you specify the BACKUP(AMS) or BACKUP(DSS) parameter. You can back up the journal directly to tape when you specify the BACKUP(DSS) parameter when DFSMSrmm is active. Although the journal is initialized as RECFM=VB and LRECL=32756, when running EDGHSKP or EDGBKUP PARM='BACKUP BACKUP(DSS)' you can specify RECFM=VB,LRECL=9248 for JRNLBKUP DD to optimize dasd space.
MASTER	Identifies the DFSMSrmm control data set. This data set is optional when you code the BACKUP on the EXEC statement if DFSMSrmm is active. This data set is required for RESTORE except when DFSMSdss is used for RESTORE. Only specify the MASTER DD if the target data set already exists.
SMFIN	Identifies the data set containing the SMF records to be used during restore processing. The only SMF records supported are the IBM SMF type 42 subtype 22. All other records are ignored. SMFIN is mutually exclusive with JOURNAL for RESTORE.
SYSIN	Use the SYSIN file to select the journal records used for forward recovery of the CDS. When you specify the JOURNAL or SMFIN DD statement during RESTORE to request forward recovery, you can use the SYSIN file to specify a RESTORE command as described in “SYSIN file for EDGBKUP” on page 382 to specify a target recovery point. The SYSIN DD statement is optional.
SYSPRINT	Contains the utility program messages that IDCAMS and ADRDSSU issue when backing up the DFSMSrmm control data set. This data set can be a SYSOUT file.

SYSIN file for EDGBKUP

Use the SYSIN file to select the target recovery point for forward recovery of the CDS.

Forward recovery starts at the first record in the JOURNAL or SMFIN file that matches the CDS forward recover start point, or if the start point cannot be found, the first record in sequence after that point. Forward recovery continues until the point in time you select or to the end of the input records, whichever occurs first.

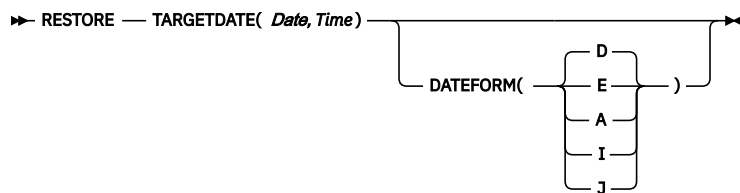


Figure 151. EDGBKUP SYSIN file

RESTORE

Use this command in the SYSIN file to select your chosen forward recovery point

When you do not specify RESTORE, DFSMSrmm uses all the available complete sets of records to perform forward recovery to the latest available point in time.

TARGETDATE(Date, Time)

Use the TARGETDATE operand to specify the date and time you have selected to be your target forward recovery point in time. You must specify both a date and a time. The date and time are the local date and time as recorded in the journal or SMF records used for forward recovery. Any input records created at a time higher than the value you specify are skipped.

Date

Specify the date in the format set by the DATEFORM operand, or if the DATEFORM operand is not specified, by the DATEFORM parmlib value. For example, if your installation set DATEFORM(J), specify: TARGETDATE(2008/123,23:22:10.0)

Time

Specify the target local time in the format hh:mm:ss.t

The '.t' (tenths of a second) is optional.

When you forward recover to a specific point in time, you have to restart DFRMM with an empty/new/cleared JOURNAL data set because DFSMSrmm will be unable to match your recovered control data set with your existing JOURNAL. If you do not ensure the JOURNAL is new or empty, DFSMSrmm start-up will issue message EDG2106D and expect the operator to handle that WTOR.

After the completion of EDGBKUP, you should either empty the JOURNAL, or allocate a new JOURNAL prior to starting DFRMM. If you start DFSMSrmm without doing that, you will receive message EDG2106D. The correct action is to reply L for LOCK, and then run EDGHSKP BACKUP of CDS and JOURNAL to cause JOURNAL to be cleared. You can then continue with tape processing.

DATEFORM (A | E | I | J | D)

Use this optional operand to specify the format for the date you enter in the TARGETDATE operand.

Value	Language	Format	Example
A	American	mm/dd/yyyy	12/15/2013
E	European	dd/mm/yyyy	15/12/2013
I	ISO	yyyy/mm/dd	2013/12/15
J	Julian	yyyy/ddd	2013/349
D	Default	Installation's default in EDGRMMxx	Initially set to Julian

The default date format for all date fields is the value specified in the parmlib member EDGRMMxx. The value is initially set to J for Julian. To change the date format for each run of EDGBKUP, use the DATEFORM parameter.

Return codes for EDGBKUP

EDGBKUP can issue the return codes shown in [Table 52 on page 384](#).

Table 52. EDGBKUP return codes

Return Code	Explanation
0	All requested functions completed successfully.
4	DFSMSrmm encountered a minor error during processing. It issues an informational message and continues processing.
8	DFSMSrmm encountered errors during the forward recovery or restore process. DFSMSrmm has updated the control record in the recovered control data set to show that recovery was not successful.
12	DFSMSrmm encountered a severe error during processing of one of the requested functions. DFSMSrmm stops the utility. DFSMSrmm has updated the control record in the recovered control data set to show that recovery was not successful.
16	DFSMSrmm encountered a severe error during a required communication with the DFSMSrmm subsystem. DFSMSrmm stops the utility.

Additional EDGBKUP return code information

If DFSMSrmm encounters an error during restore processing, DFSMSrmm completes the restore and sets a return code of 8 under these conditions:

- DFSMSrmm finds that the previous update of the control data set failed and the error was not corrected.
- DFSMSrmm cannot match the control data set and the journal. Journal copies used as input to forward recovery are out of sequence with other journal copies.
- DFSMSrmm can match the journal and the control data set, but the last set of journal records is incomplete. DFSMSrmm issues a message to indicate that forward recovery is completed to the point of the last complete set of journal records.
- DFSMSrmm detects discrepancies in the journal records used to perform a forward recovery.

For either a return code of 8 or 12, you should decide whether the restored control data set is acceptable, then take corrective actions. You can try to recover again by using the correct control data set and journal data sets. If you do not have any other backups or journals from which to restore, you will have to accept the restored control data set.

DFSMSrmm sets values in the control data set control record to indicate that the recovery did not complete satisfactorily. Until these values are reset, DFSMSrmm always prompts the operator at DFSMSrmm startup time to determine if the control data set can be used or not. To clear the values, run EDGUTIL VERIFY(ALL) to validate the recovered control data set. If processing completes successfully, DFSMSrmm resets the control data set control record and accepts the control data set for use without prompting the operator at future startup times.

If EDGUTIL finds errors in the restored control data set, correct the errors and run EDGUTIL again. Start DFSMSrmm and use the restored control data set while you issue RMM TSO subcommands. Correct the errors in the control data set by using RMM TSO subcommands or the EDGUTIL utility with the MEND parameter as described in [“Mending the control data set” on page 414](#) to update the information in the control data set.

Customizing the DSSOPT DD statement

You can use the DSSOPT DD statement to replace the DFSMSdss DUMP, COPY, and RESTORE commands that are used by DFSMSrmm. You might want to customize the operands based on the media that you used for the backup, the copy, or the resource that you have available.

Figure 152 on page 385 shows examples of the DFSMSdss DUMP, COPY, and RESTORE commands that DFSMSrmm issues. You can replace the second line of the commands that are shown in Figure 152 on page 385 with one or more DFSMSdss options. If you decide to change the commands, specify all the operands you want to have processed because DFSMSrmm uses your input in place of its own.

```
DUMP DS(INCLUDE(cds_name)) OUTDD(BACKUP) SHARE -
    COMPRESS CONCURRENT VALIDATE OPTIMIZE(1)

COPY DS(INCLUDE(cds_name)) [OUTDD(BACKUP)] SHARE -
    FR(PREF) CONCURRENT RENAMEU((*.CDS,*.COPYCDS)) REPLACEU

RESTORE DS(INCLUDE(**)) INDD(BACKUP) -
    REPLACE
```

Figure 152. DFSMSdss commands that are issued by DFSMSrmm

The command operands that you can specify in the DSSOPT DD statement are controlled and validated by DFSMSdss, not by DFSMSrmm. If you specify an unsupported command operand, DFSMSdss fails the operation.

The COPY command used by DFSMSrmm includes the OUTDD(BACKUP) keyword only if you have specified the BACKUP DD. When the OUTDD keyword is not specified, processing relies on SMS ACS processing being used to determine the target volume. You can override this processing by specifying the OUTDD keyword in DSSOPT or by specifying any other alternative DFSMSdss keyword that directs where the target copy is to be created.

The COPY command includes FR(PREF) combined with the CONCURRENT keyword. This enables the use of FlashCopy, concurrent or virtual concurrent copy if possible. If this is possible, a non-intrusive backup is performed to create a CDS copy. Otherwise, the backup continues, but CDS updates are prevented until the CDS copy completes. An alternative approach you can use with the DSSOPT DD is to cause the copy to fail if one of these methods cannot be used. For example:

```
FR(PREF) CONCURRENT(VIRTUALREQ) RENAMEU((*.CDS,*.COPYCDS,*.COPYCDS,*.COPYCDS)) REPLACEU
or
FR(PREF) CONCURRENT(ANYREQ) RENAMEU((*.CDS,*.COPYCDS,*.COPYCDS,*.COPYCDS)) REPLACEU
```

Do not specify FR(PREF) without the CONCURRENT keyword or the FR(REQ) keyword, because this causes processing to be intrusive.

The RENAMEU keyword on the COPY command specifies a filter mask (*.CDS) and renaming rule (*.COPYCDS). This changes the ending qualifier of the copied data set from 'CDS' to 'COPYCDS'. If your control data set name does not end in 'CDS' or you want to use a different renaming rule, you must use DSSOPT to override the second line of the command. See [z/OS DFSMSdss Storage Administration](#) for details of the use of RENAMEU.

The REPLACE keyword on the RESTORE command ensures that when you recover the DFSMSrmm control data set from a backup, any existing control data set is reused when possible or reallocated if necessary. If you are increasing the size of the control data set and have preallocated a larger control data set, specify the REPLACE keyword to restore to the preallocated data set.

The DSSOPT DD statement can be specified for dump, copy, and restore operations. DFSMSrmm reads all the records and uses them to replace its default command operands beginning at the second line. You can include comments in the DSSOPT records by using DFSMSdss conventions.

Example: Ensure that DFSMSdss does not compress the data and that the tape hardware is used to compress the records. This example uses EDGHSKP to back up to tape. To use EDGBKUP, the backup data sets must be allocated on DASD.

```
//EDGBKUP EXEC PGM=EDGHSKP,PARM='BACKUP(DSS)'
//SYSPRINT DD SYSOUT=*
//MESSAGE DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.CDS(+1),
// LABEL=(,SL)
//JRNLBKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.JRNL(+1),
// RECFM=VB,LRECL=9248,
// LABEL=(2,SL),VOL=REF=*.BACKUP
//DSSOPT DD *
```

```
CONCURRENT OPTIMIZE(4) VALIDATE
/*
```

Example: Rename the control data set during restore processing. The restored control data set is renamed when DFSMSdss renames each of the components of the backup copy of the original DFSMSrmm control data set. The example also shows how to restore the control data set and forward recover it to a volume different from the volume on which it was backed up. When the control data set is renamed during the restore, do not specify the MASTER DD statement unless it identifies the predefined target data set.

```
//EDGBKUP EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,UNIT=TAPE,DSN=BACKUP.CDS(0)
//JOURNAL DD DISP=SHR,DSN=BACKUP.JOURNAL(0)
// DD DISP=SHR,DSN=RMM.JOURNAL
//NEWVOL DD DISP=SHR,VOL=SER=MYVOLX
//DSSOPT DD *
/*RESTORE TO NEW VOLUME,AND
RENAME THE CDS */
OUTDD(NEWVOL)RENAMEU((*.CDS,*.NEWCDS))
```

Example: Create a copy of the control data set using fast replication. The copy of the control data set is created using a new data set name based on the current CDS dsname. The example ensures that fast replication is used so that an almost instant copy of the CDS can be created. No journal backup is created in this example. The example is run with DFSMSrmm active so that the MASTER DD is not required. The CDS data set name is obtained from the running DFSMSrmm subsystem. The data set name of the copy is created from the existing CDS data set name. The second qualifier of CDS is renamed to COPYCDS. The copy is almost instant, and this is a non-intrusive copy of the CDS. Updates to the CDS are allowed during backup.

```
//EDGBKUP EXEC PGM=EDGBKUP,PARM='BACKUP(COPY)'
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,UNIT=SYSDA,VOL=SER=myvol
//DSSOPT DD *
FR(PREF) CC(ANYREQ) RENAMEU((*.CDS.**,*.COPYCDS.**)) REPLACEU
/*
```

Backing up the control data set

Requirement: To use DFSMSdss for non-intrusive backup, set up the DFSMSdss concurrent copy environment or the virtual concurrent environment. You must have the hardware and software required to establish a concurrent copy session or a virtual concurrent copy session.

Use EDGBKUP to perform these tasks:

- Back up the control data set to a non-VSAM data set or a VSAM cluster that is described in [“Backing up the DFSMSrmm control data set and journal”](#) on page 387.
- Back up the journal described in [“Backing up the journal”](#) on page 373.
- Restore the control data set by returning a backup of the control data set and, optionally, forward recover it described in [“Restoring the control data set”](#) on page 389.
- Reorganize the control data set described in [“Reorganizing the control data set”](#) on page 393.
- Move the control data set and journal to different device types that are described in [“Moving the control data set and journal to a different device”](#) on page 397.

You can back up and restore the control data set using EDGBKUP with the DFSMSdss DUMP command and the RESTORE command or the access method services REPRO command. Use the DFSMSdss DUMP command with concurrent copy and the CONCURRENT option to perform a non-intrusive back up where DFSMSrmm allows updates to the DFSMSrmm control data set and journal. EDGBKUP links to IDCAMS or ADRDSSU to restore the control data set or back up the control data set. EDGBKUP links to IDCAMS to back up the journal data set.

You can see the commands and the messages that DFSMSrmm issues during processing in the SYSPRINT data set.

You cannot selectively copy, edit, or sort records in the DFSMSrmm journal data set or in backup copies because the records are in a format that only DFSMSrmm understands.

Backing up the DFSMSrmm control data set and journal

When DFSMSrmm is active, you do not need to specify the MASTER and JOURNAL DD statements. EDGBKUP obtains the name of the control data set and the journal from the DFSMSrmm subsystem. When the DFSMSrmm subsystem is stopped, quiesced, or when you want to back up a control data set that is not in use by DFSMSrmm, specify the MASTER DD statement and the JOURNAL DD statement. You decide which data sets are backed up. Specify the BACKUP DD to request control data set backup. Specify the JRNLBKUP DD to request journal backup. You can specify either or both of the BACKUP and JRNLBKUP DD statements. If you do not back up the journal before it is cleared, forward recovery from previous control data set backups is limited. EDGBKUP does not reset the journal after backing up the control data set. Use EDGHSKP to reset the journal after backing up the control data set.

Example: Back up the control data set by using EDGBKUP.

```
//EDGBKUP EXEC PGM=EDGBKUP,PARM='BACKUP'
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),UNIT=SYSALLDA,DSN=BACKUP.CDS(+1),
//        SPACE=(TRK,(ppp,sss),RLSE),RECFM=VB,LRECL=9216
//        BLKSIZE=0
```

Example: Back up the journal data set by using EDGBKUP.

```
//EDGBKUP EXEC PGM=EDGBKUP,PARM='BACKUP'
//SYSPRINT DD SYSOUT=*
//JRNLBKUP DD DISP=(,CATLG),UNIT=SYSALLDA,DSN=BACKUP.JRNL(+1),
//          SPACE=(TRK,(ppp,sss),RLSE),RECFM=VB,LRECL=9248
//          BLKSIZE=0,BUFNO=30
```

DFSMSrmm checks that any previous update of the control data set has completed successfully before backing up the DFSMSrmm control data set. DFSMSrmm does not back up the control data set when the previous update of the control data set fails and you are not using BACKUP(REORG). DFSMSrmm issues an informational message and sets a return code of 12.

To store backup copies of the control data set in a storage location to prepare for disaster recovery, you can place the backups on tape and define vital record specifications to move the backups off-site. When BACKUP(DSS) is used, you can back up directly to tape. When you use BACKUP(NREORG) or BACKUP(REORG), use a DASD data set for the initial backup copy. Then copy the backup copy to a tape data set in a subsequent job step or job. For disaster recovery, you can restore directly from the tape data set with the DFSMSrmm subsystem stopped or quiesced as long as you use the EDGBKUP utility. For on-site recovery, use a DASD backup for faster recovery.

Updating the active control data set

Use the EDGUPDT utility with the UPDATE parameter to update the active DFSMSrmm CDS with record updates created during testing or recovery. EDGUPDT uses a journal or journal backup or a concatenation of these data sets, which were created by DFSMSrmm running with the JRNLTRAN(YES) option in use. Using EDGUPDT ensures that, even after testing or recovery actions, you can ensure that the DFSMSrmm CDS reflects the actual content of the tape volumes in your library. The DFSMSrmm started procedure must be active when you run this utility.

When you prepare to run a test or recovery system with JRNLTRAN(YES), you would create a copy of the current production CDS – perhaps using the EDGHSKP/EDGBKUP utility with the BACKUP(COPY) parameter. Before you start DFSMSrmm on the test/recovery system create a new journal data set so that only the journal records created during the current test/recovery are available for use with EDGUPDT after all or part of testing/recovery is completed. You could, for example, backup the test/recovery system journal regularly and use the backup of the journal with EDGUPDT to maintain the production CDS. At the end of the test complete any required updates using the active test/recovery journal.

The EDGUPDT utility can only apply updates from the journal if the records have not been changed in the original CDS. Utility processing ensures that the CDS record has not been changed, deleted or added since the CDS copy was created.

If any errors occur when running EDGUPDT as a result of records being changed in the original CDS, processing continues but skips the update from the journal. Information messages are issued. At the end of processing a summary of record updates is written to the SYSPRINT file. If you rerun EDGUPDT using the same JOURNAL file to recover from a previous run error, you should expect an error message for each record that was successfully processed on the previous run.

DFSMSrmm processing ensures only that the volume and data set information in the CDS reflects processing in the test/recovery environment. DFSMSrmm does not make any changes to, nor validate that other, related system data sets are correct. For example:

TCDB

There is no consideration of whether a volume entry exists or not, or that the volume status remains in synch with the DFSMSrmm status of the volume.

UCAT

There is no consideration of whether data sets which were cataloged on the test/recovery system have corresponding catalog entries in the production environment.

Once your EDGUPDT processing is completed, you can optionally cross-check the DFSMSrmm CDS with related system data by:

1. Running EDGUTIL with PARM='VERIFY(SMSTAPE)'.

You can optionally limit the scope of the cross-checking by using the SYSIN DD statement to specify include and exclude conditions for the volumes to be processed.

2. Running EDGHSKP with PARM='VERIFY,CATSYNCH'.

Messages are issued during processing for data sets that are missing from the user catalogs, but defined as cataloged in DFSMSrmm.

The IDCAMS utility can be used to maintain either volume entries in the TCDB or data set catalog entries in the user catalogs.

JCL for EDGUPDT

This topic provides a JCL example to update the journal. [Figure 153 on page 388](#) shows the JCL for EDGUPDT.

```
//UPDATE EXEC PGM=EDGUPDT,PARM=UPDATE
//SYSPRINT DD SYSOUT=*
//JOURNAL DD DISP=SHR,DSN=TEST.SYSTEM.JRNL
//SYSIN DD *
UPDATE TARGETDATE(2009/123,12:30:00)
/*
```

Figure 153. JCL example for EDGUPDT

DD statements for EDGUPDT

The data sets that are used by the EDGUPDT utility are shown in [Table 53 on page 388](#):

Table 53. DFSMSrmm EDGUPDT data sets

DD Statement	Description
JOURNAL	Identifies the journal backups and journal to be used during update processing. When you concatenate multiple data sets, do so from oldest to newest.
SYSIN	Use the SYSIN file to select the journal records used for update of the CDS.
SYSPRINT	Contains the utility program messages issued when updating the DFSMSrmm control data set. This data set can be a SYSOUT file.

SYSIN file for EDGUPDT

Use this file to select the target point in time for update processing from the journal input records. Update processing starts at the first record in the JOURNAL. Update processing continues until the point in time you select, or to the end of the input records, whichever occurs first.

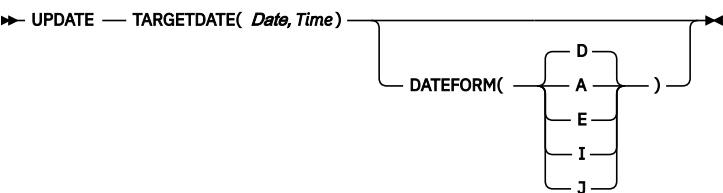


Figure 154. EDGUPDT SYSIN file

UPDATE

Use this command in the SYSIN file to select your chosen end point for journal update processing. When you do not specify UPDATE, DFSMSrmm uses all the available complete sets of records to perform update processing to the latest available point in time.

TARGETDATE

Use the TARGETDATE(*Date, Time*) operand to specify the date and time you have selected to be your end point in time for update processing. You must specify both a date and a time. The date and time are the local date and time as recorded in the journal records used for update processing. Any input records created at a time higher than the value you specify are skipped.

Date

Specify the date in the format set by the DATEFORM operand or (if the DATEFORM operand is not specified) by the DATEFORM parmlib value. For example, if your installation set DATEFORM(J), specify: TARGETDATE(2008/123,23:22:10.0)

Time

Specify the target local time in the format hh:mm:ss.t

The '.t' (tenths of a second) is optional.

DATEFORM

Use the optional DATEFORM (A | D | E | I | J) operand to specify the format for the date you enter in the TARGETDATE operand.

Value	Language	Format	Example
A	American	mm/dd/yyyy	12/15/2008
E	European	dd/mm/yyyy	15/12/2008
I	ISO	yyyy/mm/dd	2008/12/15
J	Julian	yyyy/ddd	2008/349
D	Default	Installation's default in EDGRMMxx	Initially set to Julian

The default date format for all date fields is the value specified in the parmlib member EDGRMMxx. The value is initially set to J for Julian. To change the date format for each run of EDGUPDT, use the DATEFORM parameter.

Restoring the control data set

This topic includes information about these topics:

- [“Controlling the control data set recovery point” on page 390](#)
- [“Restoring the control data set with forward recovery” on page 390](#)

- [“Restoring the control data set without forward recovery” on page 391](#)
- [“Forward recovering the control data set” on page 391](#)
- [“Restoring the control data set at a recovery site” on page 392](#)
- [“Using non-dfsmsrmm utilities to restore the control data set” on page 392](#)

Controlling the control data set recovery point

You can restore the DFSMSrmm control data set to any of these recovery points:

- The start time of the selected DFSMSrmm control data set backup.
- The end time of the selected DFSMSrmm control data set backup. Backups taken with DFSMSdss require the journal backup to have been taken at the same time as the DFSMSrmm control data set backup, otherwise the end time is no different than the start time.
- The end time of a journal backup.
- The current point in time, which can be achieved as long as you have an appropriate backup of the DFSMSrmm control data set and required backup copies of both the journal and the active journal. Also, the journal backups used must not include any time when the journal was disabled. If you have incomplete journal data because the journal was disabled or a journal backup is damaged or missing, the restored DFSMSrmm control data set might be inconsistent.

You can determine the DFSMSrmm control data set version that has actually been recovered by using information from runs of the EDGBKUP utility during forward recovery and from information in the DFSMSrmm control data set control record.

When you restore a DFSMSrmm control data set with forward recovery, use the details in message EDG6431I to identify the DFSMSrmm control data set time stamp and the time stamp of the journal records used for forward recovery. For example:

```
EDG6431I THE CONTROL DATA SET TIMESTAMPED 2001/240 06:04:47 WAS FORWARD RECOVERED FROM JOURNAL
RECORDS BETWEEN 2001/240
EDG6431I CONT:- 06:04:47 AND 2001/241 04:01:12
```

If you no longer have the EDG6431I message or you did not use forward recovery, you can use the RMM LISTCONTROL subcommand to find the time stamp. In this case, restart DFSMSrmm after your recovery, but before you start tape processing or update records in the DFSMSrmm control data set. Use the RMM LISTCONTROL CNTL subcommand to display the update date and update time values which indicate your recovery point time stamp

Restoring the control data set with forward recovery

Figure 155 on page 390 shows the JCL for restoring the DFSMSrmm control data set and for forward recovering the DFSMSrmm control data set. You can restore the control data set directly from tape by using the EDGBKUP utility when DFSMSrmm is stopped or quiesced. You do not need to use the EDGRESET utility or the MODIFY command to reset DFSMSrmm. EDGBKUP uses either the DFSMSdss RESTORE command or the AMS REPRO utility to restore the control data set based on the contents of the control data set backup.

```
//EDGBKUP EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.CDS
//*
//BACKUP DD DISP=SHR,DSN=BACKUP.CDS(0)
//*
//JOURNAL DD DISP=SHR,DSN=BACKUP.JRNL(0)
// DD DISP=SHR,DSN=RMM.JOURNAL
```

Figure 155. Restoring the control data set with forward recovery

You must specify the MASTER DD statement to restore from an AMS REPRO backup and to forward recover an existing control data set.

The MASTER DD is optional when you restore the control data set from a DFSMSDss backup. In this way, you do not have to pre-allocate the control data set, and you can optionally restore to a control data set by using a different data set name. After the DFSMSDss restore is completed, DFSMSrmm dynamically allocates the MASTER file to the restored data set prior to starting forward recovery. If the MASTER DD statement is specified, the data set name must match the data set name that DFSMSDss restores.

You can restore the DFSMSrmm control data set and forward recover the restored control data set with journal records. Use the JOURNAL DD statement to concatenate multiple journal data sets. If you forward recover using multiple journal data sets, concatenate the journal data sets in the order in which changes were originally made. The first journal data set in the concatenation must match the control data set to be forward recovered. For control data set backups that are taken with AMS REPRO, DFSMSrmm considers the matching journal to be the one that contains records that are created after the journal was cleared. For control data set backups that are taken with DFSMSDss, DFSMSrmm considers the matching journal to be the journal in use at the time the backup is taken.

Example: The example shows how to restore from the second generation backup. Concatenate journal data sets for back up using AMS REPRO.

```
//BACKUP DD DISP=SHR,DSN=BACKUP.CDS(-2)
//JOURNAL DD DISP=SHR,DSN=BACKUP.JOURNAL(-1)
// DD DISP=SHR,DSN=BACKUP.JOURNAL(0)
// DD DISP=SHR,DSN=RMM.JOURNAL
```

Example: The example shows how to restore from the second generation backup. Concatenate journal data sets for back up using DFSMSDss.

```
//BACKUP DD DISP=SHR,DSN=BACKUP.CDS(-2)
//JOURNAL DD DISP=SHR,DSN=BACKUP.JOURNAL(-2)
// DD DISP=SHR,DSN=BACKUP.JOURNAL(-1)
// DD DISP=SHR,DSN=BACKUP.JOURNAL(0)
// DD DISP=SHR,DSN=RMM.JOURNAL
```

When backup is taken using AMS REPRO and you restore the control data set from the latest control data set backup, use the active journal to forward recover to the latest point in time.

When backup is taken using BACKUP(DSS) and you restore from the latest control data set backup, both the latest journal backup and the active journal must be used for forward recovery. For information about restoring the control data set at a recovery site, see [“Restoring the control data set at a recovery site” on page 392](#).

Restoring the control data set without forward recovery

To restore the control data set without forward recovery, do not specify a JOURNAL DD statement. EDGBKUP checks the contents of the control data set backup and calls the correct utility to restore the control data set. The MASTER DD statement is optional when you have used DFSMSDss in the DFSMSrmm utilities to create the backup copy and you do not need to change the name of the DFSMSrmm control data set.

Example: To restore the control data set without forward recovery:

```
//RESTORE EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,DSN=RMM.BACKUP.CDS
//MASTER DD DISP=SHR,DSN=RMM.CDS
```

Forward recovering the control data set

You can forward recover the control data set after you have restored the control data set in a previous step.

Example: To forward recover the control data set by using EDGBKUP:

```
//FWDRECVR EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
```



```
//MASTER DD DISP=SHR,DSN=RMM.CDS
//JOURNAL DD DISP=SHR,DSN=RMM.JOURNAL
```

When you do not provide the BACKUP DD statement, DFSMSrmm does not restore the control data set. DFSMSrmm assumes that the data set identified by the MASTER DD statement identifies the correct control data set, and EDGBKUP uses data from the journal to forward recover the control data set.

You can use the JOURNAL DD statement to concatenate multiple journal data sets. See [“Restoring the control data set with forward recovery” on page 390](#) for information about concatenating journal data for recovery. You can forward recover a restored or unrestored control data set. For example, if automatic forward recovery fails, you could attempt forward recovery by using the active journal. If you have already restored the control data set but have not used journal data for forward recovery, run forward recovery as a second step. The DFSMSrmm subsystem must be stopped or quiesced during forward recovery of the active control data set.

Restoring the control data set at a recovery site

Before you start the DFRMM procedure, use the EDGBKUP utility to restore from the latest control data set backup. You can restore from either tape or DASD. When you restore from tape, even though the DFRMM procedure is not started, DFSMSrmm verifies that you are restoring a backup of the DFSMSrmm control data set and allows you to use EDGBKUP with DFSMSrmm inactive. Any backup of the control data set is a backup consistent with the time that you started the backup. You can restore that backup copy to your selected point in time, or can optionally forward recover to the end of the DFSMSrmm backup processing or some later time as long as you have the journal backups and journal data set. Usually you will not use forward recovery. To restore the control data set at a recovery site, follow these steps:

1. If you used DFSMSrmm to create the backup by using IDCAMS REPRO, allocate a new control data set. You can use the DFSMSrmm-supplied sample job EDGJMFAL to allocate a new control data set.
2. Run EDGBKUP with the RESTORE parameter to restore the control data set from a backup copy. The MASTER DD statement is optional if you used DFSMSrmm to create the backup with DFSMSdss.

```
//RESTORE EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,DSN=RMM.BACKUP.CDS
//MASTER DD DISP=SHR,DSN=RMM.CDS
```

3. If you want to forward recover to a later point in time, run EDGBKUP with the RESTORE parameter and include the JOURNAL DD statement to identify the journals to be applied.

```
//RESTORE EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.CDS
//JOURNAL DD DISP=SHR,DSN=RMM.JOURNAL.BACKUP(0)
```

This step can be combined with Step 2 to restore the control data set and journal at the same time:

```
//RESTORE EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,DSN=RMM.BACKUP.CDS
//MASTER DD DISP=SHR,DSN=RMM.CDS
//JOURNAL DD DISP=SHR,DSN=RMM.JOURNAL.BACKUP(0)
```

The MASTER DD statement is optional if you used DFSMSrmm to create the backup with DFSMSdss.

4. Allocate an empty journal data set. DFSMSrmm provides sample EDGJNLAL that you can use to allocate the journal.
5. Start DFRMM with a DFSMSrmm EDGRMMxx parmlib member that names the restored control data set and the empty journal.

Using non-dfsmsrmm utilities to restore the control data set

Recommendation: Use the DFSMSrmm EDGBKUP utility to restore the control data set to get the benefits that the DFSMSrmm utility provides and to avoid replying to operator messages if DFSMSrmm is not active.

You can use IDCAMS REPRO or DFSMSDss RESTORE to recover the DFSMSrmm control data set from a backup copy taken using those utilities. This topic contains examples for you to use IDCAMS and DFSMSDss.

If you use IDCAMS or DFSMSDss to restore the control data set from a backup, you must forward recover the control data set before it can be used by DFSMSrmm. If you do not perform forward recovery and start DFSMSrmm, you will have to reply to message EDG0123D. To avoid this, forward recover the control data set by using the EDGBKUP utility. You can use a dummy journal if you do not have an existing journal or journal backups. When forward recovery is completed, you are ready to use the control data set with DFSMSrmm.

Example: Restore the control data set by using IDCAMS.

```
//RESTORE EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,DSN=RMM.BACKUP.CDS
//MASTER DD DISP=SHR,DSN=RMM.CDS
//SYSIN DD *
REPRO INFILE(BACKUP) OUTFILE(MASTER)
/*
```

Example: Restore the control data set using DFSMSDss.

```
//RESTORE EXEC PGM=ADRDSSU
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=SHR,DSN=RMM.BACKUP.CDS
//SYSIN DD *
RESTORE DS(INCLUDE(**)) INDD(BACKUP) REPLACE
/*
```

Example: Forward recover the control data set using a dummy journal.

```
//FWDRECVR EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.CDS
//JOURNAL DD DUMMY
```

Example: CDS copy followed later by a forward recovery from that CDS copy.

1. Create almost instant copy of the current, active DFSMSrmm CDS

```
//CDSCOPY EXEC PGM=EDGHSKP,PARM='BACKUP(COPY)'
//BACKUP DD UNIT=3390,VOL=SER=RMMPK2,DISP=SHR
//SYSPRINT DD SYSOUT=*
//DSSOPT DD *
FR(PREF) CONCURRENT RENAMEU((*.*.MASTERX,*.*.R10COPY)) REPLACE /*
```

2. DFSMSrmm processing continues until some point where a recovery is needed.
3. Forward recover the CDS copy using the last active DFSMSrmm journal:

```
//FORW EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=RMMTST.Z1ASMF.R10COPY,DISP=SHR
//JOURNAL DD DSN=RMMTST.Z1ASMF.JOURNAL,DISP=SHR
```

4. Start DFSMSrmm using a parmlib member that points to the recovered CDS copy.

```
S DFRMM,M=RC
```

using parmlib member EDGRMMRC

Reorganizing the control data set

Use EDGBKUP to reorganize the control data set. The DFSMSrmm subsystem must be stopped or quiesced when you reorganize the active control data set. DFSMSrmm reorganizes the control data set by first backing up the control data set and optionally the journal. Then DFSMSrmm restores the control data

set from the backup control data set. For more information, see the topic about [Reclaiming CA Space for a KSDS in z/OS DFSMS Using Data Sets](#).

Do not use the BACKUP(DSS) parameter to reorganize the control data set. If you use BACKUP(DSS) to regularly back up for recovery purposes, use the EDGBKUP utility with the BACKUP(REORG) parameter with care. Although DFSMSrmm can determine which utility to use for recovery from any backup taken using an DFSMSrmm utility, any backup taken during BACKUP(REORG) might impact the cycles that have been maintained of regular backup copies.

Example: Reorganize the control data set without backing up the journal.

```
//REORG      EXEC PGM=EDGBKUP,PARM=' BACKUP(REORG) '  
//SYSPRINT  DD   SYSOUT=*  
//BACKUP    DD   DISP=(,CATLG),DSN=RMM.BACKUP.CDS(+1),UNIT=SYSALLDA,  
//          SPACE=(CYL,(ppp,sss),RLSE),RECFM=VB,LRECL=9216,  
//          BLKSIZE=0  
//MASTER   DD   DISP=OLD,DSN=RMM.CDS
```

Example: Reorganize the control data set and back up the journal. DFSMSrmm backs up the journal before attempting to restore the control data set because the JRNLBKUP DD and the JOURNAL DD statement are specified in this example.

```
//REORG      EXEC PGM=EDGBKUP,PARM=' BACKUP(REORG) '  
//SYSPRINT  DD   SYSOUT=*  
//BACKUP    DD   DISP=(,CATLG),DSN=RMM.BACKUP.CDS(+1),UNIT=SYSALLDA,  
//          SPACE=(CYL,(ppp,sss),RLSE),RECFM=VB,LRECL=9216  
//          BLKSIZE=0  
//MASTER   DD   DISP=OLD,DSN=RMM.CDS  
//JRNLBKUP  DD   DISP=(,CATLG),DSN=RMM.BACKUP.JOURNAL(+1),UNIT=SYSALLDA,  
//          SPACE=(CYL,(ppp,sss),RLSE),RECFM=VB,LRECL=9248,  
//          BLKSIZE=0,BUFNO=30  
//JOURNAL   DD   DISP=OLD,DSN=RMM.JOURNAL,BUFNO=30
```

These examples show how you can reorganize your current, or in use, control data set. This approach requires that the DFSMSrmm started task is stopped and a backup and restore operation is to be processed. Another approach to reorganizing the control data set is to allocate a new control data set and ensure that the control data set contents are reorganized as part of the copy to the new data set. See “Steps for moving the control data set and journal using the DFSMSrmm EDGHSKP utility with the PARM='BACKUP' parameter” on page 397 for a process you can follow that also reorganizes the control data set contents. Be sure to remember that only a control data set backup using EDGHSKP with the BACKUP or BACKUP(AMS) parameter, or EDGBKUP with the BACKUP, BACKUP(NREORG), or BACKUP(REORG) parameter, reorganizes the control data set contents.

You can significantly reduce the need to reorganize the control data set by enabling the CA reclaim function for it. For more information, see the topic about [Reclaiming CA Space for a KSDS in z/OS DFSMS Using Data Sets](#).

Monitoring the space used by the control data set

See “Step 12: Creating the DFSMSrmm control data set” on page 48 for information on creating the DFSMSrmm control data set, including information on how to calculate DASD space for the control data set, placement of the control data set, and allocating space for the control data set. We recommend that you ensure that CA Reclaim is not disabled and that you allocate the DFSMSrmm control data set with enough secondary space, so that the DFSMSrmm control data set can grow as you add new resources such as volumes and data sets. Also, allow for enough planned free space in the DFSMSrmm control data set to allow for growth and any known new-volume requirements. However, at some time, the allocated space in the DFSMSrmm control data set will fill up and new extents are allocated and used as long as there is free DASD space available. The DFSMSrmm control data set can grow in size within the limits imposed by VSAM and DFSMS.

Use the LISTCAT output from IDCAMS to check the details of the DFSMSrmm control data set and the RMM LISTCONTROL CNTL subcommand to list the calculated percentage used for the DFSMSrmm control data set. DFSMSrmm calculates the percentage of the DFSMSrmm control data set used by checking the High Used RBA and High Allocated RBA that the LISTCAT output shows. This percentage does not take

into account any available embedded free space, nor does it take into account the number of used extents or the additional free space that might be available to extend the DFSMSrmm control data set.

Decide how you will monitor and report on the DFSMSrmm control data set space used. By using the information available to you, including your predicted tape usage, decide if the DFSMSrmm control data set is large enough or needs to be reallocated. Decide on your own thresholds, and check regularly to ensure they are used to trigger corrective action. Although you can reorganize the DFSMSrmm control data set to recover free space, do not do this regularly. Ensure your DFSMSrmm control data set is large enough and can grow as required to avoid the need to reorganize it regularly.

Changing the size of the control data set and journal

To change the size of the DFSMSrmm control data set or journal (either to make the data sets larger or smaller), use the procedures documented in this topic. The basic procedure is to use backup and recovery, but the method selected depends on whether you have a new volume available for the new control data set. If you use the existing volume and must delete the existing data set to allocate one of a new size, follow the procedure documented in [“Backing up the DFSMSrmm control data set and journal” on page 387](#) and [“Restoring the control data set with forward recovery” on page 390](#). If you use a new volume, follow the procedure documented in [“Moving the control data set and journal to a different device” on page 397](#). When you allocate the new data sets, make them the required size by either increasing or decreasing the size based on your requirements.

Recovering from control data set update failures

Information about a volume in the DFSMSrmm control data set is stored in several records. A change in a volume's status can require updating several records. Since an update of multiple records takes a certain amount of time, it is possible that external events, such as a power outage, could interrupt the updating process. An interruption can leave some records that reflect the volume's old status and some its new and the control data set is then considered corrupt. It might be impossible to subsequently change the volume's status. This situation is referred to as a *multi-record update failure*.

DFSMSrmm detects a multi-record update failure at DFSMSrmm address-space start-up time when the control data set is not shared between systems. When a control data set is shared between multiple systems, the multi-record update failure can be detected at any time. A hardware failure on any of the systems could be detected by another system at the next I/O to the control data set. In this case, a SYSTEM RESET of the failing system releases the control data set reserve, and the other systems sharing that control data set, find they had experienced a multi-record update failure.

Recovery processing

The first request for I/O to the control data set, following a multi-record update failure, detects the failure, and recovery processing begins. Other attempts to access the control data set are queued behind the current request until either manual recovery is required or automatic recovery is successful.

When a multi-record update failure is first detected, DFSMSrmm attempts automatic recovery processing which requires no operator intervention. If a journal data set matching the corrupt control data set is found, DFSMSrmm issues message EDG2111I to notify the operator that automatic recovery processing is starting. DFSMSrmm issues message EDG2115I if automatic recovery is not possible and manual recovery is required. Manual recovery might be needed under these conditions:

- The journal data set is not defined in the initialization parameters
- The journal and control data sets do not match
- The journal data set has been disabled in response to message EDG2103D
- The journal data set update has been ignored in response to message EDG2103D

Manual recovery requires operator intervention. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for operator procedures for responding to messages DFSMSrmm issues during recovery processing.

Recommendation: Use EDGHSKP with the CATSYNCH parameter to synchronize catalogs after recovery.

Handling I/O requests following a failure

After recovery is completed, I/O requests for any tape processing activity and DFSMSShsm tape volume release activity are retried.

Automatic recovery

Once automatic recovery is successful the first request continues as normal, and the queued requests are processed.

Manual recovery

When manual recovery is required, each subsystem request that results in I/O to the control data set fails. For requests that can be retried, DFSMSrmm issues WTOR EDG4001D or EDG8008D.

When you perform manual recovery because the control data set is no longer valid, restore the control data set using the most recent backup copy of the control data set. Forward recover the control data set to the point of failure by using the latest control data set backup and the applicable journal backups that are concatenated with the active journal. See [“Restoring the control data set with forward recovery” on page 390](#) for information. If manual recovery is required because the control data set is full or for some other reason where the control data set information is valid except for the most recent failed record updates, use the current control data set for recovery rather than a backup copy. When you use the current control data set for recovery, you do not need to perform forward recovery because DFSMSrmm restart performs automatic forward recovery. Also do not run the EDGUTIL utility against the control data set before you resume DFSMSrmm operation.

Use one of these methods to perform manual recovery by using the current control data set.

- Run the EDGBKUP utility with PARM='RESTORE' to use the active journal to forward recover the current DFSMSrmm control data set. If processing is successful, the control data set information is correct. See [“Forward recovering the control data set” on page 391](#).
- Run the EDGBKUP utility with PARM='BACKUP(REORG)' to reorganize the current control data set and reclaim enough free space to enable processing to continue. See [“Reorganizing the control data set” on page 393](#). To correct the control data set information, refresh the DFSMSrmm started task, and attempt automatic forward recovery.
- Use IDCAMS REPRO to copy the contents of the current control data set to a new, larger control data set. Create a new EDGRMMxx parmlib member to define data set name for the new control data set. To correct the control data set information, refresh the DFSMSrmm started task, and attempt automatic forward recovery.
- Run EDGBKUP with PARM='RESTORE' to use the latest control data set backup, and the applicable journal backups that are concatenated with the active journal to forward recover. If processing is successful, the control data set information is correct. See [“Restoring the control data set with forward recovery” on page 390](#).

Manual recovery requires operator intervention. See *z/OS DFSMSrmm Managing and Using Removable Media* for operator procedures for information about stopping, quiescing, and restarting the DFSMSrmm subsystem. After manual recovery completes, the operator can reply 'RETRY' or 'CANCEL' to the EDG4001D and EDG8008D WTORs. These requests fail and must be rerun or reissued after manual recovery:

- Inventory management in progress
- DFSMSrmm utilities EDGINERS and EDGUTIL
- RMM TSO subcommands

Moving the control data set and journal to a different device

Before You Begin: See “Step 19: Starting DFSMSrmm” on page 60 for information about the stopping or quiescing of DFSMSrmm to allow processing for restoring or reorganizing the control data set.

To move the control data set or journal to a different device, use DFSMSrmm utilities. For other DFSMSrmm data sets, such as the extract data set, use the existing storage management techniques that you are familiar with to move them.

These topics provide examples for moving the control data set and journal data set that use both DFSMSrmm utilities and non-DFSMSrmm techniques. If you want to move the control data set and not the journal, modify these examples as follows:

1. Do not allocate a new journal data set.
2. Only alter the control data set name. Do not alter the journal name or change the journal name in the DFSMSrmm EDGRMMxx parmlib.

Do not change the restore step to ensure that the restore process includes forward recovery from the journal records.

Steps for moving the control data set and journal using the DFSMSrmm EDGHSKP utility with the PARM='BACKUP' parameter

Perform these steps to move your control data set and journal to a different device by using the EDGHSKP utility with the PARM='BACKUP' parameter.

1. Allocate a new control data set and journal. Refer to “Step 12: Creating the DFSMSrmm control data set” on page 48 for information on allocating the control data set. Now is a good time to consider whether you should allocate the control data set as an extended format (EF) data set and to optionally set the extended addressability (EA) attribute so that the control data set can grow beyond 4GB in size.
2. Back up the control data set and the journal.

Example: With DFSMSrmm active, run EDGHSKP,PARM='BACKUP' to back up the control data set and the journal and to clear the current journal.

```
// EXEC PGM=EDGHSKP,PARM='BACKUP'  
//MESSAGE DD DISP=SHR,DSN=messages  
//SYSPRINT DD SYSOUT=*  
//BACKUP DD DISP=(,CATLG),DSN=cds backup(+1),UNIT=SYSALLDA,  
// RECFM=VB,LRECL=9216,BLKSIZE=0  
//JRNLBKUP DD DISP=(,CATLG),DSN= journal backup(+1),UNIT=SYSALLDA,  
// RECFM=VB,LRECL=9248,BLKSIZE=0,BUFNO=30
```

This clears the current journal data set. Some records might be written to the journal if any updates are made to the control data set before the DFSMSrmm procedure is stopped. This is not a problem because the restore operation will use them in forward recovery.

3. Stop or quiesce the DFSMSrmm procedure to prevent any further updates to the control data set during recovery.
4. Restore the backup of the control data set to the new control data set allocated in step “1” on page 397.

Example: This JCL example puts the latest DFSMSrmm information into the new control data set and uses the old journal to forward recover the control data set backup to the point where you stopped the DFSMSrmm procedure.

```
// EXEC PGM=EDGBKUP,PARM='RESTORE'  
//SYSPRINT DD SYSOUT=*  
//BACKUP DD DISP=SHR,DSN=cds backup(0)  
//MASTER DD DISP=SHR,DSN=new control data set  
//JOURNAL DD DISP=SHR,DSN=old journal
```

Note: Steps 2, 3, and 4 can be optimized to minimize DFSMSrmm downtime and to reorganize the control data set in less time. To optimize the DFSMSrmm downtime, use these steps:

- a. Back up the control data set and the journal. For example, with DFSMSrmm active, run EDGHSKP,PARM='BACKUP' to back up the control data set and the journal and to clear the current journal.

```
// EXEC PGM=EDGHSKP,PARM='BACKUP'  
//MESSAGE DD DISP=SHR,DSN=messages  
//SYSPRINT DD SYSOUT=*  
//BACKUP DD DISP=SHR,DSN=new control data set  
//JRNLBKUP DD DISP=(,CATLG),DSN= journal backup(+1),UNIT=SYSALLDA  
// RECFM=VB,LRECL=9248,BLKSIZE=0,BUFNO=30
```

This copies the control data set to the new control data set and reorganizes the records and clears the current journal data set. Once the backup is completed, the new control data set is almost ready for use by DFSMSrmm. Some records might be written to the journal if any updates are made to the control data set before the DFSMSrmm procedure is stopped. This is not a problem because the forward recovery operation will use them.

- b. Stop or quiesce the DFSMSrmm procedure to prevent any further updates to the control data set during recovery.
- c. Forward recover the new control data set from step **b**. For example, this JCL example uses the old journal to forward recover the new control data set to the point where you stopped the DFSMSrmm procedure.

```
// EXEC PGM=EDGBKUP,PARM='RESTORE'  
//SYSPRINT DD SYSOUT=*  
//MASTER DD DISP=SHR,DSN=new control data set  
//JOURNAL DD DISP=SHR,DSN=old journal
```

5. Implement the new data sets by using one of these techniques:

- Use IDCAMS ALTER command to rename the new control data set and new journal, after renaming the old data sets, or
- Create a new EDGRMMxx parmlib member with the new names, or
- Update the current parmlib member to include the names of the new control data set and journal.

6. If you stopped DFSMSrmm, start the DFSMSrmm procedure, using the updated parmlib member or the new parmlib member. If you quiesced DFSMSrmm, use the MODIFY command to specify the parmlib member suffix to be used.

If you are keeping multiple control data set and journal backups for error recovery situations, perform step “2” on page 397 again to backup the control data set and journal.

Recommendation: Back up the new data sets now to avoid keeping the old journal for recovery. As your backup copies are created in the future, your requirement for the old journal will be eliminated.

You are done when you have successfully moved the control data set and journal.

Steps for moving the control data set and journal using DFSMSrmm utility EDGHSKP utility with the PARM='BACKUP(DSS)' parameter

Before you begin: Use the BACKUP(DSS) option on a concurrent copy capable device to enable DFSMSrmm to continue to process requests while the backup is taken.

Perform these steps to move your control data set and journal to a different device.

1. With DFSMSrmm active, back up both the control data set and the journal.

Example: Run EDGHSKP,PARM='BACKUP(DSS)' to back up the control data set and the journal and to clear the current journal. Use the BACKUP(DSS) option on a concurrent copy capable device to enable DFSMSrmm to continue processing requests while the backup is taken. DFSMSrmm might write some records to the journal if any updates are made to the control data set before the DFSMSrmm procedure is stopped. This is not a problem as the restore operation will use them in forward recovery.

```
//EXEC PGM=EDGHSKP,PARM='BACKUP(DSS)'  
//MESSAGE DD DISP=SHR,DSN=messages
```



```
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),DSN=cds_backup(+1),UNIT=SYSALLDA
//JRNLBKUP DD DISP=(,CATLG),DSN=journal_backup(+1),UNIT=SYSALLDA
```

2. Stop or quiesce the DFSMSrmm procedure to prevent any further updates to the control data set during recovery.
3. Restore the backup of the control data set to the new control data set. If the BACKUP(DSS) option is used, you can use the DSSOPT DD statement to specify the new control data set data set name. This puts the latest DFSMSrmm information into the new control data set and uses the old journal to forward recover the control data set backup to the point when the you stopped the DFSMSrmm procedure.

This JCL example restores the back up to a different device using a new data set name:

```
//MOVECDS EXEC PGM=EDGBKUP,PARM='RESTORE'
//SYSPRINT DD SYSOUT=*
//DSSOPT DD *
//RENAMEU(*.CDS,*.NEWCDS) OUTDYNAM(SHRPK2) NULLSTORCLAS BYPASSACS(*)
//BACKUP DD DISP=SHR,DSN=cds_backup(0)
//JOURNAL DD DISP=SHR,DSN=journal_backup(0)
//DD DISP=SHR,DSN=old_journal
```

4. Implement the new data sets by using one of these techniques:
 - Use IDCAMS ALTER command to rename the new control data set and journal after you rename the old data sets.
 - Create a new EDGRMMxx parmlib member with the new journal names and control data set names.
 - Update the current parmlib member to include the names of the new control data set and journal.
5. If you stopped DFSMSrmm, start the DFSMSrmm procedure, using the updated parmlib member or the new parmlib member. If you quiesced DFSMSrmm, use the MODIFY command to specify the parmlib member suffix to be used.

If you are keeping multiple control data set and journal backups for error recovery situations, perform step “1” on page 398 again to backup the control data set and journal.

Recommendation: Back up the new data sets now to avoid the requirement to keep the old journal for recovery. As your backup copies are created in the future, your requirement for the old journal will be eliminated.

You are done when you have successfully moved the control data set and journal.

Moving the journal using DFSMSrmm utilities

The journal cannot, strictly speaking, be moved. You move it by allocating a new journal data set and later deleting the old one.

If you tried to copy the old journal into a new journal, when DFSMSrmm is started, the journal data set will be re-initialized and be overwritten. Therefore, it is critical that you backup your journal using EDGBKUP with DFSMSrmm stopped or quiesced in the event that you need the journal for forward recovery.

To move your journal to a different device, follow this procedure:

1. Allocate a new journal data set.
2. Stop or quiesce the DFSMSrmm procedure to prevent any further updates to the control data set and journal.
3. With DFSMSrmm stopped or quiesced, run EDGBKUP,PARM='BACKUP' to backup both the control data set and journal as shown in [Figure 156 on page 400](#).

```
// EXEC PGM=EDGBKUP,PARM=' BACKUP '
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=cds name
//JOURNAL DD DISP=SHR,DSN=journal name
//BACKUP DD DISP=(,CATLG),DSN=cds backup(+1),UNIT=SYSALLDA,
//        SPACE=(CYL,(ppp,sss),RLSE),RECFM=VB,LRECL=9216
//JRNLBKUP DD DISP=(,CATLG),DSN= journal backup(+1),UNIT=SYSALLDA,
//        SPACE=(CYL,(ppp,sss),RLSE),RECFM=VB,LRECL=9248
```

Figure 156. JCL example for backing up the control data set and journal

This provides you with a valid point-in-time backup of the control data set and the old journal data set. After this step, if you need to recover the control data set, you can use this control data set backup, and the new journal data set.

Optionally you can back up just the journal by removing the BACKUP DD statement. Because you are only moving the journal, you only need to back up the journal.

4. Implement the journal by using one of these techniques:

- Use IDCAMS ALTER command to rename the new journal, having first renamed the old data set, or
- Create a new EDGRMMxx parmlib member, or update the current parmlib member to include the name of the new journal.

5. If DFSMSrmm was stopped, start the DFSMSrmm procedure, using the current, updated, or the new parmlib member. If DFSMSrmm was quiesced, use the MODIFY command to specify the parmlib member suffix to be used. Doing so, uses the unmovd control data set and the newly allocated journal.

6. Delete the old journal.

Steps for moving the control data set using non-dfsmsrmm utilities

Perform these steps to move your control data set to a different device by using non-DFSMSrmm utilities such as AMS REPRO and EXPORT/IMPORT, or DFSMSdss ADRDSSU.

1. Allocate a new control data set only if you are planning to use AMS REPRO.
2. Stop or quiesce the DFSMSrmm procedure to prevent any further updates to the control data set during recovery.
3. Copy the old control data set to the new control data set by using one of these utilities:
 - AMS REPRO.
 - AMS EXPORT followed by IMPORT.
 - DFSMSdss ADRDSSU utility.

Here is an example of copying the old control data set to the new control data set by using the DFSMSdss ADRDSSU utility.

```
//COPYCDS EXEC PGM=ADRDSSU,REGION=8M
//SYSPRINT DD SYSOUT=*
//SYSIN DD *
COPY -
LOGINDYNAM (SHRPK3) -
DS(INC(RMM.CDS)) -
OUTDYNAM(SHRPK2) -
SPHERE -
RENAMEU(*.CDS,*.NEWCDS)
```

4. Implement the data sets by using one of these techniques:

- Use IDCAMS ALTER command to rename the new control data set, having first renamed the old data set, or
- Create a new EDGRMMxx parmlib member, or
- Update the current parmlib member to include the name of the new control data set.

5. If you stopped DFSMSrmm, start the DFSMSrmm procedure, using the updated parmlib member or the new parmlib member. If you quiesced DFSMSrmm, use the MODIFY command to specify the parmlib member suffix to be used. This will use the moved control data set and the unmoved journal.

Using EDGUTIL for tasks such as creating and verifying the control data set

Use EDGUTIL to perform these tasks:

- Create the control data set control record in an empty VSAM data set described in [“Creating or updating the control data set control record”](#) on page 409.
- Update an existing control data set control record described in [“Creating or updating the control data set control record”](#) on page 409.
- Verify control data set information to diagnose errors in the control data set described in [“Verifying the contents of the control data set”](#) on page 411.
- Check that control data set information about a volume's status and the library in which it resides are consistent with the TCDB information, and optionally the library manager database information described in [“Verifying the control data set and tape configuration database”](#) on page 413. You can optionally generate control statements to enable mismatches to be synchronized at a later time using the EDGSPLCS utility. The control statements always include the "Q" verify request option to enable serialization of the return to scratch processing performed by EDGSPLCS.
- Synchronize the TCDB information and library manager information for system-managed volumes with information in the DFSMSrmm control data set that is described in [“Synchronizing the contents of the control data set”](#) on page 413
- Detect consistency errors in control data set information created during conversion activities and fix the errors that were detected. Run the mend function against a VSAM copy of the control data set as the first step in fixing a production control data set. The mend function should only be used with guidance from IBM Support Center to determine the underlying cause of any errors in the control data set. See [“Mending the control data set”](#) on page 414 for restrictions that you should be aware of when you use the mend function.
- Enable selected functions:
 - [“Setting up DFSMSrmm stacked volume support”](#) on page 415
 - [“Enabling extended bin support”](#) on page 417

You can run multiple copies of EDGUTIL (on the same system or on different systems), and each copy can process a different subset of volumes. If you use each copy to process a different subset of volumes, you can potentially reduce the time to complete processing of all volumes. See [“SYSIN file for VERIFY and MEND processing”](#) on page 406 for additional information on running multiple copies of the EDGUTIL utility.

JCL for EDGUTIL

This topic provides JCL for creating the DFSMSrmm control data set, verifying the contents of the control data set, updating the control data set, and mending the control data set. The MASTER DD statement is required.

JCL for creating the control data set

The JCL shown in [Figure 157](#) on page 402 can be used to create a control data set with the name MYCDS.

```
//EDGUTIL EXEC PGM=EDGUTIL,PARM='CREATE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=RMM.CONTROL.DSET,DISP=SHR
//SYSIN DD *
CONTROL CDSID(MYCDS)
/*
```

Figure 157. Creating the control data set

JCL for updating the control data set

Use the sample JCL in [Figure 158 on page 402](#) to update the control data set with the name MYCDS.

```
//UTIL EXEC PGM=EDGUTIL,PARM='UPDATE'
//SYSPRINT DD SYSOUT=*
//MASTER DD DISP=SHR,DSN=RMM.CONTROL.DSET
//SYSIN DD *
CONTROL CDSID(MYCDS) RACKFREE(1234)
/*
```

Figure 158. Updating the control data set

JCL for verifying the contents of the control data set

Use the sample JCL in [Figure 159 on page 402](#) to verify the contents of the control data set.

```
//UTIL EXEC PGM=EDGUTIL,PARM='VERIFY(ALL)'
//SYSPRINT DD utility message data set
//MASTER DD DSN=RMM.CONTROL.DSET,DISP=SHR
//SYSIN DD DUMMY
```

Figure 159. Verifying the contents of the control data set

JCL for mending the control data set

Use the sample JCL in [Figure 160 on page 402](#) to mend the control data set.

```
//UTIL EXEC PGM=EDGUTIL,PARM='MEND'
//SYSPRINT DD SYSOUT=*
//MASTER DD DSN=RMM.CONTROL.DSET,DISP=SHR
//SYSIN DD DUMMY
```

Figure 160. Mending the control data set

EXEC parameters for EDGUTIL

[Figure 161 on page 403](#) shows the EXEC parameters for EDGUTIL.

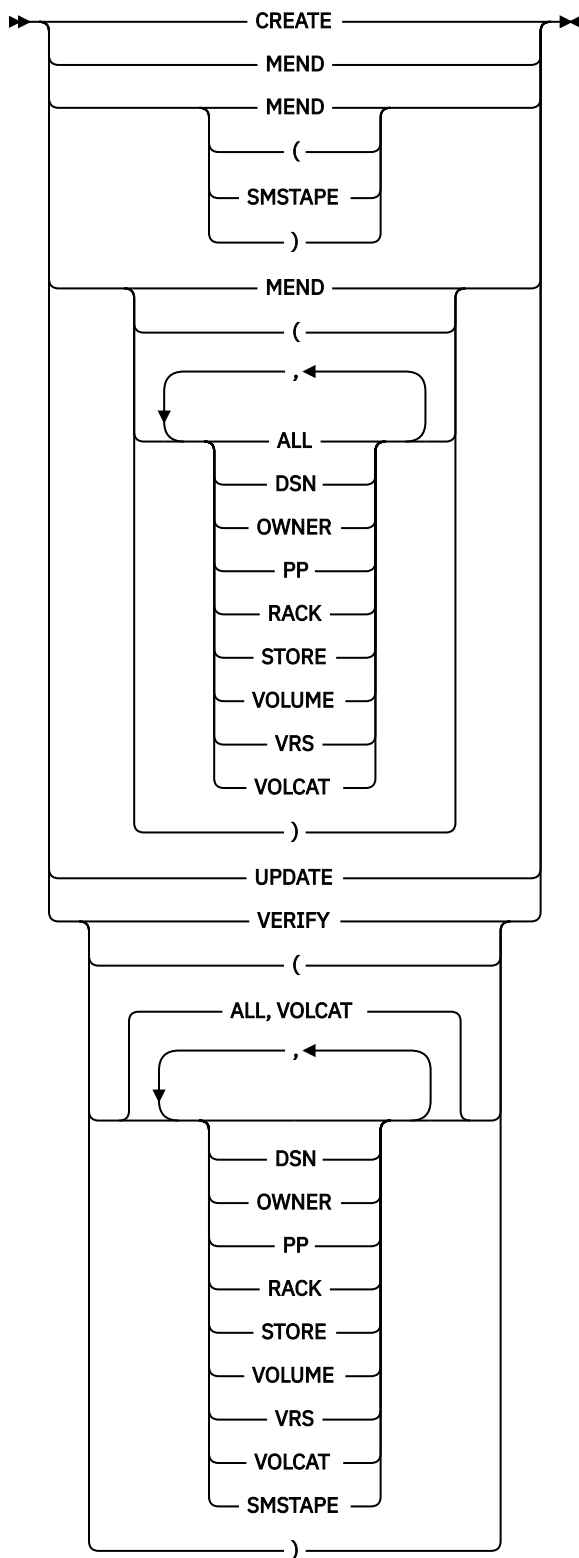


Figure 161. EDGUTIL EXEC parameters

CREATE

Use CREATE to create a new control data set control record.

MEND

Use MEND to detect and fix errors in your control data set. If you have system-managed volumes, DFSMSrmm also uses information from the TCDB and the library manager database. The errors you

encounter might have been created during conversion activities or as a result of system failures. When you specify the MEND parameter with no other values, the default processing performed by DFSMSrmm checks for all errors in your control data set, and in addition, if you have system managed volumes, DFSMSrmm uses information from both the TCDB and the library manager data base. Use the MEND function only with guidance from the IBM Support Center or to update the control data set once stacked volumes support is enabled. Run MEND on an unused control data set or with DFSMSrmm inactive. See [“Mending the control data set”](#) on page 414 for more information.

MEND(SMSTAPE)

Use MEND(SMSTAPE) to update the TCDB and the library manager database using information from the DFSMSrmm control data set. Use MEND to update the DFSMSrmm control data set based on information from the TCDB and the library manager database. Before running MEND(SMSTAPE), you should first use the VERIFY(SMSTAPE) option to find information that is not the same in the DFSMSrmm control data set, TCDB, and the library manager database. After running VERIFY(SMSTAPE) and before running MEND(SMSTAPE), you can update the control data set using DFSMSrmm TSO subcommands to correct DFSMSrmm information.

MEND(ALL,DSN,OWNER,PP,RACK,STORE,VOLUME,VRS,VOLCAT)

Use MEND to correct errors in the control data set. You can correct all the control data set information or select specific types of information.

The values you can specify on MEND are:

ALL

DFSMSrmm uses control data set information only and corrects all control data set information at once. No processing of the TCDB or library manager is performed.

DSN

DFSMSrmm corrects data set information based on comparing data set information to volume information.

OWNER

DFSMSrmm corrects owner information based on comparing owner information to volume information.

PP

DFSMSrmm corrects product information based on comparing software product information to volume and library shelf location information.

RACK

DFSMSrmm corrects rack number information based on comparing library shelf location information to volume information.

STORE

DFSMSrmm corrects bin number information based on comparing storage location shelf information to volume information.

VOLCAT

DFSMSrmm compares volume status and library name information in its control data set with the same information in the TCDB. If the information is different, DFSMSrmm corrects the control data set information. Not all of the information found to be different is corrected by DFSMSrmm MEND processing. EDGUTIL messages will be issued for differences found and whether they are corrected.

VOLUME

DFSMSrmm corrects volume information based on comparing information about data sets, software products, owners, and shelf locations in the library and storage locations.

VRS

When correcting vital record specification errors, DFSMSrmm validates the next vital record specification information to see if a name vital record specification exists. If DFSMSrmm does not find a next vital record specification, DFSMSrmm fixes the information about the next vital record specification. DFSMSrmm checks for generic data set name, job name, and volser masks. DFSMSrmm also checks that the location information in each vital record specification is valid by comparing it to the LOCDEF entries. If an invalid or an unsupported generic mask is

found, DFSMSrmm deletes the vital record specification. If an incorrect location type is found, DFSMSrmm corrects it based on the LOCDEF and SMS library definitions.

UPDATE

Use UPDATE to:

- Update an existing control data set control record.
- Mark the DFSMSrmm control data set as synchronized or not synchronized with the user catalogs so the control data set is synchronized at a later time.
- Enable extended bin support.

VERIFY(ALL,DSN,OWNER,PP,RACK,SMSTAPE,STORE,VOLUME,VRS,VOLCAT)

Use VERIFY to verify the information in the control data set and identify errors. You can verify all the information in the control data set at once or select specific values to verify individual pieces of information. If stacked volume support is enabled, DFSMSrmm checks the consistency of stacked volumes and the volumes in the stacked volumes.

To correct inconsistencies found during VERIFY processing, use the DFSMSrmm TSO subcommands to correct the inconsistencies. Use MEND(SMSTAPE) to drive changes to the TCDB and library manager database from the DFSMSrmm control data set or use access method services commands to correct errors in the TCDB.

If both VOLCAT and SMSTAPE are specified with VERIFY, DFSMSrmm does only the SMSTAPE processing.

The values you can specify on VERIFY are:

ALL

DFSMSrmm verifies all information at once, except for VOLCAT and SMSTAPE, which do consistency checking against the TCDB.

DSN

DFSMSrmm validates data set information and compares data set information to volume information.

OWNER

DFSMSrmm validates owner information and compares owner information to volume information.

PP

DFSMSrmm validates product information and compares software product information to volume and library shelf location information.

RACK

DFSMSrmm validates rack number information and compares library shelf location information to volume information.

SMSTAPE

DFSMSrmm performs extra processing with the TCDB and library manager database when SMSTAPE is specified. DFSMSrmm scans both the DFSMSrmm control data set and the TCDB sequentially to find DFSMSrmm volumes that are not in the TCDB and TCDB volumes that are not in the DFSMSrmm control data set. DFSMSrmm also checks any volume that is found to be system-managed, either by definition to DFSMSrmm or retrieved from the TCDB, against the library manager database for an IBM automated tape library.

STORE

DFSMSrmm validates bin number information and compares storage location shelf information to volume information.

VOLCAT

DFSMSrmm compares volume status and library name information in its control data set with the same information in the TCDB. If the information is different, DFSMSrmm issues an information message, but sets a minimum return code of 0.

VOLUME

DFSMSrmm compares volume information to information about data sets, software products, owners, and shelf locations in the library and storage locations.

VRS

For vital record specification checking, DFSMSrmm validates the next vital record specification information to see if a name vital record specification exists. DFSMSrmm checks for generic data set name, job name, and volser masks. DFSMSrmm also checks that the location information in each vital record specification is valid by comparing it to the LOCDEF entries. If DFSMSrmm does not find a next vital record specification, it issues an information message, but sets a minimum return code of 0.

The default is VERIFY(ALL,VOLCAT).

SYSIN file for VERIFY and MEND processing

An optional SYSIN file allows you to select the subset from the available locations, and volume entries during verification of volumes. Verification of volumes includes VERIFY/MEND(SMSTAPE/VOLCAT), VERIFY or MEND, VERIFY/MEND(ALL), and VERIFY/MEND(VOL) processing. By default, all volumes are verified.

The SYSIN commands in [Figure 162 on page 406](#) can be used to select the subset of volumes.

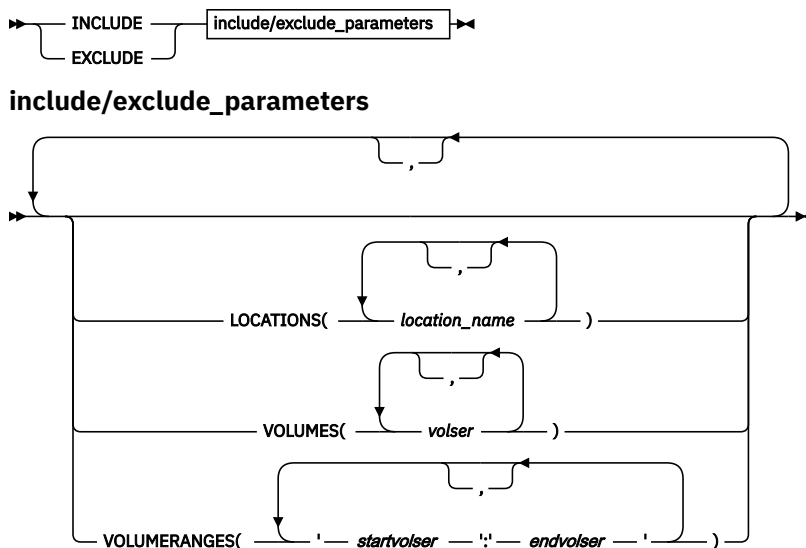


Figure 162. EDGUTIL SYSIN file

EXCLUDE

Specifies the exclusion criteria for EDGUTIL processing. You can specify this command one time only. You can specify INCLUDE and EXCLUDE commands in any order.

You can specify one or more of these optional operands:

LOCATIONS(*location_name*)

Specifies volumes to be excluded based on the volume's current location. For 3-way audit and VOLCAT processing, specify the system-managed library location names. For VOL processing, any system-managed library, storage location name known to DFSMSrmm, or SHELF can be specified. A *location_name* is one-to-eight characters and can be a location name mask. Each *location_name* can be specified in one of these ways:

- Specify a specific location using one-to-eight character names.
- Specify all locations using a single asterisk (*).
- Specify all locations that begin or end with specific characters, such as ATL* or *DR, or multiple locations by using * within a location name.
- Use % (percent sign) in the location name to replace a single character. You can specify up to eight % in a location name mask.

DFSMSrmm does not validate the specified location names against the DFSMSrmm LOCDEF entries or the names of the SMS libraries.

When validation of a location name fails, the EDGUTIL utility stops processing and ends with the return code of 12.

Any *location_names* specified are used to exclude volumes based on the TCDB volume record library name and the DFSMSrmm volume record current location. You can specify a list of values.

VOLUMES(*volser*)

Specifies a list of volumes to be excluded. You can specify the volumes as fully qualified or as a *volser* prefix ending in *. A fully qualified volume is one-to-six alphanumeric, national or special characters, but the first character must not be blank. Quotation marks are required for special characters. Any value ending in *, even if it is enclosed in quotation marks, is considered to be a *volser* prefix. You can specify a list of values.

VOLUMERANGES('startvolser':'endvolser')

Specifies a subset of volumes based on the starting and ending *volser*s to be excluded. The *volser*s must be one-to-six alphanumeric, national, or special characters, but the first character must not be blank. Single quotation marks are required for each value regardless of the use of special characters. The end of range must not be lower than the start of the range. You can specify a list of values.

The default is that no volumes are excluded.

INCLUDE

Specifies the inclusion criteria for EDGUTIL processing. You can specify this command one time only. You can specify INCLUDE and EXCLUDE commands in any order.

You can specify one or more of these optional operands:

LOCATIONS(*location_name*)

Specifies a subset of the available volumes based on the volume's current location for processing. For 3-way audit and VOLCAT processing, specify the system-managed library location names. For VOL processing, any system-managed library, storage location name known to DFSMSrmm, or SHELF can be specified. A *location_name* is one-to-eight characters and can be a location name mask. Each *location_name* can be specified in one of these ways:

- Specify a specific location using one-to-eight character names.
- Specify all locations using a single asterisk (*).
- Specify all locations that begin or end with specific characters, such as ATL* or *DR, or multiple locations by using * within a location name.
- Use % (percent sign) in the location name to replace a single character. You can specify up to eight % in a location name mask.

DFSMSrmm does not validate the specified location names against the DFSMSrmm LOCDEF entries or the names of the SMS libraries.

When validation of a location name fails, the EDGUTIL utility stops processing and ends with the return code of 12.

Any *location_names* specified are used to select volumes based on the TCDB volume record library name and the DFSMSrmm volume record current location. You can specify a list of values.

VOLUMES(*volser*)

Specifies a list of volumes to be processed. You can specify the volumes as fully qualified or as a *volser* prefix ending in *. A fully qualified volume is one-to-six alphanumeric, national or special characters, but the first character must not be blank. Quotation marks are required for special characters. Any value ending in *, even if it is enclosed in quotation marks, is considered to be a *volser* prefix. You can specify a list of values.

VOLUMERANGES(*startvolser*:*endvolser*)

Specifies a subset of volumes based on the starting and ending *volser*s to be processed. The *volser*s must be one-to-six alphanumeric, national, or special characters, but the first character

must not be blank. Quotation marks are required for each value regardless of the use of special characters. The end of range must not be lower than the start of the range. You can specify a list of values.

Here are some examples of using the SYSIN command to select a subset of volumes:

```
//SYSIN DD *  
INCLUDE VOLUMES(ABC001,ABC002) LOCATIONS(ATL1)  
/*
```

If the volumes with names 'ABC001' or 'ABC002', it is included or if it is located at the location 'ATL1', it is also included.

```
//SYSIN DD *  
INCLUDE VOLUMERANGES('XYZ001':'XYZ099') VOLUMES(001*)  
/*
```

This statement includes volumes in the volser range from 'XYZ001' to 'XYZ099', or additionally it will also include any volume with a volser starting with "001".

```
//SYSIN DD *  
INCLUDE VOLUMES(Z*,B*,VOL001)  
EXCLUDE VOLUMES(B5*)  
/*
```

In this scenario, the statement includes any volume with volser starting with 'Z', 'B', or 'VOL001'; except for volumes starting with "B5".

The default is that all volumes defined in the TCDB (when 3-way audit or VOLCAT processing is requested) and all volumes defined to DFSMSrmm are processed.

The EDGUTIL utility processes the SYSIN file to determine if a subset of volumes is to be processed. EDGUTIL processing is independent of the order of the commands. As each volume is about to be processed, it is checked first against the INCLUDE selection and then against the EXCLUDE selection. The volume selection process applies to all volume processing in EDGUTIL. For 3-way audit processing, the library names known to the volume catalogs and DFSMSrmm control data set volume records determine the system-managed libraries volume details to be retrieved, unless the LOCATIONS operand has restricted the subset to specific libraries. The library manager volume details are retrieved directly from the candidate system-managed libraries as needed.

How EDGUTIL performs VERIFY and MEND processing for volumes

The EDGUTIL utility performs this processing:

- For 3-way audit processing:
 - All TCDB volume entries are retrieved from all connected volume catalogs and compared with DFSMSrmm volume information. TCDB volumes not defined to DFSMSrmm are reported unless excluded from processing by SYSIN.
 - Any DFSMSrmm-defined volume, unless excluded from processing by volume or location, known from the TCDB or DFSMSrmm control data set to be in a system managed library, is checked against library manager data.
 - EDGUTIL does not retrieve volumes from the library manager unless information is needed for a volume known to the TCDB or to DFSMSrmm.
- For 3-way audit, VOLCAT, and VOL processing:
 - Each DFSMSrmm-defined volume is processed only if selected for processing by default or by LOCATIONS, VOLUMES or VOLUMERANGES, and is not excluded.
- For VOL processing:
 - When you select a subset of DFSMSrmm-defined volumes to be processed using one of these: VERIFY, VERIFY with ALL/VOL, MEND, or MEND with ALL/VOL:

- Processing includes the previous and next volume regardless of whether the previous and next volumes are selected to be verified.
- Processing excludes ensuring that stacked volume information is consistent.

Creating or updating the control data set control record

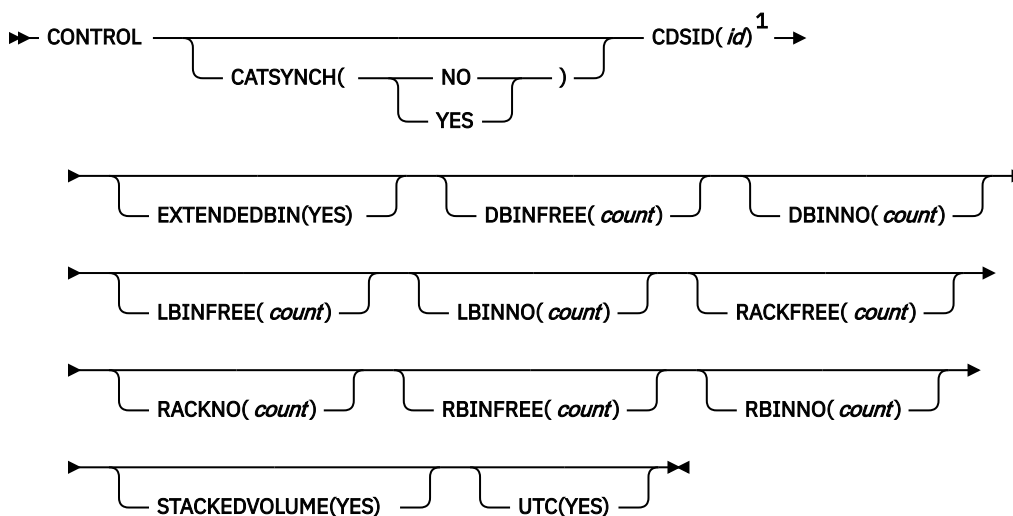
Create the control data set control record the first time you run EDGUTIL. This normally occurs during DFSMSrmm implementation or conversion to DFSMSrmm. The control data set control record contains information about the number of shelf locations in the library and storage locations. To create or update the control record, the user of EDGUTIL must have UPDATE or higher RACF access to the DFSMSrmm control data set.

Once you have created a control record, defined shelf locations to your installation, and have begun managing these shelves with DFSMSrmm, use the RMM ADDRACK subcommand to add shelf locations to DFSMSrmm. Do not use EDGUTIL to change the number of shelf locations.

You should only need to update the control record to correct rack or bin counts. DFSMSrmm lets you update the control data set and the control record only if inventory management and backup, restore and reorganize are not in progress.

As part of CREATE processing or UPDATE processing, EDGUTIL ensures that DFSMSrmm is not active and opens the control data set for load processing. The information provided on the CONTROL command in SYSIN is used to build a control record, which is written to the control data set.

The SYSIN commands are required to create or update the control data set control record.



Notes:

¹ The CDSID operand is required only for CREATE

Figure 163. EDGUTIL SYSIN commands

CATSYNCH(NO|YES)

Specifies whether or not the DFSMSrmm control data set and system catalogs are synchronized or not. When the DFSMSrmm control data set is not synchronized with the system catalogs, DFSMSrmm always retrieves catalog information to determine the catalog status of a data set.

When you indicate that catalogs are fully shared with the EDGRMMxx parmlib OPTION CATSYSID(*) as described in “Defining system options: OPTION” on page 161, DFSMSrmm automatically marks the DFSMSrmm control data set as synchronized when you run the EDGHSKP utility with the CATSYNCH parameter as described in “EXEC parameters for EDGHSKP” on page 337. You must use CATSYNCH(YES) to indicate to DFSMSrmm that catalogs are synchronized when you use DFSMSrmm with unshared catalogs. Do not specify CATSYNCH(YES) if you have not run the EDGHSKP utility with

the CATSYNCH parameter on each system to synchronize the DFSMSrmm control data set with the user catalogs.

CATSYNCH(YES) sets the last synchronization date and time to the current date and time. CATSYNCH(NO) clears the last synchronization date and time. Specify CATSYNCH(NO) to force synchronization of the DFSMSrmm control data set and user catalogs the next time inventory management is run. DFSMSrmm cannot track catalog updates when the DFSMSrmm subsystem is stopped and issues messages when the updates cannot be made.

CDSID(id)

Specifies 1-to-8 characters that identify the control data set by name. There is no default.

At DFSMSrmm startup time, DFSMSrmm matches this CDSID value with the CDSID operand in parmlib member EDGRMMxx. The CDSID value in EDGUTIL will set or change the control data set ID. EDGUTIL will fail an attempt to change the CDSID value while DFSMSrmm is active. If EDGUTIL UPDATE processing runs while DFSMSrmm is not active and the CDSID value was specified in the SYSIN DD, EDGUTIL matches it with the CDSID in the control data set control record. If they do not match, a WTOR EDG6527D is issued prompting that the CDSID change is requested and asking the operator to verify that DFSMSrmm is inactive across any other systems that share the control data set. Reply to the WTOR with either "Y" to continue with the CDSID change or "N" to cancel the request. Ensure that the GRSRNLxx parmlib member is updated to reflect any CDSID changes you make. See [“Step 5: Updating SYS1.PARMLIB members”](#) on page 25. You must always set a CDSID using the EDGUTIL utility. The CDSID ensures that only the correct DFSMSrmm systems start up and use the control data set that you created for them.

This operand is required for CREATE.

CONTROL

Specifies to update or create the control record.

DBINFREE(nnnnnn)

Specifies the number of empty bin numbers in the DISTANT storage location in a range from 0 to 999999.

DBINNO(nnnnnn)

Specifies the number of bin numbers in the DISTANT storage location in a range from 0 to 999999.

EXTENDED BIN(YES)

Enables DFSMSrmm extended bin support, which allows the reuse of bins at the start of a move.

When extended bin support is enabled, DFSMSrmm records additional information in the volume and bin record while a volume is moving from or to a bin-managed storage location.

Do not enable extended bin support until you have made sure that the same level of code has been installed for all the DFSMSrmm systems that share a control data set.

Once it is enabled, extended bin support cannot be disabled.

Extended bin support must be enabled, if you want to use DFSMSrmm parmlib OPTION command REUSEBIN(STARTMOVE) operand to reuse bins when a volume moves from a bin.

LBINFREE(nnnnnn)

Specifies the number of empty bin numbers in the LOCAL storage location in a range from 0 to 999999.

LBINNO(nnnnnn)

Specifies the number of bin numbers in the LOCAL storage location in a range from 0 to 999999.

RACKFREE(nnnnnn)

Specifies the number of empty rack numbers in the library for location SHELF and for system-managed libraries in a range from 0 to 2147483647.

RACKNO(nnnnnn)

Specifies the number of rack numbers in the library for location SHELF and for system-managed libraries in a range from 0 to 2147483647.

RBINFREE(nnnnnn)

Specifies the number of empty bin numbers in the REMOTE storage location in a range from 0 to 999999.

RBINNO(nnnnnn)

Specifies the number of bin numbers in the REMOTE storage location in a range from 0 to 999999.

STACKEDVOLUME(YES)

Enables DFSMSrmm stacked volume support.

When stacked volumes have been defined to DFSMSrmm but you have not enabled stacked volume support, DFSMSrmm manages the movement of the volumes that are in containers using the individual volume location. Volume movement is limited to non-shelf-managed storage locations. When you enable stacked volume support, DFSMSrmm uses the stacked volume records to manage the movement of the volumes contained in the stacked volumes. See [“Setting up DFSMSrmm stacked volume support” on page 415](#).

You cannot remove stacked volume support after it is enabled.

UTC(YES)

Enables DFSMSrmm common time support. Prior to enabling this support, ensure all systems in the RMMplex have toleration maintenance installed or are at the current z/OS release, and all applications dependent on the correct date and time information from DFSMSrmm are updated to support the new time zone support, if required.

You must only enable common time support if the time of day clocks of all systems in the RMMplex are set to GMT.

Once you enable common time support, DFSMSrmm starts to record the control data set record dates and times in common time and converts existing values, as required, from local times to common time. See [“Setting up DFSMSrmm common time support” on page 416](#) for more information.

You cannot disable common time support after it is enabled.

Verifying the contents of the control data set

Specify VERIFY on the EXEC parameter of EDGUTIL to verify the contents of the control data set. For VERIFY processing, EDGUTIL reads sequentially through the different record types in the control data set. The record types are identified by the VERIFY options you specify. For each record DFSMSrmm validates key fields and checks information with related records in the control data set. DFSMSrmm issues an informational message to the SYSPRINT file for each discrepancy that is identified. You can verify all the information in the control data set at once, or select specific values to verify individual pieces of information.

For example, if you specify the STORE value as shown in [Figure 164 on page 411](#), DFSMSrmm reads all the storage location shelf information in the control data set. DFSMSrmm then reads volume information for only those volumes in the storage locations and verifies that the volumes include the correct location information.

```
//UTIL      EXEC PGM=EDGUTIL,PARM='VERIFY(STORE)'  
//SYSPRINT DD SYSOUT=*  
//MASTER   DD DISP=SHR,DSN=RMM.CONTROL.DSET
```

Figure 164. Example of JCL for VERIFY(STORE)

Note: The SYSIN commands in [Figure 162 on page 406](#) can be used to select a subset of volumes.

An optional output file, EDGSPLCS, can be specified with VERIFY(SMSTAPE) to request that control statements are generated that can be used with the EDGSPLCS utility. If changes to either the TCDB or library manager data are required, message EDG6846I is issued to SYSPRINT for each change. When VERIFY(SMSTAPE) processing is completed, you can edit the output data set to select those for further processing. When you have reviewed the statements and selected those you wish to process, you can run the EDGSPLCS utility to process the chosen statements. This process might be quicker than running MEND(SMSTAPE) once you have run VERIFY(SMSTAPE). See [“Using EDGSPLCS to issue commands to](#)

OAM for system-managed volumes” on page 418 for additional information about the EDGSPLCS utility. See “EDGSPLCS file for the EDGUTIL utility” on page 418 for information about the EDGSPLCS file for the EDGUTIL utility.

When you specify VERIFY(ALL), and this completes successfully, DFSMSRmm resets the error indicator that is set when the control data set recovery processing was not successful. For VERIFY(VOLCAT), DFSMSRmm compares TCDB information with information in the DFSMSRmm control data set. For VERIFY(SMSTAPE), DFSMSRmm also retrieves the library manager information for each system-managed volume and compares the information to the TCDB and DFSMSRmm information. The function uses the DFSMSRmm control data set as the master. Use EDGUTIL MEND(SMSTAPE) to synchronize the TCDB and library manager database from DFSMSRmm. When you specify MEND, without SMSTAPE, EDGUTIL checks that DFSMSRmm is not active or that the DFSMSRmm control data set is not in use. MEND processing is performed the same way as VERIFY(ALL) and VERIFY(VOLCAT) processing. MEND processing cannot fix all discrepancies but those that can be fixed automatically are corrected by updating the control data set.

For some VERIFY functions when EDGUTIL is running, DFSMSRmm uses the information from the parmlib options of the running DFSMSRmm. For example, DFSMSRmm checks LOCDEF entries for the types and names of locations defined, and also checks the access system definitions for system managed libraries. If DFSMSRmm has never been started on the system where EDGUTIL is run, the parmlib options cannot be part of the verification. To cause certain parmlib information to be used, start DFRMM with the chosen parmlib member, and run EDGUTIL.

During VERIFY, EDGUTIL issues messages to indicate what stage of processing has been reached. These messages also go in the SYSPRINT file. An example of the SYSPRINT message file can be seen in [Figure 165 on page 412](#).

```
EDG6433I STARTING VERIFICATION OF RACK      RECORDS
EDG6433I STARTING VERIFICATION OF VOLUME   RECORDS
EDG6433I STARTING VERIFICATION OF DATA SET RECORDS
EDG6433I STARTING VERIFICATION OF OWNER    RECORDS
EDG6433I STARTING VERIFICATION OF PRODUCT  RECORDS
EDG6434I NO PRODUCT      RECORDS IN CONTROL DATA SET
EDG6433I STARTING VERIFICATION OF STORE    RECORDS
EDG6434I NO EMPTY BIN   RECORDS IN CONTROL DATA SET
EDG6434I NO INUSE BIN    RECORDS IN CONTROL DATA SET
EDG6433I STARTING VERIFICATION OF VRS      RECORDS
EDG6417I CONTROL DATA SET VERIFY SUCCESSFUL
EDG6901I UTILITY EDGUTIL COMPLETED WITH RETURN CODE 0
```

Figure 165. Sample EDGUTIL SYSPRINT output

You can correct errors that are found in the control data set by using one of these methods:

- Restore the control data set to a level where the errors are not present. See “[Restoring the control data set](#)” on page 389 for additional information.
- Correct information in the control data set by using the RMM TSO ADD, CHANGE, DELETE, and LIST subcommands. See *z/OS DFSMSRmm Managing and Using Removable Media* for additional information.
- Mend the control data set by specifying MEND on the EXEC parameter of EDGUTIL. See “[Mending the control data set](#)” on page 414 for additional information.
- Synchronize the TCDB and the library manager database from the DFSMSRmm control data set by specifying MEND(SMSTAPE) on the EDGUTIL EXEC parameter. See “[Synchronizing the contents of the control data set](#)” on page 413 for additional information.
- Synchronize the DFSMSRmm control data set from the TCDB by specifying MEND(VOLCAT) on the EDGUTIL EXEC parameter. See “[Synchronizing the contents of the control data set](#)” on page 413 for additional information.

Note: The SYSIN commands in [Figure 162 on page 406](#) can be used to select a subset of volumes.

While verify is running in parallel with DFSMSRmm active, the DFSMSRmm control data set can be updated by other processing, and as a result, verify can report inconsistencies for resources that are updated during the verify processing. To avoid rerunning verify to clean up these inconsistencies; run EDGUTIL when there is little activity that updates the control data set.

Note: For VERIFY(SMSTAPE), MEND(SMSTAPE), and MEND(VOLCAT) only, DFSMSrmm obtains the latest information from the TCDB and library manager before any inconsistency is reported or corrected. This ensures that concurrent system processing is better tolerated.

DFSMSrmm processes each resource in turn, dependent on the VERIFY parameter, and validates related information as follows:

- For every shelf location for a scratch volume, the associated volume must have the correct shelf location and media name and must be recorded as a scratch volume.
- For every data set, the corresponding volume must be defined and the next and previous data set names in sequence must also be correctly defined.
- For every defined owner, the corresponding volumes are checked for correct information.

Verifying the control data set and tape configuration database

You can use EDGUTIL VERIFY(VOLCAT) to check the consistency of information in the control data set and the tape configuration database (TCDB). EDGUTIL checks volume status and the library where the volume resides. You can use EDGUTIL VERIFY(SMSTAPE) to include the library manager database in the consistency checking. You can use MEND(SMSTAPE) to update the TCDB and library manager database with information from the DFSMSrmm control data set. You can use the EDGSPLCS DD name to get the EDGSPLCS statement out during VERIFY(SMSTAPE) so that errors can be corrected without having to run MEND(SMSTAPE) as well.

For 3-way audit with system managed libraries, VERIFY(SMSTAPE), MEND, and MEND(SMSTAPE) processing exploits the use of a host library interface to return multiple volumes in a single request. In addition, DFSMSrmm allows the selection of libraries and subsets of volumes. This processing reduces the EDGUTIL elapsed time. EDGUTIL constructs a series of requests for the libraries containing the volumes to be verified. The information is retrieved as required and is processed together with entries from the TCDB and volume information from the DFSMSrmm control data set. When a mismatch is detected between the TCDB, DFSMSrmm control data set, and the library manager data, DFSMSrmm uses the CBRXLCS QVR request so that any timing related change can be detected. Processing of all volumes, regardless of function, is subject to the subsetting through location and volume selection.

Figure 166 on page 413 shows JCL for verifying the control data set and tape configuration database:

```
//UTIL      EXEC      PGM=EDGUTIL,PARM='VERIFY(VOLCAT) '
//SYSPRINT DD          utility message data set
//MASTER   DD          control data set
```

Figure 166. Sample JCL for verifying the control data set and the TCDB

Synchronizing the contents of the control data set

You can synchronize the contents of the DFSMSrmm control data set, TCDB, and the library manager database by using the EDGUTIL utility. First, use the verify function to check for inconsistencies in the contents of the DFSMSrmm control data set with the TCDB and the Library Manager. Then use EDGUTIL to automatically fix the inconsistencies found between the control data set and the TCDB and Library Manager. Table 54 on page 413 lists the information checked for inconsistencies by EDGUTIL in the CDS, Library Manager, and TCDB.

Table 54. Information checked by EDGUTIL for inconsistencies			
Information Checked	DFSMSrmm CDS	Library Manager	TCDB (Volume Catalog)
Category		X	
Error category ¹		X	
Error type ¹			X
Library name			X

Table 54. Information checked by EDGUTIL for inconsistencies (continued)

Information Checked	DFSMSrmm CDS	Library Manager	TCDB (Volume Catalog)
Location/Destination	X		
Media Type	X	X	X
Residency	X	X	X
Status	X		
Storage group	X		X
Use attribute			X
1. In order to recover volumes from the error category, DFSMSrmm will change the volume to SCRATCH for volumes found to be in Error Category with any of these volume error statuses: CHECKPT, LNGTHERR, MEDIAMNT, MED2MNT, NOTINLIB, PASSPROT, RACFPROT, TRKCMPAT, and UNEXPIR. These errors are explained in <i>z/OS DFSMS OAM Planning, Installation, and Storage Administration Guide for Tape Libraries</i>.			

You can use EDGUTIL VERIFY(SMSTAPE) to check the synchronization of the DFSMSrmm control data set with the TCDB and Library Manager database. Specify EDGUTIL MEND(SMSTAPE) to synchronize the TCDB and the library manager database from the DFSMSrmm control data set. You can run the function against an active control data set because the function does not update the control data set. The function uses the DFSMSrmm control data set as the master and makes changes to the TCDB and library manager database such as status and storage group information.

Specify EDGUTIL MEND(VOLCAT) with DFSMSrmm inactive to update the DFSMSrmm control data set from the TCDB with information such as location, media type, storage group, and in-transit status.

Mending the control data set

Recommendations:

The most important part of the EDGUTIL MEND(ALL) procedure is to BACKUP the CDS before you start. Here are the recommended steps to follow (you may prefer to practice with MEND on a backup VSAM data set on a test system first).

1. Always run the MEND function on a VSAM copy of the control data set first. Taking a back up of the control data set is essential because the control data set is unusable if MEND processing fails for any reason.
2. Do not run the MEND function when DFSMSrmm is running on the same system and is using a control data set with the same name as the control data set that the MEND function is expected to correct.
3. Do not enable stacked volume support until all systems using the control data set are on a supporting release level.
4. Quiesce all tape activity on all images sharing the CDS while this procedure is taking place.
5. Backup the CDS and journal to disk using EDGHSKP with PARM='BACKUP'.
6. Stop RMM ('P DFRMM') or quiesce RMM ('F DFRMM,QUIESCE') on all images.
7. Restore the flat-file CDS backup to a VSAM data set allocated with the same characteristics as in CDS. Thus, you create the VSAM copy of the CDS.
8. Run MEND(ALL) against the VSAM copy of the CDS.
9. You can run EDGUTIL with PARM='VERIFY(ALL)' if you want to ensure the errors in question have been repaired, but it's probably redundant at this point.
10. If MEND(ALL) against the VSAM copy of the CDS finished successfully, you can either:
 - a. backup the VSAM copy and restore it to the current CDS, or
 - b. rename the VSAM data sets so that the VSAM copy becomes the current CDS, or

c. run MEND(ALL) against the current CDS.

If you run into any problems, you'll always have the original backup (item 5 above) to restore from.

11. Restart RMM ('S DFRMM,M=xx') or reinit RMM ('F DFRMM,M=xx') on all systems sharing the CDS.

You can use EDGUTIL VERIFY as described in [“Verifying the contents of the control data set” on page 411](#) to find most control data set errors. Obtain guidance from the IBM Support Center to use EDGUTIL MEND to fix the errors found in the DFSMSrmm control data set. When you use the EDGUTIL MEND function to fix errors for system-managed tape volumes, MEND updates the DFSMSrmm control data set from information from the TCDB and the library manager database.

You can also use the MEND function to enable stacked volume support as described in [“Setting up DFSMSrmm stacked volume support” on page 415](#). MEND processing checks to see if you have enabled stacked volume support with the EDGUTIL UPDATE option STACKEDVOLUME(YES). DFSMSrmm then creates stacked volume information from the existing volume 'In container' values. DFSMSrmm marks the control data ready for stacked volume support if processing is successful.

Note: The SYSIN commands in [Figure 162 on page 406](#) can be used to select a subset of volumes.

During MEND processing, DFSMSrmm creates stacked volumes in the DFSMSrmm control data set that are marked as media type HPCT and recording format 128TRACK. DFSMSrmm obtains the media name from pool definitions defined using the DFSMSrmm parmlib member VLPOOL command. Start DFSMSrmm at least once with a parmlib member that contains the VLPOOL commands, before you run EDGUTIL, to ensure that EDGUTIL can use the VLPOOL information. DFSMSrmm marks the stacked volumes as 'Created during MEND' and obtains location information from the last volume in sequence that is found to be in the container.

After MEND processing completes, you might need to add or change some volume information. Use the RMM CHANGEVOLUME subcommand to set or change any information.

Setting up DFSMSrmm stacked volume support

To enable stacked volume support, perform these tasks:

1. Update all systems sharing a control data set to the level of code that contains stacked volume support.
2. Correct any information about stacked volumes that you have defined to DFSMSrmm before running the DFSMSrmm EDGUTIL utility. EDGUTIL changes the volume type to stacked but does not set the correct location information for the volume. You can set the correct volume type and location information prior to running EDGUTIL MEND using the RMM CHANGEVOLUME subcommand. [Figure 167 on page 415](#) shows an example of how you can use the RMM SEARCHVOLUME subcommand to build a CLIST that can be used to change volume information. Specify operands that are based on the way that you have defined volumes to DFSMSrmm. Then run the CLIST produced by the command to make changes to the volumes.

```
RMM SEARCHVOLUME VOLUME(ST*) OWNER(*) LIMIT(*) -  
CLIST('RMM CHANGEVOLUME ', 'TYPE(STACKED) LOCATION(vts_name) NORACK')
```

Figure 167. Changing volume type and volume location

3. Run the EDGUTIL utility with UPDATE with the STACKEDVOLUME(YES) operand on the CONTROL statement of the SYSIN file to enable stacked volume support.
4. To check the stacked volume information, you can use EDGUTIL with VERIFY(VOLUME) to check whether the container information is correct. Use the RMM LISTCONTROL CNTL subcommand to display the status of support. DFSMSrmm marks the support status as MIXED if there is any container information in the control data set volume records. If the support status shows MIXED, you can run EDGUTIL MEND as described in [“Mending the control data set” on page 414](#) to make the container information consistent. During MEND processing, DFSMSrmm creates the necessary stacked volumes if you have not previously defined them using the DFSMSrmm subcommands.
5. Run EDGHSKP storage location management processing to clean up location and bin number information in volumes that are in a container.

Setting up DFSMSrmm common time support

Before DFSMSrmm common time support (UTC), also known as GMT, is enabled, all dates and times are stored in the DFSMSrmm control data set in local time. When the control data set is shared, and the sharing systems are set to run in different time zones, the local dates and times in the control data set could be from any of your systems. When you display information or extract records, you need to be aware of how the records were created, on which system, and where they might have been updated in order to interpret the dates and times shown. The same consideration also applies for records created or updated prior to enabling common time support because DFSMSrmm assumes they are times local to the system running the DFSMSrmm subsystem and converts the values based on that assumption.

When you enable common time support, DFSMSrmm maintains the records in the control data set in common time. Most date and time fields are paired together to enable an accurate conversion to and from common time and between different time zones. In some cases, DFSMSrmm has date fields in control data set records, and there is no associated time field. For these date fields, DFSMSrmm uses an internal algorithm that approximates conversion between time zones based on the time zone offsets involved.



Attention: Using the SET system command with either the DATE or the TIME keyword, or both, or replying to message IEA888A to run the system on future or past dates can affect the way that DFSMSrmm calculates local times. In order to get the correct results from DFSMSrmm processing when you need to test with future or past dates, you should alter the TOD clock and keep the time zone offset as before.

To enable common time support, perform these tasks:

1. Ensure all systems in the RMMplex have toleration maintenance installed or are at the current z/OS release, and all applications dependent on the correct date and time information from DFSMSrmm are updated to support the new time zone support if required. DFSMSrmm subcommand output remains in local time, so most applications do not need to change unless they are to exploit the availability of the time zone offset.
2. Ensure the system time of day (TOD) clock is set to GMT on all systems in the RMMplex. It is common practice for the system to use local time based either on the TIMEZONE value in the CLOCKxx member of parmlib or from an external time source.
3. Run the EDGUTIL utility with UPDATE with the UTC(YES) operand on the CONTROL statement of the SYSIN file to enable common time support. See [“Creating or updating the control data set control record” on page 409](#).

When common time support is enabled, any newly recorded dates and times are stored in common time and any existing records are converted to common time as they are updated by DFSMSrmm processing. Note: You will continue to see dates and times in local time because DFSMSrmm converts from common time to your local time.

When you use TSO subcommands in batch TMP or in native TSO, the TSO subcommands return data in your local time. It is important to specify dates and times as local time values when using subcommands in this environment.

For REXX variables and the DFSMSrmm ISPF dialog, you have the option of returning the date in any selected time zone so that you can, when needed, view data in times local to other systems managed by DFSMSrmm.

- For the REXX environment, you can optionally set SYSAUTH.EDGTZ to your selected time zone offset so that subcommands return dates and times in your selected time zone. You can also include the TZ operand on ADD and CHANGE subcommands, when required, to indicate the time zone for dates and times specified on the subcommand.
- For ISPF, the DFSMSrmm dialog has the ability for each user to select their own time zone value or run using the existing systems local time. The dialog exploits the SYSAUTH.EDGTZ setting and the TZ subcommand operand to make use of different time zones simple and straightforward for the user. The user operates completely in their selected time zone when reading returned data and when making changes to DFSMSrmm information. You can also change the setting during an ISPF session to select a new value for the next dialog interactions.

The DFSMSrmm application programming interface always returns values in local time. In addition, a SFI indicates the time zone offset used so that any application dependent either on the SFI or XML values can make its own time conversions based on the time zone offset.

The report extract data set contains date and time values in the local time of the running system. The extract header record includes a field that lists the time zone offset.

Use the RMM LISTCONTROL CNTL subcommand or the CONTROL dialog to see whether DFSMSrmm common time support is enabled.

Daylight savings time considerations

It is recommended that you QUIESCE DFSMSrmm when you make your daylight savings time changes on the system. This ensures that date and time fields in DFSMSrmm control data set records are handled consistently.

When you switch from daylight savings time, there are dates and times in the system that are repeated. DFSMSrmm journal records are time-stamped with local time values, so either:

1. QUIESCE DFSMSrmm until there is no chance of repeated times, or
2. Accept that during forward recovery with journal records from this time change period, you will get message EDG6429W.

IBM recommends that you take the second approach.

Enabling extended bin support

To enable extended bin support, create or update the control data set control record using the EDGUTIL utility with the EXTENDEDDBIN(YES) option. See [“Creating or updating the control data set control record” on page 409](#) for a detailed description of the EXTENDEDDBIN parameter.

When extended bin support is enabled, DFSMSrmm records additional volume information and bin information to keep track of the volume's location when a volume is moving from a bin-managed storage location or to a bin-managed storage location.

DFSMSrmm keeps track of this information for a volume: Destination bin number, Destination bin media name for a volume, current bin number, current bin media name, Old bin number, and Old bin media name. When a volume starts moving to a bin-managed storage location, DFSMSrmm updates the destination bin fields with the bin number and the bin media name. When the move has been confirmed, DFSMSrmm updates the current bin fields with the destination bin fields.

If a volume moves from a bin-managed storage location, DFSMSrmm does not change the current bin fields until the move has been confirmed. DFSMSrmm changes the old bin number to the current bin number and clears the current bin number when a move is confirmed. When extended bin support is not enabled and a volume is moving, DFSMSrmm shows the source bin in the old bin fields and the target bin in the current bin fields.

DFSMSrmm keeps track of additional volume information in the bin record when extended bin support is enabled: Moving-in volume, Moving-out volume, and Old volume. When extended bin support is enabled, a volume, for which a move to the bin has been started, is shown as the 'moving-in volume' in the bin record. When the volume move is confirmed, the volume is shown as the current volume in the bin record. A volume, for which a move from the bin has been started, is shown as the 'moving-out volume' in the bin record. When the volume move out of the bin is confirmed, the volume is shown as the 'old volume'. When extended bin support is not enabled, DFSMSrmm shows a volume as the 'current volume' in the bin record from the time the move to the bin has been started until the time that the move from this bin has been confirmed.

Before you enable extended bin support, perform these steps:

1. Complete all outstanding volume moves from and to bin-managed storage locations.
2. Run inventory management vital record processing or inventory management expiration processing to complete the confirmation of the volume moves.

3. Review user-written programs, REXX EXECs, and reports that contain information about bins. You might need to modify the programs and reports to incorporate information provided with extended bin support. In an RMMplex, your system-managed libraries must be connected to at least one system which either runs z/OS Version 1 Release 3 or higher or has APAR OW49863 installed.

EDGSPLCS file for the EDGUTIL utility

The EDGSPLCS file is written to during EDGUTIL VERIFY(SMSTAPE) processing. If this DD name is allocated, it causes EDGUTIL to create statements to be used with the EDGSPLCS utility. [Figure 168 on page 418](#) shows JCL for allocating the EDGSPLCS file during EDGUTIL processing:

```
//EDGSPLCS DD DISP=(,CATLG),UNIT=SYSALLDA,SPACE=(TRK,(1,1)),LRECL=80,RECFM=FB,  
// DSN=MY.SPLCS.DATA
```

Figure 168. Sample JCL for allocating the EDGSPLCS file during EDGUTIL processing

Return codes for EDGUTIL

EDGUTIL issues these return codes shown in [Table 55 on page 418](#).

Table 55. EDGUTIL return codes

Return Code	Explanation
0	All requested functions completed successfully.
4	DFSMSrmm encountered a minor error during processing. It issues a warning message and continues processing.
8	DFSMSrmm has encountered an error opening the control data set.
12	DFSMSrmm encountered a severe error during processing of one of the requested functions. DFSMSrmm stops the utility.
16	DFSMSrmm encountered a severe error during a required communication with the DFSMSrmm subsystem. DFSMSrmm stops the utility.

Using EDGSPLCS to issue commands to OAM for system-managed volumes

You can use the EDGSPLCS utility to issue supported commands to OAM for system-managed volumes. DFSMSrmm builds the input commands for this utility automatically during EDGUTIL VERIFY(SMSTAPE) processing and EDGHSKP EXPROC processing when you request them.

You can run multiple copies of EDGSPLCS. Using different parameters, EDGSPLCS can be processing in parallel for multiple libraries or multiple drives within a library, but this utility does not ensure that each parameter is different from any other currently running.

You need ALTER authority to the relevant volume catalog in order to use the EDGSPLCS utility to update the TCDB. For example, if you use just one volume catalog and use the default volume catalog prefix, you need ALTER access to SYS1.VOLCAT.VGENERAL

When the INDD control statement includes a 'Q' verify request, EDGSPLCS exploits a SYSTEMs level ENQ using *qname* SYSZRMM *rname* EDGSPLCS.*volser* to ensure that a volume in the list can only be processed by one instance of EDGSPLCS. Each instance goes down the list looking for volumes that match the execution parameters and selects the first volume that matches the criteria, and when the Q verify request is specified, issues an ENQ and if it cannot obtain the ENQ, moves to the next volume in the list. If an ENQ is obtained, it verifies that the volume is still in private status and issues CBRXLCS CUA (change use attribute to move the volume to scratch), then finally DEQs, and moves to the next volume in the list.

EXEC parameters for EDGSPLCS

Figure 169 on page 419 shows the EXEC parameters for EDGSPLCS.

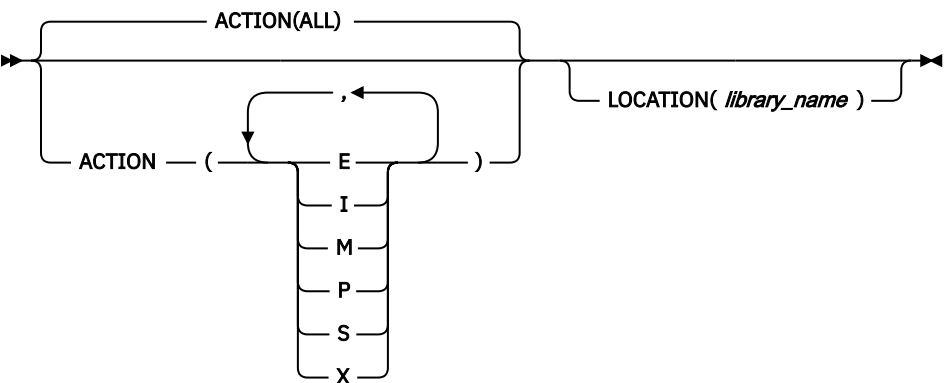


Figure 169. EDGSPLCS EXEC parameters

ACTION

Specifies that only the specified requests in the input file are processed. You can optionally provide the name of a library to restrict the processing to only those requests. You can specify one or more of the possible actions.

ALL

All actions.

E

Eject volume.

I

Import volume.

M

Manual cartridge entry.

P

Set to **P**rivate status.

S

Set to **S**cratch status.

X

EXport volume.

If you do not specify the ACTION parameter, the default value is ALL.

LOCATION

LOCATION(*library_name*) specifies the name of the system-managed library for which the EDGSPLCS utility will process commands during this run. By default, all locations are considered. However, you can select a subset based on the library name using this parameter. Because EDGSPLCS serializes the volumes for processing and ensures the volumes are in the correct status, you can use the same input file for different libraries by running multiple copies of EDGSPLCS in parallel.

INDD input file

Table 56 on page 420 displays an existing file of RECFM=F LRECL 80 containing control statements that direct the processing that EDGSPLCS utility performs.

The control statements take this format:

av	volser	options	library	message
12	4	11	27	36

Table 56. INDD input file for the EDGSPLCS utility

Symbol	Explanation	Values
a	Action character	One of these values: • E - Eject volume • I - Import volume • M - Manual Cartridge Entry • P - set to Private status • S - set to Scratch status • X - eXport volume
v	Verify request	One of these values: • Q - Serialize the volume for processing and ensure the volume is in the correct status • V - Verify volume is resident • blank - Do not verify volume is resident
volser	Volume serial	
options	Depends on action character	Action specific values starting in column 11: • S • P - Storage group name or blank followed by optional owner ID or blank. 'Owner ID' starts in column 19. • I - Cancel request. Specify C to cancel an existing import. Distributed library name to start an import in a specific library of a PtP VTS. • X - Cancel request. Specify C to cancel an existing export. Distributed library name to start an export in a specific library of a PtP VTS. • M - Library name into which the volume is to be entered. Column 20; media type of volume to be entered - for example; 5. • E - Eject destination. Either C (convenience) or B (high capacity).

<i>Table 56. INDD input file for the EDGSPLCS utility (continued)</i>		
Symbol	Explanation	Values
library	Library name	Starting in column 27, this is an eight character field for you to specify the library name. This field is used by the LOCATION execution parameter and is only required if LOCATION parameter is specified.
message	Output area for EDGSPLCS	After processing, this area contains a function specific message from the EDGSPLCS utility.

OUTDD output file

This is the output file that is written by the EDGSPLCS utility. It contains a copy of each of the input control statements, and each statement contains a completion message.

Return codes for EDGSPLCS

EDGSPLCS issues these return codes shown in [Table 57 on page 421](#).

<i>Table 57. EDGSPLCS return codes</i>	
Return Code	Explanation
0	All requested functions completed successfully. Refer to OUTDD records for individual messages from the input actions.
4	DFSMSrmm encountered a minor error during processing. Refer to OUTDD records for individual messages from the input actions.
8	At least one requested action was not supported. Refer to OUTDD records for individual messages from the input actions.
12	DFSMSrmm encountered a severe error during processing. DFSMSrmm stops the utility.

Sharing the DFSMSrmm control data set

When you use multiple DFSMSrmm systems, they can share the same control data set. You can use the DFSMSrmm ISPF dialog and RMM TSO subcommands to display all information that has been recorded in the control data set. When, however, you share a DFSMSrmm control data set where at least one system does not support system-managed tape libraries, there are restrictions on using the RMM TSO subcommands from DFSMSrmm on a non-system-managed tape system to add and change information in the control data set for system managed tape volumes.

Also see [Chapter 5, “Running DFSMSrmm with system-managed tape libraries,” on page 99](#) for additional considerations.

Running DFSMSrmm inventory management when sharing the control data set

Use the EDGHSKP utility to run inventory management activities that include: vital record processing, expiration processing, storage location management processing, backing up the control data set and

journal, and creating an extract data set. See [Chapter 13, “Performing inventory management,”](#) on page 327 for more information.

When you have at least one non-system-managed tape environment, you should perform inventory management using DFSMSrmm in a system-managed tape environment. DFSMSrmm ensures that the TCDB is updated with the correct volume status as volumes are returned to scratch, and that volume movement is automatically confirmed to a system managed tape library.

Running EDGINERS when sharing the control data set

If you have at least one non-system-managed tape environment, run EDGINERS with DFSMSrmm in a system-managed tape environment so that you can take advantage of the dynamic allocation capability of EDGINERS.

Defining volume information when sharing the control data set

When you have at least one non-system-managed tape environment, issue RMM TSO ADDVOLUME, DELETEVOLUME, and CHANGEVOLUME EJECT subcommands using DFSMSrmm in the system-managed tape environment so the TCDB is automatically updated.

Confirming volume movement when sharing the control data set

Run inventory management on the DFSMSrmm that is running in the system-managed tape environment. Use the RMM CHANGEVOLUME subcommand with these operands to confirm that pending volume movement has taken place:

```
RMM CHANGEVOLUME * CMOVE(from_location,to_location)
```

You can specify a system-managed library name as the *from_location* or *to_location* name when using DFSMSrmm even if system-managed tape is not in use.

When you run inventory management on the system managed tape system, the volume information in the control data set is updated to confirm that any volumes pending movement have been moved. If you run inventory management on the non-system managed tape system, your request does not fail but volume information is not updated.

Returning volumes to scratch when sharing the control data set

When volumes that reside in an IBM system managed tape library are returned to scratch by DFSMSrmm, information needs to be updated in the TCDB.

When you have some systems without system-managed tape active, ensure that inventory management runs on a system with system-managed tape active so that system-managed volumes returning to scratch are updated in the TCDB automatically during inventory management expiration processing.

Chapter 15. Initializing, erasing, and scanning tape volumes

DFSMSrmm samples provided in SAMPLIB

- EDGJINER Sample JCL for Using the EDGINERS Utility for Initializing and Erasing Tapes
- EDGLABEL Sample Started Procedure for Initializing, Erasing, and Scanning Tapes

Use EDGINERS to initialize, erase, and scan volumes. Run EDGINERS regularly as part of your inventory management processing.

EDGINERS writes BCD labels on 7-track tape volumes and ASCII (ISO/ANSI format) labels on tape cartridges or 9-track tape volumes. EDGINERS writes 7-track tape labels in even parity (translator on, converter off). You can label tape cartridges, 7-track tape volumes, or 9-track tape volumes with EDGINERS.

EDGINERS provides support for ISO/ANSI version 3 VOL1 and HDR1 labels and for ISO/ANSI version 4 VOL1 and HDR1 labels.

If you are authorized to change the label type, you can relabel tape volumes by specifying the label type in your JCL. This is done at the time of volume use to avoid a separate mount of the volume. When you change the volume label type at the time of use, you do not need to use either EDGINERS or the INIT action. DFSMSrmm allows the tape volume label to be created or overwritten at any time, regardless of the status of the volume, if the user has the required access to a security resource. See Chapter 9, [“Authorizing DFSMSrmm users and ensuring security,”](#) on page 215 for more information about volume labels.

See *z/OS DFSMSrmm Managing and Using Removable Media* for operator procedures that describe operator tasks like responding to initialization messages, tape mount messages, and using the LABEL procedure to request EDGINERS processing. See [“Using the LABEL procedure”](#) on page 459 for a description of the EDGLABEL procedure provided by DFSMSrmm.

Before initializing or erasing a volume, DFSMSrmm ensures that the correct volume is mounted by reading the volume label. For DFSMSrmm-defined volumes, it also ensures that the requested action is actually required.

When DFSMSrmm reads an existing volume label, the request might fail because the existing volume requires formatting of the servo tracks. If DFSMSrmm detects that a mounted volume has a servo track formatting error, DFSMSrmm issues message EDG6658I and fails the request to initialize or erase the volume.

Note: The recording technology and media associated with IBM new tape architecture products, such as 3590 and 3592, uses servo track. See *IBM 3590 High Performance Tape Subsystem Introduction and Planning Guide, GA32-0330* and *IBM TotalStorage Enterprise Tape 3592: Presentation Guide Introduction and Planning Guide, GA32-0464* for more information.

DFSMSrmm erases volumes using the hardware security erase feature when it is available. When erasing volumes, DFSMSrmm also reinitializes them so that the correct volume labels are written and the volumes are ready for reuse. If the hardware security erase feature is not available, DFSMSrmm overwrites volumes with a bit pattern of hex FF.

The EDGINERS SCAN function reads the VOL1 and header labels for the first file on the volume. If the labels exist and are read successfully, the label contents are displayed in the SYSPRINT file. For the operator command, the information is also displayed, but truncated, on the operator console. For SYSIN manual processing, the information is displayed in the SYSPRINT file. You can scan the labels of a volume regardless of whether it is defined to DFSMSrmm. If the volume is already defined to DFSMSrmm, the information read from the volume labels is compared with the information defined to DFSMSrmm. Any

undefined volume is not automatically defined to DFSMSrmm. See [“Using the LABEL procedure”](#) on page 459 for a description of the EDGLABEL procedure provided by DFSMSrmm.

Replacing IEHINITT with EDGINERS

You can use the DFSMSrmm EDGINERS utility or the IEHINITT utility to initialize and erase tape volumes.

Use the IEHINITT utility to initialize tapes you do not want to be defined to DFSMSrmm. Use the RMM CHANGEVOLUME subcommand to inform DFSMSrmm that the volume has been initialized. If you use tapes that are initialized using a utility other than EDGINERS and do not inform DFSMSrmm, DFSMSrmm can issue message EDG4026I at OPEN time.

EDGINERS, if you decide to use it, performs these tasks:

- Reads and validates volume labels.
- Maintains RACF profiles.
- Uses and updates information in the DFSMSrmm control data set.
- Provides a facility to erase tapes.
- Defines volumes you initialize or erase that are not yet defined in the control data set.
- Ends processing when two consecutive errors are detected on the same volume to prevent subsequent volumes from being used incorrectly when a cartridge loader is in use. For example, if an incorrect volume label is read, the volume is demounted and a new mount request issued. Processing is dependent on the operator response to DFSMSrmm message EDG6663D and the WRONGLABEL processing described in [“EXEC parameters for EDGINERS”](#) on page 426.
- Issues a WTOR to the operator when a mount request is issued. A reply to the WTOR is not always required as processing continues as soon as the volume is mounted. The WTOR is issued to allow the operator to skip a volume if the volume cannot be mounted for some reason.
- Bypasses any IOS000I messages for an 'NCA' error (tape not capable message) when reading the label on a volume that has not been initialized.
- In addition, EDGINERS can also be used to scan volume labels and compare the label information with the information defined to DFSMSrmm.

For information about preventing or limiting the use of IEHINITT, see [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,”](#) on page 215.

Using EDGINERS

You can use EDGINERS in either automatic processing mode and manual processing mode. Initialize and erase actions that are defined in the control data set drive automatic processing. With automatic processing, volumes are initialized and erased without operator intervention. SYSIN within JCL or operator commands drive manual processing. You might set up a job that initializes new volumes using automatic processing to minimize librarian intervention. Use manual processing for initializing volumes when you want to change a known, existing volume serial number to another volume serial number. Scan can be performed in manual mode only.

For volumes already defined to DFSMSrmm, the initialize and erase processing is dependent on the action being set for the volume. You can only erase volumes that have the ERASE release action pending, and you can only initialize volumes that have the INIT release action pending. These are some important things to remember:

- You can set the INIT action for any volume at any time, but be careful not to do this for the wrong volume.
 - To set the INIT action, issue this command: `RMM CV volser INIT(Y)`
- To set the INIT release action pending, set the INIT release action first and then when the volume is released, the INIT action is set pending.
 - To set the INIT release action pending, issue this command: `RMM CV volser RELEASEACTION(INIT)`

- To set the ERASE release action pending, set the ERASE release action first and then when the volume is released, the ERASE action is set pending.
 - To set the ERASE release action pending, issue these two commands:
 - RMM CV *volser* RELEASEACTION(ERASE)
 - RMM DV *volser* RELEASE
 - Be careful when you use this way to force the ERASE action. Once the volume is erased, there is no way to recover the data.

DFSMSrmm also provides these automated ways to set the ERASE release action:

- By the SECCLS parmlib option. When data sets are created, DFSMSrmm automatically sets the ERASE action for the volume based on the SECCLS parmlib option. See [“Defining security classes: SECCLS” on page 206](#) for additional information about the SECCLS parmlib option.
- By the RACF 'erase on scratch' attributes from the DATASET class profiles. This way is only available to you when the DEVSUPxx parmlib option TAPEAUTHDSN=YES is in use. When data sets are created, DFSMSrmm automatically sets the ERASE action for the volume based on the DATASET profile 'erase on scratch' attribute.

EDGINERS supports a TAPE DD that might have been allocated dynamically using the S99TIOEX, S99ACUCB, and S99DSABA options of dynamic allocation.

Initializing and erasing volumes automatically

To initialize and erase volumes without operator intervention, you can set up automatic processing by specifying any of these EDGINERS EXEC parameters: COUNT, INITIALIZE, ERASE, LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT. When you request automatic processing, DFSMSrmm does not process any SYSIN commands you specify.

During DFSMSrmm expiration processing, DFSMSrmm records the volumes that need to be initialized or erased when they are released or returned to scratch. If you use the EXEC parameters in your JCL to set up automatic processing, DFSMSrmm initializes or erases these volumes without operator intervention. EDGINERS issues write-to-operator messages and MSGDISP requests to the operator and the drive to get a volume mounted and demounted. If operators are unable to mount a volume, DFSMSrmm allows them to skip processing the current requested volume.

To initialize scratch volumes in a non-system-managed library that you are adding to DFSMSrmm, use the RMM ADDVOLUME subcommand with the INIT(Y) operand to mark the volumes you want initialized before they are available as scratch. Specify the INITIALIZE EXEC parameter in your EDGINERS JCL. DFSMSrmm initializes all the volumes marked as requiring initialization. See [“Initializing scratch volumes in system-managed libraries” on page 107](#) for information on initialization for volumes in a system-managed library.

Initializing,Erasing, and scanning volumes manually

Recommendation: Use manual processing with automatic cartridge loaders and volumes with old labels. DFSMSrmm performs manual processing that requires operator intervention when you do not specify any EXEC parameters that select automatic processing. These parameters are described in [“Initializing and erasing volumes automatically” on page 425](#). Specify commands in the SYSIN file or reply to operator messages to when you want operator intervention.

The only required SYSIN command operand is the volume serial number. When you specify the volume serial number, DFSMSrmm gets the rest of the information about the volume from the control data set.

If you supply MEDIANAME, POOL, or RACK operands on the INIT or ERASE SYSIN command that do not match the existing volume entry, DFSMSrmm issues an error message and stops processing the current volume.

Initializing and erasing volumes automatically using multiple tape drives

You can use multiple tape drives when initializing or erasing volumes by running one copy of EDGINERS for each tape drive that you wish to use. To run multiple copies of EDGINERS, submit multiple jobs or start multiple procedures. You can specify the same parameters or different parameters for each EDGINERS job you run. For example, you can split volumes between jobs by specifying the MEDIANAME operand, POOL operand, or the LOCATION operand on separate runs of EDGINERS to initialize different volumes. If you use the same parameters, each EDGINERS job you run shares the volumes that are to be initialized. During EDGINERS processing, DFSMSrmm serializes the use of the volume by using a SYSTEMS ENQ. Serializing the use of the volume ensures that no other invocation of EDGINERS attempts to initialize the volume. EDGINERS skips those volumes already processed or that are serialized. When you run multiple copies of EDGINERS with the same parameters, specify the BATCH(0) parameter to ensure that EDGINERS processing continues until all volumes are initialized or erased. Refer to [“EXEC parameters for EDGINERS”](#) on page 426 for information about the EDGINERS BATCH parameter and the EDGINERS COUNT parameter. Specify the COUNT(x) parameter if you are using the VERIFY parameter or if you are using cartridge loaders to control the batch size.

JCL for EDGINERS

[Figure 170 on page 426](#) shows sample JCL for automatic processing.

```
//INIT      EXEC  PGM=EDGINERS,  
//          PARM='MEDIANAME(3480),VERIFY'  
//SYSPRINT DD    program message data set  
//TAPE      DD    UNIT=(TAPE,,DEFER)
```

Figure 170. JCL for EDGINERS automatic processing

[Figure 171 on page 426](#) shows sample JCL for manual processing.

```
//INIT      EXEC  PGM=EDGINERS  
//SYSPRINT DD    program message data set  
//TAPE      DD    UNIT=(TAPE,,DEFER)  
//SYSIN     DD    optional control command input
```

Figure 171. JCL for EDGINERS manual processing

The SYSPRINT DD statement is always required. It contains all the messages issued by EDGINERS processing, even when processing uses the operator console for input. The messages issued by EDGINERS are normally written both to SYSPRINT and as a WTO message to the job log, console, and syslog. Some messages issued as WTO are truncated because of the message length limits, but the complete message text is available in the SYSPRINT file.

The TAPE DD statement is only required if any of the volumes to be processed are not in a system-managed tape library.

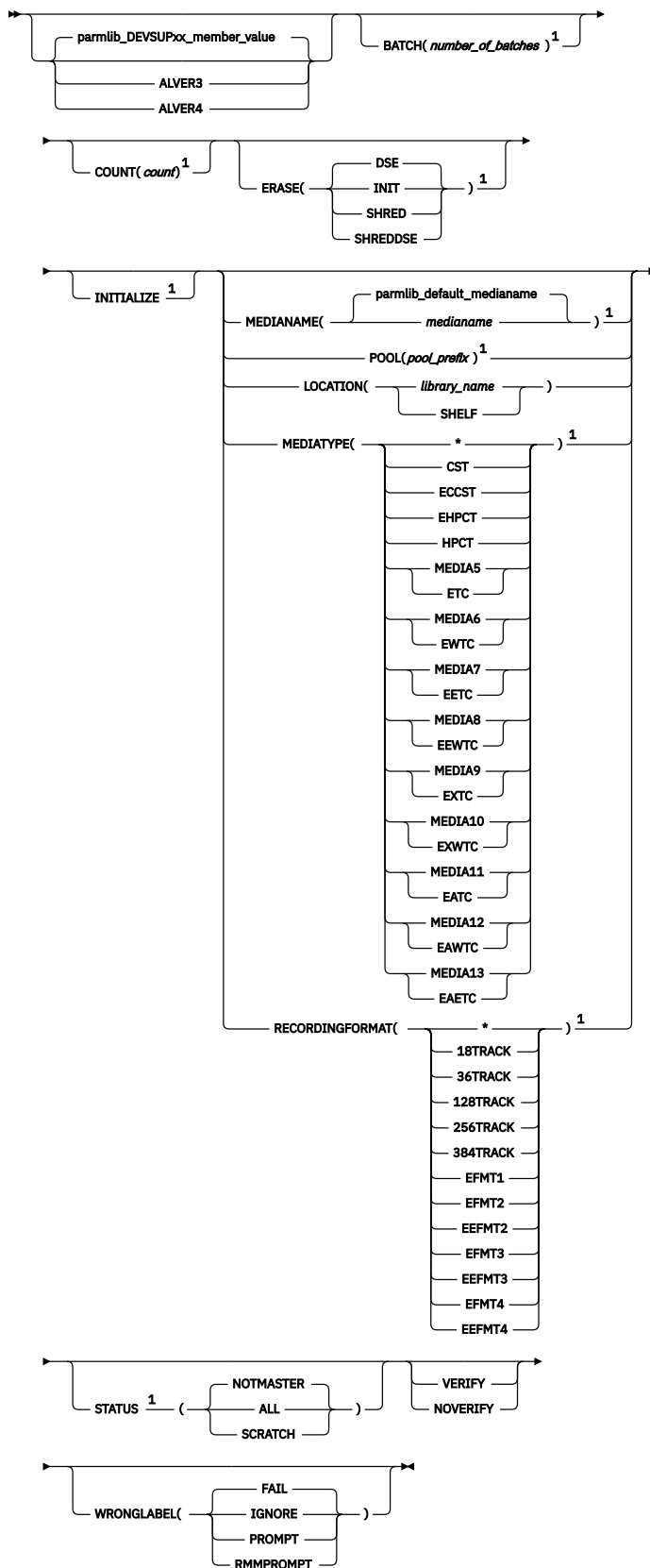
[Figure 172 on page 426](#) shows sample JCL for initializing volumes with ISO/ANSI version 4 tape labels. The EXEC JCL statement can be overridden by control information from SYSIN, operator replies to messages, or information from the DFSMSrmm control data set.

```
//INIT      EXEC  PGM=EDGINERS,PARM='...ALVER4,...'  
//SYSPRINT DD    SYSOUT=A  
//SYSIN     DD    DUMMY
```

Figure 172. JCL for initializing volumes with ISO/ANSI Version 4 VOL1 and HDR1 labels

EXEC parameters for EDGINERS

[Figure 173 on page 427](#) describes the EXEC parameters.



Notes:

¹ Specify this parameter for automatic processing.

Figure 173. EDGINERS EXEC parameters

ALVER3

Use ALVER3 to set the EDGINERS processing default value for ISO/ANSI label tapes to version 3. To understand how the DFSMSrmm assigns the label version, see [“How DFSMSrmm selects an ISO/ANSI label Version”](#) on page 441.

ALVER4

Use ALVER4 to set the EDGINERS processing default value for ISO/ANSI label tapes to version 4. To understand how the DFSMSrmm assigns the label version, see [“How DFSMSrmm selects an ISO/ANSI label Version”](#) on page 441.

BATCH(number_of_batches)

Use BATCH to specify the number of batches of volumes to be processed in a single run of EDGINERS automatic processing. Use the COUNT parameter to specify the number of volumes in each batch. The COUNT is the number of volumes that are initialized or erased before DFSMSrmm verifies the volumes. After DFSMSrmm verifies the volumes in a batch, EDGINERS starts again to initialize or erase the volumes in the next batch.

If you specified the NOVERIFY parameter, the number of volumes that are processed is the BATCH value or its default, multiplied by the value of COUNT or its default. However, DFSMSrmm does not batch the processing of these volumes.

The default for BATCH is BATCH(1). To process all volumes that have actions pending, specify BATCH(0). DFSMSrmm treats BATCH(0) as BATCH(X'FFFFFFFF'), which is the upper limit for the number of batches that DFSMSrmm can process.

COUNT(count)

Use COUNT to specify the number of volumes to initialize or to erase when DFSMSrmm performs automatic processing. Use COUNT with the BATCH parameter to specify the number of volumes in each batch of volumes to be processed. The maximum value that you can specify is 99. If automatic processing is in effect but COUNT is omitted, then the default value is 10. When you specify COUNT, DFSMSrmm performs automatic processing.

ERASE(DSE|INIT|SHRED|SHREDDSE)

Use ERASE to request that DFSMSrmm selects volumes that have the erase action pending. If automatic processing is in effect but ERASE is not specified then DFSMSrmm will only select volumes with the initialize action pending. When you specify ERASE, DFSMSrmm performs automatic processing.

ERASE can be specified either as ERASE or as **ERASE(value)**. Specifying ERASE without a value (that is, as ERASE) is the same as specifying ERASE (DSE). You can optionally specify one of the following values for ERASE to select the action to be performed by the tape drive:

DSE

Specifies that a Data Secure Erase (DSE) should be attempted. This exploits the tape drive hardware capability to erase data from the volume. This is the default value if the ERASE parameter is omitted or specified without a value.

INIT

Specifies that an ERASE action equates to an INIT action; no secure erase is attempted and the volume is relabelled as if the INIT action had been requested.

SHRED

For encrypted volumes, this value specifies that the Data Key should be made unusable by the drive. For non-encrypted volumes the DSE action is attempted.

SHREDDSE

For encrypted volumes, this value specifies that the Data Key should be made unusable by the drive, and that any non-encrypted residual data on the volume should be subject to DSE. For non-encrypted volumes, the DSE action is attempted.

INITIALIZE

Use INITIALIZE to request that DFSMSrmm selects volumes that have the initialize action pending. If automatic processing is in effect but neither INITIALIZE nor ERASE are specified, then INITIALIZE is the default. You can also specify INITIALISE for INITIALIZE. When you specify INITIALIZE, DFSMSrmm performs automatic processing.

LOCATION(library_name)

Use LOCATION to specify a subset of volumes for automatic processing. The *library_name* must be the name of a system-managed tape library that is on the running system or SHELF. If you specify LOCATION, you cannot specify MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT.

There is no default *library_name* value. If you do not specify LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT, DFSMSrmm uses MEDIANAME as the default parameter for automatic processing. This means that all volumes defined with the default medianame are selected if they have the required action pending.

MEDIANAME(medianame | parmlib_default_medianame)

Use MEDIANAME to specify a subset of volumes for automatic processing. If you specify MEDIANAME, you cannot specify LOCATION, MEDIATYPE, POOL, or RECORDINGFORMAT. If you do not specify LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT, DFSMSrmm uses MEDIANAME as the default parameter for automatic processing. This means that all volumes defined with the default medianame are selected if they have the required action pending.

DFSMSrmm does not use MEDIANAME to set a default for the SYSIN INIT and ERASE commands MEDIANAME operand.

The default MEDIANAME is the value that you define with the EDGRMMxx parmlib OPTION MEDIANAME operand described in [“Defining system options: OPTION”](#) on page 161.

MEDIATYPE(* | CST | ECCST | EHPCT | HPCT | MEDIA5 | MEDIA6 | MEDIA7 | MEDIA8 | MEDIA9 | MEDIA10 | MEDIA11 | MEDIA12 | MEDIA13)

Use MEDIATYPE to specify a subset of volumes for automatic processing. Specifies the volume's physical media type. Use one of these:

The volume is not a cartridge.

CST

Cartridge System Tape

ECCST

Enhanced Capacity Cartridge System Tape

EHPCT

Extended High Performance Cartridge Tape

HPCT

High Performance Cartridge Tape

MEDIA5/ETC

IBM TotalStorage Enterprise Tape Cartridge

MEDIA6/EWTC

IBM TotalStorage Enterprise WORM Tape Cartridge 3592

MEDIA7/EETC

IBM TotalStorage Enterprise Economy Tape Cartridge 3592

MEDIA8/EEWTC

IBM TotalStorage Enterprise Economy WORM Tape Cartridge 3592

MEDIA9/EXTC

IBM TotalStorage Enterprise Extended Tape Cartridge 3592

MEDIA10/EXWTC

IBM TotalStorage Enterprise Extended WORM Tape Cartridge 3592

MEDIA11/EATC

IBM Enterprise Advanced Tape Cartridge

MEDIA12/EAWTC

IBM Enterprise Advanced WORM Tape Cartridge

MEDIA13/EAETC

IBM Enterprise Advanced Economy Cartridge

When you specify MEDIATYPE, DFSMSrmm performs automatic processing. If you specify MEDIATYPE, you cannot specify LOCATION, MEDIANAME, POOL, or RECORDINGFORMAT.

There is no default MEDIATYPE value. If you do not specify LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT, DFSMSrmm uses MEDIANAME as the default parameter for automatic processing. This means that all volumes defined with the default medianame are selected if they have the required action pending.

POOL(*pool_prefix*)

Use POOL to specify a subset of volumes for automatic processing. A pool prefix is one-to-five alphanumeric, national, or special characters followed by an asterisk (*). The pool must be one that is defined to DFSMSrmm on the running system. If you specify POOL, you cannot specify LOCATION, MEDIANAME, MEDIATYPE, or RECORDINGFORMAT.

There is no default *pool_prefix* value. If you do not specify LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT, DFSMSrmm uses MEDIANAME as the default parameter for automatic processing. This means that all volumes defined with the default medianame are selected if they have the required action pending.

RECORDINGFORMAT(*| 18TRACK | 36TRACK | 128TRACK | 256TRACK | 384TRACK | EFMT1 | EFMT2 | EEFMT2 | EFMT3 | EEFMT3 | EFMT4 | EEFMT4)

Use RECORDINGFORMAT to specify a subset of volumes for automatic processing. RECORDINGFORMAT specifies the basic recording format for tape volumes.

An asterisk indicates that the format is unknown or that the volume is not a tape volume.

18TRACK

Data has been written to the volume in 18-track format. A recording format of 18TRACK is valid with MEDIATYPE(CST) and MEDIATYPE(ECCST) only.

36TRACK

Data has been written to the volume in 36-track format. A recording format of 36TRACK is valid with MEDIATYPE(CST) and MEDIATYPE(ECCST) only.

128TRACK

Data has been written to the volume in 128-track format. A recording format of 128TRACK is valid with MEDIATYPE(EHPCT) and MEDIATYPE(HPCT) only.

256TRACK

Data has been written to the volume in 256-track format. A recording format of 256TRACK is valid with MEDIATYPE(EHPCT) and MEDIATYPE(HPCT) only.

384TRACK

Data has been written to the volume in 384-track format. A recording format of 384TRACK is valid with MEDIATYPE(EHPCT) and MEDIATYPE(HPCT) only.

EFMT1

Data has been written to the volume in EFMT1 (enterprise format 1) recording format. A recording format of EFMT1 is valid with MEDIATYPE(MEDIA5), MEDIATYPE(MEDIA6), MEDIATYPE(MEDIA7), and MEDIATYPE(MEDIA8) only.

EFMT2

Data has been written to the volume in EFMT2 (enterprise format 2) recording format. A recording format of EFMT2 is valid with MEDIATYPE(MEDIA5), MEDIATYPE(MEDIA6), MEDIATYPE(MEDIA7), MEDIATYPE(MEDIA8), MEDIATYPE(MEDIA9), and MEDIATYPE(MEDIA10) only.

EEFMT2

Data has been written to the volume in EEFMT2 (enterprise encrypted format 2) recording format. A recording format of EEFMT2 is valid with MEDIATYPE(MEDIA5), MEDIATYPE(MEDIA6), MEDIATYPE(MEDIA7), MEDIATYPE(MEDIA8), MEDIATYPE(MEDIA9), and MEDIATYPE(MEDIA10) only.

EFMT3

Data has been written to the volume in EFMT3 (enterprise format 3) recording format. A recording format of EFMT3 is valid with MEDIATYPE(MEDIA5, MEDIA6, MEDIA7, MEDIA8, MEDIA9, and MEDIA10) only.

EEFMT3

Data has been written to the volume in EEFMT3 (enterprise encrypted format 3) recording format. A recording format of EEFMT3 is valid with MEDIATYPE(MEDIA5, MEDIA6, MEDIA7, MEDIA8, MEDIA9, and MEDIA10) only.

EFMT4

Data has been written to the volume in EFMT4 (enterprise format 4) recording format. A recording format of EFMT4 is valid with MEDIATYPE(MEDIA9, MEDIA10, MEDIA11, MEDIA12, and MEDIA13) only.

EEFMT4

Data has been written to the volume in EEFMT4 (enterprise encrypted format 4) recording format. A recording format of EEFMT4 is valid with MEDIATYPE(MEDIA9, MEDIA10, MEDIA11, MEDIA12, and MEDIA13) only.

There is no default RECORDINGFORMAT. If you do not specify LOCATION, MEDIANAME, MEDIATYPE, POOL, or RECORDINGFORMAT, DFSMSrmm uses MEDIANAME as the default parameter for automatic processing. This means that all volumes defined with the default medianame are selected if they have the required action pending.

STATUS

Use STATUS to control the kind of tapes that you want DFSMSrmm to initialize or erase. The default for STATUS is NOTMASTER. Specifying STATUS requests automatic processing.

ALL

EDGINERS processes all volumes that have the INITIALIZE or ERASE action pending.

NOTMASTER

EDGINERS processes all volumes in SCRATCH, USER, INIT, ENTRY or PENDING RELEASE status that have the INITIALIZE or ERASE action pending. EDGINERS does not process any volumes in MASTER status. NOTMASTER is the default.

SCRATCH

EDGINERS processes volumes in SCRATCH, INIT, ENTRY or PENDING RELEASE status that have the INITIALIZE or ERASE action pending. EDGINERS does not process any volumes in MASTER or USER status.

VERIFY|NOVERIFY

Use VERIFY to request that DFSMSrmm ask the operator to remount each volume that has been successfully erased or labeled. The volumes are requested in reverse order, and the volume labels read to ensure no operator errors have occurred, for example, a mismatch between the internal label and the external label.

For automatic processing VERIFY is the default. For manual processing NOVERIFY is the default.

WRONGLABEL

Use WRONGLABEL to specify the processing that DFSMSrmm performs when a wrong volume is mounted. WRONGLABEL processing does not apply to NL tapes. For NL tapes, DFSMSrmm issues the WTOR EDG6628A to obtain the volume serial number or rack number for the volume that has been mounted. You can use WRONGLABEL when you are running EDGINERS in automatic mode and manual mode.

FAIL

DFSMSrmm does not prompt the operator to accept a mounted volume that does not match the requested volume. The mount request is rejected, the volume demounted, and DFSMSrmm issues message EDG6661E or message EDG6662E. FAIL is the default.

IGNORE

When the wrong volume is mounted, DFSMSrmm does not issue any operator prompt. DFSMSrmm issues message EDG6661E or EDG6662E to log the relabeling and processing proceeds. This

is an extremely dangerous option and should be used with caution because any volume can be relabeled as long as the requested volume has the INIT action or is not defined to DFSMSrmm. Use of this option requires CONTROL access to RACF FACILITY class resource STGADMIN.EDG.INERS.WRONGLABEL.

PROMPT

When an incorrect volume label is detected by EDGINERS for the mounted volume, the operator is always prompted to confirm the processing to be performed. DFSMSrmm issues message EDG6661E or message EDG662E, followed by message EDG6663D. Processing continues according to the response to message EDG6663D. This option should be used with caution because any volume can be relabeled as long as the requested volume is either known to DFSMSrmm and has the INIT action, or is not known to DFSMSrmm. No additional authorization is required, other than the authorization required for running EDGINERS.

RMM PROMPT

When the volume serial number of the mounted volume that is defined to DFSMSrmm does not match the volume serial number of the requested volume, DFSMSrmm issues message EDG6663D to prompt the operator to confirm processing. If the magnetic volume serial number of the tape is not known to DFSMSrmm, initialization continues as if the tape had no magnetic label. If the volume is known to DFSMSrmm, DFSMSrmm issues messages EDG6662E and EDG6663D to prompt the operator or issues message EDG6661E to log the relabeling. Use this option when your installation has defined all its volumes to DFSMSrmm; otherwise caution is required. Use of this option requires UPDATE access to RACF FACILITY class resource STGADMIN.EDG.INERS.WRONGLABEL.

SYSIN commands for EDGINERS

Use the SYSIN commands for manual processing. You must use separate SYSIN commands for each volume you want initialized, erased, or scanned.

ERASE and INIT commands

The format of the SYSIN commands for erasing and initializing are shown in [Figure 174 on page 433](#).

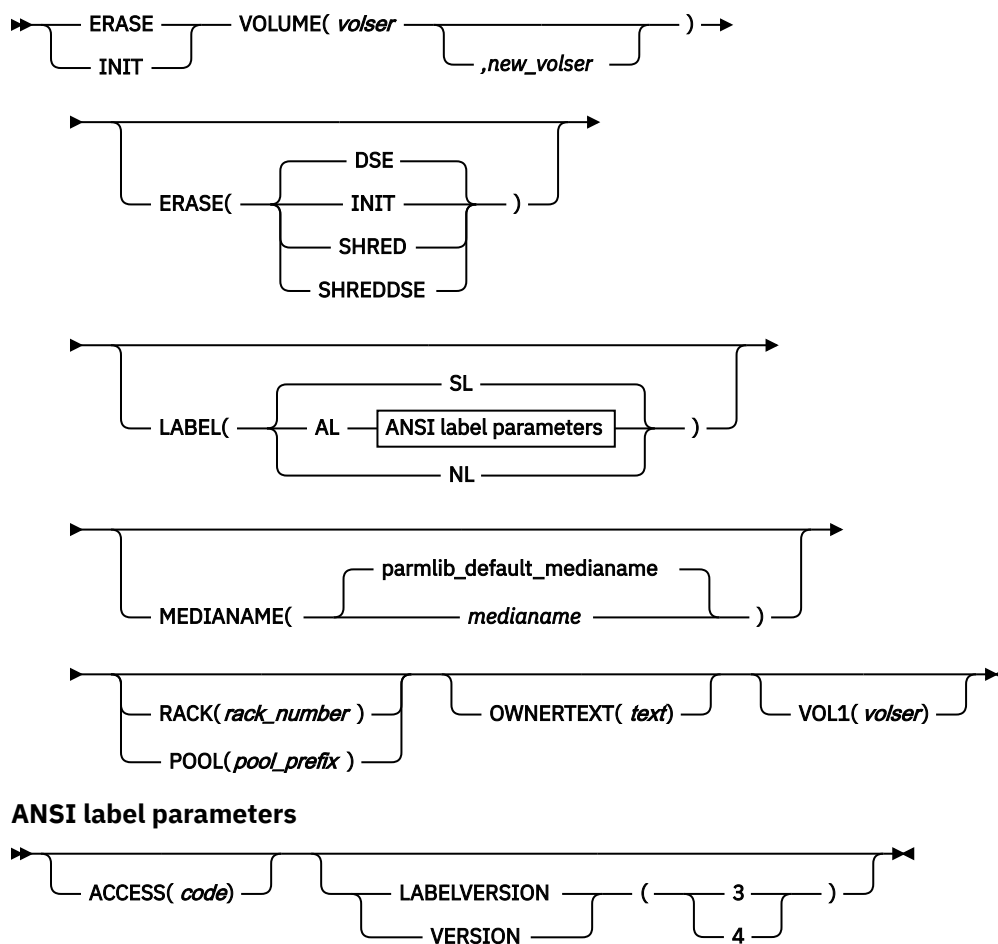


Figure 174. EDGINERS SYSIN commands (erasing and initializing)

ERASE

Specify ERASE to security erase a volume and write a new label on it.

You must specify either ERASE or INIT.

INIT

Specify INIT to initialize a volume.

You must specify either INIT or ERASE.

Note: DFSMSrmm ensures that the requested action is pending for the volume. If this action is not pending, DFSMSrmm fails the request. You can use RMM CV volser INITIALIZE(Y) to set the INIT action pending for any volume at any time regardless of the volume status, so be careful not to do this for the wrong volume, as the status of the volume is not checked (MASTER or NOTMASTER) during initialization using manual mode (SYSIN DD *).

VOLUME(volser,new_volser)

volser specifies the volume serial number of the volume to be initialized or erased. *volser* is required. *volser* is one to 6 alphanumeric, national, or special characters. Enclose it in single quotation marks if it contains any special characters. If you are adding volumes with a volume serial number less than six characters, you must supply a rack number or a pool, otherwise DFSMSrmm issues an error message.

If the volume is already defined in the DFSMSrmm control data set, DFSMSrmm ensures that the requested action is pending for the volume. If this action is not pending, DFSMSrmm fails the request.

If the volume mounted is already labeled, DFSMSrmm reads the label to ensure that the volume serial number matches the one you specify. If the volume mounted does not have a recognizable

volume label but contains data (no label tapes or nonstandard label tapes), DFSMSrmm issues a WTOR. The operator must reply to this message before DFSMSrmm can initialize or erase the volume.

If the volume is not defined in the DFSMSrmm control data set and you do not specify a new volume serial number, DFSMSrmm adds the volume to the control data set.

The value for the variable *new_volser* specifies a new volume serial number. Use it if you want to relabel a volume with a new volume serial number. If this new volume is already defined in the DFSMSrmm control data set, DFSMSrmm fails the request.

DFSMSrmm adds information about the new volume to the DFSMSrmm control data set, using information recorded for the volume you are replacing, and then deletes information about the volume you are replacing.

ACCESS(*code*)

Specifies the ISO/ANSI volume accessibility code. Specify *code* as any character in the ISO/ANSI X3.4–1986 character set. You must specify LABEL(AL) if you specify an accessibility code.

You must modify the volume access installation exit routine in z/OS to allow subsequent use of the volume if you specify ACCESS.

The default is blank, allowing unlimited access to the volume.

ERASE(DSE|INIT|SHRED|SHREDDSE)

Use the ERASE operand on an ERASE command to tailor the ERASE action to be performed.

You can optionally specify an operand value for ERASE to select the action to be performed by the tape drive for the ERASE action. The following values can be specified:

DSE

Specifies that a Data Secure Erase (DSE) should be attempted. This exploits the tape drive hardware capability to erase data from the volume.

INIT

Specifies that an ERASE action equates to an INIT action; no secure erase is attempted and the volume is relabelled as if the INIT action had been requested.

SHRED

For encrypted volumes, this value specifies that the Data Key should be made unusable by the drive. For non-encrypted volumes the DSE action is attempted.

SHREDDSE

For encrypted volumes, this value specifies that the Data Key should be made unusable by the drive, and that any non-encrypted residual data on the volume should be subject to DSE. For non-encrypted volumes, the DSE action is attempted.

When you do not specify the ERASE operand, ERASE(DSE) is the default.

LABEL(NL|SL|AL)

Use LABEL to specify the type of label to be written on the volume:

AL

Specifies an ISO/ANSI Label.

NL

Specifies no label.

SL

Specifies an IBM standard label.

If you do not specify the label type and the volume is already defined in DFSMSrmm, DFSMSrmm uses the label type defined in the DFSMSrmm control data set.

If you do not specify the label type and the volume is not already defined in the control data set, DFSMSrmm uses IBM standard label (SL) as the default.

LABELVERSION(3|4)

Use LABELVERSION to specify the ISO/ANSI volume label version. Specify LABELVERSION to update the DFSMSrmm control data set with the required label version for ISO/ANSI output tapes. To

understand how DFSMSrmm assigns a label version if you do not specify the LABELVERSION, see [“How DFSMSrmm selects an ISO/ANSI label Version” on page 441.](#)

3

Use 3 to initialize tape volumes with ISO/ANSI version 3 VOL1 and HDR1 labels

4

Use 4 to initialize tape volumes with ISO/ANSI version 4 VOL1 and HDR1 labels.

MEDIANAME(*medianame*)

Specifies the volume's media name.

The media name that you specify must match the media name defined in the control data set. If the media names do not match, DFSMSrmm fails the request.

If the volume is not defined in the DFSMSrmm control data set, DFSMSrmm uses the value that you specify when you add the volume. If you do not specify a media name, DFSMSrmm uses the value that you defined for your installation with the EDGRMMxx parmlib OPTION MEDIANAME operand.

The default is the EDGRMMxx parmlib OPTION MEDIANAME operand value.

OWNERTEXT(*text*)

Specifies the owner's name or similar identification. *text* is fourteen characters. Enclose in single quotation marks if it includes blanks or special characters. The text must be 10 bytes for SL, 14 bytes for AL.

The information is specified as character constants up to 10 bytes long for EBCDIC and BCDIC volume labels and up to 14 bytes long for volume labels written in ASCII.

POOL(*pool_prefix*)

Specifies a pool prefix for a pool to which you want to assign the volume. If the volume is not defined to DFSMSrmm, DFSMSrmm selects an available rack number for the volume in the pool you specify. If the volume is already defined in the DFSMSrmm control data set, DFSMSrmm changes the volume's rack number to move the volume.

If you do not supply a pool prefix or a rack number for a volume already defined in the DFSMSrmm control data set, DFSMSrmm uses the volume's existing rack number. If the volume is not defined in the control data set and you do not supply a pool prefix or a rack number, DFSMSrmm assigns the volume a rack number matching its volume serial number.

RACK(*rack_number*)

Specifies a shelf location for the volume. If you do not supply a pool ID or a rack number for a volume already defined in the control data set, DFSMSrmm uses the volume's existing rack number. If the volume is not defined in the DFSMSrmm control data set and you do not supply a pool ID or a rack number, DFSMSrmm assigns the volume a rack number matching its volume serial number.

VOL1(*volser*)

Use the VOL1 operand to specify the VOL1 label volser that is to be written in the tape label when it is required to be different than the external volser. The value for the variable *volser* is 1-to-6 alphanumeric, national, or special characters.

Volumes with a VOL1 value are treated as duplicate volumes by DFSMSrmm. If you are labeling a duplicate volume, DFSMSrmm uses the previously defined VOL1 value for the volume when you do not specify a VOL1 value for the volume. If you specify a VOL1 value, DFSMSrmm uses that VOL1 in place of the VOL1 value that was previously defined to DFSMSrmm. You can only label a volume as a duplicate when you are initializing or erasing volumes manually

For more information about tape label validation or volume access code, see [z/OS DFSMS Using Magnetic Tapes](#).

SCAN command

The format of the SYSIN commands for scanning are shown in [Figure 175 on page 436.](#)

➡ SCAN — VOLUME(*volser*) ➡

Figure 175. EDGINERS SYSIN command (for scanning)

SCAN

Specify SCAN to scan the labels of a volume.

You can scan the labels of a volume regardless of whether it is defined to DFSMSrmm. If the volume is already defined to DFSMSrmm the information read from the volume labels is compared with the information defined to DFSMSrmm. Any undefined volume is not automatically defined to DFSMSrmm.

The SCAN function reads the VOL1 and header labels for the first file on the volume. If the labels exist and are read successfully, the label contents are displayed in the SYSPRINT file. For the operator command the information is also displayed, but truncated, on the operator console. For SYSIN, manual, processing the information is displayed in the SYSPRINT file.

If the volume is defined to DFSMSrmm and the label information does not match that defined to DFSMSrmm the differences are highlighted in the output, see the example in Figure 4 File 1 output summaries below.

For each SYSIN command, you must specify one of; ERASE, INIT, and SCAN.

VOLUME

VOLUME(*volser*) specifies the volume serial number of the volume to be scanned. *volser* is required. *volser* is one to 6 alphanumeric, national, or special characters. Enclose it in single quotation marks if it contains any special characters.

You can scan the labels of any volume, including system-managed scratch volumes. Although the system prevents a specific mount of a system-managed scratch volume, you can scan them because EDGINERS handles the change of use attribute between SCRATCH and PRIVATE in the TCDB. As an alternative to relying on this EDGINERS processing, you can change a scratch volume to master status, then run EDGINERS with SCAN and release the volume after the scan. Depending on the results of the scan, you might also want to re-label the volume to clean up any mismatch between DFSMSrmm recorded information and the actual volume contents.

Volumes that are not standard label (such as IBM SL, and ANSI Label (AL)), and are not labeled NL by the EDGINERS utility, are treated as no label (NL). This can include true no label volumes, non-standard label (NSL), and those with a leading tape mark (TM).

SCAN output

The scan output is written to the SYSPRINT file for all SCAN commands. When EDGINERS is run from the console, it is also issued as a multiline message to the operator console. See [“WTO SCAN results messages” on page 437](#) for the format of truncated messages written by WTO.

```
EDG6679I SCAN RESULTS:
* * * * Device 0590, TAPE, VOLSER=A03503
VOL1 label = VOL1A035030 VSSI VTAPE
-----
Data set 0001          1...5...10...15...20...25...30...35...40...45...50...55...60...65...70...75...80
HDR1 label =          HDR1RMMTST.TRG.VOL2 A0350300010001 01002000000000000000IBM OS/VS 370
HDR2 label =          HDR2F0008000080000Z1DDAC11/TRG1 B 12120
* Tape mark
-----
          LBL volser Dsname                                Vseq Dseq Cdate  Jobname Step    RECF LRECL
BLKSZ
On vol  SL  A03503                                RMMTST.TRG.VOL2 0001 00001 2010/020 Z1DDAC11 TRG1    FB      80
80
          Mismatch(*)
RMM data SL  A03503                                RMMTST.TRG.VOL2 0001 00001 1999/032 Z1DDAC11 SOURCE    FB      80
80
EDG6683I MISMATCH ON Cdate Step
EDG6685I DATA SET ATTRIBUTES WERE PREVIOUSLY CHANGED USING COPYFROM
```

Figure 176. Example of scan output

The output summary lines (the last four lines of the example in [Figure 176 on page 436](#)) are displayed for standard label volumes to enable the label information and DFSMSRmm information to be compared. Standard label volumes can be either SL or AL, or EDGINERS created RMNL, with or without User labels. The differences are highlighted by an "*" in the appropriate column of the "Mismatch(*)" output line. If EDGINERS detects that data set attributes have been copied by a tape copy application, the message EDG6685I is issued to help the storage administrator understand why the mismatch is shown. See the topic [Handling volume discrepancies in z/OS DFSMSRmm Managing and Using Removable Media](#).

The output summary lines are included under these circumstances:

Heading line

When one or more of the following summary lines are required

On vol

When one or more labels were read from the volume (VOL1/HDR1/RMNL)

Mismatch(*)

When there is a difference between the volume labels and DFSMSRmm information

RMM data

When a volume record is available in DFSMSRmm. The data used is partly from the volume record and the remainder from the data set record for the first file if it exists.

WTO SCAN results messages

When EDGINERS is run from the console with a SCAN command entered from the console, the results of scanning are written to both the SYSPRINT file and a truncated set of results messages to the operator console.

```
EDG6682I SCAN RESULTS (TRUNCATED): FOR FULL DETAILS - REFER TO SYSPRINT
* * * * Device 0910, TAPE, VOLSER=A06640
VOL1A066400                                VSSI VTAPE
-----
1...5...10...15...20...25...30...35...40...45...50...55...60...65...70.
HDR1R.MFTAPE.DSN00001A0664000010001      01030700000000000000IBM OS/VS 3
HDR2F0008000080000RMMUSERJ/WRITMF P   B   12120
      * Tape mark
-----
LBL volser Dsname                                Vseq Dseq Crda
SL  A06640                                R.MFTAPE.DSN00001 0001 00001 2010
      Mismatch(*)
SL  A06640                                RMMUSER.MFTAPE.DSN00001 9999 00001 2010

EDG6683I MISMATCH ON Vseq
```

For more information, see the section [Scanning a tape volume label in z/OS DFSMSRmm Managing and Using Removable Media](#).

Label data comparison with DFSMSRmm data

Table 58 on page 437 shows the comparison between label data and DFSMSRmm data.

Table 58. Label data comparison with DFSMSRmm data						
Label ID	Position	Name	EDG6679I heading	SL	AL	RMM Extract Field Name
n/a	n/a	Label type	LBL	Y	Y	RVLABEL
VOL1	5	Volume Serial number	volser	Y	Y	RVVOLSER or RVVOL1 for duplicate volumes
HDR1	5	Data set identifier	Dsname	Y	Y	RVDSN1
HDR1	28	Volume sequence number	Vseq	Y	Y	RVVOLSEQ

Table 58. Label data comparison with DFSMSrmm data (continued)

Label ID	Position	Name	EDG6679I heading	SL	AL	RMM Extract Field Name
HDR1	32	Data set sequence number	Dseq	Y	Y	RVLABNO1
HDR1	42	Creation Date Format: cyyddd, century 0=20	Crdate	Y	Y	RDCRDATE
HDR2	SL(5, 37, 39) AL(5)	Record format	RECF	Y	Y ¹	RDRECFM
HDR2	6	Block length ²	BLKSZ	Y	Y	RDBLKSZ
HDR2	11	Record length	LRECL	Y	Y	RDLRECL
HDR2	18	Job identification	Jobname	Y	Y ¹	RDCRTJBN
HDR2	27	Job step identification	Step	Y	Y ¹	RDSTEPNM
HDR2	71	Large Block Length ³	BLKSZ	Y	N	RDBLKSZ

Note: Values for volume sequence number in HDR1 and DFSMSrmm data may be different. Volume sequence number in HDR1 indicates the order of the volume within the multivolume aggregate (set of volume serial numbers for one multivolume dataset), whereas the volume sequence number in DFSMSrmm indicates the sequence number of a volume in a multivolume set.

Using EDGINERS with system-managed tape libraries

With DFSMS tape support, you can associate tape drives with specific system-managed tape libraries. As a result, you can mount volumes resident in a system-managed tape library only on the drives associated with that library.

Checking volumes in system-managed tape libraries

EDGINERS performs this checking to make sure a volume can be used in a system-managed tape library:

- EDGINERS determines if a volume in a system-managed tape library can be mounted on the current system. If the volume cannot be mounted, possibly because it is defined in a TCDB on another system, DFSMSrmm skips that volume.
- In order to initialize a volume in a system-managed tape library, a volume must be in a private category because the automated tape library does not support specific mounts of scratch volumes. RMM TSO command and release processing attempts to ensure that the volume is in the correct category so that EDGINERS does not have to check the volume status. Refer to [“Setting status for a volume in a system-managed tape library” on page 440](#) for more information about setting the status for a volume.

Note: Although the system prevents a specific mount of a system-managed scratch volume, you can scan them because EDGINERS handles the change of use attribute between SCRATCH and PRIVATE in the TCDB.

¹ For AL tapes, these fields can only be fully compared to DFSMSrmm recorded information when the system code in the first header label is IBMZLA.

² Also see HDR2 position 71

³ Also see HDR2 position 6

- You must define a volume in a system-managed tape library to DFSMSrmm before you can initialize or erase it. Any volume not defined to DFSMSrmm will be mounted on the drive allocated by the TAPE DD statement in the JCL for EDGINERS as long as the drive is not in a system-managed library.

Requesting volume mounts for system-managed tape libraries

EDGINERS requests volume mounts only on those drives on which the volumes can be mounted, by testing volume eligibility and using dynamic allocation to obtain a suitable drive for each volume to be processed.

DFSMSrmm uses the pre-allocated drive if it is available. If the pre-allocated drive is not available, EDGINERS dynamically allocates a drive using the first volume obtained from the control data set. Further processing checks that the subsequent volumes are eligible for use on the same drive. If the pre-allocated drive is not in a system-managed tape library, DFSMSrmm uses it for all volumes selected that are not resident in a system-managed tape library.

When dynamic allocation of a tape drive fails, EDGINERS sets return code to 4 and skips processing of the current volume. Messages describing the failure are issued. These might include messages prefixed with IKJ and CBR. In most cases you will see a return and reason code returned from OAM. For example, OAM issues return code 8 and reason code 51 to indicate that the requested volume is in scratch status. This error might be the result of a mismatch between the DFSMSrmm control data set and TCDB volume status. You can correct this by updating the status in the TCDB to match that known by DFSMSrmm. Refer to [z/OS DFSMSdfp Diagnosis](#) for information about OAM return and reason codes.

Running EDGINERS on multiple system complexes

You can run EDGINERS on multiple system complexes to ensure that all required volumes are erased or initialized. To select a subset of volumes to initialize or erase, you can specify POOL and LOCATION with the EDGINERS EXEC parameter.

Running EDGINERS on a 3494 in manual mode

DFSMSrmm can determine the status of the vision system and if the library is in manual mode. EDGINERS can automatically handle some errors when an automated tape library is fully functional. For example, when the library is fully functional, DFSMSrmm uses the vision system volume serial number to verify that the correct volume is mounted.

Mounting and demounting volumes

EDGINERS provides a way to direct to an automated tape library the WTOR messages and MSGDISP requests to the operator and drive to get a volume mounted and demounted. These messages are issued through the library automation communication services component of OAM. If a mount on an automated tape library drive fails, DFSMSrmm uses the return and reason codes set by the library automation communication services component to test if a volume should be skipped.

During demount processing, DFSMSrmm ensures that errors detected on volumes mounted in an automated tape library are reflected in the TCDB. For example, DFSMSrmm ensures the TCDB contains information about write-protected, wrong volume, and wrong label type errors. DFSMSrmm skips the volume rather than having the operator correct the error.

Using DFSMSdfp processing to label volumes

For volumes in an automated tape library, you have the option to use DFSMSdfp OPEN processing as an alternative to using EDGINERS to label scratch volumes.

If the automated tape library is fully functional (vision system working) and the VOL1 label for a scratch volume does not match the external label, DFSMSdfp rewrites the VOL1 label with the correct volume serial number.

DFSMSrmm defers the initialization of the volumes to DFSMSdfp if you request the initialization prior to entering a scratch volume into the automated tape library. DFSMSrmm turns off the initialize action.

For example you use the RMM ADDVOLUME subcommand to define scratch volumes to DFSMSrmm as shown in [Figure 177 on page 440](#). Specify INIT(Y) to request that the volume is initialized prior to first use. If you later enter the volumes into an automated tape library, during entry processing DFSMSrmm turns off the initialize action to defer the labeling to OPEN processing under DFSMSdfp control.

```
RMM ADDVOLUME A12345 INIT(Y) STATUS(SCRATCH)
```

Figure 177. Defining scratch volumes to be initialized

If you request that the initialize action is set for a scratch volume that is already resident in the automated tape library, as shown in [Figure 178 on page 440](#) using the RMM CHANGEVOLUME subcommand, DFSMSrmm does not defer the initialization to DFSMSdfp. You must use EDGINERS to label the tape before it can be used.

```
RMM CHANGEVOLUME A12345 INIT(Y)
```

Figure 178. Changing the initialization action for a volume

Setting status for a volume in a system-managed tape library

DFSMSrmm maintains volume status in the TCDB for volumes that are defined in the DFSMSrmm control data set. The status is updated using the RMM TSO commands or when volumes are released. Use the DFSMSrmm EDGRMMxx parmlib OPTION command SMSTAPE operand to control when the TCDB is updated. If the TCDB is not updated, the request to initialize or erase a volume in a system-managed tape library might fail because the volume is in a scratch library category. If DFSMSrmm is still running in record-only mode, use the AMS ALTER VOLUMEENTRY command to change the volume status. If DFSMSrmm is running in any other mode, you can use the RMM TSO commands to correct the volume status.

To set the correct status for the volume, issue two commands. Issue this command to complete the failed request.

```
RMM CHANGEVOLUME A12345 CONFIRMRELEASE(INIT)
```

Then issue this command to request that the volume be initialized.

```
RMM CHANGEVOLUME A12345 INIT(Y)
```

Labeling new tape volumes with EDGINERS

When you use the EDGINERS utility to label new tape volumes, EDGINERS reads the VOL1 tape label header of any volume that is to be initialized. When DFSMSrmm reads the VOL1 tape label header for new volumes, this can result in IOS000I messages with NCA (Not Capable). EDGINERS identifies the sense information and initializes the volume without further checking or intervention.

When IOS returns the message "Format incompatible" for volumes that do not have readable header information, DFSMSrmm cannot determine if the correct volume is mounted. DFSMSrmm issues message EDG6656E and message EDG6661E. In message EDG6661E, DFSMSrmm displays *FMTIC, which is a dummy volume serial number used by EDGINERS.

```
EDG6656E FORMAT OF VOLUME M00021 IS NOT COMPATIBLE WITH CURRENT DEVICE
EDG6661E INCORRECT VOLUME MOUNTED ON DEVICE 0281 - REQUESTED VOLUME M00021
MOUNTED VOLUME *FMTIC.
```

To ensure that new tape volumes are labeled, you can use the EDGINERS WRONGLABEL parameter described in [“EXEC parameters for EDGINERS” on page 426](#) to specify what DFSMSrmm does when it encounters a new tape volume with unreadable header information. Use the WRONGLABEL(RMMPROMPT) parameter if you want DFSMSrmm to prompt the operator for a reply or the WRONGLABEL(IGNORE) parameter when you want EDGINERS to continue without intervention.

Using the automatic cartridge loader with EDGINERS

You can use cartridge loaders with the EDGINERS utility to automate the mounting of volumes that are to be erased or labeled. To use cartridge loaders set to automatic mode, do not pre-mount volumes. Mount the volumes after EDGINERS issues the first mount message because DFSMSrmm processing depends on the mount message, which is not issued when the cartridge loader is set to automatic mode and if a volume is already loaded.

If you use brand new volumes or volumes that might cause I/O errors, Automatic Volume Recognition (AVR) can reject premounted volumes at allocation time. AVR processing is not under DFSMSrmm control. AVR dismounts the premounted and readied volumes when an I/O error is detected and issues message IEF503I.

When you use cartridge loaders with EDGINERS, you can optionally use the COUNT operand to specify how many volumes you plan to load into the cartridge loader. When you use the VERIFY operand, specify the COUNT operand to enable the batch of volumes to be reloaded for verify processing before any other volumes are processed. Use the BATCH operand to specify how many batches of volumes you want to process in a single run of EDGINERS.

Controlling access to EDGINERS

You should control access to EDGINERS because it can overwrite previously labeled tapes regardless of expiration date and security protection. DFSMSrmm helps you prevent unauthorized users from using EDGINERS by using the RACF security resource STGADMIN.EDG.OPERATOR. Only users with access to STGADMIN.EDG.OPERATOR can use EDGINERS.

For more information about using STGADMIN.EDG.OPERATOR, see [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,” on page 215.](#)

How DFSMSrmm selects an ISO/ANSI label Version

You can specify an ISO/ANSI label version, or can use the system default to label tape volumes with ISO/ANSI labels.

You can specify the volume label version in several different ways. EDGINERS uses this selection order for each volume in order to determine the label version, then uses the first value that it finds to assign the label version to the volume.

1. The LABELVERSION parameter in SYSIN command of EDGINERS
2. The REQUIREDLABELVERSION in the DFSMSrmm control data set volume record
3. The EDGINERS EXEC parameter ALVER3 or ALVER4
4. The parameter ALVERSION() in the parmlib DEVSUPxx member
5. The system default is 3

Producing label symmetry

If you initialize an ISO/ANSI label using EDGINERS, the labels do not frame an empty or null data set as required for interchange. To produce a label symmetry that meets ISO/ANSI standards, at least a minimal open and close sequence must be performed. For example, a volume previously initialized with EDGINERS results in label symmetry when you use the data set utility IEBGENER before the volume leaves the system for interchange, as shown in [Figure 179 on page 441.](#)

```
//STEP1 EXEC PGM=IEBGENER
//SYSUT1 DD DUMMY,DCB=(RECFM=F,BLKSIZE=80,LRECL=80)
//SYSUT2 DD DSN=DUMMY,UNIT=(tape,,DEFER),LABEL=(,AL),
           DCB=(RECFM=F,BLKSIZE=80,LRECL=80)
//SYSIN DD DUMMY
```

Figure 179. Using IEBGENER

How EDGINERS processing works

Before initializing or erasing a tape volume, DFSMSrmm verifies that the correct volume is mounted by reading any existing label, obtaining the volume serial number from the tape library vision system, or prompting the operator to confirm that the external label is correct. EDGINERS labels tape volumes as follows:

- For standard label volumes, DFSMSrmm creates a standard volume label, an 80-byte dummy header label, and a tape mark.

A standard volume label with a serial number you specify and a blank security byte. The format of the ISO/ANSI label is constructed either for Version 3 or Version 4, depending on the options, of the label standard. For a complete description of IBM standard volume labels and ISO/ANSI Version 3 volume labels, see [z/OS DFSMS Using Magnetic Tapes](#).

An 80-byte dummy header label. For IBM standard labels, this header consists of the characters HDR1 followed by zeros. For ISO/ANSI labels, this header consists of the characters HDR1 followed by zeros, with these exceptions:

- Position 54, which contains an ASCII space
 - A 1 in the file section, file sequence, and generation number fields
 - A leading space in the creation and expiration date fields
 - A system code of IBMZLA, followed by 13 spaces, to identify the operating system creating the label
- For tapes with no label, DFSMSrmm writes an 80-byte record that is not recognized as any valid label format, however, the DFSMSrmm utility can recognize it as an unlabeled tape once it is initialized.

The first time a tape that is labeled by EDGINERS is used for output, these sequence of events occurs:

1. The tape mark that EDGINERS created is overwritten.
2. The dummy header label that EDGINERS created is filled in with operating system data and device-dependent information.
3. A HDR2 record, containing data set characteristics, is created.
4. User header labels are written if exits to user label routines are provided.
5. A new tape mark is written.
6. Data is placed on the receiving volume.

When relabeling a volume defined to DFSMSrmm, DFSMSrmm uses the information recorded about the old volume as part of the new volume information recorded in the control data set.

This information is taken from the old volume record: Expiration Date, Density, Use Count, Store Id, Bin Number, Old Bin Number, Loan Location, Previous Location, Last Read and Write dates, Assigned Date and Time, Owner, Status, Label Type, Release actions, Actions Pending (excluding initialize and erase), Volume access and accessors, Unit name, Rack number, Temporary / Permanent Read / Write error counts.

When a volume that resides in an automated system-managed tape library is rejected at OPEN time because the tape media servo tracks require formatting, DFSMSrmm updates the volume information in the control data set. DFSMSrmm sets the volume INIT action to indicate that the volume must be initialized. If EDGINERS is used to relabel a volume and servo tracks have not been formatted, DFSMSrmm cannot initialize the volume.

Return codes for EDGINERS

EDGINERS issues one of the return codes shown in [Table 59 on page 442](#).

Table 59. EDGINERS return codes	
Return Code	Explanation
0	All requested functions completed successfully.

Table 59. EDGINERS return codes (continued)

Return Code	Explanation
4	DFSMSrmm encountered a minor error during processing. It issues a warning message and continues processing.
8	DFSMSrmm has stopped at least one requested function. It continues processing the next requested function.
12	DFSMSrmm encountered a severe error during processing of one of the requested functions. DFSMSrmm stops the utility.
16	DFSMSrmm encountered a severe error during a required communication with the DFSMSrmm subsystem. DFSMSrmm stops the utility.

EDGINERS examples

These examples illustrate some of the uses of EDGINERS. To use the examples, replace the **tape** specified in the examples with actual device numbers or esoteric unit names, unless your installation has the required device numbers defined to the esoteric name 'TAPE'. The actual device numbers and esoteric unit names depend on how your installation has defined the devices to your system. See [z/OS DFSMS Using Magnetic Tapes](#) for devices supported by this utility.

Example 1: Write IBM standard labels on three tapes

Figure 180 on page 443 is a manual processing example. Serial numbers 001234, 001235, and 001236 are placed on three tape volumes. The labels are written in EBCDIC at 800 bits per inch. Each volume labeled is mounted, when it is required, on a single 9-track tape unit. You must specify SYSIN commands for each volume you want to label, as [Figure 180 on page 443](#) shows.

```
//LABEL1  JOB  ...
//STEP1   EXEC PGM=EDGINERS
//SYSPRINT DD  SYSOUT=A
//TAPE    DD  DCB=DEN=2,UNIT=(tape,1,DEFER)
//SYSIN   DD  *
INIT VOLUME(001234) LABEL(SL)
INIT VOLUME(001235) LABEL(SL)
INIT VOLUME(001236) LABEL(SL)
/*
```

Figure 180. Writing IBM standard labels on three tapes

Control statement description:

- TAPE DD defines the tape unit used in the labeling operation.
- SYSIN DD defines the control data set, which follows in the input stream.
- INIT specifies that the tapes are to be labeled.

Example 2: Write an ISO/ANSI label on a tape

Figure 181 on page 444 is a manual processing example. Serial number 001001 is placed on one ISO/ANSI tape volume; the label is written at 800 bits per inch. The volume labeled is mounted, when it is required, on a 9-track tape unit. You must specify SYSIN commands for each volume you want to label, as [Figure 181 on page 444](#) shows.

```
//LABEL2 JOB ...
//STEP1 EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=A
//TAPE DD DCB=DEN=2,UNIT=(tape,1,DEFER)
//SYSIN DD *
INIT VOLUME(001001) LABEL(AL)
/*
```

Figure 181. Writing an ISO/ANSI label on a tape

Control statement description:

- TAPE DD defines the tape unit to be used in the labeling operation.
- SYSIN DD defines the control data set, which follows in the input stream.
- INIT specifies the serial number and label type for the volume that is being labeled.

Example 3: Place two groups of serial numbers on six tape volumes

Figure 182 on page 444 is a manual processing example. Two groups of serial numbers (001234, 001235, 001236, and 001334, 001335, 001336) are placed on six tape volumes. The labels are written in EBCDIC at 800 bits per inch. Each volume labeled is mounted, when it is required, on a single 9-track tape unit. You must specify SYSIN commands for each volume you want to label, as [Figure 181 on page 444](#) shows.

```
//LABEL3 JOB ...
//STEP1 EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=A
//TAPE DD DCB=DEN=2,UNIT=(tape,1,DEFER)
//SYSIN DD *
INIT VOLUME(001234) LABEL(SL)
INIT VOLUME(001235) LABEL(SL)
INIT VOLUME(001236) LABEL(SL)
INIT VOLUME(001334) LABEL(SL)
INIT VOLUME(001335) LABEL(SL)
INIT VOLUME(001336) LABEL(SL)
/*
```

Figure 182. Numbering tape volumes

Control statement description:

- TAPE DD defines the tape unit to be used in the labeling operation.
- SYSIN DD defines the control data set, which follows in the input stream.
- INIT specifies that the tapes are to be labeled.

Example 4: Place serial numbers on eight tape volumes

Figure 183 on page 445 is a manual processing example. Serial numbers 001234, 001244, 001254, 001264, 001274, and so on in a sequence, are placed on eight tape volumes. The labels are written in EBCDIC at 800 bits per inch. Each volume labeled is mounted, when it is required, on a single 9-track tape unit. You must specify SYSIN commands for each volume you want to label, as [Figure 183 on page 445](#) shows.

```
//LABEL4 JOB ...
//STEP1 EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=A
//TAPE DD DCB=DEN=2,UNIT=(tape,1,DEFER)
//SYSIN DD *
INIT VOLUME(001234) LABEL(SL)
INIT VOLUME(001244) LABEL(SL)
INIT VOLUME(001254) LABEL(SL)
INIT VOLUME(001264) LABEL(SL)
INIT VOLUME(001274) LABEL(SL)
INIT VOLUME(001284) LABEL(SL)
INIT VOLUME(001294) LABEL(SL)
INIT VOLUME(001304) LABEL(SL)
/*
```

Figure 183. Placing serial numbers on eight tape volumes

Control statement description:

- TAPE DD defines the tape unit used in the labeling operation.
- SYSIN DD defines the control data set, which follows in the input stream.
- The INIT statements specify the tapes to be labeled.

Example 5: Relabel a volume

Figure 184 on page 445 is a manual processing example for relabeling a volume. Supply both the current and the new volume serial number in your SYSIN statement. In the example the volume that is currently labeled as CURR01 is relabeled to NEW001.

If volume CURR01 is already defined to DFSMSrmm, DFSMSrmm defines a new volume entry NEW001 to the DFSMSrmm control data set using information from the existing volume entry CURR01 and deletes the existing volume entry. If volume CURR01 is not defined to DFSMSrmm, DFSMSrmm defines a new volume entry NEW001 in the control data set.

```
//INIT EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(tape,,DEFER)
//SYSIN DD *
INIT VOLUME(CURR01,NEW001) LABEL(SL)
```

Figure 184. Relabeling a volume

Control statement description:

- TAPE DD defined the tape unit to be used in the labeling operation.
- INIT specifies the current and new tape labels.

Example 6: Automatically initialize or erase 3480 volumes

Figure 185 on page 445 is an automatic processing example. EDGINERS requests that DFSMSrmm scan its control data set for the first ten 3480 volumes waiting to be initialized or erased.

```
//LABEL5 JOB ...
//STEP1 EXEC PGM=EDGINERS,PARM='INITIALIZE,ERASE,COUNT(10)'
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(3480,,DEFER)
```

Figure 185. Automatically initialize or erase 3480 volumes

Control statement description:

- TAPE DD defines the tape unit used in the labeling operation.

Example 7: Automatically initialize and erase volumes in a system-managed library

In Figure 186 on page 446 an automatic run of EDGINERS is scheduled to find, initialize, and erase up to 20 volumes residing in an automated tape library called MYATL. All tape cartridges will be labeled as appropriate for the drive type on which they are mounted, and for their current media characteristics. Because SHRED is specified, the Data Key will be made unusable by the drive for encrypted volumes. For non-encrypted volumes, the DSE action is attempted.

```
//LABEL6 JOB ....
//STEP1 EXEC PGM=EDGINERS,
//      PARM='COUNT(20),LOCATION(MYATL),INITIALIZE,ERASE(SHRED)'
//SYSPRINT DD SYSOUT=A
```

Figure 186. Automatically initialize and erase volumes in a system-managed library

Control statement description:

- TAPE and SYSIN DD are not required.
- PARM values request automatic processing with the default of VERIFY of all labeled volumes.

Example 8: Automatically initialize 50 scratch enhanced capacity cartridges

In Figure 187 on page 446, an automatic run of EDGINERS is used to initialize 50 scratch enhanced capacity cartridges defined to DFSMSrmm in the previous job step, and initialize them for use in a non-system managed tape library. This example assumes that no other volumes in the pool need to be initialized.

```
//LABEL7 JOB ...
//STEP1 EXEC PGM=IKJEFT01
//SYSTPRINT DD SYSOUT=A
//SYSTSIN DD *
      RMM ADDVOLUME A00100 STATUS(SCRATCH) INIT(Y) -
      COUNT(50) LOCATION(SHELF) MEDIATYPE(ECCST)
/*
//STEP2 EXEC PGM=EDGINERS,
//      PARM='COUNT(50),POOL(A*),INITIALIZE,NOVERIFY'
//SYSPRINT DD SYSOUT=A
//TAPE DD UNIT=(tape,1,DEFER)
```

Figure 187. Initialize 50 scratch enhanced capacity cartridges

Control statement description:

- SYSTSIN DD includes commands to define required volumes.
- TAPE DD allocates a drive capable of labeling enhanced capacity cartridges.

Example 9: Manually erase a volume

In Figure 188 on page 446, volume 0A7100 is erased and defined to DFSMSrmm. The volume is successfully erased and the volume label is written as an ISO/ANSI label.

```
//LABEL8 JOB ....
//STEP1 EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=A
//TAPE DD UNIT=(tape,,DEFER)
//SYSTSIN DD *
      ERASE VOLUME(0A7100) LABEL(AL) ERASE(SHREDDSE)
/*
```

Figure 188. Erase a volume

Control statement description:

- TAPE DD allocates a drive suitable for this volume.

- SYSIN DD includes the command requested to erase the volume.

Example 10: Automatically initialize volumes using multiple tape drives

Figure 189 on page 447 shows how to run multiple copies of EDGINERS to automatically label volumes driven by initialize actions in the DFSMSrmm control data set. Two jobs are provided but you can run one for each tape drive that you want to use. Change the jobname for each copy.

```
//LABEL10A JOB .....
//AUTOINIT EXEC PGM=EDGINERS,PARM='INITIALIZE,BATCH(0)',
// 'NOVERIFY,MEDIANAME(CARTS)'
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(tape,,DEFER)
//
//LABEL10B JOB .....
//AUTOINIT EXEC PGM=EDGINERS,PARM='INITIALIZE,BATCH(0)',
// 'NOVERIFY,MEDIANAME(CARTS)'
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(tape,,DEFER)
```

Figure 189. Initialize volumes using multiple tape drives

Control statement description:

- TAPE DD allocates a drive suitable for this volume.

Example 11: Manually labeling duplicate volumes using EDGINERS

You can label volumes as duplicate volumes using the EDGINERS utility that uses manual processing, with the VOL1 operand on the command in the SYSIN file, as shown in the example in Figure 190 on page 447. The VOL1 value is written to the tape label.

```
//MANUAL EXEC PGM=EDGINERS
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(tape,,DEFER)
//SYSIN DD *
INIT VOLUME(D000001) LABEL(SL) VOL1(ABC123)
/*
```

Figure 190. Labeling duplicate volumes using EDGINERS

Control statement description:

- TAPE DD allocates a drive suitable for this volume.

Example 12: Selecting EHPCT volumes for processing automatically

You can select volumes for processing as shown in the example in Figure 191 on page 447.

```
//INIT EXEC PGM=EDGINERS,
// PARM='MEDIATYPE(EHPCT),NOVERIFY,BATCH(0),STATUS(NOTMASTER)'
//SYSPRINT DD SYSOUT=*
//TAPE DD UNIT=(TAPEEH,,DEFER)
```

Figure 191. Selecting volumes for automatic processing

Control statement description:

- MEDIATYPE(EHPCT) is used to select only those volumes that are EHPCT media. EDGINERS processing is performed without operator intervention by using the DFSMSrmm control data set as input.
- BATCH(0) ensures that all volumes that need to be processed are processed.
- STATUS(NOTMASTER) is used to select only scratch volumes and volumes that are pending release.
- TAPE DD uses your installation's unit names to select a suitable drive on which to mount EHPCT volumes. Change the TAPEEH value to the value required for your installation. If all tapes are system-

managed, you can remove the TAPE DD, because DFSMSrmm dynamically allocates the drives that are required and the TCDB media type ensures the correct drives are allocated.

Example 13: Manually scanning a system managed DFSMSrmm volume

You can scan volumes as shown in the example in [Figure 192 on page 448](#). Volume S10017 is defined to DFSMSrmm and is resident in a system managed library. The example shows a manual processing run of EDGINERS with the SCAN command in SYSIN.

```
//LABEL8 JOB ....  
//STEP1 EXEC PGM=EDGINERS  
//SYSPRINT DD SYSOUT=*  
//SYSIN DD *  
SCAN VOLUME(S10017)  
/*
```

Figure 192. Scanning a system managed volume

Control statement description:

- TAPE DD is not required because EDGINERS dynamically allocates a suitable drive for a system managed volume.
- SYSIN DD includes the command request to scan the volume.

Chapter 16. Customizing DFSMSrmm

This topic helps you customize DFSMSrmm to best meet your installation's needs.

Changing the initial entry point to the DFSMSrmm dialog

Normally, when you enter the DFSMSrmm ISPF dialog, the first panel you see is the Primary Option Menu. You can change this initial entry point so tape librarians always see the Librarian Menu when they enter DFSMSrmm or general users see the DFSMSrmm User Menu.

To change the initial entry point, use the RMMISPF exec with one of its optional parameters which changes the panel navigation to go directly to a lower level. [Figure 193 on page 449](#) shows the operands you can use with the RMMISPF exec.

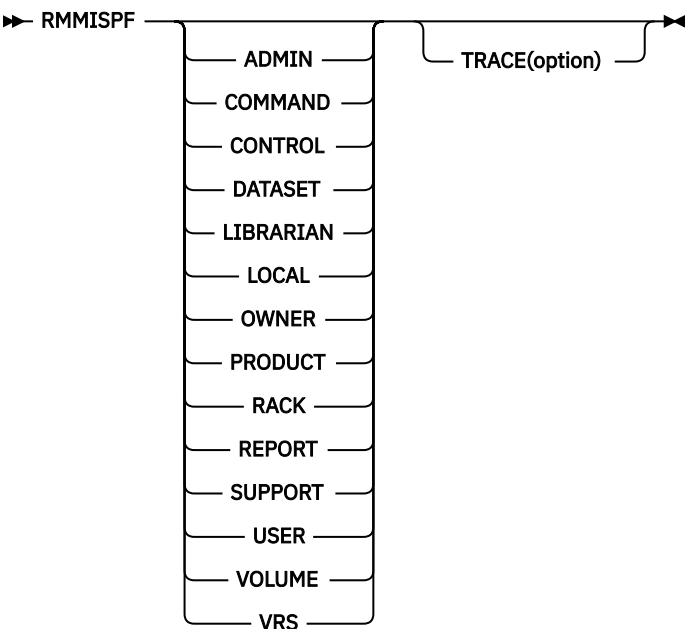


Figure 193. RMMISPF exec syntax diagram

Each of the operands, except for TRACE, represents a specific user or function menu from which you can request functions. Use the TRACE(option) operand to diagnose problems in any of the REXX execs supplied in the dialog. For more information about using TRACE, see [z/OS DFSMSrmm Diagnosis Guide](#).

To start the librarian at the Librarian Menu, add this line to the ISPF selection panel as shown in [Figure 194 on page 450](#).

```
% R +DFSMSrmm - Librarian dialog
```

Although the example shows the ISRUTIL panel, you can use any other selection panel to make the changes.

```

%----- UTILITY SELECTION MENU -----
%OPTION  ==>_ZCMD
%
% 1 +LIBRARY - Compress or print data set. Print index listing.
+          Print, rename, delete, or browse members
% 2 +DATASET - Allocate, rename, delete, catalog, uncatalog, or
+          display information of an entire data set
% 3 +MOVE/COPY - Move, copy, or promote members or data sets
% 4 +DSLIST - Print or display (to process) list of data set names
+          Print or display VTOC information
% 5 +RESET - Reset statistics for members of ISPF library
% 6 +HARDCOPY - Initiate hardcopy output
% 8 +OUTLIST - Display, delete, or print held job output
% 9 +COMMANDS - Create/change an application command table
% 10 +CONVERT - Convert old format menus/messages to new format
% 11 +FORMAT - Format definition for formatted data Edit/Browse
% 12 +SUPERCE - Compare data sets (Standard dialog)
% 13 +SUPERCE - Compare data sets (Extended dialog)
% 14 +SEARCH-FOR - Search data sets for strings of data
% R +DFSMSrmm - Librarian dialog
)INIT
  .HELP = ISR30000
)PROC
  &ZSEL = TRANS( TRUNC (&ZCMD, '.')
    1, 'PGM(ISRUDA) PARM(ISRUDA1)'
    2, 'PGM(ISRUDA) PARM(ISRUDA2)'
    3, 'PGM(ISRUMC)'
    4, 'PGM(ISRUDL) PARM(ISRUDLP)'
    5, 'PGM(ISRURS)'
    6, 'PGM(ISRUHC)'
    8, 'PGM(ISRUOLP)'
    9, 'PANEL(ISPUCMA)'
    10, 'PGM(ISRQCM) PARM(ISRQCMP)'
    11, 'PGM(ISRFMT)'
    12, 'PGM(ISRSSM)'
    13, 'PGM(ISRSEPRM) NOCHECK'
    14, 'PGM(ISRSFM)'
    R, 'CMD(%RMMISPF LIBRARIAN) NEWAPPL(EDG)'
    , , ,
    *, '?' )
  &ZTRAIL = .TRAIL
)END

```

Figure 194. Adding DFSMSrmm Librarian Option to ISPF

Adding local dialog extensions

You can add your own local extensions to the DFSMSrmm ISPF dialog. DFSMSrmm has a dummy panel, EDGP@LCL, that provides easy access to local dialog extensions. You can use these extensions without having to modify DFSMSrmm. Use these steps to add your extensions:

1. Create a selection menu, EDGP@LCL, that includes the extensions you want to add.
2. For each extension, create the required procedures and panels.
3. Make the local menu available to ISPF in a panel library so that it is selected ahead of the menu DFSMSrmm supplies.
4. Start the DFSMSrmm ISPF dialog.
5. Select Option 6 (LOCAL). You should see the modified local selection menu you just created.

You do not have to limit the local extensions to functions that use DFSMSrmm commands and utilities. Add any function that is useful to DFSMSrmm users, such as the tape librarian and storage administrator. For example, you could add:

- Service level reporter charts on tape activities
- A link to ISMF or RACF dialogs
- Lists of DFSMSShm volumes to compare with DFSMSrmm definitions
- A volume loan facility that enforces local standards for location names

When including RMM TSO subcommands, use the REXX procedure language instead of CLIST. RMM TSO subcommands set REXX variables so you do not need to trap and interpret the command output. Refer

to the REXX procedures DFSMSrmm supplies for examples of using the subcommands and receiving variables. Do not modify the supplied execs because they can change from one release to the next.

For more information on using the RMM TSO subcommands within REXX and the variables each subcommand issues, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

Customizing the local dialog with 'U' line command

You can customize the dummy panel, EDGP@LCL, to add facilities to the ISPF dialog. The 'U' line command allows you locally-provided line command support and calls the EDGRCLCL exec. A set of variables are saved in the variable pool for use by the EDGRCLCL exec. You can use this exec to implement any local extensions to the search results line commands. If the exec does not exist, for example, the installation has not provided one, one of these error messages is displayed: U line command not in use or You must use the EDGRCLCL exec to implement the U line command.

The set of variables saved for use by the EDGRCLCL exec depends on the search results list being processed. Each of the fields in the current row of the table are available as well as the table name, an indication of what type of search the results are for, the current row, and other table settings that might be required by the exec. The exec does not need to process all rows of the table, only the current row. It is called for each row of the table for which the 'U' line command has been specified.

Customize point-and-shoot fields in the DFSMSrmm dialog

To easily see the fields that are enabled for point-and-shoot, you must customize the color, intensity and highlighting of the point-and-shoot fields. Issue the ISPF system command PSCOLOR from any ISPF command line and adjust the Point-and-Shoot Panel Element as desired. Refer to [z/OS ISPF User's Guide Vol II](#) for additional information.

Changing the ADD product volume defaults

When you use the DFSMSrmm ISPF dialog to define product volumes to DFSMSrmm, the dialog issues either the RMM TSO CHANGEVOLUME or ADDVOLUME subcommand.

When ADDVOLUME is used, the dialog has some hard-coded default values set for RETPD and RELEASEACTION. You can modify the dialog EXEC EDGRPADV to change the values to ones that suit your installation.

The EDGRPADV dialog default is:

```
command = edgcmd" ADDVOLUME "edg@vol" STATUS(MASTER)" ,
            "NUMBER('"edgraddq(edg@pnum)"') LEVEL("edg@ver")",
            "FEAT("edg@fcd")",
            "MEDIANAME('"edgraddq(edg@medn)"')",
            "LOCATION("edg@loc") RETPD(90) RELEASE(RETURN)"
```

To modify the values that the EDGRPADV EXEC uses, install your changes using an SMP/E USERMOD after editing a copy of the EDGRPADV EXEC. The only modifications you should make are to either add new operands to the ADDVOLUME command, or to change the values set for the MASTER, RETPD, and RELEASE operands.

Customizing DFSMSrmm messages for report titles and user notification

DFSMSrmm provides messages for report titles and user notification. You can customize them by performing these tasks:

1. Updating the text of a message in the DFSMSrmm message table EDGMTAB.
2. Applying changes to EDGMTAB by creating an SMP/E-installable USERMOD.

Customizing DFSMSrmm report titles

You can customize the title text on DFSMSrmm reports. For example, you can add your company's name to the bottom of each report page.

To customize, update the text of a message in the DFSMSrmm message table EDGMTAB. The report utilities get the message text from the message table and use it when creating the report trailer lines. [Table 60 on page 452](#) shows the message numbers DFSMSrmm uses for each report utility:

Table 60. Customizing report titles

Utility	Message Number	Used For
EDGAUD	5839	security report trailer 1
EDGAUD	5840	security report trailer 2
EDGAUD	5846	audit report trailer 1
EDGAUD	5847	audit report trailer 2
EDGHSKP	2203	EDGHSKP REPORT file header
EDGRPTD	5825	trailer line 1

To customize the message text, edit the message text in the EDGMTAB source, supplied with the product. You could change trailer 1 for EDGRPTD, as shown in these examples. [Figure 195 on page 452](#) shows the text before modification.

```
EDGMSG 5825,TYPE=I,          X
      '          ',          X
      MOD=NO
```

Figure 195. Before modifying the Trailer 1 for EDGRPTD

[Figure 196 on page 452](#) shows the text after modification.

```
EDGMSG 5825,TYPE=I,          X
      'Warwick Corporation',  X
      MOD=NO
```

Figure 196. After modifying the Trailer 1 for EDGRPTD

The report utility centers the text before printing.

Customizing notification messages and notes

With DFSMSrmm you can set up automatic notification to owners by setting a parmlib option and defining owner electronic address to DFSMSrmm. Refer to [Chapter 8, “Using the parmlib member EDGRMMxx,” on page 143](#) for information on setting up automatic notification and controlling message text case.

DFSMSrmm provides a series of messages in the DFSMSrmm message module, EDGMTAB. You can modify them to provide specific information to users:

Messages EDG2405 through EDG2409

Notify users that volumes they own are eligible for release.

Messages EDG2700 through EDG2713

Create a note you send to product owners when new volumes for a product are entered into the library.

When modifying messages, you should:

- Keep message variables in the same order as in the original message.
- Use the same number of substitution characters that is shown in the original message text.
- Keep the messages in the same order they are displayed.

- Do not delete any messages from EDGMTAB. If you do not want to use a particular message, leave the message text blank.

Modifying text for release notification

Modify messages 2405-2409 to change the message text to notify users that their volumes are eligible for release.

You can substitute the message text without restriction for all the messages except message 2406. For message 2406, you must specify the secondary currency character X'4A' to represent substitution characters for variables that DFSMSrmm supplies. The secondary currency character shown in [Figure 197 on page 453](#) is ¢. You must also keep the variables in the order shown in [Figure 197 on page 453](#). The &NAM variable shown in message 2405 is replaced with the product name and need not be specified.

[Figure 197 on page 453](#) shows the messages in EDGMTAB before modification.

```
EDGMSG 2405,TYPE=I, X
  'Subject: &NAM volume expiration', X
  MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2406,TYPE=I, X
  'Volume ¢¢¢¢¢ assigned to owner ¢¢¢¢¢¢¢ on ¢¢¢¢¢¢¢¢¢ X
  at ¢¢:¢¢:¢¢', X
  MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2407,TYPE=I, X
  'is now pending release.', X
  MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2408,TYPE=I, X
  'If you wish the volume to be retained, please take immeX
  diate action.', X
  MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2409,TYPE=I, X
  'You can use the dialog functions or the RMM CHANGEVOLUMX
  E TSO command.', X
  MOD=NO,MSGID=NO
```

Figure 197. Notify owner messages

[Figure 198 on page 453](#) shows the notification text that results from [Figure 197 on page 453](#).

```
Subject: DFSMSrmm volume expiration
Volume LAUREN assigned to owner KRISTINE on 1998/01/01 at 09:45:11
is now pending release.
If you wish the volume to be retained, please take immediate action.
You can use the dialog functions or the RMM CHANGEVOLUME TSO COMMAND.
```

Figure 198. Default notification text

[Figure 199 on page 454](#) shows the same messages after modification.

```

EDGMSGB 2405,TYPE=I, X
'Subject: Bld 88 Tape Volume Expiration X
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2406,TYPE=I, X
'Volume ##### assigned to owner ##### on ##### X
at ##:##:##', X
MOD=YES,MSGID=NO
SPACE 2
EDGMSGB 2407,TYPE=I, X
'has now expired. Please come to the Computer Room Tapex
Library Window', x
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2408,TYPE=I, X
'to pick up the tape. Contact the tape librarian at 555x
-5796 for any', x
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2409,TYPE=I, X
'questions. DO NOT REPLY to this automated system messax
ge. Thanks. ', x
MOD=NO,MSGID=NO
SPACE 2

```

Figure 199. Modified messages

Figure 200 on page 454 shows the text that results from [Figure 199 on page 454](#).

```

Subject: Bld 88 Tape Volume Expiration.
Volume LAUREN assigned to owner KRISTINE on 1998/01/01 at 09:45:11
has now expired. Please come to the Computer Room Tape Library Window
to pick up the tape. Contact the tape librarian at 555-5796 for any
questions. DO NOT REPLY to this automated system message. Thanks.

```

Figure 200. Modified notification text

Modifying text in notes to product owners

DFSMSrmm provides note text for notifying product owners under these conditions:

- A new software product volume is added to DFSMSrmm.
- A change is made to add a volume to a product.
- The software level of a product volume is changed.

You can modify the note by modifying messages EDG2700 through EDG2713. [Figure 201 on page 454](#) shows an example of a note produced by modifying EDG2700 through EDG2713.

```

Subject: Volume PP0001 has been added for software product 5647-A01
A volume has been added on 1999/12/27 at 20:19:31 to a DFSMSrmm
software product which you own:

```

```

Product Number = 5694-A01 Level = V01R01M00
Name           = z/OS Version 1.1
Description    = includes rmm

```

Volume	Rack	Feature Code	Description
PP0001	PP1233	5678	z/OS V1.1 (volume 1 of 1)

Figure 201. Notifying product owner

Here are the messages you can modify. The messages must be displayed in sequence to produce the note. Several messages consist of blank lines so you have flexibility in the note you send. You can also line up the columns in your message text as shown in message EDG2710.

```

EDGMSGB 2700,TYPE=I, X
'Subject: Volume ##### has been added for software prodX
uct #####', X
MOD=YES,MSGID=NO

```

```

SPACE 2
EDGMSG 2701,TYPE=I,
'A volume has been added on ##### at :::: to a
&NAM',
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2702,TYPE=I,
'software product which you own:',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2703,TYPE=I,
',
',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2704,TYPE=I,
' Product Number = ##### Level = VcRcM',
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2705,TYPE=I,
' Name = #####',
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2706,TYPE=I,
' Description = #####',
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2707,TYPE=I,
',
',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2708,TYPE=I,
' Volume Rack Feature Code Description',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2709,TYPE=I,
' -----X
-----',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2710,TYPE=I,
' #####
#####',
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2711,TYPE=I,
',
',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2712,TYPE=I,
',
',
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2713,TYPE=I,
',
',
MOD=NO,MSGID=NO

```

Modifying notify messages

When you use Internet electronic mail addresses with NOTIFY processing, DFSMSrmm uses a different set of messages to create the notification message. This enables you to use a different message format for Internet mail than for non-Internet mail. For example, you can use HTML tags in the Internet mail message to select attributes such as color and font of the messages. You can also select the form of electronic mail used at the individual user level and use the appropriate message format. When you customize messages, ensure that you do not exceed a message length of 80 characters. Your individual messages are merged and flowed subject to the HTML tags once the e-mail message is sent.

See [Figure 202 on page 456](#) for an example of customizing message numbers 2450-2463 for release notification.

```

SPACE 2
EDGMSGB 2450,TYPE=I,
'HELO $$$$$$', host name substituted
MOD=YES,MSGID=NO
SPACE 2
EDGMSGB 2451,TYPE=I,
'MAIL FROM:<RMM@YOURMVS>'), edit in required id
MOD=YES,MSGID=NO
SPACE 2
EDGMSGB 2452,TYPE=I,
('RCPT TO:<',63C'$','>'), internet email substituted
MOD=YES,MSGID=NO
SPACE 2
EDGMSGB 2453,TYPE=I,
'DATA',
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2454,TYPE=I,
'Subject: &NAM volume expiration',
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2455,TYPE=I,
'CONTENT-TYPE: TEXT/HTML',
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2456,TYPE=I,MOD=YES,MSGID=NO,
('Volume <FONT COLOR=RED>$$$$$</FONT> ',
'assigned to owner<b>$$$$$$$</b> ')
SPACE 2
EDGMSGB 2457,TYPE=I,MOD=YES,MSGID=NO,
'on $$$$$$$$ at $:$:$'
SPACE 2
EDGMSGB 2458,TYPE=I,MOD=NO,MSGID=NO,
('is now pending release.<p>',
'If you wish the volume to be retained, please ')
SPACE 2
EDGMSGB 2459,TYPE=I,MOD=NO,MSGID=NO,
('<font size="+2" color=blue><b>take immediate action ',X
'</font></b>.<br>')
SPACE 2
EDGMSGB 2460,TYPE=I,MOD=NO,MSGID=NO,
('You can use the <b>dialog functions</b> ',X
'or the <b>RMM CHANGEVOLUME</b>')
SPACE 2
EDGMSGB 2461,TYPE=I,MOD=NO,MSGID=NO,
'TSO command.'
SPACE 2
EDGMSGB 2462,TYPE=I,
',', Period to end the data
MOD=NO,MSGID=NO
SPACE 2
EDGMSGB 2463,TYPE=I,
'QUIT',
MOD=NO,MSGID=NO

```

Figure 202. Customizing message numbers 2450-2463 for release notification

See [Figure 203 on page 456](#) for an example of this message.

```

From: <RMM@YOURMVS>
Date/Time: 02/28/2014 09:18AM

To: Undisclosed-recipients;
cc:
Subject: DFSMSrmm volume expiration

```

```

Volume VOLSER assigned to owner OWNER001 on 2013/333 at 07:14:13 is now pending release.
If you wish the volume to be retained, please take immediate action.
You can use the dialog functions or the RMM CHANGEVOLUME TSO command.

```

Figure 203. Example of customizing message numbers 2450-2463 for release notification

Note: 'VOLSER' would be shown in the color red and 'take immediate action' would be shown in the color blue.

Here is an example of customizing message numbers 2720-2739 for product notification.

```
SPACE 2
EDGMSG 2720,TYPE=I,
'HELO $$$$$$', host name substituted X
MOD=YES,MSGID=NO X
SPACE 2
EDGMSG 2721,TYPE=I, X
'MAIL FROM:<RMM@YOURMVS>', edit in required id X
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2722,TYPE=I, X
('RCPT TO:<',63C'$','>'), internet email substituted X
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2723,TYPE=I, X
'DATA', X
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2724,TYPE=I, X
'CONTENT-TYPE: TEXT/HTML ', X
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2725,TYPE=I,MOD=YES,MSGID=NO, X
('Subject: Volume $$$$ ', X
'has been added for software product $$$$')
SPACE 2
EDGMSG 2726,TYPE=I,MOD=YES,MSGID=NO, X
('<FONT FACE=COURIER>', X
'A volume has been added on $$$$ at $:.$:.$ ')
SPACE 2
EDGMSG 2727,TYPE=I,MOD=YES,MSGID=NO, X
'to a &NAM software product which you own :</FONT> <p>'
SPACE 2
EDGMSG 2728,TYPE=I,MOD=YES,MSGID=NO, X
('<table border="0"><tr><td><b>Product Number</b>', X
'</td><td>$$$$</td></tr>')
SPACE 2
EDGMSG 2729,TYPE=I, X
'<tr><td><b>Level</b></td><td>V$$$M$</td></tr> ', X
MOD=YES,MSGID=NO
SPACE 2
EDGMSG 2730,TYPE=I,MOD=YES,MSGID=NO, X
('<tr><td><b>Name</b></td>', X
'<td>$$$$$$$$$$$$$$$$</td></tr> ')
SPACE 2
EDGMSG 2731,TYPE=I,MOD=YES,MSGID=NO, X
('<tr><td><b>Description</b></td>', X
'<td>$$$$$$$$$$$$$$$$</td></tr> ')
SPACE 2
EDGMSG 2732,TYPE=I, X
'</table><p>', X
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2733,TYPE=I, X
'<table border="1"> ', X
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2734,TYPE=I,MOD=NO,MSGID=NO, X
('<tr><td><b>Volume</b></td><td><b>Rack</b></td>', X
'<td><b>')
SPACE 2
EDGMSG 2735,TYPE=I,MOD=NO,MSGID=NO, X
('Feature Code</b></td>', X
'<td><b>Description</b></td></tr>')
SPACE 2
EDGMSG 2736,TYPE=I,MOD=YES,MSGID=NO, X
('<tr><td>$$$$</td><td>$$$$</td>', X
'<td>$$$$</td>')
SPACE 2
EDGMSG 2737,TYPE=I,MOD=YES,MSGID=NO, X
('<td>$$$$$$$$$$$$$$$$</td></tr>', X
'</table>')
SPACE 2
EDGMSG 2738,TYPE=I, X
',', Period to end message X
MOD=NO,MSGID=NO
SPACE 2
EDGMSG 2739,TYPE=I, X
```

The results of this customization would look like this:

From: <RMM@YOURMVS>
Date/Time: 02/28/2014 01:52PM

To: Undisclosed-recipients;
cc:
Subject: Volume VOLSER has been added for software product PPPPPPP

A volume has been added on 2013/321 at 08:22:12 to a DFSMSrmm software product that you own:

Product Number	\$\$\$\$\$\$\$
Level	V\$\$\$\$M\$
Name	\$\$\$\$\$\$\$\$\$\$\$\$\$
Description	\$\$\$\$\$\$\$\$\$\$\$\$\$

Managing VM tape volumes

DFSMSrmm samples provided in SAMPLIB

- EDGCLIBQ Sample Exec to Create Reports for VM Tape Volumes
- EDGJVME Sample JCL for Creating Reports for VM Tape Volumes

DFSMSrmm provides SAMPLIB members to help you manage VM tapes. The sample, EDGJVME, is a batch job that creates a list of volumes that are flagged for VM use. EDGJVME gets the information from the extract data set. The job sends this list to the relevant VM system.

When you have received the list on the VM system, you can use the sample, EDGCLIBQ. EDGCLIBQ is an exec that produces information about the volumes on the list, including shelf location, owner, and users that are allowed to access the volumes.

DFSMSrmm provides the LIBQ exec to help you query information about VM volumes. Use the LIBQ exec to get the location where a volume resides. This example describes one scenario for using the LIBQ exec:

1. A user requests a volume to be mounted on VM by sending a message to the operator.
2. The operator issues the LIBQ exec, with the volume serial number requested as 'LIBQ volser'.
3. The LIBQ exec returns the rack number where the volume resides to the operator.
4. The operator decides whether to mount the volume as requested by the user.
5. The operator attaches the drive with the volume mounted, as requested.

To use the LIBQ exec to find a scratch volume, modify the exec to search for scratch volumes in the file it processes. You can tailor the rules you want to use for 'in use' or scratch volumes, or modify the exec and use it as part of your VM tape automation. You might also tailor the process to work with other non-z/OS platforms.

Replenishing scratch volumes in a system-managed library

DFSMSrmm sample provided in SAMPLIB

- EDGXPROC Sample to Replenish Scratch Volumes in a System-Managed Library
- EDGSETT Sample that you can use in IBM Tivoli Workload Scheduler for z/OS in place of EDGXPROC.

When a tape library detects a low-on-scratch condition, where more scratch volumes are needed, OAM issues WTO messages CBR3660A, CBR3792E, and CBR3794A. DFSMSrmm intercepts these messages and starts the procedure you define with the SCRATCHPROC value in parmlib. See [“Defining system options: OPTION” on page 161](#) for more information about specifying SCRATCHPROC. You must run

DFSMSrmm with a scratch procedure. You can modify the DFSMSrmm supplied sample, EDGXPROC, to support your location procedures. You can use the scratch procedure to take any action you would like. For example, you can code the procedure to trigger the required inventory management expiration processing job, to run inventory management, or to take no action.

You can use the DFSMSrmm supplied sample procedure, EDGXPROC, which runs DFSMSrmm expiration processing to replenish the automated tape library's scratch volumes. If you use the DFSMSrmm supplied sample, EDGXPROC, you must define the procedure in the installation procedure library, SYS1.PROCLIB as described in [“Step 8: Updating the procedure library”](#) on page 43.

You can modify this procedure, for example, to alert the operator if scratch volumes are not replenished. [Figure 204 on page 459](#) shows the EDGXPROC procedure.

```
//EDGXPROC PROC OPT=EXPROC
//EDGHSKP EXEC PGM=EDGHSKP,
//          PARM='&OPT.'
//SYSPRINT DD SYSOUT=*
//MESSAGE DD DSN=MESSAGE.FILE.NAME,DISP=SHR
```

Figure 204. EDGXPROC procedure

DFSMSrmm keeps track of the time and date the procedure starts and when it last ran expiration processing to ensure that one procedure completes processing before a new procedure begins. You can display this information by using the RMM LISTCONTROL subcommand with the CNTL operand.

Automating backup

DFSMSrmm sample provided in SAMPLIB

EDGBETT sample that you can use in the IBM Tivoli Workload Scheduler for z/OS for automating backup.

Create a backup procedure in the system procedure library. [Figure 205 on page 459](#) shows an example of a procedure that runs backup as part of inventory management. Alternatively, you might want to use the procedure to submit a batch job to perform the backup or to inform your job scheduler to submit the job.

Specify your backup procedure name with the BACKUPPROC value in parmlib. See [“Defining system options: OPTION”](#) on page 161 for more information about specifying BACKUPPROC.

```
//CDSBKUP PROC
//EDGHSKP EXEC PGM=EDGHSKP,PARM='BACKUP(DSS)'
//MESSAGE DD DISP=SHR,DSN=RMM.MESSAGE
//SYSPRINT DD SYSOUT=*
//BACKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.CDS(+1),
//        LABEL=(,SL)
//JRNLBKUP DD DISP=(,CATLG),UNIT=TAPE,DSN=BACKUP.JRNL(+1),
//        LABEL=(2,SL),VOL=REF=*.BACKUP
//        PEND
```

Figure 205. Example BACKUPPROC procedure

Using the LABEL procedure

You can run the EDGINERS utility as an operator started procedure, so that the operator or librarian can make requests for tape labeling, erasing, and scanning.

[Figure 206 on page 460](#) shows a copy of the sample procedure you can use in your installation. Rename the procedure to LABEL. Specify additional processing options that suit the needs of your installation.

You must define the LABEL procedure as a RACF started procedure. The LABEL procedure requires UPDATE access to the STGADMIN.EDG.OPERATOR RACF facility class profile for INIT and ERASE functions; for the SCAN function, READ access is required. See [Chapter 9, “Authorizing DFSMSrmm users and ensuring security,”](#) on page 215 for information about STGADMIN.EDG.OPERATOR.

See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information on using the LABEL procedure as part of your operator's tasks.

```
//LABEL    PROC OPT=NOVERIFY,U=3480,SOUT=DUMMY
//*
//*      RMM procedure for volume labelling, erasing and scanning
//*      See DFSMSrmm Managing and Using Removable Media
//*      for details of the LABEL utility.
//*
//*      &OPT are the options for the EDGINERS utility.
//*      See DFSMSrmm Implementation and Customization Guide
//*      for details of the EDGINERS utility.
//*
//*      &U is the device type or number requested for a tape drive
//*
//*      &SOUT Allows you to specify that the message file is
//*      printed. For example S LABEL,SOUT='SYSOUT=A'
//*
//*      If either VERIFY OR NOVERIFY is the only option specified
//*      in the &OPT parameter, requests are entered via the
//*      system console as WTOR replies.
//*
//INIT      EXEC PGM=EDGINERS,PARM='&OPT'
//SYSPRINT DD  &SOUT
//TAPE      DD  UNIT=(&U,,DEFER)
```

Figure 206. Sample label procedure

Processing NL label tapes: EDG019VM

The EDG019VM sample exit that DFSMSrmm provides can be used as a replacement for IFG019VM. The exit uses the tape volume mount exit to obtain tape label information from the operator when an NL tape is mounted as a scratch tape on a non-specific NL mount request.

Input

The invocation environment must be identical to the environment that is provided at entry to the exit IFG019VM.

Output

The parameter list provided as input is updated. R15 is set on exit with one of the possible values for the return codes:

- 0**
Accept the volume. The parameter list might have been updated.
- 4**
Continue normal processing. The parameter list has not been updated.
- 8**
Reject the volume.

Processing

EDG019VM returns the rack number as the mounted volume for use by open processing.

```
RMMV01D device REPLY WITH RACK NUMBER FOR NL REQUEST – OR REPLY "REJECT"
```

Environment

See [z/OS DFSMS Installation Exits](#) for information about setting up the exit.

Chapter 17. Using the Problem Determination Aid Facility

Perform this step once for each z/OS image. You only need to perform this step if you want an external DASD record of trace data. IBM recommends that you keep a history of DFSMSrmm PDA trace data in case a problem is reported to IBM.

The problem determination aid (PDA) facility gathers DFSMSrmm processing information to enable analysis to pinpoint module flow and resource usage related to DFSMSrmm problems. The PDA facility is required for IBM service because it traces module and resource flow. The PDA facility consists of in-storage trace, optional DASD log data sets, EDGRMMxx parmlib member options, and operator commands to control tracing.

DFSMSrmm accumulates problem determination information at specific module points in the form of trace data, and it records this data in main storage. At predetermined intervals, the trace data is scheduled for output to DASD. The DFSMSrmm trace recording function receives the trace data scheduled for output and writes this data to a file on DASD. The PDA facility consists of two separate log data sets. DFSMSrmm recognizes these log data sets by their DD names, EDGPDOX and EDGPDOY. Recording takes place in the data set defined by EDGPDOX. When that data set is filled, the two data set names are swapped, and recording continues on the newly renamed data set.

When this data set is filled, the names are again swapped, and the output switches to the other data set, thus overlaying the previously recorded data. The larger the data sets, the longer the period of time that is represented by the accumulated data.

Establish a procedure that automatically copies the EDGPDOY data set to tape as a generation-data-group data set each time DFSMSrmm issues message EDG9117I. This practice provides a sequential history of trace data over time so that the data is available when needed for resolving problems.

DFSMSrmm trace data can be formatted with the DFSMSshm ARCPRPDO utility. You are authorized to use ARCPRPDO even if you are not licensed to use DFSMSshm. See *z/OS DFSMSshm Diagnosis* for details about ARCPRPDO. *z/OS DFSMSrmm Diagnosis Guide* provides information about using the DFSMSrmm trace data to determine possible sources of errors. See *z/OS MVS System Messages, Vol 1 (ABA-AOM)* and *z/OS MVS System Messages, Vol 2 (ARC-ASA)* for the messages issued by the DFSMSshm ARCPRPDO utility.

Roadmap for using the Problem Determination Aid

This table shows the tasks and associated procedures for using the problem determination aid.

Task	Associated procedure
Plan to use the PDA Facility.	“Planning to use the PDA facility” on page 462
Determine how long to keep trace information.	“Determining how long to keep trace information” on page 462
Determine the size of the PDA log data set.	“Determining Problem Determination Aid (PDA) log data set size” on page 462
Enable the PDA facility.	“Enabling the Problem Determination Aid (PDA) facility” on page 462
Allocate the PDA log data sets.	“Allocating the Problem Determination Aid (PDA) log data sets” on page 463
Increase, if necessary, the size of the PDA log data sets.	“Increasing the size of the Problem Determination Aid (PDA) log data sets” on page 463

Task	Associated procedure
Maintain a history of the PDA log data sets.	“Maintaining a history of Problem Determination Aid (PDA) log data” on page 464
Print the PDA log data sets.	“Printing the Problem Determination Aid (PDA) log data sets” on page 465

Planning to use the PDA facility

Before you can use the PDA facility, you need to perform these tasks:

1. Determine how long you want to keep trace information.
2. Optionally allocate storage on DASD for the PDA log data sets, EDGPDOX and EDGPDOY.
3. Implement the PDA facility based on how long you want to keep trace data.

Determining how long to keep trace information

How many hours, days, or weeks of trace history does your site want to keep? The minimum recommended trace history is four hours; however, IBM recommends a long-term trace history. A longer trace history gives a greater span both forward and backward in time. Your choice of a trace history interval falls into one of these categories:

Short-term trace history

One to two days is typically considered a short-term trace history interval. Short-term trace histories can be obtained without using generation data groups (GDGs).

Long-term trace history

Two or more days is typically considered a long-term trace history interval. Long-term trace histories are best implemented with the use of generation data sets (GDSs) that are appended sequentially to form a generation data group (GDG).

A long-term trace history is preferred because some DFSMSrmm processing occurs only on a periodic basis. With a longer trace history, you might be able to see more patterns to help you perform problem analysis.

Determining Problem Determination Aid (PDA) log data set size

Allocate storage for your PDA log data sets based on the amount of trace data activity at your site and on how long you want to keep trace information. If you choose to keep trace information for two days or less, see [Appendix E, “Problem Determination Aid log data set size work sheet for short-term trace history,” on page 543](#). If you choose to keep trace information for longer than two days, see [Appendix D, “Problem Determination Aid log data set size work sheet for long-term trace history,” on page 541](#).

Enabling the Problem Determination Aid (PDA) facility

The PDA facility default operating mode is trace enabled at DFSMSrmm startup.

You should continuously enable PDA tracing when DFSMSrmm is active since the processing overhead is minimal. If you define the EDGPDOX and EDGPDOY DD statements in the DFSMSrmm startup procedure, the EDGPDOX and EDGPDOY data sets are swapped and trace output is logged in the data sets.

You can control PDA tracing by using these z/OS MODIFY command keywords. You can also enable or disable the PDA facility by using the parmlib OPTION command PDA operands that are described in [“Defining system options: OPTION” on page 161](#).

PDA=ON|OFF

Specify to turn PDA tracing on or off.

PDALOG=ON|OFF|SWAP

Specify to control the LOGGING function during PDA tracing.

Example: Use the z/OS MODIFY command to enable PDA tracing.

```
F DFRMM,PDA=ON
```

The PDA log data sets are automatically swapped at DFSMSrmm startup. After startup, use the z/OS MODIFY command PDALOG=SWAP to manually swap the data sets as required. For information on how to manually swap the PDA log data sets, see *z/OS DFSMSrmm Diagnosis Guide* manual for details.

Allocating the Problem Determination Aid (PDA) log data sets

To allocate and catalog the problem determination log data sets:

```
//*****  
//* SAMPLE JOB THAT ALLOCATES AND CATALOGS THE PDA LOG DATA SETS.  */  
//*****  
//ALLOPDD JOB MSGLEVEL=1  
//STEP1 EXEC PGM=IEFBR14  
//DD1 DD DSN=?UID..?HOSTID..RMMPDOX,DISP=(,CATLG),  
// UNIT=?TRACEUNIT.,  
// VOL=SER=?TRACEVOL.,SPACE=(CYL,(30))  
//DD2 DD DSN=?UID..?HOSTID..RMMPDOY,DISP=(,CATLG),  
// UNIT=?TRACEUNIT.,  
// VOL=SER=?TRACEVOL.,SPACE=(CYL,(30))
```

Change the User ID (?UID.), z/OS system image ID (?HOSTID.), the trace unit (?TRACEUNIT.), and the volume serial number (?TRACEVOL.) parameters to names that are valid for your environment. The LRECL and RECFM fields will be set by DFSMSrmm when the data set is opened and are not required in the JCL. Do not add the RLSE parameter to the DD statement.

The EDGPDOX and EDGPDOY data sets must be allocated to the same volume. The EDGPDOX and EDGPDOY data sets must be cataloged, variable blocked physical sequential and must not be striped. The data sets are required only if you want to keep a permanent history of the trace data on DASD. Begin with an initial data set size of 30 cylinders. You can adjust the size as you gain more experience with the amount of activity that is traced in your installation. If you allocate them as SMS-managed data sets they must be associated with a storage class having the GUARANTEED SPACE attribute. They should not be associated with a storage class that will conflict with the required data set attributes.

You must define a pair of trace output data sets for each z/OS image. Do not share the trace data sets with multiple DFSMSrmm systems or with other system components.

Increasing the size of the Problem Determination Aid (PDA) log data sets

You might need to increase the size of the EDGPDOX and EDGPDOY data sets (for example, in response to messages EDG9114I, EDG9115I, or EDG9116I).

The options for increasing the size of the PDA log data sets are:

1. Stop DFSMSrmm, reallocate larger EDGPDOX and EDGPDOY data sets, and then restart DFSMSrmm.
2. Reallocate the data sets without stopping DFSMSrmm. Suppose (for example) the current PDA log data sets are DFRMM.ZOS1A.EDGPDOX and DFRMM.ZOS1A.EDGPDOY, with both allocated with 20 cylinders, and you wish to reallocate them with 25 cylinders. You can do this with these steps:
 - a. Ensure that your procedure for automatically copying the EDGPDOY data set to tape as a generation-data-group data set each time DFSMSrmm issues message EDG9117I has completed successfully.

- b. Rename DFRMM.ZOS1A.EDGPDOY to DFRMM.ZOS1A.EDGPDOY.OLD
 - c. Allocate a new DFRMM.ZOS1A.EDGPDOY with 25 cylinders, following the requirements in [“Allocating the Problem Determination Aid \(PDA\) log data sets” on page 463.](#)
 - d. Issue MODIFY DFRMM,PDALOG=SWAP
 At this point, you have DFRMM.ZOS1A.EDGPDOX allocated with 25 cylinders and DFRMM.ZOS1A.EDGPDOY allocated with 20 cylinders.
 - e. Ensure that your procedure for automatically copying the EDGPDOY data set to tape as a generation-data-group data set each time DFSMSrmm issues message EDG9117I again completes successfully.
 - f. Rename DFRMM.ZOS1A.EDGPDOY to DFRMM.ZOS1a.EDGPODY.OLD2
 - g. Allocate a new DFRMM.ZOS1a.EDGPDOY with 25 cylinders, following the requirements in [“Allocating the Problem Determination Aid \(PDA\) log data sets” on page 463.](#)
- Now, DFRMM.ZOS1A.EDGPDOX and DFRMM.ZOS1A.EDGPDOY are both allocated with 25 cylinders.

Maintaining a history of Problem Determination Aid (PDA) log data

IBM recommends that you keep a history of DFSMSrmm PDA trace data in case a problem is reported to IBM. To do this, you should copy each log data set as it fills up to a generation data group (GDG) data set on tape.

DFSMSrmm writes trace data to the data set name defined by the EDGPDOX DD statement. When DFSMSrmm swaps the output data sets, trace data recorded prior to the swap is available in the data set name defined by the EDGPDOY DD statement. To archive the trace data, copy the EDGPDOY data set at the time DFSMSrmm issues message EDG9117I.

Example: Define the generation data group (GDG) name for the archived problem determination output data set. The z/OS system image ID (?HOSTID.) must be valid for your environment.

```

/*****
/* SAMPLE JOB THAT DEFINES THE GENERATION DATA GROUP NAME FOR THE      */
/* ARCHIVED PDA LOG DATA SET.                                          */
/*****
/*
//DEFGDG   JOB MSGLEVEL=1
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN     DD *
//          DEFINE   GDG(NAME(' ?UID..?HOSTID..RMMTRACE' ) -
//                  LIMIT(30) NOSCRATCH NOEMPTY)
/*

```

The data set name for the DD statement, SYSUT1, must correspond to the name specified for the EDGPDOY data set. Change the z/OS system image ID (?HOSTID.) to an ID that is valid for your environment. To copy the inactive trace data set to a scratch tape as a generation data set (GDS):

```

/*****
/* SAMPLE JOB THAT COPIES THE INACTIVE PDA LOG DATA SET TO TAPE      */
/*****
/*
//PDCCOPY   JOB MSGLEVEL=1
//STEP1     EXEC PGM=IEBGGENER
//SYSPRINT  DD SYSOUT=A
//SYSIN     DD DUMMY
//SYSUT1    DD DSN=?UID..?HOSTID..RMMPDOY,DISP=SHR
//SYSUT2    DD DSN=?UID..?HOSTID..RMMTRACE(+1),
//          UNIT=TAPE,
//          DISP=(NEW,CATLG,DELETE),
//          DCB=(?UID..?HOSTID..RMMPDOY)

```


Printing the Problem Determination Aid (PDA) log data sets

For information about printing the problem determination aid logs, see [*z/OS DFSMSHsm Diagnosis*](#) for information about the ARCPRPDO utility.

Chapter 18. Setting up DFSMSrmm disposition processing

DFSMSrmm disposition processing is optional and provides support for these functions:

- Providing operators with information to help them perform tasks like moving a tape to a specific location
- Generating sticky labels
- Updating the location where a volume resides.

You can request DFSMSrmm disposition processing by identifying a disposition control data set using the DFSMSrmm parmlib EDGRMMxx OPTION DISPDDNAME operand and optionally coding the same DD name in a job step that creates or references a tape data set. [Figure 208 on page 470](#) shows how you might code the DD name in a job step. DFSMSrmm disposition processing occurs at CLOSE and EOVS for each volume. DFSMSrmm does not provide support for asynchronous processing in a separate job step.

When you do not code the optional disposition DD name, you can create sticky labels, but you must use the EDG_EXIT100 installation exit to request the label to be produced.

At CLOSE and EOVS time, DFSMSrmm always prepares a sticky label using one of the default label layouts depending on the volume mounted. The prepared label is only printed when the sticky label is requested. You request the label either by using the options in the disposition control file or by using the EDG_EXIT100 installation exit to request the label. In any case, you can customize the label and the location processing using the EDG_EXIT100 installation exit.

If you use the input disposition control file and, optionally, customize with the EDG_EXIT100 installation exit, DFSMSrmm disposition processing can be expanded to support:

- Messages that are issued to one or more route codes by defining multiple messages in the disposition control file.
- Suppression of label output you might have set up using the control file. Modify label output by using the EDG_EXIT100 installation exit.
- Assignment of loan location, storage location, or library location for a volume.

Implementing DFSMSrmm disposition control file processing

Follow these steps to implement DFSMSrmm disposition processing.

1. When you plan on using disposition control files to trigger sticky label production, define disposition control information in a disposition control file as described in [“Modifying the contents of the disposition control file” on page 468](#). You can define separate disposition control files for individual users, a separate job, or a separate job step. Define the data sets with LRECL 80. The data sets can be sequential files or can be members of a partitioned data set. You can also include data in your JCL stream instead of defining a data set.
2. When you do not plan to use disposition control files, but you would still like to create sticky labels, you must customize the supplied EDGUX100 exit module. You can use the default label prepared by DFSMSrmm, or do your own custom label processing.
3. Add the disposition control file DD name to the batch job step of the tape jobs that process the tape files for which messages or labels or location assignment is required. The DD name is the same name as you specified in DISPDDNAME in parmlib.
4. Define the disposition control file DD name in the DFSMSrmm EDGRMMxx parmlib member by using the OPTION command DISPDDNAME operand as described in [“Defining system options: OPTION” on page 161](#). When you plan on using disposition control files, you can also define a message ID in parmlib by using the OPTION command DISPMMSGID operand. DFSMSrmm builds a WTO message by using the message number from the DISPMMSGID parmlib option and the message text defined in the disposition control file.

5. If you do not add the OUTPUT JCL statement, sticky labels are produced using a WTO on route code 13. Add the OUTPUT JCL statement to the DFRMM started procedure as shown in [“Step 8: Updating the procedure library” on page 43](#) if you want DFSMSrmm to dynamically allocate a SYSOUT file for each label. Use the attributes of the OUTPUT statement to define how the label output should be printed.
6. Document the procedures you use for printing label output and applying sticky labels to the correct volumes. If you are using DFSMSrmm disposition control for the first time, work with your operations staff to develop, test, and document procedures for responding to DFSMSrmm messages, printing labels, and using the EDG_EXIT100 exit to modify output. See [z/OS DFSMSrmm Managing and Using Removable Media](#) for information about procedures for your operators.

Tip: DFSMSrmm disposition processing can fail when your application closes a DASD data set and a tape data set at the same time. The system issues error code 50D under these conditions.

- When the DASD data set was allocated with RLSE for the SPACE parameter in the DD statement or RELEASE in the TSO ALLOCATE command,
- When the SMS management class specifies YI or CI for the partial release attribute.

Partial release processing during CLOSE holds an exclusive enqueue on the task input/output table (SYSZTIOT) resource and can prevent DFSMSrmm from opening the disposition control file at the same time. To ensure that the disposition control file can be opened, code a separate CLOSE for every data set or remove the RLSE parameter from the DD statement for the DASD data set.

Modifying the contents of the disposition control file

You identify the input disposition control file by using DFSMSrmm EDGRMMxx parmlib DISPDDNAME operand. The disposition control file is a fixed format sequential file with a record length of 80 characters. All the parameters including the message text are position dependent. [Figure 207 on page 468](#) shows the layout for the disposition control file.

```
ddname_1nrlOUT=vvvv,message text
ddname_1nrlmessage text
ddname_2nrlLOC=vvvvvvvv,message text
ddname_3nrlLOC=vvvvvvvv,message text
```

Figure 207. Disposition control file record format

ddname

1-to-8 character ddname of an input file or output file. This *ddname* must match the ddname you defined for the input tape file or the output tape file processed by the current job step. DFSMSrmm only processes the statements that match to the DD name of the file being processed by CLOSE or EOVS.

You can code multiple lines for each ddname specified and can include as many different ddnames as is required to cover tape data sets requiring disposition support. As each file is processed by CLOSE or EOVS, DFSMSrmm uses only those statements that match the ddname of that file. *ddname* must be coded in position 1 through 8.

n

Code any non-blank character in position 9 to cause the message to be issued in a non-roll deletable message to the console.

r

Code a character to define the route code to use for the WTO. Use one of the characters shown in [Table 61 on page 469](#) to use specific route codes. Use any character other than the characters in [Table 61 on page 469](#) to use the default route code 13.

Table 61. Setting the message route code

If You Code	Then You Are Requesting This Route Code
Any	13 (default value). Any value other than those results in the default value being used.
A	2
B	3
C	5
*	11
F	2,11
G	3,11
H	5,11

L

Indicates if a sticky label is to be produced as part of disposition processing and must be coded in position 11. DFSMSrmm ignores any other value and no label is produced. The valid text codes are shown in [Table 62 on page 469](#).

Table 62. Coding sticky label text

If You Code	Then You Are Requesting That
L	a label is produced by disposition processing. You can code multiple L's with the same ddname and route code if more message text is required.
M	a label is produced by disposition processing but the message text is passed as user data to the EDG_EXIT100 installation exit for further processing.

OUT=vvvvvvvv

This is an optional keyword starting in position 12. Use OUT to change the location for a volume. When you use OUT to change a volume's location, you do not need to confirm the volume move. When OUT is specified, OUT must be followed by a comma, if you also specified message text. vvvvvvvv is a 1 to 8 character location name, that is to be used by DFSMSrmm as a location name for the current volume.

LOC=vvvvvvvv

This is an optional keyword. If you specify LOC, it must be followed by a comma if the optional message text is specified. vvvvvvvv is a 1 to 8 character location name. DFSMSrmm uses the value as a location name for the current volume. When you use LOC=, you request that DFSMSrmm update the volume's destination so that the move can be tracked and confirmed later.

LOC and OUT are mutually exclusive. If you specify multiple statements for the same DD name, DFSMSrmm uses the last location name that you specify for the volume.

You can optionally blank pad location names on the right up to the maximum length of a location name.

DFSMSrmm checks the location name you specify against the location names you defined using the DFSMSrmm LOCDEF location information defined to DFSMSrmm at startup time. If the location you specified, is not defined using LOCDEF DFSMSrmm treats the location as if it is a loan location. If the location you specified is identified as a storage location, use OUT= if you do not want to confirm the move. Use LOC= if you want the move confirmed at a later time.

You can use EDG_EXIT100 to control how the location name is used as described in [“Changing location information with EDG_EXIT100” on page 287](#). You can override the location type determined from the LOCDEF location information to specify that a location is a loan location. You can control whether a confirm move is required for DFSMSrmm storage locations.

When you specify a location name, DFSMSrmm uses this information just as if you had entered the value on an RMM CHANGEVOLUME subcommand with either LOANLOC or LOCATION. If the location is a bin-managed storage location, the required location is set to this value and inventory management DSTORE processing assigns a bin number. You can override any location assignment by defining vital record specification movement policies.

message_text

This is up to 69 characters of message text to be issued as a WTO. It must begin in position 12 or after the comma which separates the message text from the location name. If additional text is required, include another record in the control file specifying the same ddname and route code. Each message is issued as a separate WTO.

If the *l* value is M, the message text is limited to up to 69 characters of text and is treated as user data for label processing. No additional ddname records are supported for the M user data option.

Figure 208 on page 470 shows the sample JCL, if you want to perform these tasks:

- Issue a WTO on route code 2 and route code 1
- Generate a sticky label
- Set the volume location to a location called FICH
- Confirm that the volume is moved to the location FICH. In Figure 208 on page 470, when IEBGENER closes the SYSUT2 output file, DFSMSrmm scans the DISPDD file for the SYSUT2 DD statement, if the DISPDD file is defined. Figure 208 on page 470 shows two entries for SYSUT2, which is the file being closed. The first entry requests that DFSMSrmm issue the message 'SEND THIS TAPE TO THE FICHE PRINTER' on route codes 2 and 11, that a sticky label is generated, and that the volume's move to location FICH is already confirmed. The second entry provides user data that is passed to the EDG_EXIT100 installation exit. DFSMSrmm includes the user data in the sticky label it generates as described in “Creating sticky labels” on page 281

```
// EXEC PGM=IEBGENER
//SYSUT2 DD DISP=(,KEEP),DSN=MY.FICHE.DATA,UNIT=TAPE,LABEL=(,SL)
//SYSUT1 DD DISP=SHR,DSN=MASTER.FICHE.DATA
//SYSIN DD DUMMY
//DISPDD DD *
SYSUT2 FLOUT=FICH,SEND THIS TAPE TO THE FICHE PRINTER
SYSUT2 MUSER DATA TO BE SENT TO EDGUX100
/*
```

Figure 208. Sample JCL to request disposition processing

The message DFSMSrmm issues is EDG4054I, but you can use the DISPMMSGID parmli option to change the message number to an installation selected value.

Selecting the method used for label processing

You can request that DFSMSrmm produce labels using a WTO on route code 13 or using the OUTPUT JCL statement to send labels to a spool file or a printer. You control the method DFSMSrmm uses by specifying an OUTPUT JCL statement in the DFSMSrmm started procedure as described in “Step 8: Updating the procedure library” on page 43. The name on the OUTPUT JCL statement must exactly match that specified for the DD name of the disposition control file.

If you use the OUTPUT JCL statement method, DFSMSrmm dynamically allocates a sysout file for each label using the DISPDDNAME OUTPUT JCL statement. You use the attributes of the OUTPUT statement to define how the label output is to be printed. For example, you can route the output to another system, specify a special forms type or use any of the OUTPUT statement keywords.

If you do not provide an OUTPUT JCL statement in the DFSMSrmm started procedure, you must configure a console to accept WTO messages on route code 13 so that the labels can be printed.

Modifying tape labels

The default label is 10 rows with a maximum of 80 characters per row. The default LRECL is set to 80. The maximum number of data characters supported in a label is 2000 characters. DFSMSrmm provides two default label styles for your use. You can modify these label styles using the EDG_EXIT100 installation exit. Figure 209 on page 471 shows the default label for cartridges. The default label consists of eight data rows, two of which are used for spacing. Cartridge labels are identified by media type other than *, and a density of either *, IDRC, or 3480. Figure 210 on page 471 shows the default label for all other types of volumes. The default label consists of seven data rows, 3 of which are used for spacing the labels. You can use the EDG_EXIT100 installation exit to update the default labels as described in “Modifying DFSMSrmm label output” on page 284.

```
|...+...1...+...2...+...3...+...4...+...5...+...6...+...7.
dsname-----
userdata-----
      jobname_      crdate____
              expdt____
dens comp lrecl  blksiz recf
volser seqn lab      devc
```

Figure 209. Default label format for a tape cartridge

```
|...+...1...+...2...+...3...+...4...+...5...+...6...+...7.
dsname-----
userdata-----
      jobname_      crdate____
              expdt____
dens comp lrecl  blksiz recf expdt____
              volser seqn lab      devc
```

Figure 210. Default label format for a round tape

The values for the variables shown in Figure 209 on page 471 and Figure 210 on page 471 are:

dsname

The data set name of the file being processed. 1 to 44 characters.

userdata

The user data specified by message text in the disposition control file. 0 to 69 characters.

jobname

The current job name. 1 to 8 characters.

crdate

The data set create date. 1 to 10 characters in the date format specified by the DATEFORM parmlib option.

expdate

The data set expiration date. 1 to 10 characters in the date format specified by the DATEFORM parmlib option.

dens

The recording density of the volume. 1 to 4 characters.

comp

Indication of compaction of data on the volume. 4 characters.

lrecl

The logical record length of the data. 1 to 5 characters.

blksiz

The block size of the data. 1 to 6 characters.

recf

The record format. 1 to 4 characters.

volser

The volume serial number. 1 to 6 characters.

seqn

The volume sequence number. 1 to 4 characters.

lab

The volume label type. 1 to 3 characters.

devc

The number of the drive on which the file is processed. 4 characters.

Use the EDGSLAB macro to map the label data area as described in [“Sticky label data: EDGSLAB”](#) on page [529](#).

Chapter 19. Running DFSMSrmm with JES3

DFSMSrmm provides SMP/E USERMODs in SAMPLIB that you can apply to the standard JES3 user exits and other JES3 modules. Use EDG3UX71, EDG3UX29, and EDG3UX62 to set up DFSMSrmm with JES3. Install the USERMODs to prevent JES3 from validating certain volume mounts, to update JES3 fetch and mount messages, and to enable the use of no label tape volumes with JES3.

In a JES3 system, JES3 validates volume mounts when JES3 is managing tape drives and performing pre-processing setup. JES3 ensures that scratch tapes have reached their expiration date and that volumes are correctly write-enabled or protected. JES3 validation can conflict with DFSMSrmm and RACF processing. For example, you can use 'logical write protect' with IBM tape drives to prevent a user from writing to a tape even if the tape is physically write-enabled. Since DFSMSrmm ensures that all scratch tapes are valid and provides features to ignore the expiration dates on volumes, you might want to prevent JES3 from validating certain volume mounts.

Preventing JES3 from validating volumes

Use the EDG3UX29 USERMOD to prevent JES3 from validating volumes to avoid conflicts with DFSMSrmm validation.

In a JES3 system, the default value for EXPDTCHK is YES. This value can conflict with the tape management system function that allows you to use expiration date protected tapes. EDG3UX29 sets EXPDTCHK=NO.

In a JES3 system, the default value for RINGCHK is YES. This value can conflict with the tape management system function, DFSMSdfp, and RACF function that forces logical write protect or allows a user to open a tape data set for READ even though the tape is write enabled. EDG3UX29 sets RINGCHK=NO.

Updating JES3 fetch and mount messages

DFSMSrmm provides USERMODs for use when updating JES3 fetch and mount messages with shelf location and pool information.

When you are using EDG_EXIT100 for scratch pooling as described in [“Using the DFSMSrmm EDG_EXIT100 installation exit”](#) on page 267, use the EDG3UX71 USERMOD to ensure the correct tape pool is requested.

In a JES3 complex all systems running DFSMSrmm must have the same EDG_EXIT100 installation exit and the same parmlib VLPOOL operand values to ensure that a scratch volume is not rejected because it is from the wrong pool. DFSMSrmm also provides USERMODs EDG3LVVR and EDG3IIP1 that you can use to ensure that the correct tape pool is requested. If you do not use one of the USERMODs which allows DFSMSrmm to update a message or tape display, DFSMSrmm cannot provide tape pool information and the wrong tape pool might be requested. Any incorrect tape mounted is rejected by DFSMSrmm volume validation and a remount requested. The remount request does use the correct pool.

Steps for using the EDG3UX71 USERMOD

Before you begin: Validate that the EDG3UX71 USERMOD does not conflict with any modifications that you have made to the IATUX71 exit.

DFSMSrmm supplies a USERMOD to IATUX71, called EDG3UX71, in SAMPLIB. Use EDG3UX71 to update IATUX71 in order to replace and append text to JES3 fetch-and-mount messages and to provide text for tape drive displays. Perform these steps to use the EDG3UX71 USERMOD to update the IAT5210 message:

1. Install the USERMOD by using SMP/E.

2. Set the SETPARAM DSN option in the JES3 initialization deck to a value other than 0. For exit-selected scratch pooling, the EDG_EXIT100 installation exit depends on the data set name being included in the fetch-and-mount messages. Ensure that you have the DSN keyword coded on the JES3 SETPARAM statement in the initialization deck, and that the data set name length value is sufficient to provide selection control in the EDG_EXIT100 installation exit. The value is the length of the data set name to be included in mount messages and fetch messages that are issued by JES3.

Recommendations:

- a. Set the data set name length to 9 if you use only the first qualifier for pooling.
- b. Set the data set name length to 31 if you want the EDG_EXIT100 installation exit to use the data set name for pool selection.

Result: The IAT5210 message is updated.

Using the EDG3IIP1 USERMOD

Use the EDG3IIP1 USERMOD in SAMPLIB to update IATIIP1 to force DEFER for all tape requests. If you use EDG3UX71, you do not need to use EDG3IIP1. The EDG3IIP1 USERMOD updates IATIIP1 to mark tape allocations for deferred processing. This is equivalent to coding UNIT=(unit,,DEFER) as a JCL keyword.

Before you begin: Validate that the EDG3IIP1 USERMOD does not conflict with any other modifications to the IATIIP1 module.

Install the USERMOD by using SMP/E.

Using the EDG3LVVR USERMOD

Use the EDG3LVVR USERMOD in SAMPLIB to update the tape drive display with the correct exit selected pool. The EDG3LVVR USERMOD updates IATLVVR to AWAIT MSGDISP for scratch mount. If you use EDG3UX71, you do not need to use EDG3LVVR.

Before you begin: Validate that the EDG3LVVR USERMOD does not conflict with any other modifications to the IATLVVR module.

The EDG3LVVR USERMOD updates IATLVVR to add a short wait to the verify function. The wait allows the setup function time to issue the IAT5210 message. A JES3 MCS console is required when using this USERMOD.

1. Define a JES3 MCS console.

Refer to for details on how to define MCS consoles for use with JES3. You need to define an MCS console with logical association to JES3 and use it as the tape operator console to receive the mount messages as updated by DFSMSrmm. Use the JES3 CONSOLE statement with the TYPE=MCS keyword.

Use the MCS console routing codes to select which JES3 issued messages are seen on the MCS console. JES3 routes the tape messages to MCS consoles with a route code of 3.

This method cannot be used in a multi-system JES3 complex where tape drives are attached to a JES3 local processor, because DFSMSrmm can only correctly update the tape drive display on the global processor.

2. Install the USERMOD by using SMP/E.

Using the EDG3UX62 USERMOD to create and mount no label tapes

Before you begin: Validate that the EDG3UX62 USERMOD does not conflict with any other modifications to the IATUX62 module.

Installing the EDG3UX62 USERMOD is optional. If you decide to install it, use SMP/E.

Install EDG3UX62 when you are using JES3 pre-execution setup for tape volumes.

After you implement EDG3UX62, you can accomplish these tasks:

- Create NL tape volumes from scratch volumes.
- Use duplicate volumes when using deferred tape mount processing.
- Mount a volume with standard labels for a non-specific NL request.
- Process any standard label volume when requesting a specific volume where the specific volume and the mounted volume have the same label type. For example, the user requests an AL tape volume and the operator mounts an AL tape volume.
- Specify volumes with duplicate volume serial numbers.

The EDG3UX62 USERMOD updates IATUX62 to override the JES3 decision to reject a tape when a non-specific NL tape is requested and a standard label write-enabled tape is mounted. During JES3 verify processing, JES3 ensures that the mounted volume matches the requested volume. If an NL tape is requested, JES3 ensures that an NL tape is mounted. For specific volume requests this is correct processing, but for non-specific requests DFSMSrmm forces all scratch volumes to contain a standard volume label. As a result, JES3 rejects any scratch volume that is mounted.

Chapter 20. Running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS

DFSMSrmm provides sample jobs and procedures that you can modify to set up automation software, such as the IBM Tivoli Workload Scheduler for z/OS, to run DFSMSrmm tasks. The sample jobs and procedures are shipped in SAMPLIB.

Recommendation: Before you use the DFSMSrmm-supplied jobs, customize the jobs for your installation. You must make the job control language (JCL) available to the IBM Tivoli Workload Scheduler for z/OS for submission. The procedures must be in a procedure library that you use with jobs that are submitted by the IBM Tivoli Workload Scheduler for z/OS.

The naming conventions are EDGJxxxx for IBM Tivoli Workload Scheduler for z/OS jobs and EDGPxxxx for IBM Tivoli Workload Scheduler for z/OS procedures. Each EDGJxxxx job contains JCL SET statements for variables that are required for correct JCL generation. For example, you must set the variable RMMPREF to the data set name prefix for use on all output data sets. You can tailor the jobs so you can use the jobs for normal job submission or for recovery or restart. Each EDGPxxxx procedure can be tailored as well. You might have to customize space requirements for the files that are allocated for use as output files.

The supplied jobs support DFSMSrmm tasks that are performed daily, weekly, and monthly and fall into these categories:

1. Scheduled tasks that are run on a regularly scheduled basis.
2. Event triggered tasks that are run on as-needed basis. See [“Event triggered tracking” on page 482](#) for descriptions of DFSMSrmm event triggered tasks.

Using a Tivoli special resource when running DFSMSrmm with the IBM Tivoli Workload Scheduler for z/OS

Use an IBM Tivoli Workload Scheduler for z/OS special resource to perform these functions:

- Allow some DFSMSrmm jobs to run in parallel.
- Prevent more than one EDGHSKP job from running at the same time.
- Avoid jobs failing because inventory management is already running.
- Prevent the EDGUTIL VERIFY job from running at the same time that other DFSMSrmm inventory management jobs are running.
- Allow the IBM Tivoli Workload Scheduler for z/OS to handle the dependencies for jobs that do not always need to run.

The special resource is named in the DFSMSrmm sample IBM Tivoli Workload Scheduler for z/OS loader job EDGJLOPC. The special resource is defined when you load the DFSMSrmm application to the IBM Tivoli Workload Scheduler for z/OS. You can customize the name of the special resource in EDGJLOPC. Be sure to change all occurrences of the special resource name in the application definition, so that the IBM Tivoli Workload Scheduler for z/OS job scheduling can maintain inventory management serialization properly.

Most of the DFSMSrmm tasks in the job flow are automated. These tasks require manual intervention.

1. Confirmation of moves requires intervention at an IBM Tivoli Workload Scheduler for z/OS terminal to mark volume moves completed. You must confirm volume movement before you can run the confirm job.
2. During restart or recovery of the DFSMSrmm-supplied Main job, you might have to perform visual checking and some other recovery actions based on the reason for the failure. For example, when recovery completed, the Main job restarts from the beginning. For EDGHSKP, return code 4 is an acceptable completion code. Any higher return code value triggers restart or recovery processing.

DFSMSrmm provides a sample procedure called EDGJLOPC. The sample procedure includes code that supports the inventory management schedule that is recommended in [“Scheduling DFSMSrmm utilities”](#) on page 327.

Setting up DFSMSrmm to use the IBM Tivoli Workload Scheduler for z/OS

Follow this procedure to set up IBM Tivoli Workload Scheduler for z/OS for management of DFSMSrmm tasks:

1. Customize the batch loader statements in EDGJLOPC as described in [“Customizing the IBM Tivoli Workload Scheduler for z/OS batch loader statements”](#) on page 481.
2. Ensure that the IBM Tivoli Workload Scheduler for z/OS workstations that are required by the DFSMSrmm applications are defined to the IBM Tivoli Workload Scheduler for z/OS as described in [“Setting up IBM Tivoli Workload Scheduler for z/OS workstations”](#) on page 481.
3. Make the customized jobs and procedures available to IBM Tivoli Workload Scheduler for z/OS and to your running systems as described in [“Descriptions of DFSMSrmm jobs to run with the IBM Tivoli Workload Scheduler for z/OS”](#) on page 478.
4. Run EDGJHKPA to define GDG bases and to create first generations if needed. DFSMSrmm provides a job called preparation, that is described in [“Descriptions of DFSMSrmm jobs to run with the IBM Tivoli Workload Scheduler for z/OS”](#) on page 478, that you can use to create the GDGs.
5. Change the DFSMSrmm PARMLIB OPTION command BACKUPPROC and SCRATCHPROC operands to use the new sample procedures for use with the IBM Tivoli Workload Scheduler for z/OS as described in [“Defining system options: OPTION”](#) on page 161.
6. Run the DFSMSrmm sample EDGJLOPC job as described in [“Descriptions of DFSMSrmm jobs to run with the IBM Tivoli Workload Scheduler for z/OS”](#) on page 478.
7. Add the event trigger tracking entries to the IBM Tivoli Workload Scheduler for z/OS by using the dialog as described in [“Event triggered tracking”](#) on page 482.
8. Set up the IBM Tivoli Workload Scheduler for z/OS restart management to handle restart activities. Restart management uses the DFSMSrmm application programming interface to issue commands based on the IBM Tivoli Workload Scheduler for z/OS restart/recovery options that you define. If you do not use IBM Tivoli Workload Scheduler for z/OS, you can issue DFSMSrmm TSO subcommands to change information as needed.

If not all of the data sets on a volume are created successfully, you can use the subcommand shown in [Figure 211](#) on page 478 to override normal vital record specification management processing.

```
RMM CHANGEDATASET dsname VOLUME(volser) SEQ(number) ABEND
```

Figure 211. Overriding vital record specification management processing with the RMM CHANGEDATASET TSO subcommand

Descriptions of DFSMSrmm jobs to run with the IBM Tivoli Workload Scheduler for z/OS

All the sample jobs that are used with the IBM Tivoli Workload Scheduler for z/OS job management include the required control statements about recovery, restart, or modifications to make at submit time. The JCL uses procedures that are provided to avoid duplicate JCL and to simplify job construction.

Preparation

You must first allocate files that are required for the regular processing. EDGHSKP has special requirements for the data sets that are used for processing. Use EDGJHKPA to define the GDGs and set up the first generation of the files as required. EDGJHKPA uses the procedures EDGPHKPA, EDGPVRSR, EDGPMSG, EDGPACTA and EDGPRPTA, and REXX procedure EDGRHKPA. Specify parameters in EDGHSKP to customize the GDG names and limits. Customize the file size by altering the SPACE values in the lowest level procedures.

The GDGMODEL should be customized to match a model DSCB available on your system. Any DCB attributes are acceptable because DFSMSrmm overrides the values at OPEN time. For system-managed data sets, a model DSCB name is not required, and you can specify GDGMODEL="" in that case.

CDS Verify

Run this job when you want to verify the contents of the DFSMSrmm control data set. The EDGJVIFY job runs EDGUTIL with PARM=VERIFY on the current DFSMSrmm control data set. The expected completion is return code 0. You must perform manual recovery before dependent jobs can be run when the return code is greater than 0.

CDS Backup

Run this job when you want to back up the DFSMSrmm control data set. The EDGJBKP1 and EDGBKP2 jobs run EDGHSKP with PARM=BACKUP. Backup is performed at the start and at the end of the main inventory management application processing. You can modify the jobs to use DFSMSdss or AMS for backup. You make the choice of backup to DASD or tape when running the preparation jobs.

Jobs EDGJBKP1 and EDGJBKP2 run procedure EDGPBKP to perform the backup. Expected completion is return code 4 or less. You must perform manual recovery before dependent jobs can be run when the return code is greater than 4.

Inventory Management

Run this job when you want to perform DFSMSrmm inventory management processing. These jobs run EDGHSKP with VRSEL and EXPROC. These jobs also include running storage location management on a weekly basis to trigger movement decisions for off-site storage. Job EDGJDHKP and job EDGJWHKP run procedures EDGPHSKP, EDGPMSG, and EDGPVRS to process and set up the files for the next run. The job copies the message file to SYSOUT for easier information analysis. The expected completion is return code 4 or less. You must perform manual recovery before dependent jobs can be run when the return code is greater than 4. As part of the recovery, run the EDGJVRSV job to run VRSEL with VERIFY and to report on the ACTIVITY file for analysis of VRSMIN and VRCHANGE conditions. For return code 8, you might have to change vital record specifications or add additional empty bin numbers. Higher return codes or abends need to be researched by the support programmer.

Erasing and Labeling Volumes

Run this job when you want to initialize and erase volumes depending on the release actions set for the volumes. The EDGJINER job runs an automatic EDGINERS job to handle the INIT and ERASE release actions. You can run dependent jobs that are run regardless of successful completion.

Scratch Processing

Run this job when you want to process volumes that are returning to scratch status. The EDGJSCR job runs EDGHSKP with the EXPROC parameter. This job handles the return to scratch of volumes requiring confirmed actions; whether INIT/ERASE or movement actions. You can run dependent jobs regardless of successful completion.

Scratch Reporting

Run this job when you want to obtain information about volumes in scratch status. The EDGJSCR job produces an up-to-date extract and uses EDGRPTD with SCRLIST and NEWSR files to produce the latest scratch list reports. You can run dependent jobs regardless of successful completion.

Ejecting Volumes

Run this job to eject volumes from system-managed tape libraries. This is an optional job that you can use if your installation uses system-managed tape. The job also includes support for VTS export processing. You can run dependent jobs regardless of successful completion. You can customize this job to include similar processing that is required for any non-system-managed volumes that you have in your library.

Producing Reports

Run this job to produce volume movement reports. The EDGJMOVE job is dependent on the inventory management job and optionally the ejecting volume job processing. This job creates an up-to-date extract and uses EDGRPTD to produce the movement reports. You can run dependent jobs regardless of successful completion.

Move Confirmation

Run this job to perform global confirmation of volume movement. The EDGJCMOV job issues a global confirm move. You can run this job after a manual action is taken by the installation to mark the movement complete at the dependent IBM Tivoli Workload Scheduler for z/OS workstation. You can run dependent jobs regardless of successful completion.

Daily jobs are always run on processing days. Weekly and monthly jobs are included in the job flow and the IBM Tivoli Workload Scheduler for z/OS automatically adjusts dependencies that are based on the rules you specify. When a job fails, the IBM Tivoli Workload Scheduler for z/OS automatically prevents dependent jobs from processing if any manual recovery actions have been defined. To alter the dependencies, you can modify the sample IBM Tivoli Workload Scheduler for z/OS applications that are described in [“IBM Tivoli Workload Scheduler for z/OS applications for DFSMSrmm”](#) on page 480.

CDS backup and scratch processing are functions that are always run in the daily cycle. CDS backup and scratch processing can also be added dynamically to the cycle as required. When CDS backup and scratch processing are added dynamically, they run independently. They are added automatically to the schedule when the write-to-operator (WTO) from DFSMSrmm is detected and the BACKUPPROC or SCRATCHPROC is triggered. When a started procedure matches an application defined to the IBM Tivoli Workload Scheduler for z/OS running on a started task processor, IBM Tivoli Workload Scheduler for z/OS modifies the current plan.

IBM Tivoli Workload Scheduler for z/OS applications for DFSMSrmm

The application definition implements the schedule suggested in the [“Scheduling DFSMSrmm utilities”](#) on page 327. In addition, EDGJLOPC includes support for events that are triggered by alerts such as the CBR3660A, CBR3792E, and CBR3794A (short on scratch) messages and the EDG2107E (journal threshold reached) message.

The applications defined in the DFSMSrmm sample IBM Tivoli Workload Scheduler for z/OS setup job are shown in [Table 63](#) on page 480:

Table 63. IBM Tivoli Workload Scheduler for z/OS applications

Application	Description
GRMMDAY	GRMMDAY is a grouping application used to set a common start time for daily jobs and applications. GRMMDAY is scheduled to run every working day of the year.
GRMMMTH	GRMMMTH is a grouping application used to set a common start time for monthly jobs and applications. GRMMMTH is scheduled to run once each month on the first Friday of the month.
GRMMWK	GRMMWK is a grouping application used to set a common start time for weekly jobs and applications. GRMMWK is scheduled to run every Monday.
RMMBKP	RMMBKP is the first daily application. RMMBKP backs up the control data set and journal data set, and clears the journal. RMMBKP is dependent on the monthly RMMMTH job EDGJVYF.
RMMEXP	RMMEXP runs expiration processing and produces scratch list reports. RMMEXP is dependent on prior daily jobs and the weekly movement processing performed by the RMMMOVES job.
RMMHKPD	RMMHKPD is the main daily inventory management run. RMMHKPD is replaced once a week with a weekly inventory management run that includes movement processing.

Table 63. IBM Tivoli Workload Scheduler for z/OS applications (continued)

Application	Description
RMMMOVES	RMMMOVES includes jobs to: • Eject system-managed volumes that are moving • Produce and print movement reports • Confirm volume moves once volumes have been moved RMMMOVES includes a manual action to complete a task at a workstation when volumes have been moved. RMMEXP is dependent on the completion of RMMMOVES.
RMMMTH	RMMMTH performs a monthly verify of the DFSMSrmm control data set. When you run RMMMTH, all other applications are dependent on RMMMTH running successfully.
RMMPOST	RMMPOST backs up the control data set and journal data set, and clears the journal. RMMPOST runs EDGINERS to process any new release actions.
RMMWK	The main inventory management run once each week instead of the daily inventory management application. RMMWK is dependent on RMMBKP.
RMMVRSVER	RMMVRSVER is an application that is set up only for use during recovery of the main inventory management application. RMMVRSVER is dynamically added if the EDGHSKP step fails with return code 8. RMMVRSVER runs VRSEL with VERIFY and produces a report from the ACTIVITY file.

You can use the sample job EDGJLOPC to run the IBM Tivoli Workload Scheduler for z/OS batch loader utility to define DFSMSrmm as an application to IBM Tivoli Workload Scheduler for z/OS to manage the regular scheduling of DFSMSrmm functions.

Customize the EDGJLOPC sample by:

- Modifying the supplied JCL to run in your environment
- Tailoring and customizing the sample application definition and schedule to meet your specific needs.

Then run the sample to load the application definition to the active control file or to a VSAM file to be used with the IBM Tivoli Workload Scheduler for z/OS subsystem.

Customizing the IBM Tivoli Workload Scheduler for z/OS batch loader statements

The DFSMSrmm sample job EDGJLOPC contains the application definitions for the applications and jobs described in [“IBM Tivoli Workload Scheduler for z/OS applications for DFSMSrmm”](#) on page 480. You can alter or add to the definitions before running the IBM Tivoli Workload Scheduler for z/OS batch loader to load the definitions to your IBM Tivoli Workload Scheduler for z/OS subsystem or to another VSAM file you plan to use as an IBM Tivoli Workload Scheduler for z/OS AD database.

Setting up IBM Tivoli Workload Scheduler for z/OS workstations

You should ensure that you define these workstations to your IBM Tivoli Workload Scheduler for z/OS subsystem or alter them to those you want to use with the DFSMSrmm applications.

The sample definitions use these workstations:

STC1

A computer workstation for the running started tasks

CPU1

A computer workstation for the running batch jobs

PRT1

A workstation for printing movement reports

TLIB

A manual workstation for use by the tape librarian or operator to mark movement of volumes completed.

Event triggered tracking

The applications are set up to take advantage of the IBM Tivoli Workload Scheduler for z/OS event trigger tracking for these conditions:

- Journal threshold is reached - backup is required to clear the journal
- Low on scratch - expiration processing is required to return pending release volumes to scratch status and produce new scratch lists.

To use the applications, follow these steps:

1. Define these tasks to the IBM Tivoli Workload Scheduler for z/OS under Event Trigger Tracking; EDGSETT triggers RMMEXP, and EDGBETT triggers RMMBKP.
2. Change the DFSMSrmm parmlib OPTION command BACKUPPROC and SCRATCHPROC operands to name special procedures EDGBETT and EDGSETT. Specify EDGBETT and EDGSETT to enable the event trigger processing.

The options for each entry in the ETT table are:

- Event type=J
- Job replace=Y
- Dependency resolution=N
- Availability status=N

EDGBETT and EDGSETT sample procedures are only intended for tracking by IBM Tivoli Workload Scheduler for z/OS because they only execute IEFBR14.

Chapter 21. Using the parmlib member EDGDEFxx

Introduction to the defaults table

The EDGDEFxx member in the parmlib defines the defaults table, or DEFTABLE. The defaults table is a way to dynamically assign retention parameters to newly written data sets and the volumes on which they reside, based on a set of selection criteria. The defaults table is a simple alternative to using the EDGCVRSX sample for the EDG_EXIT100 installation exit, which relies on a table called UXTABLE.

Alternatively, retention policies can be managed using Management Classes and SMS ACS routines, with the MCATTR option. The settings in the management class take precedence over those defined in the defaults table.

Here is a sample defaults table:

Data set name mask	Job name mask	KEYDATE	RETPD	VX	Last reference days	VRS Value	WHILE CATALOG	Retention Method
ABC.*	JOB02	*	100	N				VRSEL
**	A*	96000						EXPDT
**	*	98020		Y	20	M98020		EXPDT
**	*	98040			40	M98040		EXPDT
**	*	99000				M99000	ON	EXPDT
SYS1.**	*	*	PERMANENT	Y				EXPDT
**	*	*	5,OVERRI DE	Y	20			

Columns 1-3: selection criteria. "Program Name Mask" and "Continue" are also part of the selection criteria. These are not displayed due to space constraints.

Columns 4-8: data set attributes assigned to a data set if the selection criteria match.

Column 9: volume attribute assigned to scratch volume when writing its first data set if the selection criteria match. Other volume attributes that can be set using the defaults table are "Retain By", "EDM" and "POOL". These are not displayed due to space constraints.

Here is how the defaults table is defined in the EDGDEFxx member:

```
DEFAULT DSN(ABC.*) -  
        JOB(JOB02) -  
        KEYDATE(*) -  
        RETPD(100) -  
        VX(NO) -  
        RM(VRSEL)  
DEFAULT JOB(A*) -  
        KEYDATE(96000) -  
        RM(EXPDT) -  
DEFAULT KEYDATE(98020) -  
        VRSVALUE(M98020) -  
        LASTREF(20) -  
        RM(EXPDT)  
DEFAULT KEYDATE(98040) -  
        VRSVALUE(M98040) -  
        LASTREF(40) -  
        RM(EXPDT)  
DEFAULT KEYDATE(99000) -  
        VRSVALUE(M99000) -  
        WHILECATALOG(ON) -
```

```

RM(EXPDT)
DEFAULT DSN(SYS1.***) -
  RETPD(PERMANENT) -
  VX(YES) -
  RM(EXPDT) -
DEFAULT -
  RETPD(5,OVERRRIDE) -
  VRSELEXCLUDE(YES) -
  LASTREF(20) -

```

Here are additional examples on how you can code the defaults table:

ONLYIF SYSNAME(PROD)	/* ONLY if SYSNAME symbol is PROD	*/ -
DEFAULT DSN(**.G%%V%%)	/* DSNAME or PREFIX	*/ -
WHILECATALOG(ON)	/* SET WHILECATALOG CHECK to ON	*/ -
RM(EXPDT)	/* RETENTION METHOD EXPDT used	*/
ONLYIF SYSNAME(TEST)	/* ONLY if SYSNAME symbol is TEST	*/ -
DEFAULT PGMNAME(XYZ)	/* Program that created the DSN	*/ -
VRSVAL(D99365)	/* Specifies the VRS managementvalue	*/
DEFAULT DSNAME(**.HMIGTAPE.DATASET)	/* DSNAME or PREFIX	*/ -
PGM(ARCCTL)	/* program that created the DSN	*/ -
RETPD(PERM,OVERRRIDE)	/* RETPD and override the JCL expdt	*/ -
RM(EXPDT)	/* RETENTION METHOD EXPDT used	*/ -
RETAINBY(VOLUME)	/* RETAINBY VOLUME	*/ -
WHILECATALOG(OFF)	/* SET WHILECATALOG CHECK to OFF	*/
DEFAULT DSNAME(**.BACKTAPE.DATASET)	/* DSNAME or PREFIX	*/ -
PGM(ARCCTL)	/* Program that created the DSN	*/ -
RETPD(PERM,OVERRRIDE)	/* RETPD and override the JCL expdt	*/ -
RM(EXPDT)	/* RETENTION METHOD EXPDT used	*/ -
RETAINBY(VOLUME)	/* RETAINBY VOLUME	*/ -
WHILECATALOG(OFF)	/* SET WHILECATALOG CHECK to OFF	*/
DEFAULT PGM(ARCWCTL)	/* ABARS program that created DSN	*/ -
RM(EXPDT)	/* RETENTION METHOD EXPDT used	*/ -
RETPD(PERM,OVERRRIDE)	/* RETPD and override the JCL expdt	*/ -
WHILECATALOG(OFF)	/* SET WHILECATALOG CHECK to OFF	*/
DEFAULT DSNAME(DFHSM.**)	/* DSNAME or PREFIX	*/ -
PGM(ARCCTL)	/* Program that created the DSN	*/ -
RM(EXPDT)	/* RETENTION METHOD EXPDT used	*/ -
RETAINBY(SET)	/* RETAINBY VOLUME	*/ -
RETPD(PERM,OVERRRIDE)	/* RETPD and override the JCL expdt	*/ -
WHILECATALOG(OFF)	/* SET WHILECATALOG CHECK to OFF	*/
DEFAULT DSNAME(**)	/* DSNAME or PREFIX	*/ -
JOBNAME(*)	/* All jobname, this is the default	*/ -
NOKEYDATE	/* NO Keydates specified	*/ -
VRSVAL(CATALOG)	/* Specifies the VRS management value	*/

Examples of Key dates that have "special meaning expiration dates". For all these definitions, a VRS entry for each VRSVALUE must be defined, so that all the matching data sets are correctly retained:

```

DEFAULT KEYDATE(99000)          /* WHILECATALOG          */ -
      VRSVALUE(CATALOG)         /* Specifies the VRS management value */ -
      RETPD(5,OVERRIDE)         /* RETPD and override the JCL expdt   */
DEFAULT KEYDATE(99001)          /* 001 cycles of a data set are to be held */ -
      VRSVALUE(D99001)         /* Specifies the VRS management value */ -
      RETPD(5,OVERRIDE)         /* RETPD and override the JCL expdt   */
DEFAULT KEYDATE(99002)          /* 002 cycles of a data set are to be held */ -
      VRSVALUE(D99002)         /* Specifies the VRS management value */ -
      RETPD(5,OVERRIDE)         /* RETPD and override the JCL expdt   */

```

Whenever a new data set is written onto a tape, the defaults table is searched from the top down until a matching entry is found. Once a matching entry is found, the search stops, unless the matching entry has **CONTINUE** specified. An entry only matches if all four selection criteria for the data set are satisfied: the data set name mask matches, the job name mask matches, the program name mask matches, and the key date matches. From the matching entry, any attributes that are not blank are copied to the data set and the volume. The volume attributes are only used if the data set is the first in the volume chain, not when writing to a master or a user volume.

Certain attributes are only valid for a specific retention method. Last reference days, WHILECATALOG and Retain By are only used when writing to a volume with retention method of EXPDT. VRSELEXCLUDE applies only to volumes with a retention method of VRSEL, since volumes with RM=EXPDT are always excluded from VRSEL processing. The VRSEL management value is assigned to any data set, however, it is only used by VRSEL processing during housekeeping.

The priority of the defaults table is below that of the management class and the user exit 100, except for POOLING decisions. It is recommended not to combine the defaults table with using management classes and ACS selection routines, or the EDG_EXIT100 installation exit and the UXTABLE.

The EDGDEFxx suffix is specified in the DEFTABLE system option. See [“Defining system options: OPTION”](#) on page 161 for more information.

Note: Expiration dates derived from Model DSCB (DCB=model.dsname) or LIKE=dsname are not evaluated as KEYDATE by EDGUX100, DEFTABLE or SMS Management Class ACS routine. Only EXPDT=yyddd specified in JCL, DATACLAS or dynamic allocation are evaluated as KEYDATE. However, EXPDT derived from Model DSCB will be stored in DFRMM Volume and Data set records and will be used as valid expiration dates if no further matches occur from DEFTABLE or VRS.

Converting from using UXTABLE to using the defaults table

The UXTABLE is a table that is created during conversion to DFSMSrmm and is used together with the EDGCVRSX sample for the EDG_EXIT100 installation exit. It functions like the defaults table, and you can move away from using the UXTABLE by converting it to the defaults table. To convert, you need to have the source of the UXTABLE that is used on your system. The following UXTABLE option is not available in the defaults table: ACL1OPT.

Use the EDGJDEF sample JCL, which is available in SYS1.SAMPLIB, to convert the UXTABLE to the defaults table. EDGJDEF expects the UXTABLE source to be specified in the //VRSTABLE DD statement and saves the defaults table to the data set specified in the //EDGDEF DD statement. The EDGJDEF job uses the EDGRDEF script to do the conversion. This script is also located in SYS1.SAMPLIB. The EDGRDEF script expects one parameter, which is the global retention period default on your system. If not changed, this parameter has a value of 5: PARM='%EDGRDEF 5'. See the EDGRDEF script for more information about this parameter. The defaults table is different from the UXTABLE as follows:

1. Additional options present in the defaults table, missing from UXTABLE:

CONTINUE, EDM/NOEDM, LASTREF, NOKEYDATE, PGMNAME, RETAINBY, WHILECATALOG

2. Unlike the UXTABLE, the defaults table allows you to combine selection criteria DSNAME and JOBNAME together with KEYDATE. All entries of the defaults table are processed together, while the UXTABLE processes KEYDATE entries separately from the DSNAME/JOBNAME entries.
3. The RO(YES) option of the UXTABLE, used to indicate that the retention period or expiration date that is specified in the JCL must be replaced, has the equivalent option OVERRIDE, a subparameter of RETPD, in the defaults table. An example of specifying OVERRIDE is RETPD(50,OVERRIDE). In the defaults table, OVERRIDE must be specified together with a retention period, unlike the UXTABLE where RO(YES) would clear the retention period that is specified in the JCL even if RETPD is not specified in the UXTABLE. Also, in the UXTABLE, KEYDATE entries have RO(YES) implied, which is not the case for the defaults table.
4. The OVERRIDE option of the defaults table has no effect on the priority of the defaults table entry. This is unlike the UXTABLE, where RO(NO) and RO(YES) are also used to choose between a KEYDATE entry and a DSN/JOBNAME entry, when both match the data set. In the defaults table, the NODATE parameter can be used instead of RO(NO) to indicate that the current entry is not to be used if KEYDATE or RETPD is specified in the JCL, or original expiry date is assigned to dataset via Data class or Model DSCB.
5. The UXTABLE allows you to combine entries in such a way that a DSN/JOBNAME with RO(NO) is used to make a pooling decision while a KEYDATE entry is used to choose all the remaining retention settings. The same behavior might be achieved in the defaults table by using the CONTINUE keyword. The CONTINUE keyword allows you to make any pooling decisions in the beginning of the defaults table, and if CONTINUE is specified in the entries that choose the POOL. The rest of the defaults table is able to assign any other parameters.

Modifying the defaults table

It is possible to modify the defaults table by changing the EDGDEFxx parmlib member, and then restarting the DFSMSrmm subsystem with the following command, where DFRMM is the name of the task used for DFSMSrmm:

```
F DFRMM,M=nn
```

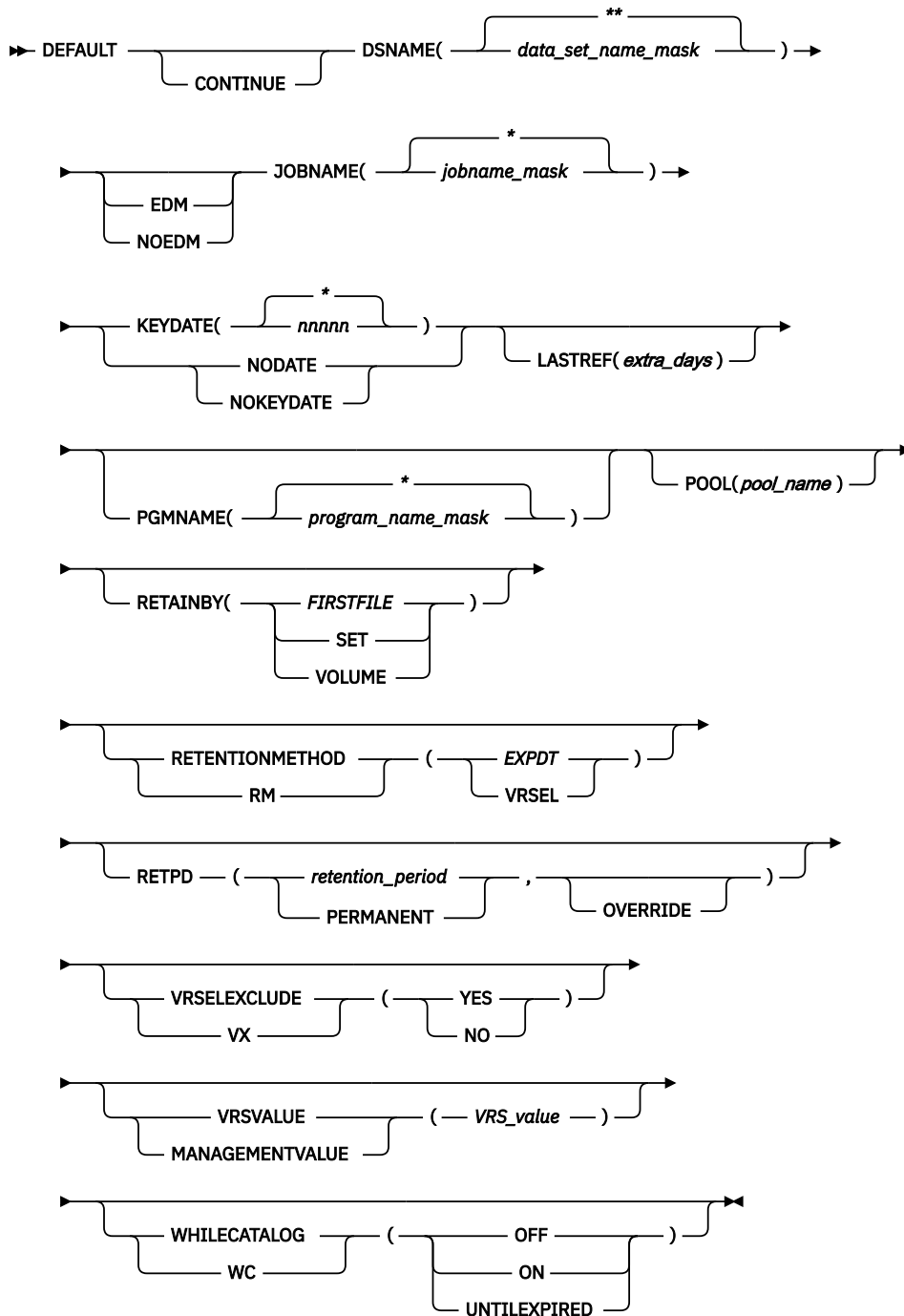
Defining defaults table entries: DEFAULT

Use the default command in the EDGDEFxx parmlib member to define the defaults table entries for DFSMSrmm.

DEFAULT command syntax

Figure 212 on page 486 shows the syntax of the DEFAULT command:

Figure 212. Parmlib member EDGDEFxx DEFAULT command syntax



DEFAULT command operands

CONTINUE

When CONTINUE is specified, if this defaults table entry is selected, RMM remembers the settings defined in the entry and continues searching to see if any other defaults table entries also match. The searching stops when a matching defaults entry is found that does not have CONTINUE specified, or when the end of the defaults table is reached. If more than one matching entries specify the same parameter, the value of the last matching entry is used.

For example, in the following case:

- DEFAULT DSNAME(ABC.*) POOL(ABC1*) CONTINUE
- DEFAULT DSNAME(ABC.*) RETPD(999)

Data sets that start with ABC will use the ABC1* scratch pool, and will have a retention period of 999 days.

DSNAME(data_set_name_mask)

Select this defaults table entry if the name of the data set written to tape matches the data set name mask. You can supply a fully qualified data set name or a data set name mask that contains wildcard characters. The default is ** to select all data sets.

Note: DFSMSrmm does not fold data set names to uppercase letters when you specify quoted data set names. When you specify data set names or data set name masks, be sure to specify the correct case for each character. DFSMSrmm does not check quoted data set names for valid characters. Any string of up to 44 characters is accepted, except those that start with a blank or x'00'.

Data set names without quotes and data set name masks without quotes must pass these data set naming rules:

- A data set name can have one or more qualifiers
- Each qualifier is one to eight characters. The first character alphabetic (A to Z) or national (# @ \$). The remaining seven characters can be either alphabetic, numeric (0 - 9), national, or a hyphen (-)
- Qualifiers are separated by a period
- The data set name must not include a member name.

In addition, data set name masks must pass these data set mask naming rules:

- You can use * or % in a data set name mask
- A single * represents a single qualifier of any number of characters
- A single % when used within a qualifier represents zero or more characters
- More than one single * can be used within a qualifier as long as a character precedes or follows the *
- ** represents zero or more qualifiers. At the end of the mask, ** indicates to ignore any remaining characters
- ** indicates to select all data sets. ** is the default.

Percent sign (%): A place holder for a single character.

Period (.): A leading or trailing period is not allowed. Consecutive periods are also not allowed.

Double asterisk (**): Cannot be specified within a data set name qualifier.

EDM

Specifies if the volume on which the matching data set is located is to be marked as EDM (External Data Manager) managed. Setting the EDM attribute prevents the volume from being expired during DFSMSrmm expiration processing. External Data Managers such as HSM release such volumes by themselves using the EDGTVEXT or EDGDFHSM programming interfaces.

By default, the global EDM option is turned on for DFSMSrmm, marking any volumes written by the following program names as EDM-managed: ANZSRVR, ANRSERV, ARCCTL, ARCWCTL, CBROAM, DSMSEVR. The EDM parameter of the defaults table allows to specify additional volumes to be marked as EDM-managed.

JOBNAME(jobname_mask)

Select this defaults table entry if the job that created the data set has a name that matches the mask provided. It can contain one-to-eight alphanumeric characters or \$, #, or @. Also, use % in your generic job name mask to match any one character and * to match any character string inside the job name.

The default is * to select all job names.

KEYDATE(keydate_value)

The KEYDATE is matched against the EXPDT value used in the JCL. KEYDATE has the form yyddd where yy indicates the years from 1900 to 1999, and can have the value from 00 to 99 and ddd indicates the days, and can have the value from 000 to 366.

The default is * to select all dates

NOEDM

Specifies that the volume on which the matching data set is located is not to be marked as EDM (External Data Manager) managed. Setting the EDM attribute prevents the volume from being expired during DFSMSrmm expiration processing. External Data Managers such as HSM release such volumes by themselves using the EDGTVEXT or EDGDFHSM programming interfaces.

Note that NOEDM does NOT reset the EDM attribute if it was set for the volume previously. Using NOEDM allows to make exceptions to the list of programs that turn EDM on automatically with the EDM(Y) global option - ANZSRVR, ANRSERV, ARCCTL, ARCWCTL, CBROAM, DSMSESV.

NOKEYDATE

It is possible to specify NOKEYDATE instead of KEYDATE. NOKEYDATE means that the dataset only matches if it does not have a KEYDATE specified in the JCL or there is no matching KEYDATE in the DEFAULTS table. A KEYDATE is any date that can be found used in the KEYDATE selection criteria in the defaults table.

For example, if only the following two defaults exist in the defaults table:

```
DEFAULT DSNAME NOKEYDATE VRSVALUE(ABCVAL)
DEFAULT KEYDATE(99000) VRSVAL(WHILECAT)
```

If a data set is written with EXPDT=99000 coded in the EXPDT value in the JCL, it is assigned a VRS value of WHILECAT. If any other EXPDT is specified, the data set has the VRSVALUE of ABCVAL.

NODATE

It is possible to specify NODATE instead of KEYDATE. NODATE means that the dataset only matches if it does not have a KEYDATE (EXPDT) or Retention period specified in the JCL or original expiry date is assigned via Data Class or Model DSCB. A KEYDATE is any date that can be found used in the KEYDATE selection criteria in the defaults table.

For example, if only the following two defaults exist in the defaults table:

```
DEFAULT DSNAME NODATE VRSVALUE(XYZVAL)
DEFAULT KEYDATE(99000) VRSVAL(WHILECAT)
```

If a data set is written with EXPDT=99000 coded in the EXPDT value in the JCL, it is assigned a VRS value of WHILECAT. If EXPDT and RETPD is not specified on the data set has the VRSVALUE of XYZVAL.

LASTREF(extra_days)

LASTREF(extra_days) specifies the number of days that the matching data set will be retained after the data set was last referenced.

LASTREF(extra_days) is only set for new data sets on volumes managed by the EXPDT retention method.

extra_days is a decimal number between 0 and 93000. DFSMSrmm uses the LASTREF extra days to evaluate the data set expiration date. The extra days are added to the date of last reference. The data set expiration date is set to the maximum of the date calculated using LASTREF extra days, and the current expiration date. Any reference to the data set by a read or write operation will redetermine the expiration date. An *extra_days* value of 0 means that DFSMSrmm does not consider the data set last reference date when evaluating the data set expiration date.

For a volume set retained by VOLUME or SET, DFSMSrmm ensures that the LASTREF(extra_days) attribute is the same for all files of a multivolume data set.

The LASTREF(extra_days) value for new data sets can be taken from the following sources, listed in priority order, with the highest priority first:

1. "Expire after Days Non-usage" in the Management Class
2. LASTREF(extra_days) in the defaults table in EDGDEFxx.
3. The default EDGRMMxx parmlib OPTION RM(EXPDT(LASTREF(value))).

PGMNAME(program_name_mask)

Select this defaults table entry if the program that created the data set has a name that matches the mask provided. It can contain one to eight alphanumeric characters or \$, #, or @. Also, use % in your program name mask to match any one character and * to match any character string inside the program name.

The default is * to select all program names.

POOL(pool_mask)

Selects the scratch pool prefix for the write request when using an MTL (manual tape library) or a stand-alone tape drive. The selected pool is appended to the write to operator mount messages.

During OPEN processing DFSMSrmm validates that a scratch volume has been mounted from the selected pool.

POOL is not compatible with KEYDATE, NOKEYDATE and PGMNAME operands. The scratch pool prefix must also be defined in the EDGRMMxx parmlib member using the VLPOOL command.

The z/OS MONITOR DSNNAME option must be in effect to use the data set name mask when selecting a scratch pool.

The scratch pool selection can be taken from the following sources, listed in priority order with the highest priority first:

1. Storage group name as determined by the ACS routines - for non system managed libraries when SMSACS(YES) is in effect.
2. POOL(pool_mask) in the defaults table in EDGDEFxx.
3. PL100_POOL returned by the EDG_EXIT100 exit routine.
4. Storage group name as determined by the ACS routines for Manual Tape Libraries (ignored if the EDG_EXIT100 exit routine sets the PL100_SET_IGNORE_SGNAME bit. If you are using an unmodified version of EDGUX100 or EDGCVR SX for your user exit, this bit is set by default).
5. DFSMSrmm system-based scratch pooling.

For example: POOL(ABC1*)

RETAINBY(FIRSTFILE|SET|VOLUME)

Specifies how DFSMSrmm is to retain a volume or multivolume set. RETAINBY is only set for volumes managed by the EXPDT retention method if the matching data set is the first data set written on the volume chain.

FIRSTFILE

The expiration date of the first file is used to set the expiration date of a single volume or a multivolume set. The dates of additional data sets on the volume or to a multivolume set have no influence on the volume expiration date.

SET

All volumes in the set will have the same expiration date. DFSMSrmm uses the highest expiration date of all volumes in the set and each file on a volume set can increment it.

VOLUME

The expiration date of the volume is considered for each volume separately and each file on a volume can increment the volume expiration date.

The RETAINBY value for volumes can be taken from the following sources, listed in priority order, with the highest priority first:

1. "Volume Set Management Level" in the Management Class
2. RETAINBY in the defaults table in EDGDEFxx
3. The default EDGRMMxx parmlib OPTION RM(EXPDT(RETAINBY(value))).

RETENTIONMETHOD(EXPDT|VRSEL)

Specifies the retention method for a tape volume or multivolume set. RETENTIONMETHOD is only set if the matching data set is the first data set written on the volume chain.

Specify EXPDT to set the retention method for a tape volume set to be based on EXPDT. Data sets and volumes managed by this retention method are never processed by VRSEL inventory management. The expiration date for volumes and data sets is updated dynamically, also it is possible to use the LASTREF and WHILECATALOG options.

Specify VRSEL to set the retention method for a tape volume set to be VRSEL. This option enables DFSMSrmm inventory management to attempt to match data sets and volumes to vital record specifications, and if a match is found, to determine if the data set or volumes are to be retained by VRS.

The RETENTIONMETHOD for volumes can be taken from the following sources, listed in priority order, with the highest priority first:

1. "Retention Method" in the Management Class
2. PL100_RETENTIONMETHOD, returned by the EDG_EXIT100 exit routine
3. RETENTIONMETHOD in the defaults table in EDGDEFxx
4. The default EDGRMMxx parmlib OPTION RM.

The retention method can also be specified manually in the ADDVOLUME or CHANGEVOLUME subcommands.

RETENTIONMETHOD can be specified as RM.

RETPD(*retention_period* [,*override*])

Specifies the number of days that the matching data set will be retained before being considered for release. The value is a decimal number from 0 to 93000 and is added to the current date and time to calculate the new expiration date and time. If PERMANENT is specified, the expiration date is set to 1999/365.

RETPD(nnnn, OVERRIDE) causes the RETPD value in the defaults table to override what is specified in the JCL. If OVERRIDE is not specified, and either RETPD or EXPDT are present in the JCL, the JCL value is used.

The expiration date for newly written data sets can be taken from the following sources, listed in priority order, with the highest priority first:

1. The value in the JFCB control block, which can come from the following three sources:
 - a. The RETPD or EXPDT value specified in the JCL
 - b. The RETPD or EXPDT value specified in the DATACLAS
 - c. The EDG_EXIT100 installation exit, which can modify the JFCB containing the JCL parameters and pointed to by PL100_JFCBPTR. If the sample RO option in the UXTABLE defines whether the exit overrides the JCL value.
2. "Expire after Date/Days" in the management class
3. RETPD in the defaults table. You can use OVERRIDE in the defaults table to override the JFCB value.
4. The default EDGRMMxx parmlib option REPTD(value), along with RM=EXPDT specific options RM(EXPDT(NOGDG(RETPD(value)))) and RM(EXPDT(GDG(RETPD(value)))).

The retention period or expiration date can also be specified manually in the RMM ADDDATASET, RMM ADDVOLUME, RMM CHANGEDATASET and RMM CHANGEVOLUME commands. Using WHILECATALOG and LASTREF allows the expiration date to be modified dynamically, depending on catalog status or the last reference date.

VRSELEXCLUDE(YES|NO)

Specifying VRSELEXCLUDE causes the new data set to be excluded from VRSEL processing.

The VRSELEXCLUDE value from the defaults table has no effect on data sets on volumes managed by the EXPDT retention method, since they always have VRSELEXCLUDE set to yes.

The VRSELEXCLUDE value for newly written data sets can be taken from the following sources, listed in priority order, with the highest priority first:

1. “Exclude from VRSEL “ in the management class
2. PL100_SET_VRSELEXCLUDE, returned by the EDG_EXIT100 exit routine
3. VRSELEXCLUDE(YES/NO) as specified in the defaults table.

VRSELEXCLUDE can also be changed manually in the CHANGEDATASET subcommand.

VRSELEXCLUDE can be abbreviated as VX.

VRSVALUE(VRS_management_value)

Specifies the vital record specification management value to be associated with the data set. A vital record specification management value is one-to-eight characters other than blank, comma, and semicolon. The name you specify must be a single qualifier.

VRS policies can reference data sets with a specific VRS management value.

The VRS management value for newly written data sets can be taken from the following sources, listed in priority order, with the highest priority first:

1. PL100_VRS, returned by the EDG_EXIT100 exit routine
2. VRSVALUE as specified in the defaults table.

The VRS management value can also be changed manually in the CHANGEDATASET subcommand.

VRSVALUE can be specified as MANAGEMENTVALUE.

WHILECATALOG(OFF|ON|UNTILEXPIRED)

WHILECATALOG allows to retain data sets on volumes managed by the EXPDT retention method based on their catalog status. WHILECATALOG is not set for new data sets on volumes managed the VRSEL retention method.

WHILECATALOG(OFF)

The matching data set will have the WHILECATALOG attribute set to OFF. Its retention will depend only on the expiration date and not on its catalog status.

WHILECATALOG(ON)

The matching data set will have the WHILECATALOG attribute set to ON. As a result, it will not be expired as long as it is cataloged, nor will it be expired prior to its expiration date. The expiration date of the new data set will be set to the greater of the EXPDT value or (creation date + CATRETPD hours). The CATRETPD option is used to ensure that new data sets are not prematurely expired before the catalog signal is received, when EDGHSKP housekeeping functions are running at the same time the tape is being created. If WHILECATALOG is ON and the data set is cataloged, the data set expiration date is displayed with (KeptByCatlg) appended to the date.

WHILECATALOG(UNTILEXPIRED)

The matching data set will have the WHILECATALOG attribute set to UNTILEXPIRED. As a result, it will expire after being uncataloged or if the expiration date is reached. When DFSMSrmm determines that a data set is no longer cataloged, the DFSMSrmm resets the expiration date of the data set to a date equal to the date the data set was uncataloged plus the number of days specified by the CATLGDAYS option. WHILECATALOG(UNTILEXPIRED) has no effect on data sets that are never cataloged. If WHILECATALOG is UNTILEXPIRED and the data set is cataloged, the data set expiration date is displayed with (OrUncatlg) appended to the date.

The WHILECATALOG value for new data sets can be taken from the following sources, listed in priority order, with the highest priority first:

1. “Retain While Cataloged” in the Management Class
2. WHILECATALOG in the defaults table in EDGDEFxx
3. The default EDGRMMxx parmlib OPTIONS: RM(EXPDT(NOGDG(WHILECATALOG(value)))) and RM(EXPDT(GDG(WHILECATALOG(value)))).

Sharing the defaults table on multiple systems

ONLYIF allows to specify parmlib options that are only valid for a certain system. This makes it possible to use one common PARMLIB member to define the RMM setup for different LPARs. The command that follows ONLYIF is disregarded on all systems except the system specified in the ONLYIF parameter.

The OPTION command can be specified more than once to allow to separate options valid for all systems from options for a specific system. ONLYIF can also be specified before the DEFAULT command in the EDGDEFxx parmlib member.

ONLYIF is valid before the EDGDEFxx DEFAULT command, and also before any EDGRMMxx command. For more information, see [“ONLYIF” on page 156](#).

Chapter 22. The DFSMSrmm WUI

The DFSMSrmm Web GUI provides an alternative to 3270 RMM green screens. It is deployed as a z/OSMF plug-in and interacts with RMM by using the z/OSMF REST API interface and RMM TSO API commands. It offers a subset of those features that can be controlled or displayed by the RMM ISPF panels as well as new functionality not available in RMM ISPF panels..

Setting up the DFSMSrmm WUI

The DFSMSrmm WUI plug-in for z/OSMF represents a web user interface for DFSMSrmm. It provides a subset of functionality that is implemented by ISPF screens of DFSMSrmm.

WUI depends on:

- DFSMS Removable Media Manager (RMM)
- z/OSMF Nucleus
- z/OSMF Administration tasks
- z/OSMF TSO/E address space services

Installing the DFSMSrmm WUI

Follow these steps to install the DFSMSrmm WUI plug-in for z/OSMF:

1. Before installation, make sure that DFSMSrmm and z/OSMF are running and properly configured by using their configuration guides.
 - a. To install and configure DFSMSrmm, follow [Chapter 2, “Implementing DFSMSrmm,”](#) on page 23.
 - b. Install and configure z/OSMF with RMM. In this step, the defaults and suggestions used in the following manuals are used. If customizations are made, match the replacements as they differ from z/OSMF documentation.
 - i) Complete the following instructions to install and configure core features of z/OSMF:
 - [Create a z/OSMF nucleus on your system](#) in the *IBM z/OS Management Facility Configuration Guide*
 - [Configure the z/OSMF administration tasks](#) in the *IBM z/OS Management Facility Configuration Guide*
 - ii) In all cases, it is recommended to modify the Common TSO Logon Proc that was created or copied during the z/OSMF Nucleus installation (default is **IZUFPROC**) and specified in **IZUPRM** as the **COMMON_TSO PROC ()** by adding the RMM executable library to the **SYSEXEC** parameter.

By default, this library is at **SYS1.SEDGEXE1** but may have been customized when RMM was installed or later moved. This is required for installations that do not have the RMM Executable Library in the **SYS1.SEDGEXE1** Data Set. It is not required if the library is available at the default location.
 - c. z/OSMF TSO API must be authorized to SAF. You can check the existing API Authorizations in [Security Configuration Assistant](#) tab in the *IBM z/OS Management Facility Configuration Guide*.
 - i) In case the TSO API is not authorized, configure the TSO/E address space services. For more information, see [Configure the TSO/E address space services](#) in the *IBM z/OS Management Facility Configuration Guide*
 - d. Make sure that the user that is using the DFSMSrmm WUI is added to either the IZUUSER or the IZUADMIN SAF groups.
2. Open z/OSMF in your browser and run the z/OSMF "Import Manager" plug-in. You need to be authorized to access the "Import Manager" task. For more information, see [Configure the z/OSMF administration tasks](#) in *IBM z/OS Management Facility Configuration Guide*.

3. In the z/OSMF **Import Manager** plug in, specify the full path to the file and click **Import**.
 - a. The standard installation path for US-based English is `.../dfsmsrmm_wui/plugin-install.properties`.
 - b. The standard installation path for Japanese is `.../dfsmsrmm_wui/plugin-install-ja.properties`.
 - c. Review the selected `plugin-install.properties` file:
`izu.externalapp.code.root` parameter is a path to directory with the plug-in's web executable files. It is not recommended to change this path, but this path should point to where your executable files have been unpacked and will remain. If needed to update, you can update this path from UNIX System Services where the file exists.
 - d. On validation and confirmation of paths, click **Import**.
4. To use the DFSMSrmm WUI plug-in, authorize user groups IZUADMIN and IZUUSER (or your custom user group) the class of the plug-in SAF, issue the following commands.

```
RDEFINE ZMFAPLA IZUDFLT.ZOSMF.DFSMS.RMM.WUI UACC(NONE)

PERMIT IZUDFLT.ZOSMF.DFSMS.RMM.WUI CLASS(ZMFAPLA)
ID(IZUADMIN) ACCESS(CONTROL)

PERMIT IZUDFLT.ZOSMF.DFSMS.RMM.WUI CLASS(ZMFAPLA)
ID(IZUUSER) ACCESS(READ)

SETROPTS RACLIST(ZMFAPLA) REFRESH
```

Refresh the z/OSMF web page to accept changes of SAF permissions.

5. Add the DFSMSrmm WUI TSO log on procedure:
 - a. Add a TSO logon procedure with member name EDGWUILP to PDS, which contains your TSO logon procedures.
 - b. Add the following contents to the logon procedure.

```
//EDGWUILP EXEC PGM=IKJEFT01,REGION=6144K,
// DPRTY=(10,10),TIME=1440,DYNAMNBR=50
```

You can also find the sample EDGWUILP logon procedure inside the **SYS1.SEDGEXE1** library or the `../assets/logon/` directory of the DFSMSrmm WUI.

- c. Define the logon procedure to SAF. Sample commands:

```
RDEFINE TSOPROC EDGWUILP UACC(NONE)
PERMIT EDGWUILP CLASS(TSOPROC) ID(IZUADMIN) ACCESS(READ)
PERMIT EDGWUILP CLASS(TSOPROC) ID(IZUUSER) ACCESS(READ)
SETROPTS RACLIST(TSOPROC) REFRESH
```

- d. It is recommended to enable no less than 2 simultaneous TSO sessions for a user. Otherwise, the DFSMSrmm WUI might encounter stability issues.
 - e. To check that the logon procedure was successfully installed, log on to the TSO terminal by using the EDGWUILP procedure. If the TSO logon is successful and no errors or prompts for user input are received at logon, the logon procedure installation is successful.

The EDGWUILP procedure is not designed to work with ISPF.

6. Run the DFSMSrmm WUI plug-in from the z/OSMF web interface, and open the settings tab.

Check all settings in the web application block, except the Request limit setting. They must be the same as the settings used to successfully connect to your TSO terminal on step “5.e” on page 496.

The logon procedure name that is used must be the 'EDGWUILP' procedure that is installed in step “5” on page 496.

It is recommended to log off from other active TSO sessions for the current user.

7. To accept TSO connection settings, click **Save** and reload the **DFSMSrmm WUI** page to finish the installation.

To update the existing installation, import the plug-in again as defined in steps “2” on page 495 and “3” on page 496 of the installation instructions.

Using the DFSMSrmm WUI

The DFSMSrmm WUI consists of the four main tabs (Volumes, Data Sets, Defaults Table, and Monitoring) and one Settings tab. Tabs are accessed from the top navigation panel of the WUI. It also contains two additional views: Volumes and Data Sets relation view and the Network Logs.

The Volumes tab is used to display the main table with a list of Volumes that can be filtered by using the applied search filters. This tab has a panel that displays detailed information for a selected Volume.

The Data Sets tab is used to display the main table with a list of Data Sets which can be filtered by using the applied search filters. This tab has a panel that displays detailed information for a selected Data Set.

The Volumes and Data Sets relation view shows Volumes containing the selected Data Set for a multi-volume Data Set, or Data Sets that are on a selected Volume. This view is accessible from a context (right-click) menu of Volumes or Data Sets tabs.

The Defaults Table tab is used to display a table with RMM Defaults Table policies. This tab has a Policy Helper feature which displays default RMM policies that would be assigned to data sets created with specified criteria. This feature is also accessed in the Data Sets tab by right-clicking on a data set row and selecting Show Policies.

The Monitoring tab is used to display information about the current RMM status. It has general information about the system, PARMLIB, RMM CDS (Control Data Set), the RMM Journal. Additionally, it contains network information that is available if RMM is being used in a client/server setup.

The Settings tab is used to display and modify current settings of the DFSMSrmm WUI. From this tab, you can go to the network logs for service purposes.

The Network logs view is used to display network requests that returned abnormal (non-2XX) HTTP response codes. It can be used to check if there are some issues that are detected during the exploitation of the DFSMSrmm WUI. This log can be downloaded for the service analysis.

Volumes tab

The Volumes tab is used to display filtered list of volumes and detailed information on a volume that is stored in RMM.

To use the Volumes tab, you need READ access to the STGADMIN.EDG.MASTER resource profile.

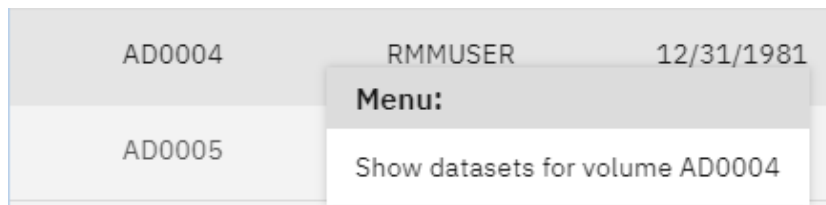
In addition, if COMMANDAUTH(DSN) is in effect, you need READ access to the first file data set name in the DATASET class. If there is no first file that is defined to DFSMSrmm and the volume is in master status or user status, you need READ access to the volume in the TAPEVOL class.

When the RACF® SETROPTS MLNAMES command has been used to activate the name-hiding function, or when COMMANDAUTH(DSN) is in use, to list and search all entries independent of the access that is granted to the DATASET and TAPEVOL class, you need either CONTROL access to the STGADMIN.EDG.MASTER profile resource or CONTROL access to the STGADMIN.EDG.LIST profile resource.

The Volumes tab consists of the following elements:

Volume Serial	Owner	Assigned Date	Expiration Date	Retention Conditions	Retention Date	Status	Location	System ID	Size
AD0001	IBMUSER	12/31/2016	01/01/2007			MASTER	SHELF		1 KB
AD0002		12/31/2020				SCRATCH	SHELF		0 KB
AD0003	RMMUSER	12/31/2014	12/31/2000			MASTER	SHELF		0 KB
AD0004	RMMUSER	12/31/1981	12/31/2020	Expiration date not reached		MASTER	SHELF		2 KB
AD0005	RMMUSER	10/02/2017	12/31/1997			USER	SHELF		0 KB
AD0006		05/06/2004				SCRATCH	SHELF		0 KB
AD0007	IBMUSER	02/29/2004	02/29/2028	Expiration date not reached		USER	SHELF		1 KB
AD0008	RMMUSER	01/01/2029	02/29/1984			USER	SHELF		0 KB
AD0009		12/31/2024				SCRATCH	SHELF		0 KB
AD0010		12/31/2051				SCRATCH	SHELF		0 KB
AD0011		04/05/2036				SCRATCH	SHELF		0 KB

1. **Top Navigation Panel:** Contains tabs representing the main sections of the DFSMSrmm WUI.
2. **Volume List:** Contains volume search results depending on applied filters. Information is retrieved using RMM TSO commands LISTVOLUME and SEARCHVOLUME. A right-click on this table opens a menu that allows to show Data Sets for the selected.



3. **CSV Export Button:** Exports the table in CSV format.
4. **Filters Button:** Opens a modal window with filters that are set for the list of volumes that are displayed in the Volumes tab.
5. **Refresh Button:** Updates the list of volumes.
6. **Volume Details:** Contains detailed information about selected volume. This panel is hidden until a volume from the list of volumes is clicked.
7. **Notification Button:** Opens a window with the history of notifications that appear during the current session.
8. **Volume List Navigation Bar:** Displays the total number of volumes and pages. Also, used to navigate through the list.

Filters applied to the list of volumes are set in the following modal window:

Volumes filters [X]

Volume serial ⓘ Job Name ⓘ NOJOBNAME

* < Job Name [OFF]

Owner ⓘ Status ⓘ

* ALL [v]

Expiration Date ⓘ Create Date ⓘ

mm/dd/yyyy — mm/dd/yyyy mm/dd/yyyy — mm/dd/yyyy

Reference Date ⓘ Retention Date ⓘ

mm/dd/yyyy — mm/dd/yyyy Retention Date

Reset Apply

Note: The following list contains descriptions of the fields that are used to filter the list of volumes. All fields correspond to the same named parameters in the RMM SEARCHVOLUME command. For more information about usage and restrictions for the SEARCHVOLUME command, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

Volume serial field

Specifies the serial numbers of the volumes that are being searched. This field has a switcher button to turn on/off the search volumes interval.

Owner field

Specifies a specific owner ID to list volumes belonging to that owner.

Expiration date field

Specifies the search criteria of retention expiration date of the searched volumes.

Reference date field

Specifies the search criteria of created date of the searched volumes.

Job name field

Specifies the name of a job that created the volumes.

Status field

Allows to filter volumes that have specific status.

Create date field

Specifies the search criteria of created date of the searched volumes.

Retention date field

Specifies the retention date of the volumes that are being searched. User can also select special retention dates, including “CATRETPD”, “WHILECATLG”, and “CYCL/cccc”, where ccccc is the number of cycles to be retained.

Data Sets tab

The Data Sets tab is used to display a filtered list of data sets and detailed information on a data set stored in RMM.

To use the Data Sets tab, you need READ access to the STGADMIN.EDG.MASTER resource profile.

In addition, if COMMANDAUTH(DSN) is in effect, you need READ access to the data set name in the DATASET class.

When the RACF® SETROPTS MLNAMES command is used to activate the name-hiding function, or when COMMANDAUTH(DSN) is in use, to list and search all entries independent of the access that is granted to the DATASET and TAPEVOL class, you need either CONTROL access to the STGADMIN.EDG.MASTER profile resource or CONTROL access to the STGADMIN.EDG.LIST profile resource.

The Data Sets tab consists of the following elements:

The screenshot displays the DFSMSrmm Data Sets tab. The interface includes a top navigation bar with tabs: DFSMSrmm, Volumes, Data Sets (highlighted with a green box and callout 1), Defaults Table, Monitoring, and Settings. To the right of the tabs are icons for search, filter, refresh, and notifications. The main area contains a table of data sets with columns: Data Set Name, Volume Serial, Owner, SEQ, Expiration Date, and Retention Date. The table lists 11 data sets. To the right of the table is a details panel for the selected data set, RMMUSER.TDS1.D4. The details panel includes fields for Data Set size, Management class, Job name, Program name, VRSEL Exclude, Whilecatalog, Last Ref Extra Days, VRS, Secondary VRS, Create date, Date last written, and Date last read. The bottom of the interface shows a pagination bar with 'Items per page: 1 - 11 of 176 items' and '1 of 16 pages'.

Data Set Name	Volume Serial	Owner	SEQ	Expiration Date	Retention Date
RMMUSER.TDS1.D4	AD0004	RMMUSER	1	10/26/2020	
RMMUSER.TDS2.D4	AD0004	RMMUSER	2	10/26/2020	
RMMWUL.V0001.TDS1	AD0001	IBMUSER	1	01/01/2007	
RMMWUL.V0001.TDS2	AD0001	IBMUSER	2	12/31/2000	
RMMWUL.V0007.TDS1	AD0007	IBMUSER	1	02/29/2028	
RMMWUL.V0013.TDS1	AD0013	RMMUSER	1	02/29/2000	
RMMWUL.V0017.TDS1	AD0017	RMMUSER	1	12/31/2000	
RMMWUL.V0026.TDS1	AD0026	IBMUSER	1	08/14/2012	
RMMWUL.V0027.TDS1	AD0027	IBMUSER	1	01/01/2032	
RMMWUL.V0029.TDS1	AD0029	RMMUSER	1	12/31/2056	
RMMWUL.V0036.TDS1	AD0036	RMMUSER	1	02/29/2056	

Details for Data Set: RMMUSER.TDS1.D4

Data Set size: 1 KB

Management class: -

Job name: RMMUSERJ

Program name: PRGRM

VRSEL Exclude: NO

Whilecatalog: -

Last Ref Extra Days: -

VRS: -

Secondary VRS: -

Create date: 06/05/2020

Date last written: 06/05/2020

Date last read: 06/05/2020

Items per page: 1 - 11 of 176 items

1 of 16 pages

1. **Top Navigation Panel:** Contains tabs representing the main sections of the DFSMSrmm WUI.
2. **Data Sets List:** Contains data sets search results depending on applied filters. Information is retrieved using RMM TSO commands LISTDATASET and SEARCHDATASET. A right-click on this table opens a menu that allows users to show the volumes for the selected data set, or the applicable policies from the defaults table:

Data Set Name	Volume Serial	Owner	SEQ	Expiration Date	Retention Date
RMMUSER.A10400DS.N1	A10401	RMMUSER	1	04/12/2022	
RMMUSER.A10400DS.N2		RMMUSER	1	04/12/2022	
RMMUSER.A10400DS.N3		RMMUSER	1	05/02/2022	

3. **Download table:** Starts to download current search results to PC in *.csv format.
4. **Filters Button:** Opens a modal window with filters that are set for the list of data sets that are displayed in Data Sets tab.
5. **Refresh Button:** Updates the list of data sets.
6. **Data Set Details:** Contains detailed information about the selected data set. This panel is hidden until a data set from the list of data sets is clicked.
7. **Notification Button:** Opens a window with the history of notifications that appear during the current session.
8. **Data Sets List Navigation Bar:** Displays the total number of data sets and pages. Also, used to navigate through the list.

Filters applied to the list of data sets are set in the following modal window:

Data Sets filters

Data Set Name ⓘ

Volume Serial ⓘ

Job Name ⓘ

NOJOBNAME

*

*

Job Name

OFF

Owner ⓘ

Program Name ⓘ

NOPROGRAMNAME

*

Program Name

OFF

Expiration Date ⓘ

Create Date ⓘ

mm/dd/yyyy — mm/dd/yyyy

mm/dd/yyyy — mm/dd/yyyy

Refetence Date ⓘ

Retention Date ⓘ

mm/dd/yyyy — mm/dd/yyyy

Retention Date

Reset

Apply

Note: The following list contains descriptions of the fields that are used to filter the list of data sets. All fields correspond to the same named parameters in the RMM SEARCHDATASET command. For more information about usage and restrictions for the SEARCHDATASET command, see [z/OS DFSMSrmm Managing and Using Removable Media](#).

Dataset name field

Specifies the mask or a fully qualified data set name to list only those data sets.

Volume serial field

Specifies the serial number of the volume where data sets are being searched.

Owner field

Specifies a specific owner ID to list data sets belonging to that owner.

Expiration Date field

Specifies the search criteria of expiration date of the searched data sets.

Reference Date field

Filters the list based on last read date and last write date parameters, using the most recent of both dates.

Job Name field

Specifies the name of a job that created the data sets.

Program Name field

Specifies the name of a job step program running at the time the data sets were opened for output.

Create Date field

Specifies the search criteria of created date of the searched data sets.

Retention Date field

Specifies the retention date of the data sets that are being searched.

Defaults Table tab

The Defaults Table tab displays RMM defaults table, currently in read-only mode.

To use the Defaults Table tab, you need CONTROL access to the STGADMIN.EDG.MASTER resource profile when the STGADMIN.EDG.LISTCONTROL resource profile is not defined.

Or, you might need CONTROL access to the STGADMIN.EDG.LISTCONTROL resource profile when the STGADMIN.EDG.LISTCONTROL resource profile is defined.

The Defaults Table tab consists of the following elements:

The screenshot shows the DFSMSrmm Defaults Table tab. At the top, there is a navigation bar with tabs: DFSMSrmm, Volumes, Data Sets, Defaults Table (selected), Monitoring, and Settings. Below the navigation bar, there are two main sections: Selection Criteria and Resulting parameters. The Selection Criteria section has a table with columns: DSN, Job name, KEYDATE, and Prg name. The Resulting parameters section has a table with columns: RM, Pool, EDM, VRS, VX, Last ref days, Ret PD, RO, RetainBy, and WHILECAT. Below these sections is a table titled 'Defaults table entries that matched'. This table has columns: Data Set Name mask, Job Name mask, KEYDATE, Program Name, Continue, Retention Method, Pool, EDM, VRS Value, VRSEL Exclude, Last Ref extra days, Retention Period, Retention Override, Retain By, and While Catalog. The table contains 10 rows of data. At the bottom, there is a navigation bar showing 'Results: 17' and '1 of 2 pages'.

Data Set Name mask	Job Name mask	KEYDATE	Program Name	Continue	Retention Method	Pool	EDM	VRS Value	VRSEL Exclude	Last Ref extra days	Retention Period	Retention Override	Retain By	While Catalog
RMMUSER.A*DS.N1	*	*	*	X	EXPDT	BIG*		VRSVLO##	N					UX
RMMUSER.**N1	*	*	*	↓	VRSEL				Y					
RMMUSER.**N1	*	*	*	↓	EXPDT						PERM	N		
RMMUSER.A*DS.N2	NOJOB	*	*	↓	EXPDT			VRSNVR	N	95	10	N	VOLUME	
RMMUSER.A*DS.N2	*	*	*	↓	EXPDT		Y			10			FIRSTFILE	OFF
RMMUSER.A*DS.N2	DEFTTEST	*	*	↓	VRSEL	SM*	N	VRSVLF##	N	25				ON
RMMUSER.A*DS.N2	*	*	*	X	EXPDT		N	VRSMLP##		5				OFF
RMMUSER.**N2	*	*	*	X	VRSEL		Y	VRSV###	N					OFF
RMMUSER.A*DS.N3	*	*	*	↓	VRSEL		N		N	5			VOLUME	ON

1. **Top Navigation Panel:** Contains tabs representing the main sections of the DFSMSrmm WUI.
2. **Download Table Button:** Starts to download current table to PC in *.csv format.
3. **Policy Helper Button:** Opens a modal window that allows the user to specify data set selection criteria. The defaults table policies that would be assigned to a data set created with the specified criteria are displayed in the Defaults Table entries that matched.
4. **Refresh Button:** Updates the list of default table entries.
5. **Notification Button:** Opens window with history of notifications that appear during current session.
6. **Selection Criteria:** Contains data set selection criteria that is input by the user.
7. **Resulting Parameters:** Contains the resulting policies assigned to a data set created with the entire set of specified selection criteria.
8. **Defaults Table:** Contains the default table entries from the applied EDGDEFxx or entries that RMM identifies as a match for a data set created with the entire set of specified selection criteria.
9. **Defaults Table navigation bar:** Displays the total number of items and pages and is used to navigate through the list.
10. **Screen Options Button:** Opens a modal window that allows a user to set specific user settings for this view. Currently, the only settings toggle the visibility of the columns in the primary data table of the Defaults Table tab. The checkbox group relates to each column of the table, checked means visible, unchecked means hidden.

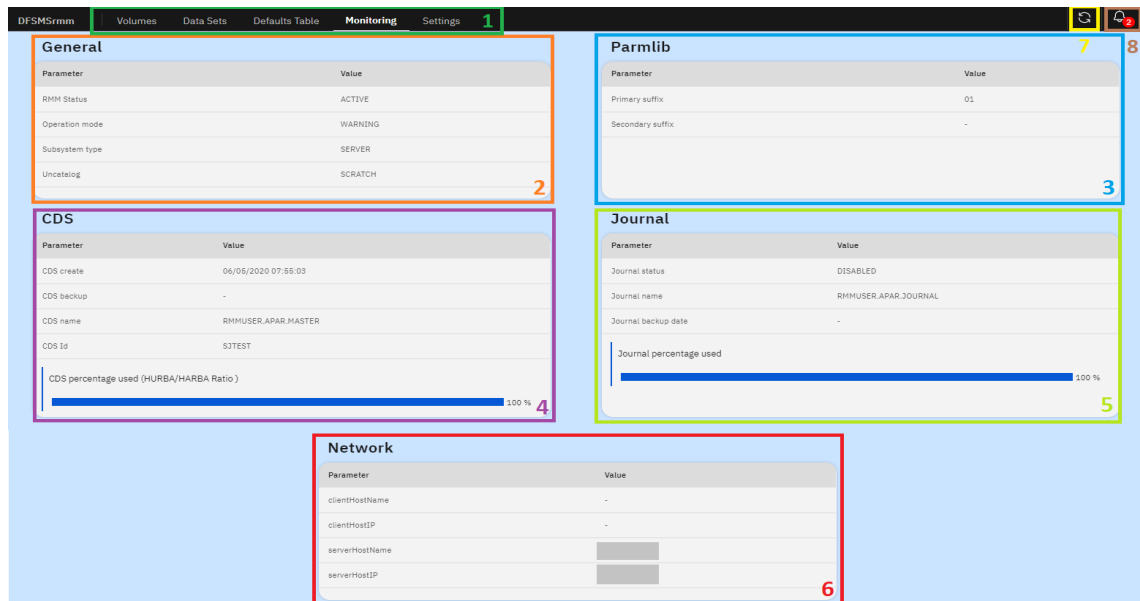
Monitoring tab

The Monitoring tab displays the current RMM status.

To use the Monitoring tab, you need CONTROL access to the STGADMIN.EDG.MASTER resource profile when the STGADMIN.EDG.LISTCONTROL resource profile is not defined.

Or, you might need CONTROL access to the STGADMIN.EDG.LISTCONTROL resource profile when the STGADMIN.EDG.LISTCONTROL resource profile is defined.

The Monitoring tab consists of the following elements:



1. **Top Navigation Panel:** Contains tabs representing the main sections of the DFSMSrmm WUI.
2. **General Status Panel:** Contains general information about the system.
3. **Parmlib Panel:** Contains information about the Parmlib.
4. **CDS Panel:** Contains information about the CDS (Control Data Set).
5. **Journal Panel:** Contains information about the Journal.
6. **Network Information Panel:** Displays connection status when the Subsystem type is Server or Client.
7. **Refresh Button:** Updates the monitoring information.
8. **Notification Button:** Opens window with history of notifications that appear during the current session.

Settings tab

The Settings tab displays connection-related settings specific to the RMM WUI.

The Settings tab consists of the following elements:

1. **Upper Navigation Panel:** Contains tabs that represent the main sections of the DFSMSrmm WUI.
2. **Web Application settings:** Contains parameters of the TSO/E logon procedure that is used to log on to the TSO/E address space, and the **Request Limit** field, which controls the maximum number of elements that are displayed in the **Volumes** and **Data Sets** tabs.
3. **Network logs:** Contains buttons to show logs for the current user and clear logs for the current user.
4. **Page form buttons:**
 - a. **Apply:** Accepts changes only for the current session.
 - b. **Save:** Saves changes for current and all subsequent sessions
 - c. **Restore:** Restores saved settings from the server, overwriting any applied changes on the current session.
5. **Notification Button:** Opens window with history of notifications that appear during the current session.
6. **Remove App Settings button:** Opens window with a method to delete saved application settings that are managed through the z/OSMF persistence API. No RMM CDS data is at risk from this options.

The DFSMSrmm WUI uses utility REXX programs to process response data from RMM TSO commands and convert it to a web-acceptable format (JSON).

The Settings tab has a link to the Network Logs tab that contains request information with non-200 response codes.

Network Logs tab

The Network Logs tab displays connection-related settings specific to RMM.

The Network Logs tab consists of the following elements:

Date	Status	Method	URL	Request body	Response body
Thu, 29 Oct 2020 11:29:24	500	PUT	https://	/* REXX */\nArg sessionNumber limit command\nsysauth....	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:23	500	PUT	https://	/* REXX */\nARG sessionNumber limit VolumeSer\nsysaut...	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:21	500	PUT	https://	/* REXX */\n\nsysauth.edgdate = AMERICAN\nARG sessionN...	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:20	500	PUT	https://	/* REXX */\n\nARG sessionNumber limit DataSet VolumeSeri...	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:18	500	PUT	https://	/* REXX */\n\nsysauth.edgdate = AMERICAN\nARG session...	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:17	500	PUT	https://	/* REXX */\n\nsysauth.edgdate = AMERICAN\nARG \$session...	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:14	500	PUT	https://	/* REXX */\n\nsysauth.edgdate = AMERICAN\nARG sessionN...	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...
Thu, 29 Oct 2020 11:29:13	500	PUT	https://	/* REXX */\n\nARG sessionNumber limit command\nsysauth....	{\"rc\":4,\"reason\":3,\"details\":{\"IEC032I E37-04,IFG0554P,IBMUS...

1. **Top Navigation Panel:** Contains tabs representing the main sections of the DFSMSrmm WUI.
2. **Network logs list:** Contains information about requests that have non-200 response codes. This information can also be tracked in the "Network" tab of the Developer Tools in a browser.
3. **Download table Button:** Downloads the current table to PC in *.csv format.
4. **Notification Button:** Opens window with history of notifications that appear during the current session.

Troubleshooting

Errors when creating TSO session:

Symptom: The DFSMSrmm WUI takes too long to load data into volume table and displays the following message: ISPWB000 Client requested ISPF session initialization

Possible cause: Log on procedure that is defined in Settings, needs actions which the DFSMSrmm WUI cannot interpret.

Corrective action: Use the sample logon procedure stored in the './TSO' directory of the DFSMSrmm WUI installation package.

Symptom: The DFSMSrmm WUI does not load data from the server and displays the following message: Error: The procedure name "XXXXXX" has not been defined for use

Possible cause: Current logon procedure does not exist or is not allowed for the current user.

Corrective action:

- Make sure that the TSO logon procedure name is specified correctly.
- Authorize the current DFSMSrmm WUI user to execute TSO logon procedure that is specified in the DFSMSrmm WUI settings or specify another TSO logon procedure.

Symptom: The DFSMSrmm WUI does not load data from the server and displays the following message: IZUG1121E: The TSO/E address space could not be created because an error occurred with the logon procedure or the user settings

Possible cause: Current user configuration might not allow the creation of a new TSO session in addition to the existing ones.

Corrective action:

1. Force reconnect to the existing TSO sessions must be turned off.

Change user configuration to stop force reconnect to the existing TSO session while creating new TSO session.

2. Parallel TSO session limit might be exceeded.

Logoff from existing TSO sessions and try to open the DFSMSrmm WUI again. You can also cancel the TSO address spaces from the System Display and Search Facility of z/OS.

A new TSO session is created on each restart of the DFSMSrmm WUI. To avoid future issues, enable no less than 2 simultaneous TSO sessions for all users that are using the DFSMSrmm WUI.

If there are other problems that are related to initialization of TSO sessions, you might do the next steps to investigate possible cause of the problem:

- Try to log on with the standard ISPF plug-in for z/OSMF using the logon procedure specified in the DFSMSrmm WUI Settings. Note that DFSMSrmm WUI logon procedure is not able to open ISPF panels. It is not intended to do this. However, you might receive a message from TSO and this indicates that the TSO address space is created successfully.
- Check the Web Application settings, they need to be the same as the settings that the DFSMSrmm WUI user is using to connect to 3270 terminal.
- Try to connect to 3270 terminal with the same settings.

Errors while working with the DFSMSrmm WUI:

Symptom: The DFSMSrmm WUI does not load any data and shows the following warning message: Unable to get settings from the server for the current user. Please, set your settings and press the SAVE button.

Possible cause: The DFSMSrmm WUI did not find user settings on the z/OSMF server. This situation happens when the user opened the DFSMSrmm WUI for the first time or never saved settings before.

Corrective action: Open Settings tab, specify required fields and press Save.

Note: If you change Web Application settings, you need to restart the DFSMSrmm WUI to accept the changes.

Symptom: Response time is too long; data on Volumes or Data Sets tabs takes too long to load.

Possible cause: The DFSMSrmm WUI is processing a large amount of data.

Corrective action: Specify a smaller value in the Request limit field in the DFSMSrmm WUI Settings tab.

Symptom: The DFSMSrmm WUI does not load any data and shows the following error message: Can 't get data from the server. Check your connection and try again.

Possible cause: The DFSMSrmm WUI sent the request, but did not receive a response from the server.

Corrective action: Check your internet connection and press the Refresh button in the right corner above the table.

Symptom: The DFSMSrmm WUI does not load any data and shows the following error message: Can 't parse response from the server

Possible cause: The plug-in received a message from the server, but did not recognize it.

Corrective action:

- Press the “Refresh” button in the right corner above the table.
- Reload the DFSMSrmm WUI.
- Check that z/OSMF APAR PH26103 is installed. As a temporary measure, try to decrease the Request limit parameter in the DFSMSrmm WUI Settings.

Symptom: The DFSMSrmm WUI does not load any data and shows the following error message: Can 't parse settings

Possible cause: The DFSMSrmm WUI received saved user settings from the z/OSMF server, but did not read them.

Corrective action: Enter all settings again on the Settings tab and press the Save button and reload DFSMSrmm WUI.

Symptom: The DFSMSrmm WUI does not load any data and shows error message: DFSMSRMM SUBSYSTEM IS NOT ACTIVE

Possible cause: The DFSMSrmm is not active on the server.

Corrective action: Start DFSMSrmm on the server.

Symptom: The DFSMSrmm WUI tries to load data for a long time and shows the following error message: The request has timed out.

Possible cause: The DFSMSrmm WUI cannot receive a response from the server.

Corrective action: Check your internet connection and connection to the server. Then, reload the DFSMSrmm WUI.

Symptom: The DFSMSrmm WUI does not load any data and shows error message: Data Sets “XXXXXX” not in catalog or catalog can not be accessed or Member “XXXXXX” not in Dataset “YYYYYY”

Possible cause: The DFSMSrmm WUI cannot find some REXX procedures.

Corrective action: The RMM REXX procedures for the z/OSMF plug-in are installed to the SYS1.SEDGEXE1 data set, make sure that it is accessible.

Note: For other types of errors, if they cause the DFSMSrmm WUI to be unstable, reload the DFSMSrmm WUI by closing its window in the z/OSMF user interface or by refreshing the current web browser tab.

Appendix A. DFSMSrmm mapping macros

Note

The descriptions of the following mapping macros are in *z/OS DFSMSrmm Reporting*:

- Report Extract Data Set Mapping Macros in SYS1.MACLIB.

You use the extract data set as input to the DFSMSrmm utility EDGRPTD to create reports.

The extract data set contains information extracted from the DFSMSrmm control data set.

The extract data set records contain all major key fields so that you can select fields and sort them for reports. Variable length fields are expanded to maximum length and redundant control information is removed.

- SMF Records Mapping Macros in SYS1.MODGEN.

DFSMSrmm requires two record types to support audit needs and security needs. You specify the exact SMF record types in EDGRMMxx, using the SMFAUD macro for auditing and the SMFSEC macro for security records.

DFSMSrmm provides these macros as programming interfaces for customers.



Attention:

Do not use as programming interfaces any DFSMSrmm macros other than those identified in this topic.

- Library Control System Interface Macro in SYS1.MODGEN.

[“OAM interface: EDGLCSUP” on page 511](#)

- DFSMSrmm Installation Exit Mapping Macros in SYS1.MODGEN.

[“Installation exit mapping macro: EDGPL100” on page 517](#)

[“Installation exit mapping macro: EDGPL200” on page 524](#)

[“Installation exit mapping macro: EDGPL300” on page 526](#)

- DFSMSrmm Sticky Label Mapping Macro in SYS1.MACLIB.

– [“Sticky label data: EDGSLAB” on page 529](#)

OAM interface: EDGLCSUP

EDGLCSUP maps the Library Control System interface parameter list. See [“Managing system-managed tape library volumes: EDGLCSUX” on page 251](#) for more information about using the OAM interface.

```
Common name: EDGLCSUX Parameter List
Macro ID: EDGLCSUP
DSECT name: LCSUP
Owning component: DFSMSrmm (DF186)
Eye-catcher ID: None
Storage attributes: Subpool: 230
                    Key: N/A
                    Residency: N/A
Size: See LCSUP_LENGTH
Created by: Callers of EDGLCSUX
Pointed to by: Register 1 on entry to EDGLCSUX
Serialization: None
Function: Used by EDGLCSUX and its callers to map
         requests made from LCS installation exits
```

Table 64. Structure LCSUP

Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
0	(0)	STRUCTURE	160	LCSUP	EDGLCSUX parameter list
0	(0)	CHARACTER	16	LCSUP_HDR	Control block header
0	(0)	CHARACTER	8	LCSUP_IDENT	Control block ID
8	(8)	UNSIGNED	1	LCSUP_VERN0	Control block version number
9	(9)	UNSIGNED	1	LCSUP_REVNO	Control block revision number
10	(A)	UNSIGNED	2	LCSUP_SUBPOOL	Control block subpool number
12	(C)	UNSIGNED	4	LCSUP_LENGTH	Control block length
Start of input fields					
16	(10)	BIT(8)	1	LCSUP_FUNCTION	Requested function
		1...		LCSUP_ENT	Caller is CBRUXENT
		.1..		LCSUP_EJC	Caller is CBRUXEJC
		..1.		LCSUP_CUA	Caller is CBRUXCUA
		...1		LCSUP_VNL	Caller is CBRUXVNL
		 1...		LCSUP_ACTVNL	Reentry from CBRUXVNL
18	(12)	BIT(8)	1	LCSUP_STATUSD	Corresponds to MVSFLGD flag
		1...		LCSUP_MVOREAD	Owner may read volume
		.1..		LCSUP_MVOUPD	Owner may update volume
		..1.		LCSUP_MVOALT	Owner may alter volume
		...1		LCSUP_MVPROTR	Read-Only protection
		 1...		LCSUP_MVPROTU	Update protection
		1..		LCSUP_MVMVSUSE	May be used on MVS system
		1.		LCSUP_MVMVUSE	May be used on VM system
19	(13)	BIT(8)	1	LCSUP_STATUSE	Corresponds to MVSFLGE flag
		1...		LCSUP_MVRETSCR	Return to scratch pending
		.1..		LCSUP_MVREPREL	Replace tape pending
		..1.		LCSUP_MVREINIT	Init pending
		...1		LCSUP_MVDEGAUS	Degauss/security erase pending
		 1...		LCSUP_MVROWNER	Return to owner pending
		1..		LCSUP_MVNOWNER	Notify owner pending
20	(14)	SIGNED	4	LCSUP_LCSPL	Pointer to LCS parameter list
Start of output fields					
24	(18)	SIGNED	4	LCSUP_LCSRC	Return code for LCS
28	(1C)	SIGNED	4	LCSUP_LCSRS	Reason code for LCS
Output fields for CBRUXVNL					
32	(20)	CHARACTER	8	LCSUP_LOANLOC	Loan location
40	(28)	CHARACTER	8	LCSUP_LOCATION	Current volume location
48	(30)	UNSIGNED	1	LCSUP_LOCTYPE	Current location type
49	(31)	CHARACTER	8	LCSUP_DEST	Current volume destination
57	(39)	UNSIGNED	1	LCSUP_DESTYPE	Current destination type
58	(3A)	CHARACTER	8	LCSUP_HOME	Volume home location

Table 64. Structure LCSUP (continued)

Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
66	(42)	UNSIGNED	1	LCSUP_HOMETYPE	Volume home location type
67	(43)	CHARACTER	6	LCSUP_RACK	Rack number
73	(49)	CHARACTER	6	LCSUP_BIN	Bin number
79	(4F)	BIT(8)	1	LCSUP_STATUS	Volume status (MVFLGA)
		1... ..		LCSUP_MSTFLG	Volume is master
		.1.. ..		LCSUP_RLSFLG	Volume pending release
		..1.		LCSUP_VRFLG	Vital record - do not release
		...1		LCSUP_ASSFLG	User tape (assigned by library)
		 1...		LCSUP_LONFLG	Tape is on loan
		1..		LCSUP_OPNFLG	Tape opened and not yet closed
		1.		LCSUP_SCRFLG	Volume is scratch
		1		LCSUP_OCEFLG	Volume recorded by O/C/E0V
80	(50)	BIT(8)	1	LCSUP_STATUSX	Volume status (MVFLGAX)
		1... ..		LCSUP_GVCFLG	Scratch volume claimed via GETVOLUME
		.1..		LCSUP_XINFLG	Scratch volume has never been initialised
		..1.		LCSUP_INIFLG	Scratch volume with init action pending
		...1		LCSUP_ENTFLG	Scratch volume waiting to enter into ATL
		 1...		LCSUP_FABEND	Abend in process when a data set closed
		1..		LCSUP_FOCEAB	Abend probably in O/C/E0V
		1.		LCSUP_ATIFLG	Init required for ATL volume
81	(51)	BIT(8)	1	LCSUP_FLAGS	Flag byte
		1... ..		LCSUP_TRANSIT	Volume moving status
		.1..		LCSUP_8197	If on, message EDG8197 is issued
82	(52)	UNSIGNED	1	LCSUP_OPMODE	OPMODE (like TLVOPFLG)
		.1..		LCSUP_OPMODE_MAN	MANUAL mode
		..1.		LCSUP_OPMODE_REC	RECORDING mode
		...1		LCSUP_OPMODE_WARN	WARNING mode
		 1...		LCSUP_OPMODE_PROT	PROTECT mode
84	(54)	SIGNED	4	LCSUP_TDSI	Tape device selection information
88	(58)	CHARACTER	72	LCSUP_VER2SEC	Version 2 section
88	(58)	CHARACTER	16	LCSUP_INCONTAINER	Container
88	(58)	CHARACTER	6	LCSUP_STV	Stacked volume
104	(68)	UNSIGNED	1	LCSUP_VOLUMETYPE	Volume type
105	(69)	BIT(8) IsA(LOCFLAGS)	1	LCSUP_LOCFLAGS	
		 1...		LCSUP_HSTORE	Location type is STORE and HOME
106	(6A)	BIT(8) IsA(LOCFLAGS)	1	LCSUP_DESFLAGS	
		 1...		LCSUP_HSTORE	Location type is STORE and HOME
End output fields for CBRUXVNL					

Table 64. Structure LCSUP (continued)					
Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
160	(A0)	CHARACTER	0	LCSUP_END	End of LCSUP

Table 65. Constants for LCSUP				
Len	Type	Value	Name	Description
Constants used to initialize the LCSUP header section				
8	CHARACTER	EDGLCSUP	LCSUP_IDENTV	Control block ID
1	DECIMAL	1	LCSUP_VER1#	Version 1
1	DECIMAL	2	LCSUP_VER2#	Version 2
1	DECIMAL	2	LCSUP_VER#	Version
1	DECIMAL	0	LCSUP_REV#	Revision number
1	DECIMAL	0	LCSUP_SP#	Subpool number
2	DECIMAL	160	LCSUP_LEN#	Control block length
2	DECIMAL	88	LCSUP_LEN1#	
2	DECIMAL	160	LCSUP_LEN2#	
Constants used to test the location type fields				
1	DECIMAL	0	LCSUP_TYPE_SHELF	Shelf location type
1	DECIMAL	1	LCSUP_TYPE_STORE	Store location type
1	DECIMAL	2	LCSUP_TYPE_MTL	MTL location type
1	DECIMAL	3	LCSUP_TYPE_ATL	ATL location type
1	NUMB HEX	00	LCSUP_VOLUMETYPE_PHYSICAL	
1	NUMB HEX	01	LCSUP_VOLUMETYPE_LOGICAL	
Constants for return codes in R15				
1	DECIMAL	0	LCSUP_RC_OK	Success reason code is set
1	DECIMAL	4	LCSUP_RC_SSNA	DFSMSrmm subsystem not available
1	DECIMAL	8	LCSUP_RC_LERR	Logical error
1	DECIMAL	12	LCSUP_RC_ENV	Environment error
Constants for reason codes in R0 when R15 = LCSUP_RC_OK				
1	DECIMAL	0	LCSUP_RS_OK	Request successfully processed
1	DECIMAL	1	LCSUP_RS_NOACTION	No action performed by RMM
1	DECIMAL	2	LCSUP_RS_DONT	Do not need RMM exits to be called
Constants for reason codes in R0 when R15 = LCSUP_RC_ENV				
1	DECIMAL	1	LCSUP_RS_IDENT	Incorrect value in LCSUP_IDENT
1	DECIMAL	2	LCSUP_RS_VERN0	Incorrect value in LCSUP_VERN0
1	DECIMAL	3	LCSUP_RS_REVNO	Incorrect value in LCSUP_REVNO
1	DECIMAL	4	LCSUP_RS_SUBPOOL	Incorrect value in LCSUP_SUBPOOL
1	DECIMAL	5	LCSUP_RS_LENGTH	Incorrect value in LCSUP_LENGTH
1	DECIMAL	6	LCSUP_RS_FUNCTION	Incorrect value in LCSUP_FUNCTION
1	DECIMAL	7	LCSUP_RS_NSUPV	EDGLCSUX not in supervisor state

Table 65. Constants for LCSUP (continued)				
Len	Type	Value	Name	Description
1	DECIMAL	8	LCSUP_RS_LCSUP	EDGLCSUX parameter list could not be addressed
1	DECIMAL	9	LCSUP_RS_CBRPL	CBRUXXP parameter list could not be addressed
1	DECIMAL	10	LCSUP_RS_ABEND	EDGLCSUX abended
Constants for reason codes returned in LCSUP_LCSRS				
1	DECIMAL	1	LCSUP_RS_PBD	Inconsistent parameter list
1	DECIMAL	2	LCSUP_RS_NMV	Volume not to be used with MVS
1	DECIMAL	3	LCSUP_RS_DEB	Specified destination is not the current library
1	DECIMAL	4	LCSUP_RS_RJP	Undefined volume is rejected by reject prefix
1	DECIMAL	5	LCSUP_RS_SCR	Private to scratch change invalid
1	DECIMAL	6	LCSUP_RS_IVU	User ID not valid for RMM
1	DECIMAL	7	LCSUP_RS_RPX	Retention period exceeds installation maximum
1	DECIMAL	8	LCSUP_RS_NRM	Volume is not RMM managed
1	DECIMAL	9	LCSUP_RS_RIU	Rack to match volume serial number is not available
1	DECIMAL	10	LCSUP_RS_NSL	Label type is not supported in a library
1	DECIMAL	11	LCSUP_RS_IRK	Volume rack inconsistent
1	DECIMAL	12	LCSUP_RS_REL	Volume pending release
1	DECIMAL	13	LCSUP_RS_STA	Volume status is scratch
1	DECIMAL	14	LCSUP_RS_INI	Volume init action pending
1	DECIMAL	15	LCSUP_RS_DUPLV	Logical volume duplicates physical volume
1	DECIMAL	16	LCSUP_RS_NOTEXP	Logical volume is not exported
1	DECIMAL	17	LCSUP_RS_DUPPV	Physical volume duplicates logical volume
1	DECIMAL	18	LCSUP_RS_SMM	Entry volume status mismatch
1	DECIMAL	19	LCSUP_RS_DUPSV	Volume duplicates stacked volume
1	DECIMAL	20	LCSUP_RS_IDL	Volume entry not processed
1	DECIMAL	21	LCSUP_RS_PRI	Volume ignored by PRITITION TYPE(RMM)
1	DECIMAL	22	LCSUP_RS_PNI	Volume ignored by PRITITION TYPE(NORMM)

Table 66. Cross Reference for LCSUP			
Name	Offset	Hex Tag	Level
LCSUP	0		1
LCSUP_ACTVNL	10	08	3
LCSUP_ASSFLG	4F	10	3
LCSUP_ATIFLG	50	02	3
LCSUP_BIN	49		2
LCSUP_CUA	10	20	3
LCSUP_DESFLAGS	6A		3

Table 66. Cross Reference for LCSUP (continued)			
Name	Offset	Hex Tag	Level
LCSUP_DEST	31		2
LCSUP_DESTYPE	39		2
LCSUP_EJC	10	40	3
LCSUP_END	A0		2
LCSUP_ENT	10	80	3
LCSUP_ENTFLG	50	10	3
LCSUP_FABEND	50	08	3
LCSUP_FLAGS	51		2
LCSUP_FOCEAB	50	04	3
LCSUP_FUNCTION	10		2
LCSUP_GVCFLG	50	80	3
LCSUP_HDR	0		2
LCSUP_HOME	3A		2
LCSUP_HOMETYPE	42		2
LCSUP_HSTORE	69	08	4
LCSUP_HSTORE	6A	08	4
LCSUP_IDENT	0		3
LCSUP_INCONTAINER	58		3
LCSUP_INIFLG	50	20	3
LCSUP_LCSPL	14		2
LCSUP_LCSRC	18		2
LCSUP_LCSRS	1C		2
LCSUP_LENGTH	C		3
LCSUP_LOANLOC	20		2
LCSUP_LOCATION	28		2
LCSUP_LOCFLAGS	69		3
LCSUP_LOCTYPE	30		2
LCSUP_LONFLG	4F	08	3
LCSUP_MSTFLG	4F	80	3
LCSUP_MVDEGAUS	13	10	3
LCSUP_MVMVSUSE	12	04	3
LCSUP_MVNOWNER	13	04	3
LCSUP_MVOALT	12	20	3
LCSUP_MVOREAD	12	80	3
LCSUP_MVOUPD	12	40	3
LCSUP_MVPROTR	12	10	3
LCSUP_MVPROTU	12	08	3
LCSUP_MVREINIT	13	20	3
LCSUP_MVREPREL	13	40	3
LCSUP_MVRETSCR	13	80	3
LCSUP_MVROWNER	13	08	3
LCSUP_MVVMUSE	12	02	3

Table 66. Cross Reference for LCSUP (continued)			
Name	Offset	Hex Tag	Level
LCSUP_OCEFLG	4F	01	3
LCSUP_OPMODE	52		2
LCSUP_OPMODE_MAN	52	40	3
LCSUP_OPMODE_PROT	52	08	3
LCSUP_OPMODE_REC	52	20	3
LCSUP_OPMODE_WARN	52	10	3
LCSUP_OPNFLG	4F	04	3
LCSUP_RACK	43		2
LCSUP_REVNO	9		3
LCSUP_RLSFLG	4F	40	3
LCSUP_SCRFLG	4F	02	3
LCSUP_STATUS	4F		2
LCSUP_STATUSD	12		2
LCSUP_STATUSE	13		2
LCSUP_STATUSX	50		2
LCSUP_STV	58		4
LCSUP_SUBPOOL	A		3
LCSUP_TDSI	54		2
LCSUP_TRANSIT	51	80	3
LCSUP_VERNO	8		3
LCSUP_VER2SEC	58		2
LCSUP_VNL	10	10	3
LCSUP_VOLUMETYPE	68		3
LCSUP_VRFLG	4F	20	3
LCSUP_XINFLG	50	40	3
LCSUP_8197	51	40	3

Installation exit mapping macro: EDGPL100

EDGPL100 maps the EDG_EXIT100 installation exit parameter list. See [“Using the DFSMSrmm EDG_EXIT100 installation exit” on page 267](#) for information about using the EDG_EXIT100 installation exit parameter list.

A DSECT addressed by PL100_LABINFO is passed to EDG_EXIT100 user exit. This information is provided by DFSMSrmm to allow sticky label processing by EDG_EXIT100.

```

Common name: EDG_EXIT100 Parameter List
Macro ID: EDGPL100
DSECT name: PL100
Owning component: DFSMSrmm (DF186)
Eye-catcher ID: EDGPL100
Storage attributes: Subpool: 230
                    Key: N/A
                    Residency: N/A
Size: See PL100_LENGTH
Created by: Callers of EDG_EXIT100 exit
Pointed to by: Register 1 on entry to EDG_EXIT100 exit
               modules
Serialization: None
Function: Used by EDG_EXIT100 exit modules and their
          callers to map the parameter list

```

Table 67. Structure PL100

Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
0	(0)	STRUCTURE	288	PL100	EDG_EXIT100 parameter list
0	(0)	CHARACTER	16	PL100_HDR	Control block header
0	(0)	CHARACTER	8	PL100_IDENT	Control block ID
8	(8)	UNSIGNED	1	PL100_VERN0	Control block version number
9	(9)	UNSIGNED	1	PL100_REVNO	Control block revision number
10	(A)	UNSIGNED	2	PL100_SUBPOOL	Control block subpool number
12	(C)	UNSIGNED	4	PL100_LENGTH	Control block length
Start of input fields					
16	(10)	BIT(8)	1	PL100_VALID	Valid functions
		1...		PL100_CAN_IGNORE	Requested volume can be ignored
		.1..		PL100_CAN_VRS	Requested VRS can be updated
		..1.		PL100_CAN_RACKNO	Rack number can be returned
		...1		PL100_CAN_IGNORE_FILE2_TON	
					Data set record can be ignored
		 1...		PL100_CAN_COPYFROM	Tape Copyfrom supported
		1..		PL100_CAN_POOL	Scratch pool name can be set
		1.		PL100_ITS_CLOSE	Exit called at CLOSE/EOV
		1		PL100_CAN_RETMET	Can assign retention method
17	(11)	BIT(8)	1	PL100_INFO	Information byte
		1...		PL100_INFO_IGNORE	Volume ignored by RMM
		.1..		PL100_INFO_NOTRMM	Volume not RMM managed
		..1.		PL100_INFO_DISPDD	Disposition control file entry processed
		...1		PL100_INFO_CMOVE	Disposition control location assignment requests move confirmation at a later time
		 1...		PL100_INFO_USERDATA	USERDATA provided
		1..		PL100_INFO_MTL	Allocated tape drive MTL
		1.		PL100_INFO_DISPLAB	Label requested by disposition DD entry
		1		PL100_INFO_IGNORE_REQUEST_BYRULE	
18	(12)	BIT(8)	1	PL100_VALID2	Valid functions 2
		1...		PL100_CAN_VRSELEXCLUDE	VRSELEXCLUDE supported
20	(14)	CHARACTER	6	PL100_REQ_VOLSER	Requested volume serial number
26	(1A)	CHARACTER	6	PL100_MOUNT_VOLSER	Mounted volume serial number
32	(20)	ADDRESS	4	PL100_WTOPTR	Address of WTO message
Start of output fields					
36	(24)	BIT(8)	1	PL100_FUNCTION	Requested function
		1...		PL100_SET_IGNORE	Volume is ignored
		.1..		PL100_SET_IGNORE_MOUNTED	Mounted volume is ignored
		..1.		PL100_SET_IGNORE_REQUESTED	
					Requested volume is ignored

Table 67. Structure PL100 (continued)

Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
		...1		PL100_SET_IGNORE_FILE2_TON	
					Data set record is ignored
		 1...		PL100_SET_NOLABEL	Suppress sticky label
		1..		PL100_SET_POOL	Scratch pool name is set
		1		PL100_SET_ACLOFF	Request no mount from ACL
37	(25)	BIT(8)	1	PL100_FUNCTION2	Requested function 2
		1...		PL100_SET_CMOVE	Confirm move is required
		.1..		PL100_SET_NOCMOVE	Confirm move is not required
		..1.		PL100_SET_IGNORE_SGNAME	Use system based scratch pooling
		...1		PL100_SET_NOTBYPASS_SAFRC8	
					Do not bypass SAF return code 8
		 1...		PL100_SET_RETMET	Retention method assigned
		1..		PL100_SET_COPYFROM	Tape Copyfrom requested
		1.		PL100_SET_VRSELEXCLUDE	VRSELEXCLUDE requested
38	(26)	UNSIGNED	1	PL100_RETENTIONMETHOD	Retention method
40	(28)	ADDRESS	4	PL100_JFCBPTR	Pointer to copy of JFCB
44	(2C)	CHARACTER	8	PL100_VRS	New VRS management value
52	(34)	CHARACTER	6	PL100_RACKNO	External volume serial number
52	(34)	CHARACTER	6	PL100_POOL	Scratch pool name
60	(3C)	ADDRESS	4	PL100_LABINFO	Address of label information block
Start of version 2 fields					
64	(40)	CHARACTER	69	PL100_LAB_USERDATA	User data for label processing
136	(88)	ADDRESS	4	PL100_LABPTR	Address of prepared label
140	(8C)	CHARACTER	8	PL100_LOCATION	Target location name
148	(94)	BIT(8)	1	PL100_LOCTYPE	Target location type
152	(98)	CHARACTER	8	PL100_DDNAME	DD name
160	(A0)	CHARACTER	8	PL100_DISPDD	Disposition DD name
168	(A8)	CHARACTER	8	PL100_ACCODE	ACC CODE parameter value or blank if no ACC CODE or 'ACC CODE=' is specified
176	(B0)	ADDRESS	4	PL100_ACEROPTR	Address of IGDACERO
Start of version 3 fields					
192	(C0)	CHARACTER	44	PL100_COPYFROM_DSN	TAPE COPYFROM DSN
236	(EC)	CHARACTER	6	PL100_COPYFROM_VOLSER	TAPE COPYFROM DSN VOLSER
244	(F4)	UNSIGNED	4	PL100_COPYFROM_DSEQ	TAPE COPYFROM DSN SEQ#
248	(F8)	CHARACTER	8	PL100_COPYFROM_OWNER	TAPE COPYFROM OWNER (OPT)
288	(120)	CHARACTER	0	PL100_END	End of PL100

The following DSECT is passed to EDG_EXIT100 exit modules and will be addressed by PL100_LABINFO. The information is provided by DFSMSrmm to allow sticky label processing by EDG_EXIT100.

Table 68. Structure PL100_LABDS

Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
0	(0)	STRUCTURE	137	PL100_LABDS	Label information block
0	(0)	CHARACTER	44	PL100_DSN	Data set name
44	(2C)	CHARACTER	1	PL100_LTYP	Label type
		.1..		PL100_AL	ANSI label
		...1		PL100_BLP	Bypass label processing
		 1...		PL100_UL	User label
		1..		PL100_NSL	None standard label
		1.		PL100_SL	Standard label
		1		PL100_NL	No label
The following 2 date fields are copied from the JFCB and thus contain 'YYDDDD' in which the 'YY' is an offset from 1900 and 'DDDD' contain the Julian day of the year. Example: X'590008' represents January 8, 1989.					
45	(2D)	CHARACTER	3	PL100_CRDT	Creation date YYDDDD
48	(30)	CHARACTER	3	PL100_XPDT	Expiration date YYDDDD
51	(33)	CHARACTER	1	PL100_OFLAG	Open flags
		1...		PL100_FOUT	Data set opened for output
		..1.		PL100_FEOV	Call for end of volume
52	(34)	CHARACTER	6	PL100_VOLSER	Volume serial number
58	(3A)	CHARACTER	8	PL100_JOBNAME	Job name
66	(42)	CHARACTER	8	PL100_STPNAM	Step name
74	(4A)	CHARACTER	8	PL100_SYSTEM	System ID
82	(52)	SIGNED	2	PL100_LRECL	Logical record length
84	(54)	SIGNED	2	PL100_BLKSI	Blocksize
86	(56)	CHARACTER	2	PL100_UNIT	Unit address (binary)
88	(58)	SIGNED	4	PL100_BLK#	Number of blocks written
92	(5C)	UNSIGNED	2	PL100_FSCT	Data set sequence count
94	(5E)	CHARACTER	1	PL100_RECFCM	Record format
		11..		PL100_UND	Undefined
		1...		PL100_FIX	Fixed
		.1..		PL100_VAR	Variable
		..1.		PL100_RFO	Track overflow
		...1		PL100_RFB	Blocked
		 1...		PL100_RFS	Standard/spanned record
		1..		PL100_ASA	ASA control characters
		1.		PL100_MAC	Machine control characters
95	(5F)	CHARACTER	1	PL100_NVOL	Volume sequence number
96	(60)	CHARACTER	1	PL100_DEN	Tape density from JFCB
PL100_TDSI1/2 are no longer used - use PL100_TEPMTDS1/2/3/4 instead					
97	(61)	BIT(8)	1	PL100_TDSI1_OLD	TDSI byte 1
98	(62)	CHARACTER	1	PL100_TDSI2_OLD	TDSI byte 2

Table 68. Structure PL100_LABDS (continued)					
Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
Start of VERSION 2 fields					
99	(63)	CHARACTER	8	PL100_MEDIANAME	Volume media name
107	(6B)	CHARACTER	4	PL100_CRDATE	Data set creation date as packed decimal YYYYDDD
111	(6F)	CHARACTER	4	PL100_CRTIME	Data set creation time as packed decimal HHMMSS
115	(73)	CHARACTER	4	PL100_XPDATE	Data set expiration date as packed decimal YYYYDDD
119	(77)	CHARACTER	6	PL100_PREVOL	Previous volume in sequence
125	(7D)	CHARACTER	8	PL100_CRJOB	Creating job name
133	(85)	CHARACTER	4	PL100_TEPMTDSI	TDSI from IFGTPE work area
133	(85)	CHARACTER	1	PL100_TEPMTDS1	Recording technology
134	(86)	CHARACTER	1	PL100_TEPMTDS2	Media type
135	(87)	CHARACTER	1	PL100_TEPMTDS3	Compaction
136	(88)	CHARACTER	1	PL100_TEPMTDS4	Special attributes

Table 69. Constants for PL100				
Len	Type	Value	Name	Description
Constants used to initialize the PL100 header section				
8	CHARACTER	EDGPL100	PL100_IDENTV	Control block ID
1	DECIMAL	3	PL100_VER#	Version number
1	DECIMAL	0	PL100_REV#	Revision number
1	DECIMAL	0	PL100_SP#	Subpool number
2	DECIMAL	288	PL100_LEN#	Control block length
Constants to compaction and read compatibility for 4 byte TDSI				
1	DECIMAL	1	PL100_TEPM_NOCMP	Compaction not used
1	DECIMAL	2	PL100_TEPM_BIDRC	Compaction yes (IDRC)
1	DECIMAL	1	PL100_TEPM_RDCOM	Read compatibility
Constants to define location type				
1	HEX	00	PL100_LOC_LOAN	Loan location
1	HEX	01	PL100_LOC_STORE	Storage location
1	HEX	02	PL100_LOC_LIBRARY	Library
Constants to define retention method				
1	DECIMAL	0	PL100_RM_VRSEL	
1	DECIMAL	1	PL100_RM_EXPDT	

Table 70. Cross Reference for PL100			
Name	Offset	Hex Tag	Level
PL100	0		1

Table 70. Cross Reference for PL100 (continued)			
Name	Offset	Hex Tag	Level
PL100_ACCODE	A8		2
PL100_ACEROPTR	B0		2
PL100_AL	2C	40	3
PL100_ASA	5E	04	3
PL100_BLK#	58		2
PL100_BLKSI	54		2
PL100_BLP	2C	10	3
PL100_CAN_COPYFROM	10	08	3
PL100_CAN_IGNORE	10	80	3
PL100_CAN_IGNORE_FILE2_TON	10	10	3
PL100_CAN_POOL	10	04	3
PL100_CAN_RACKNO	10	20	3
PL100_CAN_RETMET	10	01	3
PL100_CAN_VRS	10	40	3
PL100_CAN_VRSELEXCLUDE	12	80	3
PL100_COPYFROM_DSEQ	F4		2
PL100_COPYFROM_DSN	C0		2
PL100_COPYFROM_OWNER	F8		2
PL100_COPYFROM_VOLSER	EC		2
PL100_CRDATE	6B		2
PL100_CRDT	2D		2
PL100_CRJOB	7D		2
PL100_CRTIME	6F		2
PL100_DDNAME	98		2
PL100_DEN	60		2
PL100_DISPDD	A0		2
PL100_DSN	0		2
PL100_END	120		2
PL100_FE0V	33	20	3
PL100_FIX	5E	80	4
PL100_FOUT	33	80	3
PL100_FSCT	5C		2
PL100_FUNCTION	24		2
PL100_FUNCTION2	25		2
PL100_HDR	0		2
PL100_IDENT	0		3
PL100_INFO	11		2
PL100_INFO_CMOVE	11	10	3
PL100_INFO_DISPDD	11	20	3
PL100_INFO_DISPLAB	11	02	3
PL100_INFO_IGNORE	11	80	3
PL100_INFO_IGNORE_REQUEST_BYRULE	11	01	3

Table 70. Cross Reference for PL100 (continued)			
Name	Offset	Hex Tag	Level
PL100_INFO_MTL	11	04	3
PL100_INFO_NOTRMM	11	40	3
PL100_INFO_USERDATA	11	08	3
PL100_ITS_CLOSE	10	02	3
PL100_JFCBPTR	28		2
PL100_JOBNAME	3A		2
PL100_LAB_USERDATA	40		2
PL100_LABDS	0		1
PL100_LABINFO	3C		2
PL100_LABPTR	88		2
PL100_LENGTH	C		3
PL100_LOCATION	8C		2
PL100_LOCTYPE	94		2
PL100_LRECL	52		2
PL100_LTYP	2C		2
PL100_MAC	5E	02	3
PL100_MEDIANAME	63		2
PL100_MOUNT_VOLSER	1A		2
PL100_NL	2C	01	3
PL100_NSL	2C	04	3
PL100_NVOL	5F		2
PL100_OFLAG	33		2
PL100_POOL	34		3
PL100_PREVOL	77		2
PL100_RACKNO	34		2
PL100_RECFM	5E		2
PL100_REQ_VOLSER	14		2
PL100_RETENTIONMETHOD	26		2
PL100_REVNO	9		3
PL100_RFB	5E	10	3
PL100_RFO	5E	20	3
PL100_RFS	5E	08	3
PL100_SET_ACLOFF	24	01	3
PL100_SET_CMOVE	25	80	3
PL100_SET_COPYFROM	25	04	3
PL100_SET_IGNORE	24	80	3
PL100_SET_IGNORE_FILE2_TON	24	10	3
PL100_SET_IGNORE_MOUNTED	24	40	3
PL100_SET_IGNORE_REQUESTED	24	20	3
PL100_SET_IGNORE_SGNAME	25	20	3
PL100_SET_NOCMOVE	25	40	3
PL100_SET_NOLABEL	24	08	3

Table 70. Cross Reference for PL100 (continued)			
Name	Offset	Hex Tag	Level
PL100_SET_NOTBYPASS_SAFRC8	25	10	3
PL100_SET_P00L	24	04	3
PL100_SET_RETMET	25	08	3
PL100_SET_VRSELEXCLUDE	25	02	3
PL100_SL	2C	02	3
PL100_STPNAM	42		2
PL100_SUBP00L	A		3
PL100_SYSTEM	4A		2
PL100_TDSI1_OLD	61		2
PL100_TDSI2_OLD	62		2
PL100_TEPMTDSI	85		2
PL100_TEPMTDS1	85		3
PL100_TEPMTDS2	86		3
PL100_TEPMTDS3	87		3
PL100_TEPMTDS4	88		3
PL100_UL	2C	08	3
PL100_UND	5E	C0	3
PL100_UNIT	56		2
PL100_VALID	10		2
PL100_VALID2	12		2
PL100_VAR	5E	40	4
PL100_VERNO	8		3
PL100_VOLSER	34		2
PL100_VRS	2C		2
PL100_WTOPTR	20		2
PL100_XPDATE	73		2
PL100_XPDT	30		2

Installation exit mapping macro: EDGPL200

EDGPL200 maps the parameter list for the EDG_EXIT200 installation exit. See “Using the EDG_EXIT200 installation exit” on page 300 for information about using the EDG_EXIT200 installation exit.

```

Common name: EDG_EXIT200 Parameter List
Macro ID: EDGPL200
DSECT name: PL200
Owning component: DFSMSrmm (DF186)
Eye-catcher ID: EDGPL200
Storage attributes: Subpool: 0
                    Key: N/A
                    Residency: N/A
Size: See PL200_LENGTH
Created by: Callers of EDG_EXIT200 exit
Pointed to by: Register 1 on entry to EDG_EXIT200 exit
               modules
Serialization: None
Function: Used by EDG_EXIT200 exit modules and their
          callers to map the parameter list

```

Table 71. Structure PL200

Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
0	(0)	STRUCTURE	272	PL200	EDG_EXIT200 parameter list
0	(0)	CHARACTER	16	PL200_HDR	Control block header
0	(0)	CHARACTER	8	PL200_IDENT	Control block ID
8	(8)	UNSIGNED	1	PL200_VERN0	Control block version number
9	(9)	UNSIGNED	1	PL200_REVNO	Control block revision number
10	(A)	UNSIGNED	2	PL200_SUBPOOL	Control block subpool number
12	(C)	UNSIGNED	4	PL200_LENGTH	Control block length
Start of input fields					
16	(10)	BIT(8)	1	PL200_VALID	Valid functions
		1...		PL200_CAN_SCRTCH	Can handle return to scratch
20	(14)	CHARACTER	6	PL200_VOLSER	RMM defined volume serial number
26	(1A)	CHARACTER	6	PL200_RACK_NUMBER	RMM defined rack number
32	(20)	CHARACTER	8	PL200_MEDIA_NAME	Volume media name
40	(28)	CHARACTER	8	PL200_LOCATION	Volume location
48	(30)	UNSIGNED	2	PL200_VOLSEQ	Volume sequence number
50	(32)	CHARACTER	44	PL200_DSNAME	First file data set name
94	(5E)	BIT(8)	1	PL200_VOLUME_FLAGS	Status flags for volume
		1...		PL200_SMS_VOL	Volume is SMS managed
		.1..		PL200_HOME_LOCDEF	Volume is in storage location defined as home
		..1.		PL200_MANUAL_SCRATCH	Volume is in VLP00L with AUTOSCRATCH(NO)
Start of output fields					
95	(5F)	BIT(8)	1	PL200_FUNCTION	Requested function
		1...		PL200_SET_NOSCRTCH	Do not return to scratch
		.1..		PL200_SET_IGNORE_DSN	Ignore data set information
96	(60)	CHARACTER	30	PL200_DESCRIPTION	User description
126	(7E)	CHARACTER	8	PL200_OWNER	Volume owner ID
136	(88)	SIGNED	4	PL200_EDGVREC_ADDR	Address of volume information
140	(8C)	CHARACTER	8	PL200_CATSYSID(16)	CATSYSID list for the running system
272	(110)	CHARACTER	0	PL200_END	End of PL200

Table 72. Constants for PL200

Len	Type	Value	Name	Description
Constants used to initialize the PL200 header section				
8	CHARACTER	EDGPL200	PL200_IDENTV	Control block ID
1	DECIMAL	1	PL200_VER#	Version number
1	DECIMAL	1	PL200_REV#	Revision number
1	DECIMAL	0	PL200_SP#	Subpool number
2	DECIMAL	272	PL200_LEN#	Control block length

Table 73. Cross Reference for PL200			
Name	Offset	Hex Tag	Level
PL200	0		1
PL200_CAN_SCRATCH	10	80	3
PL200_CATSYSID	8C		2
PL200_DESCRIPTION	60		2
PL200_DSNAME	32		2
PL200_EDGVREC_ADDR	88		2
PL200_END	110		2
PL200_FUNCTION	5F		2
PL200_HDR	0		2
PL200_HOME_LOCDEF	5E	40	3
PL200_IDENT	0		3
PL200_LENGTH	C		3
PL200_LOCATION	28		2
PL200_MANUAL_SCRATCH	5E	20	3
PL200_MEDIA_NAME	20		2
PL200_OWNER	7E		2
PL200_RACK_NUMBER	1A		2
PL200_REVNO	9		3
PL200_SET_IGNORE_DSN	5F	40	3
PL200_SET_NOSCRATCH	5F	80	3
PL200_SMS_VOL	5E	80	3
PL200_SUBPOOL	A		3
PL200_VALID	10		2
PL200_VERNO	8		3
PL200_VOLSEQ	30		2
PL200_VOLSER	14		2
PL200_VOLUME_FLAGS	5E		2

Installation exit mapping macro: EDGPL300

EDGPL300 maps the DFSMSrmm installation exit, EDG_EXIT300, parameter list. See “Using the EDG_EXIT300 installation exit” on page 305 for information about using the EDG_EXIT300 installation exit.

```

Common name: EDG_EXIT300 Parameter List
Macro ID: EDGPL300
DSECT name: PL300
Owning component: DFSMSrmm (DF186)
Eye-catcher ID: EDGPL300
Storage attributes: Subpool: 230
                    Key: N/A
                    Residency: N/A
Size: See PL300_LENGTH
Created by: Callers of EDG_EXIT300 exit
Pointed to by: Register 1 on entry to EDG_EXIT300 exit
               modules
Serialization: None
Function: Used by EDG_EXIT300 exit modules and their
          callers to map the parameter list

```

Table 74. Structure PL300					
Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
0	(0)	STRUCTURE	80	PL300	EDG_EXIT300 parameter list
0	(0)	CHARACTER	16	PL300_HDR	Control block header
0	(0)	CHARACTER	8	PL300_IDENT	Control block ID
8	(8)	UNSIGNED	1	PL300_VERN0	Control block version number
9	(9)	UNSIGNED	1	PL300_REVNO	Control block revision number
10	(A)	UNSIGNED	2	PL300_SUBPOOL	Control block subpool number
12	(C)	UNSIGNED	4	PL300_LENGTH	Control block length
Start of input fields					
16	(10)	CHARACTER	28	PL300_INPUT	Start input fields
16	(10)	BIT(8)	1	PL300_FUNCTION	Requested function
		1... ..		PL300_MEDINF	Gather media information
20	(14)	ADDRESS	4	PL300_LSTTEP	IFGTEP tape exit parameter
24	(18)	ADDRESS	4	PL300_LSTMDNFP	Media information table
28	(1C)	CHARACTER	6	PL300_LSTVOL	Volume serial number
34	(22)	CHARACTER	2	PL300_LSTUNIT	Unit address (binary)
36	(24)	CHARACTER	8	PL300_LSTMDNF	Volume media information name
Start of output fields					
44	(2C)	CHARACTER	36	PL300_OUTPUT	Start output fields
44	(2C)	BIT(32)	4	PL300_LSTOFLGS	Media information processing flags
44	(2C)	BIT(8)	1	PL300_LSTOFLG1	Media information first processing flag byte
		1... ..		PL300_LSTOFMV	Volume is not an IBM media
		.1... ..		PL300_LSTOFFX	Returned data external format
48	(30)	SIGNED	4	PL300_LSTORC	Vendor API return code
52	(34)	SIGNED	4	PL300_LSTORS	Vendor API reason code
56	(38)	CHARACTER	4	PL300_LSTOTDSI	Tape data set information (CBRTDSI)
56	(38)	CHARACTER	1	PL300_LSTOTDSI_RECTK	Recording technology
57	(39)	CHARACTER	1	PL300_LSTOTDSI_MEDIA	Mediatype
58	(3A)	CHARACTER	1	PL300_LSTOTDSI_COMP	Compaction
59	(3B)	CHARACTER	1	PL300_LSTOTDSI_SATR	Special attributes
60	(3C)	CHARACTER	8	PL300_LSTOMDTX	Media type external format
68	(44)	CHARACTER	8	PL300_LSTOMDRX	Recording format external format
76	(4C)	UNSIGNED	4	PL300_LSTOMCAP	Medium capacity (MB)
80	(50)	CHARACTER	0	PL300_END	End of PL300

Table 75. Constants for PL300				
Len	Type	Value	Name	Description
Constants used to initialize the PL300 header section				
8	CHARACTER	EDGPL300	PL300_IDENTV	Control block ID
1	DECIMAL	1	PL300_VER#	Version number

Table 75. Constants for PL300 (continued)				
Len	Type	Value	Name	Description
1	DECIMAL	0	PL300_REV#	Revision number
1	DECIMAL	0	PL300_SP#	Subpool number
2	DECIMAL	80	PL300_LEN#	Control block length

Table 76. Cross Reference for PL300			
Name	Offset	Hex Tag	Level
PL300	0		1
PL300_END	50		2
PL300_FUNCTION	10		3
PL300_HDR	0		2
PL300_IDENT	0		3
PL300_INPUT	10		2
PL300_LENGTH	C		3
PL300_LSTMDNF	24		3
PL300_LSTMDNFP	18		3
PL300_LSTOFFX	2C	40	5
PL300_LSTOFLGS	2C		3
PL300_LSTOFLG1	2C		4
PL300_LSTOFMV	2C	80	5
PL300_LSTOMCAP	4C		3
PL300_LSTOMDRX	44		3
PL300_LSTOMDTX	3C		3
PL300_LSTORC	30		3
PL300_LSTORS	34		3
PL300_LSTOTDSI	38		3
PL300_LSTOTDSI_COMP	3A		4
PL300_LSTOTDSI_MEDIA	39		4
PL300_LSTOTDSI_RECTK	38		4
PL300_LSTOTDSI_SATR	3B		4
PL300_LSTTEP	14		3
PL300_LSTUNIT	22		3
PL300_LSTVOL	1C		3
PL300_MEDINF	10	80	4
PL300_OUTPUT	2C		2
PL300_REVNO	9		3
PL300_SUBPOOL	A		3
PL300_VERNO	8		3

Sticky label data: EDGSLAB

EDGSLAB maps the DFSMSrmm sticky label data area. See Chapter 18, “Setting up DFSMSrmm disposition processing,” on page 467 for more information about the default sticky labels you can request with DFSMSrmm disposition processing.

Common name: Sticky Label Layout
 Macro ID: EDGSLAB
 DSECT name: SLAB
 Owning component: DFSMSrmm (DF186)
 Eye-catcher ID: EDGSLAB
 Storage attributes: Subpool: 230
 Key: N/A
 Residency: N/A
 Size: See SLABSIZE
 Created by: RMM close processing
 Pointed to by: EDGPL100 on entry to EDG_EXIT100
 Serialization: None
 Function: Used by EDG_EXIT100 and its callers to map
 the sticky label layout

Table 77. Structure SLAB					
Offset Dec	Offset Hex	Type	Len	Name(Dim)	Description
0	(0)	STRUCTURE	2024	SLAB	
0	(0)	CHARACTER	8	SLABID	Identifier 'EDGSLAB '
8	(8)	UNSIGNED	1	SLABSPL	Subpool number
9	(9)	UNSIGNED	3	SLABSIZE	Total size
12	(C)	UNSIGNED	1	SLABKEY	Protection key
13	(D)	UNSIGNED	1	SLABVER	Version number
14	(E)	UNSIGNED	1	SLABLRECL	Output file LRECL
15	(F)	BIT(8)	1	SLABTYPE	Label type
		1...		SLABTYPE_CART	Cartridge label built
		.1..		SLABTYPE_REEL	Reel label built
16	(10)	UNSIGNED	1	SLABCOL	Number of columns
17	(11)	UNSIGNED	1	SLABROW	Number of rows
20	(14)	CHARACTER	2000	SLABLAB	Sticky label
20	(14)	CHARACTER	2000	SLABMAX	Maximum size
20	(14)	CHARACTER	800	SLABCART	Cartridge label layout
20	(14)	CHARACTER	80	SLABCLN1	Record 1
20	(14)	CHARACTER	44	SLABCDN	Cartridge label data set name
100	(64)	CHARACTER	80	SLABCLN2	Record 2
100	(64)	CHARACTER	69	SLABCUSR	User data
180	(B4)	CHARACTER	80	SLABCLN3	Record 3
260	(104)	CHARACTER	80	SLABCLN4	Record 4
261	(105)	CHARACTER	8	SLABCJBN	Cartridge label job name
274	(112)	CHARACTER	10	SLABCCRD	Cartridge label creation date
340	(154)	CHARACTER	80	SLABCLN5	Record 5
354	(162)	CHARACTER	10	SLABCEXP	Cartridge label expiration date
420	(1A4)	CHARACTER	80	SLABCLN6	Record 6
421	(1A5)	CHARACTER	4	SLABCDEN	Cartridge label density
426	(1AA)	CHARACTER	4	SLABCCMP	Cartridge label compaction
431	(1AF)	CHARACTER	5	SLABCLRC	Cartridge label LRECL

Table 77. Structure SLAB (continued)					
Offset Dec	Offset Hex	Type	Len	Name (Dim)	Description
437	(1B5)	CHARACTER	6	SLABCLBK	Cartridge label BLKSIZE
445	(1BD)	CHARACTER	4	SLABCRCF	Cartridge label RECFM
500	(1F4)	CHARACTER	80	SLABCLN7	Record 7
580	(244)	CHARACTER	80	SLABCLN8	Record 8
581	(245)	CHARACTER	6	SLABCVSL	Cartridge label volume serial number
588	(24C)	CHARACTER	4	SLABCSQN	Cartridge label volume sequence
593	(251)	CHARACTER	3	SLABCLAB	Cartridge label volume label label type
603	(25B)	CHARACTER	4	SLABCDVC	Cartridge label device number
660	(294)	CHARACTER	80	SLABCLN9	Record 9
740	(2E4)	CHARACTER	80	SLABCLNA	Record 10
20	(14)	CHARACTER	800	SLABTAPE	Tape label layout
20	(14)	CHARACTER	80	SLABTLN1	Record 1
20	(14)	CHARACTER	44	SLABTDSN	Tape label data set name
100	(64)	CHARACTER	80	SLABTLN2	Record 2
100	(64)	CHARACTER	69	SLABTUSR	Tape data
180	(B4)	CHARACTER	80	SLABTLN3	Record 3
184	(B8)	CHARACTER	8	SLABTJBN	Tape label job name
210	(D2)	CHARACTER	10	SLABTCRD	Tape label creation date
260	(104)	CHARACTER	80	SLABTLN4	Record 4
340	(154)	CHARACTER	80	SLABTLN5	Record 5
341	(155)	CHARACTER	4	SLABTDEN	Tape label density
346	(15A)	CHARACTER	4	SLABTCMP	Tape label compaction
351	(15F)	CHARACTER	5	SLABTLRC	Tape label LRECL
357	(165)	CHARACTER	6	SLABTLBK	Tape label BLKSIZE
365	(16D)	CHARACTER	4	SLABTRCF	Tape label RECFM
370	(172)	CHARACTER	10	SLABTEXP	Tape label expiration date
420	(1A4)	CHARACTER	80	SLABTLN6	Record 6
500	(1F4)	CHARACTER	80	SLABTLN7	Record 7
515	(203)	CHARACTER	6	SLABTVSL	Tape label volume serial number
522	(20A)	CHARACTER	4	SLABTSQN	Tape label volume sequence
527	(20F)	CHARACTER	3	SLABTLAB	Tape label volume label type
536	(218)	CHARACTER	4	SLABTDVC	Tape label device number
580	(244)	CHARACTER	80	SLABTLN8	Record 8
660	(294)	CHARACTER	80	SLABTLN9	Record 9
740	(2E4)	CHARACTER	80	SLABTLNA	Record 10
2024	(7E8)	CHARACTER	0	SLABEND	End of sticky label layout

Table 78. Constants for SLAB				
Len	Type	Value	Name	Description
1	DECIMAL	1	SLABVER#	Version number
1	DECIMAL	5	SLABKEY#	Key number
1	DECIMAL	230	SLABSP#	Subpool number

Table 78. Constants for SLAB (continued)				
Len	Type	Value	Name	Description
1	DECIMAL	10	SLABROW#	Default number of rows
1	DECIMAL	80	SLABCOL#	Default number of columns
1	DECIMAL	80	SLABLRECL#	Default number of columns
2	DECIMAL	2024	SLABLNG	

Table 79. Cross Reference for SLAB			
Name	Offset	Hex Tag	Level
SLAB	0		1
SLABCART	14		3
SLABCBK	1B5		5
SLABCCMP	1AA		5
SLABCCRD	112		5
SLABCDEN	1A5		5
SLABCDSN	14		5
SLABCDVC	25B		5
SLABCEXP	162		5
SLABCJBN	105		5
SLABCLAB	251		5
SLABCLNA	2E4		4
SLABCLN1	14		4
SLABCLN2	64		4
SLABCLN3	B4		4
SLABCLN4	104		4
SLABCLN5	154		4
SLABCLN6	1A4		4
SLABCLN7	1F4		4
SLABCLN8	244		4
SLABCLN9	294		4
SLABCLRC	1AF		5
SLABCOL	10		2
SLABCRCF	1BD		5
SLABCSQN	24C		5
SLABCUSR	64		5
SLABCVSL	245		5
SLABEND	7E8		2
SLABID	0		2
SLABKEY	C		2
SLABLAB	14		2
SLABLRECL	E		2
SLABMAX	14		3
SLABROW	11		2
SLABSIZE	9		2
SLABSPL	8		2

Table 79. Cross Reference for SLAB (continued)			
Name	Offset	Hex Tag	Level
SLABTAPE	14		3
SLABTBLK	165		5
SLABTCMP	15A		5
SLABTCRD	D2		5
SLABTDEN	155		5
SLABTDSN	14		5
SLABTDVC	218		5
SLABTEXP	172		5
SLABTJBN	B8		5
SLABTLAB	20F		5
SLABTLNA	2E4		4
SLABTLN1	14		4
SLABTLN2	64		4
SLABTLN3	B4		4
SLABTLN4	104		4
SLABTLN5	154		4
SLABTLN6	1A4		4
SLABTLN7	1F4		4
SLABTLN8	244		4
SLABTLN9	294		4
SLABTLRC	15F		5
SLABTRCF	16D		5
SLABTSQN	20A		5
SLABTUSR	64		5
SLABTVSL	203		5
SLABTYPE	F		2
SLABTYPE_CART	F	80	3
SLABTYPE_REEL	F	40	3
SLABVER	D		2

Appendix B. Using DFSMSrmm samples

DFSMSrmm provides several samples in SAMPLIB, SMPSTS, and SYS1.SEDGEXE1. [Table 80 on page 533](#) lists the samples that are available and where they can be found after SMP/E APPLY processing. After SMP/E ACCEPT processing, samples in SAMPLIB move to ASAMPLIB and samples in SMPSTS move to the AEDGSRCL library.

You can use the IBM Tivoli Workload Scheduler for z/OS sample jobs or procedures with other scheduling systems. In some cases, you must modify the sample jobs.

Table 80. SAMPLIB and SMPSTS Members

Member Name	Shows You How To	Supplied In
CBRUXCUA	Use programming interface to EDGLCSUX	SMPSTS
CBRUXEJC	Use programming interface to EDGLCSUX	SMPSTS
CBRUXENT	Use programming interface to EDGLCSUX	SMPSTS
CBRUXVNL	Use programming interface to EDGLCSUX	SMPSTS
EDG3IIP1	Update IATIIP1 to force DEFER for all tape requests	SAMPLIB
EDG3LVVR	Update IATLVVR to AWAIT MSGDISP for scratch mounts	SAMPLIB
EDG3UX29	Install a JES3 USERMOD	SAMPLIB
EDG3UX62	Update IATUX62 to override JES3 rejection of standard label tapes	SAMPLIB
EDG3UX71	Update IATUX71 to replace and append text to JES3 fetch and mount messages and to provide text for tape drive displays	SAMPLIB
EDGAPISR	Use the DFSMSrmm API by means of EDGXCI	SMPSTS
EDGBETT	Sample procedure for Tivoli event trigger tracking of backup	SAMPLIB
EDGCLIBQ	Use reports for VM tape volumes	SAMPLIB
EDGDFRMM	Create a procedure in SYS1.PROCLIB	SAMPLIB
EDGGAUD1	SMF Audit of Volumes by Volser	SAMPLIB
EDGGAUD2	SMF Audit of Volume by Rack	SAMPLIB
EDGGAUD3	SMF42 Audit of Volumes by Volser	SAMPLIB
EDGGAUD4	SMF42 Audit of Volume by Rack	SAMPLIB
EDGGDSNM	Mixed Case data sets Retained by VRS	SAMPLIB
EDGGREPL	Volumes to be replaced	SAMPLIB
EDGGREPV	Volumes to be replaced based on defined criteria	SAMPLIB
EDGGR01	Scratch tapes by volume serial	SAMPLIB
EDGGR02	List of SCRATCH Volumes by Dataset Name	SAMPLIB
EDGGR03	Inventory List by Volume Serial	SAMPLIB
EDGGR04	Inventory List by Dataset Name	SAMPLIB
EDGGR06	Inventory of Volumes by Location	SAMPLIB

Table 80. SAMPLIB and SMPSTS Members (continued)

Member Name	Shows You How To	Supplied In
EDGGR07	Inventory of Dataset by Location	SAMPLIB
EDGGR08	Inventory of Bin by Location	SAMPLIB
EDGGR09	Datasets in Loan Location	SAMPLIB
EDGGR10	Volumes in Loan Location	SAMPLIB
EDGGR11	List MultiVolume and MultiFile Sets	SAMPLIB
EDGGR12	Movement Report by Dataset	SAMPLIB
EDGGR13	Movement Report by Bin	SAMPLIB
EDGGR14	Movement Report by Volume Serial	SAMPLIB
EDGGR15	Volume Inventory Including Volume Count	SAMPLIB
EDGGSEC1	Report of Accesses to Secure Volumes	SAMPLIB
EDGGSEC2	SMF42 Report of Accesses to Secure Volumes	SAMPLIB
EDGHCLT	Sample shows how to issue RMM subcommands using DFSMSrmm classes and methods	SAMPLIB
EDGIVPPM	Parmlib member for supplied Installation Verification Program (IVP)	SAMPLIB
EDGIVP1	IVP job 1 - initializes tape volumes	SAMPLIB
EDGIVP2	IVP job 2 - uses tape volumes	SAMPLIB
EDGJACTP	Print the ACTIVITY file	SAMPLIB
EDGJAUDM	Create a monthly archive from weekly audit reports	SAMPLIB
EDGJAUDW	Create a weekly archive from daily audit reports	SAMPLIB
EDGJBCAV	Build RMM ADDVOLUME subcommands from a list of barcode scanned volumes	SAMPLIB
EDGJBKP1	Sample Tivoli job for running backup	SAMPLIB
EDGJBKP2	Sample Tivoli job for running backup	SAMPLIB
EDGJBKUP	Sample JCL for using the backup program	SAMPLIB
EDGJCEXP	Sample job to report on copies of logical volumes exported from TS7700 Virtualization Engine.	SAMPLIB
EDGJCMOV	Sample Tivoli job for confirming volume moves	SAMPLIB
EDGJCOMB	Audit tape library using a list of barcode scanned volumes	SAMPLIB
EDGJCVB	Create a report of volumes in a storage location	SAMPLIB
EDGJDEF	Converts the UXTABLE to Defaults Table	SAMPLIB
EDGJDHKP	Sample Tivoli job for running daily inventory management	SAMPLIB
EDGJDSN	Create a report of data sets sorted by data set name	SAMPLIB
EDGJEJC	Sample Tivoli job for ejecting volumes from system-managed libraries	SAMPLIB
EDGJEXP	Sample Tivoli job for running expiration processing	SAMPLIB

Table 80. SAMPLIB and SMPSTS Members (continued)

Member Name	Shows You How To	Supplied In
EDGJHKPA	Sample JCL for allocating the data sets required for inventory management	SAMPLIB
EDGJHSKP	Sample JCL for using the utility program EDGHSKP	SAMPLIB
EDGJIMPC	Sample JCL to create an import list from CLIST output	SAMPLIB
EDGJINER	Sample JCL for using the utility program EDGINERS	SAMPLIB
EDGJLOPC	Sample JCL for running the IBM Tivoli Workload Scheduler for z/OS batch loader utility to define DFSMSrmm as an application to IBM Tivoli Workload Scheduler for z/OS	SAMPLIB
EDGJMFAL	Sample JCL for allocating the control data set	SAMPLIB
EDGJMOVE	Sample Tivoli job for creating movement reports	SAMPLIB
EDGJNLAL	Sample JCL for allocating the journal	SAMPLIB
EDGJNSCR	Create a report of volumes recently returned to scratch status	SAMPLIB
EDGJRACK	Create a report based on rack number prefixes	SAMPLIB
EDGJRECL	Create a report containing information about lost volumes	SAMPLIB
EDGJRECV	Build RMM subcommands to add volumes to DFSMSrmm	SAMPLIB
EDGJROWN	Create a report about owners sorted by name and department number	SAMPLIB
EDGJRPT	Sample JCL to create reports using the extended report extract file	SAMPLIB
EDGJRVOL	Create a report about volumes; by volume serial number, by rack number, by security level, by owner, and by expiration date	SAMPLIB
EDGJSCRL	Sample Tivoli job for creating scratch listings	SAMPLIB
EDGJSMF	Create a report of SMF records	SAMPLIB
EDGJSMFP	Create a list of types of SMF record found	SAMPLIB
EDGJSTM0	Check REXX execs for use of removed .0 stem variables.	SAMPLIB
EDGJUTIL	Sample JCL for initializing the control data set	SAMPLIB
EDGJVFY	Sample Tivoli job for verifying control data set contents	SAMPLIB
EDGJVLT	Create a report about volumes currently in storage locations sorted by volume serial number	SAMPLIB
EDGJVLTM	Create a report about volumes moving to storage locations	SAMPLIB
EDGJVME	Sample JCL for creating reports for VM tape volumes	SAMPLIB
EDGJVOL	Create a report about volumes sorted by volume serial number	SAMPLIB

Table 80. SAMPLIB and SMPSTS Members (continued)

Member Name	Shows You How To	Supplied In
EDGJVRSV	Sample Tivoli job for running vital record processing trial run	SAMPLIB
EDGJWHKP	Sample Tivoli job for running weekly inventory management	SAMPLIB
EDGLABEL	Started procedure for initializing and erasing tapes	SAMPLIB
EDGMEDOP	Sample MEDINF PARMLIB commands	SAMPLIB
EDGMEDST	Sample MEDINF PARMLIB commands	SAMPLIB
EDGPACTA	Sample Tivoli procedure for allocating ACTIVITY report files	SAMPLIB
EDGPACTC	Sample Tivoli procedure sort input	SAMPLIB
EDGPACTD	Sample Tivoli procedure sort input	SAMPLIB
EDGPACTI	Sample Tivoli procedure ICETOOL control statements	SAMPLIB
EDGPACTM	Sample Tivoli procedure sort input	SAMPLIB
EDGPACTP	Sample Tivoli procedure reporting on ACTIVITY file	SAMPLIB
EDGPACTT	Sample Tivoli procedure sort input	SAMPLIB
EDGPACTV	Sample Tivoli procedure sort input	SAMPLIB
EDGPBKUP	Sample Tivoli procedure for control data set backup	SAMPLIB
EDGPCMOV	Sample Tivoli procedure for global volume move confirmation	SAMPLIB
EDGPEJC	Sample Tivoli procedure for ejecting volumes	SAMPLIB
EDGPEXP	Sample Tivoli procedure for running expiration processing	SAMPLIB
EDGPHKP	Sample Tivoli procedure for running inventory management	SAMPLIB
EDGPHKPA	Sample Tivoli procedure for allocating inventory management data sets	SAMPLIB
EDGPINER	Sample Tivoli procedure for labeling and erasing tapes	SAMPLIB
EDGPMOVE	Sample Tivoli procedure for creating movement reports	SAMPLIB
EDGPMSGGA	Sample Tivoli procedure for allocating the next generation of the MESSAGE file	SAMPLIB
EDGPMSGC	Sample Tivoli procedure for copying the MESSAGE file and creating the next generation of the MESSAGE file	SAMPLIB
EDGPRPTA	Sample Tivoli procedure for allocating the next generation of the report extract file	SAMPLIB
EDGPRPTX	Sample Tivoli procedure for creating the report extract file	SAMPLIB
EDGPSCRL	Sample Tivoli procedure for creating the scratch list report	SAMPLIB
EDGPVFY	Sample Tivoli procedure for verifying the contents of the control data set	SAMPLIB

Table 80. SAMPLIB and SMPSTS Members (continued)

Member Name	Shows You How To	Supplied In
EDGPVRSR	Sample Tivoli procedure for allocating the next generation of the REPORT file and the ACTIVITY file	SAMPLIB
EDGRHKPA	Sample Tivoli exec for defining GDG bases	SAMPLIB
EDGRRPTE	REXX Exec to create reports using the extended report extract file	EDGEXE1
EDGRVCLN	REXX Exec to report and update existing vital record specifications	EDGEXE1
EDGSETT	Sample Tivoli procedure for event trigger tracking of low-on-scratch volume condition	SAMPLIB
EDGUX100	Use the installation exit EDG_EXIT100	SAMPLIB
EDGUX200	Use the installation exit EDG_EXIT200	SAMPLIB
EDGUX300	Use the installation exit EDG_EXIT300	SAMPLIB
EDGXMP1	REXX EXEC to list all volumes in a multivolume set	SAMPLIB
EDGXMP2	REXX EXEC to list all data set information for a given volume	SAMPLIB
EDGXMP3	REXX EXEC to show how the EDGRLCL exec can be coded to handle the 'U' line command.	SAMPLIB
EDGXPROC	Replenish scratch volumes in a automated tape library	SAMPLIB
IGXMSGEX	Use programming interface to EDGMSGEX	SMPSTS

Appendix C. Evaluating removable media management needs

Use the list of questions to assess your current tape management practices and anticipate future requirements. You need your answers to these questions later, when you assess direct access storage device (DASD) needs for the control data set, journal, and report extract data set.

If you plan to change anything about the removable media library, such as increasing the number of volumes, consider the changes shown in [Table 81 on page 539](#) when identifying your DASD needs.

Table 81. Evaluating Removable Media Management Needs

Task	Subtask
Determine the number of resources you have in your removable media library.	How many volumes do you have in your removable media library? A <i>volume</i> is any type of removable media, such as a tape cartridge or an optical disk. Add an average of five volumes in your count for each software product in your installation.
	How many shelf locations or slots do you maintain in your removable media library and in your storage locations? A <i>shelf location</i> is a single space on a shelf where you store a volume. Count all shelves in the library and in your storage locations. For DFSMSrmm subcommands and the ISPF dialog, shelf locations in the removable media library are called <i>rack numbers</i> . Shelf locations in storage locations are called <i>bin numbers</i> .
	How many data sets do you have on removable media? Count any data sets on your removable media.
	How many different individuals or groups use removable media? DFSMSrmm can keep track of owners and of removable media in the DFSMSrmm control data set.
Determine the number of requests submitted to your removable media library.	How many scratch tape mounts are performed daily? • A <i>scratch</i> tape mount is a non-specific tape mount as, for example, when someone requests a blank tape.
	How many non-scratch tape mounts are performed daily? A <i>non-scratch</i> tape mount is a specific tape mount as, for example, when someone requests a tape he or she owns or a software product tape.

Table 81. Evaluating Removable Media Management Needs (continued)

Task	Subtask
Determine the types of activities taking place in your media library.	What activities are performed to support disaster recovery and vital records management? How many volumes enter and leave your removable media library daily? This includes volumes moving to storage locations for disaster recovery and vital records, as well, as foreign tapes entering your library. How many volumes are returned to scratch daily? This number can be used to calculate the space required for the journal. How many volumes expire daily? This number can be used to calculate the space required for the journal. How many logical volumes are imported and exported daily? This number can be used to calculate the space required for the journal.
Determine the number of information changes that might be made to DFSMSrmm information.	This number can be used to calculate the space required for the journal. Changes include information about data sets, owners, software products, or volumes made by using the DFSMSrmm TSO subcommands or DFSMSrmm ISPF dialog.

Appendix D. Problem Determination Aid log data set size work sheet for long-term trace history

Use the work sheet to calculate the size of your PDA log data set (long term).

1. Fill in the blanks with values for your installation.

-----	=	?UID	- The high-level qualifier you want to use for the PDA log data sets.
-----	=	?HOSTID	- The identifier for the processing unit at your site.
-----	=	?TRACEUNIT	- The unit identifier for the device on which you want to allocate the PDA log data sets.
-----	=	?TRACEVOL	- The serial number for the volume on which you want to put your PDA log data sets.

2. Allocate the minimum recommended storage for PDA log data sets: 20 cylinders.

Substitute the values you have provided in step 1 of this work sheet, and run the JCL job shown in [“Allocating the Problem Determination Aid \(PDA\) log data sets” on page 463](#) to allocate and catalog the PDA log data sets.

If you allocated these data sets as SMS-managed data sets, they must be allocated on a specific volume and they must be associated with a storage class having the GUARANTEED SPACE attribute.

3. Allocate a generation data group (GDG) in which you can archive your site's trace history data.

The example defines the generation data group (GDG) name for the archived problem determination output data set. Substitute the applicable values you provided in step 1 of this work sheet, and run the JCL job shown in [“Maintaining a history of Problem Determination Aid \(PDA\) log data” on page 464](#) to create a generation data group.

4. Develop a procedure to automatically copy your PDA log data sets to tape.

The example shows you how to copy the inactive trace data set to tape as a generation data set (GDS). Substitute the applicable values you have provided in step 1 of this work sheet, and run the JCL job shown in [“Maintaining a history of Problem Determination Aid \(PDA\) log data” on page 464](#) to automatically copy your PDA log data sets to tape.

Appendix E. Problem Determination Aid log data set size work sheet for short-term trace history

Use the work sheet to calculate the size of your PDA log data set (short term).

1. Fill in the blanks with values for your installation.

-----	=	?tracehours	-	The number of hours of trace history you want to retain.
-----	=	?UID	-	The high-level qualifier you want to use for the PDA log data sets.
-----	=	?HOSTID	-	The identifier for the processing unit at your site.
-----	=	?TRACEUNIT	-	The unit identifier for the device on which you want to allocate the PDA log data sets.
-----	=	?TRACEVOL	-	The serial number for the volume on which you want to put your PDA log data sets.

2. Allocate the minimum recommended storage for PDA log data sets, which is 20 cylinders.

Substitute the values you used in step 1 of this work sheet, and run the JCL job shown in [Figure 213 on page 543](#) to allocate and catalog the PDA log data sets.

```
//ALLOPDO JOB MSGLEVEL=1, TYPRUN=HOLD
//STEP1 EXEC PGM=IEFBR14
//DD1 DD DSN=?UID..?HOSTID..RMMPDOX, DISP=(,CATLG),
// UNIT=?TRACEUNIT., VOL=SER=?TRACEVOL., SPACE=(CYL,(20))
//DD2 DD DSN=?UID..?HOSTID..RMMPDOY, DISP=(,CATLG),
// UNIT=?TRACEUNIT., VOL=SER=?TRACEVOL., SPACE=(CYL,(20))
```

Figure 213. JCL for allocating and cataloging PDA log data sets

If you have allocated these data sets as SMS-managed, they must be allocated on a specific volume and they must be associated with a storage class having the GUARANTEED SPACE attribute.

3. Measure the cylinders per hour trace history generation rate at your site.

After one hour of processing (during a time of high DFSMSrmm activity), measure the amount of storage used to record that hour's trace activity. Issue the MODIFY command to swap the EDGPDOX and EDGPDOY data sets. After you have swapped these data sets, the EDGPDOY data set will be ready to measure and the EDGPDOX data set will be ready to receive additional trace data.

```
F DFRMM,PDALOG=SWAP
```

Use the information gathered in this step to calculate the cylinders per hour.

Cylinders/hr = cylinders per hour of trace history

4. Calculate the total amount of cylinders required for your site's trace history data.

((tracehours = -----) x (cylinders/hr = -----)) = -----
Total = total number of cylinders of trace data

5. Divide in half the total cylinders required for your short-term trace history interval. If the result is a fraction, round up to the next whole number.

(Total = -----)

2 = -----

This step provides the total number of cylinders to allocate for each data set.

Appendix F. Accessibility

Accessible publications for this product are offered through [IBM Documentation for z/OS \(www.ibm.com/docs/en/zos\)](http://www.ibm.com/docs/en/zos).

If you experience difficulty with the accessibility of any z/OS documentation see [How to Send Feedback to IBM](#) to leave documentation feedback.

Notices

This information was developed for products and services that are offered in the USA or elsewhere.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:

*IBM Director of Licensing
IBM Corporation
North Castle Drive, MD-NC119
Armonk, NY 10504-1785
United States of America*

For license inquiries regarding double-byte character set (DBCS) information, contact the IBM Intellectual Property Department in your country or send inquiries, in writing, to:

*Intellectual Property Licensing
Legal and Intellectual Property Law
IBM Japan Ltd.
19-21, Nihonbashi-Hakozakicho, Chuo-ku
Tokyo 103-8510, Japan*

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

This information could include missing, incorrect, or broken hyperlinks. Hyperlinks are maintained in only the HTML plug-in output for IBM Documentation. Use of hyperlinks in other output formats of this information is at your own risk.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact:

*IBM Corporation
Site Counsel
2455 South Road*

Poughkeepsie, NY 12601-5400
USA

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by IBM under terms of the IBM Customer Agreement, IBM International Program License Agreement or any equivalent agreement between us.

Any performance data contained herein was determined in a controlled environment. Therefore, the results obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

All statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

Terms and conditions for product documentation

Permissions for the use of these publications are granted subject to the following terms and conditions.

Applicability

These terms and conditions are in addition to any terms of use for the IBM website.

Personal use

You may reproduce these publications for your personal, noncommercial use provided that all proprietary notices are preserved. You may not distribute, display or make derivative work of these publications, or any portion thereof, without the express consent of IBM.

Commercial use

You may reproduce, distribute and display these publications solely within your enterprise provided that all proprietary notices are preserved. You may not make derivative works of these publications, or

reproduce, distribute or display these publications or any portion thereof outside your enterprise, without the express consent of IBM.

Rights

Except as expressly granted in this permission, no other permissions, licenses or rights are granted, either express or implied, to the publications or any information, data, software or other intellectual property contained therein.

IBM reserves the right to withdraw the permissions granted herein whenever, in its discretion, the use of the publications is detrimental to its interest or, as determined by IBM, the above instructions are not being properly followed.

You may not download, export or re-export this information except in full compliance with all applicable laws and regulations, including all United States export laws and regulations.

IBM MAKES NO GUARANTEE ABOUT THE CONTENT OF THESE PUBLICATIONS. THE PUBLICATIONS ARE PROVIDED "AS-IS" AND WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, AND FITNESS FOR A PARTICULAR PURPOSE.

IBM Online Privacy Statement

IBM Software products, including software as a service solutions, ("Software Offerings") may use cookies or other technologies to collect product usage information, to help improve the end user experience, to tailor interactions with the end user, or for other purposes. In many cases no personally identifiable information is collected by the Software Offerings. Some of our Software Offerings can help enable you to collect personally identifiable information. If this Software Offering uses cookies to collect personally identifiable information, specific information about this offering's use of cookies is set forth below.

Depending upon the configurations deployed, this Software Offering may use session cookies that collect each user's name, email address, phone number, or other personally identifiable information for purposes of enhanced user usability and single sign-on configuration. These cookies can be disabled, but disabling them will also eliminate the functionality they enable.

If the configurations deployed for this Software Offering provide you as customer the ability to collect personally identifiable information from end users via cookies and other technologies, you should seek your own legal advice about any laws applicable to such data collection, including any requirements for notice and consent.

For more information about the use of various technologies, including cookies, for these purposes, see IBM's Privacy Policy at ibm.com/privacy and IBM's Online Privacy Statement at ibm.com/privacy/details in the section entitled "Cookies, Web Beacons and Other Technologies," and the "IBM Software Products and Software-as-a-Service Privacy Statement" at ibm.com/software/info/product-privacy.

Policy for unsupported hardware

Various z/OS elements, such as DFSMSdfp, JES2, and MVS, contain code that supports specific hardware servers or devices. In some cases, this device-related element support remains in the product even after the hardware devices pass their announced End of Service date. z/OS may continue to service element code; however, it will not provide service related to unsupported hardware devices. Software problems related to these devices will not be accepted for service, and current service activity will cease if a problem is determined to be associated with out-of-support devices. In such cases, fixes will not be issued.

Minimum supported hardware

The minimum supported hardware for z/OS releases identified in z/OS announcements can subsequently change when service for particular servers or devices is withdrawn. Likewise, the levels of other software products supported on a particular release of z/OS are subject to the service support lifecycle of those

products. Therefore, z/OS and its product publications (for example, panels, samples, messages, and product documentation) can include references to hardware and software that is no longer supported.

- For information about software support lifecycle, see: [IBM Lifecycle Support for z/OS \(www.ibm.com/software/support/systemsz/lifecycle\)](http://www.ibm.com/software/support/systemsz/lifecycle)
- For information about currently-supported IBM hardware, contact your IBM representative.

Programming interface information

This publication documents intended Programming Interfaces that allow the customer to write programs to obtain the services of DFSMSrmm.

Trademarks

IBM, the IBM logo, and [ibm.com](http://www.ibm.com) are trademarks or registered trademarks of International Business Machines Corporation in the United States, other countries, or both. If these and other IBM trademarked terms are marked on their first occurrence in this information with a trademark symbol (® or ™), these symbols indicate U.S. registered or common law trademarks owned by IBM at the time this information was published. Such trademarks might also be registered or common law trademarks in other countries. A current list of IBM trademarks is available on the Web at [Copyright and Trademark information \(www.ibm.com/legal/copytrade.shtml\)](http://www.ibm.com/legal/copytrade.shtml).

Java and all Java-based trademarks and logos are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.

Linux is a trademark of Linus Torvalds in the United States, other countries, or both.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Windows is a trademark of Microsoft Corporation in the United States, other countries, or both.

Index

A

ABARS (aggregate backup and recovery support)
 defining ABARS to RACF [47](#)
ABEND
 retention of data sets closed by abend processing [6](#)
ABEND vital record specification [355](#)
access method services REPRO command [386](#)
accessibility
 contact IBM [545](#)
accessibility code [297](#)
ACCODE processing [281](#)
accompany tapes [321](#)
ACCOUNTING, EDGRMMxx operand [168](#)
acero [278](#)
ACS processing [89](#)
active requests, number of [63](#)
ACTIVITY file
 description [352](#)
 printing [352](#)
 viewing [352](#)
adding
 local dialog extensions [450](#)
 volumes to DFSMSrmm [64](#)
ADDOWNER subcommand [63](#)
ADDVOLUME subcommand [106](#)
ADDVRS subcommand
 planning [66](#), [315](#)
AGE operand
 MEDINF REPLACE command [152](#)
allocating data sets
 backup copies [333](#)
 control data set [51](#)
 extract data set [335](#), [336](#)
 inventory management [333](#)
 journal [54](#)
alternate tape [318](#)
American date format [171](#), [340](#), [383](#), [389](#)
AMS LIBRARY command [129](#), [130](#)
ANYUSE, EDGRMMxx operand [203](#)
application programmer
 tasks [21](#)
ARCTVEXT programming interface
 managing DFSMSHsm tapes [249](#), [251](#)
 planning [249](#), [267](#)
 updating during implementation [24](#)
assigning
 bin numbers [9](#)
 bin numbers using DSTORE [357](#)
 expiration dates with EDGUX100 [289](#)
 storage locations [357](#)
assigning a storage group [82](#), [89](#)
assigning a storage group [213](#)
assistive technologies [545](#)
associating a pool with a system [79](#)
audit tape library using a list of barcode scanned volumes
[534](#)

authorization
 administrator functions [228](#)
 for EDGINERS utility [224](#)
 for users [55](#)
 ignoring duplicate or undefined volumes [279](#)
 initialize and erase functions [224](#)
 librarian functions [229](#)
 operator functions [230](#)
 performing inventory management [230](#)
 resources [219](#), [225](#)
 system programmer functions [229](#)
 user functions [228](#)
 users [215](#), [241](#)
authorization to DFSMSrmm resources [312](#)
authorizing users
 for use of DFSMSrmm subcommands [26](#)
 for use of DFSMSrmm utilities [27](#)
 users by defining resources to RACF [16](#)
automated labeling by DFSMSdtp [439](#)
automated tape library
 cartridge entry processing [100](#)
 defining volumes for [100](#)
 description [2](#)
 ejecting volumes from [101](#)
 moving volumes to [105](#)
 replenishing scratch volumes [458](#)
 reserving shelf space for ejected volumes [63](#)
 scratch pool restrictions [75](#)
 specifying a name [100](#)
automated WTO response
 updating AUTORxx [30](#)
automatic message handling [211](#)
automatic recovery [395](#)
Automatic Volume Recognition (AVR) [441](#)
automating control data set backup [370](#)
automating volume mounts [441](#)
AUTORRM member [30](#)
AUTORRP member [30](#)
AUTORxx
 customizing [30](#)
AUTORxx member [30](#)

B

backing up
 DFSMSrmm control data set [377](#)
 journal [387](#)
 using EDGBKUP with access method services REPRO
 command [386](#)
 using EDGBKUP with DFSMSdss DUMP [386](#)
 using EDGHSKP with access method services REPRO
 command [371](#)
 using EDGHSKP with DFSMSdss DUMP [370](#)
 when the DFSMSrmm subsystem is active [370](#)
 when the DFSMSrmm subsystem is inactive [370](#)
backup function
 control data set [387](#)

- backup function (*continued*)
 - DFSMSrmm control data set [370](#)
 - journal [373](#)
- BACKUP parameter [337](#), [372](#), [374](#)
- backup procedure
 - automating [374](#)
 - sample procedure [459](#)
 - specifying the procedure name [168](#)
- BACKUPPROC, EDGRMMxx operand [168](#)
- Basic Tape Library Support (BTLS)
 - defining scratch pools [129](#)
 - description [129](#)
 - inventory management considerations [130](#)
 - scratch management [131](#)
 - updating the catalog [131](#)
 - using EDGINERS with [130](#)
- batch processing [428](#)
- bin number
 - assignment using DSTORE [357](#)
 - estimating the number of [539](#)
- BLP, EDGRMMxx operand [169](#)
- BUFFER.CONTROL resource symbolic name [29](#)
- building
 - ADDVOLUME subcommands from a list of barcode scanned volumes [534](#)
 - RMM CHANGEVOLUME subcommands for volumes in storage locations [534](#)
 - RMM subcommands to add volumes to DFSMSrmm [535](#)
- built-in storage locations
 - description [3](#), [133](#)
 - priority of moves to [3](#)
 - shelf-management [133](#)
 - switching to installation defined locations [140](#)
- bypass label processing
 - defined in installation options [4](#)
 - DFSMSrmm control of [169](#)
 - processing support [14](#)
- bypassing tape label processing [232](#)

C

- calculating space for
 - control data set [49](#)
 - journal [53](#)
- cartridge eject parameter list
 - processing for [261](#)
- cartridge entry processing for logical volume cartridges [109](#)
- catalog control [351](#)
- catalog processing
 - catalog support [193](#)
 - retaining data sets while cataloged [355](#)
 - return to scratch [359](#)
- catalog retention period
 - displayed in vital records retention report [351](#)
 - setting [169](#)
- catalog sharing [332](#)
- catalog status tracking
 - identifying system IDs [170](#)
- catalog synchronization setup with CATSYSID, EDGRMMxx operand [170](#)
- cataloging the DFSMSrmm control data set in a shared user catalog [50](#)
- catalogs, synchronizing with the DFSMSrmm control data set [339](#), [409](#)
- CATLGDAY, EDGRMMxx operand [183](#)
- CATRETPD, EDGRMMxx operand [169](#)
- CATSYSID, EDGRMMxx operand [170](#)
- CBRSPPIM — sample JCL for import list volume private request [115](#), [120](#)
- CBRSPPIM-sample JCL for import list volume scratch request [115](#)
- CBRSPPXP — sample JCL for export list volume scratch request [119](#)
- CBRSPPXP-sample JCL for export list volume scratch request [114](#)
- CBRSPSIM — sample JCL for import list volume scratch request [115](#), [120](#)
- CBRSPSIM-sample JCL for import list volume scratch request [115](#)
- CBRSPSXP — sample JCL for export list volume scratch request [119](#)
- CBRSPSXP-sample JCL for export list volume scratch request [114](#)
- CBRUXCUA exit
 - description [533](#)
 - reason codes [253](#)
 - return codes [253](#)
 - volume status change [125](#)
 - where to find source code [24](#)
- CBRUXEJC exit
 - description [533](#)
 - return codes [253](#)
 - where to find source code [24](#)
- CBRUXENT and CBRUXEJC
 - set up parallel processing for [266](#)
- CBRUXENT exit
 - description [533](#)
 - entering volumes [100](#)
 - modifying [127](#)
 - return codes [253](#)
 - where to find source code [24](#)
- CBRUXVNL exit
 - description [533](#)
 - processing support [102](#)
 - retrieving information about a volume [261](#)
 - retrieving information about VTS volumes [262](#)
 - return codes [253](#)
 - where to find source code [24](#)
- CDSID, EDGRMMxx operand [168](#)
- chaining vital record specifications [6](#)
- CHANGEVOLUME subcommand [218](#)
- changing
 - ADD product volume dialog panel defaults [451](#)
 - DFSMSrmm dialog panel navigation [449](#)
 - pool definition [79](#)
 - running modes [67](#), [180](#)
- changing owner information [256](#)
- changing storage location managementtype [139](#)
- changing storage location medianame [139](#)
- changing storage locations [139](#)
- character set
 - chart xxxiii
 - use in statement xxxiii
- checking
 - DATASET class resource [244](#)
 - TAPEVOL class resource [245](#)
- checking DFSMSrmm status [62](#)
- checkpoint data set creation [17](#)

- clearing the journal [370](#), [377](#)
- client systems
 - authorization [71](#)
 - firewall [71](#)
 - implementing [72](#)
 - inventory management considerations [332](#), [378](#)
 - setting up [71](#)
 - using [73](#)
 - utility considerations [332](#), [378](#)
- CLIST operand [16](#)
- combining retention types [354](#)
- common time support
 - enabling support [411](#)
- completing the export processing [114](#)
- concatenated parmlib support [45](#), [47](#)
- confirming
 - global release actions [369](#)
 - global volume movement [369](#)
 - volume movement in the system-managed tape environment [422](#)
 - volume movement to system-managed libraries [105](#)
 - volume release actions [360](#)
- considerations for multiple z/OS images or RMMplexes [23](#)
- contact
 - z/OS [545](#)
- control data set
 - allocating the index and data components [51](#)
 - audit information [232](#)
 - backing up [8](#), [386](#)
 - calculating space [49](#)
 - cataloging in a shared user catalog [50](#)
 - control record [409](#)
 - controlling [390](#)
 - controlling access to [50](#)
 - creating during installation
 - allocating space [51](#)
 - backing up [52](#)
 - initializing [52](#)
 - creating the control record [409](#)
 - decrease size [395](#)
 - defining [49](#)
 - displaying information [421](#)
 - EDGJMFAL SAMPLIB member [51](#)
 - EDGJUTIL SAMPLIB member [52](#)
 - ensuring a consistent copy of [372](#)
 - ensuring access in shared environment [50](#)
 - forward recovering [391](#)
 - forward recovery of
 - planning for [52](#)
 - fuzzy copy of [372](#)
 - global resource serialization [50](#)
 - GUARANTEED SPACE attribute, using [50](#)
 - improving performance [50](#)
 - increase size [395](#)
 - index and data components [51](#)
 - JCL for backing up [372](#)
 - mirrored copy of
 - planning for [52](#)
 - monitoring [394](#)
 - moving to a different device [397](#)
 - naming [171](#)
 - placement of [50](#)
 - recovering [8](#), [391](#)
 - reorganizing [392](#), [393](#)
- control data set (*continued*)
 - restoring [390–392](#)
 - scheduling back up [327](#)
 - sharing [421](#)
 - size increase [50](#)
 - specifying ID [168](#)
 - steps for moving [397](#), [398](#)
 - synchronizing with system catalogs [339](#), [409](#)
 - update failures [395](#)
 - updating the control record [409](#)
 - updating volume status [125](#)
 - validating against TCDB [413](#)
 - verifying the contents [411](#), [413](#)
- control data set control record
 - creating [409](#)
 - updating during recovery [384](#)
- controlling
 - controlling access to the control data set [50](#)
 - data set recording [285](#)
 - message case [179](#)
- controlling tape erasure [431](#)
- controlling tape initialization [431](#)
- controlling the control data set [390](#)
- controlling volume movement [179](#)
- controlling volume retention
 - EXPDT retention method [186](#)
 - system-wide default [183](#)
 - VRSEL retention method [183](#)
- conversion
 - changing duplicate volume serial numbers [96](#)
 - detecting errors during [401](#)
 - fixing errors resulting from [401](#)
 - ignoring duplicate volume serial numbers [275](#)
- converting
 - DFSMSrmm systems [72](#)
 - duplicate volume serial numbers [96](#)
- converting CLIST output [114](#), [115](#), [119](#), [120](#)
- copy export [116](#)
- copy services
 - DFSMSrmm support for [52](#)
- correcting rack or bin number counts [409](#)
- creating
 - a monthly archive from weekly audit reports [534](#)
 - a report about owners sorted by name and department number [535](#)
 - a report about volumes [535](#)
 - a report based on rack number prefixes [535](#)
 - a report containing information about lost volumes [535](#)
 - a report of data sets sorted by data set name [534](#)
 - a report of volumes recently returned to scratch status [535](#)
 - a report using the extended report extract file [535](#)
 - a weekly archive from daily audit reports [534](#)
 - an import list from CLIST output [535](#)
 - extract data set [335](#)
 - reports [15](#)
 - table for controlling data set recording [286](#)
 - volume label [442](#)
- creating checkpoint data sets [17](#)
- creating non-checkpoint data sets [17](#)
- customizing
 - ADD Product Volume dialog panel defaults [451](#)
 - DFSMSrmm messages [451](#)
 - DSSOPT DD Statement [384](#)

customizing (*continued*)

- EDGP@LCL, dummy panel [450](#)
- EDGXPROC procedure [458](#)
- installation exits [249](#), [267](#)
- ISPF dialog [449](#)
- local dialog extensions [451](#)
- notification messages and notes [452](#)
- report trailer lines [452](#)
- RMMISPF exec [449](#)
- user exits [249](#)

customizing notify messages [455](#)

D

data set

- allocating for inventory management [333](#)
- managing [14](#)
- order of matching during vital record processing [355](#)
- protection [239](#)
- recording information [285](#)
- retention by job name [194](#)
- types of retention [5](#)
- uncataloging during expiration processing [361](#)

data set profile [239](#)

data set recording

- table for controlling data set recording [286](#)
- using EDGUX100 [285](#)

data set retention period

setting [183](#)

data set vital record specification

- example [6](#)
- for managing special dates [279](#)

data sets

- excluding from vital record processing [352](#)
- excluding from VRSEL processing [274](#)

date format

- extract data set [336](#)
- setting for use in reports and messages [171](#)

DATEFORM

- in EDGHSKP [336](#), [340](#), [383](#)
- in EDGUPDT [389](#)
- operand in EDGRMMxx [171](#)

DEFAULT command

operands [487](#)

default retention period, specifying [187](#)

defining

- a volume in a system-managed tape library [100](#)
- ABARS user ID to RACF [47](#)
- DFSMSshm user ID to RACF [47](#), [311](#)
- DFSMSrmm subsystem name to z/OS [25](#)
- home location [5](#)
- MCS console [271](#)
- mount messages [154](#)
- original expiration date [211](#)
- pools in parmlib member with the VLPOOL command [209](#)
- RACF profiles [215](#)
- security classes [206](#)
- SMF audit records [189](#)
- SMF records generated by DFSMSrmm [27](#)
- SMF security records [189](#)
- TAPEVOL class resource [245](#)
- volumes to DFSMSrmm [64](#)

defining storage locations [134](#)

DELETED

retention of data set closed by abend processing [6](#)

DELETED vital record specification [355](#)

DELETEVOLUME subcommand [218](#)

DELETEVRS subcommand [218](#)

deleting storage locations [139](#)

delimiters [xxxiii](#)

designing

- rack pool [80](#)
- scratch pool [80](#)

determining RMM versus NORMM [277](#)

device types supported [2](#), [3](#)

DFSMSdftp automated labeling [439](#)

DFSMSdss

- changing options [384](#)
- clearing the journal during back up processing [373](#)
- commands used by DFSMSrmm [384](#)
- DSSOPT DD statement [333](#), [382](#)
- EDGSPLCS DD statement [333](#)
- inventory management considerations [371](#)
- STGADMIN.ADR.DUMP.CNCURRNT [371](#)
- using with EDGBKUP [386](#)

DFSMSshm

- ADDVRS examples for retaining DFSMSshm tapes [315](#)
- alternate tape [318](#)
- authorization to DFSMSrmm resources [311](#)
- authorization to RACF [311](#)
- defining DFSMSshm to RACF [47](#), [311](#)
- disaster recovery using DFSMSshm alternate tapes [324](#)
- expiration date protection [314](#)
- retaining
 - ABARS accompany tapes [321](#)
 - backup tapes [317](#)
 - control data set backup tapes [322](#)
 - dump tapes [319](#)
 - migration tapes [316](#)
 - TAPECOPY tapes [318](#)
 - tapes [314](#)
 - tapes written by ABARS [321](#), [323](#)
- running with DFSMSrmm [311](#), [325](#)
- using EDGDFHSM programming interface [251](#)
- using EDGTVEXT programming interface [249](#)
- validating data set name [18](#)

DFSMSshm tapes

- managing with EDGDFHSM [250](#)
- managing with VRSEL retention method [315](#)

DFSMSrmm

- adding local dialog extensions [450](#)
- application programmer tasks [21](#)
- authorization checking [241](#)
- basic tape library support [129](#)
- changing ADD product volume dialog panel defaults [451](#)
- changing DFSMSrmm dialog panel navigation [449](#)
- changing volume dialog panel defaults [451](#)
- concatenated parmlib support [45](#), [47](#)
- control record [409](#)
- customizing local dialog extensions [451](#)
- customizing messages [451](#)
- defining owner to [63](#)
- description [1](#)
- duplicate volume serial number support [95](#)
- EDG_EXIT100 installation exit [267](#)
- EDG_EXIT200 installation exit [300](#)
- general user tasks [20](#)

DFSMSrmm (*continued*)

- initializing the subsystem [66](#)
- mapping macros [511](#)
- modes of operation [18](#)
- operator tasks [21](#)
- panel navigation [449](#)
- protecting resources [215](#)
- RACF considerations [239](#)
- refreshing DFSMSrmm installation exits [294](#), [302](#), [307](#)
- removing from the system [244](#)
- running modes [180](#)
- running utilities [7](#)
- security classification [232](#)
- storage administrator tasks [21](#)
- storage group name support [108](#)
- system programmer tasks [21](#)
- system-managed tape libraries support [99](#)
- tape initialization and erase control [431](#)
- tape librarian tasks [20](#)
- tape mount validation rules [17](#)
- undefined volume serial number support [97](#)
- Updating the Workload Management service definition [60](#)
- validating tape volumes [17](#)
- volume rejection rules [18](#)
- wrong label processing [431](#)

DFSMSrmm application programming interface [16](#)

DFSMSrmm catalog processing [366](#)

DFSMSrmm control data set

- synchronizing with user catalogs
- in fully shared catalog environment [367](#)

DFSMSrmm resources

- setting the level of access for [219](#)

DFSMSrmm utility

- EDGBKUP, backing up the control data set [377](#)
- EDGHSKP, inventory management program [327](#)
- EDGINERS, initializing and erasing volumes [233](#), [423](#)
- EDGRESET utility, removing DFSMSrmm from the system [244](#)
- EDGUPDT, updating the active control data set [377](#)
- EDGUTIL, verifying control data set contents [377](#)

DFSORT

- sample EDGJACTP print job [352](#)

diagnosing errors

- using SYSPRINT data set [386](#)
- using the message file [412](#)
- using the TRACE operand [449](#)

dialog customization [449](#)

disabling

- automatic cartridge loader [270](#)
- PDA trace facility [182](#)
- the DFSMSrmm subsystem interface [61](#)

disabling the autoloader [269](#)

DISPDDNAME, EDGRMMxx operand [171](#)

displaying information [421](#)

DISPMSGID, EDGRMMxx operand [171](#), [172](#)

disposition control

- defining the message returned by disposition processing [171](#), [172](#)
- modifying label output using EDGUX100 [284](#)
- naming a disposition control file [171](#)
- setting up [43](#), [467](#)

DISTANT storage location [3](#)

DITTO, using [122](#)

DSNAME, EDGRMMxx operand [171](#)

DSNEXPIRE, EDGRMMxx operand [172](#)

DSSOPT

- changing options [384](#)
- description [333](#), [382](#)

DSTORE

- examples [357](#)
- INSEQUENCE [340](#)
- LOCATION [340](#)
- parameter [340](#)
- REASSIGN [341](#)

duplicate or undefined volumes [279](#)

duplicate volume

- adding into system-managed tape library [97](#)

duplicate volume serial number

- changing duplicate volume serial numbers [96](#)

- defining duplicate volume serial numbers to DFSMSrmm [95](#)

- ignoring [275](#)

- labeling duplicate volume serial numbers [96](#)

- managing duplicate volume serial numbers [95](#)

- using EDG_EXIT100 [267](#)

dynamic shelf-management [357](#)

E

EBCDIC labels [443](#)

EDG_EXIT100

- parameter list [295](#)

- specifying retention method [271](#)

EDG_EXIT200

- parameter list [303](#)

EDG_EXIT200 installation exit

- EDGPL200 [524](#)

- parameter list for [524](#)

EDG_EXIT300

- exit routine processing [305](#)

- parameter list [308](#)

- Setting up routine environment [306](#)

EDG0154I [61](#)

EDG019VM, programming interface [460](#)

EDG2107E [175](#)

EDG2108E [175](#)

EDG3IIP1 [474](#)

EDG3IIP1 SAMPLIB member

- using [474](#)

EDG3LVVR [474](#)

EDG3LVVR SAMPLIB member

- description [533](#)

- using [474](#)

EDG3UX29 [473](#)

EDG3UX29 SAMPLIB member [24](#)

EDG3UX62 [473](#)

EDG3UX62 SAMPLIB member

- description [533](#)

EDG3UX71 [473](#)

EDG3UX71 SAMPLIB member

- description [533](#)

- using [473](#)

EDG3X71, programming interface [264](#)

EDG4026I [424](#)

EDGAPISR SMPSTS member

- description [533](#)

EDGBETT [533](#)

- EDGBKUP
 - backing up the control data set with
 - planning for [52](#)
 - SYSIN file for [382](#)
- EDGBKUP control data set backup program
 - backing up the control data set and journal [387](#)
 - backing up the journal [373](#)
 - controlling the control data set recovery point [390](#)
 - description [533](#)
 - DFSMSrmm control data set back up and restore program [386](#)
 - exec parameters [380](#), [382](#)
 - restoring the control data set [390–392](#)
 - return codes [384](#)
- EDGCLIBQ SAMPLIB member
 - description [533](#)
 - using [458](#)
- EDGDFHSM [250](#)
- EDGDFHSM, programming interface [251](#)
- EDGDFRMM SAMPLIB member
 - description [533](#)
- EDGHCLT [534](#)
- EDGHSKP
 - backing up the control data set with
 - planning for [52](#)
 - EXEC parameters for [337](#)
- EDGHSKP EXPROC utility
 - multitasking [364](#)
- EDGHSKP inventory management utility
 - control data set backup processing [370](#)
 - description [327](#)
 - expiration processing [359](#)
 - extract data set processing [335](#)
 - return codes [374](#)
 - storage location management processing [357](#)
 - vital record processing [345](#)
- EDGHSKP utility
 - SYSIN file [342](#)
- EDGHSKP/EDGBKUP
 - backing up the control data set with
 - using [372](#)
- EDGINERS
 - EXEC parameters for [426](#)
 - using [424](#)
- EDGINERS initializing and erasing volumes utility
 - automatic processing [425](#)
 - creating volume label [442](#)
 - description [423](#)
 - differences and similarities to IEHINITT [8](#)
 - initialize and erase program [423](#)
 - ISO/ANSI label support [423](#)
 - JCL [426](#)
 - label symmetries [441](#)
 - manual processing [425](#)
 - replacing IEHINITT with EDGINERS [424](#)
 - return codes [442](#)
 - running for BTLs [130](#)
 - running when sharing the control data set [422](#)
 - tape volumes with ISO/ANSI labels [441](#)
 - using EDGINERS instead of IEHINITT [233](#)
 - wrong label processing [431](#)
- EDGINERS utility
 - SYSIN commands for [432](#)
- EDGINERS.volser resource symbolic name [29](#)
- EDGIVP1 SAMPLIB member
 - description [533](#)
- EDGIVP2 SAMPLIB member
 - description [533](#)
- EDGIVPPM SAMPLIB member
 - description [533](#)
- EDGJACTP SAMPLIB member
 - description [533](#)
- EDGJAUDM [534](#)
- EDGJAUDW [534](#)
- EDGJBCAV [534](#)
- EDGJBKP1 [534](#)
- EDGJBKP2 [534](#)
- EDGJCMOV [534](#)
- EDGJCOMB [534](#)
- EDGJCVB [534](#)
- EDGJDHKP [534](#)
- EDGJDSN [534](#)
- EDGJEJC [534](#)
- EDGJEXP [534](#)
- EDGJHKPA [535](#)
- EDGJHKPA SAMPLIB member
 - description [533](#)
- EDGJHSKP [535](#)
- EDGJHSKP SAMPLIB member
 - description [533](#)
- EDGJIMPC [535](#)
- EDGJIMPC sample [120](#)
- EDGJINER [535](#)
- EDGJINER SAMPLIB member
 - description [533](#)
- EDGJLOPC [535](#)
- EDGJLOPC SAMPLIB member
 - description [533](#)
- EDGJMFAL [535](#)
- EDGJMFAL SAMPLIB member
 - description [533](#)
 - using during implementation [51](#)
- EDGJMOVE [535](#)
- EDGJNLAL [535](#)
- EDGJNLAL SAMPLIB member
 - description [533](#)
 - using during implementation [54](#)
- EDGJNSCR [535](#)
- EDGJRACK [535](#)
- EDGJRECL [535](#)
- EDGJRECV [535](#)
- EDGJROWN [535](#)
- EDGJRVOL [535](#)
- EDGJSCRL [535](#)
- EDGJSMF [535](#)
- EDGJSMFP [535](#)
- EDGJSTM0 [535](#)
- EDGJUTIL [535](#)
- EDGJUTIL SAMPLIB member
 - description [533](#)
 - using during implementation [52](#)
- EDGJVfy [535](#)
- EDGJVLT [535](#)
- EDGJVLTm [535](#)
- EDGJVME [535](#)
- EDGJVOL [535](#)
- EDGJVRSV [536](#)
- EDGJWHKP [536](#)

- EDGLABEL SAMPLIB member
 - defining in ICHRIN03 [46](#)
 - description [533](#)
 - RACF requirement [46](#)
 - using [459](#)
- EDGLCSUP
 - input [252](#)
- EDGLCSUP macro programming interface [511](#)
- EDGLCSUX
 - managing automated tape library volumes [251](#)
 - output [253](#)
 - planning [249](#)
 - return codes [253](#)
- EDGLIBQ SAMPLIB member
 - description [533](#)
 - using [458](#)
- EDGMSGEX, programming interface [263](#)
- EDGP@LCL, dummy panel [450](#)
- EDGPACTA [536](#)
- EDGPACTC [536](#)
- EDGPACTD [536](#)
- EDGPACTI [536](#)
- EDGPACTM [536](#)
- EDGPACTP [536](#)
- EDGPACTT [536](#)
- EDGPACTV [536](#)
- EDGPBKUP [536](#)
- EDGPCMOV [536](#)
- EDGPDOX PDA trace data set [461](#)
- EDGPDOY PDA trace data set [461](#)
- EDGPEJC [536](#)
- EDGPEXP [536](#)
- EDGPHKP [536](#)
- EDGPHKPA [536](#)
- EDGPINER [536](#)
- EDGPL100 macro programming interface [517](#)
- EDGPL300 macro programming interface [526](#)
- EDGPMOVE [536](#)
- EDGPMSGA [536](#)
- EDGPMSGC [536](#)
- EDGPRPTA [536](#)
- EDGPRPTX [536](#)
- EDGPSCRL [536](#)
- EDGPVIFY [536](#)
- EDGPVRSA [537](#)
- EDGRESET utility
 - invoking by the DFSMSrmm procedure [231](#)
 - removing DFSMSrmm from the system [224](#), [244](#)
- EDGRHKPA [537](#)
- EDGRMMxx, DFSMSrmm parmlib member
 - creating parmlib member definitions [47](#)
 - defining during implementation [47](#)
 - example [143](#)
 - MNTMSG command [154](#)
 - OPTION command [161](#)
 - REJECT command [202](#), [238](#)
 - SECCLS command [206](#)
 - specifying options for [143](#)
 - VLPOOL command [209](#)
- EDGRVCLN [537](#)
- EDGRVCLN exec
 - description [533](#)
- EDGSETT [537](#)
- EDGSLAB macro programming interface [529](#)
- EDGSPLCS [418](#)
- EDGSSSI [25](#)
- EDGTVEXT
 - description [249](#)
- EDGTVEXT, programming interface [249](#)
- EDGUPDT
 - exec parameters [388](#)
 - SYSIN file for [389](#)
- EDGUTIL control data set contents verification program
 - creating the control record [409](#)
 - DFSMSrmm control data set create and verify program [401](#)
 - enabling stacked volume support [409](#)
 - exec parameters [402](#)
 - JCL [401](#)
 - mending the control data set [414](#)
 - return codes [418](#), [421](#)
 - verify system-managed volume information [107](#)
 - verifying control data set contents [413](#)
- EDGUTIL utility
 - multitasking [406](#)
- EDGUX100 SAMPLIB member
 - assigning expiration dates [289](#)
 - bypassing tape label processing [232](#)
 - data set recording [285](#)
 - description [533](#)
 - ignoring duplicate volumes [275](#)
 - installing [293](#)
 - managing duplicate volumes [275](#)
 - managing scratch pools [269](#)
 - modifying disposition control processing label output [284](#)
 - refreshing [294](#)
 - return codes [300](#)
 - tailoring [278](#), [280](#)
 - using [85](#), [91](#), [267](#)
- EDGUX200 SAMPLIB member
 - description [533](#)
 - installing [301](#)
 - refreshing [302](#)
 - return codes [305](#)
 - using [300](#)
- EDGUX300 SAMPLIB member
 - installing [306](#)
 - refreshing [307](#)
 - return codes [309](#)
 - using [305](#)
- EDGXMP1 SAMPLIB member
 - description [533](#)
- EDGXMP2 SAMPLIB member
 - description [533](#)
- EDGXPROC SAMPLIB member
 - defining in parmlib [188](#)
 - RACF requirement [46](#)
 - using EDGXPROC procedure [458](#)
- ejecting volumes from system-managed libraries [101](#), [358](#)
- enablement policy
 - for products or product features [42](#)
- enabling
 - DFSMSrmm and tape recording [26](#)
 - DFSMSrmm subsystem interface [66](#)
 - extended bin support [417](#)
 - ISPF DSLIST Support [58](#)
 - management class expiration attributes [177](#)

- enabling (*continued*)
 - PDA trace facility [182](#)
 - SMS management class attributes [93](#)
 - stacked volume support [409](#)
- erase volume release action [370](#)
- ERASE, EDGRMMxx operand [208](#)
- erasing tape volumes using EDGINERS [423](#)
- error diagnosis [449](#)
- European date format [171](#), [340](#), [383](#), [389](#)
- evaluating removable media management needs [539](#)
- excluding data sets from vital record processing [352](#)
- EXEC parameters
 - for EDGINERS [426](#)
- exec, LIBQ [458](#)
- EXPDTCHECK, EDGRMMxx operand [211](#)
- EXPDTDROP
 - effect on expiration processing [360](#)
- EXPDTDROP, EDGRMMxx operand [172](#)
- expiration date
 - assigning with EDGUX100 [289](#)
 - protecting with VLPOOL EXPDTCHECK(Y) [211](#)
 - setting [187](#)
- expiration processing
 - description [360](#)
 - EXPDT retention method [360](#)
 - EXPDTDROP limit [360](#)
 - retaining DFSMSShsm tapes [314](#)
 - scheduling [327](#)
 - VRSEL retention method [360](#)
- expiration time [12](#)
- exploiting high speed cartridge tape positioning [17](#)
- export processing [113](#)
- EXPROC parameter [337](#), [359](#)
- extended bin support
 - enabling [417](#)
 - managing bin reuse [340](#)
 - REUSEBIN(CONFIRMMOVE) [188](#)
 - REUSEBIN(STARTMOVE) [188](#)
- external data manager considerations [250](#)
- extract data set
 - placement of [336](#)
 - scheduling creation [327](#)

F

- fast replication
 - backing up the control data set with
 - planning for [52](#)
- FEATURENAME(DFSMSRMM)
 - in IFAPRDxx [42](#)
- finding samples [xxix](#)
- FlashCopy
 - backing up the control data set with
 - using [372](#)
- formatting a list of logical volumes from RMM
 - SEARCHVOLUME CLIST output [120](#)
- forward recovery [8](#), [391](#)
- fuzzy backup of control data set [372](#)

G

- GDG, EDGRMMxx operand [173](#)
- general user

- general user (*continued*)
 - access to DFSMSrmm resources [228](#)
 - tasks [20](#)
- generation data groups
 - cycle retention handling [173](#)
- generic volume serial number [7](#)
- global confirmation [369](#)
- Global Mirror
 - backing up the control data set with
 - planning for [52](#)
 - using [372](#)
- global resource serialization
 - control data set [50](#)
- global resource serialization (GRS)
 - reducing global resource serialization traffic [177](#)
- global resource serialization(GRS)
 - converting the RESERVE to a SYSTEMS enqueue [28](#)
 - converting the SYSTEMS enqueue to a local SYSTEM enqueue [27](#)
 - reserve names [28](#)
 - SYSZRMM [28](#)
 - updating GRSRNLxx [27](#)
- GRS (global resource serialization)
 - reducing global resource serialization traffic [177](#)
- GRS(global resource serialization)
 - converting the RESERVE to a SYSTEMS enqueue [28](#)
 - converting the SYSTEMS enqueue to a local SYSTEM enqueue [27](#)
 - reserve names [28](#)
 - SYSZRMM [28](#)
 - updating GRSRNLxx [27](#)
- GRSRNLxx [27](#)
- GUARANTEED SPACE attribute
 - control data set [50](#)
 - journal [54](#)

H

- hierarchy of moves [3](#)
- hierarchy of storage location names [355](#)
- high speed cartridge tape positioning support [17](#)
- home location
 - changing [123](#), [127](#)
 - definition [5](#)
 - updating [102](#)
- host name [71](#)

I

- I/O errors on a volume [12](#)
- IATIIP1 [474](#)
- IATLVVR [474](#)
- IATUX71 [473](#)
- IBM standard and user header or trailer labels (SUL) [13](#)
- IBM standard labels (SL) [13](#)
- IBM Tivoli Tape Optimizer [152](#)
- IBM Tivoli Workload Scheduler for
 - z/OS
 - EDGBETT sample procedure [533](#)
 - EDGJBKP1 sample procedure [534](#)
 - EDGJBKP2 sample procedure [534](#)
 - EDGJCMOV sample procedure [534](#)
 - EDGJDHKP sample procedure [534](#)

IBM Tivoli Workload Scheduler for z/OS (continued)

- EDGJEJC sample procedure [534](#)
- EDGJEXP sample procedure [534](#)
- EDGJMOVE sample procedure [535](#)
- EDGJSCRL sample procedure [535](#)
- EDGJVFY sample procedure [535](#)
- EDGJVRSV sample procedure [536](#)
- EDGJWHKP sample procedure [536](#)
- EDGPACTA sample procedure [536](#)
- EDGPACTC sample procedure [536](#)
- EDGPACTD sample procedure [536](#)
- EDGPACTI sample procedure [536](#)
- EDGPACTM sample procedure [536](#)
- EDGPACTP sample procedure [536](#)
- EDGPACTT sample procedure [536](#)
- EDGPACTV sample procedure [536](#)
- EDGPBKUP sample procedure [536](#)
- EDGPCMOV sample procedure [536](#)
- EDGPEJC sample procedure [536](#)
- EDGPEXP sample procedure [536](#)
- EDGPHKP sample procedure [536](#)
- EDGPHKPA sample procedure [536](#)
- EDGPINER sample procedure [536](#)
- EDGPMOVE sample procedure [536](#)
- EDGPMSGA sample procedure [536](#)
- EDGPMSGC sample procedure [536](#)
- EDGPRPTA sample procedure [536](#)
- EDGPRPTX sample procedure [536](#)
- EDGPSCRL sample procedure [536](#)
- EDGPVfy sample procedure [536](#)
- EDGPVRSa sample procedure [537](#)
- EDGRHKPA sample procedure [537](#)
- EDGSETT sample procedure [537](#)
- IBM-assigned SMF record types
 - switching [196](#)
- ICETOOL, DFSORT utility
 - description [15](#), [16](#)
- ICHRIN03, started procedure table
 - defining ABARS user ID [47](#)
 - defining DFSMSHsm user ID [47](#), [311](#)
 - defining DFSMSrmm user ID [46](#)
- ID, EDGRMMxx operand [155](#)
- IEC507D [85](#), [211](#)
- IEC704A [107](#)
- IEFRDER DD [44](#)
- IEFSSNxx [25](#)
- IEHINITT
 - description [8](#)
 - differences with EDGINERS [424](#)
 - limiting use of [233](#)
 - replacing with EDGINERS [424](#)
- IFAPRDxx
 - FEATURENAME requirement [42](#)
- IGDACERO, mapping macro [281](#)
- IGDACSXT, pre-ACS installation exit [281](#)
- ignoring
 - volumes [275](#)
- IGXMSGEX programming interface
 - displaying DFSMSrmm messages [263](#)
 - planning [249](#), [267](#)
 - updating during implementation [24](#)
- IKJEFTxx member [27](#)
- IKJTSOxx [26](#)
- implementation

implementation (continued)

- adding volumes [64](#)
- assigning a RACF user ID [46](#)
- authorizing users [55](#)
- creating the control data set [48](#)
- defining
 - EDGRMMxx [47](#)
 - owner information [63](#)
 - shelf locations [63](#)
- dynamically adding DFSMSrmm [26](#)
- enabling DFSMSrmm and tape recording [26](#)
- initializing the DFSMSrmm subsystem [66](#)
- installation defined storage locations [134](#)
- installing
 - JES3 USERMOD [24](#)
 - with SMP/E [23](#)
- journal [53](#)
- modifying ISPF [55](#)
- planning vital record specifications [66](#)
- protect mode [67](#)
- restarting z/OS [59](#)
- running the IVP [24](#)
- setting up utilities [68](#)
- START command [60](#)
- starting DFSMSrmm [60](#)
- storage locations as home locations [135](#)
- tailoring EDGRMMxx [47](#)
- tasks [23](#), [68](#)
- updating
 - ARCTVEXT [24](#)
 - AUTORxx [30](#)
 - GRSRNLxx [27](#)
 - IEFSSNxx [25](#)
 - IGXMSGEX [24](#)
 - IKJTSOxx [26](#)
 - operational procedures [66](#)
 - procedure library [43](#)
 - SMFPRMxx [27](#)
 - SYS1.PARMLIB members [25](#)
- implementing
 - volume replacement policies [151](#)
- import processing [115](#), [116](#), [119](#)
- INACTIVE resource symbolic name [29](#)
- initialize volume release action [370](#)
- initializing
 - DFSMSrmm control data set [52](#)
 - DFSMSrmm subsystem [66](#)
 - volumes in system-managed libraries [107](#)
 - volumes with unknown volume serial numbers [431](#)
- initializing and erasing volumes
 - batch processing [428](#)
 - examples [443](#)
 - replacement for IEHINITT [424](#)
 - scheduling [327](#)
 - using EDGINERS [423](#)
 - using LABEL procedure [459](#)
 - volumes in system-managed libraries [107](#)
- input only volume [126](#)
- INSEQUENCE [340](#)
- installation defined storage locations
 - defining location names [144](#)
 - defining with the LOCDEF parmlib command [134](#)
 - description [3](#), [133](#)
 - implementing [134](#), [135](#)

installation defined storage locations (*continued*)

- priority of moves to [3](#)
- segregating shelf locations [133](#)
- setting a destination for volumes [133](#)
- shelf-management [133](#)
- switching built-in storage locations [140](#)

installation exit

- CBRUXCUA [251](#)
- CBRUXEJC [251](#)
- CBRUXENT [251](#)
- CBRUXVNL [251](#)
- displaying DFSMSrmm messages [264](#)
- EDG_EXIT100 [267](#)
- EDGTVEXT [249](#)
- IATUX71 [264](#)
- IFG019VM [460](#)
- IGXMSGEX [263](#)
- processing NL label tapes [460](#)
- running in parallel [264](#)

installation verification program (IVP)

- description of EDGIVPPM, EDGIVP1, EDGIVP2 [533](#)

integrated catalog facility [332](#)

inventory management

- allocating data sets [333](#)
- backing up the DFSMSrmm control data set [370](#)
- creating an extract data set [335](#)
- creating reports [335](#)
- description [8](#)
- EDGHSKP, inventory management program [327](#)
- expiration processing [359](#)
- management reports [327](#)
- message file [327](#), [412](#)
- processing order [337](#)
- report file [348](#)
- return codes for [374](#)
- running [328](#)
- scheduling [327](#)
- storage location management [357](#)
- using VRSCCHANGE EDGRMMxx operand to set up trial run [193](#)
- vital record processing [345](#)
- where to run [130](#), [421](#)

IOS000I [424](#)

IP address [71](#)

IPL date checking [174](#)

IPLDATE, EDGRMMxx operand [174](#)

ISO date format [171](#), [340](#), [383](#), [389](#)

ISO/ANSI accessibility [297](#)

ISO/ANSI and user header or trailer labels (AUL) [13](#)

ISO/ANSI label versions, specifying [434](#)

ISO/ANSI labels [443](#)

ISO/ANSI labels (AL) [13](#)

ISO/ANSI tape labels, EDGINERS example [446](#), [447](#)

ISO/ANSI X3.4–1986 character set [434](#)

ISPF

- adding a selection to an ISPF panel [56](#)
- changing ADD product volume dialog panel defaults [451](#)
- making the ISPF dialog available [57](#)
- modifying during installation [55](#)

ISPF DSLIST Support [58](#)

J

JES2

JES2 (*continued*)

- installing a USERMOD during implementation [24](#)
- setting subsystem name example [25](#)

JES3

- defining an MCS console [271](#)
- DFSMSrmm SAMPLIB member EDG3IIP1 [474](#)
- DFSMSrmm SAMPLIB member EDG3LVVR [474](#)
- DFSMSrmm SAMPLIB member EDG3UX29 [473](#)
- DFSMSrmm SAMPLIB member EDG3UX62 [474](#)
- DFSMSrmm SAMPLIB member EDG3UX71 [473](#)
- message IATUX29 [24](#)
- mount and fetch messages [77](#), [154](#), [269](#)
- running with DFSMSrmm [473](#)
- setting the SETPARAM DSN option [474](#)

job name

- controlling policy selection [194](#)
- description [5](#)
- order of matching during vital record processing [355](#)
- using for pool selection [290](#)

jobs from non-z/OS systems, defining owner information [63](#)

journal

- allocating space for [54](#)
- backing up [55](#), [373](#)
- calculating space for [53](#)
- clearing [370](#), [377](#)
- decrease size [395](#)
- defining a threshold [175](#)
- EDGJNLAL SAMPLIB member [54](#)
- GUARANTEED SPACE attribute, using [54](#)
- increase size [395](#)
- JCL for backing up [372](#)
- moving
 - using DFSMSrmm utilities [399](#)
- moving to a different device [397](#)
- naming [175](#)
- placement of [54](#)
- pre-update copy of updated record [175](#)
- protecting [55](#)
- resetting [371](#)
- scheduling back up [327](#)
- steps for moving [397](#), [398](#)

JOURNAL DD [44](#)

JOURNALFULL, EDGRMMxx operand [175](#)

JRNLNNAME, EDGRMMxx operand [175](#)

JRNLTRAN, EDGRMMxx operand [175](#)

Julian date format [171](#), [340](#), [383](#), [389](#)

K

KB [49](#)

keyboard

- navigation [545](#)
- PF keys [545](#)
- shortcut keys [545](#)

L

LABEL procedure [459](#)

label types supported [13](#)

labeling duplicate volumes [447](#)

LASTREF extra days

- RETENTIONMETHOD suboption [184](#)

LIBQ exec [458](#)

- library
 - automated tape [1](#)
 - manual tape [1](#)
 - non-system-managed [1](#)
 - removable media [1](#)
 - system-managed [1](#)
- LIBRARY command [129](#), [131](#)
- limiting searches [176](#)
- limiting tape volume usage at the system level [202](#)
- LINECOUNT, EDGRMMxx operand [175](#)
- LISTCONTROL subcommand [218](#)
- LISTVRS subcommand [218](#)
- LOCAL storage location [3](#)
- LOCALTASKS EDGRMMxx operand [175](#)
- location
 - defining installation defined storage locations [144](#)
 - for more than one destination request [355](#)
 - specifying move locations [6](#)
- LOCATION [340](#)
- LOCDEF command syntax [144](#)
- LOCDEF, command in EDGRMMxx
 - AUTOMOVE operand [145](#)
 - LOCATION operand [144](#)
 - MANAGEMENTTYPE operand [145](#)
 - MEDIANAME operand [145](#)
 - PRIORITY operand [146](#)
 - TYPE operand [146](#)
 - using [147](#)
- logical volume cartridge entry processing [109](#)
- logical volume support [108](#)
- low-on-scratch condition [458](#)

M

- macros
 - EDG_EXIT300 [526](#)
 - EDGLCSUP [511](#)
 - EDGPL100 [517](#)
 - EDGPL200 [524](#)
 - EDGSLAB [529](#)
 - Installation Exit mapping macro — EDG_EXIT300 [526](#)
 - Installation Exit mapping macro — EDGPL100 [517](#)
 - Installation Exit mapping macro — EDGPL200 [524](#)
 - Library Control System interface — EDGLCSUP [511](#)
 - sticky label data — EDGSLAB [529](#)
- magnetic media
 - life expectancy of [152](#)
- maintaining the user access list [238](#)
- management class
 - assigning [92](#)
 - defining for volume retention [91](#)
 - exploiting expiration attributes of [177](#)
 - exploiting management class attributes [93](#)
 - retaining
 - non-system-managed volumes [89](#)
 - system-managed volumes [91](#)
 - usage in vital record processing [353](#)
- management class names [6](#)
- managing
 - DFSMSHsm tapes [311](#)
 - managing scratch tape pools [85](#)
 - scratch tape pools with EDG_EXIT100 [269](#)
 - special dates with vital record management values [279](#)
 - storage locations [133](#)

- managing (*continued*)
 - VM tapes [458](#)
- managing DFSMSHsm tapes
 - with VRSEL retention method [315](#)
- managing stacked volumes [110](#)
- manual cartridge entry processing [100](#)
- manual mode
 - implementation, during [61](#)
 - specifying [180](#)
- manual move control [138](#)
- manual recovery [395](#)
- manual tape library
 - adding volumes to [100](#)
 - description [2](#)
 - ejecting volumes from [101](#), [358](#)
 - moving volumes to [106](#)
- marking volumes for replacement [363](#)
- MASK, EDGRMMxx operand [208](#)
- MASTER DD [44](#)
- master status [9](#)
- master status volumes, controlling tape initialization [431](#)
- master volume
 - controlling overwriting of [176](#)
 - validating owner information [260](#)
- MASTER.RESERVE resource symbolic name [29](#)
- MASTEROVERWRITE, EDGRMMxx operand [176](#)
- matching order for vital record specifications [355](#)
- MAXHOLD, EDGRMMxx operand [176](#)
- maximum retention period, specifying [177](#)
- MAXRETPD, EDGRMMxx operand [177](#)
- MB [49](#)
- MCATTR, EDGRMMxx operand [93](#), [177](#)
- MCS console [271](#)
- media name
 - assigning for storage locations [137](#)
 - defining for storage locations [145](#)
 - defining for volume pools [79](#)
 - using to move volumes to non-shelf-managed storage locations [138](#)
 - using to segregate storage locations [137](#)
- media shape [137](#)
- media type pooling [80](#)
- MEDIANAME, EDGRMMxx operand [178](#), [212](#)
- MEDINF command
 - syntax [149](#)
- MEDINF command, in EDGRMMxx [148](#)
- MEDINF REPLACE command
 - values for [151](#)
 - volume replacement [151](#)
- MEDINF STK command
 - example of using [153](#)
- MEMBER, EDGRMMxx operand [178](#)
- mending the control data set [414](#)
- message file [327](#)
- messages
 - CBR3660A [458](#)
 - controlling message case [179](#)
 - customizing messages [451](#)
 - EDG2103D [395](#)
 - EDG2111I [395](#)
 - EDG2115I [395](#)
 - EDG2235E [365](#)
 - EDG2236I [365](#)
 - EDG2237E [365](#)

messages (*continued*)

- EDG2307I [364](#)
- EDG2404W [355](#), [364](#)
- EDG2405 through EDG2409 [452](#)
- EDG2420I [364](#)
- EDG2421I [364](#)
- EDG2422I [364](#)
- EDG2423I [364](#)
- EDG2424I [364](#)
- EDG2425I [364](#)
- EDG2426I [364](#)
- EDG2429I [364](#)
- EDG2700 through EDG2713 [452](#)
- EDG3017I [371](#)
- EDG3018I [371](#)
- EDG3212E [371](#)
- EDG4001D [396](#)
- EDG4010D [371](#)
- EDG4026I [424](#)
- EDG6401I [364](#)
- EDG6417I [412](#)
- EDG6433I [412](#)
- EDG6434I [412](#)
- EDG6901I [412](#)
- EDG8008D [396](#)
- EDG8121D [261](#)
- EDG8122D [261](#)
- EDG8123D [262](#)
- EDG8124I [261](#)
- IATUX29 [24](#)
- IEC507D [211](#)
- IEC704A [107](#)
- IOS000I [424](#)
- release notification [453](#)
- sample EDGUTIL SYSPRINT output [412](#)
- sample messages issued by RMM ADDRACK TSO subcommand [371](#)
- setting date format for [171](#)
- updating [154](#)

Metro

- backing up the control data set with
 - planning for [52](#)
 - using [372](#)

MHKP.ACTIVE resource symbolic name [29](#)

migrating to vts [109](#)

mixed case messages [179](#)

MNTMSG command syntax [154](#)

MNTMSG, command in EDGRMMxx

- ID operand [155](#)
- MSGID operand [155](#)
- RACK operand [155](#)
- VOLUME operand [156](#)

mode of operation

- changing during implementation [67](#)
- definitions [18](#)
- description [18](#)
- in parmlib member EDGRMMxx [180](#)
- type [180](#)

modifying

- tape labels [471](#)

monitoring the space used by control data set [394](#)

monthly archive from weekly audit reports [534](#)

movement and retention policies [345](#)

movement hierarchy [7](#)

movement priority number [7](#)

moving

- DFSMSrmm data sets to different devices [397](#)

- moving volumes to a system-managed tape library [127](#)

- volumes to storage locations [137](#)

- volumes using vital record specification chains [6](#)

moving volumes in a multivolume set [179](#)

MSG, EDGRMMxx operand [179](#), [208](#)

MSGID, EDGRMMxx operand [155](#)

multi-record update failure [395](#)

multitasking of utilities [364](#), [406](#)

N

name hiding support [241](#)

name set vital record specification

- checking for [406](#)

- correcting next vital record specification information [404](#)

name vital record specification [6](#)

NAME, EDGRMMxx operand [208](#)

navigation

- keyboard [545](#)

NETVIEW [66](#)

no label tapes

- setting up RACF profiles to use [223](#)

- usage [13](#)

- using BLP to create [169](#)

NOLASTREF

- RETENTIONMETHOD suboption [186](#)

non-checkpoint data set creation [17](#)

non-IBM media

- information

- implementing [153](#)

non-scratch tape [539](#)

non-system-managed tape library

- moving from [127](#)

- sharing the control data set [421](#)

nonstandard tape label [13](#)

NOTIFY processing

- customizing messages [455](#)

NOTIFY, EDGRMMxx operand [179](#)

notifying

- customizing messages and notes [452](#)

- during inventory management [360](#)

- owners [179](#)

- release action [360](#)

- volume and software product owners [360](#)

- volume release [179](#)

NUMBER, EDGRMMxx operand [208](#)

O

OAM (object access method)

- cartridge entry processing [100](#)

- EDGLCSUP macro [511](#)

- manual cartridge entry processing [100](#)

- volume-not-in-library processing [103](#)

OAM support

- DFSMSrmm processing for [255](#)

obtaining updated versions of IKJEFTxx member [27](#)

ONLYIF command syntax [156](#)

OPEN

- OPEN (*continued*)
 - retention of data sets that are currently open [6](#)
- open data sets
 - during expiration processing [364](#)
 - retaining [355](#)
- OPEN vital record specification [355](#)
- OPENRULE command
 - examples of using [201](#)
 - syntax [157](#)
- OPENRULE parmlib command
 - implementing [200](#)
- operational procedures [66](#)
- operator
 - access to DFSMSrmm resources [228](#)
 - tasks [21](#)
- OPMODE
 - EDGRMMxx operand [180](#)
 - manual mode [180](#)
 - protect mode [181](#)
 - record-only mode [180](#)
 - warning mode [180](#)
- OPTION command
 - operands [168](#)
- OPTION, command in EDGRMMxx
 - ACCOUNTING operand [168](#)
 - BACKUPPROC operand [168](#)
 - BLP operand [169](#)
 - CATLGDAY operand [183](#)
 - CATRETPD operand [169](#)
 - CATSYSID operand [170](#)
 - CDSID operand [168](#)
 - CLIENT operand [170](#)
 - COMMANDAUTH operand [170](#)
 - DATEFORM operand [171](#)
 - DISPDDNAME operand [171](#)
 - DISPMSGID operand [171, 172](#)
 - DSNAME operand [171](#)
 - DSNEXPIRE operand [172](#)
 - EXPDTDROP operand [172](#)
 - GDG operand [173](#)
 - IPLDATE operand [174](#)
 - JOURNALFULL operand [175](#)
 - JRNLNNAME operand [175](#)
 - JRNLTRAN operand [175](#)
 - LINECOUNT operand [175](#)
 - LOCALTASKS operand [175](#)
 - MASTEROVERWRITE operand [176](#)
 - MAXHOLD operand [176](#)
 - MAXRETPD operand [177](#)
 - MCATTR operand [177](#)
 - MEDIANAME operand [178](#)
 - MEMBER operand [178](#)
 - MOVEBY operand [179](#)
 - MSG operand [179](#)
 - NOTIFY operand [179](#)
 - OPMODE operand [180](#)
 - PDA operand [182](#)
 - PDABLKCT operand [182](#)
 - PDABLSZ operand [182](#)
 - PDALOG operand [182](#)
 - PREACS operand [182](#)
 - RETAINBY operand [183](#)
 - RETENTIONMETHOD operand
 - LASTREF suboption [184](#)

- OPTION, command in EDGRMMxx (*continued*)
 - RETENTIONMETHOD operand (*continued*)
 - NOLASTREF suboption [186](#)
 - RETAINBY suboption [186](#)
 - RETPD operand [187](#)
 - REUSEBIN operand [188](#)
 - SCRATCHPROC operand [188](#)
 - SERVER operand [188](#)
 - setting during startup [60](#)
 - SMFAUD operand [189](#)
 - SMFSEC operand [189](#)
 - SMSACS operand [189](#)
 - SYSID operand [190](#)
 - TPRACF operand [191](#)
 - VRSCCHANGE operand [193](#)
 - VRSDROP operand [194](#)
 - VRSEL operand [194](#)
 - VRJOBNAME operand [194](#)
 - VRSMIN operand [195](#)
 - VRRETAIN operand [195](#)
- original expiration date [232](#)
- OUTPUT, EDGRMMxx operand [204](#)
- owner
 - defining owner information [63](#)
 - information recorded in the TCDB [106](#)
 - managing information [15](#)

P

- panel navigation [449](#)
- parallel processing
 - setting up outside of SMP/E [266](#)
- parallel running of exits [264](#)
- parmlib member definitions [47](#)
- parmlib member EDGRMMxx
 - defining
 - implementation, during [47](#)
 - mount and fetch messages — MNTMSG [154](#)
 - pools — VLPOOL [209](#)
 - security classes — SECCLS [206](#)
 - system options — OPTION [161](#)
 - tables not available on systems — REJECT [202](#)
 - description [5](#)
 - specifying options [143](#)
- partitioning a system-managed tape library [100, 125](#)
- partitioning a VTS [125](#)
- partitioning an automated tape library [127](#)
- PDA, EDGRMMxx operand [182](#)
- PDABKLSZ, EDGRMMxx operand [182](#)
- PDABLKCT, EDGRMMxx operand [182](#)
- PDALOG, EDGRMMxx operand [182](#)
- PERM operand
 - MEDINF REPLACE command [152](#)
- permanent errors [363](#)
- planning
 - evaluating removable media management needs [539](#)
 - owner information [63](#)
 - pooling [75](#)
 - programming interfaces [249, 267](#)
 - user exits [249, 267](#)
 - vital record specifications [66](#)
- policies
 - volume replacement [151](#)
- policy

policy (*continued*)

- defining for data sets and volumes [5](#)
- defining for management class and vital record management values [91](#)
- defining retention and movement policies [5](#)
- expiration management [91](#)
- retention and movement [345](#)

pool

- changing definition [79](#)
- changing definitions [79](#)
- creating [78](#)
- default [78](#), [209](#)
- designing
 - rack pools [80](#)
 - scratch pools [80](#)
- name [79](#), [153](#)
- operator messages [77](#)
- prefixes [78](#)
- size [78](#)
- tape drive availability [77](#)
- tape drive displays [77](#)
- types [77](#)
- types of [75](#)
- VLPOOL command [78](#), [209](#)

pooling

- based on media type [80](#)
- considerations [77](#)
- defining shelf locations during implementation [63](#)
- definition [9](#), [75](#)
- example [84](#)
- planning [75](#), [81](#)
- rack [9](#)
- scratch [10](#)
- segregating WORM tapes in separate scratch pools [97](#)
- selection using EDG_EXIT100 installation exit [290](#)
- selection using job name [290](#)
- selection using system name [290](#)
- types [9](#)
- using a BTLS category name [213](#)
- using VLPOOL to specify [209](#)
- within system-managed libraries [10](#)

port [71](#)

PPRC

- backing up the control data set with
 - planning for [52](#)
 - using [372](#)

pre-acfs processing [281](#)

pre-ACS processing [89](#)

predicting when a volume is released [340](#)

PREFIX, EDGRMMxx operand [213](#)

preventing

- a volume from returning to scratch status [300](#)
- the use of IEHINITT [233](#)
- volume use on specific systems [202](#)

primary vital record specification [354](#)

priority

- of location names [355](#)
- setting with PRIORITY operand [146](#)
- volume moves [3](#)

private volume

- controlling overwriting of [176](#)
- definition [9](#)
- validating owner information [260](#)

Problem Determination Aid (PDA)

Problem Determination Aid (PDA) (*continued*)

- controlling tracing [462](#)
- data sets [461](#)
- description [43](#), [461](#)
- PDA parmlib option for enabling and disabling PDA trace [182](#)

problem diagnosis [449](#)

procedure library

- updating [43](#)

procedure library updates

- during implementation [43](#)

processing ACCODE [281](#)

products or product features

- enablement policy for [42](#)

programming interfaces

- EDG019VM [460](#)

- EDG3X71 [264](#)

- EDGDFHSM [251](#)

- EDGLCSUP [511](#)

- EDGLCSUX [251](#)

- EDGMSGEX [263](#)

- EDGSLAB [529](#)

- planning [249](#), [267](#)

protect mode

- running during implementation [67](#)
- setting to validate data set names [238](#)
- specifying [180](#)

PRRTITION command

- examples of using [201](#)

- syntax [197](#)

PRRTITION parmlib command

- implementing [200](#)

pseudo-GDG [320](#)

PTFs

- installing [69](#)

Q

QNAME [29](#)

quiescing the DFSMSrmm subsystem interface [62](#)

R

RACF

- profiles for no label tapes [223](#)

RACF (Resource Access Control Facility)

- assigning DFSMSrmm a user ID during implementation [46](#)

- audit information [232](#)

- authorizing users [16](#), [215](#), [238](#)

- checking for DATASET class resource [244](#)

- checking for TAPEVOL class resource [245](#)

- considerations for using under DFSMSrmm [239](#)

- controlling profile processing [233](#)

- data set profile [239](#)

- defining a TAPEVOL class resource [245](#)

- defining ABARS to RACF [47](#)

- defining DFSMSHsm to RACF [47](#), [311](#)

- defining profiles [215](#)

- defining resource classes [278](#)

- maintaining the user access list [238](#)

- no protection [233](#)

- protected DFSMSrmm resources [215](#)

RACF (Resource Access Control Facility) (*continued*)

- RACF operand in EDGRMMxx [213](#)
- recording changes to vital record specifications [218](#)
- STGADMIN.ADR.DUMP.CNCURRNT [371](#)
- tape profiles, setting DFSMSrmm's use of [191](#)
- tape protection [233](#)
- TAPEDSN [233](#)
- TAPEVOL [233](#)
- updating ICHRIN03 [46](#)
- using profile processing [237](#)
- using tape profiles [13](#)
- RACF SETROPTS MLNAMES considerations [241](#)
- rack number
 - assigning a [435](#)
 - estimating the number of [539](#)
 - inserting into a message [155](#)
- rack pool
 - definition [9](#)
 - designing [80](#)
 - using [85](#)
 - using VLPOOL to define in parmlib member [209](#)
- RACK, EDGRMMxx operand [155](#)
- RACROUTE calls [241](#)
- read-only mode
 - running during implementation [65](#)
 - specifying [180](#)
 - switching modes [67](#)
- REASSIGN [341](#)
- reclaiming volume from pending release [329](#)
- recording technology
 - life expectancy of [152](#)
- recovery
 - description [8](#)
 - from control data set update failures [395](#)
 - updating the DFSMSrmm control record during [384](#)
 - using DITTO [122](#)
- refreshing DFSMSrmm installation exits [294](#), [302](#), [307](#)
- REJECT command
 - syntax [203](#)
- REJECT commands
 - converting [204](#)
 - converting to OPENRULE and PRTITION commands [205](#)
- reject processing [202](#)
- REJECT, command in EDGRMMxx
 - ANYUSE operand [203](#)
 - OUTPUT operand [204](#)
- rejecting
 - a range of tape volumes [202](#)
 - volumes [18](#)
 - volumes on specific systems [202](#), [238](#)
- relabeling tapes [231](#)
- release action
 - confirming [360](#)
 - erase [370](#)
 - initialize [370](#)
 - notify owner [360](#)
 - pending [360](#)
 - replace [363](#)
 - return to owner [363](#)
 - return to scratch [363](#)
 - types [12](#)
- release notification messages [453](#)
- releasing
 - considerations for applications that manage tape [250](#)

releasing (*continued*)

- volume from pending release status [329](#)
- volumes at the pool-level [76](#)
- REMOTE storage location [3](#)
- removable media library
 - definition [1](#)
 - organizing [75](#)
- removing DFSMSrmm from the system [244](#)
- reorganizing the control data set [392](#), [393](#)
- replace volume release action [363](#)
- replacement policies
 - implementing [151](#), [361](#)
- report
 - about owners sorted by name and department number [535](#)
 - about volumes [535](#)
 - based on rack number prefixes [535](#)
 - containing information about lost volumes [535](#)
 - creating [15](#)
 - customizing titles [452](#)
 - data sets sorted by data set name [534](#)
 - default lines per page, specifying [175](#)
 - EDGAUD DFSMSrmm security and audit report [8](#)
 - EDGRPTD DFSMSrmm movement and inventory report [16](#)
 - EDGRRPTE exec [8](#)
 - Expiration Processing Report [364](#)
 - extract data set [335](#)
 - monthly archive from weekly audit report [534](#)
 - on volumes to be replaced [152](#)
 - Report Generator [15](#)
 - sample JCL for [533](#)
 - setting date format for [171](#)
 - SMF records [535](#)
 - types of SMF record found [535](#)
 - Vital Records Retention Report [348](#)
 - volumes currently in storage locations sorted by volume serial number [535](#)
 - volumes moving to storage locations [535](#)
 - volumes recently returned to scratch status [535](#)
 - volumes sorted by volume serial number [535](#)
 - weekly archive from daily audit reports [534](#)
- report trailer lines, customizing [452](#)
- resetting the journal [371](#), [386](#)
- resource symbolic names [29](#)
- resources
 - defining [63](#)
- restoring the control data set [390](#)
- restoring the control data set at a recovery site [392](#)
- restoring the control data set with forward recovery [391](#)
- RETAINBY
 - RETENTIONMETHOD suboption [186](#)
- RETAINBY, EDGRMMxx operand [183](#)
- retaining
 - ABARS accompany [321](#)
 - based on catalog status [351](#)
 - data set retention types [5](#)
 - data sets [5](#)
 - DFSMSHsm backup tapes [317](#)
 - DFSMSHsm control data set backup tapes [322](#)
 - DFSMSHsm dump tapes [319](#)
 - DFSMSHsm migration tapes [316](#)
 - DFSMSHsm TAPECOPY tapes [318](#)
 - DFSMSHsm tapes [314](#)

- retaining (*continued*)
 - tapes written by ABARS [321](#), [323](#)
 - types of retention [5](#)
 - volumes [279](#)
- retaining accompany tapes [321](#)
- retaining backup tapes [323](#)
- retaining data sets closed during ABEND processing [355](#)
- retaining data sets closed with normal disposition DELETE [355](#)
- retaining tapes written by [321](#)
- retaining volumes forever [281](#)
- retaining volumes in a multivolume set
 - EXPDT retention method [186](#)
 - VRSEL retention method [183](#)
- retention and movement policies [345](#)
- retention by job name [194](#)
- retention method
 - assigning policies [87](#)
 - for new tape volumes [183](#)
 - for volumes [10](#)
 - selecting [4](#)
- retention period
 - default, specifying [187](#)
 - maximum, specifying [177](#)
- retention types [5](#)
- RETENTIONMETHOD, EDGRMMxx operand [183](#)
- RETPD, EDGRMMxx operand [187](#)
- return codes
 - EDG_EXIT100 [300](#)
 - EDG_EXIT200 [305](#)
 - EDG_EXIT300 [309](#)
 - EDGBKUP [384](#)
 - EDGHSKP [374](#)
 - EDGINERS [442](#)
 - EDGSPLCS [421](#)
 - EDGUTIL [418](#)
 - OAM [253](#)
- return to owner release action [363](#)
- return to scratch release action [363](#)
- returning volumes to scratch
 - when sharing the control data set [422](#)
- REUSEBIN(CONFIRMMOVE) [188](#)
- REUSEBIN(STARTMOVE) [188](#)
- reusing bins [188](#)
- REXX execs
 - check for use of removed .0 stem variables [535](#)
- RMM ADDVRS subcommand [218](#)
- RMM CHANGEVOLUME subcommand [218](#)
- RMM DELETEVOLUME subcommand [218](#)
- RMM DELETEVRS subcommand [218](#)
- RMM LISTCONTROL subcommand [218](#)
- RMM LISTVRS subcommand [218](#)
- RMM SEARCHVRS subcommand [218](#)
- RMMISPF exec [449](#)
- RMMISPF EXEC [55](#)
- RMMplex
 - authorizing users [55](#)
 - creating the control data set [48](#)
 - description [1](#)
 - managing catalogs [74](#)
 - sharing the control data set [50](#)
- RNAME [29](#)
- RPTXT command
 - syntax [344](#)

- RPTXT parameter [337](#)
- rules for tape mount validation [17](#)
- Running inventory management [328](#)
- running mode
 - description [18](#)
 - initial setting in parmlib member EDGRMMxx [65](#)
 - manual [180](#)
 - protect [181](#)
 - record-only [180](#)
 - set for full validation and recording [67](#)
 - types [180](#)
 - warning [180](#)

S

- SAF (System Authorization Facility)
 - authorization checking [241](#)
 - description [16](#)
 - processing to ignore a volume [276](#)
 - using the interface [241](#)
- SAMPLIB members
 - EDG3UX29 installing a JES3 USERMOD [24](#)
 - EDGCLIBQ [458](#)
 - EDGDFRMM [43](#)
 - EDGIVP1 IVP Job 1 initializing tape volumes [24](#)
 - EDGIVP2 IVP Job 2 using tape volumes [24](#)
 - EDGIVPPM installation verification procedure [24](#)
 - EDGJBKUP [377](#)
 - EDGJHKPA [333](#)
 - EDGJHSPK [327](#)
 - EDGJINER [423](#)
 - EDGJMFAL [51](#)
 - EDGJNLAL [54](#)
 - EDGJUTIL [52](#)
 - EDGJVME [458](#)
 - EDGLABEL [43](#)
 - EDGLIBQ [458](#)
 - EDGPHKP [327](#)
 - EDGPHKPA [333](#)
 - EDGUX100 [24](#)
 - EDGUX200 [24](#)
 - EDGUX300 [24](#)
 - EDGXPROC [43](#)
 - list of [533](#)
- SCAN output
 - example of [436](#)
- scheduling
 - back up [327](#)
 - inventory management [8](#), [327](#)
- scratch management for BTLS [131](#)
- scratch mount management [124](#)
- scratch pool
 - definition [10](#)
 - designing [80](#)
 - managing with EDG_EXIT100 installation exit [269](#)
 - using VLPOOL to define in parmlib member [209](#)
- scratch pooling using storage groups [82](#)
- scratch volumes
 - replenishing in an automated tape library [458](#)
- SCRATCHPROC, EDGRMMxx operand [188](#)
- searches, limiting [176](#)
- SEARCHVRS subcommand [218](#)
- SECCLS command
 - syntax [207](#)

- SECCLS, command in EDGRMMxx
 - DESCRIPTION operand [208](#)
 - ERASE operand [208](#)
 - MASK operand [208](#)
 - MSG operand [208](#)
 - NAME operand [208](#)
 - NUMBER operand [208](#)
 - SMF operand [209](#)
- secondary vital record specification [354](#)
- security
 - audit trails [232](#)
 - considerations when running DFSMSHsm and DFSMSrmm [325](#)
 - controlling RACF profile processing [233](#)
 - controlling volume use in a system complex [238](#)
 - DFSMSrmm resources [215](#)
 - security classes — SECCLS [206](#)
 - SMFAUD [232](#)
 - special processing [232](#)
 - using RACF profile processing [237](#)
- selecting a volume pool [213](#)
- selecting volumes for automatic processing [447](#), [448](#)
- server systems
 - authorization [71](#)
 - firewall [71](#)
 - implementing [72](#)
 - inventory management considerations [332](#), [378](#)
 - setting up [71](#)
 - using [73](#)
 - utility considerations [332](#), [378](#)
- SETPARAM DSN option in JES3 [474](#)
- setting
 - maximum retention period [12](#)
 - message case [179](#)
 - mode of operation [180](#)
 - setting location priority [146](#)
 - volume expiration date and time [12](#)
- setting up parallel processing outside of SMP/E [266](#)
- shared user catalog [50](#)
- sharing
 - catalogs [332](#)
 - the control data set [421](#)
- shelf location
 - defining during implementation [63](#)
 - defining new locations [63](#)
 - definition of [539](#)
 - management [9](#)
 - prefix, in pools [78](#)
- shelf management
 - built-in storage location [3](#)
 - description [9](#)
 - for storage locations [133](#)
 - installation defined storage location [3](#)
 - using LOCDEF MANAGEMENTTYPE operand to specify [145](#)
- shortcut keys [545](#)
- SHUTDOWN resource symbolic name [29](#)
- SMF record number
 - collecting audit information [232](#)
 - defining audit records [189](#)
 - defining security records [189](#)
- SMF, EDGRMMxx operand [209](#)
- SMFAUD, EDGRMMxx operand [189](#)
- SMFPRMxx [27](#)
- SMFSEC, EDGRMMxx operand [189](#)
- SMP/E
 - implementation, installing DFSMSrmm during [23](#)
- SMPSTS members
 - ARCTVEXT [24](#)
 - CBRUXCUA [24](#)
 - CBRUXEJC [24](#)
 - CBRUXENT [24](#)
 - CBRUXVNL [24](#)
 - IGXMSGEX [24](#)
 - list of [533](#)
- SMS management class attributes
 - exploiting [93](#)
- SMS pre-ACS interface
 - using [93](#)
- SMS tape storage groups [81](#)
- space requirement
 - calculating for control data set [49](#)
 - calculating for journal [53](#)
- special expiration date [59](#)
- specifying ISO/ANSI label versions [434](#)
- specifying pool prefixes [78](#)
- specifying the ISO/ANSI volume accessibility code [434](#)
- specifying, EDGRMMxx operand [47](#)
- stacked volume
 - enabling support [411](#)
 - using [113](#)
- stacked volume support [112](#), [118](#)
- START command [60](#)
- starting
 - DFSMSrmm [60](#)
 - DFSMSrmm subsystem address space [43](#)
- STGADMIN.ADR.DUMP.CNCURRNT [371](#)
- STGADMIN.EDG.HOUSEKEEP
 - READ access [230](#)
 - user [230](#)
- STGADMIN.EDG.IGNORE.TAPE
 - example [243](#)
 - using [276](#)
- STGADMIN.EDG.IGNORE.TAPE.NORMM
 - definition [217](#)
 - setting access [222](#)
- STGADMIN.EDG.IGNORE.TAPE.RMM
 - definition [217](#)
 - setting access [221](#)
- STGADMIN.EDG.LABEL
 - controlling tape relabeling [231](#)
- STGADMIN.EDG.LISTCONTROL
 - CONTROL access [228](#)
 - user functions [228](#)
- STGADMIN.EDG.MASTER
 - administrator functions [228](#)
 - CONTROL access [228–230](#)
 - librarian functions [229](#)
 - operator functions [230](#)
 - READ access [228](#)
 - system programmer functions [229](#)
 - user functions [228](#)
- STGADMIN.EDG.NOLABEL
 - controlling tape relabeling [231](#)
- STGADMIN.EDG.OPERATOR
 - librarian functions [230](#)
 - operator functions [231](#)
 - READ access [230](#)

- STGADMIN.EDG.OPERATOR (*continued*)
 - system programmer functions [229](#)
 - UPDATE access [229](#), [231](#)
- STGADMIN.EDG.OWNER
 - administrator functions [229](#)
 - READ access [228](#)
 - UPDATE access [229](#)
 - user functions [228](#)
- STGADMIN.EDG.RELEASE
 - READ access [228](#)
 - user functions [228](#)
- STGADMIN.EDG.VRS
 - administrator functions [229](#)
 - librarian functions [230](#)
 - READ access [228](#)
 - system programmer functions [229](#)
 - UPDATE access [229](#)
 - user functions [228](#)
- STGADMIN.IGG.LIBRARY [125](#)
- sticky label support [282](#)
- sticky labels
 - modifying label output using EDGUX100 [284](#)
- stopping the DFSMSRmm subsystem interface [61](#)
- storage administrator
 - access to DFSMSRmm resources [228](#)
 - tasks [21](#)
- storage group
 - assigning [89](#)
 - name recorded in DFSMSRmm control data set [125](#)
 - name validation [108](#)
- storage group, assigning [82](#)
- storage groups for DFSMSRmm scratch pooling [82](#)
- storage location
 - assigning bin numbers [9](#)
 - built-in [3](#), [133](#)
 - changing [139](#)
 - defining [134](#)
 - definition [3](#)
 - deleting [139](#)
 - DISTANT [3](#)
 - installation defined [3](#), [133](#)
 - LOCAL [3](#)
 - managing using DSTORE [357](#)
 - more than one destination request [355](#)
 - moving volumes to [137](#)
 - priority of storage location names [355](#)
 - REMOTE [3](#)
 - segregating by media name [79](#)
 - shelf-management [3](#), [133](#)
- storage location management
 - description [357](#)
 - scheduling [327](#)
- storage requirements
 - control data set [49](#)
 - extract data set [335](#), [336](#)
 - journal [53](#)
- summary of changes xxxvii
- switching volumes to a system-managed tape library [127](#)
- syntax diagrams
 - how to read xxx
- SYS1.PARMLIB
 - updating during implementation [25](#)
- SYSID, EDGRMMxx operand [79](#), [190](#), [214](#)
- SYSIN file

- SYSIN file (*continued*)
 - EDGHSKP utility [342](#)
- SYSPRINT data set [386](#)
- system
 - name, defining [190](#)
 - options, defining [161](#)
- system catalogs, synchronizing with the DFSMSRmm control data set [339](#), [409](#)
- system programmer
 - access to DFSMSRmm resources [228](#)
 - tasks [21](#)
- system-managed tape library
 - adding duplicate volume [97](#)
 - confirming volume movement to [105](#)
 - defining existing volumes [106](#)
 - defining volumes to DFSMSRmm [122](#)
 - description [2](#)
 - DFSMSRmm defining name of [100](#)
 - DFSMSRmm support of [99](#)
 - ejecting volumes from [101](#), [358](#)
 - initializing volumes in [107](#)
 - partitioning [125](#)
 - pool selection [290](#)
 - removing DFSMSRmm from [224](#)
- SYSZRMM [29](#)

T

- tape
 - initializing [428](#)
 - labeling [423](#)
 - processing tape labels [17](#)
 - validating mounts [17](#)
- tape configuration database
 - definition [2](#)
 - obtaining volume information from [64](#)
 - rebuilding after system failure [107](#)
 - updates performed by DFSMSRmm [106](#)
 - verifying against the control data set [413](#)
- tape label
 - bypassing tape label processing [232](#)
 - creating a VOL1 [423](#)
 - expiration date [211](#)
 - processing [20](#)
 - restriction [131](#)
 - types supported [13](#)
 - validation [20](#), [423](#)
- tape labels
 - modifying [471](#)
- tape librarian
 - access to DFSMSRmm resources [228](#)
 - tasks [20](#)
- tape mount validation rules [17](#)
- tape profile
 - creating a tape profile [238](#)
 - processing [233](#)
 - protection options [233](#)
- TAPEDSN [191](#), [233](#)
- TAPEVOL [191](#), [233](#)
- tasks
 - application programmer [21](#)
 - general user [20](#)
 - operator [21](#)
 - storage administrator [21](#)

tasks (*continued*)

system programmer [21](#)

tape librarian [20](#)

TCP/IP

error, WTOR [71](#)

identifying information [71](#)

IP address [71](#)

tracing IP communication [71](#)

TEMP operand

MEDINF REPLACE command [152](#)

temporary read error

information recorded by DFSMSrmm [442](#)

listed in the extract data set [337](#)

TPRACF, EDGRMMxx operand [191](#), [325](#)

tracing

IP communication [71](#)

tracing errors [449](#)

trial run

description [356](#)

set based on EDGRMMxx VRSCCHANGE operand [193](#)

validating vital record specifications without control data set update [68](#)

vital record specification before they are processed [341](#)

TYPE, EDGRMMxx operand [214](#)

U

UNCATALOG, EDGRMMxx operand [193](#)

uncataloging

controlling with UNCATALOG EDGRMMxx operand [193](#)

data sets [193](#)

data sets during expiration processing [361](#)

updating

AUTORxx [30](#)

GRSRNLxx [27](#)

ICHRIN03 [46](#)

SMFPRMxx [27](#)

volume expiration date [187](#)

Workload Management service definition for DFSMSrmm [60](#)

upper case messages [179](#)

user access list [238](#)

user exits

planning [267](#)

using [249](#)

user group pooling [80](#)

user interface

ISPF [545](#)

TSO/E [545](#)

user status [9](#)

user status volumes, controlling tape initialization [431](#)

using

a shared user catalog [50](#)

EDG_EXIT100 installation exit [267](#)

EDG_EXIT200 installation exit [300](#)

EDG_EXIT300 installation exit [305](#)

manual move control [138](#)

media shape to segregate shelf locations [137](#)

using scratch pooling [85](#)

using a stacked volume [113](#)

using SMS tape storage groups [81](#)

UTC

enabling support [411](#)

utility

utility (*continued*)

EDGAUD, security and audit [8](#)

EDGBKUP, backing up control data set [8](#), [377](#)

EDGHSKP, inventory management [8](#), [327](#)

EDGINERS, initializing and erasing volumes [8](#), [423](#)

EDGRESET utility [244](#)

EDGRPTD, movement and inventory [16](#)

EDGSPLCS [418](#)

EDGUTIL, verifying control data set contents [8](#), [107](#), [377](#)

multitasking [364](#), [406](#)

V

validating

control data set integrity [411](#)

magnetic tape mounts [17](#)

owner information on a master volume [260](#)

primary vital record specification [354](#)

storage group name [108](#)

vital record specification chains [353](#)

vital record specifications without control data set update [68](#)

VERIFY parameter [337](#)

verifying DFSMSrmm control data set contents [401](#)

virtual tape server

completing export processing [114](#)

confirming volume moves for exported volumes [119](#)

creating a volume export list [113](#)

creating a volume import list [120](#)

creating an import list [535](#)

creating export list of volumes [114](#), [119](#)

creating import list of volumes [115](#), [120](#)

DFSMSrmm support for [104](#)

export processing [113](#), [118](#)

import processing [115](#), [116](#), [119](#)

logical volume cartridge entry processing [109](#)

logical volume support [108](#)

migrating to [109](#)

partitioning [125](#)

stacked volume support [112](#), [118](#)

volume-not-in-library processing [103](#)

virtual tape server subsystem [2](#)

virtualization engine [116](#)

vital record processing

description [352](#)

identifying volumes to be retained [93](#), [281](#)

scheduling [327](#)

trial run set up VRSCCHANGE EDGRMMxx operand [193](#)

Vital Records Retention Report [348](#)

vital record specification

chaining [6](#)

checking for [353](#)

defining [92](#)

for data sets closed during ABEND processing [355](#)

for data sets still open during inventory management [355](#)

management values [279](#)

matching order [355](#)

minimum number [353](#)

order of matching [355](#)

planning [66](#)

recording changes to [218](#)

secondary vital record specification [354](#)

- vital record specification (*continued*)
 - testing vital record specification before they are processed [341](#)
 - validating vital record specification chains [353](#)
- vital record specification management value
 - assigned by DFSMSrmm EDGUX100 sample [6](#)
 - defining [279](#)
 - description [6](#)
 - retaining volumes using [279](#)
 - setting with EDG_EXIT100 installation exit [268](#)
- Vital Records Retention Report
 - customizing trailer lines [452](#)
 - using [348](#)
- VLPOOL command
 - syntax [209](#)
- VLPOOL definitions [129](#)
- VLPOOL MASTEROVERWRITE, EDGRMMxx operand [212](#)
- VLPOOL, command in EDGRMMxx
 - AUTOSCRATCH operand [210](#)
 - DESCRIPTION operand [211](#)
 - EXPDTCHECK operand [211](#), [314](#)
 - MEDIANAME operand [212](#)
 - PREFIX operand [213](#)
 - RACF operand [213](#)
 - RELEASEACTION operand [214](#)
 - SYSID operand [214](#)
 - TYPE operand [214](#)
 - VLPOOL MASTEROVERWRITE operand [212](#)
- VM tape
 - getting rack number with LIBQ exec [458](#)
 - managing [458](#)
- volume
 - adding details [64](#)
 - automatic notification of release [179](#)
 - changing ADD product volume dialog panel defaults [451](#)
 - confirming moves [369](#)
 - defining [106](#), [122](#)
 - defining details [64](#)
 - defining existing [106](#)
 - duplicate serial numbers [95](#)
 - ejecting from system-managed libraries [101](#), [358](#)
 - foreign volumes [97](#)
 - ignoring [275](#)
 - initializing volumes in system-managed libraries [107](#)
 - input only [126](#)
 - managing volumes with special expiration dates [91](#)
 - marking for replacement [363](#)
 - master status [9](#)
 - moving [105](#)
 - preventing use on specific systems in a complex [238](#)
 - private [9](#)
 - protecting [325](#)
 - reclaiming volume from pending release [329](#)
 - rejecting [202](#)
 - retaining
 - DFSMSHsm tape volumes [314](#)
 - using management class [91](#)
 - using vital record specification [279](#)
 - retention types [5](#)
 - returning to scratch [125](#)
 - returning volumes to scratch status [361](#)
 - scratch volume [9](#)
 - segregating by media shape [137](#)
 - support duplicate volumes [475](#)

- volume (*continued*)
 - undefined serial numbers [97](#)
 - updating expiration date [187](#)
 - updating status [125](#)
 - user status [9](#)
 - using manual move control [138](#)
 - validating mounts [17](#)
 - VM tapes, managing [458](#)
 - volume access [238](#)
 - with duplicate volume serial numbers [95](#)
 - with permanent errors [363](#)
 - with special expiration date [59](#)
- volume DFSMSrmm ISPF dialog panel defaults [451](#)
- volume replacement policies
 - implementing [151](#), [361](#)
 - MEDINF REPLACE command [151](#)
- volume serial number
 - changing duplicate volume serial numbers [96](#)
 - external [2](#)
 - ignoring duplicates [275](#)
 - labeling duplicate volume serial numbers [96](#)
 - managing duplicate volume serial numbers [95](#)
 - rules for matching shelf location [75](#)
 - using EDG_EXIT100 to manage duplicates [267](#)
- volume-not-in-library parameter list
 - processing for [261](#)
- volume-not-in-library processing [103](#)
- VOLUME, EDGRMMxx operand [156](#)
- volumes to be replaced
 - report on [152](#)
- VRSBYJOBNAME
 - description [5](#)
- VRSCHANGE, EDGRMMxx operand [193](#)
- VRSDROP, EDGRMMxx operand [194](#)
- VRSEL parameter [337](#), [347](#)
- VRSEL processing
 - excluding specific data sets from [274](#)
- VRSEL retention method
 - manage DFSMSHsm tapes with [315](#)
- VRSEL, EDGRMMxx operand [194](#)
- VRSELEXCLUDE attribute
 - excluding data sets from vital record processing [352](#)
 - setting with EDGUX100 [274](#)
- VRSJOBNAME, EDGRMMxx operand [194](#)
- VRSMIN, EDGRMMxx operand [195](#)
- VRSRETAIN, EDGRMMxx operand [195](#)

W

- waiting requests, number of [63](#)
- warning mode
 - running during implementation [65](#)
 - specifying [180](#)
- weekly archive from daily audit reports [534](#)
- WHILECATALOG
 - catalog date format, WHILECATLG [351](#)
- WMC operand
 - MEDINF REPLACE command [152](#)
- Workload Management service definition for DFSMSrmm
 - updating [60](#)
- worm tape
 - segregating WORM tapes in separate scratch pools [97](#)
- writing
 - EBCDIC labels [443](#)

writing (*continued*)

ISO/ANSI labels

[443](#)

wrong label processing [431](#)

WUI

install [495](#)

network logs tab [507](#)

troubleshooting [508](#)

Using the DFSMSrmm WUI [497](#)

X

XRC

backing up the control data set with

planning for [52](#)

using [372](#)



Product Number: 5655-ZOS

SC23-6874-60

